Decision-Analytics

Chakra Roy

2026-10-20

Table of contents

Pengantar **3**

Video pengantar . . . 3
Mengapa buku ini ada . . . 3
Satu perjalanan, tiga tahap . . . 4
Untuk siapa buku ini . . . 4
Bagaimana buku ini disusun . . . 5
Cara menggunakan buku ini . . . 5
Prasyarat dan persiapan . . . 5
Dataset pendamping . . . 6
Konvensi . . . 6
Umpan balik dan kontribusi . . . 6
Ucapan terima kasih . . . 7

Introduction **8**

Apa itu Decision Analytics? . . . 8
Tangga Analytics: Dari Data ke Keputusan . . . 9
Siklus Pengambilan Keputusan . . . 10
Sneak Peek: Satu Contoh, Empat Tingkat Analytics . . . 11
Tingkat 1 — Descriptive: Apa yang terjadi? . . . 11
Tingkat 2 — Diagnostic: Mengapa? . . . 12
Tingkat 3 — Predictive: Apa yang akan terjadi? . . . 13
Tingkat 4 — Prescriptive: Apa yang sebaiknya dilakukan? . . . 13
Apa yang baru saja kita lihat? . . . 15
Peta Disiplin Ilmu . . . 16
Nilai: Dari Insight ke Impact . . . 16
Dataset Pendamping . . . 17
Tiga Bagian Buku Ini . . . 17
Kosakata Kunci . . . 18
Cara Membaca Buku Ini . . . 19
Rangkuman dan Jembatan ke Bagian I . . . 19

I Bagian I — Descriptive & Diagnostic Analytics 21

1 Fondasi Decision Analytics 22
1.1 Mengapa Fondasi? . . . 22
1.2 Menyiapkan Lingkungan Python . . . 22
1.3 Struktur Data Inti: `Series` dan `DataFrame` . . . 25
1.4 Tipe Data dan Mengapa Ia Penting . . . 27
1.5 Memuat Dataset Pendamping . . . 28
1.6 Inspeksi Awal: Lima Langkah Sistematis . . . 29
1.6.1 Langkah 1 — Lihat Bentuk dan Kolom . . . 29
1.6.2 Langkah 2 — Periksa Tipe Data . . . 30
1.6.3 Langkah 3 — Lihat Beberapa Baris Pertama . . . 31
1.6.4 Langkah 4 — Nilai Hilang . . . 31
1.6.5 Langkah 5 — Ringkasan Statistik & Distribusi Kategorikal . . . 31
1.7 Variabel Target dan Variabel Fitur . . . 33
1.8 Konvensi Dataset untuk Buku Ini . . . 34
1.9 Latihan Terbimbing . . . 35
1.10 Kotak "Jembatan": Dari Data ke Keputusan . . . 36
1.11 Rangkuman . . . 37
1.12 Jembatan ke Bab 2 . . . 37

2 Data Wrangling & Manajemen Data 39
2.1 Mengapa Wrangling? . . . 39
2.2 Sumber Data di Dunia Nyata . . . 40
2.3 Kualitas Data: Enam Dimensi . . . 40
2.4 Memuat Ulang Dataset . . . 41
2.5 Diagnosis Nilai Hilang . . . 41
2.5.1 Jenis Nilai Hilang: MCAR, MAR, MNAR . . . 41
2.5.2 Menemukan Nilai Hilang yang Tersembunyi . . . 42
2.5.3 Memutuskan Strategi . . . 43
2.6 Duplikat dan Konsistensi . . . 44
2.6.1 Duplikat Baris . . . 44
2.6.2 Konsistensi Nilai Kategorikal . . . 45
2.7 Outlier: Deteksi dan Penanganan . . . 46
2.7.1 Metode IQR (Interquartile Range) . . . 46
2.7.2 Metode Z-Score . . . 46
2.7.3 Memutuskan: Hapus, Pertahankan, atau Transformasi? . . . 47
2.8 Feature Engineering Dasar . . . 48
2.8.1 Binning: Mengubah Kontinu menjadi Kategorikal . . . 48
2.8.2 Encoding Kategori . . . 49
2.8.3 Fitur Turunan yang Berguna . . . 49
2.9 Menyimpan Dataset Bersih . . . 50
2.10 Latihan Terbimbing . . . 51

2.11 Kotak “Jembatan”: Dari Data ke Keputusan . . . 52
2.12 Rangkuman . . . 52
2.13 Jembatan ke Bab 3 . . . 53

3 Statistik Deskriptif & Eksplorasi Data 54
3.1 Dari Data ke Pertanyaan . . . 54
3.2 Memuat Dataset Bersih . . . 54
3.3 Ukuran Pusat: Tiga Cara Menjawab “Berapa Rata-rata?” . . . 55
3.4 Ukuran Sebaran: Seberapa Lebar Data Tersebar? . . . 57
3.5 Bentuk Distribusi: Kemencengan dan Ekor . . . 58
3.6 Eksplorasi Univariat: Variabel Kategorikal . . . 59
3.7 Eksplorasi Bivariat: Apakah Churn Berbeda Antar Kelompok? . . . 62
3.7.1 Kategorikal × Target: Cross-Tabulation . . . 62
3.7.2 Numerik × Target: Grouped Comparison . . . 63
3.8 Uji Statistik: Apakah Perbedaan Ini Nyata? . . . 65
3.8.1 Kerangka Uji Hipotesis . . . 65
3.8.2 Uji Chi-Square untuk Kategorikal . . . 66
3.8.3 Uji-t untuk Numerik . . . 66
3.9 Effect Size: Seberapa Besar Perbedaannya? . . . 67
3.9.1 Untuk Kategorikal: Cramér’s V . . . 67
3.9.2 Untuk Numerik: Cohen’s d . . . 68
3.10 Korelasi Antar Variabel Numerik . . . 69
3.11 Probabilitas: Jembatan ke Bagian II . . . 71
3.12 Latihan Terbimbing . . . 71
3.13 Kotak “Jembatan”: Dari Data ke Keputusan . . . 72
3.14 Rangkuman . . . 73
3.15 Jembatan ke Bab 4 . . . 73

4 Visualisasi Data & Dashboard 74
4.1 Mengapa Visualisasi? . . . 74
4.2 Grammar of Graphics: Bahasa Universal Visualisasi . . . 74
4.3 Prinsip Tufte: Data-Ink Ratio . . . 75
4.4 Memilih Chart yang Tepat . . . 76
4.5 Visualisasi Statis: `matplotlib` dan `seaborn` . . . 77
4.5.1 Setup Gaya Global . . . 77
4.5.2 Histogram dan Density . . . 78
4.5.3 Bar Chart yang Diurutkan . . . 79
4.5.4 Boxplot: Perbandingan Distribusi antar Kelompok . . . 80
4.5.5 Small Multiples: Faceting . . . 80
4.6 Visualisasi Interaktif: `plotly` . . . 81
4.6.1 Plotly Express: API Tingkat Tinggi . . . 81
4.6.2 Bar Chart Interaktif dengan Grouping . . . 82
4.6.3 Tren dan Waktu (Pratinjau) . . . 83

4.7 Kesalahan Visualisasi yang Harus Dihindari . . . 83
4.7.1 1. Sumbu Y yang Tidak Dimulai dari Nol . . . 84
4.7.2 2. Menggunakan Warna untuk Mengkodekan Terlalu Banyak Kategori . 84
4.7.3 3. Dual Y-Axis . . . 85
4.8 Membangun Dashboard: Prinsip dan Praktik . . . 86
4.8.1 Prinsip Dashboard yang Baik . . . 86
4.8.2 Contoh Dashboard Sederhana dengan Plotly . . . 86
4.8.3 Dashboard Interaktif dengan Widget . . . 88
4.9 Menyimpan Grafik untuk Buku . . . 88
4.10 Latihan Terbimbing . . . 88
4.11 Kotak "Jembatan": Dari Data ke Keputusan . . . 89
4.12 Rangkuman . . . 90
4.13 Jembatan ke Bab 5 . . . 91

5 Diagnostic Analytics 92
5.1 Dari "Apa" ke "Mengapa" . . . 92
5.2 Pola Pikir Diagnostik . . . 93
5.3 Stratifikasi: Mengurai Pola yang Tampak Sederhana . . . 93
5.3.1 Contoh Pertama: Apakah Efek Kontrak Bertahan di Semua Tingkat Tenure? . . . 93
5.4 Simpson's Paradox: Ketika Agregasi Menyesatkan . . . 95
5.4.1 Contoh Klasik . . . 96
5.4.2 Mencari Simpson's Paradox di Dataset Telco . . . 96
5.5 Drill-Down: Menelusuri Ke Dalam . . . 97
5.5.1 Contoh: Churn Rate per Segmen Bertingkat . . . 97
5.6 Analisis Kohort: Bagaimana Perilaku Berubah Sepanjang Waktu . . . 99
5.6.1 Churn Rate per Kelompok Tenure . . . 99
5.6.2 Kurva "Survival" Sederhana . . . 101
5.7 Root Cause Analysis: Kerangka Sistematis . . . 103
5.7.1 5 Whys . . . 103
5.7.2 Diagram Ishikawa (Fishbone) . . . 104
5.8 Visual Analytics untuk Diagnosis . . . 104
5.8.1 1. Heatmap Stratifikasi . . . 104
5.8.2 2. Small Multiples untuk Perbandingan . . . 105
5.8.3 3. Scatter dengan Anotasi . . . 106
5.9 Batas Diagnosis: Korelasi vs Kausalitas . . . 108
5.9.1 Tiga Syarat Kausalitas . . . 108
5.9.2 Metode Kausal yang Sederhana . . . 108
5.10 Studi Kasus: Diagnosis Churn Telco . . . 109
5.10.1 Langkah 1: Identifikasi Segmen Berisiko Tinggi . . . 109
5.10.2 Langkah 2: Drill-Down ke Segmen "Sangat Tinggi" . . . 110
5.10.3 Langkah 3: 5 Whys untuk Segmen Ini . . . 110
5.10.4 Langkah 4: Implikasi untuk Keputusan . . . 111

5.11 Latihan Terbimbing . . . 112
5.12 Kotak "Jembatan": Dari Data ke Keputusan . . . 112
5.13 Rangkuman . . . 113
5.14 Jembatan ke Bab 6 . . . 113

6 Studi Kasus & Komunikasi Insight **115**
6.1 Skenario: Dua Minggu untuk Menyelamatkan Pelanggan . . . 115
6.2 Kerangka SCQA: Cara Berpikir Sebelum Menulis Kode . . . 116
6.3 Menyusun Analisis: Alur Kerja End-to-End . . . 117
6.3.1 Langkah 1: Memuat Data Bersih . . . 117
6.3.2 Langkah 2: Menetapkan Baseline . . . 117
6.3.3 Langkah 3: Diagnosis — Di Mana Churn Terkonsentrasi? . . . 119
6.3.4 Langkah 4: Menghitung Dampak Finansial per Segmen . . . 120
6.4 Dari Analisis ke Narasi: Data Storytelling . . . 121
6.4.1 Tiga Tingkat Data Storytelling . . . 121
6.4.2 Struktur Narasi: Piramida Terbalik . . . 121
6.5 Dashboard Eksekutif . . . 122
6.5.1 Tiga KPI Utama . . . 122
6.5.2 Dashboard Detail: Tiga Grafik yang Menjawab "Siapa, Mengapa, Berapa" 124
6.6 Menyampaikan ke Stakeholder . . . 126
6.6.1 1. Buka dengan Jawaban, Bukan Metodologi . . . 126
6.6.2 2. Gunakan Analogi, Bukan Jargon . . . 126
6.6.3 3. Tunjukkan Ketidakpastian Secara Eksplisit . . . 126
6.6.4 4. Akhiri dengan Ajakan Bertindak yang Spesifik . . . 127
6.7 Tiga Rekomendasi Prioritas . . . 127
6.8 Batas Analisis Ini . . . 128
6.9 Kotak "Jembatan": Dari Diagnosis ke Prediksi . . . 128
6.10 Latihan Terbimbing . . . 129
6.11 Rangkuman . . . 130
6.12 Penutup Bagian I . . . 130
6.13 Jembatan ke Bagian II . . . 131

II Bagian II — Predictive Analytics **132**

7 Kerangka Predictive Analytics **133**
7.1 Dari "Mengapa" ke "Siapa" . . . 133
7.2 Formulasi Masalah: Sebelum Menyentuh Kode . . . 134
7.2.1 1. Apa yang Diprediksi? (Target) . . . 134
7.2.2 2. Untuk Siapa Prediksi Ini? (Audiens) . . . 135
7.2.3 3. Kapan Prediksi Dibutuhkan? (Waktu) . . . 135
7.2.4 4. Apa Konsekuensi Kesalahan? (Cost Matrix) . . . 135
7.2.5 5. Apa Baseline yang Harus Dikalahkan? (Baseline) . . . 136

7.3 Alur Kerja Pemodelan Prediktif . . . 136
7.4 Membagi Data: Train, Validation, Test . . . 137
7.4.1 Mengapa Kita Tidak Bisa Mengukur Performa pada Data Latih? . . . 137
7.4.2 Mengapa Tiga Pembagian, Bukan Dua? . . . 137
7.4.3 Menerapkan Pembagian Data . . . 138
7.4.4 Verifikasi: Apakah Distribusi Konsisten? . . . 139
7.5 Cross-Validation: Estimasi yang Lebih Stabil . . . 140
7.6 Bias-Variance Tradeoff: Inti dari Semua Pemodelan . . . 142
7.6.1 Dua Jenis Kesalahan . . . 143
7.6.2 Underfitting vs Overfitting . . . 143
7.6.3 Demonstrasi: Overfitting pada Decision Tree . . . 144
7.7 Data Leakage: Musuh yang Tidak Terlihat . . . 147
7.7.1 Contoh Data Leakage . . . 147
7.7.2 Contoh Temporal Leakage di Dataset Telco . . . 147
7.7.3 Mencegah Preprocessing Leakage . . . 148
7.8 Baseline Model: Yang Harus Dikalahkan . . . 148
7.9 Latihan Terbimbing . . . 151
7.10 Kotak “Jembatan”: Dari Data ke Keputusan . . . 152
7.11 Rangkuman . . . 152
7.12 Jembatan ke Bab 8 . . . 153

8 Regresi: Fondasi Pemodelan Statistik 154
8.1 Mengapa Regresi? . . . 154
8.2 Regresi Linear: Memodelkan Target Kontinu . . . 155
8.2.1 Bentuk Umum . . . 155
8.2.2 Contoh: Memprediksi Tagihan Bulanan . . . 155
8.2.3 Melatih Model dan Mengukur Performa . . . 156
8.3 Interpretasi Koefisien: Membaca Model . . . 157
8.4 Diagnostik Residual: Memeriksa Asumsi . . . 159
8.4.1 Plot Residual vs Fitted . . . 159
8.5 Multikolinearitas: Ketika Fitur Saling Berkorelasi . . . 160
8.5.1 Mendeteksi dengan VIF . . . 160
8.5.2 Solusi: Regularisasi . . . 161
8.6 Regresi Logistik: Memodelkan Target Biner . . . 163
8.6.1 Bentuk Umum . . . 163
8.6.2 Melatih Model pada Data Churn . . . 164
8.6.3 Interpretasi Koefisien sebagai Odds Ratio . . . 165
8.6.4 Perhatikan Tanda Koefisien . . . 166
8.7 Regresi Linear vs Logistik: Kapan Memakai Apa . . . 167
8.8 Menghubungkan Kembali ke Keputusan . . . 168
8.9 Latihan Terbimbing . . . 168
8.10 Kotak “Jembatan”: Dari Model ke Keputusan . . . 169
8.11 Rangkuman . . . 170

8.12 Jembatan ke Bab 9 . . . 170

9 Klasifikasi: Membandingkan Model dan Memilih yang Tepat 172
9.1 Dari Satu Model ke Enam . . . 172
9.2 Menyiapkan Data . . . 173
9.3 Enam Model Klasifikasi . . . 174
9.3.1 1. Logistic Regression — Baseline yang Kuat . . . 175
9.3.2 2. K-Nearest Neighbors — Berdasarkan Kemiripan . . . 223
9.3.3 3. Naive Bayes — Probabilitas Sederhana . . . 269
9.3.4 4. Decision Tree — Aturan yang Dapat Dibaca . . . 292
9.3.5 5. Random Forest — Kekuatan Ensemble . . . 343
9.3.6 6. Gradient Boosting — Model Terkuat di Kelas Tabular . . . 397
9.4 Evaluasi: Metrik yang Berbeda Menjawab Pertanyaan yang Berbeda . . . 451
9.4.1 Confusion Matrix: Fondasi Semua Metrik . . . 451
9.4.2 Precision, Recall, F1 . . . 453
9.4.3 ROC Curve: Trade-off Recall dan False Positive Rate . . . 454
9.4.4 Precision-Recall Curve: Lebih Informatif untuk Kelas Tidak Seimbang . . . 456
9.5 Memilih Metrik Berdasarkan Konsekuensi Bisnis . . . 458
9.5.1 Skenario 1: Program Retensi Murah . . . 458
9.5.2 Skenario 2: Program Retensi Mahal . . . 458
9.5.3 Skenario 3: Prioritisasi dengan Anggaran Terbatas . . . 458
9.6 Class Imbalance: Ketika Kelas Minoritas Terlupakan . . . 459
9.6.1 1. `class_weight='balanced'` . . . 459
9.6.2 2. Threshold Adjustment . . . 460
9.6.3 3. Resampling (SMOTE, Oversampling, Undersampling) . . . 462
9.7 Kalibrasi: Apakah Probabilitas Dapat Dipercaya? . . . 463
9.7.1 Mengukur Kalibrasi . . . 463
9.8 Feature Importance: Mengapa Model Memprediksi Apa yang Dia Prediksi . . . 465
9.9 Memilih Model Akhir . . . 467
9.9.1 Kerangka Keputusan . . . 467
9.9.2 Rekomendasi untuk Dataset Telco . . . 467
9.10 Latihan Terbimbing . . . 468
9.11 Kotak “Jembatan”: Dari Prediksi ke Keputusan . . . 469
9.12 Rangkuman . . . 470
9.13 Jembatan ke Bab 10 . . . 471

10 Time Series & Forecasting 472
10.1 Mengapa Kita Beralih Dataset . . . 472
10.2 Memuat Dataset Retail . . . 473
10.2.1 Inspeksi Awal . . . 474
10.3 Dari Cross-Sectional ke Time Series . . . 475
10.4 Agregasi: Dari Transaksi ke Deret Waktu . . . 475

10.5 Dekomposisi Time Series . . . 477
10.5.1 Dekomposisi Aditif vs Multiplikatif . . . 477
10.5.2 Melakukan Dekomposisi . . . 478
10.6 Uji Stasioneritas . . . 480
10.7 Forecasting: Meramalkan Masa Depan . . . 482
10.7.1 Metode 1: Naive Forecast (Baseline) . . . 482
10.7.2 Metode 2: Exponential Smoothing (Holt-Winters) . . . 482
10.7.3 Metode 3: ARIMA . . . 483
10.7.4 Membandingkan Ketiga Metode . . . 483
10.8 Evaluasi Forecast: Metrik yang Berbeda . . . 484
10.9 Rolling-Origin Cross-Validation . . . 486
10.10 Analisis Survival: Kembali ke Telco . . . 488
10.10.1 Konsep Dasar . . . 488
10.10.2 Membangun Kurva Survival . . . 489
10.10.3 Uji Log-Rank: Apakah Kurva Berbeda Secara Signifikan? . . . 491
10.10.4 Implikasi untuk Keputusan . . . 491
10.11 Menghubungkan Kembali ke Dataset Retail . . . 492
10.12 Latihan Terbimbing . . . 492
10.13 Kotak “Jembatan”: Dari Prediksi ke Simulasi . . . 493
10.14 Rangkuman . . . 494
10.15 Jembatan ke Bab 11 . . . 494

11 Unsupervised Learning: Menemukan Struktur Tersembunyi 495
11.1 Mengapa Unsupervised Learning? . . . 495
11.2 Menyiapkan Data . . . 496
11.3 K-Means Clustering . . . 498
11.3.1 Memilih Jumlah Cluster (K) . . . 498
11.3.2 Melatih K-Means dengan K Optimal . . . 500
11.3.3 Visualisasi Segmen . . . 501
11.4 Hierarchical Clustering . . . 503
11.4.1 Membangun Dendrogram . . . 503
11.4.2 Membandingkan dengan K-Means . . . 504
11.5 PCA: Mereduksi Dimensi . . . 506
11.5.1 Mengapa PCA? . . . 506
11.5.2 Menerapkan PCA . . . 506
11.5.3 Visualisasi 2D dengan PCA . . . 508
11.6 Association Rules: Market Basket Analysis . . . 510
11.6.1 Konsep Dasar . . . 510
11.6.2 Menyiapkan Data Transaksi . . . 511
11.6.3 Menerapkan Apriori . . . 511
11.6.4 Interpretasi Hasil Komputasi Analitik . . . 514
11.6.5 Catatan tentang Support 0,123 . . . 515
11.6.6 Visualisasi Rules . . . 516

11.7 Menghubungkan Segmen ke Keputusan . . . 517
11.7.1 Dari Segmentasi ke Alokasi Anggaran . . . 517
11.7.2 Dari Association Rules ke Strategi Bundling . . . 518
11.8 Latihan Terbimbing . . . 519
11.9 Kotak "Jembatan": Dari Struktur ke Optimasi . . . 520
11.10Rangkuman . . . 521
11.11Jembatan ke Bab 12 . . . 521

12 Simulasi Monte Carlo **522**
12.1 Dari Prediksi ke Distribusi . . . 522
12.2 Konsep Dasar Simulasi Monte Carlo . . . 523
12.2.1 Tiga Langkah . . . 523
12.2.2 Contoh Sederhana: Estimasi . . . 523
12.3 Membangkitkan Distribusi dari Data . . . 526
12.3.1 Pendekatan 1: Distribusi Teoretis . . . 526
12.3.2 Pendekatan 2: Distribusi Empiris (Bootstrap) . . . 528
12.4 Aplikasi 1: Ketidakpastian Churn (Telco) . . . 529
12.4.1 Model . . . 529
12.4.2 Simulasi . . . 530
12.4.3 Interpretasi untuk Keputusan . . . 531
12.5 Aplikasi 2: Ketidakpastian Permintaan (Retail) . . . 532
12.5.1 Model . . . 532
12.5.2 Simulasi Keputusan: Berapa Stok Optimal? . . . 533
12.6 Risiko dan Value-at-Risk . . . 535
12.7 Memilih Distribusi: Panduan Praktis . . . 536
12.8 Batas Simulasi Monte Carlo . . . 537
12.9 Latihan Terbimbing . . . 538
12.10Kotak "Jembatan": Dari Simulasi ke Optimasi . . . 539
12.11Rangkuman . . . 539
12.12Jembatan ke Bagian III . . . 540

III Bagian III — Prescriptive Analytics **541**

13 Fondasi Prescriptive Analytics **542**
13.1 Dari "Apa yang Akan Terjadi?" ke "Apa yang Harus Dilakukan?" . . . 542
13.2 Anatomi Masalah Keputusan . . . 543
13.2.1 Contoh: Keputusan Retensi Pelanggan . . . 543
13.3 Payoff Table: Memformalkan Keputusan . . . 544
13.3.1 Membangun Payoff Table untuk Keputusan Retensi . . . 544
13.4 Kriteria Keputusan di Bawah Ketidakpastian . . . 545
13.4.1 1. Maximin (Pesimis) . . . 546
13.4.2 2. Maximax (Optimis) . . . 546

13.4.3 3. Minimax Regret (Menyesali Keputusan yang Salah) . . . 547
13.4.4 4. Expected Value (Rasional-Netral terhadap Risiko) . . . 548
13.5 Decision Tree: Memformalkan Keputusan Berurutan . . . 550
13.5.1 Contoh: Keputusan Retensi Dua Tahap . . . 551
13.6 Nilai Informasi: Seberapa Berharga Informasi Tambahan? . . . 554
13.6.1 Expected Value of Perfect Information (EVPI) . . . 554
13.6.2 Expected Value of Sample Information (EVSI) . . . 555
13.7 Menghubungkan Kembali ke Model Prediktif . . . 557
13.7.1 Probabilitas Churn sebagai Input . . . 557
13.8 Dari Payoff Table ke Optimasi . . . 558
13.8.1 Contoh: Alokasi Anggaran Retensi . . . 558
13.9 Latihan Terbimbing . . . 560
13.10 Kotak “Jembatan”: Dari Keputusan ke Optimasi . . . 561
13.11 Rangkuman . . . 562
13.12 Jembatan ke Bab 14 . . . 562

14 Linear Programming: Optimasi di Bawah Kendala 563
14.1 Mengapa Linear Programming? . . . 563
14.2 Anatomi Model Linear Programming . . . 564
14.2.1 Asumsi Linear Programming . . . 564
14.3 Contoh Klasik: Optimasi Bauran Produksi . . . 565
14.3.1 Formulasi . . . 565
14.3.2 Solusi Grafis . . . 565
14.4 Implementasi dengan PuLP . . . 569
14.5 Aplikasi 1: Alokasi Anggaran Retensi . . . 570
14.5.1 Skenario . . . 570
14.5.2 Formulasi LP . . . 571
14.5.3 Analisis Sensitivitas: Shadow Price . . . 573
14.6 Aplikasi 2: Optimasi Bauran Produksi Retail . . . 574
14.6.1 Formulasi . . . 575
14.7 Kapan LP Tidak Cukup? . . . 578
14.8 Latihan Terbimbing . . . 578
14.9 Kotak “Jembatan”: Dari LP ke Keputusan Nyata . . . 579
14.10 Rangkuman . . . 580
14.11 Jembatan ke Bab 15 . . . 581

15 Integer & Nonlinear Programming 582
15.1 Mengapa Integer Programming? . . . 582
15.2 Tiga Kelas Model Integer . . . 583
15.2.1 Kompleksitas Komputasi . . . 583
15.3 Binary Programming: Keputusan Ya/Tidak . . . 584
15.3.1 Formulasi Umum . . . 584
15.3.2 Aplikasi 1: Pemilihan Pelanggan untuk Intervensi Retensi . . . 584

15.3.3 Formulasi Binary Programming . . . 585
15.3.4 Membandingkan dengan LP-Relaxation . . . 587
15.4 Logical Constraints: Kekuatan Binary Programming . . . 595
15.4.1 Empat Pola Logika yang Umum . . . 595
15.4.2 Aplikasi 2: Lokasi Fasilitas (Facility Location) . . . 596
15.4.3 Formulasi Facility Location . . . 597
15.4.4 Aplikasi 3: Penjadwalan Shift . . . 598
15.5 Nonlinear Programming . . . 601
15.5.1 Contoh: Optimasi Anggaran Iklan dengan Diminishing Returns . . . 601
15.5.2 Kapan Nonlinear Programming Digunakan? . . . 604
15.6 Memilih Teknik yang Tepat . . . 604
15.7 Latihan Terbimbing . . . 605
15.8 Kotak “Jembatan”: Dari Optimasi ke Keputusan Nyata . . . 606
15.9 Rangkuman . . . 607
15.10Jembatan ke Bab 16 . . . 608

16 Network & Multi-Objective Optimization 609
16.1 Mengapa Network Optimization? . . . 609
16.2 Anatomi Jaringan . . . 610
16.3 Transportation Problem: Distribusi Barang . . . 610
16.3.1 Formulasi . . . 610
16.3.2 Aplikasi: Distribusi Retail . . . 611
16.3.3 Solusi dengan PuLP . . . 612
16.4 Assignment Problem: Penugasan Sumber Daya . . . 615
16.4.1 Formulasi . . . 615
16.4.2 Aplikasi: Penugasan Sales ke Wilayah . . . 615
16.4.3 Solusi Assignment . . . 616
16.5 Multi-Objective Optimization . . . 617
16.5.1 Konsep Pareto Optimality . . . 618
16.5.2 Metode Weighted Sum . . . 618
16.5.3 Aplikasi: Alokasi Anggaran Retensi dengan Dua Tujuan . . . 618
16.5.4 Memilih Titik di Pareto Frontier . . . 623
16.6 Minimum Cost Flow: Kerangka Umum . . . 625
16.7 Latihan Terbimbing . . . 626
16.8 Kotak “Jembatan”: Dari Jaringan ke Simulasi Keputusan . . . 627
16.9 Rangkuman . . . 628
16.10Jembatan ke Bab 17 . . . 628

17 Simulation for Decision Making 630
17.1 Mengapa Optimasi Determinstik Tidak Cukup . . . 630
17.2 Tiga Pendekatan untuk Keputusan di Bawah Ketidakpastian . . . 631
17.2.1 1. Expected Value Optimization . . . 631
17.2.2 2. Robust Optimization . . . 631

17.2.3 3. Stochastic Programming 632
17.3 Simulation-Optimization: Kerangka Kerja 632
17.4 Aplikasi 1: Alokasi Retensi di Bawah Ketidakpastian 633
17.4.1 Menyiapkan Model Probabilistik 633
17.4.2 Membangkitkan Skenario 634
17.4.3 Strategi 1: Expected Value Optimization 635
17.4.4 Strategi 2: Robust Optimization 636
17.4.5 Evaluasi Out-of-Sample: Membandingkan Strategi 637
17.5 Aplikasi 2: Inventori Retail di Bawah Ketidakpastian 640
17.5.1 Model 640
17.5.2 Pendekatan: Optimasi untuk Setiap Skenario 640
17.5.3 Value-at-Risk untuk Keputusan Inventori 643
17.6 Scenario-Based Optimization: Pendekatan Lebih Kaya 644
17.6.1 Formulasi 645
17.7 Latihan Terbimbing 645
17.8 Kotak “Jembatan”: Dari Simulasi ke Integrasi 646
17.9 Rangkuman 647
17.10 Jembatan ke Bab 18 648

18 Integrasi & Proyek Akhir **649**
18.1 Skenario: Rapat Direksi Telko Nusantara 649
18.2 Ringkasan Perjalanan: Dari Data ke Keputusan 650
18.3 Fase 1: Business Understanding 650
18.3.1 Pertanyaan Strategis 650
18.3.2 Definisi Keberhasilan 651
18.4 Fase 2: Descriptive & Diagnostic — Mengingat Kembali 651
18.5 Fase 3: Predictive — Model Churn 653
18.6 Fase 4: Prescriptive — Optimasi Alokasi 655
18.7 Fase 5: Simulation — Evaluasi di Bawah Ketidakpastian 660
18.8 Fase 6: Executive Dashboard 662
18.9 Fase 7: Rekomendasi untuk Direksi 666
18.9.1 Tiga Rekomendasi Prioritas 666
18.9.2 Satu Halaman Eksekutif 666
18.10 Fase 8: Deployment & Monitoring 666
18.10.1 Rencana Deployment 667
18.10.2 Monitoring Model 667
18.11 Latihan Terbimbing 667
18.12 Kotak “Jembatan”: Dari Proyek ke Praktik 668
18.13 Refleksi Akhir: Perjalanan Anda 669
18.14 Rangkuman 669
18.15 Penutup: Setelah Buku Ini 670

Ringkasan & Refleksi **671**
Perjalanan yang Telah Kita Tempuh . . . 671
Tiga Pergeseran Pola Pikir . . . 672
1. Dari Data ke Keputusan . . . 672
2. Dari Kepastian ke Probabilitas . . . 672
3. Dari Korelasi ke Kausalitas (dan Kembali Lagi) . . . 672
Sepuluh Pelajaran Kunci . . . 672
Yang Tidak Diajarkan Buku Ini . . . 673
Untuk Pembaca yang Berbeda . . . 674
Jika Anda Mahasiswa . . . 674
Jika Anda Pengajar . . . 675
Jika Anda Praktisi . . . 675
Setelah Buku Ini: Langkah Berikutnya . . . 675
Untuk Pembelajar Mandiri . . . 675
Untuk Pengajar . . . 675
Penutup . . . 676

References **677**
Daftar Pustaka . . . 677
Bacaan Lanjutan . . . 678
Fondasi Analytics & Statistika . . . 679
Machine Learning & Predictive Analytics . . . 679
Optimasi & Riset Operasi . . . 679
Simulasi & Analisis Keputusan . . . 679
Time Series . . . 680
Visualisasi & Komunikasi . . . 680
Causal Inference (Topik Lanjutan) . . . 680
MLOps & Deployment (Topik Lanjutan) . . . 680
Etika & Dampak Sosial . . . 681
Cara Mengutip Buku Ini . . . 681
Lisensi Konten . . . 681

Pengantar

Buku ini lahir dari satu kebiasaan sederhana: **belajar sambil mengajar**. Setiap bab yang Anda baca adalah hasil dari proses menyiapkan materi diskusi, pelatihan, maupun kuliah, mengujinya di ruang diskusi, training dan kelas, lalu merevisinya berdasarkan pertanyaan partisipan. Karena itu, buku ini bukan monograf yang selesai sekali jadi—ia adalah buku ajar yang tumbuh bersama pembacanya.

Video pengantar

Tip

Mengapa buku ini ditulis — pengantar singkat tentang latar belakang, tujuan, dan filosofi buku.
https://www.youtube.com/embed/XXXXXXXXXXX

Mengapa buku ini ada

Keputusan yang baik jarang lahir dari satu teknik tunggal. Ia biasanya merupakan hasil perjalanan panjang: memahami data apa adanya (*descriptive*), mencari tahu mengapa sesuatu terjadi (*diagnostic*), memperkirakan apa yang akan terjadi (*predictive*), lalu memilih tindakan terbaik di bawah ketidakpastian (*prescriptive*). Sayangnya, keempat tahap ini sering diajarkan terpisah—statistika di satu semester atau pada satu course, machine learning di judul yang lain, dan riset operasi di kuliah atau training terpisah—tanpa benang merah yang jelas.

Buku ini mencoba menyatukan keempatnya dalam **satu narasi**, menggunakan **satu bahasa (Python)**, dan **satu atau lebih dataset pendamping** yang relevan atau sama dari awal hingga akhir. Tujuannya bukan menjadikan Anda ahli di setiap teknik, melainkan membekali Anda dengan cara berpikir yang utuh: dari data mentah hingga rekomendasi keputusan.

Satu perjalanan, tiga tahap

> Tip
>
> **Peta perjalanan buku** — animasi singkat yang menunjukkan alur tiga bagian dan bagaimana satu dataset yang sama dianalisis secara berbeda di setiap bagian.
> https://www.youtube.com/embed/YYYYYYYYYY

Buku ini terbagi menjadi tiga bagian besar yang mencerminkan tiga tahap perjalanan tersebut:

Bagian	Pertanyaan Kunci	Kompetensi yang Diharapkan
I. Descriptive & Diagnostic	Apa yang telah terjadi, dan mengapa?	Membersihkan, mengeksplorasi, dan mendiagnosis data menjadi insight
II. Predictive	Apa yang kemungkinan besar akan terjadi?	Membangun dan memvalidasi model prediksi
III. Prescriptive	Apa tindakan terbaik yang harus diambil?	Merumuskan dan menyelesaikan model optimasi serta simulasi keputusan

Bagian I—VI dapat dibaca secara berurutan sebagai satu buku utuh, tetapi juga dirancang agar tiap bagian bisa dipakai sebagai modul course mandiri.

Untuk siapa buku ini

Buku ini ditulis untuk tiga kelompok pembaca yang sering kali tumpang tindih:

- **Mahasiswa** tingkat akhir S1 atau awal S2 di bidang bisnis, ekonomi, teknik industri, statistika, atau ilmu komputer yang ingin melihat gambaran utuh analytics—bukan hanya satu teknik.
- **Praktisi** yang sudah mengenal satu alat (misalnya Excel atau SQL) dan ingin memperluas cakrawala ke pemodelan prediktif dan optimasi.
- **Dosen dan pengajar** yang mencari kerangka ajar siap pakai, dengan contoh kode yang bisa direproduksi dan studi kasus yang bisa langsung diadaptasi.

Bagaimana buku ini disusun

Setiap bab mengikuti pola yang sama agar mudah diikuti:

1. **Tujuan pembelajaran** — tiga sampai empat butir yang menjadi target bab.
2. **Konsep inti** — narasi yang menjelaskan *mengapa*, bukan hanya *bagaimana*.
3. **Contoh kode Python** — dapat dijalankan langsung dari notebook.
4. **Latihan terbimbing** — untuk memperkuat pemahaman melalui praktik.
5. **Studi kasus mini** — menerapkan konsep pada dataset pendamping.
6. **Rangkuman** — poin-poin kunci dan jembatan ke bab berikutnya.

Selain itu, bab-bab di Bagian II dan III memuat **kotak "Jembatan"** yang menghubungkan hasil prediksi ke keputusan nyata—supaya Anda tidak berhenti di angka akurasi atau profit, tetapi sampai pada implikasi manajerialnya.

Cara menggunakan buku ini

- **Jika Anda pembelajar mandiri**, baca berurutan dari Bagian I. Kerjakan latihan di setiap bab sebelum melanjutkan—buku ini paling bermanfaat jika Anda *menulis kode*, bukan hanya membacanya.
- **Jika Anda pengajar**, Bagian I–III dapat dipetakan langsung ke tiga course. Materi slide dan video pendek dapat ditemukan di folder `slides/` pada repositori pendamping.
- **Jika Anda praktisi**, lompat langsung ke bagian yang relevan. Bab 1 (Fondasi) tetap disarankan sebagai titik masuk karena ia memperkenalkan dataset dan konvensi buku.

Prasyarat dan persiapan

> Tip
>
> **Menyiapkan lingkungan Python** — video 15 menit yang memandu instalasi, pembuatan virtual environment, dan pengujian `pandas` serta `scikit-learn`.
> https://www.youtube.com/embed/ZZZZZZZZZZZ

Buku ini mengasumsikan Anda sudah mengenal dasar-dasar Python (variabel, fungsi, list, dan dictionary). Jika belum, banyak tutorial pengantar gratis yang bisa menjadi bekal awal. Yang perlu Anda siapkan:

- **Python 3.10 atau lebih baru** (disarankan lewat Miniconda atau `uv`).
- **Editor**: VS Code dengan ekstensi Python dan Quarto, atau Jupyter Lab.

- **Paket Python** yang tercantum dalam `requirements.txt` di repositori buku. Cukup jalankan:

```
pip install -r requirements.txt
```

- **Quarto CLI** (versi 1.4 ke atas) jika Anda ingin merender buku atau mengubah bab.

Semua kode di buku ini dijalankan dengan `random_state` tetap agar hasilnya dapat direproduksi. Jika Anda memperoleh angka yang sedikit berbeda pada model stokastik, hal ini wajar.

Dataset pendamping

Seluruh buku menggunakan **satu dataset pendamping** sebagai benang merah—sebuah dataset publik tentang hal yang paling relevan di era saat ini. Dataset yang sama akan:

- dieksplorasi dan didiagnosis di **Bagian I**,
- dimodelkan untuk prediksi di **Bagian II**,
- dan dijadikan dasar rekomendasi keputusan di **Bagian III**.

Dengan cara ini, Anda melihat bagaimana ***satu kumpulan data dapat menghasilkan jenis pertanyaan dan jawaban yang berbeda*** tergantung pada tahap analytics yang sedang Anda kerjakan.

Konvensi

- **Kode dalam blok abu-abu** dapat dijalankan langsung jika Anda membuka buku dalam format notebook atau HTML interaktif.
- **Kotak "Jembatan"** menandai transisi dari analisis ke keputusan.
- **Kotak "Catatan"** berisi peringatan atau asumsi penting.
- **Rujukan** menggunakan format BibTeX dan dikelola melalui references.bib. Anda dapat melihat daftar lengkapnya di bagian akhir buku.

Umpan balik dan kontribusi

Buku ini terbuka untuk koreksi dan perbaikan. Jika Anda menemukan kekeliruan, memiliki saran studi kasus, atau ingin menerjemahkan bab ke bahasa lain, silakan buka issue atau kirim *pull request* melalui repositori:

> **Repositori**: github.com/username/decision-analytics

Ucapan terima kasih

Buku ini tidak akan sampai ke tangan Anda tanpa dukungan dari yang saya sayangi *Argia* dan *Nurlina*, serta rekan-rekan tim peneliti: *Dr. Rusbani, Dr. Dien Kurtanty*, dan *Dr. Ade Heryana*. Terima kasih khusus juga kepada para kontributor yang, melalui pertanyaan-pertanyaan mereka di ruang diskusi atau kelas, tanpa sadar telah menulis ulang banyak bagian dari buku ini.

Selamat membaca—dan selamat mengambil keputusan yang lebih baik.

Introduction

> Bab ini adalah **peta jalan**. Ia tidak mengajarkan satu teknik pun secara mendalam, tetapi memberi Anda kerangka berpikir yang akan dipakai di seluruh buku: apa itu *decision analytics*, bagaimana keempat tingkat analytics saling terhubung, dan mengapa urutan tersebut penting. Di akhir bab, Anda akan melihat **satu contoh utuh**—dari data mentah hingga rekomendasi keputusan—dalam bentuk yang sangat disederhanakan.

Apa itu Decision Analytics?

Decision Analytics adalah disiplin yang menggunakan data, model, dan penalaran untuk **meningkatkan kualitas keputusan**. Kata kuncinya adalah *keputusan*, bukan *analisis*. Banyak organisasi menghasilkan laporan dan dashboard yang rapi tetapi tetap mengambil keputusan dengan perut—bukan karena analisisnya salah, melainkan karena analisisnya tidak pernah dirancang untuk berujung pada tindakan.

Tiga gagasan membedakan decision analytics dari sekadar "analisis data":

1. **Fokus pada keputusan, bukan pada data.** Pertanyaan pertama bukan "data apa yang kita punya?" melainkan "keputusan apa yang akan diambil, dan informasi apa yang dapat memperbaikinya?"
2. **Nilai datang dari tindakan, bukan dari insight.** Model yang akurat tetapi tidak pernah dipakai tidak memiliki nilai. Sebaliknya, model sederhana yang mengubah cara orang mengambil keputusan bisa bernilai besar.
3. **Data, model, dan penilaian manusia bekerja bersama.** Model memberi rekomendasi; manusia memutuskan dengan mempertimbangkan konteks, etika, dan batasan yang tidak tertangkap dalam data.

Definisi kerja. *Decision Analytics adalah proses mengubah data menjadi rekomendasi tindakan yang dapat dieksekusi, dengan memperhitungkan ketidakpastian dan keterbatasan dunia nyata.*

Tangga Analytics: Dari Data ke Keputusan

Literatur analytics biasanya membedakan empat tingkat kematangan, yang secara kolektif dikenal sebagai **tangga analytics**:

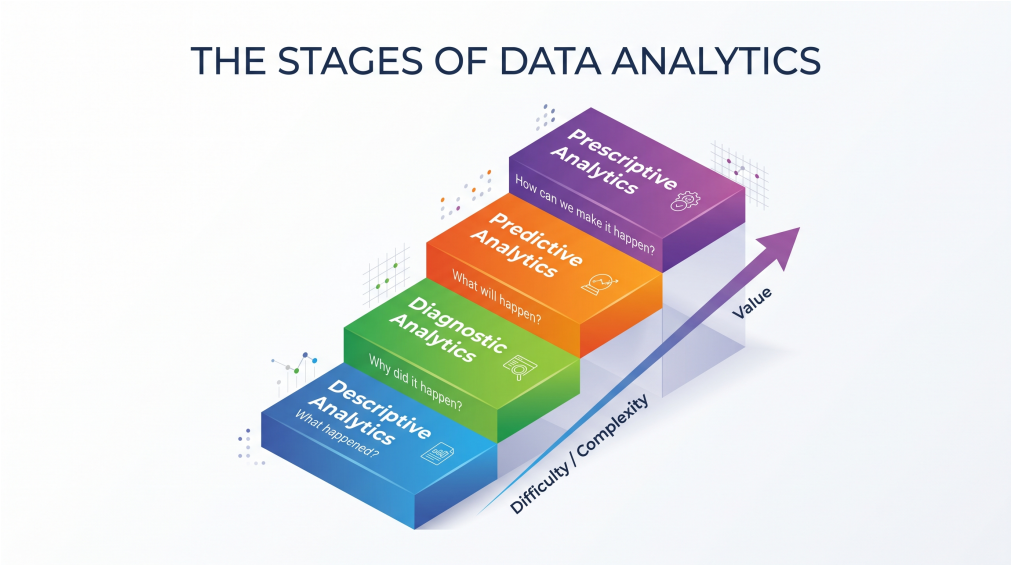


Figure 1: Tangga analytics: descriptive, diagnostic, predictive, prescriptive.

Tingkat	Pertanyaan	Contoh Keluaran	Disiplin Utama
Descriptive	Apa yang telah terjadi?	Ringkasan penjualan bulanan, dashboard KPI	Statistika, business intelligence
Diagnostic	Mengapa hal itu terjadi?	Analisis akar masalah, segmentasi penyebab	Statistika, visual analytics
Predictive	Apa yang kemungkinan besar akan terjadi?	Forecast permintaan, skor risiko churn	Machine learning, ekonometrika
Prescriptive	Apa yang sebaiknya kita lakukan?	Jadwal optimal, alokasi anggaran, kebijakan harga	Riset operasi, simulasi, teori keputusan

Diagnostic sering diperlakukan sebagai "sub-tingkat" dari descriptive, karena keduanya bekerja pada data historis. Namun secara praktik, lompatan dari *apa* ke *mengapa* membutuhkan teknik dan kebiasaan berpikir yang cukup berbeda—sehingga dalam buku ini ia mendapat porsi tersendiri di Bagian I.

> **Tip**
>
> Tangga ini bukan hierarki nilai—setiap tingkat bernilai jika menjawab pertanyaan yang tepat. Perusahaan yang belum memiliki dashboard yang andal tidak akan banyak memperoleh manfaat dari model optimasi canggih. Sebagian besar kegagalan analytics bukan karena tekniknya terlalu sederhana, melainkan karena fondasinya belum kokoh.

Siklus Pengambilan Keputusan

Analytics bukan kegiatan sekali jalan. Ia adalah bagian dari siklus yang berulang:



Figure 2: Siklus pengambilan keputusan berbasis analytics.

1. **Framing** — Merumuskan keputusan secara eksplisit: apa pilihan yang tersedia, apa tujuannya, apa kendalanya.
2. **Data** — Mengumpulkan dan menyiapkan data yang relevan dengan keputusan tersebut.
3. **Analysis** — Menerapkan teknik deskriptif, prediktif, atau preskriptif sesuai kebutuhan.
4. **Decision** — Memilih tindakan, dengan mempertimbangkan hasil analisis dan konteks manusiawi.
5. **Action** — Melaksanakan keputusan.
6. **Learning** — Mengukur hasil, membandingkan dengan ekspektasi, dan memperbaiki model maupun proses.

Banyak kegagalan analytics terjadi karena satu atau lebih tahap ini dilewati. Model prediksi yang bagus tetapi tidak pernah sampai ke **Action** hanya menghasilkan laporan. Keputusan yang diambil tanpa **Framing** yang jelas akan menghasilkan analisis yang tidak relevan.

Sneak Peek: Satu Contoh, Empat Tingkat Analytics

Sebelum masuk ke bab-bab teknis, mari lihat bagaimana keempat tingkat analytics bekerja pada **satu kasus yang sama**. Contoh ini sangat disederhanakan—detailnya akan dibahas di bagian-bagian berikutnya—tetapi tujuannya adalah memperlihatkan bahwa keempat tingkat itu bukan empat topik terpisah, melainkan **empat lapisan dari satu percakapan yang sama**.

Skenario. Sebuah kedai kopi ingin memahami penjualannya dan memutuskan berapa banyak barista yang perlu dijadwalkan setiap jam.

```
import numpy as np
import pandas as pd

# Membangkitkan data sintetis: penjualan per jam selama 30 hari
np.random.seed(42)
jam = pd.date_range("2025-01-01", periods=30 * 24, freq="h")
pola_harian = 20 + 10 * np.sin(np.arange(len(jam)) * 2 * np.pi / 24)
penjualan = np.maximum(0, pola_harian + np.random.normal(0, 3, len(jam)))

df = pd.DataFrame({"waktu": jam, "penjualan": penjualan})
df["jam"] = df["waktu"].dt.hour
df["hari"] = df["waktu"].dt.day_name()
```

Tingkat 1 — Descriptive: Apa yang terjadi?

```
ringkasan = df.groupby("jam")["penjualan"].mean().round(1)
print(ringkasan)
```

```
jam
0     20.3
1     22.0
2     25.0
3     26.8
4     28.5
5     29.7
6     30.2
7     29.3
8     28.7
9     27.8
```

```
10    25.0
11    23.1
12    19.9
13    16.8
14    14.5
15    13.1
16    10.5
17    11.7
18    11.0
19    10.4
20    10.9
21    12.4
22    14.2
23    17.4
Name: penjualan, dtype: float64
```

Keluaran ini memberi kita gambaran: pada jam berapa penjualan tinggi, pada jam berapa rendah. Ini adalah pekerjaan **descriptive analytics**—meringkas masa lalu agar dapat dipahami.

Tingkat 2 — Diagnostic: Mengapa?

```
per_hari = df.groupby("hari")["penjualan"].mean().round(1)
print(per_hari)
```

```
hari
Friday       20.0
Monday       19.9
Saturday     20.0
Sunday       20.2
Thursday     20.0
Tuesday      20.3
Wednesday    19.6
Name: penjualan, dtype: float64
```

Jika kita melihat bahwa Senin pagi secara konsisten lebih rendah dari hari lain, kita bisa mulai bertanya *mengapa*—apakah karena lokasi, cuaca, kompetitor, atau karena memang pola pelanggan. Ini adalah pekerjaan **diagnostic analytics**: tidak berhenti pada "apa", tetapi mengejar "mengapa".

Tingkat 3 — Predictive: Apa yang akan terjadi?

```
# Forecast sederhana: rata-rata historis per jam sebagai prediksi
prediksi_per_jam = df.groupby("jam")["penjualan"].mean().round(1)
print(prediksi_per_jam)
```

```
jam
0     20.3
1     22.0
2     25.0
3     26.8
4     28.5
5     29.7
6     30.2
7     29.3
8     28.7
9     27.8
10    25.0
11    23.1
12    19.9
13    16.8
14    14.5
15    13.1
16    10.5
17    11.7
18    11.0
19    10.4
20    10.9
21    12.4
22    14.2
23    17.4
Name: penjualan, dtype: float64
```

Model prediksi yang sesungguhnya (dibahas di Bagian II) akan jauh lebih canggih—mempertimbangkan hari dalam seminggu, musim, promosi, dan sebagainya. Tetapi idenya sama: mengubah data historis menjadi **estimasi tentang masa depan**.

Tingkat 4 — Prescriptive: Apa yang sebaiknya dilakukan?

```python
from pulp import LpVariable, LpMinimize, LpProblem, lpSum, LpInteger, value

# Setiap barista melayani ~5 pelanggan/jam; tambahkan margin keamanan 10%
kapasitas_per_barista = 5
margin = 1.1

barista = LpVariable.dicts("barista", range(24), lowBound=1, cat=LpInteger)
model = LpProblem("penjadwalan_barista", LpMinimize)
model += lpSum([barista[h] for h in range(24)])  # minimalkan total barista

for h in range(24):
    model += barista[h] * kapasitas_per_barista >= prediksi_per_jam.iloc[h] * margin

model.solve()

jadwal = {h: int(value(barista[h])) for h in range(24)}
print(jadwal)
```

```
Welcome to the CBC MILP Solver
Version: 2.10.3
Build Date: Dec 15 2019

command line - /Users/chakraroy/miniconda3/envs/decanal/lib/python3.11/site-
packages/pulp/apis/../solverdir/cbc/osx/i64/cbc /var/folders/67/ntcn5rhj6r98wjkx_hg095lm0000g
pulp.mps -timeMode elapsed -solve -printingOptions all -solution /var/folders/67/ntcn5rhj6r98
pulp.sol (default strategy 1)
At line 2 NAME          MODEL
At line 3 ROWS
At line 29 COLUMNS
At line 126 RHS
At line 151 BOUNDS
At line 176 ENDATA
Problem MODEL has 24 rows, 24 columns and 24 elements
Coin0008I MODEL read with 0 errors
Option for timeMode changed from cpu to elapsed
Continuous objective value is 105.424 - 0.00 seconds
Cgl0004I processed model has 0 rows, 0 columns (0 integer (0 of which binary)) and 0 elements
Cbc3007W No integer variables - nothing to do
Cuts at root node changed objective from 118 to -1.79769e+308
Probing was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cu
Gomory was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cut
```

```
Knapsack was tried 0 times and created 0 cuts of which 0 were active after adding rounds of c
Clique was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cu
MixedIntegerRounding2 was tried 0 times and created 0 cuts of which 0 were active after addi
FlowCover was tried 0 times and created 0 cuts of which 0 were active after adding rounds of
TwoMirCuts was tried 0 times and created 0 cuts of which 0 were active after adding rounds o
ZeroHalf was tried 0 times and created 0 cuts of which 0 were active after adding rounds of c

Result - Optimal solution found

Objective value:                118.00000000
Enumerated nodes:               0
Total iterations:               24
Time (CPU seconds):             0.00
Time (Wallclock seconds):       0.00

Option for printingOptions changed from normal to all
Total time (CPU seconds):       0.00   (Wallclock seconds):       0.00

{0: 5, 1: 5, 2: 6, 3: 6, 4: 7, 5: 7, 6: 7, 7: 7, 8: 7, 9: 7, 10: 6, 11: 6, 12: 5, 13: 4, 14:
```

Sekarang kita tidak lagi sekadar memprediksi—kita **merekomendasikan tindakan**: berapa barista yang harus dijadwalkan pada setiap jam agar kapasitas cukup, dengan jumlah barista seminimal mungkin. Ini adalah **prescriptive analytics**.

Apa yang baru saja kita lihat?

Satu dataset, empat pertanyaan berbeda, empat jenis keluaran:

Tingkat	Pertanyaan	Output pada contoh ini
Descriptive	Apa yang terjadi?	Rata-rata penjualan per jam
Diagnostic	Mengapa?	Perbedaan penjualan antahari
Predictive	Apa yang akan terjadi?	Estimasi penjualan per jam esok hari
Prescriptive	Apa yang sebaiknya dilakukan?	Jadwal barista yang optimal

Dalam praktik nyata, setiap tingkat membutuhkan teknik, data, dan pertimbangan yang jauh lebih kaya. Tetapi struktur berpikirnya sama—dan itulah yang akan Anda kuasai melalui buku ini.

> **Note**
>
> Demo di atas menggunakan data sintetis agar dapat dijalankan tanpa dataset eksternal. Dataset pendamping buku ini (lihat Dataset Pendamping) akan menjadi bahan untuk versi yang jauh lebih realistis di setiap bagian.

Peta Disiplin Ilmu

Decision analytics bukan disiplin tunggal; ia adalah simpul dari beberapa tradisi intelektual yang lebih tua:

- **Statistika** memberi kita bahasa untuk mengukur ketidakpastian, menguji hipotesis, dan memisahkan sinyal dari noise.
- **Machine learning** memperluas statistika ke data berdimensi tinggi dan pola non-linear yang sulit dinyatakan dalam model klasik.
- **Riset operasi (operations research)** memberi kita alat untuk mengoptimalkan—merumuskan keputusan sebagai masalah matematis dengan tujuan dan kendala.
- **Business intelligence** dan **visual analytics** menyediakan jembatan antara model dan pengambil keputusan manusia.
- **Ekonometrika** dan **ilmu keputusan (*decision science*)** mengingatkan kita bahwa keputusan selalu tertanam dalam konteks kausal dan organisasional.

Buku ini tidak berusaha menggantikan pelatihan mendalam di masing-masing disiplin. Ia berusaha menyusun mereka dalam satu alur kerja yang koheren.

Nilai: Dari Insight ke Impact

Salah satu temuan paling konsisten dalam literatur analytics adalah bahwa **sebagian besar nilai hilang di "last mile"**—bukan karena modelnya salah, melainkan karena rekomendasinya tidak pernah sampai ke pengambil keputusan, atau sampai tetapi tidak dipercaya.

Tiga penghalang yang paling umum:

1. **Relevansi.** Analisis menjawab pertanyaan yang bukan pertanyaan pengambil keputusan.
2. **Kepercayaan.** Model dianggap kotak hitam; pengambil keputusan tidak dapat memeriksa asumsinya.
3. **Kelayakan.** Rekomendasi dianggap tidak dapat dieksekusi mengingat kendala nyata (anggaran, kebijakan, budaya).

Karena itu, di setiap bab Bagian II dan III Anda akan menemukan **kotak "Jembatan"**—kotak yang secara eksplisit menanyakan: *jika hasil ini benar, keputusan apa yang akan berubah, dan siapa yang harus meyakinkannya?*

Dataset Pendamping

Seluruh buku menggunakan **satu atau lebih dataset pendamping**—yang relevan dengan subyek yang sedang dibahas. Dataset ini dipilih karena ia dapat dianalisis pada keempat tingkat analytics tanpa kehilangan koherensi.

Tingkat	Pertanyaan yang akan kita ajukan pada dataset pendamping
Descriptive	Bagaimana pola pelanggan, produk, atau transaksi saat ini?
Diagnostic	Mengapa pola ini muncul? Segmen mana yang paling berbeda?
Predictive	Pelanggan mana yang berisiko tinggi, produk mana yang akan laris?
Prescriptive	Retensi, alokasi stok, atau intervensi apa yang optimal?

Dengan menggunakan satu atau lebih dataset yg konsisten dari awal sampai akhir, Anda akan melihat bagaimana **pertanyaan yang berbeda menghasilkan analisis yang berbeda, bahkan ketika datanya sama**.

Tiga Bagian Buku Ini

Bagian I — Descriptive & Diagnostic Analytics. Fondasi: cara membaca, membersihkan, dan memahami data. Anda akan berkenalan dengan Python, pandas, visualisasi, dan alat diagnostik. Bagian ini menjawab ***apa dan mengapa***.

Bagian II — Predictive Analytics. Cara memodelkan hubungan antarvariabel dan memprediksi masa depan. Anda akan mempelajari regresi, klasifikasi, time series, unsupervised learning, dan simulasi Monte Carlo sebagai jembatan ke Bagian III. Bagian ini menjawab ***apa yang akan terjadi***.

Bagian III — Prescriptive Analytics. Puncak perjalanan: mengambil keputusan optimal di bawah ketidakpastian dan keterbatasan. Anda akan mempelajari linear programming, integer programming, optimasi jaringan, simulasi keputusan, dan integrasi prediksi ke dalam model optimasi. Bagian ini menjawab ***apa yang sebaiknya dilakukan***.

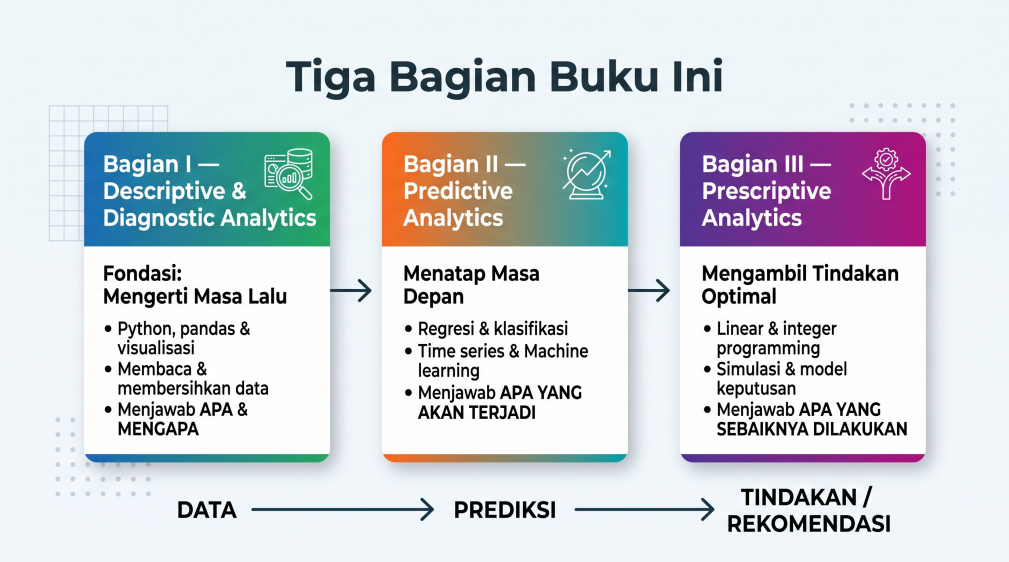


Figure 3: alur 3 bagian Buku

Bagaimana ketiganya terhubung. Bagian I menyiapkan data; Bagian II mengubahnya menjadi prediksi; Bagian III mengubah prediksi menjadi rekomendasi tindakan. Anda tidak dapat melompat ke Bagian III tanpa fondasi Bagian I dan II—bukan karena aturan akademis, tetapi karena keputusan yang baik membutuhkan prediksi yang baik, dan prediksi yang baik membutuhkan data yang dipahami dengan baik.

Kosakata Kunci

Sepanjang buku, beberapa istilah akan muncul berulang kali. Berikut versi ringkas yang akan diperluas di bab-bab selanjutnya.

Analytics — Proses mengubah data menjadi insight dan insight menjadi tindakan. Mencakup keempat tingkat pada Figure 1.
Model — Representasi matematis atau komputasional dari sebagian dunia nyata, dirancang untuk tujuan tertentu.
Fitur (feature) — Variabel masukan dalam sebuah model prediktif. Disebut juga prediktor atau variabel independen.
Overfitting — Kondisi ketika model terlalu menyesuaikan diri pada data latih sehingga buruk dalam memprediksi data baru.
Fungsi tujuan (objective function) — Kuantitas yang ingin dimaksimalkan atau diminimalkan dalam model optimasi.
Kendala (constraint) — Batasan yang harus dipenuhi solusi dalam model optimasi.
Ketidakpastian (uncertainty) — Keadaan ketika hasil dari suatu tindakan tidak dapat diketahui dengan pasti; ditangani melalui probabilitas dan simulasi.

Glosarium lengkap tersedia di bagian akhir buku.

Cara Membaca Buku Ini

Buku ini dirancang agar dapat dipakai dengan beberapa cara berbeda:

- **Pembelajar mandiri** sebaiknya membaca berurutan dari Bagian I, mengerjakan latihan di setiap bab sebelum melanjutkan. Kode dapat dijalankan langsung dari notebook.
- **Pengajar** dapat memetakan Bagian I–III langsung ke tiga course. Materi slide tersedia di folder slides/ pada repositori pendamping.
- **Praktisi** yang ingin fokus pada satu topik dapat masuk langsung ke bab yang relevan, tetapi Bab 2 tetap disarankan sebagai titik masuk karena ia memperkenalkan konvensi dan dataset pendamping.
- **Pembaca yang ingin gambaran cepat** dapat membaca bab ini, lalu melompat ke bab rangkuman setiap bagian sebelum memutuskan di mana masuk lebih dalam.

Tip

Jika Anda mengajar dengan buku ini, gunakan demo sneak peek di atas sebagai kelas pembuka. Ia memberi mahasiswa gambaran utuh sebelum masuk ke detail teknis—dan menjadi jangkar yang bisa dirujuk balik setiap kali mereka merasa tersesat di tengah detail bab.

Rangkuman dan Jembatan ke Bagian I

Dalam bab ini kita telah menetapkan:

- **Decision analytics** berfokus pada keputusan, bukan pada data.
- Empat tingkat analytics—**descriptive, diagnostic, predictive, prescriptive**—membentuk satu tangga yang saling bergantung.
- Analytics adalah bagian dari **siklus pengambilan keputusan** yang berulang, bukan aktivitas sekali jalan.
- Nilai analytics sering hilang di **last mile**; karena itu buku ini secara eksplisit menghubungkan setiap analisis ke keputusan.
- Buku ini disusun dalam **tiga bagian** dengan **satu dataset pendamping** dan **satu bahasa (Python)**.

Bab berikutnya, **Fondasi Decision Analytics**, akan mulai membangun fondasi teknis: menyiapkan lingkungan Python, memahami struktur data, dan memperkenalkan dataset pendamping yang akan menemani Anda sampai akhir buku. Jika Anda baru mengenal Python, bab itu juga menyediakan jalur percepatan singkat agar Anda dapat mengikuti bab-bab selanjutnya tanpa hambatan.

Part I

Bagian I — Descriptive & Diagnostic Analytics

1 Fondasi Decision Analytics

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Menyiapkan dan memverifikasi lingkungan Python yang digunakan sepanjang buku. 2. Memahami struktur data inti `pandas`—`Series` dan `DataFrame`—serta kapan menggunakannya. 3. Memuat dataset pendamping dan melakukan inspeksi awal yang sistematis. 4. Membedakan tipe data dan memahami mengapa tipe yang tepat penting untuk analisis selanjutnya.

1.1 Mengapa Fondasi?

Sebelum kita dapat memprediksi atau mengoptimalkan apa pun, kita perlu **memegang data dengan benar**. Di dunia nyata, sebagian besar waktu seorang analis dihabiskan bukan untuk membangun model, melainkan untuk memastikan data yang masuk ke model tersebut dapat dipercaya. Bab ini membangun kebiasaan itu sejak awal.

Tip

Jika Anda sudah berpengalaman dengan Python dan `pandas`, Anda dapat membaca cepat bab ini dan langsung ke Bab 2. Namun tetap perhatikan **Bagian 1.5** (dataset pendamping) karena konvensi yang diperkenalkan di sana akan dipakai di seluruh buku.

1.2 Menyiapkan Lingkungan Python

Buku ini menggunakan environment Conda bernama `decanal` yang telah disiapkan sebelumnya (lihat `README.md` pada repositori buku). Jika Anda mengikuti dari awal, pastikan environment tersebut aktif:

```
conda activate decanal
```

Anda dapat memverifikasi bahwa semua paket inti tersedia dengan menjalankan blok berikut. Jika ada paket yang hilang, jalankan `pip install -r requirements.txt` dari root proyek.

```
import sys
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print(f"Python  : {sys.version.split()[0]}")
print(f"pandas  : {pd.__version__}")
print(f"numpy   : {np.__version__}")
```

```
Python  : 3.11.13
pandas  : 2.3.3
numpy   : 2.4.6
```

Anda juga dapat memeriksa bahwa eksekusi benar-benar terjadi di environment `decanal`:

```
import sys
print(sys.executable)
```

```
/Users/chakraroy/miniconda3/envs/decanal/bin/python
```

Jika path yang dicetak mengarah ke `.../envs/decanal/bin/python` (Linux/macOS) atau `...\envs\decanal\python.exe` (Windows), Anda siap melanjutkan.

> **Note**
>
> Sepanjang buku, blok kode diberi opsi `#| label:` agar dapat direferensikan silang dan direproduksi. Opsi `#| echo: true` menampilkan kode di buku; `#| echo: false` menyembunyikannya tetapi tetap menjalankannya. Anda tidak perlu menghafal ini sekarang—akan diperkenalkan bertahap.

Cara paling andal agar Quarto secara otomatis menggunakan *environment* Python tertentu (seperti `venv` atau `conda`) adalah dengan mendaftarkan *environment* tersebut sebagai kernel Jupyter, lalu mendeklarasikannya di *header* YAML dokumen Anda. Berikut adalah beberapa metode yang dapat digunakan untuk melakukan itu:

Metode 1: Mendaftarkan Environment sebagai Kernel Jupyter (Direkomendasikan) Pendekatan ini mengunci dokumen `.qmd` ke *environment* yang spesifik, sehingga siapapun yang merender dokumen (yang memiliki kernel dengan nama sama) akan menggunakan dependensi yang tepat.

```yaml
---
title: "Analisis Data dengan Python"
jupyter: quarto_env
---
```

Metode 2: Menggunakan Ekstensi VS Code Jika Anda menulis dan merender Quarto menggunakan Visual Studio Code (dengan ekstensi Quarto terinstal), Anda bisa memanfaatkan deteksi *interpreter* bawaan VS Code.

1. Tekan `Ctrl + Shift + P` (atau `Cmd + Shift + P` di Mac).
2. Ketik dan pilih **Python: Select Interpreter**.
3. Pilih *environment* (Conda/Venv) yang sesuai untuk proyek Anda.
4. Saat Anda mengeklik tombol **Render** di VS Code, Quarto akan secara otomatis mengeksekusi kode menggunakan *environment* Python yang terpilih tersebut tanpa perlu mengubah YAML.

Metode 3: Pewarisan Terminal (CLI) Jika Anda mengeksekusi Quarto murni dari *Command Line* tanpa menyetel nama kernel di YAML:

- Quarto dirancang untuk langsung mendeteksi `python` yang berjalan di path terminal Anda.
- Aktifkan *environment* Anda di terminal (misal: `conda activate myenv`).
- Jalankan perintah `quarto preview nama_file.qmd` atau `quarto render`. Quarto akan otomatis menggunakan *environment* yang sedang aktif.

Metode 4: Khusus Pengguna Engine Knitr (Kombinasi R dan Python) Jika dokumen Anda menggunakan *engine* `knitr` (R) sebagai dasar, namun berisi *chunk* Python (via paket `reticulate`), Anda harus memberi tahu R *environment* mana yang harus digunakan. Tambahkan *chunk* R ini di awal dokumen `.qmd` Anda:

```r
#| include: false
library(reticulate)
use_virtualenv("./nama_env", required = TRUE)
# atau gunakan: use_condaenv("nama_env", required = TRUE)
```

Jika cara-cara itu belum berhasil, mungkin Anda perlu melakukan ini:

Hapus Cache Eksekusi Quarto Terkadang Quarto menggunakan hasil eksekusi lama yang tersimpan di cache, sehingga perubahan konfigurasi tidak langsung terlihat.

- Hapus folder `_freeze` (jika ada) di direktori proyek Anda.
- Atau, jalankan render melalui terminal dengan perintah:

```
quarto render nama_file.qmd --execute-cache-clear
```

1.3 Struktur Data Inti: `Series` dan `DataFrame`

pandas menyediakan dua struktur data yang menjadi tulang punggung hampir semua analisis di buku ini:

Struktur	Dimensi	Analogi	Penggunaan khas
`Series`	1D	Satu kolom dengan indeks	Satu variabel, satu urutan waktu, hasil agregasi
`DataFrame`	2D	Tabel dengan baris & kolom	Seluruh dataset, hasil penggabungan, matriks fitur

Mari kita lihat keduanya secara langsung.

```
import pandas as pd

# Series: satu variabel dengan indeks
penjualan_harian = pd.Series([120, 135, 118, 150, 142],
                             index=["Sen", "Sel", "Rab", "Kam", "Jum"],
                             name="penjualan")
penjualan_harian
```

```
Sen    120
Sel    135
Rab    118
Kam    150
Jum    142
Name: penjualan, dtype: int64
```

```
# DataFrame: kumpulan Series yang berbagi indeks
toko = pd.DataFrame({
    "kota": ["Jakarta", "Bandung", "Surabaya", "Medan"],
    "jumlah_karyawan": [45, 22, 30, 18],
    "pendapatan_juta": [1200, 560, 780, 420],
})
toko
```

	kota	jumlah_karyawan	pendapatan_juta
0	Jakarta	45	1200
1	Bandung	22	560
2	Surabaya	30	780
3	Medan	18	420

Operasi dasar yang akan sering Anda gunakan:

```
print("--- Bentuk (baris, kolom) ---")
print(toko.shape)

print("\n--- Nama kolom ---")
print(toko.columns.tolist())

print("\n--- Tipe data per kolom ---")
print(toko.dtypes)

print("\n--- Akses satu kolom (Series) ---")
print(toko["pendapatan_juta"])

print("\n--- Ringkasan statistik ---")
toko.describe()
```

```
--- Bentuk (baris, kolom) ---
(4, 3)

--- Nama kolom ---
['kota', 'jumlah_karyawan', 'pendapatan_juta']

--- Tipe data per kolom ---
kota               object
jumlah_karyawan     int64
pendapatan_juta     int64
dtype: object

--- Akses satu kolom (Series) ---
0    1200
1     560
2     780
3     420
Name: pendapatan_juta, dtype: int64
```

```
--- Ringkasan statistik ---
```

	jumlah_karyawan	pendapatan_juta
count	4.00000	4.000000
mean	28.75000	740.000000
std	11.92686	340.587727
min	18.00000	420.000000
25%	21.00000	525.000000
50%	26.00000	670.000000
75%	33.75000	885.000000
max	45.00000	1200.000000

Tip

Aturan praktis. Jika Anda membutuhkan satu variabel, gunakan `Series`. Jika Anda membutuhkan tabel, gunakan `DataFrame`. Hampir semua operasi agregasi (`groupby`, `mean`, `sum`) mengembalikan `Series`, dan hampir semua operasi penggabungan mengembalikan `DataFrame`.

1.4 Tipe Data dan Mengapa Ia Penting

Salah satu kesalahan paling umum pemula adalah **mengabaikan tipe data**. `pandas` memberi setiap kolom sebuah *dtype*, dan tipe yang salah akan menyebabkan analisis yang salah—sering tanpa error sama sekali.

Kelompok dtype	Contoh	Konsekuensi jika salah
Numerik (`int64`, `float64`)	`usia=34`, `harga=2.5e6`	Operasi statistik gagal atau menghasilkan `NaN`
Boolean (`bool`)	`is_active=True`	Perbandingan logika tidak berjalan
Kategorikal (`category`)	`kota="Jakarta"`	Mengonsumsi memori besar & memperlambat grouping
Object (`object`)	Campuran string dan angka	Sering menjadi tanda data kotor
Datetime (`datetime64`)	`waktu="2025-01-01"`	Analisis time series tidak mungkin dilakukan

> **Warning**
>
> Kolom numerik yang **terbaca sebagai string** adalah salah satu jebakan paling umum. Ini sering terjadi ketika file CSV memuat angka dengan pemisah ribuan (misalnya `"1,200"`) atau simbol mata uang (`"Rp 1.200"`). Selalu periksa `dtypes` setelah memuat dataset.

Untuk mengonversi tipe data secara eksplisit:

```
contoh = pd.DataFrame({
    "harga_str": ["1,200", "560", "780"],
    "tanggal_str": ["2025-01-01", "2025-01-02", "2025-01-03"],
    "kategori_str": ["A", "B", "A"],
})

contoh["harga"] = (
    contoh["harga_str"].str.replace(",", "", regex=False).astype(float)
)
contoh["tanggal"] = pd.to_datetime(contoh["tanggal_str"])
contoh["kategori"] = contoh["kategori_str"].astype("category")

contoh[["harga", "tanggal", "kategori"]].dtypes
```

```
harga              float64
tanggal     datetime64[ns]
kategori          category
dtype: object
```

1.5 Memuat Dataset Pendamping

Seluruh buku ini menggunakan **satu dataset pendamping**: *Telco Customer Churn*—data pelanggan perusahaan telekomunikasi dengan label biner `Churn` (berhenti berlangganan atau tidak). Dataset ini ideal untuk buku kita karena:

- Memiliki **variabel numerik** (tenure, monthly charges, total charges).
- Memiliki **variabel kategorikal** (kontrak, layanan internet, metode pembayaran).
- Memiliki **target biner** yang memungkinkan klasifikasi prediktif.
- Memiliki **struktur waktu** (tenure dalam bulan) yang memungkinkan analisis survival dan time series sederhana.
- Memiliki **makna bisnis langsung** yang memungkinkan perumusan model optimasi di Bagian III.

Dataset tersedia di folder `data/raw/telco_churn.csv` pada repositori buku. Jika Anda belum mengunduhnya, jalankan blok berikut untuk mengunduh ke folder `data/raw/`:

```
from pathlib import Path
import urllib.request

URL = ("https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/"
       "master/data/Telco-Customer-Churn.csv")
target = Path("data/raw/telco_churn.csv")
target.parent.mkdir(parents=True, exist_ok=True)
urllib.request.urlretrieve(URL, target)
print(f"Dataset tersimpan di {target.resolve()}")
```

> **Note**
>
> Blok di atas diberi `#| eval: false` karena tidak perlu dijalankan ulang setiap kali buku dirender—cukup sekali. Setelah file ada di `data/raw/`, blok-blok berikutnya akan memuatnya dari disk.

Sekarang muat dataset:

```
from pathlib import Path
import pandas as pd

DATA_PATH = Path("data/raw/telco_churn.csv")
df = pd.read_csv(DATA_PATH)

print(f"Bentuk dataset: {df.shape[0]} baris × {df.shape[1]} kolom")
```

```
Bentuk dataset: 7043 baris × 21 kolom
```

1.6 Inspeksi Awal: Lima Langkah Sistematis

Setiap kali Anda menerima dataset baru—baik dari repositori publik maupun dari klien—jalankan **lima langkah inspeksi** berikut sebelum melakukan apa pun. Kebiasaan ini akan menyelamatkan Anda dari banyak masalah di kemudian hari.

1.6.1 Langkah 1 — Lihat Bentuk dan Kolom

```
print(f"Baris   : {df.shape[0]:,}")
print(f"Kolom   : {df.shape[1]}")
print(f"\nKolom-kolom: {df.columns.tolist()}")
```

```
Baris   : 7,043
Kolom   : 21

Kolom-kolom: ['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'Pl
```

1.6.2 Langkah 2 — Periksa Tipe Data

```
df.dtypes
```

```
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object
```

Perhatikan kolom `TotalCharges`—secara logis ia seharusnya numerik (total tagihan), tetapi sering terbaca sebagai `object` karena ada nilai kosong yang direpresentasikan sebagai string `" "`. Kita akan membahas ini di Bab 2 (wrangling), tetapi catat dulu bahwa **ini adalah masalah yang akan kita selesaikan**.

1.6.3 Langkah 3 — Lihat Beberapa Baris Pertama

```
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone servic
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone servic
4	9237-HQITU	Female	0	No	No	2	Yes	No

1.6.4 Langkah 4 — Nilai Hilang

```
missing = df.isna().sum().sort_values(ascending=False)
missing[missing > 0]
```

```
Series([], dtype: int64)
```

Jika hasilnya kosong (atau hanya sedikit), Anda beruntung. Pada dataset ini, `NaN` dari `isna()` mungkin tidak menangkap nilai kosong yang direpresentasikan sebagai string. Kita akan melihat cara yang lebih komprehensif di Bab 2.

1.6.5 Langkah 5 — Ringkasan Statistik & Distribusi Kategorikal

```
df.describe()
```

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000

	SeniorCitizen	tenure	MonthlyCharges
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

```
for col in ["gender", "Contract", "InternetService", "Churn"]:
    if col in df.columns:
        print(f"\n--- {col} ---")
        print(df[col].value_counts(dropna=False))
```

```
--- gender ---
gender
Male      3555
Female    3488
Name: count, dtype: int64

--- Contract ---
Contract
Month-to-month    3875
Two year          1695
One year          1473
Name: count, dtype: int64

--- InternetService ---
InternetService
Fiber optic    3096
DSL            2421
No             1526
Name: count, dtype: int64

--- Churn ---
Churn
No     5174
Yes    1869
Name: count, dtype: int64
```

Tip

Ringkasan lima langkah. 1. **Bentuk & kolom** — apa yang kita punya? 2. **Tipe data** — apakah tipe sesuai dengan makna logis? 3. **Cuplikan data** — apakah nilainya masuk akal? 4. **Nilai hilang** — di mana data bolong? 5. **Distribusi** — apakah sebarannya wajar, atau ada outlier ekstrem?
Jalankan kelima langkah ini setiap kali membuka dataset baru, bahkan sebelum membaca dokumentasinya. Sering kali Anda akan menemukan anomali yang tidak disebutkan dalam dokumentasi.

1.7 Variabel Target dan Variabel Fitur

Karena dataset ini akan digunakan untuk klasifikasi prediktif di Bagian II, kita perlu membedakan **variabel target** dari **variabel fitur** sejak awal.

- **Target**: kolom `Churn` (biner: `Yes`/`No`). Inilah yang akan kita coba prediksi.
- **Fitur**: semua kolom lain yang berpotensi informatif tentang target.

```
churn_counts = df["Churn"].value_counts()
churn_pct = df["Churn"].value_counts(normalize=True).mul(100).round(2)

pd.DataFrame({"jumlah": churn_counts, "persentase (%)": churn_pct})
```

Churn	jumlah	persentase (%)
No	5174	73.46
Yes	1869	26.54

Distribusi kelas pada dataset ini tidak seimbang—sekitar seperempat pelanggan churn. Ini adalah pola yang khas di dunia nyata dan akan menjadi pertimbangan penting ketika kita memilih metrik evaluasi di Bab 9.

```
import matplotlib.pyplot as plt

ax = df["Churn"].value_counts().plot(kind="bar", color=["#4C72B0", "#DD8452"])
ax.set_title("Distribusi Churn")
ax.set_xlabel("Churn")
ax.set_ylabel("Jumlah pelanggan")
ax.set_xticklabels(["No", "Yes"], rotation=0)
```

```
plt.tight_layout()
plt.show()
```

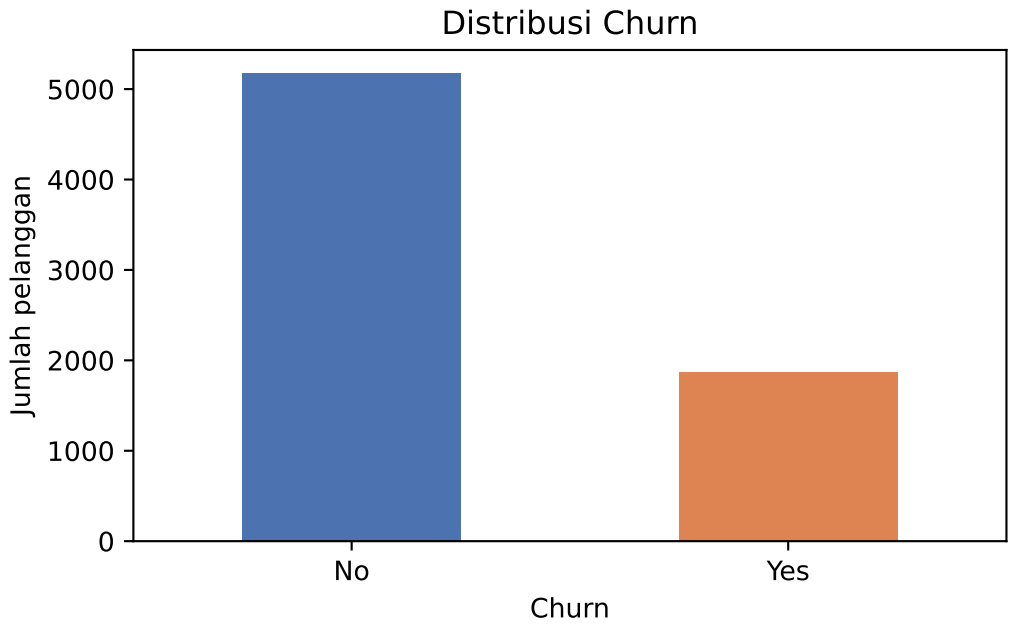


Figure 1.1: Distribusi variabel target Churn.

1.8 Konvensi Dataset untuk Buku Ini

Untuk menjaga konsistensi di seluruh bab, tetapkan konvensi berikut dan jangan mengubahnya:

Konvensi	Nilai
Nama variabel `DataFrame` utama	`df`
Nama variabel target	`y` (Series)
Nama matriks fitur	`X` (DataFrame)
Path dataset mentah	`data/raw/telco_churn.csv`
Path dataset bersih	`data/processed/telco_clean.csv`
`random_state` bawaan	`42`

> **Note**
>
> **Mengapa `random_state=42`?** Karena kita ingin hasil dapat direproduksi—Anda, pembaca lain, dan CI/CD Anda harus mendapatkan angka yang identik setiap kali buku dirender. Angka 42 hanyalah konvensi populer; yang penting adalah **konsisten di seluruh buku**.

Untuk menghindari duplikasi kode pemuatan, kita bisa membuat helper kecil di `code/utils.py`:

```
# code/utils.py
from pathlib import Path
import pandas as pd

DATA_DIR = Path("data")

def load_churn() -> pd.DataFrame:
    """Dataset pendamping buku Decision Analytics."""
    return pd.read_csv(DATA_DIR / "raw" / "telco_churn.csv")

def load_churn_clean() -> pd.DataFrame:
    """Dataset bersih hasil Bab 2."""
    return pd.read_csv(DATA_DIR / "processed" / "telco_clean.csv")
```

Di bab-bab selanjutnya, kita akan memuat data dengan:

```
import sys; sys.path.append(".")
from code.utils import load_churn
df = load_churn()
```

> **Tip**
>
> Membuat helper seperti di atas mungkin terasa berlebihan untuk satu dataset. Tetapi ketika buku Anda memiliki 18 bab yang semuanya memuat dataset yang sama, **satu titik perubahan** (misalnya jika path atau nama file berubah) jauh lebih mudah dikelola daripada mengedit 18 file.

1.9 Latihan Terbimbing

Kerjakan latihan berikut langsung di notebook Anda. Solusi tersedia di folder `solutions/` pada repositori buku.

Latihan 1.1 — Inspeksi mandiri. Muat dataset dengan `pd.read_csv("data/raw/telco_churn.csv")`. Jawab pertanyaan berikut **tanpa** melihat jawaban di bawah: - Berapa banyak baris dan kolom? - Kolom mana saja yang bertipe `object`? - Kolom mana yang seharusnya numerik tetapi bertipe `object`? - Apakah ada kolom dengan nilai kosong (`NaN`)?

Latihan 1.2 — Konversi tipe. Kolom `TotalCharges` seharusnya bertipe numerik. Lakukan:

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

Kemudian periksa berapa banyak nilai yang berubah menjadi `NaN`. Mengapa hal ini terjadi? (Petunjuk: periksa nilai unik yang bukan angka.)

Latihan 1.3 — Distribusi kategorikal. Untuk setiap kolom kategorikal (`dtype == "object"`), tampilkan `value_counts()`. Kolom mana yang memiliki **kardinalitas tinggi** (banyak nilai unik)? Mengapa kolom seperti `customerID` **tidak** boleh dimasukkan sebagai fitur dalam model prediktif?

Latihan 1.4 — Series vs DataFrame. Jelaskan mengapa `df["tenure"]` mengembalikan `Series` sedangkan `df[["tenure"]]` mengembalikan `DataFrame`. Dalam situasi apa Anda membutuhkan yang kedua?

> **Note**
>
> **Petunjuk Latihan 1.2.** Kolom `TotalCharges` memuat string kosong `" "` (satu spasi) untuk pelanggan baru dengan `tenure = 0`. `pd.to_numeric` dengan `errors="coerce"` akan mengubah string tersebut menjadi `NaN`, yang secara semantik **benar**—pelanggan baru belum memiliki total tagihan. Ini akan kita tangani secara eksplisit di Bab 2.

1.10 Kotak "Jembatan": Dari Data ke Keputusan

> **Important**
>
> **Jembatan.** Di akhir setiap bab Bagian I, kotak ini menanyakan: *jika analisis di bab ini benar, keputusan apa yang bisa berubah?*
> Bab ini belum menghasilkan insight substantif—ia menghasilkan **fondasi**. Tetapi ia sudah memengaruhi keputusan dengan cara yang halus namun penting:
>
> - **Keputusan tentang data mana yang dipercaya.** Dengan memeriksa tipe data dan nilai hilang sebelum analisis, Anda menghindari keputusan yang didasarkan pada kolom `TotalCharges` yang salah tafsir sebagai string.
> - **Keputusan tentang apa yang bisa diprediksi.** Dengan memahami distribusi

`Churn` (~26% positif), Anda sudah dapat mengantisipasi bahwa metrik akurasi sederhana akan menyesatkan—dan memilih metrik yang lebih tepat di Bab 9.
- **Keputusan tentang batas analisis.** Kolom `customerID` adalah pengenal unik, bukan fitur. Mengabaikannya adalah keputusan desain yang mencegah *data leakage* di kemudian hari.

Fondasi yang baik tidak menghasilkan rekomendasi langsung, tetapi ia menentukan apakah rekomendasi di bab-bab selanjutnya dapat dipercaya.

1.11 Rangkuman

Dalam bab ini kita telah:

- **Menyiapkan lingkungan** Python `decanal` dan memverifikasi bahwa semua paket inti tersedia.
- **Memahami dua struktur data inti** `pandas`—`Series` (1D) dan `DataFrame` (2D)—beserta kapan menggunakannya.
- **Membedakan tipe data** dan memahami mengapa tipe yang salah (terutama numerik yang terbaca sebagai `object`) adalah sumber kesalahan yang sering tidak terdeteksi.
- **Memuat dataset pendamping** *Telco Customer Churn* dan menjalankan **lima langkah inspeksi** sistematis: bentuk, tipe, cuplikan, nilai hilang, dan distribusi.
- **Menetapkan konvensi** nama variabel dan path yang akan dipakai konsisten di seluruh buku.
- **Mengidentifikasi dua masalah data** yang akan diselesaikan di Bab 2: kolom `TotalCharges` yang bertipe salah dan nilai kosong yang direpresentasikan sebagai string.
- **Menetapkan target dan fitur** untuk pemodelan prediktif di Bagian II, dengan catatan penting bahwa distribusi kelas tidak seimbang.

1.12 Jembatan ke Bab 2

Dataset yang baru kita muat masih "mentah" dalam banyak hal: tipe data yang salah, nilai hilang yang tersembunyi, kolom yang mungkin tidak informatif, dan skala yang belum seragam. Di bab berikutnya—**Data Wrangling & Manajemen Data**—kita akan membersihkan dataset ini secara sistematis:

- Menangani nilai hilang, baik dengan penghapusan, imputasi, maupun transformasi.
- Mengonversi tipe data yang salah, termasuk `TotalCharges`.
- Mendeteksi dan menangani outlier.
- Menggabungkan data dari beberapa sumber (jika diperlukan).

- Menyimpan dataset bersih ke `data/processed/telco_clean.csv` untuk digunakan di seluruh bab selanjutnya.

Setelah Bab 2 selesai, Anda akan memiliki **dataset bersih yang siap dianalisis**—dan itulah fondasi sesungguhnya untuk semua yang akan kita bangun di Bagian II dan III.

2 Data Wrangling & Manajemen Data

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Mengenali sumber data yang umum di organisasi dan memilih cara memuatnya dengan tepat. 2. Mendiagnosis masalah kualitas data secara sistematis—bukan hanya "menemukan yang hilang", tetapi memahami **mengapa** ia hilang. 3. Menerapkan strategi penanganan nilai hilang, duplikat, dan outlier dengan pertimbangan konteks, bukan aturan mekanis. 4. Melakukan *feature engineering* dasar untuk menyiapkan dataset bagi pemodelan di Bagian II. 5. Menyimpan dataset bersih dengan tata kelola yang dapat direproduksi.

2.1 Mengapa Wrangling?

Di Bab 1 kita berkenalan dengan dataset mentah dan menemukan dua masalah: kolom `TotalCharges` bertipe salah, dan beberapa nilai kosong tersembunyi. Di dunia nyata, **inilah norma, bukan pengecualian**. Sebuah studi yang sering dikutip menyebutkan bahwa analis data menghabiskan 60–80% waktunya untuk menemukan, membersihkan, dan mengorganisasi data—bukan untuk membangun model.

Karena itu, wrangling bukan "pekerjaan kotor sebelum pekerjaan sesungguhnya". Ia **adalah** pekerjaan sesungguhnya. Model terbaik sekalipun akan menghasilkan keputusan yang salah jika dilatih pada data yang salah—sebuah prinsip yang dikenal sebagai *garbage in, garbage out*.

Tip

Jika Anda baru mengenal `pandas`, bab ini mungkin terasa padat. Fokuslah pada **pola pikir**: setiap keputusan wrangling harus dapat dijelaskan alasannya. Kode dapat dicari; penalaran harus dibangun.

2.2 Sumber Data di Dunia Nyata

Sebelum masuk ke teknik, penting untuk mengenali bahwa data datang dalam berbagai bentuk. Buku ini fokus pada **file tabular** karena itulah bentuk paling umum di lingkungan pengajaran, tetapi Anda perlu tahu lanskapnya.

Sumber	Contoh	Alat Python	Kapan dipakai
File datar	CSV, TSV, Excel	`pandas.read_csv`, `read_excel`	Data kecil–menengah, transfer manual
Database relasional	PostgreSQL, MySQL, SQLite	`SQLAlchemy` + `pandas.read_sql`	Data besar, banyak pengguna, integritas transaksional
API	REST endpoints (JSON)	`requests`, `httpx`	Data real-time dari layanan eksternal
Semi-terstruktur	JSON, XML, Parquet	`pandas.read_json`, `read_parquet`	Log, event streams, data bersarang
Data warehouse	BigQuery, Snowflake, Redshift	Driver native atau `dbt`	Skala organisasi, analitik historis

Untuk buku ini kita memakai **file CSV** karena paling mudah direproduksi dan disebarkan. Namun pola yang sama berlaku untuk sumber lain: **muat → inspeksi → perbaiki → simpan**.

> **Note**
>
> **Mengapa Parquet lebih baik dari CSV untuk dataset besar?** CSV menyimpan semuanya sebagai teks; Parquet menyimpan tipe data secara biner, terkompresi, dan kolumnar. Untuk dataset berukuran gigabyte, Parquet bisa 10× lebih cepat dibaca. Kita tetap memakai CSV di buku ini karena lebih portabel, tetapi di produksi Parquet hampir selalu lebih baik.

2.3 Kualitas Data: Enam Dimensi

Kualitas data bukan satu konsep tunggal. Literatur kualitas data biasanya membedakan **enam dimensi**, dan setiap dimensi memiliki teknik diagnosis dan perbaikan yang berbeda:

Dimensi	Pertanyaan	Contoh masalah
Completeness	Apakah semua data yang seharusnya ada, ada?	`TotalCharges` kosong untuk pelanggan baru

Dimensi	Pertanyaan	Contoh masalah
Accuracy	Apakah nilainya benar?	Usia tercatat 200 tahun
Consistency	Apakah nilainya konsisten antarsumber/kolom?	Kota "Jakarta" vs "DKI Jakarta"
Timeliness	Apakah datanya cukup baru untuk keputusan?	Data churn 3 tahun lalu untuk prediksi hari ini
Uniqueness	Apakah ada duplikasi?	Pelanggan yang sama tercatat dua kali
Validity	Apakah nilainya memenuhi aturan domain?	`tenure = -5` (bulan negatif)

Bab ini akan menyentuh kelima dimensi pertama; **timeliness** akan menjadi fokus khusus di Bab 10 (Time Series).

2.4 Memuat Ulang Dataset

Mari kita muat ulang dataset dengan pola yang konsisten—menggunakan helper `code/utils.py` yang sudah diperkenalkan di Bab 1.

```
import sys
sys.path.insert(0,".")
import pandas as pd
import numpy as np
from decanal.utils import load_churn

df = load_churn()
print(f"Bentuk awal: {df.shape[0]} baris × {df.shape[1]} kolom")
```

```
Bentuk awal: 7043 baris × 21 kolom
```

2.5 Diagnosis Nilai Hilang

2.5.1 Jenis Nilai Hilang: MCAR, MAR, MNAR

Tidak semua nilai hilang diciptakan sama. Dalam statistika, kita membedakan tiga mekanisme:

- **MCAR** (*Missing Completely at Random*): hilangnya nilai tidak bergantung pada apa pun. Contoh: sensor rusak secara acak. Penanganan paling aman: hapus atau imputasi sederhana.
- **MAR** (*Missing at Random*): hilangnya nilai bergantung pada variabel lain yang **teramati**. Contoh: pelanggan muda lebih jarang mengisi survei pendapatan. Imputasi berbasis model cocok.
- **MNAR** (*Missing Not at Random*): hilangnya nilai bergantung pada **nilai itu sendiri**. Contoh: pelanggan berpenghasilan sangat tinggi enggan mengisi pendapatan. Penanganan paling sulit; sering kali perlu pemodelan eksplisit atau analisis sensitivitas.

> **Warning**
>
> Membedakan MCAR, MAR, dan MNAR dari data saja **tidak selalu mungkin**. Yang penting adalah **menyadari mekanismenya** dan tidak melakukan imputasi yang menghasilkan bias sistematis. Sebagai aturan praktis: jika proporsi nilai hilang kecil (<5%) dan tidak berpola, penghapusan sederhana biasanya aman.

2.5.2 Menemukan Nilai Hilang yang Tersembunyi

Pelajaran pertama Bab 1 terulang di sini: **`isna()` tidak selalu cukup**. Nilai kosong bisa direpresentasikan sebagai string `" "`, `"NA"`, `"?"`, `"-"`, `0` (padahal 0 berarti tidak ada), atau `999` (kode sentinel).

```
missing = df.isna().sum().sort_values(ascending=False)
print("--- Nilai NaN eksplisit ---")
print(missing[missing > 0])
```

```
--- Nilai NaN eksplisit ---
Series([], dtype: int64)
```

Hasilnya kosong atau sangat sedikit. Namun kita tahu dari Bab 1 bahwa `TotalCharges` bermasalah. Mari kita periksa dengan pendekatan yang lebih teliti:

```
tc = df["TotalCharges"]
print(f"Tipe data: {tc.dtype}")
print(f"Nilai unik yang bukan angka: {sorted(set(tc) - set(pd.to_numeric(tc, errors='coerce')
print(f"Jumlah string kosong: {(tc.str.strip() == '').sum() if tc.dtype == object else 0}")
```

```
Tipe data: object
Nilai unik yang bukan angka: [' ', '1010', '1024', '1032', '1033']
Jumlah string kosong: 11
```

Nilai `" "` (satu spasi) itulah yang membuat `pandas` menganggap seluruh kolom sebagai `object`. Ini adalah contoh sempurna dari **nilai hilang tersembunyi**: secara logis `NaN`, tetapi secara tekstual tampak "ada".

2.5.3 Memutuskan Strategi

Untuk `TotalCharges`, konteksnya sangat jelas: pelanggan dengan `tenure = 0` (baru mendaftar) belum memiliki total tagihan. Ini adalah **MCAR yang ditentukan domain**—hilangnya nilai adalah konsekuensi dari status pelanggan, bukan dari nilai yang hilang itu sendiri.

Tiga strategi yang mungkin:

Strategi	Kelebihan	Kekurangan
Drop baris	Sederhana	Kehilangan 11 pelanggan (dari 7.043) — tidak signifikan
Imputasi 0	Masuk akal domain	Menciptakan nilai yang secara semantik berarti "belum ada tagihan"
Imputasi median	Mempertahankan distribusi	Menyesatkan: pelanggan baru dianggap punya tagihan rata-rata

Keputusan: imputasi `0` untuk pelanggan dengan `tenure = 0`, dan **drop** baris yang masih kosong setelahnya (jika ada). Alasan: pelanggan baru *memang* belum memiliki tagihan; `0` adalah representasi yang benar secara semantik, bukan sekadar "menambal" angka.

```
# Konversi ke numerik; string kosong menjadi NaN
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Diagnostik: siapa yang NaN?
mask_nan = df["TotalCharges"].isna()
print(f"Baris dengan TotalCharges NaN: {mask_nan.sum()}")
print(f"Tenure pada baris tersebut: {df.loc[mask_nan, 'tenure'].unique()}")

# Imputasi 0 untuk pelanggan baru
df.loc[mask_nan & (df["tenure"] == 0), "TotalCharges"] = 0.0

# Sisa NaN (jika ada) kita drop dengan log
sisa = df["TotalCharges"].isna().sum()
print(f"NaN tersisa setelah imputasi: {sisa}")
```

```
Baris dengan TotalCharges NaN: 11
Tenure pada baris tersebut: [0]
NaN tersisa setelah imputasi: 0
```

```
print(f"Tipe akhir: {df['TotalCharges'].dtype}")
print(f"Rentang  : [{df['TotalCharges'].min():.2f}, {df['TotalCharges'].max():.2f}]")
df[["tenure", "MonthlyCharges", "TotalCharges"]].describe().round(2)
```

```
Tipe akhir: float64
Rentang  : [0.00, 8684.80]
```

	tenure	MonthlyCharges	TotalCharges
count	7043.00	7043.00	7043.00
mean	32.37	64.76	2279.73
std	24.56	30.09	2266.79
min	0.00	18.25	0.00
25%	9.00	35.50	398.55
50%	29.00	70.35	1394.55
75%	55.00	89.85	3786.60
max	72.00	118.75	8684.80

2.6 Duplikat dan Konsistensi

2.6.1 Duplikat Baris

```
dup_rows = df.duplicated().sum()
dup_ids = df["customerID"].duplicated().sum()
print(f"Baris duplikat penuh  : {dup_rows}")
print(f"customerID duplikat   : {dup_ids}")
```

```
Baris duplikat penuh  : 0
customerID duplikat   : 0
```

Jika ada duplikat, keputusan standarnya adalah menyimpan yang pertama (`df.drop_duplicates()`), **kecuali** ada alasan domain untuk menyimpan yang lain (misalnya, catatan terbaru berdasarkan timestamp).

2.6.2 Konsistensi Nilai Kategorikal

Nilai kategorikal sering masuk dalam variasi yang tidak konsisten: `"Yes"` vs `"yes"`, `"Jakarta"` vs `" DKI Jakarta "`. Periksa nilai unik untuk kolom kategorikal penting:

```
for col in ["gender", "Contract", "PaymentMethod", "InternetService"]:
    if col in df.columns:
        uniques = df[col].dropna().unique()
        print(f"\n--- {col} ({len(uniques)} nilai unik) ---")
        print(uniques)
```

```
--- gender (2 nilai unik) ---
['Female' 'Male']

--- Contract (3 nilai unik) ---
['Month-to-month' 'One year' 'Two year']

--- PaymentMethod (4 nilai unik) ---
['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']

--- InternetService (3 nilai unik) ---
['DSL' 'Fiber optic' 'No']
```

Gunakan `.str.strip()` untuk menghapus spasi di tepi, dan `.str.title()` atau mapping manual untuk menyamakan kapitalisasi jika perlu. Pada dataset ini nilainya sudah konsisten—tetapi **periksa selalu**, karena dataset publik pun kadang tidak konsisten.

```
# Normalisasi defensif: strip spasi di semua kolom string
obj_cols = df.select_dtypes(include="object").columns
for col in obj_cols:
    if col != "customerID":  # ID tidak boleh diubah
        df[col] = df[col].str.strip()
print(f"Kolom string yang dinormalisasi: {list(obj_cols)}")
```

```
Kolom string yang dinormalisasi: ['customerID', 'gender', 'Partner', 'Dependents', 'PhoneSer
```

2.7 Outlier: Deteksi dan Penanganan

Outlier adalah nilai yang **jauh dari mayoritas data**. Mereka bisa menjadi sumber insight (misalnya, pelanggan premium) atau sumber noise (misalnya, kesalahan entri). Yang penting adalah **mendeteksinya secara sadar**, bukan menghapusnya membabi buta.

2.7.1 Metode IQR (Interquartile Range)

Aturan paling umum: nilai di luar rentang `[Q1 - 1.5×IQR, Q3 + 1.5×IQR]` dianggap outlier.

```
def count_iqr_outliers(series: pd.Series) -> int:
    q1, q3 = series.quantile([0.25, 0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    return ((series < lower) | (series > upper)).sum()

for col in ["tenure", "MonthlyCharges", "TotalCharges"]:
    print(f"{col:16s}: {count_iqr_outliers(df[col]):3d} outlier")
```

```
tenure          :   0 outlier
MonthlyCharges  :   0 outlier
TotalCharges    :   0 outlier
```

2.7.2 Metode Z-Score

Alternatif: nilai dengan `|z| > 3` dianggap outlier. Cocok ketika distribusi mendekati normal.

```
from scipy import stats

for col in ["tenure", "MonthlyCharges", "TotalCharges"]:
    z = np.abs(stats.zscore(df[col]))
    print(f"{col:16s}: {(z > 3).sum():3d} outlier (|z| > 3)")
```

```
tenure          :   0 outlier (|z| > 3)
MonthlyCharges  :   0 outlier (|z| > 3)
TotalCharges    :   0 outlier (|z| > 3)
```

2.7.3 Memutuskan: Hapus, Pertahankan, atau Transformasi?

> **!** Important
>
> **Aturan praktis.** Jangan hapus outlier hanya karena **terdeteksi secara statistik**. Tanyakan dulu:
>
> 1. **Apakah nilainya valid?** Pelanggan dengan `MonthlyCharges = 120` mungkin memang ada (paket premium). Bukan outlier untuk dihapus.
> 2. **Apakah nilainya kesalahan?** Usia `200` atau tagihan `-50` adalah kesalahan—perbaiki atau hapus.
> 3. **Apakah ia mengganggu model yang akan kita bangun?** Model berbasis pohon (random forest, gradient boosting) tahan terhadap outlier. Regresi linear dan KNN lebih sensitif—untuk keduanya, transformasi (log) atau *winsorization* lebih baik dari penghapusan.

Untuk dataset Telco, hasil pemeriksaan IQR menunjukkan **nol outlier** pada ketiga kolom numerik. Ini adalah kabar baik: seluruh nilai berada dalam rentang yang wajar untuk domainnya. Ini juga pengingat bahwa **outlier bukanlah masalah universal**—banyak dataset publik sudah dibersihkan sebelumnya.

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(10, 3))
for ax, col in zip(axes, ["tenure", "MonthlyCharges", "TotalCharges"]):
    ax.boxplot(df[col], vert=True)
    ax.set_title(col)
    ax.set_ylabel("Nilai")
plt.tight_layout()
plt.show()
```

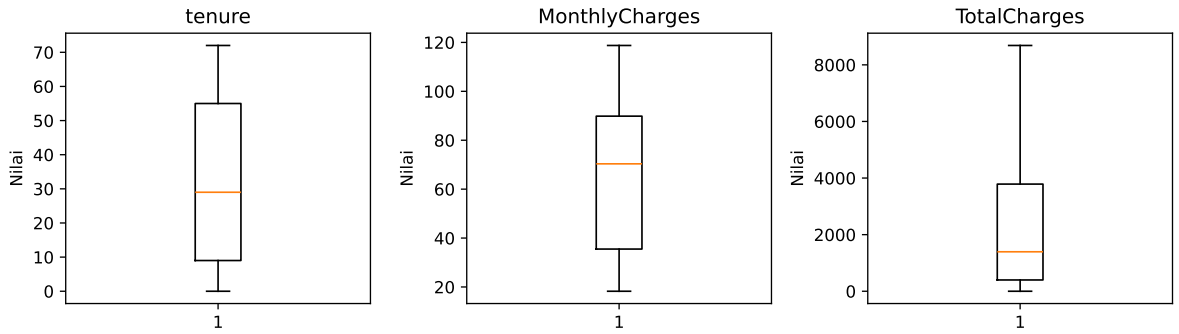


Figure 2.1: Distribusi tiga kolom numerik utama setelah wrangling. Tidak terlihat outlier ekstrem.

2.8 Feature Engineering Dasar

Feature engineering adalah seni membuat variabel baru dari variabel yang ada untuk memperkaya model. Ia bukan sekadar trik teknis—ia adalah tempat **pengetahuan domain bertemu data**.

2.8.1 Binning: Mengubah Kontinu menjadi Kategorikal

Variabel seperti `tenure` (bulan) sulit diinterpretasikan dalam bentuk mentah. Mengelompokkannya ke dalam *bin* membuatnya lebih bermakna secara bisnis dan sering meningkatkan performa model.

```
# Bin tenure ke kelompok bermakna secara bisnis
bins = [-1, 6, 12, 24, 48, np.inf]
labels = ["0-6 bln", "7-12 bln", "13-24 bln", "25-48 bln", ">48 bln"]

df["tenure_group"] = pd.cut(df["tenure"], bins=bins, labels=labels)
df["tenure_group"].value_counts().sort_index()
```

```
tenure_group
0-6 bln      1481
7-12 bln      705
13-24 bln    1024
25-48 bln    1594
>48 bln      2239
Name: count, dtype: int64
```

2.8.2 Encoding Kategori

Model machine learning hanya memahami angka. Variabel kategorikal harus dikodekan. Dua strategi utama:

- **One-hot encoding** (`pd.get_dummies`): membuat kolom biner untuk setiap kategori. Cocok untuk variabel nominal (tanpa urutan).
- **Ordinal encoding**: memberi angka sesuai urutan. Cocok hanya untuk variabel **ordinal** (misalnya `Contract`: month-to-month < one year < two year).

> **Warning**
>
> **Jangan ordinal-encode variabel nominal** seperti `PaymentMethod` atau `InternetService`. Memberi angka sembarang akan menciptakan **hubungan palsu** yang menyesatkan model linear. Untuk variabel nominal, selalu one-hot.
> Untuk buku ini, kita **tidak** melakukan encoding di Bab 2. Encoding akan dilakukan di dalam *pipeline* scikit-learn di Bab 8–9 agar tidak terjadi *data leakage* antara data latih dan uji. Bab 2 hanya menyiapkan kolom kategorikal apa adanya.

2.8.3 Fitur Turunan yang Berguna

Beberapa fitur turunan yang masuk akal untuk dataset Telco:

```
# Jumlah layanan berlangganan (proxy "keterikatan pelanggan")
service_cols = ["PhoneService", "MultipleLines", "OnlineSecurity",
                "OnlineBackup", "DeviceProtection", "TechSupport",
                "StreamingTV", "StreamingMovies"]
df["num_services"] = (df[service_cols] == "Yes").sum(axis=1)

# Rasio tagihan bulanan terhadap total (proxy "lama berlangganan efektif")
df["charge_ratio"] = df["TotalCharges"] / (df["MonthlyCharges"] * df["tenure"] + 1)

print(df[["num_services", "charge_ratio"]].describe().round(2))
```

```
       num_services  charge_ratio
count       7043.00       7043.00
mean           3.36          0.99
std            2.06          0.06
min            0.00          0.00
25%            1.00          0.97
50%            3.00          1.00
```

```
75%             5.00          1.02
max             8.00          1.53
```

Fitur-fitur ini akan **dipertimbangkan** saat membangun model di Bagian II, bukan langsung dipakai. Kadang fitur turunan membantu; kadang ia hanya menambah noise. Kita akan mengujinya secara empiris.

2.9 Menyimpan Dataset Bersih

Setelah semua transformasi, simpan dataset bersih ke `data/processed/`. Ini memberi Anda **satu titik referensi** untuk semua bab selanjutnya—jika terjadi kesalahan, Anda tahu persis versi data apa yang digunakan.

```python
from pathlib import Path

out = Path("data/processed/telco_clean.csv")
out.parent.mkdir(parents=True, exist_ok=True)

# Drop kolom yang tidak akan dipakai
cols_to_drop = ["customerID"]  # pengenal unik, bukan fitur
df_clean = df.drop(columns=cols_to_drop)

df_clean.to_csv(out, index=False)
print(f"Dataset bersih: {df_clean.shape[0]} baris × {df_clean.shape[1]} kolom")
print(f"Disimpan di   : {out.resolve()}")
```

```
Dataset bersih: 7043 baris × 23 kolom
Disimpan di   : /Users/chakraroy/Documents/Ebook/Decision-Analytics/data/processed/telco_clea
```

> **Tip**
>
> **Selalu catat transformasi yang dilakukan.** Idealnya dalam file `data/README.md` atau skrip `code/prepare_data.py` yang menjalankan seluruh alur dari mentah ke bersih secara otomatis. Prinsipnya: **jika Anda harus mengulang wrangling dari awal, Anda tidak ingin mengingat-ingat apa yang sudah dilakukan.** Skrip yang dapat dijalankan ulang adalah dokumentasi terbaik.

Verifikasi bahwa dataset tersimpan dengan benar:

```
df_check = pd.read_csv("data/processed/telco_clean.csv")
print(f"Bentuk  : {df_check.shape}")
print(f"Kolom   : {df_check.columns.tolist()}")
print(f"Tipe    : \n{df_check.dtypes.value_counts()}")
```

```
Bentuk  : (7043, 23)
Kolom   : ['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'Mul
Tipe    :
object     17
int64       3
float64     3
Name: count, dtype: int64
```

2.10 Latihan Terbimbing

Latihan 2.1 — Mekanisme nilai hilang. Untuk setiap kolom yang masih memiliki `NaN` (jika ada), tentukan apakah mekanismenya MCAR, MAR, atau MNAR. Jelaskan alasannya dalam satu paragraf. Jika tidak ada `NaN` tersisa, jelaskan mengapa Bab 1 menemukan "nilai hilang" padahal Bab 2 tidak.

Latihan 2.2 — Encoding alternatif. Kode di Bab 2 memakai `pd.cut` untuk `tenure`. Coba ganti dengan `pd.qcut` (kuantil). Bandingkan distribusi kelompok yang dihasilkan. Kapan Anda memilih `cut` (bin yang ditentukan domain) dan kapan `qcut` (bin yang ditentukan data)?

Latihan 2.3 — Fitur turunan yang diuji. Fitur `charge_ratio` yang kita buat punya rumus `TotalCharges / (MonthlyCharges × tenure + 1)`. Untuk pelanggan dengan `tenure = 0`, nilai ini menjadi `0 / (0 + 1) = 0`. Apakah ini masuk akal? Bagaimana Anda akan mendefinisikannya ulang agar lebih bermakna? (Petunjuk: lihat pelanggan dengan `tenure` 1 saja.)

Latihan 2.4 — Skrip yang dapat dijalankan ulang. Buat file `code/prepare_data.py` yang menjalankan **seluruh** wrangling di bab ini dari mentah ke bersih, dengan satu perintah `python code/prepare_data.py`. Ini adalah latihan penting untuk **reproduksibilitas**—keterampilan yang akan terus Anda pakai di sepanjang karier.

> **Note**
>
> **Petunjuk Latihan 2.1.** Jika Anda hanya menemukan `TotalCharges` yang bermasalah di Bab 1, itu karena nilai hilang di kolom lain **tidak ada**—dataset Telco relatif bersih. Ketika Anda bekerja dengan dataset nyata, Anda akan menemukan bahwa nilai hilang tersebar di banyak kolom, dan diagnosis mekanisme menjadi jauh lebih penting.

2.11 Kotak "Jembatan": Dari Data ke Keputusan

> **!** Important
>
> **Jembatan.** Bab 2 memengaruhi keputusan di **tiga tingkat**:
>
> 1. **Keputusan tentang kepercayaan.** Setelah wrangling, Anda kini dapat **menjawab pertanyaan** dengan `TotalCharges` tanpa ragu bahwa nilainya mencerminkan string atau bukan. Analisis rata-rata tagihan pelanggan baru kini valid.
> 2. **Keputusan tentang metrik.** Mengetahui bahwa 11 pelanggan baru memiliki `TotalCharges = 0` (setelah imputasi) memberi tahu Anda bahwa **rata-rata tagihan akan bias ke bawah** jika tidak difilter. Keputusan pengukuran KPI pelanggan baru harus menggunakan median atau subset.
> 3. **Keputusan tentang fitur.** Menambahkan `tenure_group` dan `num_services` bukan sekadar trik teknis—ini adalah **hipotesis**: *"Pelanggan baru mungkin berperilaku berbeda dari pelanggan lama; pelanggan dengan lebih banyak layanan mungkin lebih setia."* Hipotesis ini akan **diuji** di Bagian II. Jika terbukti, mereka memengaruhi strategi retensi; jika tidak, mereka hanya menambah noise dan harus dibuang.
>
> Wrangling yang baik adalah **pertanyaan yang diajukan dengan hati-hati kepada data**—bukan sekadar pembersihan mekanis.

2.12 Rangkuman

Dalam bab ini kita telah:

- **Memetakan sumber data** dan memahami kapan setiap sumber sesuai untuk konteks tertentu.
- **Membedakan enam dimensi kualitas data** dan teknik diagnosis yang sesuai untuk masing-masing.
- **Menyelesaikan masalah `TotalCharges`** dengan keputusan berbasis domain—imputasi 0 untuk pelanggan baru, bukan drop yang kehilangan informasi.
- **Memeriksa duplikat dan konsistensi** nilai kategorikal sebelum analisis.
- **Mendeteksi outlier** dengan dua metode (IQR dan Z-score), dan—yang lebih penting—**menahan diri untuk tidak menghapusnya** tanpa alasan domain.
- **Membuat fitur turunan** yang masuk akal secara bisnis (`tenure_group`, `num_services`, `charge_ratio`) yang akan **diuji secara empiris** di Bagian II, bukan langsung dipakai.
- **Menyimpan dataset bersih** ke `data/processed/telco_clean.csv` dan memverifikasi hasilnya.

2.13 Jembatan ke Bab 3

Dataset kita kini bersih, konsisten, dan tersimpan. Tetapi kita belum benar-benar **melihatnya**. Berapa rata-rata pelanggan yang churn? Apakah pelanggan dengan kontrak bulanan lebih sering churn? Apakah `MonthlyCharges` berbeda antara pelanggan yang churn dan yang bertahan?

Bab berikutnya—**Statistik Deskriptif & Eksplorasi Data**—akan mulai menjawab pertanyaan-pertanyaan ini secara sistematis. Kita akan berkenalan dengan ukuran pusat dan sebaran, uji statistik dasar, dan mulai membangun **intuisi tentang dataset** yang akan menjadi fondasi semua analisis berikutnya.

3 Statistik Deskriptif & Eksplorasi Data

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Memilih ukuran pusat dan sebaran yang tepat untuk setiap jenis distribusi. 2. Membaca bentuk distribusi (kemencengan, ekor) dan memahami implikasinya bagi analisis selanjutnya. 3. Melakukan eksplorasi univariat dan bivariat secara sistematis. 4. Menguji secara statistik apakah perbedaan antar kelompok bermakna atau hanya kebetulan. 5. Melaporkan besaran efek (*effect size*), bukan hanya nilai p. 6. Memahami probabilitas sebagai jembatan dari deskriptif ke prediktif.

3.1 Dari Data ke Pertanyaan

Dataset yang kita miliki di Bab 2 sudah bersih. Tetapi data bersih bukanlah pengetahuan. Bab ini adalah tentang **mengajukan pertanyaan kepada data**—dan menjawabnya dengan angka yang dapat diinterpretasikan.

Kita akan memulai dari statistik paling sederhana (rata-rata), lalu bergerak menuju pertanyaan yang lebih menarik: **Apakah pelanggan yang churn berbeda secara sistematis dari yang bertahan?** Jika ya, dalam hal apa? Seberapa besar perbedaannya? Dan apakah perbedaan itu signifikan secara statistik, atau hanya kebetulan?

Tip

Statistik deskriptif bukan sekadar "meringkas". Ia adalah **bahasa untuk mengajukan pertanyaan**. Setiap ukuran yang kita pilih—rata-rata vs. median, standar deviasi vs. IQR—mengandung asumsi tentang data dan mengarahkan kita pada jenis kesimpulan tertentu. Bab ini mengajarkan Anda memilih dengan sadar.

3.2 Memuat Dataset Bersih

Kita mulai dari titik di mana Bab 2 berhenti:

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from decanal.utils import load_churn_clean

df = load_churn_clean()
print(f"Bentuk: {df.shape[0]:,} baris × {df.shape[1]} kolom")
```

```
Bentuk: 7,043 baris × 23 kolom
```

Kita juga perlu menghitung **churn rate**—metrik paling fundamental dalam dataset ini—sebagai jangkar untuk seluruh bab:

```
n_total = len(df)
n_churn = (df["Churn"] == "Yes").sum()
churn_rate = n_churn / n_total

print(f"Total pelanggan : {n_total:,}")
print(f"Churn           : {n_churn:,}")
print(f"Churn rate      : {churn_rate:.2%}")
```

```
Total pelanggan : 7,043
Churn           : 1,869
Churn rate      : 26.54%
```

Sekitar 1 dari 4 pelanggan churn. Angka ini akan menjadi referensi di seluruh bab: setiap kali kita membandingkan kelompok, kita akan bertanya "apakah kelompok ini churn lebih sering atau lebih jarang dari 26,5%?"

3.3 Ukuran Pusat: Tiga Cara Menjawab "Berapa Rata-rata?"

Ada tiga ukuran pusat yang perlu Anda kuasai, dan memilih salah satunya bukan sekadar preferensi—ia mengandung asumsi tentang distribusi dan tujuan analisis.

Ukuran	Kapan tepat dipakai	Kapan menyesatkan
Mean (rata-rata)	Distribusi simetris, tidak ada outlier ekstrem	Distribusi miring (mis. pendapatan, tagihan)
Median (nilai tengah)	Distribusi miring, ada outlier	Data kategorikal, distribusi multimodal
Mode (nilai tersering)	Data kategorikal, deteksi pola	Data kontinu (hampir tidak pernah berulang)

Mari kita lihat ketiganya pada tiga kolom numerik utama:

```
numeric_cols = ["tenure", "MonthlyCharges", "TotalCharges"]

ringkasan = pd.DataFrame({
    "mean":   [df[c].mean()   for c in numeric_cols],
    "median": [df[c].median() for c in numeric_cols],
    "mode":   [df[c].mode().iloc[0] for c in numeric_cols],
    "selisih (mean - median)": [
        df[c].mean() - df[c].median() for c in numeric_cols
    ],
}, index=numeric_cols).round(2)

ringkasan
```

	mean	median	mode	selisih (mean - median)
tenure	32.37	29.00	1.00	3.37
MonthlyCharges	64.76	70.35	20.05	-5.59
TotalCharges	2279.73	1394.55	0.00	885.18

Cara membaca tabel ini. Jika `mean` ≈ `median`, distribusinya kurang lebih simetris. Jika `mean > median`, distribusinya **miring ke kanan** (*right-skewed*): ada beberapa nilai sangat tinggi yang menarik rata-rata ke atas. Jika `mean < median`, distribusinya miring ke kiri.

Perhatikan `TotalCharges`: `mean` jauh lebih besar dari `median`. Ini adalah pola klasik distribusi miring kanan—beberapa pelanggan dengan `tenure` panjang dan tagihan bulanan tinggi menghasilkan `TotalCharges` yang sangat besar, sementara sebagian besar pelanggan memiliki total yang lebih kecil.

> **!** Important
>
> **Mengapa ini penting untuk keputusan?** Jika Anda melaporkan "rata-rata pelanggan menghabiskan Rp X", Anda mengasumsikan **setiap pelanggan** berbelanja di sekitar angka itu. Kenyataannya, sebagian besar pelanggan berbelanja lebih sedikit dari rata-rata—mean ditarik oleh minoritas pelanggan bernilai tinggi. Untuk keputusan pricing, **median lebih jujur**; untuk keputusan alokasi anggaran (di mana total yang penting, bukan rata-rata per pelanggan), **mean tetap relevan**.

3.4 Ukuran Sebaran: Seberapa Lebar Data Tersebar?

Ukuran pusat saja menyesatkan tanpa sebaran. Dua kelompok bisa memiliki rata-rata yang sama tetapi sangat berbeda dalam keragaman—dan keragaman itu sendiri adalah informasi.

Ukuran	Rentang nilai	Kelebihan	Kekurangan
Range	`[min, max]`	Sederhana, langsung terlihat	Sangat dipengaruhi outlier
Varians	0	Fondasi matematis	Satuannya kuadrat—sulit diinterpretasikan
Std. deviasi	0	Satuannya sama dengan data	Sensitif terhadap outlier
IQR (Q3 − Q1)	0	Tahan terhadap outlier	Tidak menggunakan semua titik data
Coefficient of Variation	0	Memungkinkan perbandingan antar skala	Tidak bermakna jika mean 0

```
sebaran = pd.DataFrame({
    "std":  [df[c].std() for c in numeric_cols],
    "min":  [df[c].min() for c in numeric_cols],
    "Q1":   [df[c].quantile(0.25) for c in numeric_cols],
    "Q3":   [df[c].quantile(0.75) for c in numeric_cols],
    "IQR":  [df[c].quantile(0.75) - df[c].quantile(0.25) for c in numeric_cols],
    "max":  [df[c].max() for c in numeric_cols],
}, index=numeric_cols).round(2)

sebaran["CV (%)"] = (
    df[numeric_cols].std() / df[numeric_cols].mean() * 100
```

```
).round(2)

sebaran
```

	std	min	Q1	Q3	IQR	max	CV (%)
tenure	24.56	0.00	9.00	55.00	46.00	72.00	75.87
MonthlyCharges	30.09	18.25	35.50	89.85	54.35	118.75	46.46
TotalCharges	2266.79	0.00	398.55	3786.60	3388.05	8684.80	99.43

Coefficient of Variation (CV) memungkinkan Anda membandingkan keragaman antar variabel dengan satuan berbeda. `MonthlyCharges` dan `TotalCharges` tidak bisa dibandingkan dengan std deviasi mentah (satuannya berbeda), tetapi CV-nya bisa: nilai CV yang lebih tinggi berarti variabel tersebut **lebih bervariasi secara relatif** terhadap rata-ratanya.

3.5 Bentuk Distribusi: Kemencengan dan Ekor

Dua angka menangkap "bentuk" distribusi:

- **Skewness** (kemencengan): 0 = simetris; > 0 = miring kanan; < 0 = miring kiri.
- **Kurtosis** (keruncingan): 0 = normal; > 0 = ekor gemuk (banyak outlier); < 0 = ekor tipis.

```
from scipy import stats

bentuk = pd.DataFrame({
    "skewness": [stats.skew(df[c]) for c in numeric_cols],
    "kurtosis": [stats.kurtosis(df[c]) for c in numeric_cols],
}, index=numeric_cols).round(3)

bentuk
```

	skewness	kurtosis
tenure	0.239	-1.387
MonthlyCharges	-0.220	-1.257
TotalCharges	0.963	-0.229

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.2))
for ax, col in zip(axes, numeric_cols):
    ax.hist(df[col], bins=30, color="#4C72B0", edgecolor="white")
    ax.axvline(df[col].mean(), color="red", linestyle="--", label=f"mean")
    ax.axvline(df[col].median(), color="orange", linestyle=":", label="median")
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel("Frekuensi")
axes[0].legend(fontsize=8)
plt.tight_layout()
plt.show()
```

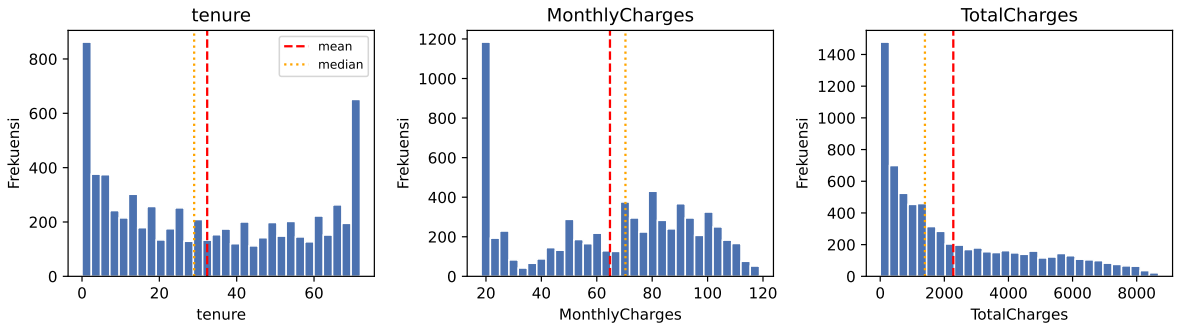


Figure 3.1: Distribusi tiga kolom numerik utama. Perhatikan bentuk miring kanan pada TotalCharges.

> **Note**
>
> **Implikasi untuk Bagian II.** Distribusi `TotalCharges` yang miring kanan akan menjadi perhatian ketika kita membangun model regresi di Bab 8. Model regresi linear mengasumsikan residual yang mendekati normal—dan distribusi yang miring parah sering melanggar asumsi ini. Salah satu solusinya adalah **transformasi logaritmik** (`np.log1p`), yang akan kita uji di Bab 8. Bab ini hanya menandai bahwa transformasi itu **mungkin diperlukan**; keputusan akhirnya diambil setelah kita melihat residual model.

3.6 Eksplorasi Univariat: Variabel Kategorikal

Untuk variabel kategorikal, ukuran pusat dan sebaran tidak berlaku sama. Yang lebih informatif adalah **distribusi frekuensi** dan **proporsi**.

```python
cat_cols = ["gender", "SeniorCitizen", "Partner", "Dependents",
            "Contract", "PaymentMethod", "InternetService"]

for col in cat_cols:
    vc = df[col].value_counts(normalize=True).mul(100).round(1)
    print(f"\n--- {col} ---")
    print(vc.to_string())
```

```
--- gender ---
gender
Male      50.5
Female    49.5

--- SeniorCitizen ---
SeniorCitizen
0    83.8
1    16.2

--- Partner ---
Partner
No     51.7
Yes    48.3

--- Dependents ---
Dependents
No     70.0
Yes    30.0

--- Contract ---
Contract
Month-to-month    55.0
Two year          24.1
One year          20.9

--- PaymentMethod ---
PaymentMethod
Electronic check             33.6
Mailed check                 22.9
Bank transfer (automatic)    21.9
Credit card (automatic)      21.6
```

```
--- InternetService ---
InternetService
Fiber optic    44.0
DSL            34.4
No             21.7
```

Sekilas, sebagian besar variabel kategorikal di dataset ini terdistribusi cukup seimbang. Tetapi ada satu yang mencolok: **`Contract`**—pelanggan bulanan (`Month-to-month`) mendominasi. Ini bukan kebetulan; ini adalah salah satu prediktor churn terkuat, dan kita akan melihatnya di bagian bivariat.

```
fig, ax = plt.subplots(figsize=(7, 3.5))
df["Contract"].value_counts().plot(kind="bar", ax=ax, color="#4C72B0")
ax.set_title("Distribusi Tipe Kontrak")
ax.set_xlabel("Tipe kontrak")
ax.set_ylabel("Jumlah pelanggan")
ax.set_xticklabels(ax.get_xticklabels(), rotation=15)
plt.tight_layout()
plt.show()
```

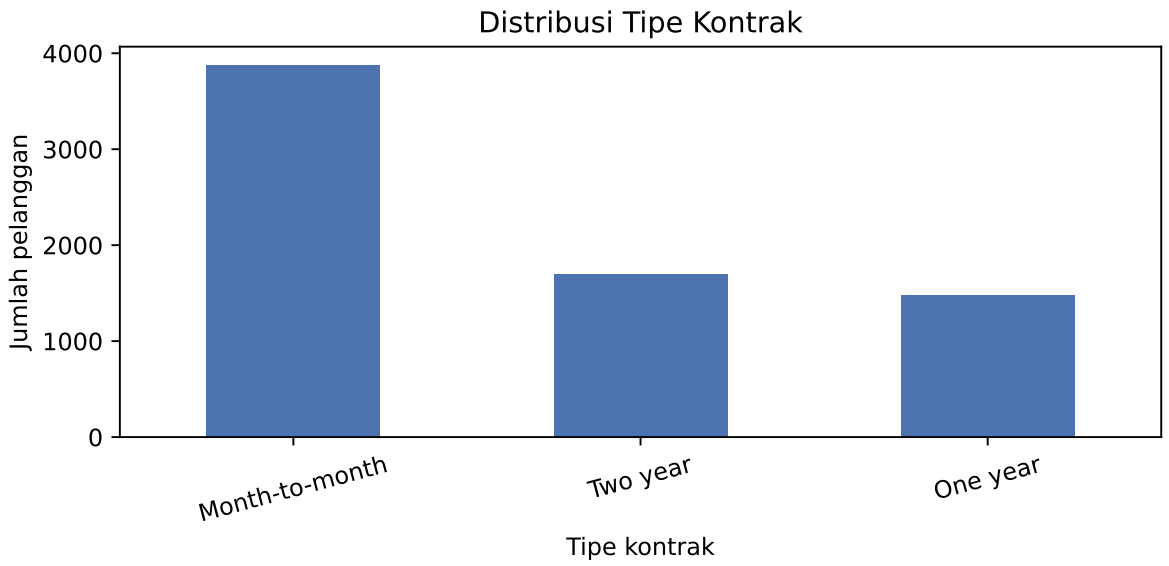


Figure 3.2: Distribusi tipe kontrak. Kontrak bulanan mendominasi dataset.

3.7 Eksplorasi Bivariat: Apakah Churn Berbeda Antar Kelompok?

Sekarang kita masuk ke bagian yang paling substantif: **membandingkan kelompok**. Pertanyaan kita adalah:

> *"Apakah pelanggan yang churn berbeda secara sistematis dari yang bertahan?"*

Kita akan memeriksa dua jenis pasangan: **kategorikal × target** dan **numerik × target**.

3.7.1 Kategorikal × Target: Cross-Tabulation

```
ct = pd.crosstab(df["Contract"], df["Churn"], normalize="index").mul(100).round(1)
ct.columns.name = "Churn (%)"
ct
```

Churn (%) Contract	No	Yes
Month-to-month	57.3	42.7
One year	88.7	11.3
Two year	97.2	2.8

Cara membaca. Setiap baris berjumlah 100%. Angka di kolom `Yes` adalah churn rate **di dalam** kelompok tersebut. Pelanggan dengan kontrak bulanan churn jauh lebih sering daripada pelanggan dengan kontrak dua tahun. Perbedaan ini tidak kecil—ini adalah **sinyal paling kuat** di dataset ini.

Mari kita bandingkan beberapa variabel kategorikal secara bersamaan:

```
ringkasan_kategori = []
for col in ["Contract", "InternetService", "PaymentMethod", "Partner", "Dependents"]:
    churn_by_group = df.groupby(col)["Churn"].apply(
        lambda s: (s == "Yes").mean() * 100
    ).round(1)
    for group, rate in churn_by_group.items():
        ringkasan_kategori.append({
            "variabel": col,
            "kelompok": group,
            "churn_rate (%)": rate,
        })
```

```
pd.DataFrame(ringkasan_kategori).pivot(
    index="variabel", columns="kelompok", values="churn_rate (%)"
).round(1)
```

kelompok variabel	Bank transfer (automatic)	Credit card (automatic)	DSL	Electronic check	Fiber optic
Contract	NaN	NaN	NaN	NaN	NaN
Dependents	NaN	NaN	NaN	NaN	NaN
InternetService	NaN	NaN	19.0	NaN	41.9
Partner	NaN	NaN	NaN	NaN	NaN
PaymentMethod	16.7	15.2	NaN	45.3	NaN

Tip

Pola berpikir. Ketika Anda melihat churn rate yang sangat berbeda antar kelompok, ada tiga kemungkinan yang perlu dipertimbangkan:

1. **Kausalitas langsung** — jenis kontrak *menyebabkan* churn lebih tinggi.
2. **Kebalikan kausalitas** — pelanggan yang *ingin* churn sengaja memilih kontrak bulanan (mereka tidak mau terikat).
3. **Variabel perancu** — pelanggan baru cenderung memilih kontrak bulanan *dan* juga cenderung churn karena belum berkomitmen.

Kita tidak dapat membedakan ketiganya hanya dari tabel ini. Itu akan membutuhkan desain eksperimen atau analisis kausal—di luar cakupan buku ini. Tetapi menyadari bahwa **perbedaan bukan berarti sebab-akibat** adalah salah satu kebiasaan berpikir terpenting dalam analisis data.

3.7.2 Numerik × Target: Grouped Comparison

Untuk variabel numerik, kita bandingkan distribusi antara dua kelompok churn:

```
perbandingan = df.groupby("Churn")[numeric_cols].agg(["mean", "median", "std"]).round(2)
perbandingan
```

	tenure			MonthlyCharges			TotalCharges		
	mean	median	std	mean	median	std	mean	median	std
Churn									
No	37.57	38.0	24.11	61.27	64.43	31.09	2549.91	1679.52	2329.95
Yes	17.98	10.0	19.53	74.44	79.65	24.67	1531.80	703.55	1890.82

Perhatikan `tenure`: pelanggan yang churn memiliki **rata-rata tenure jauh lebih pendek** daripada yang bertahan. Ini konsisten dengan intuisi: pelanggan baru lebih mungkin berhenti daripada pelanggan lama. Tetapi ada **aspek yang lebih halus**: perbedaan ini juga konsisten dengan hipotesis bahwa **intervensi retensi paling efektif dilakukan pada awal siklus hidup pelanggan**.

```
fig, ax = plt.subplots(figsize=(8, 4))
for label, color in [("No", "#4C72B0"), ("Yes", "#DD8452")]:
    subset = df[df["Churn"] == label]["tenure"]
    ax.hist(subset, bins=30, alpha=0.6, label=f"Churn = {label}", color=color)
ax.set_title("Distribusi Tenure berdasarkan Status Churn")
ax.set_xlabel("Tenure (bulan)")
ax.set_ylabel("Jumlah pelanggan")
ax.legend()
plt.tight_layout()
plt.show()
```

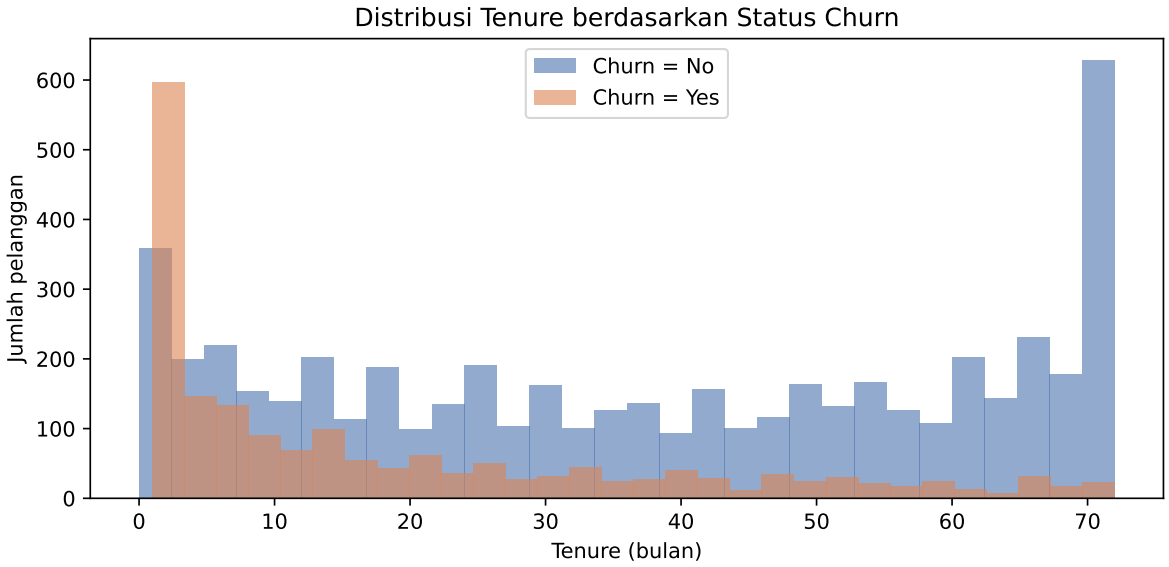


Figure 3.3: Distribusi tenure berdasarkan status churn. Pelanggan churn terkonsentrasi di tenure pendek.

3.8 Uji Statistik: Apakah Perbedaan Ini Nyata?

Sebuah perbedaan yang terlihat di data sampel belum tentu mencerminkan perbedaan yang sesungguhnya di populasi. **Uji hipotesis** membantu kita memutuskan apakah perbedaan yang kita amati cukup besar untuk tidak dijelaskan oleh kebetulan statistik.

3.8.1 Kerangka Uji Hipotesis

1. **H (hipotesis nol)**: tidak ada perbedaan antara kelompok.
2. **H (hipotesis alternatif)**: ada perbedaan.
3. Hitung **statistik uji** yang mengukur seberapa jauh data dari H .
4. Bandingkan dengan distribusi teoretis untuk mendapatkan **p-value**.
5. Tolak H jika p-value < (biasanya 0,05).

> **Warning**
>
> **p-value bukan probabilitas bahwa H benar.** Ia adalah probabilitas **mengamati data seperti ini** (atau lebih ekstrem) **jika H benar**. Ini adalah salah satu kesalahpahaman paling umum dalam statistika. Menolak H tidak berarti "membuktikan H "; ia hanya berarti "data tidak konsisten dengan H ".

3.8.2 Uji Chi-Square untuk Kategorikal

Chi-square menguji apakah dua variabel kategorikal **independen** satu sama lain.

```
from scipy.stats import chi2_contingency

for col in ["Contract", "InternetService", "PaymentMethod", "Partner"]:
    ct = pd.crosstab(df[col], df["Churn"])
    chi2, p, dof, _ = chi2_contingency(ct)
    print(f"{col:16s}: chi² = {chi2:8.2f}, p = {p:.2e}")
```

```
Contract        : chi² =  1184.60, p = 5.86e-258
InternetService : chi² =   732.31, p = 9.57e-160
PaymentMethod   : chi² =   648.14, p = 3.68e-140
Partner         : chi² =   158.73, p = 2.14e-36
```

Semua nilai p sangat kecil—jauh di bawah 0,05. Artinya, kita **menolak H** untuk setiap variabel: tidak ada variabel ini yang independen dari `Churn`. Tetapi, seperti yang akan kita lihat sebentar lagi, **nilai p kecil bukan berarti efek besar**.

3.8.3 Uji-t untuk Numerik

Uji-t membandingkan rata-rata dua kelompok.

```
from scipy.stats import ttest_ind, mannwhitneyu

for col in numeric_cols:
    grup_no = df[df["Churn"] == "No"][col]
    grup_yes = df[df["Churn"] == "Yes"][col]

    t, p_t = ttest_ind(grup_no, grup_yes, equal_var=False)
    u, p_u = mannwhitneyu(grup_no, grup_yes, alternative="two-sided")

    print(f"{col:16s}: t = {t:7.2f}, p(t-test) = {p_t:.2e}, p(Mann-Whitney) = {p_u:.2e}")
```

```
tenure          : t =   34.82, p(t-test) = 1.20e-232, p(Mann-Whitney) = 2.42e-
208
MonthlyCharges  : t =  -18.41, p(t-test) = 8.59e-73, p(Mann-Whitney) = 3.31e-
54
TotalCharges    : t =   18.71, p(t-test) = 5.90e-75, p(Mann-Whitney) = 5.69e-
83
```

Mengapa dua uji? Uji-t mengasumsikan distribusi yang mendekati normal (atau sampel yang sangat besar). Mann-Whitney tidak membuat asumsi distribusi—ia menguji apakah satu kelompok cenderung memiliki nilai lebih tinggi dari yang lain. Jika kedua uji memberi kesimpulan yang sama, kita bisa lebih yakin. Jika mereka berbeda, itu tanda bahwa asumsi uji-t mungkin dilanggar.

3.9 Effect Size: Seberapa Besar Perbedaannya?

Nilai p memberitahu kita **apakah** ada perbedaan. Ia tidak memberitahu kita **seberapa besar**. Dengan 7.043 baris, hampir semua perbedaan sepele akan signifikan secara statistik. Yang kita butuhkan adalah **effect size**.

3.9.1 Untuk Kategorikal: Cramér's V

```
from scipy.stats import chi2_contingency

def cramers_v(x: pd.Series, y: pd.Series) -> float:
    ct = pd.crosstab(x, y)
    chi2, _, _, _ = chi2_contingency(ct)
    n = ct.sum().sum()
    return np.sqrt(chi2 / (n * (min(ct.shape) - 1)))

efek_kat = pd.DataFrame({
    col: [cramers_v(df[col], df["Churn"])]
    for col in ["Contract", "InternetService", "PaymentMethod", "Partner"]
}, index=["Cramér's V"]).T.round(3)

efek_kat
```

	Cramér's V
Contract	0.410
InternetService	0.322
PaymentMethod	0.303
Partner	0.150

Interpretasi kasar Cramér's V: - $<$ **0,10**: efek sangat kecil - **0,10–0,30**: efek kecil - **0,30–0,50**: efek menengah - $>$ **0,50**: efek besar

3.9.2 Untuk Numerik: Cohen's d

```
def cohens_d(x: pd.Series, y: pd.Series) -> float:
    pooled_std = np.sqrt(((len(x) - 1) * x.var() + (len(y) - 1) * y.var()) /
                         (len(x) + len(y) - 2))
    return (x.mean() - y.mean()) / pooled_std

efek_num = pd.DataFrame({
    col: [cohens_d(df[df["Churn"] == "Yes"][col],
                   df[df["Churn"] == "No"][col])]
    for col in numeric_cols
}, index=["Cohen's d"]).T.round(3)

efek_num["|d| kategori"] = efek_num["Cohen's d"].abs().apply(
    lambda v: "kecil" if v < 0.2 else "menengah" if v < 0.5 else "besar"
)
efek_num
```

	Cohen's d	|d| kategori
tenure	-0.852	besar
MonthlyCharges	0.446	menengah
TotalCharges	-0.458	menengah

Interpretasi kasar Cohen's d: - $<$ **0,20**: efek sangat kecil - **0,20–0,50**: efek kecil - **0,50–0,80**: efek menengah - $>$ **0,80**: efek besar

> **!** Important
>
> **Pelajaran penting.** `tenure` adalah salah satu prediktor terkuat di dataset ini, tetapi Cohen's d-nya masih di kategori "menengah" atau bahkan "kecil" tergantung cutoff. Ini tidak berarti `tenure` tidak penting—ia tetap informatif ketika dikombinasikan dengan variabel lain. Tetapi **tidak ada satu variabel pun yang dengan sendirinya menjelaskan churn**. Ini adalah pelajaran penting yang akan diuji ketika kita membangun model di Bab 9: model yang baik memerlukan **banyak prediktor lemah** yang bekerja bersama, bukan satu prediktor kuat yang dominan.

3.10 Korelasi Antar Variabel Numerik

Sebelum masuk ke model, kita perlu memeriksa **hubungan antar fitur**—bukan hanya antar fitur dan target.

```
corr = df[numeric_cols].corr().round(3)
corr
```

	tenure	MonthlyCharges	TotalCharges
tenure	1.000	0.248	0.826
MonthlyCharges	0.248	1.000	0.651
TotalCharges	0.826	0.651	1.000

```
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(corr, annot=True, cmap="coolwarm", center=0, fmt=".2f",
            square=True, cbar_kws={"shrink": 0.8}, ax=ax)
ax.set_title("Korelasi antar variabel numerik")
plt.tight_layout()
plt.show()
```

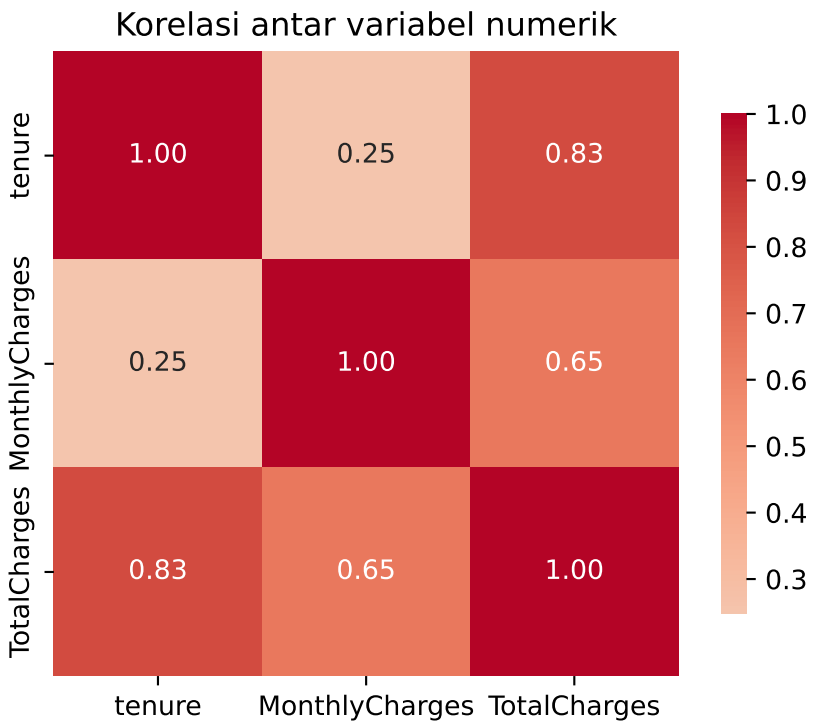


Figure 3.4: Matriks korelasi antar variabel numerik. Perhatikan korelasi tinggi antara tenure dan TotalCharges.

Korelasi `tenure` dan `TotalCharges` tinggi—dan ini **masuk akal secara domain**: pelanggan yang lebih lama secara alami memiliki total tagihan yang lebih besar. Ini bukan masalah *per se*, tetapi menjadi perhatian ketika kita membangun model regresi linear di Bab 8—**multicolinearity** dapat membuat koefisien sulit diinterpretasikan. Untuk model berbasis pohon (Bab 9), multicolinearity tidak menjadi masalah.

> **Warning**
>
> **Korelasi bukan kausalitas.** Korelasi `tenure`–`TotalCharges` tidak berarti tenure *menyebabkan* tagihan tinggi. Keduanya adalah **konsekuensi bersama** dari fakta sederhana: semakin lama pelanggan berlangganan, semakin banyak yang mereka bayarkan. Ini adalah contoh sempurna dari **variabel perancu** yang harus diwaspadai—dan alasan mengapa analisis kausal membutuhkan desain yang lebih cermat daripada sekadar korelasi.

3.11 Probabilitas: Jembatan ke Bagian II

Bab ini telah membahas **statistik deskriptif**—menggambarkan apa yang ada di data. Bab berikutnya akan mulai berbicara tentang **probabilitas**—menggambarkan **seberapa besar kemungkinan** sesuatu terjadi.

Mengapa probabilitas penting di sini? Karena seluruh Bagian II dibangun di atas gagasan bahwa:

> *"Model prediktif adalah fungsi yang menghasilkan **probabilitas** atau **estimasi**, bukan kepastian."*

Churn rate 26,5% yang kita hitung di awal bab bukan hanya statistik deskriptif—ia juga bisa dibaca sebagai **estimasi probabilitas**: jika kita memilih pelanggan secara acak, ada sekitar 26,5% kemungkinan ia akan churn. Model prediktif di Bab 9 akan memperbaiki estimasi ini dengan menggunakan informasi tambahan: pelanggan dengan kontrak bulanan, tenure pendek, dan tagihan tinggi memiliki probabilitas churn yang **lebih tinggi** dari 26,5%; yang lain lebih rendah.

> **Tip**
>
> **Latihan berpikir.** Ambil satu pelanggan dari dataset. Tanpa model, tebakan terbaik Anda tentang probabilitas churn-nya adalah 26,5% (churn rate keseluruhan). Setelah membaca bab ini, Anda tahu bahwa pelanggan dengan kontrak bulanan memiliki churn rate sekitar 43%, sementara pelanggan dengan kontrak dua tahun hanya sekitar 3%. **Informasi ini sudah memperbaiki tebakan Anda**—dan itulah inti dari semua yang akan kita pelajari di Bagian II.

3.12 Latihan Terbimbing

Latihan 3.1 — Mean vs median untuk keputusan. Anda adalah manajer produk yang ingin melaporkan "tagihan bulanan pelanggan tipikal" kepada direksi. Apakah Anda akan melaporkan `mean` atau `median` dari `MonthlyCharges`? Jelaskan pilihan Anda dalam satu paragraf, dengan mempertimbangkan siapa audiensnya dan keputusan apa yang akan diambil berdasarkan angka tersebut.

Latihan 3.2 — Effect size vs signifikansi. Untuk setiap variabel kategorikal di dataset, hitung Cramér's V-nya. Urutkan dari yang terbesar ke terkecil. Variabel mana yang memiliki efek terkuat terhadap `Churn`? Apakah urutan ini konsisten dengan intuisi Anda dari tabel churn rate per kelompok?

Latihan 3.3 — Uji normalitas. Gunakan `scipy.stats.shapiro` atau `scipy.stats.normaltest` untuk menguji apakah `tenure`, `MonthlyCharges`, dan `TotalCharges` berdistribusi normal.

Apa kesimpulan Anda? Apakah asumsi normalitas terpenuhi? Jika tidak, apa implikasinya untuk uji-t yang sudah kita jalankan?

Latihan 3.4 — Korelasi dan kausalitas. Identifikasi satu pasangan variabel di dataset yang **berkorelasi tetapi tidak mungkin memiliki hubungan kausal langsung**. Jelaskan mengapa korelasi itu muncul (petunjuk: pikirkan variabel perancu atau hubungan sebab-akibat terbalik).

Latihan 3.5 — Churn rate per segmen. Buat tabel churn rate untuk **kombinasi** dua variabel: `Contract` dan `InternetService`. Apakah ada segmen dengan churn rate di atas 50%? Segmen mana yang paling berisiko, dan bagaimana temuan ini dapat memengaruhi strategi retensi?

> **Note**
>
> **Petunjuk Latihan 3.3.** Untuk sampel sebesar 7.043 baris, uji normalitas hampir selalu menolak H , bahkan untuk penyimpangan kecil dari normalitas. Dalam praktik, kita sering lebih memperhatikan **Q-Q plot** dan **skewness** daripada nilai p uji normalitas. Ini adalah contoh lain dari mengapa effect size lebih informatif daripada signifikansi statistik.

3.13 Kotak "Jembatan": Dari Data ke Keputusan

> **Important**
>
> **Jembatan.** Bab ini menghasilkan tiga jenis insight yang langsung dapat memengaruhi keputusan:
>
> 1. **Segmentasi risiko.** Churn rate berbeda drastis antar segmen: kontrak bulanan (~43%) vs. kontrak dua tahun (~3%). Ini memberi tahu kita **di mana intervensi retensi paling dibutuhkan**. Keputusan: alokasikan anggaran retensi secara tidak proporsional ke pelanggan kontrak bulanan.
> 2. **Prioritas fitur.** Analisis effect size menunjukkan bahwa **tidak ada satu variabel pun yang dominan**—tetapi kombinasi beberapa variabel lemah dapat menjadi prediktor yang kuat. Ini mengarahkan kita untuk **membangun model multivariabel** di Bagian II, bukan mengandalkan aturan sederhana berbasis satu variabel.
> 3. **Batasan analisis ini.** Korelasi yang kuat bukan bukti kausalitas. Jika kita ingin menguji apakah intervensi retensi **benar-benar mengurangi churn** (bukan hanya berkorelasi dengan penurunan churn), kita perlu desain eksperimen—A/B test atau quasi-experimental design. Ini adalah batas dari analisis deskriptif, dan pengakuan akan batas ini adalah tanda kedewasaan analitis.

Analisis deskriptif yang baik tidak menghasilkan jawaban final—ia menghasilkan **pertanyaan yang lebih tajam**. Dan pertanyaan-pertanyaan itulah yang akan kita jawab di Bagian II dan III.

3.14 Rangkuman

Dalam bab ini kita telah:

- **Membedakan mean, median, dan mode**, dan memahami kapan masing-masing tepat digunakan—terutama pada distribusi miring seperti `TotalCharges`.
- **Mengukur sebaran** dengan std deviasi, IQR, dan coefficient of variation, dan memahami bahwa setiap ukuran memiliki kekuatan dan keterbatasan.
- **Membaca bentuk distribusi** melalui skewness dan kurtosis, dan menyadari implikasinya untuk pemodelan di Bagian II (terutama perlunya transformasi logaritmik).
- **Mengeksplorasi data secara univariat dan bivariat**, mengidentifikasi bahwa `Contract`, `tenure`, dan `InternetService` adalah prediktor churn yang paling mencolok.
- **Menguji perbedaan secara statistik** dengan chi-square dan uji-t, dan—yang lebih penting—membedakan antara **signifikansi statistik** dan **besaran efek**.
- **Menghitung effect size** (Cramér's V, Cohen's d) untuk memahami seberapa besar perbedaan yang sebenarnya.
- **Memeriksa korelasi antar fitur**, mengidentifikasi multicolinearity antara `tenure` dan `TotalCharges` yang akan relevan di Bab 8.
- **Menghubungkan statistik deskriptif dengan probabilitas**, membangun jembatan konseptual dari "apa yang ada" menuju "apa yang mungkin".

3.15 Jembatan ke Bab 4

Kita telah **mengukur** dan **menguji**—tetapi belum benar-benar **melihat**. Angka dan tabel hanya sebagian dari cerita. Mata manusia jauh lebih cepat dalam mengenali pola, anomali, dan tren ketika disajikan dalam bentuk **visualisasi**.

Bab berikutnya—**Visualisasi Data & Dashboard**—akan mengajarkan Anda **bahasa visual** untuk analytics. Kita akan berkenalan dengan prinsip-prinsip visualisasi yang efektif, memilih chart yang tepat untuk pertanyaan yang tepat, dan membangun dashboard interaktif menggunakan Plotly—yang akan menjadi dasar untuk eksplorasi di bab-bab berikutnya, serta template untuk dashboard yang akan Anda bangun di dunia nyata.

4 Visualisasi Data & Dashboard

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Memilih jenis chart yang tepat untuk setiap jenis pertanyaan analitis. 2. Menerapkan prinsip *grammar of graphics* dan *data-ink ratio* untuk menghasilkan visualisasi yang efektif. 3. Menggunakan `matplotlib` dan `seaborn` untuk visualisasi statis yang siap publikasi. 4. Menggunakan `plotly` untuk visualisasi interaktif yang dapat dieksplorasi. 5. Mengenali dan menghindari kesalahan visualisasi yang paling umum—terutama yang menyesatkan. 6. Merancang dashboard sederhana yang menjawab pertanyaan manajerial.

4.1 Mengapa Visualisasi?

Di Bab 3 kita menemukan bahwa churn rate pelanggan kontrak bulanan sekitar 43%, sementara kontrak dua tahun hanya sekitar 3%. Angka itu kuat. Tetapi ketika kita **melihatnya** dalam bentuk bar chart, kekuatannya berlipat: mata manusia langsung menangkap **jarak** antara dua batang, **proporsi** area, dan **pola** yang sulit diungkapkan dalam tabel.

Visualisasi adalah **alat berpikir**, bukan hanya alat presentasi. Sebagian besar analis menemukan insight bukan dari tabel yang mereka baca, melainkan dari grafik yang mereka **lihat**—karena otak manusia jauh lebih cepat memproses pola visual daripada angka.

Tip

Dua peran visualisasi. Pertama, *exploratory*: untuk diri Anda sendiri, menemukan pola dan anomali. Kedua, *explanatory*: untuk audiens, menyampaikan insight yang sudah ditemukan. **Keduanya membutuhkan pendekatan berbeda.** Grafik eksplorasi boleh berantakan; grafik eksplanatori harus rapi dan terfokus. Bab ini mengajarkan keduanya.

4.2 Grammar of Graphics: Bahasa Universal Visualisasi

Sebelum menyentuh kode, pahami kerangka berpikir yang menjadi fondasi hampir semua library visualisasi modern (ggplot2, plotnine, plotly, altair): **grammar of graphics** (Wilkinson,

1999).

Setiap grafik dapat diuraikan menjadi komponen-komponen berikut:

Komponen	Pertanyaan	Contoh pada dataset kita
Data	Data apa yang divisualisasikan?	`df` dengan kolom `Contract`, `tenure`, `Churn`
Aesthetic (`aes`)	Variabel mana yang dipetakan ke atribut visual?	x = `Contract`, y = `tenure`, warna = `Churn`
Geometric (`geom`)	Bentuk apa yang digunakan?	Bar (`geom_bar`), titik (`geom_point`), garis (`geom_line`)
Facet	Apakah data dibagi ke panel-panel?	Panel per `InternetService`
Statistical transformation	Apakah ada agregasi?	Rata-rata per kategori, atau hitung (count)
Coordinate system	Bagaimana posisi dipetakan ke bidang?	Cartesian (default), polar (untuk pie chart)
Scale	Bagaimana nilai dipetakan ke rentang visual?	Log scale, warna kategori

Kerangka ini menjelaskan **mengapa** `ggplot2` dan `plotly` terasa lebih konsisten dibandingkan `matplotlib` murni: keduanya mengekspresikan grafik sebagai **kombinasi komponen**, bukan sebagai instruksi imperatif ("gambar garis dari sini ke sini").

Kita akan memakai `matplotlib` untuk kontrol penuh (statis) dan `plotly` untuk interaktivitas. `seaborn` menjembatani keduanya dengan API tingkat tinggi berbasis grammar of graphics.

4.3 Prinsip Tufte: Data-Ink Ratio

Edward Tufte, dalam bukunya *The Visual Display of Quantitative Information*, merumuskan prinsip yang menjadi standar emas visualisasi data:

> *"Above all else, show the data. Maximize the data-ink ratio, within reason."*

Data-ink adalah tinta (atau piksel) yang menampilkan data. **Chartjunk** adalah tinta yang tidak menampilkan data—garis grid berlebih, bayangan, gradien dekoratif, label sumbu yang berulang tanpa alasan.

Beberapa aturan praktis yang diturunkan dari prinsip ini:

Hindari	Ganti dengan
Gridlines berlebih	Gridlines minimal atau hilangkan
Border kotak penuh	Garis sumbu ringan atau hilangkan
3D bar/pie chart	2D, tanpa dimensi palsu
Warna dekoratif	Warna yang mengkodekan informasi
Label sumbu berulang	Satu label sumbu yang jelas
Legend jika warna sudah jelas	Direct labeling
Background berwarna	Background putih atau abu-abu muda

Warning

Pie chart. Pie chart hampir selalu pilihan yang buruk untuk data analitis karena mata manusia buruk dalam membandingkan sudut dan area. Pie chart hanya masuk akal jika Anda memiliki **dua kategori** dan ingin menyoroti bahwa **satu mendominasi**. Untuk semua kasus lain, bar chart lebih efektif—dan pada Bab 3 kita sudah memakai bar chart untuk `Contract`, bukan pie chart. Ini adalah pilihan yang disengaja.

4.4 Memilih Chart yang Tepat

Ini adalah tabel referensi yang perlu Anda hafal (atau simpan sebagai cheat sheet):

Pertanyaan	Chart yang tepat	Contoh pada dataset kita
Bagaimana distribusi satu variabel kontinu?	Histogram, density plot, boxplot	Distribusi `tenure`
Bagaimana distribusi satu variabel kategorikal?	Bar chart (count)	Distribusi `Contract`
Bagaimana hubungan dua variabel kontinu?	Scatter plot	`tenure` vs `MonthlyCharges`
Bagaimana hubungan variabel kontinu dengan kategorikal?	Boxplot, violin plot	`tenure` per `Contract`
Bagaimana hubungan dua variabel kategorikal?	Mosaic plot, stacked bar, heatmap	`Contract` vs `Churn`
Bagaimana tren dari waktu ke waktu?	Line chart	(tidak berlaku untuk Telco)
Bagaimana korelasi antar banyak variabel?	Correlation heatmap, scatter matrix	Korelasi antar numerik (Bab 3)

Pertanyaan	Chart yang tepat	Contoh pada dataset kita
Bagaimana komposisi (bagian dari keseluruhan)?	Stacked bar, treemap	Churn per `Contract` per `InternetService`
Bagaimana peringkat antar kategori?	Horizontal bar (sorted)	Churn rate per segmen

> **Tip**
>
> **Aturan praktis pemula.** Jika Anda ragu antara pie chart dan bar chart, pilih **bar chart**. Jika ragu antara line chart dan bar chart, tanyakan apakah x-axis adalah waktu—jika ya, **line**; jika tidak, **bar**. Jika ragu antara dua grafik apa pun, pilih yang paling sederhana dan beri label yang jelas.

4.5 Visualisasi Statis: `matplotlib` dan `seaborn`

Kita sudah memakai keduanya di Bab 3. Di sini kita akan memperkenalkan **pola yang konsisten** agar grafik Anda terlihat profesional tanpa perlu tweak berlebihan.

4.5.1 Setup Gaya Global

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from decanal.utils import load_churn_clean

# Gaya global untuk buku ini
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.dpi"] = 100
plt.rcParams["savefig.dpi"] = 150
plt.rcParams["font.size"] = 10

df = load_churn_clean()
```

> **Note**
>
> `sn.set_theme(style="whitegrid")` sudah menghapus sebagian besar chartjunk secara otomatis—gridlines ringan, latar belakang putih, dan font yang bersih. Ini adalah **default yang wajar** untuk sebagian besar kebutuhan. Jika Anda membutuhkan kontrol lebih (misalnya untuk publikasi), gunakan `style="ticks"` dan hapus spines secara manual.

4.5.2 Histogram dan Density

```
fig, ax = plt.subplots(figsize=(8, 3.5))
sns.histplot(data=df, x="tenure", kde=True, bins=30, color="#4C72B0", ax=ax)
ax.axvline(df["tenure"].median(), color="red", linestyle="--",
           label=f"Median = {df['tenure'].median():.0f} bulan")
ax.set_xlabel("Tenure (bulan)")
ax.set_ylabel("Jumlah pelanggan")
ax.set_title("Distribusi Tenure Pelanggan")
ax.legend()
plt.tight_layout()
plt.show()
```

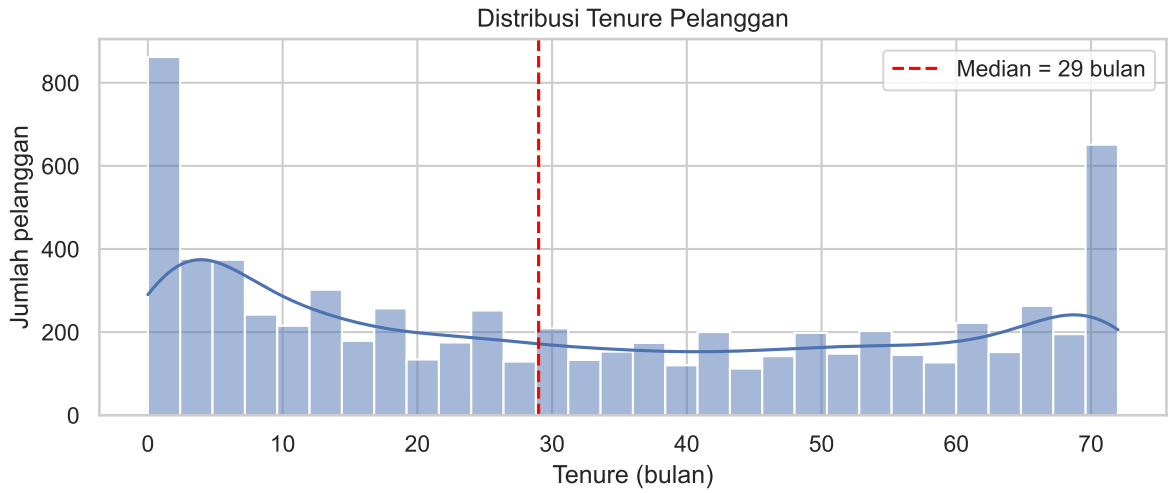


Figure 4.1: Distribusi tenure dengan density overlay. Garis putus-putus menunjukkan median.

Perhatikan distribusi **bimodal**: puncak di sekitar awal (pelanggan baru) dan puncak di akhir (~70 bulan, pelanggan loyal). Ini **tidak terlihat jelas** di tabel statistik Bab 3—dan ini adalah alasan mengapa visualisasi melengkapi statistik deskriptif, bukan menggantikannya.

4.5.3 Bar Chart yang Diurutkan

```
churn_rate = (df.groupby("Contract")["Churn"]
                .apply(lambda s: (s == "Yes").mean() * 100)
                .sort_values(ascending=True))

fig, ax = plt.subplots(figsize=(7, 3.5))
bars = ax.barh(churn_rate.index, churn_rate.values, color="#DD8452")
ax.bar_label(bars, fmt="%.1f%%", padding=3)
ax.set_xlabel("Churn rate (%)")
ax.set_ylabel("")
ax.set_title("Churn Rate per Tipe Kontrak")
ax.set_xlim(0, churn_rate.max() * 1.15)
plt.tight_layout()
plt.show()
```

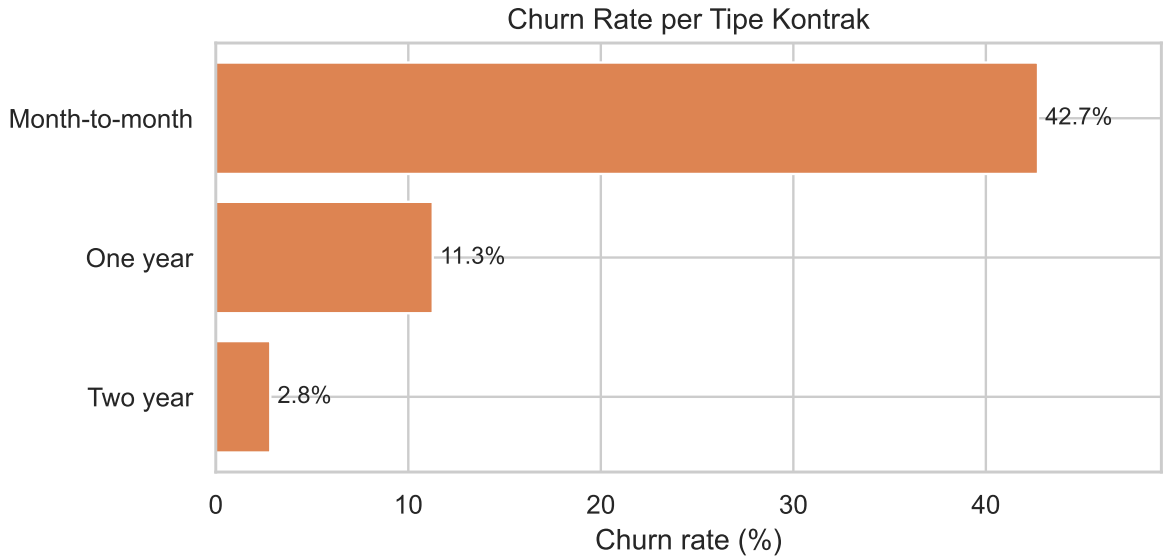


Figure 4.2: Churn rate per tipe kontrak, diurutkan dari yang tertinggi.

> **Tip**
>
> **Mengapa horizontal bar?** Untuk kategori dengan label panjang, horizontal bar menghindari rotasi label yang sulit dibaca. Dan dengan mengurutkan dari nilai terkecil ke terbesar, mata langsung tertuju pada batang terpanjang di atas. Ini adalah **pola standar** untuk "peringkat" yang sebaiknya selalu Anda gunakan.

4.5.4 Boxplot: Perbandingan Distribusi antar Kelompok

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(data=df, x="Contract", y="tenure", hue="Churn",
            palette={"No": "#4C72B0", "Yes": "#DD8452"}, ax=ax)
ax.set_xlabel("Tipe kontrak")
ax.set_ylabel("Tenure (bulan)")
ax.set_title("Tenure per Kontrak dan Status Churn")
plt.tight_layout()
plt.show()
```

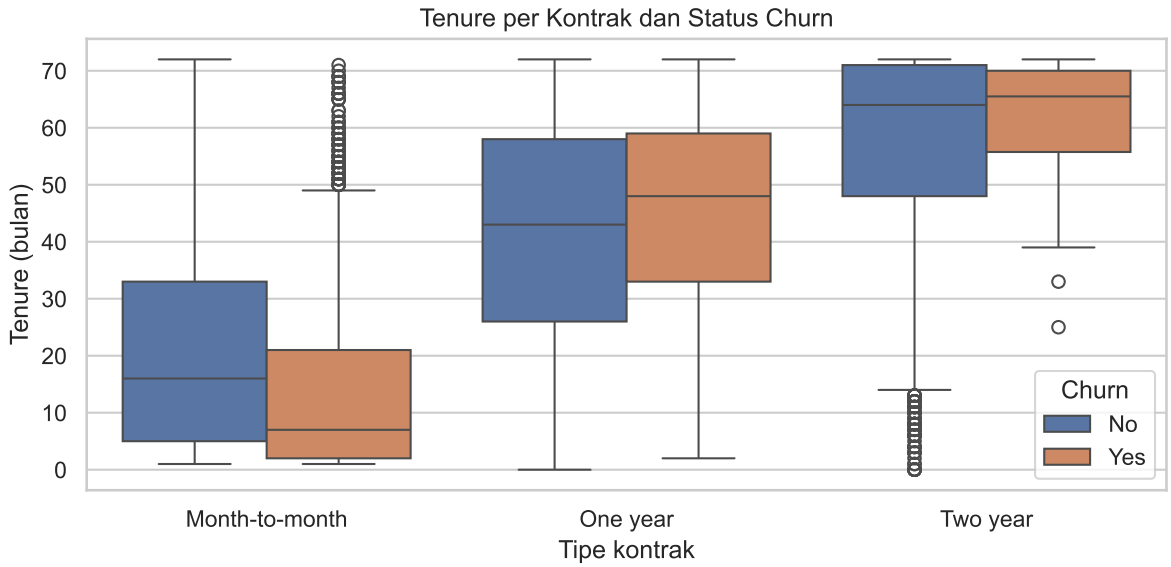


Figure 4.3: Distribusi tenure per tipe kontrak, dipisah berdasarkan status churn.

Boxplot menampilkan **lima angka penting** sekaligus: minimum, Q1, median, Q3, maksimum (di luar outlier). Ini adalah cara paling efisien untuk membandingkan distribusi antar kelompok—jauh lebih informatif daripada membandingkan rata-rata saja.

4.5.5 Small Multiples: Faceting

Ketika Anda ingin membandingkan pola yang sama di banyak kelompok, **facet** (subplot per kelompok) jauh lebih efektif daripada menumpuk semua garis di satu grafik.

```
g = sns.FacetGrid(df, col="Contract", hue="Churn", height=3, aspect=1.1,
                  palette={"No": "#4C72B0", "Yes": "#DD8452"})
g.map(sns.histplot, "tenure", bins=20, alpha=0.6)
g.add_legend(title="Churn")
g.set_axis_labels("Tenure (bulan)", "Jumlah")
g.figure.suptitle("Distribusi Tenure per Kontrak dan Churn", y=1.05)
plt.tight_layout()
plt.show()
```

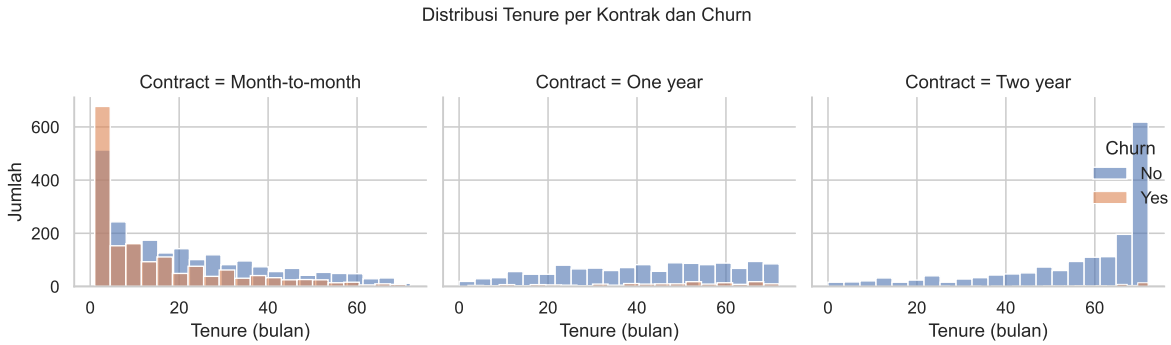


Figure 4.4: Distribusi tenure per tipe kontrak, dipecah per status churn.

4.6 Visualisasi Interaktif: `plotly`

Grafik statis baik untuk laporan dan publikasi. Tetapi untuk **eksplorasi** dan **dashboard**, interaktivitas memberi nilai tambah besar: hover untuk detail, zoom untuk fokus, filter untuk segmen.

`plotly` adalah library visualisasi interaktif paling populer di Python, dan hasilnya dapat disematkan langsung di Quarto HTML.

4.6.1 Plotly Express: API Tingkat Tinggi

```
import plotly.express as px

fig = px.scatter(
    df.sample(1000, random_state=42),  # sampel agar ringan
    x="tenure",
    y="MonthlyCharges",
```

```
    color="Churn",
    color_discrete_map={"No": "#4C72B0", "Yes": "#DD8452"},
    opacity=0.6,
    title="Tenure vs Monthly Charges (sampel 1.000 pelanggan)",
    labels={"tenure": "Tenure (bulan)", "MonthlyCharges": "Tagihan bulanan"},
)
fig.update_layout(width=750, height=420)
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

(a) Hubungan tenure vs MonthlyCharges, diwarnai berdasarkan status churn. Arahkan kursor untuk detail.

```
Unable to display output for mime type(s): text/html
```

(b)

Figure 4.5

> **Note**
>
> **Mengapa sampel 1.000 baris?** Plotly menyematkan data lengkap ke HTML, dan 7.043 titik scatter membuat file HTML besar dan browser lambat. Untuk eksplorasi, sampel representatif cukup. Untuk dashboard produksi, gunakan **WebGL** (`render_mode="webgl"`) yang jauh lebih cepat untuk data besar, atau agregasi data sebelum divisualisasikan.

4.6.2 Bar Chart Interaktif dengan Grouping

```
churn_seg = (df.groupby(["Contract", "InternetService"])["Churn"]
               .apply(lambda s: (s == "Yes").mean() * 100)
               .reset_index()
               .rename(columns={"Churn": "churn_rate"}))

fig = px.bar(
    churn_seg,
    x="Contract",
    y="churn_rate",
    color="InternetService",
    barmode="group",
```

```
    title="Churn Rate per Kontrak dan Layanan Internet",
    labels={"churn_rate": "Churn rate (%)", "Contract": "Tipe kontrak"},
    text_auto=".1f",
)
fig.update_layout(width=800, height=400, yaxis_range=[0, 70])
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

Figure 4.6: Churn rate per kombinasi kontrak dan layanan internet. Hover untuk detail.

Perhatikan pola: dalam **setiap** tipe layanan internet, churn rate pada kontrak bulanan jauh lebih tinggi daripada kontrak tahunan atau dua tahun. Ini menunjukkan bahwa **efek kontrak lebih kuat daripada efek layanan internet**—insight yang tidak langsung terlihat dari tabel tanpa memecah data dengan cara ini.

4.6.3 Tren dan Waktu (Pratinjau)

Karena Telco adalah data cross-sectional, tidak ada tren waktu. Namun untuk memberi Anda gambaran, berikut contoh bagaimana tren waktu akan divisualisasikan menggunakan dataset pendamping (Retail Store Inventory). Blok ini **tidak akan dijalankan** di Bab 4—ini hanya pratinjau untuk Bab 10.

```
# Pratinjau untuk Bab 10 - tidak dijalankan di sini
from decanal.utils import load_retail

retail = load_retail()
daily = retail.groupby("Date")["Units Sold"].sum().reset_index()

fig = px.line(daily, x="Date", y="Units Sold",
              title="Total Unit Terjual Harian (dataset retail)")
fig.show()
```

Figure 4.7

4.7 Kesalahan Visualisasi yang Harus Dihindari

Visualisasi yang buruk bukan hanya tidak informatif—ia bisa **menyesatkan secara aktif**. Berikut adalah kesalahan yang paling sering terjadi dan cara menghindarinya.

4.7.1 1. Sumbu Y yang Tidak Dimulai dari Nol

```
data = {"Kuartal": ["Q1", "Q2", "Q3", "Q4"], "Pendapatan": [100, 103, 106, 108]}
d = pd.DataFrame(data)

fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
axes[0].bar(d["Kuartal"], d["Pendapatan"], color="#DD8452")
axes[0].set_ylim(98, 110)
axes[0].set_title("Sumbu dipotong (menyesatkan)")
axes[0].set_ylabel("Pendapatan (juta)")

axes[1].bar(d["Kuartal"], d["Pendapatan"], color="#4C72B0")
axes[1].set_ylim(0, 110)
axes[1].set_title("Sumbu dari nol (jujur)")
axes[1].set_ylabel("Pendapatan (juta)")
plt.tight_layout()
plt.show()
```

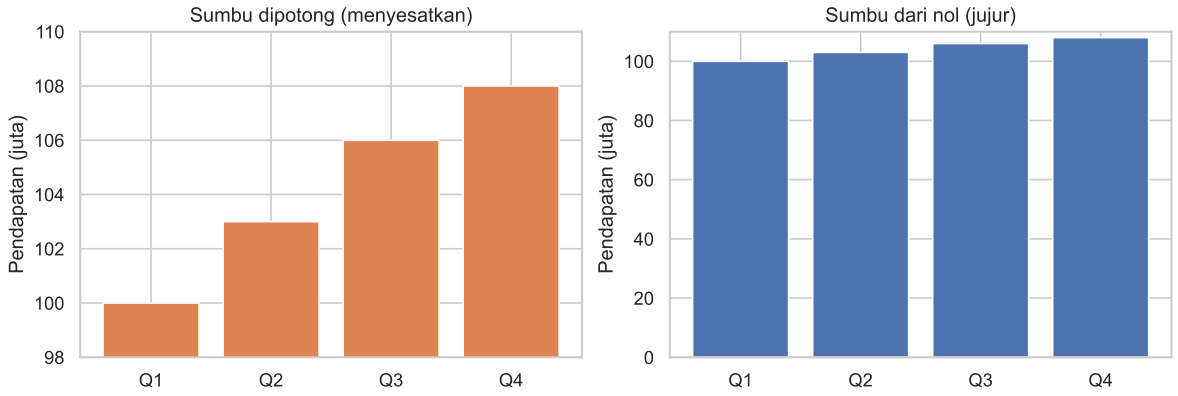


Figure 4.8: Efek sumbu Y yang dipotong. Grafik kiri menyesatkan; grafik kanan jujur.

Pada grafik kiri, pertumbuhan tampak **dramatis**—padahal dari Q1 ke Q4 hanya naik 8%. Memotong sumbu Y tidak selalu salah (kadang diperlukan untuk menonjolkan perbedaan kecil yang bermakna), tetapi harus disertai **catatan eksplisit** bahwa sumbu dipotong. Tanpa itu, ini adalah manipulasi visual.

4.7.2 2. Menggunakan Warna untuk Mengkodekan Terlalu Banyak Kategori

Mata manusia hanya bisa membedakan sekitar **7–8 warna** dengan andal. Jika kategori Anda lebih dari 7, gunakan strategi lain: **facet**, kelompokkan kategori kecil menjadi “other”, atau

gunakan **gradien** (jika kategori berurutan).

```
# Terlalu banyak kategori → sulit dibedakan
kategori = [f"Segmen {i}" for i in range(15)]
nilai = np.random.RandomState(42).randint(10, 100, 15)
fig, ax = plt.subplots(figsize=(8, 3.5))
ax.bar(kategori, nilai, color=plt.cm.tab20(np.linspace(0, 1, 15)))
ax.set_xticklabels(kategori, rotation=45, ha="right", fontsize=8)
ax.set_title("Kategori terlalu banyak - hindari ini")
plt.tight_layout()
plt.show()
```

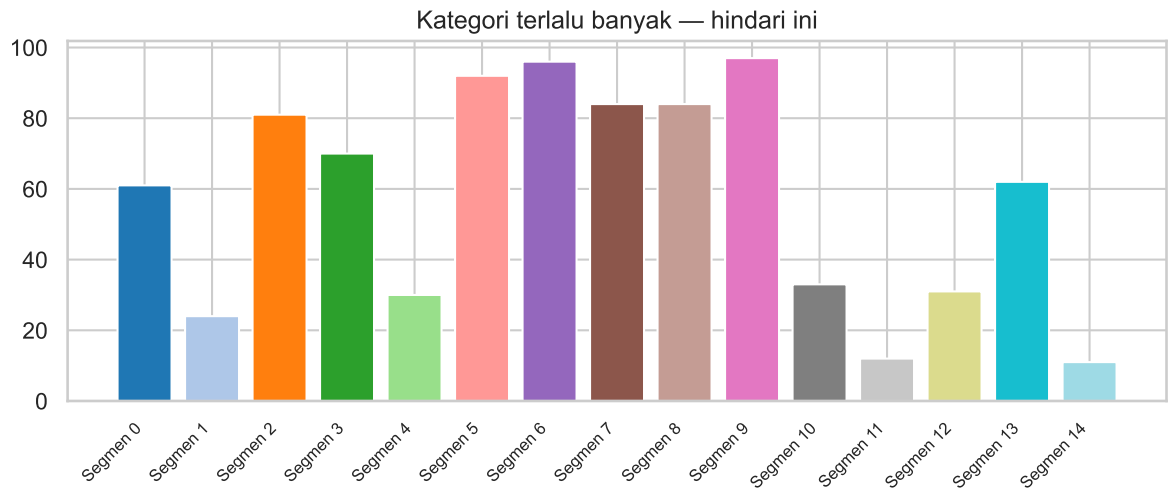


Figure 4.9: Terlalu banyak warna sulit dibedakan. Pertimbangkan faceting untuk kategori > 7.

4.7.3 3. Dual Y-Axis

Grafik dengan **dua sumbu Y** (satu kiri, satu kanan) hampir selalu menyesatkan—karena korelasi visual bergantung sepenuhnya pada skala yang Anda pilih. Korelasi yang tampak kuat dapat terlihat lemah hanya dengan mengubah skala sumbu kanan, dan sebaliknya. **Gunakan facet atau dua grafik terpisah.**

> ⚠ Warning
>
> **Aturan 3 dari 3.** Tiga kesalahan yang harus selalu Anda tanyakan sebelum mempublikasikan grafik: 1. Apakah sumbu Y dimulai dari nol (atau jujur jika tidak)? 2. Apakah ada lebih dari 7 warna kategori yang harus dibedakan? 3. Apakah ada dua sumbu Y?

> Jika ada satu saja "ya" yang bermasalah, revisi.

4.8 Membangun Dashboard: Prinsip dan Praktik

Dashboard adalah kumpulan visualisasi yang dirancang untuk menjawab **satu set pertanyaan** secara bersamaan. Ia bukan sekadar "banyak grafik di satu halaman"—ia adalah **produk** dengan audiens, tujuan, dan hierarki informasi yang jelas.

4.8.1 Prinsip Dashboard yang Baik

Prinsip	Penjelasan
Tujuan tunggal	Satu dashboard = satu pertanyaan utama. Dashboard "semua tentang bisnis" tidak berguna.
Hierarki visual	KPI utama paling besar dan paling atas; detail kecil di bawah.
Konsistensi warna	Satu palet warna di seluruh dashboard, satu warna per kategori.
Konteks	Setiap angka perlu pembanding—target, periode sebelumnya, rata-rata.
Ruang kosong	Ruang kosong bukan pemborosan—ia memisahkan dan menekankan.

4.8.2 Contoh Dashboard Sederhana dengan Plotly

Berikut dashboard dua-panel yang menjawab satu pertanyaan manajerial: *"Di mana churn paling terkonsentrasi, dan seberapa besar dampaknya?"*

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

# Data 1: Churn rate per kontrak
cr = (df.groupby("Contract")["Churn"]
        .apply(lambda s: (s == "Yes").mean() * 100)
        .sort_values())

# Data 2: Rata-rata tagihan bulanan per kontrak dan churn
```

```
charges = (df.groupby(["Contract", "Churn"])["MonthlyCharges"]
             .mean()
             .reset_index())

# Dashboard
fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=("Churn rate per kontrak (%)",
                    "Rata-rata tagihan bulanan per kontrak"),
    column_widths=[0.5, 0.5],
)

fig.add_trace(
    go.Bar(x=cr.values, y=cr.index, orientation="h",
           marker_color="#DD8452", text=cr.round(1), textposition="outside",
           name="Churn rate"),
    row=1, col=1,
)

for status, color in [("No", "#4C72B0"), ("Yes", "#DD8452")]:
    subset = charges[charges["Churn"] == status]
    fig.add_trace(
        go.Bar(x=subset["Contract"], y=subset["MonthlyCharges"],
               name=f"Churn = {status}", marker_color=color),
        row=1, col=2,
    )

fig.update_layout(
    height=420, width=1000,
    title_text="Dashboard Churn - Telco Customer Churn",
    barmode="group",
    legend_title="Status",
    margin=dict(l=150, r=20, t=60, b=60),
)
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

Figure 4.10: Dashboard churn: (kiri) churn rate per kontrak; (kanan) rata-rata tagihan bulanan per kontrak dan status churn.

> **Note**
>
> **Mengapa dashboard ini "benar"?** Ia menjawab **satu pertanyaan** (di mana churn paling terkonsentrasi dan berapa harganya?), memiliki **dua panel yang saling melengkapi** (bukan dua grafik acak), menggunakan **warna konsisten** (oranye = churn, biru = non-churn), dan menyertakan **label langsung** (angka di batang) yang mengurangi kebutuhan akan legend dan sumbu terpisah.

4.8.3 Dashboard Interaktif dengan Widget

Untuk dashboard yang lebih lengkap, Plotly dapat dipadukan dengan `ipywidgets` (di Jupyter) atau disajikan sebagai aplikasi **Dash** (di luar cakupan buku ini). Untuk keperluan pengajaran, template di atas sudah memadai. Jika Anda menginginkan dashboard yang dapat difilter, Quarto juga mendukung **Observable JS** untuk dashboard JavaScript-native.

4.9 Menyimpan Grafik untuk Buku

Ketika Anda menyimpan grafik untuk Quarto book (HTML/PDF), ada beberapa pertimbangan:

```
# Simpan grafik ke file (mis. untuk dipakai di luar Quarto)
fig_static, ax = plt.subplots(figsize=(8, 4))
sns.histplot(data=df, x="tenure", bins=30, ax=ax)
fig_static.savefig("images/tenure-distribution.png", dpi=150, bbox_inches="tight")
```

Untuk Quarto, **tidak perlu** menyimpan grafik ke file jika grafik dibuat di dalam chunk—Quarto akan menyematkannya langsung. Simpan ke file hanya jika Anda ingin memakainya di **slide** atau **bab lain**.

4.10 Latihan Terbimbing

Latihan 4.1 — Chart selection. Untuk masing-masing pertanyaan berikut, pilih satu jenis chart yang paling tepat, dan jelaskan pilihan Anda dalam satu kalimat: - (a) Bagaimana distribusi `MonthlyCharges` di antara pelanggan yang churn vs. bertahan? - (b) Berapa proporsi pelanggan yang churn di setiap kategori `PaymentMethod`? - (c) Apakah ada hubungan antara `tenure` dan `MonthlyCharges`? - (d) Bagaimana komposisi `Churn` di setiap `InternetService`?

Latihan 4.2 — Perbaiki grafik yang menyesatkan. Kode berikut membuat grafik yang **menyesatkan**. Perbaiki agar jujur secara visual:

```
fig, ax = plt.subplots()
ax.bar(["A", "B", "C"], [98, 99, 100])
ax.set_ylim(95, 101)
```

Jelaskan apa yang salah dan bagaimana Anda memperbaikinya.

Latihan 4.3 — Dashboard satu pertanyaan. Buat dashboard dua-panel untuk menjawab pertanyaan: *"Apakah pelanggan dengan lebih banyak layanan cenderung churn lebih jarang?"* Gunakan kolom `num_services` yang dibuat di Bab 2. Dashboard harus memuat: (a) churn rate per `num_services`, dan (b) jumlah pelanggan per `num_services` (untuk memastikan churn rate tidak didasarkan pada sampel kecil).

Latihan 4.4 — Plotly interaktif. Buat scatter plot interaktif dengan Plotly yang menampilkan `tenure` vs `TotalCharges`, diwarnai berdasarkan `Contract`, dengan ukuran titik sebanding dengan `MonthlyCharges`. Tambahkan judul dan label sumbu yang jelas.

Latihan 4.5 — Refleksi kritis. Cari satu grafik di berita atau laporan perusahaan (atau dari internet) yang menurut Anda **menyesatkan**. Identifikasi setidaknya satu prinsip visualisasi yang dilanggar dan jelaskan mengapa pelanggaran itu penting.

> **Note**
>
> **Petunjuk Latihan 4.2.** Grafik tersebut memotong sumbu Y dari 95 hingga 101, membuat perbedaan 2% terlihat seperti perbedaan 100%. Perbaikan: (a) mulai sumbu Y dari 0, atau (b) jika memotong sumbu Y memang diperlukan untuk menonjolkan perbedaan kecil, **beri anotasi eksplisit** di grafik bahwa sumbu dipotong. Pilihan pertama lebih baik untuk audiens umum; pilihan kedua dapat diterima untuk audiens teknis yang memahami konteks.

4.11 Kotak "Jembatan": Dari Data ke Keputusan

> **Important**
>
> **Jembatan.** Visualisasi mengubah cara keputusan dibuat:
>
> 1. **Mempercepat pemahaman.** Di Bab 3, kita butuh tabel untuk melihat bahwa churn rate berbeda antar kontrak. Di Bab 4, satu bar chart menyampaikan insight yang sama dalam **kurang dari 2 detik**. Dalam rapat manajemen, kecepatan pemahaman adalah segalanya.

2. **Mengungkap pola tersembunyi.** Distribusi `tenure` yang bimodal tidak terlihat dari statistik deskriptif—tetapi jelas dalam histogram. Pola ini memberi tahu kita bahwa ada **dua populasi pelanggan** dengan perilaku berbeda: pelanggan baru (rentan churn) dan pelanggan lama (loyal). Keputusan: strategi retensi untuk kedua populasi harus **berbeda**.

3. **Mengomunikasikan ketidakpastian.** Grafik yang jujur—dengan sumbu Y dari nol, tanpa dual axis, dengan ukuran sampel yang ditampilkan—membangun kepercayaan. Grafik yang menyesatkan, bahkan jika tidak disengaja, merusak kredibilitas analisis.

4. **Memungkinkan eksplorasi mandiri.** Dashboard interaktif memungkinkan **pengambil keputusan** mengeksplorasi data sendiri—mengubah filter, memperbesar segmen, memeriksa detail. Ini memberdayakan mereka untuk mengajukan pertanyaan yang tidak Anda antisipasi, dan sering menghasilkan insight yang tidak Anda temukan dalam analisis awal.

4.12 Rangkuman

Dalam bab ini kita telah:

- **Memahami grammar of graphics** sebagai kerangka universal untuk berpikir tentang visualisasi: data, aesthetic, geometric, facet, transformation.
- **Menerapkan prinsip Tufte**—data-ink ratio, penghindaran chartjunk—untuk menghasilkan grafik yang bersih dan efektif.
- **Memilih jenis chart yang tepat** berdasarkan jenis pertanyaan analitis (distribusi, hubungan, tren, komposisi, peringkat).
- **Menguasai `matplotlib` dan `seaborn`** untuk visualisasi statis siap publikasi: histogram, bar chart, boxplot, dan small multiples.
- **Menguasai `plotly`** untuk visualisasi interaktif yang dapat disematkan langsung di Quarto HTML.
- **Mengenali dan menghindari kesalahan visualisasi** yang paling umum: sumbu Y yang dipotong, terlalu banyak warna kategori, dual axis.
- **Merancang dashboard** yang menjawab satu pertanyaan manajerial dengan hierarki visual dan konsistensi warna.
- **Mengintegrasikan visualisasi ke dalam alur keputusan**, menyadari bahwa grafik yang baik mempercepat pemahaman dan membangun kepercayaan.

4.13 Jembatan ke Bab 5

Sampai di sini, kita telah **mengukur** (Bab 3) dan **melihat** (Bab 4). Tetapi kita belum benar-benar **mendiagnosis**—belum mengejar *mengapa* pola yang kita amati muncul. Bab berikutnya—**Diagnostic Analytics**—akan membawa kita lebih dalam:

- Mengapa pelanggan kontrak bulanan lebih sering churn?
- Apakah karena mereka lebih muda, atau lebih baru, atau karena alasan lain?
- Bagaimana kita memisahkan **efek** dari sekadar **korelasi**?
- Alat diagnostik apa yang dapat membantu kita menemukan akar masalah?

Bab 5 akan memperkenalkan teknik seperti **analisis segmentasi**, **drill-down**, **analisis akar masalah**, dan **visual analytics untuk diagnosis**—semuanya dibangun di atas fondasi statistik dan visualisasi dari dua bab sebelumnya.

5 Diagnostic Analytics

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membedakan pertanyaan deskriptif ("apa") dari pertanyaan diagnostik ("mengapa"). 2. Menerapkan stratifikasi dan *drill-down* untuk mengurai pola yang tampak sederhana. 3. Mengenali **Simpson's Paradox** dan memahami mengapa agregasi bisa menyesatkan. 4. Menggunakan analisis kohort untuk memahami bagaimana perilaku berubah sepanjang siklus hidup pelanggan. 5. Menerapkan kerangka *root cause analysis* (5 Whys, Ishikawa) pada data. 6. Memisahkan korelasi dari kausalitas secara eksplisit—dan tahu kapan diagnosis berhenti menjadi bukti.

5.1 Dari "Apa" ke "Mengapa"

Di Bab 3 kita menemukan bahwa pelanggan kontrak bulanan churn sekitar 43%, sementara kontrak dua tahun hanya sekitar 3%. Di Bab 4 kita melihatnya dalam bar chart dan langsung menangkap besarnya perbedaan.

Tetapi **mengapa**? Apakah karena kontrak bulanan *menyebabkan* churn? Atau karena pelanggan yang *berniat* churn memang memilih kontrak bulanan? Atau karena ada faktor ketiga—usia, tenure, jenis layanan—yang menjelaskan keduanya?

Pertanyaan-pertanyaan inilah yang menjadi inti **diagnostic analytics**. Ia tidak berhenti pada pola; ia mengejar **mekanisme** di balik pola.

Important

Perbedaan mendasar.

- **Descriptive**: *"Churn rate kontrak bulanan 43%."* — Fakta.
- **Diagnostic**: *"Churn rate kontrak bulanan tinggi **karena** pelanggan baru cenderung memilih kontrak bulanan, dan pelanggan baru memiliki churn rate tinggi terlepas dari kontrak."* — Mekanisme.

Yang kedua jauh lebih berguna untuk keputusan, karena ia menunjukkan **di mana intervensi harus diarahkan**.

5.2 Pola Pikir Diagnostik

Seorang analis diagnostik yang baik mengajukan pertanyaan berlapis:

1. **Apa yang kita lihat?** (deskriptif)
2. **Apakah pola ini nyata, atau artefak dari cara kita mengagregasi data?** (stratifikasi)
3. **Variabel mana yang paling menjelaskan pola ini?** (atribusi)
4. **Apakah hubungan ini kausal, atau ada penjelasan lain?** (kausalitas)
5. **Jika pola ini benar, apa implikasinya untuk keputusan?** (aksi)

Bab ini akan memandu Anda melalui kelima lapisan ini menggunakan dataset Telco.

5.3 Stratifikasi: Mengurai Pola yang Tampak Sederhana

Stratifikasi adalah teknik paling dasar dalam diagnostic analytics: memecah data menjadi subkelompok berdasarkan satu atau lebih variabel, lalu memeriksa apakah pola yang kita amati bertahan di setiap subkelompok.

5.3.1 Contoh Pertama: Apakah Efek Kontrak Bertahan di Semua Tingkat Tenure?

Di Bab 3 kita melihat bahwa kontrak bulanan memiliki churn rate jauh lebih tinggi. Tetapi pelanggan kontrak bulanan juga cenderung **baru** (tenure pendek). Apakah efek kontrak bertahan ketika kita mengontrol tenure?

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from decanal.utils import load_churn_clean

df = load_churn_clean()

# Buat kelompok tenure
df["tenure_bin"] = pd.cut(
    df["tenure"], bins=[-1, 6, 12, 24, 48, np.inf],
    labels=["0-6", "7-12", "13-24", "25-48", ">48"]
)
```

```
# Churn rate per (tenure_bin, Contract)
strat = (df.groupby(["tenure_bin", "Contract"], observed=True)["Churn"]
           .apply(lambda s: (s == "Yes").mean() * 100)
           .round(1)
           .unstack())

strat
```

Contract tenure_bin	Month-to-month	One year	Two year
0-6	55.2	10.3	0.0
7-12	42.0	10.6	0.0
13-24	37.7	8.1	0.0
25-48	32.9	10.6	2.2
>48	26.0	12.9	3.3

Cara membaca. Setiap baris adalah kelompok tenure. Setiap kolom adalah tipe kontrak. Angka di dalam sel adalah churn rate (%) untuk kombinasi tersebut.

Jika efek kontrak **bertahan di setiap tingkat tenure**, kita akan melihat pola yang konsisten: kolom `Month-to-month` selalu lebih tinggi dari `One year`, yang selalu lebih tinggi dari `Two year`. Jika efeknya **hilang** di beberapa tingkat, itu tanda bahwa efek kontrak sebenarnya **dijelaskan oleh tenure**.

```
fig, ax = plt.subplots(figsize=(8, 4))
strat.plot(kind="bar", ax=ax, color=["#4C72B0", "#55A868", "#DD8452"])
ax.set_xlabel("Kelompok tenure (bulan)")
ax.set_ylabel("Churn rate (%)")
ax.set_title("Churn Rate per Kontrak di Setiap Kelompok Tenure")
ax.legend(title="Kontrak", bbox_to_anchor=(1.02, 1), loc="upper left")
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
plt.tight_layout()
plt.show()
```

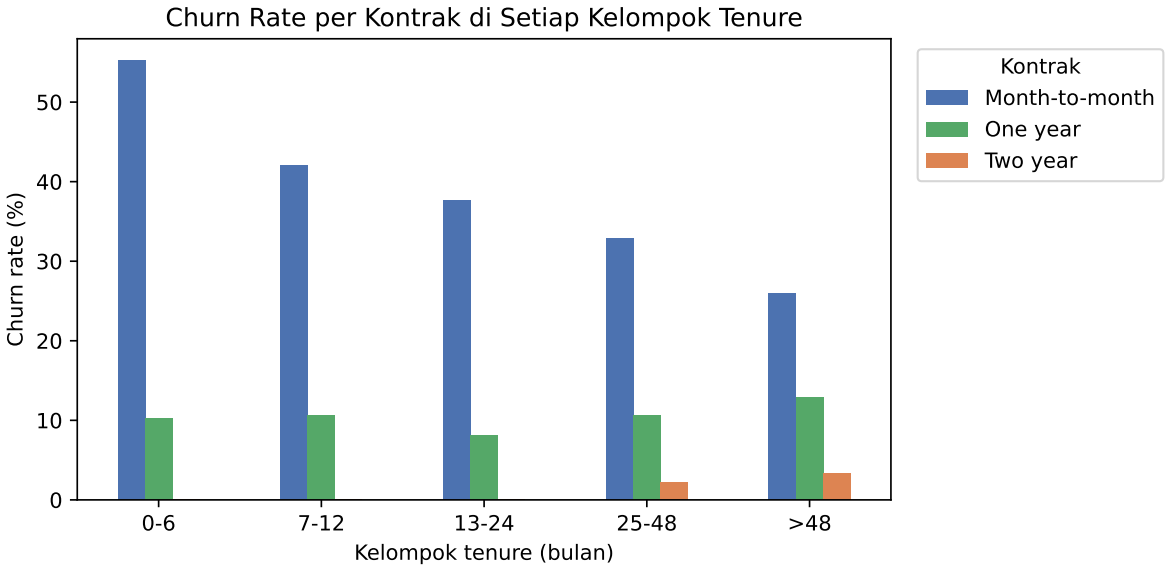


Figure 5.1: Churn rate per kontrak, dipisah per kelompok tenure. Pola konsisten di semua kelompok.

Temuan. Efek kontrak **bertahan di semua kelompok tenure**—bahkan di antara pelanggan yang baru bergabung (0–6 bulan), kontrak bulanan memiliki churn rate yang jauh lebih tinggi. Ini menunjukkan bahwa efek kontrak **bukan artefak dari tenure**. Kontrak itu sendiri (atau faktor yang berkorelasi dengannya) memang berperan.

> Tip
>
> **Prinsip stratifikasi.** Ketika Anda mencurigai bahwa pola X→Y sebenarnya dijelaskan oleh variabel Z, **stratifikasi berdasarkan Z**. Jika pola X→Y bertahan di setiap tingkat Z, Z bukan penjelas utama. Jika pola X→Y hilang setelah stratifikasi, Z-lah yang sebenarnya berperan.

5.4 Simpson's Paradox: Ketika Agregasi Menyesatkan

Simpson's Paradox adalah fenomena di mana sebuah tren muncul dalam data agregat, tetapi **terbalik** ketika data dipecah menjadi subkelompok. Ini adalah salah satu jebakan paling berbahaya dalam analisis data—dan salah satu alasan paling kuat mengapa stratifikasi bukan opsional.

5.4.1 Contoh Klasik

Bayangkan dua rumah sakit. Rumah sakit A memiliki tingkat kematian keseluruhan 10%, rumah sakit B 15%. Sekilas, A tampak lebih baik. Tetapi ketika kita memecah pasien berdasarkan tingkat keparahan:

	Pasien ringan	Pasien berat
RS A	100 pasien, 5 meninggal (5%)	900 pasien, 95 meninggal (10,6%)
RS B	800 pasien, 80 meninggal (10%)	200 pasien, 22 meninggal (11%)

Di **setiap** tingkat keparahan, RS A memiliki tingkat kematian lebih rendah. Tetapi secara agregat, RS A tampak lebih buruk—karena RS A menerima **lebih banyak pasien berat**. Ini adalah Simpson's Paradox: tren agregat menyesatkan karena distribusi subkelompok tidak seimbang.

5.4.2 Mencari Simpson's Paradox di Dataset Telco

Mari kita cari apakah ada Simpson's Paradox di dataset kita. Kita akan memeriksa hubungan antara `InternetService` dan `Churn`, dengan `Contract` sebagai variabel stratifikasi.

```
# Agregat: churn rate per InternetService
agregat = (df.groupby("InternetService")["Churn"]
             .apply(lambda s: (s == "Yes").mean() * 100)
             .round(1))
print("--- Agregat ---")
print(agregat)

# Stratifikasi: churn rate per (InternetService, Contract)
strat_internet = (df.groupby(["InternetService", "Contract"], observed=True)["Churn"]
                    .apply(lambda s: (s == "Yes").mean() * 100)
                    .round(1)
                    .unstack())
print("\n--- Stratifikasi per Contract ---")
print(strat_internet)
```

```
--- Agregat ---
InternetService
DSL            19.0
```

```
Fiber optic    41.9
No              7.4
Name: Churn, dtype: float64

--- Stratifikasi per Contract ---
Contract         Month-to-month  One year  Two year
InternetService
DSL                        32.2       9.3       1.9
Fiber optic                54.6      19.3       7.2
No                         18.9       2.5       0.8
```

Perhatikan: secara agregat, `Fiber optic` memiliki churn rate tertinggi. Tetapi ketika kita stratifikasi berdasarkan `Contract`, apakah urutan ini bertahan? Jika ya, efek `InternetService` nyata. Jika tidak—jika dalam setiap tipe kontrak urutannya berubah—kita mungkin sedang melihat Simpson's Paradox.

> **Warning**
>
> **Mengapa ini penting?** Keputusan berdasarkan data agregat bisa **sepenuhnya salah arah**. Jika kita melihat "fiber optic memiliki churn tertinggi" dan memutuskan untuk menghentikan layanan fiber optic, kita mungkin menghancurkan bisnis—padahal masalah sebenarnya adalah **komposisi pelanggan** (pelanggan fiber optic lebih sering memilih kontrak bulanan), bukan layanan itu sendiri.
> Selalu stratifikasi ketika variabel stratifikasi (di sini: `Contract`) **berkorelasi dengan variabel penjelas** (`InternetService`) **dan** dengan target (`Churn`).

5.5 Drill-Down: Menelusuri Ke Dalam

Drill-down adalah teknik memperdalam analisis dengan menambahkan dimensi. Jika agregat memberi Anda gambaran besar, drill-down memberi Anda detail yang menjelaskan gambaran itu.

5.5.1 Contoh: Churn Rate per Segmen Bertingkat

```
# Level 1: Contract
level1 = (df.groupby("Contract")["Churn"]
            .apply(lambda s: (s == "Yes").mean() * 100)
            .round(1))
print("--- Level 1: Contract ---")
```

```
print(level1)

# Level 2: Contract × InternetService
level2 = (df.groupby(["Contract", "InternetService"], observed=True)["Churn"]
            .apply(lambda s: (s == "Yes").mean() * 100)
            .round(1)
            .unstack())
print("\n--- Level 2: Contract × InternetService ---")
print(level2)

# Level 3: Contract × InternetService × Partner
level3 = (df.groupby(["Contract", "InternetService", "Partner"], observed=True)["Churn"]
            .apply(lambda s: (s == "Yes").mean() * 100)
            .round(1)
            .unstack())
print("\n--- Level 3: Contract × InternetService × Partner ---")
print(level3.head(12))
```

```
--- Level 1: Contract ---
Contract
Month-to-month    42.7
One year          11.3
Two year           2.8
Name: Churn, dtype: float64

--- Level 2: Contract × InternetService ---
InternetService   DSL  Fiber optic    No
Contract
Month-to-month   32.2         54.6  18.9
One year          9.3         19.3   2.5
Two year          1.9          7.2   0.8

--- Level 3: Contract × InternetService × Partner ---
Partner                          No   Yes
Contract       InternetService
Month-to-month DSL              35.9  24.6
               Fiber optic      57.8  49.8
               No               19.7  16.7
One year       DSL               9.3   9.3
               Fiber optic      19.7  19.1
               No                2.7   2.2
Two year       DSL               1.2   2.2
```

```
Fiber optic      10.5   6.0
No                1.4   0.5
```

Cara membaca drill-down. Di setiap level, kita menambahkan satu dimensi. Jika churn rate berubah drastis antar subkelompok, kita menemukan **segmen yang sangat berbeda**—dan segmen itulah yang paling membutuhkan perhatian.

> **Note**
>
> **Hati-hati dengan sampel kecil.** Semakin dalam drill-down, semakin kecil ukuran sampel per sel. Churn rate 50% dari 10 pelanggan jauh kurang dapat diandalkan daripada 50% dari 1.000 pelanggan. **Selalu periksa ukuran sampel** saat melakukan drill-down—ini adalah alasan kita menambahkan `count` di dashboard Bab 4.

5.6 Analisis Kohort: Bagaimana Perilaku Berubah Sepanjang Waktu

Kohort adalah kelompok pelanggan yang berbagi karakteristik umum—biasanya **waktu bergabung**. Analisis kohort menjawab pertanyaan: *"Bagaimana perilaku pelanggan berubah seiring bertambahnya usia mereka dalam layanan?"*

Untuk dataset Telco, kita tidak memiliki tanggal bergabung yang eksplisit, tetapi kita memiliki `tenure` (bulan sejak bergabung). Kita dapat membentuk kohort berdasarkan `tenure` dan melihat bagaimana churn rate berubah.

5.6.1 Churn Rate per Kelompok Tenure

```
# Churn rate per kelompok tenure (kohort berdasarkan lama berlangganan)
kohort = (df.groupby("tenure_bin", observed=True)
            .agg(
                jumlah=("Churn", "size"),
                churn_rate=("Churn", lambda s: (s == "Yes").mean() * 100)
            )
            .round(1))
kohort
```

tenure_bin	jumlah	churn_rate
0-6	1481	52.9

tenure_bin	jumlah	churn_rate
7-12	705	35.9
13-24	1024	28.7
25-48	1594	20.4
>48	2239	9.5

```
fig, ax1 = plt.subplots(figsize=(8, 4))

x = np.arange(len(kohort))
bars = ax1.bar(x, kohort["churn_rate"], color="#DD8452", alpha=0.8)
ax1.bar_label(bars, fmt="%.1f%%", padding=3)
ax1.set_xticks(x)
ax1.set_xticklabels(kohort.index, rotation=0)
ax1.set_xlabel("Kelompok tenure (bulan)")
ax1.set_ylabel("Churn rate (%)", color="#DD8452")
ax1.set_title("Churn Rate per Kelompok Tenure")

# Sumbu kedua untuk jumlah pelanggan
ax2 = ax1.twinx()
ax2.plot(x, kohort["jumlah"], color="#4C72B0", marker="o", linewidth=2)
ax2.set_ylabel("Jumlah pelanggan", color="#4C72B0")

plt.tight_layout()
plt.show()
```

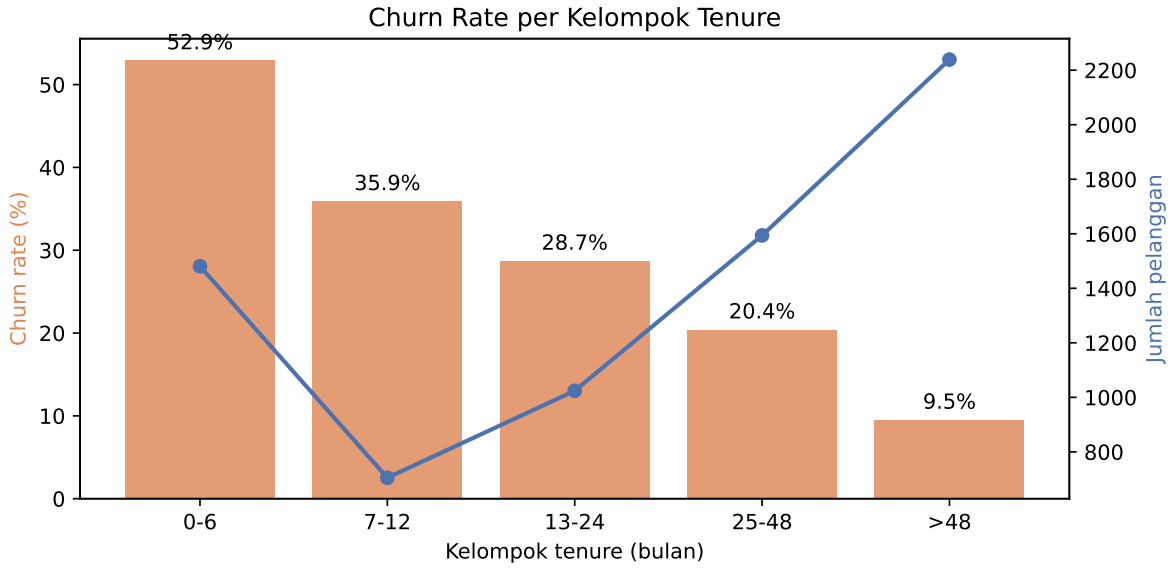


Figure 5.2: Churn rate per kelompok tenure. Churn tertinggi pada pelanggan baru; menurun seiring tenure.

> **Warning**
>
> **Kesalahan umum: dual axis.** Grafik di atas menggunakan dual axis (churn rate di kiri, jumlah di kanan). Seperti yang diperingatkan di Bab 4, dual axis bisa menyesatkan—tetapi di sini saya menggunakannya **secara sadar** untuk menunjukkan bahwa churn rate tinggi di kelompok tenure pendek **bertepatan** dengan jumlah pelanggan yang juga besar. Insight ini penting: masalah churn paling akut bukan hanya karena rate-nya tinggi, tetapi karena **volume pelanggan baru juga besar**. Jika Anda mempublikasikan grafik ini, beri anotasi eksplisit bahwa dua sumbu memiliki skala berbeda.

Temuan. Churn rate **tertinggi pada pelanggan baru** (0–6 bulan: ~53%) dan **menurun drastis** seiring tenure. Pelanggan dengan tenure > 48 bulan memiliki churn rate sangat rendah (~7%). Ini menunjukkan bahwa **risiko churn paling besar ada di awal siklus hidup pelanggan**—dan intervensi retensi paling efektif jika dilakukan **segera setelah onboarding**.

5.6.2 Kurva "Survival" Sederhana

Kita dapat memvisualisasikan ini sebagai **kurva survival**: berapa proporsi pelanggan yang bertahan setelah N bulan?

```python
# Hitung proporsi pelanggan dengan tenure >= t
tenure_range = np.arange(0, 73, 1)
survival = [(df["tenure"] >= t).mean() * 100 for t in tenure_range]

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(tenure_range, survival, color="#4C72B0", linewidth=2)
ax.fill_between(tenure_range, survival, alpha=0.2, color="#4C72B0")
ax.set_xlabel("Tenure (bulan)")
ax.set_ylabel("Proporsi pelanggan bertahan (%)")
ax.set_title("Kurva Survival Pelanggan")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

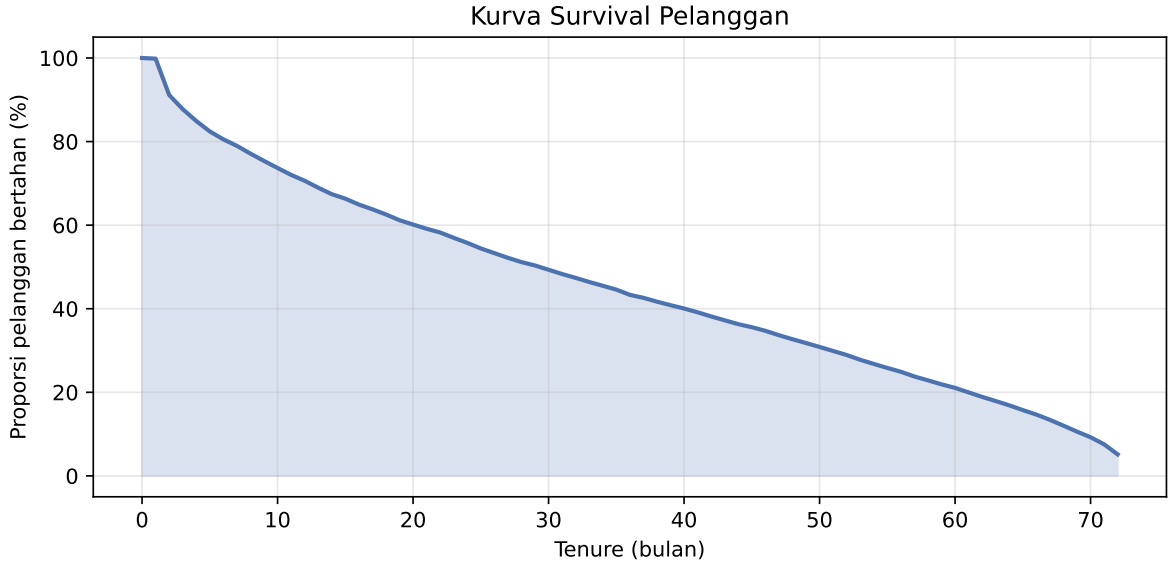


Figure 5.3: Kurva survival sederhana: proporsi pelanggan yang bertahan hingga tenure tertentu.

Kurva ini menunjukkan **penurunan tajam** di awal (bulan 0–12) dan **melandai** setelahnya. Ini adalah pola khas **"churn dini"**—sebagian besar pelanggan yang akan churn melakukannya di tahun pertama. Implikasi untuk keputusan: **program retensi harus fokus pada 12 bulan pertama**, bukan tersebar merata sepanjang siklus hidup pelanggan.

> **Note**
>
> Analisis survival formal—dengan *hazard function* dan model seperti Cox Proportional Hazards—akan dibahas di Bab 10 sebagai jembatan ke time series. Di sini kita hanya menggunakan versi deskriptif sederhana untuk membangun intuisi.

5.7 Root Cause Analysis: Kerangka Sistematis

Stratifikasi dan drill-down adalah **teknik**. Tetapi diagnosis yang baik membutuhkan **kerangka berpikir**. Dua kerangka yang paling banyak digunakan:

5.7.1 5 Whys

Teknik sederhana namun kuat: tanyakan **"mengapa"** lima kali berturut-turut, masing-masing jawaban menjadi dasar pertanyaan berikutnya.

Contoh pada dataset Telco:

1. **Mengapa churn rate tinggi?** → Karena pelanggan kontrak bulanan churn sangat sering.
2. **Mengapa pelanggan kontrak bulanan churn sangat sering?** → Karena mereka merasa tidak terikat dan mudah pindah.
3. **Mengapa mereka merasa tidak terikat?** → Karena mereka belum menggunakan banyak layanan tambahan (security, backup, tech support).
4. **Mengapa mereka belum menggunakan banyak layanan tambahan?** → Karena layanan tambahan ditawarkan sebagai add-on berbayar, dan pelanggan baru enggan menambah biaya.
5. **Mengapa pelanggan baru enggan menambah biaya?** → Karena mereka belum merasakan nilai layanan inti—dan belum ada program onboarding yang membangun keterikatan.

Akar masalah yang ditemukan: bukan kontrak bulanan itu sendiri, melainkan **kurangnya keterikatan pelanggan baru melalui layanan tambahan**. Ini mengarahkan intervensi ke **program onboarding dan bundling layanan**, bukan sekadar memaksa pelanggan pindah ke kontrak tahunan.

> **Tip**
>
> **5 Whys bukan alat statistik.** Ia adalah alat **berpikir**. Kekuatannya terletak pada memaksa Anda bergerak dari gejala ke akar. Namun ia juga punya kelemahan: ia bisa berhenti di jawaban yang salah jika asumsi Anda keliru. Karena itu, **validasi setiap "mengapa" dengan data**—jangan hanya mengandalkan intuisi.

5.7.2 Diagram Ishikawa (Fishbone)

Kerangka kedua memetakan **penyebab potensial** ke dalam kategori-kategori. Untuk churn, kategori yang masuk akal adalah:

- **Produk**: kualitas layanan, harga, fitur
- **Pelanggan**: demografi, kebutuhan, ekspektasi
- **Proses**: onboarding, dukungan pelanggan, penagihan
- **Lingkungan**: kompetitor, kondisi pasar

Setiap kategori diisi dengan hipotesis yang kemudian **diuji dengan data**. Diagram ini berguna untuk **memastikan Anda tidak melewatkan kategori penyebab**—dan untuk berkomunikasi dengan pemangku kepentingan non-teknis.

5.8 Visual Analytics untuk Diagnosis

Diagnosis yang baik membutuhkan visualisasi yang **mengungkap**, bukan sekadar **menampilkan**. Tiga jenis grafik sangat berguna untuk diagnosis:

5.8.1 1. Heatmap Stratifikasi

Heatmap memungkinkan Anda melihat pola di **banyak subkelompok sekaligus**—jauh lebih efisien daripada tabel.

```
heat_data = (df.groupby(["Contract", "InternetService"], observed=True)["Churn"]
               .apply(lambda s: (s == "Yes").mean() * 100)
               .unstack())

fig, ax = plt.subplots(figsize=(7, 3.5))
sns.heatmap(heat_data, annot=True, fmt=".1f", cmap="YlOrRd",
            cbar_kws={"label": "Churn rate (%)"}, ax=ax)
ax.set_title("Churn Rate: Contract × InternetService")
ax.set_xlabel("Layanan internet")
```

```
ax.set_ylabel("Tipe kontrak")
plt.tight_layout()
plt.show()
```

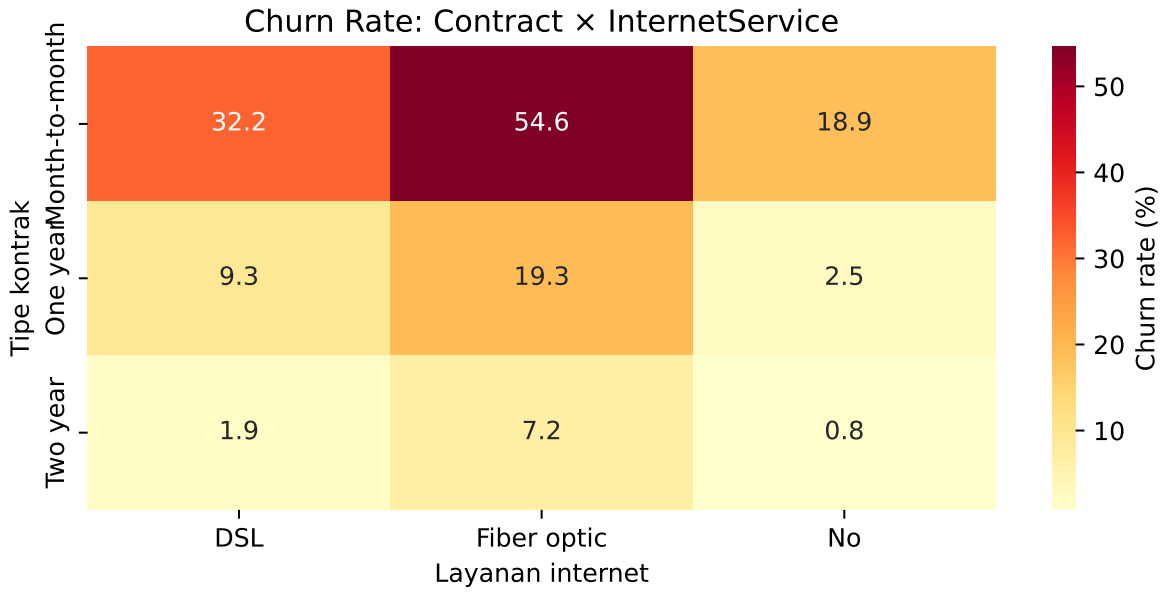


Figure 5.4: Heatmap churn rate per kombinasi Contract dan InternetService. Warna lebih gelap = churn lebih tinggi.

5.8.2 2. Small Multiples untuk Perbandingan

Ketika Anda ingin membandingkan pola yang sama di banyak segmen, **small multiples** (facet) jauh lebih efektif daripada menumpuk semua garis di satu grafik.

```
g = sns.FacetGrid(df, col="InternetService", row="Contract",
                  height=2.5, aspect=1.3, margin_titles=True)
g.map(sns.histplot, "tenure", bins=20, color="#4C72B0")
g.set_axis_labels("Tenure (bulan)", "Jumlah")
g.figure.suptitle("Distribusi Tenure per Segmen", y=1.02)
plt.tight_layout()
plt.show()
```

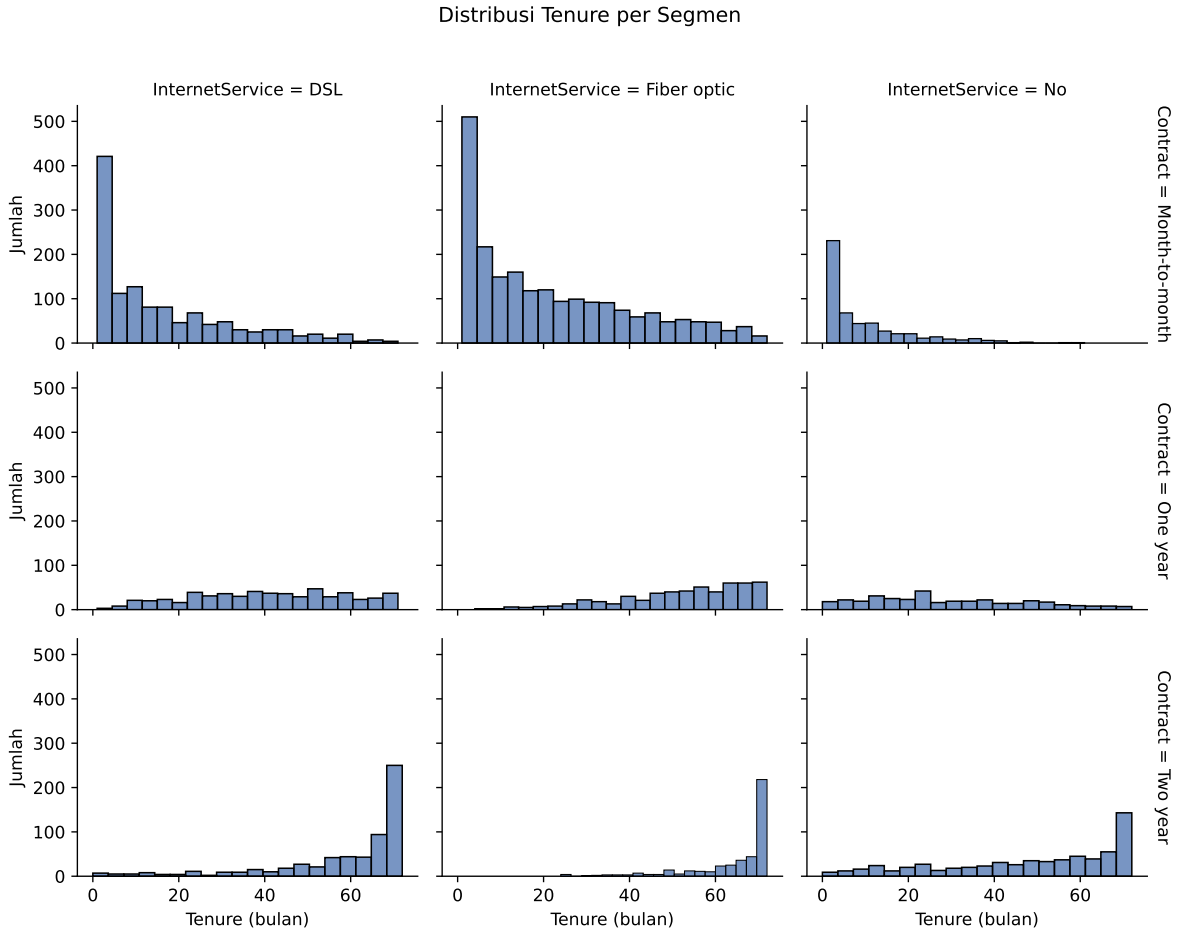


Figure 5.5: Distribusi tenure per segmen kontrak dan layanan internet.

5.8.3 3. Scatter dengan Anotasi

Scatter plot dengan anotasi memungkinkan Anda **menyoroti** titik-titik tertentu yang penting—misalnya, pelanggan dengan kombinasi fitur yang tidak biasa.

```
sample = df.sample(2000, random_state=42)
fig, ax = plt.subplots(figsize=(9, 4.5))
for status, color in [("No", "#4C72B0"), ("Yes", "#DD8452")]:
    subset = sample[sample["Churn"] == status]
    ax.scatter(subset["tenure"], subset["MonthlyCharges"],
               c=color, label=f"Churn = {status}", alpha=0.5, s=15)

# Sorot area berisiko tinggi: tenure rendah + tagihan tinggi
```

```
ax.axvspan(0, 12, alpha=0.1, color="red")
ax.axhline(df["MonthlyCharges"].median(), color="gray", linestyle=":", alpha=0.5)

ax.set_xlabel("Tenure (bulan)")
ax.set_ylabel("Tagihan bulanan")
ax.set_title("Tenure vs MonthlyCharges - Klaster Risiko Tinggi di Kuadran Kiri Atas")
ax.legend()
plt.tight_layout()
plt.show()
```

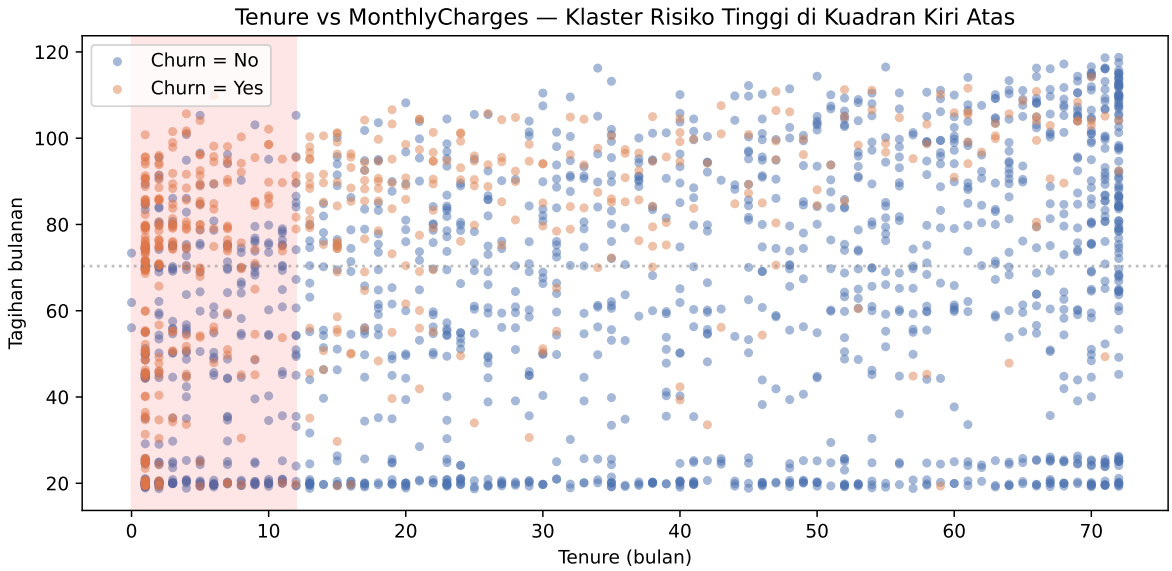


Figure 5.6: Scatter tenure vs MonthlyCharges, diwarnai berdasarkan churn. Perhatikan klaster pelanggan berisiko tinggi.

Temuan. Pelanggan dengan **tenure rendah** (0–12 bulan, area merah) dan **tagihan bulanan tinggi** (di atas median) membentuk klaster dengan proporsi churn yang sangat tinggi. Ini adalah segmen dengan **nilai tinggi tetapi risiko tinggi**—pelanggan yang membayar mahal tetapi belum berkomitmen. Keputusan: segmen ini membutuhkan **intervensi retensi prioritas**—mungkin diskon perpanjangan kontrak, atau penawaran layanan tambahan gratis selama 3 bulan pertama.

5.9 Batas Diagnosis: Korelasi vs Kausalitas

Kita telah menemukan banyak pola. Tetapi kita harus **jujur tentang batasnya**: semua teknik di bab ini menemukan **asosiasi**, bukan **kausalitas**.

5.9.1 Tiga Syarat Kausalitas

Untuk menyimpulkan bahwa X **menyebabkan** Y, kita membutuhkan tiga hal:

1. **Korelasi**: X dan Y bergerak bersama. (sudah kita punya)
2. **Urutan waktu**: X terjadi sebelum Y. (kita tidak tahu kapan churn terjadi relatif terhadap perubahan kontrak)
3. **Eliminasi penjelasan alternatif**: tidak ada variabel ketiga yang menjelaskan keduanya. (kita tidak bisa memastikan ini tanpa eksperimen atau desain kausal)

5.9.2 Metode Kausal yang Sederhana

Untuk dataset observasional seperti Telco, kita bisa mendekati kausalitas dengan beberapa teknik sederhana—meskipun tidak sekuat eksperimen:

Teknik	Prinsip	Keterbatasan
Stratifikasi	Bandingkan X→Y dalam subkelompok Z	Hanya mengontrol Z yang teramati
Regresi multivariabel	Estimasi efek X sambil mengontrol variabel lain	Tetap hanya mengontrol yang teramati; tidak menangani endogenitas
Propensity score matching	Pasangkan pelanggan dengan karakteristik serupa	Membutuhkan banyak data; tidak menangani confounder tak teramati
A/B test	Eksperimen acak	Tidak selalu mungkin atau etis

> **!** Important
>
> **Diagnosis yang baik tahu batasnya.** Kita bisa menyimpulkan bahwa "pelanggan fiber optic dengan kontrak bulanan memiliki churn rate tinggi" dengan yakin—itu fakta deskriptif yang kuat. Tetapi menyimpulkan bahwa "mengubah pelanggan dari fiber optic ke DSL akan mengurangi churn" membutuhkan eksperimen. **Diagnosis mengidentifikasi kandidat penyebab; konfirmasi membutuhkan desain yang lebih kuat.**

5.10 Studi Kasus: Diagnosis Churn Telco

Mari kita gabungkan semua teknik di atas dalam satu studi kasus yang menjawab pertanyaan:

> *"Segmen mana yang paling berisiko churn, dan mengapa?"*

5.10.1 Langkah 1: Identifikasi Segmen Berisiko Tinggi

```
# Definisikan segmen berisiko tinggi berdasarkan analisis di atas
df["risk_segment"] = np.select(
    [
        (df["Contract"] == "Month-to-month") & (df["tenure"] <= 12) & (df["InternetService"]
        (df["Contract"] == "Month-to-month") & (df["tenure"] <= 12),
        (df["Contract"] == "Month-to-month"),
    ],
    ["Sangat tinggi", "Tinggi", "Menengah"],
    default="Rendah"
)

segmen_ringkasan = (df.groupby("risk_segment", observed=True)
                      .agg(
                          jumlah=("Churn", "size"),
                          churn_rate=("Churn", lambda s: (s == "Yes").mean() * 100),
                          avg_tenure=("tenure", "mean"),
                          avg_charges=("MonthlyCharges", "mean"),
                      )
                      .round(1)
                      .sort_values("churn_rate", ascending=False))
segmen_ringkasan
```

risk_segment	jumlah	churn_rate	avg_tenure	avg_charges
Sangat tinggi	916	70.2	4.8	82.1
Tinggi	1078	35.3	4.2	37.9
Menengah	1881	33.5	32.4	75.1
Rendah	3168	6.8	49.9	62.8

5.10.2 Langkah 2: Drill-Down ke Segmen "Sangat Tinggi"

```
high_risk = df[df["risk_segment"] == "Sangat tinggi"]

print(f"Pelanggan di segmen 'Sangat tinggi': {len(high_risk):,} "
      f"({len(high_risk)/len(df)*100:.1f}% dari total)")
print(f"Churn rate segmen ini             : {(high_risk['Churn'] == 'Yes').mean()*100:.1f}%")
print(f"\nDistribusi fitur di segmen ini:")
for col in ["PaymentMethod", "Partner", "Dependents", "OnlineSecurity"]:
    vc = high_risk[col].value_counts(normalize=True).mul(100).round(1)
    print(f"\n  {col}:")
    for k, v in vc.items():
        print(f"    {k:20s}: {v:5.1f}%")
```

```
Pelanggan di segmen 'Sangat tinggi': 916 (13.0% dari total)
Churn rate segmen ini             : 70.2%

Distribusi fitur di segmen ini:

  PaymentMethod:
    Electronic check    :  68.9%
    Mailed check        :  13.4%
    Bank transfer (automatic):   9.5%
    Credit card (automatic):   8.2%

  Partner:
    No                  :  74.9%
    Yes                 :  25.1%

  Dependents:
    No                  :  87.8%
    Yes                 :  12.2%

  OnlineSecurity:
    No                  :  90.8%
    Yes                 :   9.2%
```

5.10.3 Langkah 3: 5 Whys untuk Segmen Ini

1. Mengapa segmen ini churn? → Karena mereka kontrak bulanan, tenure pendek, dan menggunakan fiber optic.

2. Mengapa mereka memilih kontrak bulanan? → Karena mereka baru—belum yakin dengan layanan, belum berkomitmen.

3. Mengapa mereka memilih fiber optic? → Mungkin karena promosi awal atau karena ingin kecepatan tinggi.

4. Mengapa mereka churn meskipun membayar mahal? → Karena ekspektasi tinggi tidak terpenuhi, atau karena mereka tidak terikat kontrak dan mudah pindah ke kompetitor.

5. Mengapa mereka tidak terikat? → Karena belum ada program onboarding yang membangun keterikatan—tidak ada diskon perpanjangan, tidak ada layanan tambahan gratis, tidak ada komunikasi proaktif.

Hipotesis akar masalah: Kurangnya strategi onboarding dan retensi dini untuk pelanggan fiber optic baru.

5.10.4 Langkah 4: Implikasi untuk Keputusan

Berdasarkan diagnosis ini, tiga intervensi yang masuk akal:

Intervensi	Target	Dasar diagnosis
Diskon perpanjangan kontrak untuk pelanggan fiber optic baru	Segmen "Sangat tinggi"	Mengurangi insentif churn dengan menciptakan biaya peralihan
Bundling layanan tambahan gratis 3 bulan pertama	Segmen "Sangat tinggi"	Membangun keterikatan melalui penggunaan fitur
Program onboarding proaktif (check-in di bulan 1, 3, 6)	Semua pelanggan baru	Mengurangi churn dini yang terlihat di kurva survival

> **Note**
>
> **Ini masih hipotesis.** Intervensi di atas masuk akal berdasarkan diagnosis, tetapi **belum terbukti efektif**. Untuk mengujinya, kita perlu **A/B test**—yang akan kita bahas sebagai bagian dari integrasi di Bab 18. Diagnosis yang baik menghasilkan **hipotesis yang dapat diuji**, bukan keputusan final.

5.11 Latihan Terbimbing

Latihan 5.1 — Stratifikasi. Apakah efek `InternetService` terhadap `Churn` bertahan ketika kita stratifikasi berdasarkan `PaymentMethod`? Buat tabel stratifikasi dan jelaskan temuan Anda dalam satu paragraf.

Latihan 5.2 — Simpson's Paradox. Cari satu pasangan variabel di dataset Telco yang menunjukkan Simpson's Paradox ketika distratifikasi oleh variabel ketiga. Tunjukkan tabel agregat dan tabel stratifikasinya, dan jelaskan mengapa paradox itu muncul.

Latihan 5.3 — Analisis kohort. Buat kurva survival untuk pelanggan yang dipisah berdasarkan `Contract`. Apakah bentuk kurva berbeda? Apa implikasinya untuk strategi retensi?

Latihan 5.4 — 5 Whys. Lakukan latihan 5 Whys untuk pertanyaan: *"Mengapa pelanggan dengan `PaperlessBilling` (jika ada di dataset Anda) memiliki churn rate yang berbeda?"* Tuliskan lima "mengapa" berturut-turut dan akar masalah yang Anda temukan. Untuk setiap "mengapa", identifikasi data apa yang akan Anda gunakan untuk memvalidasinya.

Latihan 5.5 — Batas diagnosis. Pilih satu temuan dari bab ini dan tuliskan satu paragraf yang menjelaskan mengapa temuan itu **belum** merupakan bukti kausal. Apa eksperimen yang akan Anda rancang untuk mengujinya?

> **Note**
>
> **Petunjuk Latihan 5.2.** Simpson's Paradox paling mungkin muncul ketika variabel stratifikasi berkorelasi **positif** dengan variabel penjelas **dan** berkorelasi **negatif** dengan target (atau sebaliknya). Periksa korelasi antara `InternetService`, `Contract`, dan `Churn`—kemungkinan besar Anda akan menemukan kombinasi yang menarik.

5.12 Kotak "Jembatan": Dari Data ke Keputusan

> **Important**
>
> **Jembatan.** Bab ini mengubah cara kita memandang data secara fundamental:
>
> 1. **Dari gejala ke akar masalah.** Di Bab 3 kita tahu *bahwa* kontrak bulanan memiliki churn tinggi. Di Bab 5 kita tahu *mengapa*—dan "mengapa" itu mengarahkan intervensi ke tempat yang berbeda: bukan pada kontrak itu sendiri, tetapi pada **program onboarding dan keterikatan pelanggan baru**.
> 2. **Dari agregat ke segmen.** Rata-rata churn rate 26,5% menyembunyikan kenyataan bahwa ada segmen dengan churn rate $> 50\%$ dan segmen lain dengan churn rate $<$

5%. Keputusan yang baik **menargetkan segmen**, bukan rata-rata.

3. **Dari korelasi ke hipotesis yang dapat diuji.** Kita belum bisa mengatakan "diskon perpanjangan kontrak akan mengurangi churn." Tetapi kita bisa mengatakan "berdasarkan pola stratifikasi, pelanggan fiber optic baru dengan kontrak bulanan adalah target prioritas untuk intervensi retensi." Ini adalah **hipotesis yang dapat diuji**—dan mengujinya membutuhkan **prediksi** (Bab 9) dan **optimasi** (Bab 14).

4. **Dari diagnosis ke prediksi.** Diagnosis memberi kita **variabel mana yang penting**. Prediksi akan memberi kita **model yang dapat memprediksi churn untuk pelanggan baru**—berdasarkan pola yang sama, tetapi diterapkan secara individual. Inilah jembatan ke Bagian II.

5.13 Rangkuman

Dalam bab ini kita telah:

- **Membedakan pertanyaan deskriptif dari diagnostik**—dan memahami bahwa diagnosis mengejar mekanisme, bukan sekadar pola.
- **Menerapkan stratifikasi** untuk menguji apakah pola bertahan ketika data dipecah berdasarkan variabel ketiga.
- **Mengenali Simpson's Paradox** dan memahami mengapa agregasi bisa menyesatkan—serta bagaimana menghindarinya.
- **Menggunakan drill-down** untuk menelusuri pola ke tingkat yang lebih dalam, dengan kesadaran akan bahaya sampel kecil.
- **Melakukan analisis kohort** untuk memahami bagaimana perilaku berubah sepanjang siklus hidup pelanggan—dan menemukan bahwa churn paling akut di awal.
- **Menerapkan kerangka root cause analysis** (5 Whys, Ishikawa) untuk bergerak dari gejala ke akar masalah.
- **Menggunakan visual analytics**—heatmap stratifikasi, small multiples, scatter beranotasi—untuk mengungkap pola yang tidak terlihat di tabel.
- **Mengakui batas diagnosis**: semua teknik di bab ini menemukan asosiasi, bukan kausalitas, dan konfirmasi membutuhkan desain eksperimen.

5.14 Jembatan ke Bab 6

Kita telah menyelesaikan sebagian besar Bagian I. Kita telah membersihkan data, mengukurnya, memvisualisasikannya, dan mendiagnosisnya. Sekarang saatnya **menuai hasil**—mengubah semua analisis ini menjadi **insight yang dapat dikomunikasikan** kepada pengambil keputusan.

Bab berikutnya—**Studi Kasus & Komunikasi Insight**—akan menjadi **bab penutup Bagian I**. Ia akan:

- Menggabungkan semua teknik dari Bab 1–5 dalam satu **studi kasus end-to-end** pada dataset Telco.
- Memperkenalkan **data storytelling**: bagaimana menyusun narasi dari data ke insight ke rekomendasi.
- Menunjukkan cara membangun **dashboard eksekutif** yang merangkum temuan Bagian I.
- Menjadi **jembatan formal** ke Bagian II—menunjukkan bahwa semua pertanyaan "mengapa" yang kita ajukan di sini membutuhkan model prediktif untuk dijawab secara individual.

6 Studi Kasus & Komunikasi Insight

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Menerapkan kerangka **SCQA** (Situation-Complication-Question-Answer) untuk menyusun narasi analitis. 2. Menggabungkan seluruh teknik Bagian I—wrangling, statistik, visualisasi, diagnosis—dalam satu alur kerja end-to-end. 3. Merancang **dashboard eksekutif** yang merangkum temuan untuk audiens non-teknis. 4. Mengomunikasikan ketidakpastian dan batas analisis tanpa melemahkan rekomendasi. 5. Menerjemahkan insight menjadi **rekomendasi yang dapat dieksekusi** dengan prioritas yang jelas. 6. Mengenali kapan analisis deskriptif **berhenti menjadi cukup**, dan model prediktif diperlukan.

6.1 Skenario: Dua Minggu untuk Menyelamatkan Pelanggan

Bayangkan Anda baru bergabung sebagai **Senior Data Analyst** di **Telko Nusantara**, sebuah perusahaan telekomunikasi dengan sekitar 7.000 pelanggan aktif. Pada hari Senin pagi, VP Customer Success memanggil Anda ke ruangannya:

> *"Kita kehilangan pelanggan lebih cepat dari yang bisa kita gantikan. Saya butuh diagnosis yang jelas dalam dua minggu—bukan laporan 50 halaman, tetapi sesuatu yang bisa saya bawa ke rapat direksi. Beri tahu saya: siapa yang churn, mengapa, dan apa yang harus kita lakukan."*

Anda memiliki dataset pelanggan, keterampilan Python dari Bab 1–5, dan **dua minggu**. Bagaimana Anda mendekatinya?

Tip

Ini bukan latihan akademis. Dalam pekerjaan nyata, analis jarang mendapat waktu berbulan-bulan. Yang membedakan analis yang efektif bukan hanya keterampilan teknis, tetapi kemampuan **memfokuskan energi pada pertanyaan yang tepat** dan **menyampaikan jawaban yang dapat ditindaklanjuti**.

6.2 Kerangka SCQA: Cara Berpikir Sebelum Menulis Kode

Sebelum menyentuh data, tetapkan **narasi** yang akan Anda bangun. Kerangka **SCQA** (Barbara Minto, *The Pyramid Principle*) adalah standar dalam konsultasi strategis:

Komponen	Pertanyaan	Jawaban untuk kasus ini
Situation	Apa konteks yang disepakati semua orang?	Telko Nusantara memiliki ~7.000 pelanggan aktif. Churn rate ~26,5%—sekitar 1 dari 4 pelanggan berhenti setiap tahun.
Complication	Apa masalah atau perubahan yang memicu kebutuhan ini?	Churn rate ini **tinggi** untuk industri telekomunikasi, dan biaya akuisisi pelanggan baru jauh lebih besar daripada retensi.
Question	Pertanyaan apa yang muncul dari komplikasi ini?	*Pelanggan seperti apa yang churn, mengapa mereka churn, dan intervensi apa yang paling efektif?*
Answer	Jawaban utama (yang akan diuraikan di seluruh presentasi)	*Churn terkonsentrasi pada pelanggan baru dengan kontrak bulanan dan layanan fiber optic. Intervensi retensi harus difokuskan pada 12 bulan pertama, dengan program onboarding dan bundling layanan sebagai prioritas utama.*

> **!** Important
>
> **Mengapa SCQA penting?** Karena pengambil keputusan tidak punya waktu untuk mengikuti alur berpikir Anda dari awal. Mereka perlu **jawaban lebih dulu**, baru kemudian bukti. SCQA memaksa Anda menyusun presentasi **dari kesimpulan ke detail** (deduktif), bukan dari data ke kesimpulan (induktif).

6.3 Menyusun Analisis: Alur Kerja End-to-End

Dengan SCQA di tangan, kita sekarang menjalankan analisis. Perhatikan bahwa setiap langkah di bawah ini **sudah Anda kuasai** dari Bab 1–5—yang baru adalah cara merangkainya menjadi satu alur yang koheren.

6.3.1 Langkah 1: Memuat Data Bersih

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

from decanal.utils import load_churn_clean

df = load_churn_clean()

# Konstanta warna konsisten dari Bab 4
COLOR_NO = "#4C72B0"
COLOR_YES = "#DD8452"
PALETTE_CHURN = {"No": COLOR_NO, "Yes": COLOR_YES}

print(f"Dataset: {df.shape[0]:,} pelanggan × {df.shape[1]} kolom")
```

```
Dataset: 7,043 pelanggan × 23 kolom
```

6.3.2 Langkah 2: Menetapkan Baseline

Sebelum masuk ke detail, kita perlu **baseline** yang akan menjadi konteks semua temuan:

```
n_total = len(df)
n_churn = (df["Churn"] == "Yes").sum()
churn_rate = n_churn / n_total * 100
```

```
# Estimasi dampak finansial (asumsi konservatif)
avg_monthly = df["MonthlyCharges"].mean()
avg_tenure = df["tenure"].mean()
lost_revenue_per_churn = avg_monthly * avg_tenure  # nilai seumur hidup rata-rata

baseline = pd.DataFrame({
    "Metrik": [
        "Total pelanggan",
        "Pelanggan churn",
        "Churn rate",
        "Tagihan bulanan rata-rata",
        "Tenure rata-rata (bulan)",
        "Estimasi revenue hilang per churn (Rp)",
    ],
    "Nilai": [
        f"{n_total:,}",
        f"{n_churn:,}",
        f"{churn_rate:.1f}%",
        f"Rp {avg_monthly:,.0f}",
        f"{avg_tenure:.1f}",
        f"Rp {lost_revenue_per_churn:,.0f}",
    ],
})
baseline
```

	Metrik	Nilai
0	Total pelanggan	7,043
1	Pelanggan churn	1,869
2	Churn rate	26.5%
3	Tagihan bulanan rata-rata	Rp 65
4	Tenure rata-rata (bulan)	32.4
5	Estimasi revenue hilang per churn (Rp)	Rp 2,096

Note

Asumsi harus eksplisit. Estimasi revenue hilang di atas menggunakan asumsi sederhana: pelanggan yang churn akan bertahan selama rata-rata tenure dataset. Ini **bukan** angka pasti—pelanggan yang churn mungkin memiliki profil berbeda. Tetapi untuk presentasi eksekutif, **angka perkiraan yang dapat dijelaskan** lebih berguna daripada tidak ada angka sama sekali. Selalu cantumkan asumsi di catatan kaki.

6.3.3 Langkah 3: Diagnosis — Di Mana Churn Terkonsentrasi?

```
# Stratifikasi bertingkat: Contract × tenure bin
df["tenure_bin"] = pd.cut(
    df["tenure"], bins=[-1, 6, 12, 24, 48, np.inf],
    labels=["0-6 bln", "7-12 bln", "13-24 bln", "25-48 bln", ">48 bln"]
)

segmen = (df.groupby(["Contract", "tenure_bin"], observed=True)
            .agg(
                jumlah=("Churn", "size"),
                churn_rate=("Churn", lambda s: (s == "Yes").mean() * 100),
            )
            .round(1)
            .reset_index())
segmen
```

	Contract	tenure_bin	jumlah	churn_rate
0	Month-to-month	0-6 bln	1413	55.2
1	Month-to-month	7-12 bln	581	42.0
2	Month-to-month	13-24 bln	737	37.7
3	Month-to-month	25-48 bln	802	32.9
4	Month-to-month	>48 bln	342	26.0
5	One year	0-6 bln	39	10.3
6	One year	7-12 bln	85	10.6
7	One year	13-24 bln	197	8.1
8	One year	25-48 bln	518	10.6
9	One year	>48 bln	634	12.9
10	Two year	0-6 bln	29	0.0
11	Two year	7-12 bln	39	0.0
12	Two year	13-24 bln	90	0.0
13	Two year	25-48 bln	274	2.2
14	Two year	>48 bln	1263	3.3

Dari tabel ini, kita dapat menyusun **tiga temuan utama**:

1. **Efek kontrak**: Churn rate kontrak bulanan konsisten lebih tinggi di semua kelompok tenure.
2. **Efek tenure**: Churn rate menurun tajam seiring tenure—puncaknya pada 0–6 bulan pertama.

3. **Interaksi**: Kombinasi kontrak bulanan + tenure pendek memiliki churn rate **paling ekstrem**.

6.3.4 Langkah 4: Menghitung Dampak Finansial per Segmen

Temuan statistik perlu diterjemahkan ke **bahasa bisnis**. Berapa revenue yang berisiko di setiap segmen?

```
# Definisikan segmen prioritas
df["priority_segment"] = np.select(
    [
        (df["Contract"] == "Month-to-month") & (df["tenure"] <= 12),
        (df["Contract"] == "Month-to-month") & (df["tenure"] > 12),
        (df["Contract"] == "One year"),
    ],
    ["Prioritas 1: Bulanan baru", "Prioritas 2: Bulanan lama", "Prioritas 3: Tahunan"],
    default="Prioritas 4: Dua tahun"
)

finansial = (df.groupby("priority_segment", observed=True)
               .agg(
                   jumlah=("Churn", "size"),
                   churn_rate=("Churn", lambda s: (s == "Yes").mean() * 100),
                   avg_charges=("MonthlyCharges", "mean"),
                   churn_count=("Churn", lambda s: (s == "Yes").sum()),
               )
               .round(1))
finansial["revenue_berisiko"] = (
    finansial["churn_count"] * finansial["avg_charges"] * df["tenure"].mean()
).round(0)
finansial = finansial.sort_values("revenue_berisiko", ascending=False)
finansial
```

priority_segment	jumlah	churn_rate	avg_charges	churn_count	revenue_berisiko
Prioritas 1: Bulanan baru	1994	51.4	58.2	1024	1929217.0
Prioritas 2: Bulanan lama	1881	33.5	75.1	631	1534007.0
Prioritas 3: Tahunan	1473	11.3	65.0	166	349285.0
Prioritas 4: Dua tahun	1695	2.8	60.8	48	94472.0

> **!** Important
>
> **Perhatikan pergeseran prioritas.** Segmen "Prioritas 1" mungkin memiliki jumlah pelanggan lebih kecil, tetapi karena churn rate-nya tinggi, **revenue berisiko**-nya bisa lebih besar dari segmen lain yang jumlahnya lebih banyak. Inilah mengapa analisis keuangan harus menyertai analisis statistik—untuk mengarahkan keputusan ke tempat yang dampaknya paling besar.

6.4 Dari Analisis ke Narasi: Data Storytelling

Sekarang kita memiliki semua bahan: baseline, segmen, angka finansial. Tetapi presentasi eksekutif **bukan** kumpulan tabel dan grafik. Ia adalah **cerita** dengan alur, klimaks, dan resolusi.

6.4.1 Tiga Tingkat Data Storytelling

Tingkat	Audiens	Fokus	Contoh dalam kasus kita
Laporan	Analis, tim teknis	Kelengkapan, akurasi	Tabel lengkap, kode, metode
Presentasi	Manajer, direksi	Insight dan implikasi	5–7 slide dengan grafik kunci
One-pager	Eksekutif senior	Rekomendasi dan dampak	Satu halaman dengan tiga rekomendasi

Untuk VP Customer Success, kita membutuhkan **presentasi** dan **one-pager**—bukan laporan teknis.

6.4.2 Struktur Narasi: Piramida Terbalik

1. **Jawaban (1 slide)**: *"Churn terkonsentrasi pada pelanggan bulanan baru. Intervensi di 12 bulan pertama akan menyelamatkan X pelanggan dan Y revenue."*
2. **Tiga bukti utama (3 slide)**: Setiap bukti adalah satu grafik dengan satu pesan.
3. **Rekomendasi (1 slide)**: Tiga intervensi dengan prioritas dan estimasi dampak.
4. **Lampiran**: Tabel detail, metodologi, batasan.

> Tip
>
> **Aturan 1-3-1.** Untuk presentasi eksekutif, gunakan struktur: - **1** slide pembuka dengan jawaban utama - **3** slide pendukung (satu bukti per slide) - **1** slide rekomendasi dan next steps
> Lebih dari ini, dan audiens akan kehilangan benang merah. Kurang dari ini, dan Anda terlihat kurang mendalam.

6.5 Dashboard Eksekutif

Dashboard eksekutif berbeda dari dashboard analis (Bab 4). Ia harus menjawab **satu pertanyaan besar** dan menyoroti **tiga angka kunci**—bukan menyediakan semua detail.

6.5.1 Tiga KPI Utama

```
# KPI 1: Skala masalah
total_churn_loss = finansial["revenue_berisiko"].sum()

# KPI 2: Konsentrasi
prioritas_1 = finansial.loc["Prioritas 1: Bulanan baru"]
pct_of_total_churn = (
    prioritas_1["churn_count"] / df["Churn"].eq("Yes").sum() * 100
)

# KPI 3: Potensi penyelamatan (asumsi 20% reduction dari churn prioritas 1)
potential_saved = prioritas_1["revenue_berisiko"] * 0.2

fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=(
        "Revenue berisiko",
        "Churn dari segmen prioritas 1",
        "Potensi diselamatkan (20%)",
    ),
    specs=[[{"type": "indicator"}, {"type": "indicator"}, {"type": "indicator"}]],
)

fig.add_trace(
    go.Indicator(
```

```
        mode="number",
        value=total_churn_loss / 1e9,
        number={"suffix": " M", "valueformat": ".2f"},
        title={"text": "Rp miliar"},
    ),
    row=1, col=1,
)

fig.add_trace(
    go.Indicator(
        mode="number+gauge",
        value=pct_of_total_churn,
        number={"suffix": "%", "valueformat": ".0f"},
        gauge={"axis": {"range": [0, 100]}, "bar": {"color": COLOR_YES},
               "threshold": {"line": {"color": "red", "width": 2},
                             "thickness": 0.75, "value": 50}},
    ),
    row=1, col=2,
)

fig.add_trace(
    go.Indicator(
        mode="number",
        value=potential_saved / 1e9,
        number={"suffix": " M", "valueformat": ".2f"},
        title={"text": "Rp miliar"},
    ),
    row=1, col=3,
)

fig.update_layout(
    height=350, width=1100,
    title_text="Dashboard Eksekutif - Churn Telko Nusantara",
    margin=dict(l=40, r=40, t=100, b=40),
)
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

(a) Tiga KPI utama untuk presentasi eksekutif: skala masalah, konsentrasi risiko, dan potensi penyelamatan.

```
Unable to display output for mime type(s): text/html
```

(b)

Figure 6.1

6.5.2 Dashboard Detail: Tiga Grafik yang Menjawab "Siapa, Mengapa, Berapa"

```
fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=(
        "Churn rate per segmen prioritas",
        "Kurva survival",
        "Kontribusi revenue berisiko",
    ),
    specs=[[{"type": "bar"}, {"type": "scatter"}, {"type": "domain"}]],
)

# Panel 1: Churn rate per segmen
seg_plot = finansial.reset_index().sort_values("churn_rate")
fig.add_trace(
    go.Bar(
        x=seg_plot["churn_rate"],
        y=seg_plot["priority_segment"],
        orientation="h",
        marker_color=COLOR_YES,
        text=seg_plot["churn_rate"].round(1),
        texttemplate="%{text}%",
        textposition="outside",
        showlegend=False,
    ),
    row=1, col=1,
)

# Panel 2: Kurva survival
tenure_range = np.arange(0, 73, 1)
```

```
survival = [(df["tenure"] >= t).mean() * 100 for t in tenure_range]
fig.add_trace(
    go.Scatter(
        x=tenure_range, y=survival,
        mode="lines",
        line={"color": COLOR_NO, "width": 3},
        fill="tozeroy",
        fillcolor="rgba(76, 114, 176, 0.2)",
        showlegend=False,
    ),
    row=1, col=2,
)

# Panel 3: Donut revenue
fig.add_trace(
    go.Pie(
        labels=finansial.index,
        values=finansial["revenue_berisiko"],
        hole=0.5,
        marker={"colors": ["#DD8452", "#55A868", "#4C72B0", "#8172B3"]},
        showlegend=True,
    ),
    row=1, col=3,
)

fig.update_xaxes(title_text="Churn rate (%)", row=1, col=1)
fig.update_xaxes(title_text="Tenure (bulan)", row=1, col=2)
fig.update_yaxes(title_text="Bertahan (%)", row=1, col=2)

fig.update_layout(
    height=420, width=1200,
    title_text="Di Mana Churn Terkonsentrasi - dan Berapa Biayanya",
    margin=dict(l=40, r=40, t=100, b=80),
)
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

Figure 6.2: Dashboard eksekutif detail: (kiri) churn per segmen prioritas; (tengah) kurva survival; (kanan) kontribusi revenue berisiko.

> **!** Important
>
> **Perhatikan desainnya.** Setiap panel menjawab **satu pertanyaan** yang berbeda: - Kiri: **Siapa** yang paling berisiko (segmen prioritas). - Tengah: **Kapan** risiko paling akut (kurva survival). - Kanan: **Berapa** dampak finansialnya (donut revenue).
> Ketiganya bersama-sama menjawab pertanyaan VP: *"Siapa yang churn, mengapa, dan apa yang harus kita lakukan?"* Tanpa satu pun, narasinya tidak lengkap.

6.6 Menyampaikan ke Stakeholder

Analisis yang baik bisa gagal jika **penyampaiannya** buruk. Berikut adalah prinsip-prinsip praktis yang saya sarankan Anda terapkan.

6.6.1 1. Buka dengan Jawaban, Bukan Metodologi

Jangan memulai dengan: *"Saya menggunakan stratifikasi berdasarkan tenure dan contract…"*

Mulai dengan: *"Churn kita terkonsentrasi pada 1.800 pelanggan baru dengan kontrak bulanan. Jika kita bisa menahan 20% dari mereka, kita menyelamatkan sekitar Rp X miliar setahun."*

6.6.2 2. Gunakan Analogi, Bukan Jargon

Jargon	Alternatif
"Churn rate 26,5%"	"Satu dari empat pelanggan berhenti setiap tahun"
"Distribusi bimodal"	"Ada dua kelompok pelanggan: yang baru bergabung dan yang sudah lama"
"Stratifikasi berdasarkan tenure"	"Ketika kita bandingkan pelanggan baru dengan pelanggan lama secara terpisah"
"Simpson's Paradox"	"Rata-rata keseluruhan menyesatkan karena komposisi pelanggan berbeda"

6.6.3 3. Tunjukkan Ketidakpastian Secara Eksplisit

Jangan sembunyikan ketidakpastian—**bimbing audiens untuk mempercayai Anda** dengan menunjukkannya:

"Estimasi revenue berisiko adalah Rp X miliar. Angka ini didasarkan pada asumsi bahwa pelanggan yang churn akan bertahan selama tenure rata-rata. Jika asumsi ini terlalu optimis, dampak sebenarnya bisa lebih kecil—tetapi besaran relatif antar segmen tetap sama."

6.6.4 4. Akhiri dengan Ajakan Bertindak yang Spesifik

Jangan akhiri dengan: *"Demikian presentasi saya, terima kasih."*

Akhiri dengan: *"Saya merekomendasikan tiga hal: pertama, pilot program onboarding untuk 500 pelanggan prioritas 1 selama 3 bulan. Kedua, alokasi ulang 30% anggaran retensi ke segmen prioritas 1. Ketiga, mengizinkan saya membangun model prediktif dalam 4 minggu ke depan untuk mengidentifikasi pelanggan berisiko secara individual—sehingga intervensi bisa ditargetkan lebih presisi."*

6.7 Tiga Rekomendasi Prioritas

Berdasarkan diagnosis, berikut adalah rekomendasi yang dapat disampaikan ke VP:

Prioritas	Rekomendasi	Target	Estimasi Dampak
1	Program onboarding 90 hari untuk pelanggan baru (welcome call bulan 1, check-in bulan 3, penawaran upgrade bulan 6)	Semua pelanggan baru	Mengurangi churn dini ~15–20%
2	Bundling layanan tambahan gratis 3 bulan (OnlineSecurity, TechSupport)	Pelanggan prioritas 1	Meningkatkan keterikatan; estimasi 10% pengurangan churn
3	Diskon perpanjangan kontrak untuk pelanggan yang beralih dari bulanan ke tahunan	Pelanggan prioritas 1 & 2	Meningkatkan revenue per pelanggan (LTV) dan mengurangi churn

> ⚠ Warning
>
> **Rekomendasi ini belum teruji.** Ketiganya adalah **hipotesis** berdasarkan diagnosis yang kuat, tetapi efektivitasnya harus dikonfirmasi melalui **A/B test**. Presentasi eksekutif yang baik **mengakui hal ini**—dan mengusulkan pengujian sebagai langkah berikutnya, bukan sebagai keputusan final. Ini melindungi Anda dari risiko rekomendasi yang tidak berhasil, dan membangun kepercayaan melalui kejujuran.

6.8 Batas Analisis Ini

Setiap presentasi eksekutif yang baik menyertakan **slide batasan**. Ini bukan kelemahan—ini adalah tanda kedewasaan analitis. Untuk kasus ini:

1. **Analisis korelasional, bukan kausal.** Kita tahu segmen mana yang berisiko, tetapi tidak bisa memastikan intervensi akan **menyebabkan** penurunan churn tanpa eksperimen.
2. **Data historis, bukan data real-time.** Churn rate bisa berubah jika kompetitor meluncurkan produk baru atau jika kondisi pasar berubah.
3. **Estimasi revenue berdasarkan asumsi.** Angka dampak finansial menggunakan asumsi tenure rata-rata—nilai sebenarnya bisa berbeda.
4. **Tidak ada model prediktif individual.** Analisis ini mengidentifikasi **segmen** berisiko, tetapi tidak dapat memprediksi **pelanggan spesifik** yang akan churn.

Poin keempat inilah yang menjadi **jembatan langsung ke Bagian II**.

6.9 Kotak "Jembatan": Dari Diagnosis ke Prediksi

> ❗ Important
>
> **Jembatan.** Semua yang kita lakukan di Bagian I menjawab pertanyaan **tingkat populasi**: *"Segmen mana yang berisiko?"*
> Tetapi keputusan retensi yang efektif membutuhkan jawaban **tingkat individu**: *"Pelanggan mana yang harus saya hubungi minggu ini?"*
> Di sinilah analisis deskriptif mencapai batasnya. Kita bisa mengidentifikasi bahwa **pelanggan bulanan baru berisiko**, tetapi: - Dari 1.800 pelanggan di segmen itu, mana 200 yang paling mungkin churn bulan ini? - Bagaimana kita memprioritaskan intervensi ketika anggaran terbatas? - Bagaimana kita memperkirakan probabilitas churn untuk

pelanggan **baru** yang belum kita kenal?
Pertanyaan-pertanyaan ini **tidak dapat dijawab** oleh statistik deskriptif atau diagnostik—betapapun canggihnya. Mereka membutuhkan **model prediktif** yang dapat memperkirakan probabilitas churn **untuk setiap pelanggan**, berdasarkan pola yang telah kita identifikasi di Bagian I.
Inilah tujuan Bagian II. Kita akan menggunakan semua insight dari Bagian I—variabel mana yang penting, segmen mana yang berisiko, pola mana yang muncul—sebagai **fondasi untuk membangun model** yang dapat memprediksi churn secara individual. Model itu sendiri akan menjadi input untuk **optimasi di Bagian III**: siapa yang diintervensi, dengan apa, dan berapa anggarannya.

6.10 Latihan Terbimbing

Latihan 6.1 — SCQA untuk skenario baru. Bayangkan Anda bekerja di perusahaan retail (bukan Telko) yang ingin memahami mengapa penjualan menurun. Susun SCQA untuk skenario ini—Situation, Complication, Question, dan Answer (hipotesis awal). Anda belum perlu data.

Latihan 6.2 — One-pager eksekutif. Buat satu halaman (300 kata) ringkasan untuk VP Customer Success berdasarkan analisis di bab ini. Harus mencakup: (a) temuan utama dalam satu kalimat, (b) tiga bukti pendukung, (c) tiga rekomendasi, (d) satu slide batasan. Gunakan bahasa non-teknis.

Latihan 6.3 — Kritik dashboard. Ambil dashboard eksekutif yang saya buat di bab ini. Kritik secara konstruktif: apa yang sudah baik, dan apa yang bisa diperbaiki? Pertimbangkan prinsip dari Bab 4 (data-ink ratio, chart selection) dan prinsip storytelling di bab ini.

Latihan 6.4 — Komunikasi ketidakpastian. Tulis dua paragraf yang mengomunikasikan estimasi "revenue berisiko Rp X miliar" kepada audiens eksekutif. Paragraf pertama harus **percaya diri namun jujur**; paragraf kedua harus **terlalu hati-hati** sehingga melemahkan pesan. Bandingkan keduanya—mana yang lebih efektif, dan mengapa?

Latihan 6.5 — Jembatan ke prediksi. Berdasarkan temuan Bagian I, tuliskan tiga **pertanyaan spesifik** yang menurut Anda hanya dapat dijawab oleh model prediktif. Untuk setiap pertanyaan, jelaskan mengapa statistik deskriptif tidak cukup.

> **Note**
>
> **Petunjuk Latihan 6.5.** Pertanyaan yang baik bersifat **individual** dan **prediktif**. Contoh: *"Pelanggan mana dari 1.800 pelanggan bulanan baru yang paling mungkin churn dalam 30 hari ke depan?"* — ini membutuhkan model yang dapat menghasilkan skor probabilitas per pelanggan. Statistik deskriptif hanya bisa mengatakan "segmen ini

berisiko tinggi", bukan "pelanggan A berisiko 78%, pelanggan B berisiko 12%".

6.11 Rangkuman

Dalam bab ini kita telah:

- **Menerapkan kerangka SCQA** untuk menyusun narasi analitis dari konteks ke rekomendasi—sebelum menyentuh data.
- **Menggabungkan seluruh teknik Bagian I**—wrangling, statistik deskriptif, visualisasi, diagnosis—dalam satu alur kerja end-to-end yang menjawab pertanyaan bisnis nyata.
- **Menerjemahkan temuan statistik ke dampak finansial**, menyadari bahwa prioritas segmen bisa berubah ketika revenue diperhitungkan.
- **Menerapkan prinsip data storytelling**: piramida terbalik, aturan 1-3-1, analogi bukan jargon.
- **Merancang dashboard eksekutif** yang menyoroti tiga KPI utama dan tiga grafik yang menjawab "siapa, mengapa, berapa".
- **Menyusun rekomendasi prioritas** dengan estimasi dampak dan pengakuan bahwa rekomendasi adalah hipotesis yang perlu diuji.
- **Mengomunikasikan batas analisis** secara eksplisit sebagai tanda kedewasaan analitis, bukan kelemahan.
- **Mengidentifikasi batas analisis deskriptif**: ia menjawab pertanyaan tingkat populasi, tetapi tidak dapat memberikan prediksi tingkat individu—dan inilah jembatan ke Bagian II.

6.12 Penutup Bagian I

Kita telah menyelesaikan **Bagian I — Descriptive & Diagnostic Analytics**. Mari kita lihat kembali perjalanan ini:

Bab	Yang Kita Pelajari
1	Fondasi: Python, pandas, dataset pendamping, lima langkah inspeksi
2	Wrangling: nilai hilang, duplikat, outlier, feature engineering dasar
3	Statistik: ukuran pusat, sebaran, bentuk, uji hipotesis, effect size
4	Visualisasi: grammar of graphics, prinsip Tufte, matplotlib, plotly, dashboard

Bab	Yang Kita Pelajari
5	Diagnostik: stratifikasi, Simpson's Paradox, kohort, root cause analysis
6	Studi kasus: SCQA, storytelling, dashboard eksekutif, rekomendasi

Sekarang kita memiliki: - **Dataset bersih** (`data/processed/telco_clean.csv`) dengan 21 kolom siap analisis. - **Pemahaman mendalam** tentang churn: siapa yang berisiko, mengapa, dan berapa dampaknya. - **Keterampilan komunikasi** untuk menyampaikan insight kepada pengambil keputusan. - **Hipotesis yang dapat diuji** untuk intervensi retensi.

Yang **belum** kita miliki: - **Model prediktif** yang dapat memprediksi churn untuk pelanggan individual. - **Kemampuan untuk mengoptimalkan** alokasi sumber daya retensi. - **Simulasi ketidakpastian** untuk mengevaluasi kebijakan sebelum implementasi.

Ketiganya adalah **inti Bagian II dan III**.

6.13 Jembatan ke Bagian II

Bagian I telah mengajarkan Anda untuk **melihat** dan **memahami** data. Bagian II akan mengajarkan Anda untuk **memprediksi**.

Perjalanan Bagian II:

- **Bab 7 — Kerangka Predictive**: dari deskriptif ke prediktif; bias-variance tradeoff; train-test split.
- **Bab 8 — Regresi**: memodelkan hubungan kontinu; interpretasi koefisien; diagnostik residual.
- **Bab 9 — Klasifikasi**: memprediksi probabilitas churn individual; metrik evaluasi; penanganan kelas tidak seimbang.
- **Bab 10 — Time Series**: deret waktu; forecasting; analisis survival (dengan dataset retail pendamping).
- **Bab 11 — Unsupervised**: clustering pelanggan; segmentasi berbasis data; PCA.
- **Bab 12 — Monte Carlo**: simulasi ketidakpastian; jembatan ke simulasi keputusan di Bagian III.

Setiap bab di Bagian II akan merujuk kembali ke temuan Bagian I. Variabel yang penting dalam diagnosis (Contract, tenure, InternetService) akan menjadi **fitur** dalam model. Segmen yang kita identifikasi akan divalidasi oleh model. Hipotesis yang kita bentuk akan diuji secara empiris.

Selamat datang di Bagian II.

Part II

Bagian II — Predictive Analytics

7 Kerangka Predictive Analytics

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membedakan pertanyaan prediktif dari pertanyaan deskriptif dan merumuskannya secara operasional. 2. Menjelaskan alur kerja pemodelan prediktif—dari formulasi masalah hingga evaluasi. 3. Membagi data menjadi *train*, *validation*, dan *test* dengan benar, serta memahami **mengapa** tiga pembagian diperlukan. 4. Menerapkan *cross-validation* untuk estimasi performa yang lebih stabil. 5. Menjelaskan **bias-variance tradeoff** dan mengenali tanda-tanda *overfitting* maupun *underfitting*. 6. Mengenali dan menghindari *data leakage*—kesalahan paling berbahaya dalam pemodelan prediktif. 7. Membangun *baseline model* sederhana yang menjadi pembanding untuk semua model di bab-bab selanjutnya.

7.1 Dari "Mengapa" ke "Siapa"

Di akhir Bagian I, kita berhenti pada sebuah batas yang jelas. Diagnosis kita mengatakan: *"Pelanggan bulanan baru berisiko tinggi."* Itu adalah kesimpulan tingkat **populasi**—benar untuk kelompok, tetapi tidak cukup untuk tindakan.

VP Customer Success tidak bisa menghubungi "segmen pelanggan bulanan baru". Ia harus menghubungi **individu**—dan ia tidak punya anggaran untuk menghubungi semuanya. Ia membutuhkan jawaban atas pertanyaan yang berbeda:

> *"Dari 1.800 pelanggan di segmen itu, mana 200 yang paling mungkin churn bulan ini?"*

Perhatikan pergeseran pertanyaannya:

Bagian I	Bagian II
"Segmen mana yang berisiko?"	"Pelanggan mana yang akan churn?"
Tingkat populasi	Tingkat individu
Rata-rata kelompok	Probabilitas per pelanggan
Deskriptif/diagnostik	Prediktif

Inilah wilayah **predictive analytics**: menggunakan pola yang telah kita identifikasi pada data historis untuk memperkirakan **hasil di masa depan** untuk **individu tertentu**.

> **!** Important
>
> **Prediksi bukan ramalan.** Model prediktif tidak "melihat masa depan". Ia menemukan pola dalam data historis dan mengasumsikan bahwa pola itu **bertahan**. Jika dunia berubah secara fundamental—kompetitor baru, resesi, perubahan regulasi—prediksi bisa gagal. Karena itu, model prediktif selalu merupakan **kondisional**: prediksi valid **jika** kondisi yang melatihnya masih berlaku.

7.2 Formulasi Masalah: Sebelum Menyentuh Kode

Kesalahan paling umum dalam pemodelan prediktif bukanlah algoritma yang salah, melainkan **masalah yang salah dirumuskan**. Sebelum membuka notebook, jawab lima pertanyaan berikut secara eksplisit.

7.2.1 1. Apa yang Diprediksi? (Target)

Untuk kasus kita, targetnya jelas: `Churn` (biner: `Yes`/`No`). Tetapi ini adalah pilihan yang perlu dijelaskan—mengapa biner, bukan kontinu?

Jenis target	Contoh	Algoritma yang sesuai
Biner	Churn / tidak churn	Klasifikasi: logistic regression, random forest, gradient boosting
Multikelas	Jenis kontrak yang akan dipilih	Klasifikasi multikelas
Kontinu	Nilai tagihan bulanan berikutnya	Regresi: linear regression, gradient boosting regressor
Hitungan	Jumlah bulan sebelum churn	Regresi Poisson, survival analysis
Waktu	Kapan pelanggan akan churn	Survival analysis (Bab 10)

Target biner memungkinkan kita membangun model **klasifikasi**—fokus Bab 9. Tetapi penting dicatat: **target biner menyederhanakan realitas**. Pelanggan tidak "churn" pada satu titik; mereka melalui proses. Bab 10 akan memperkenalkan pendekatan yang lebih kaya (survival analysis) sebagai pelengkap.

7.2.2 2. Untuk Siapa Prediksi Ini? (Audiens)

Prediksi untuk **tim retensi** berbeda dari prediksi untuk **direksi**:

Audiens	Butuh apa	Bentuk keluaran
Tim retensi operasional	Daftar pelanggan untuk dihubungi minggu ini	Skor probabilitas + ranking
Manajer retensi	Segmentasi risiko untuk alokasi anggaran	Distribusi skor, segmen risiko
Direksi	Dampak finansial dan tren	Agregat, forecast, confidence interval

Buku ini fokus pada keluaran **skor probabilitas per pelanggan**—yang dapat diturunkan menjadi ketiganya.

7.2.3 3. Kapan Prediksi Dibutuhkan? (Waktu)

Prediksi churn untuk **bulan depan** berbeda dari prediksi untuk **tahun depan**:

- **Horizon pendek** (1 bulan): Model dapat menggunakan fitur yang lebih spesifik dan terbaru (mis. jumlah keluhan terakhir).
- **Horizon panjang** (12 bulan): Model harus lebih umum; banyak fitur spesifik menjadi noise.

Model prediksi churn biasanya memiliki horizon 30–90 hari—cukup lama untuk intervensi, cukup pendek agar prediksi tetap relevan.

7.2.4 4. Apa Konsekuensi Kesalahan? (Cost Matrix)

Setiap model membuat dua jenis kesalahan:

	Aktual: Churn	Aktual: Tidak Churn
Prediksi: Churn	True Positive (TP)	False Positive (FP)
Prediksi: Tidak Churn	False Negative (FN)	True Negative (TN)

- **False Positive**: kita menghubungi pelanggan yang tidak akan churn. Biaya: waktu tim retensi, potensi mengganggu pelanggan.
- **False Negative**: kita melewatkan pelanggan yang akan churn. Biaya: kehilangan revenue, kehilangan pelanggan.

Dalam bisnis nyata, **FN biasanya jauh lebih mahal** dari FP—kehilangan pelanggan seharga LTV jauh lebih besar dari biaya satu panggilan telepon. Ini akan memengaruhi **metrik** yang kita optimalkan (Bab 9) dan **threshold** yang kita pilih untuk keputusan.

7.2.5 5. Apa Baseline yang Harus Dikalahkan? (Baseline)

Sebelum membangun model apa pun, kita perlu **baseline**—prediksi paling sederhana yang menjadi pembanding:

- **Baseline trivial**: prediksi semua pelanggan sebagai "tidak churn". Akurasi ~73,5% (karena 73,5% pelanggan tidak churn), tetapi **tidak berguna**—kita tidak menangkap satu pun pelanggan yang churn.
- **Baseline naif**: prediksi "churn" jika `Contract == "Month-to-month"`. Churn rate prediksi ~43%, akurasi tidak sebaik baseline trivial, tetapi **jauh lebih berguna** untuk retensi.

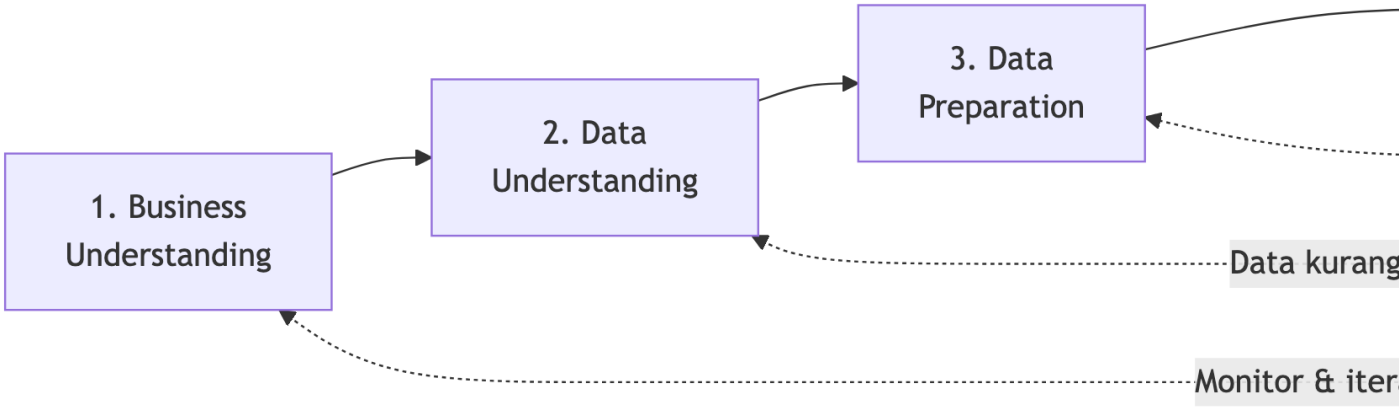
Model yang baik harus **secara signifikan mengalahkan baseline naif** dalam metrik yang relevan—bukan hanya dalam akurasi.

> Tip
>
> **Aturan penting.** Sebelum membangun model apa pun, tuliskan **baseline** yang akan Anda kalahkan. Ini mencegah Anda membangun model canggih yang sebenarnya lebih buruk dari aturan sederhana. Banyak proyek gagal karena tidak pernah menetapkan baseline.

7.3 Alur Kerja Pemodelan Prediktif

Proses pemodelan prediktif bukan satu langkah, melainkan **siklus iteratif** yang dikenal sebagai **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*):

```
flowchart LR
    A[1. Business<br/>Understanding] --> B[2. Data<br/>Understanding]
    B --> C[3. Data<br/>Preparation]
    C --> D[4. Modeling]
    D --> E[5. Evaluation]
    E --> F[6. Deployment]
    E -.->|Tidak memuaskan| C
    E -.->|Data kurang| B
    F -.->|Monitor & iterasi| A
```



Kita telah menyelesaikan tahap 1–3 di Bagian I (business understanding melalui SCQA, data understanding & preparation melalui Bab 1–2). Bab 7–9 akan fokus pada **tahap 4–5** (modeling dan evaluation); tahap 6 (deployment) akan disinggung di Bab 18.

7.4 Membagi Data: Train, Validation, Test

Ini adalah **keputusan paling penting** dalam pemodelan—dan salah satu yang paling sering salah dilakukan.

7.4.1 Mengapa Kita Tidak Bisa Mengukur Performa pada Data Latih?

Jika kita melatih model pada seluruh dataset dan mengukur performanya pada dataset yang sama, kita akan mendapat gambaran yang terlalu optimis. Model **menghafal** data latih—termasuk noise dan outlier—sehingga performanya pada data latih bisa jauh lebih baik daripada performa pada data baru.

Analoginya: seorang siswa yang belajar dengan menghafal seluruh jawaban soal tahun lalu akan tampak jenius ketika diuji dengan soal yang sama, tetapi mungkin gagal ketika diuji dengan soal baru. **Kita ingin model yang belajar *pola*, bukan *menghafal data*.**

7.4.2 Mengapa Tiga Pembagian, Bukan Dua?

Pembagian umum adalah **train (60–80%)**, **validation (10–20%)**, **test (10–20%)**. Masing-masing memiliki peran yang berbeda:

Set	Fungsi	Kapan dipakai
Train	Melatih model	Selama proses modeling
Validation	Memilih model & hyperparameter	Selama proses modeling
Test	Estimasi performa final	**Hanya sekali**, di akhir

Kesalahan klasik: menggunakan test set untuk "mencoba beberapa model dan memilih yang terbaik". Jika Anda melakukannya, test set bukan lagi test set—ia menjadi validation set kedua, dan estimasi performa Anda menjadi bias optimis.

> ⚠ Warning
>
> **Aturan emas test set.** Test set hanya boleh disentuh **satu kali**, di akhir, untuk mengukur performa final. Jika Anda melihat hasil test set dan kembali mengubah model, Anda telah melakukan *test set leakage*—estimasi performa Anda tidak akan bertahan ketika model digunakan di dunia nyata.

7.4.3 Menerapkan Pembagian Data

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np

from decanal.utils import load_churn_clean

df = load_churn_clean()

# Konvensi buku ini: random_state=42 di mana-mana
RANDOM_STATE = 42
TARGET = "Churn"

# Pisahkan fitur (X) dan target (y)
X = df.drop(columns=[TARGET])
y = df[TARGET]

print(f"Fitur : {X.shape[1]} kolom")
print(f"Target: {y.name} (distribusi: {dict(y.value_counts())})")
```

```
Fitur : 22 kolom
Target: Churn (distribusi: {'No': np.int64(5174), 'Yes': np.int64(1869)})
```

```python
from sklearn.model_selection import train_test_split

# Langkah 1: pisahkan test set (20%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,     # ← penting! lihat catatan di bawah
)

# Langkah 2: dari sisa 80%, pisahkan validation set (25% dari 80% = 20% dari total)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
    test_size=0.25,
    random_state=RANDOM_STATE,
    stratify=y_temp,
)

print(f"Train set     : {len(X_train):,} ({len(X_train)/len(X):.0%})")
print(f"Validation set: {len(X_val):,} ({len(X_val)/len(X):.0%})")
print(f"Test set      : {len(X_test):,} ({len(X_test)/len(X):.0%})")
```

```
Train set     : 4,225 (60%)
Validation set: 1,409 (20%)
Test set      : 1,409 (20%)
```

> **!** Important
>
> **Parameter `stratify=y`.** Tanpa stratifikasi, pembagian acak bisa menghasilkan distribusi kelas yang tidak seimbang antara train dan test—misalnya, train memiliki 25% churn tetapi test memiliki 30%. Dengan `stratify=y`, proporsi kelas dipertahankan di semua subset. Ini **wajib** untuk dataset dengan kelas tidak seimbang seperti Telco (26,5% churn).

7.4.4 Verifikasi: Apakah Distribusi Konsisten?

```
def churn_rate(s: pd.Series) -> str:
    return f"{(s == 'Yes').mean() * 100:.1f}%"

pd.DataFrame({
    "set": ["train", "validation", "test"],
    "n": [len(y_train), len(y_val), len(y_test)],
    "churn_rate": [churn_rate(y_train), churn_rate(y_val), churn_rate(y_test)],
})
```

	set	n	churn_rate
0	train	4225	26.5%
1	validation	1409	26.5%
2	test	1409	26.5%

Jika churn rate di ketiga set kira-kira sama (~26,5%), pembagian berhasil. Jika tidak, ada masalah dengan stratifikasi.

7.5 Cross-Validation: Estimasi yang Lebih Stabil

Pembagian train/validation tunggal memiliki kelemahan: hasilnya bergantung pada **pembagian acak tertentu**. Jika kita mengulang dengan `random_state` berbeda, hasilnya bisa berubah beberapa persen—terutama jika dataset kecil.

K-fold cross-validation mengatasi ini dengan membagi data latih menjadi K bagian, melatih model K kali (masing-masing menggunakan K-1 bagian sebagai training dan 1 bagian sebagai validation), lalu merata-ratakan hasilnya.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 3))
for fold in range(5):
    for i in range(5):
        color = "#DD8452" if i == fold else "#4C72B0"
        ax.barh(fold, 1, left=i, color=color, edgecolor="white")
    ax.text(5.1, fold, f"iterasi {fold+1}", va="center", fontsize=9)

ax.set_yticks(range(5))
ax.set_yticklabels([f"Fold {i+1}" for i in range(5)])
ax.set_xticks(np.arange(0.5, 5.5, 1))
```

```
ax.set_xticklabels([f"Bagian {i+1}" for i in range(5)])
ax.set_xlim(0, 6)
ax.set_title("5-Fold Cross-Validation")
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```

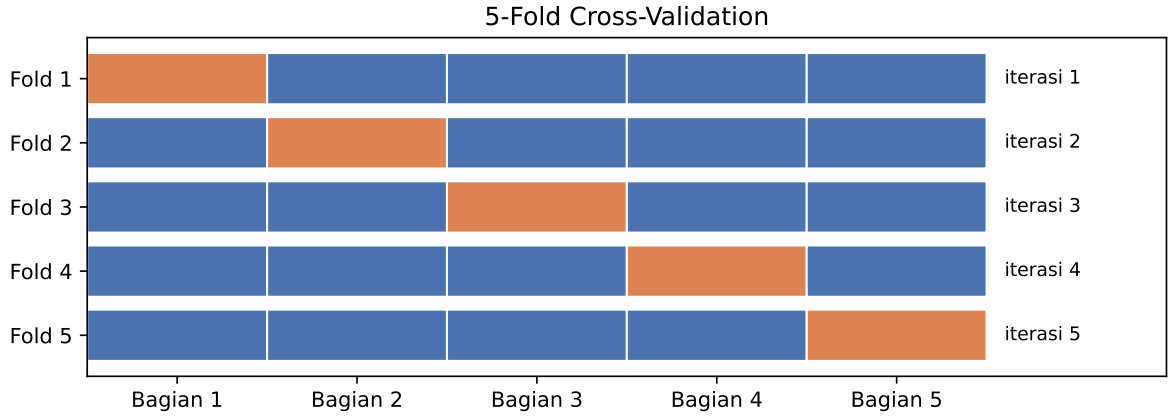


Figure 7.1: Ilustrasi 5-fold cross-validation: model dilatih 5 kali, setiap kali dengan subset validasi yang berbeda.

Oranye = validation pada iterasi tersebut. Biru = training. Model dilatih 5 kali, setiap kali subset validasi berbeda. Rata-rata performa dari 5 iterasi lebih stabil daripada satu pembagian tunggal.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

# Siapkan preprocessing dasar
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
```

```
        ("scaler", StandardScaler()),
    ]), numeric_features),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical_features),
])

# Pipeline: preprocessing + model
baseline_pipeline = Pipeline([
    ("prep", preprocessor),
    ("clf", LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)),
])

# 5-fold CV pada train set
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scores = cross_val_score(baseline_pipeline, X_train, y_train,
                         cv=cv, scoring="roc_auc")

print(f"ROC-AUC per fold: {[f'{s:.3f}' for s in scores]}")
print(f"Rata-rata       : {scores.mean():.3f} ± {scores.std():.3f}")
```

```
ROC-AUC per fold: ['0.837', '0.868', '0.834', '0.858', '0.853']
Rata-rata       : 0.850 ± 0.013
```

> **Tip**
>
> **Mengapa ROC-AUC dan bukan akurasi?** Karena kelas kita tidak seimbang (26,5% churn). Model yang selalu memprediksi "tidak churn" mendapat akurasi 73,5% tetapi ROC-AUC 0,5 (setara tebakan acak). ROC-AUC mengukur **kemampuan membedakan** antara kelas positif dan negatif—jauh lebih informatif untuk keputusan. Detail metrik akan dibahas di Bab 9.

7.6 Bias-Variance Tradeoff: Inti dari Semua Pemodelan

Bias-variance tradeoff adalah konsep paling fundamental dalam machine learning—dan salah satu yang paling sering disalahpahami. Ia menjelaskan **mengapa** model bisa gagal dengan dua cara yang berbeda.

7.6.1 Dua Jenis Kesalahan

Bayangkan kita membangun model untuk memprediksi churn. Kesalahan model dapat diuraikan menjadi tiga komponen:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Komponen	Arti	Penyebab
Bias	Seberapa jauh prediksi rata-rata model dari nilai sebenarnya	Model terlalu sederhana—tidak cukup fleksibel menangkap pola
Variance	Seberapa banyak prediksi model berubah jika data latih berubah	Model terlalu kompleks—menghafal noise dalam data latih
Irreducible Error	Noise yang tidak bisa dihilangkan	Batas fundamental dari data—tidak bisa dikurangi oleh model apa pun

7.6.2 Underfitting vs Overfitting

	Underfitting	Overfitting
Bias	Tinggi	Rendah
Variance	Rendah	Tinggi
Performa train	Buruk	Sangat baik
Performa test	Buruk	Buruk
Penyebab	Model terlalu sederhana	Model terlalu kompleks
Solusi	Model lebih kompleks, fitur lebih banyak	Regularisasi, lebih banyak data, model lebih sederhana

Analogi. Seorang siswa yang hanya membaca satu halaman buku akan gagal ujian karena **tidak cukup belajar** (underfitting). Seorang siswa yang menghafal seluruh buku tetapi tidak memahami konsep akan gagal ujian dengan soal baru karena **terlalu menghafal** (overfitting). Yang kita inginkan adalah siswa yang memahami pola—cukup belajar untuk menangkap konsep, tetapi tidak terlalu terikat pada detail spesifik.

7.6.3 Demonstrasi: Overfitting pada Decision Tree

Mari kita lihat overfitting secara langsung. Kita akan melatih decision tree dengan **kedalaman berbeda** dan membandingkan performanya pada train dan validation set.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_auc_score

depths = [1, 2, 3, 5, 7, 10, 15, 20, None]
hasil = []

for depth in depths:
    pipe = Pipeline([
        ("prep", preprocessor),
        ("clf", DecisionTreeClassifier(max_depth=depth, random_state=RANDOM_STATE)),
    ])
    pipe.fit(X_train, y_train)

    train_auc = roc_auc_score(y_train, pipe.predict_proba(X_train)[:, 1])
    val_auc = roc_auc_score(y_val, pipe.predict_proba(X_val)[:, 1])

    hasil.append({
        "max_depth": depth if depth is not None else "tanpa batas",
        "train_AUC": round(train_auc, 3),
        "val_AUC": round(val_auc, 3),
        "selisih": round(train_auc - val_auc, 3),
    })

pd.DataFrame(hasil)
```

	max_depth	train_AUC	val_AUC	selisih
0	1	0.734	0.715	0.019
1	2	0.796	0.780	0.016
2	3	0.829	0.808	0.020
3	5	0.858	0.814	0.044
4	7	0.897	0.790	0.108
5	10	0.962	0.750	0.213
6	15	0.998	0.678	0.319
7	20	1.000	0.657	0.343
8	tanpa batas	1.000	0.654	0.346

```
plot_data = pd.DataFrame(hasil)
plot_data["depth_num"] = range(1, len(plot_data) + 1)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(plot_data["depth_num"], plot_data["train_AUC"], marker="o",
        label="Train AUC", color="#DD8452", linewidth=2)
ax.plot(plot_data["depth_num"], plot_data["val_AUC"], marker="s",
        label="Validation AUC", color="#4C72B0", linewidth=2)

# Anotasi zona
ax.axvspan(0.5, 2.5, alpha=0.1, color="orange")
ax.axvspan(6.5, len(plot_data) + 0.5, alpha=0.1, color="red")
ax.text(1.5, 0.55, "Underfitting", ha="center", fontsize=9, color="darkorange")
ax.text(8, 0.85, "Overfitting", ha="center", fontsize=9, color="firebrick")

ax.set_xticks(plot_data["depth_num"])
ax.set_xticklabels(plot_data["max_depth"])
ax.set_xlabel("Kedalaman pohon (max_depth)")
ax.set_ylabel("ROC-AUC")
ax.set_title("Bias-Variance Tradeoff pada Decision Tree")
ax.legend(loc="lower right")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

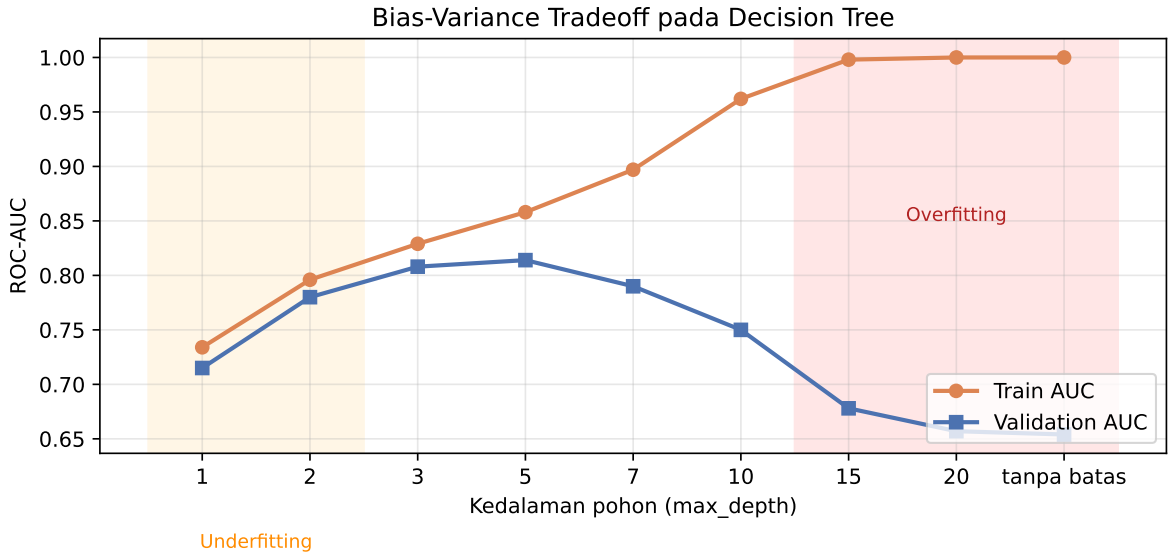


Figure 7.2: Bias-variance tradeoff. Model dengan kedalaman kecil underfitting (kedua kurva rendah). Model dengan kedalaman besar overfitting (train tinggi, validation menurun).

Cara membaca grafik ini:

- **Kedalaman 1–2 (underfitting)**: Train dan validation AUC keduanya rendah—model terlalu sederhana untuk menangkap pola.
- **Kedalaman 3–7 (zona ideal)**: Train dan validation AUC naik bersama—model menangkap pola tanpa menghafal noise. Titik di mana **validation AUC tertinggi** adalah kandidat model terbaik.
- **Kedalaman 10+ (overfitting)**: Train AUC terus naik menuju 1,0, tetapi **validation AUC mulai menurun**—model menghafal noise dalam train set, dan performanya pada data baru memburuk.

> **!** Important
>
> **Cara mendeteksi overfitting.** Tanda paling jelas adalah **selisih besar antara performa train dan validation**. Jika train AUC 0,99 tetapi validation AUC 0,75, model Anda overfitting. Selisih 5–10% masih wajar; selisih >15% adalah tanda bahaya. **Cara mendeteksi underfitting.** Jika **kedua** performa (train dan validation) rendah dan tidak ada model sederhana yang berhasil, model Anda underfitting. Solusinya bukan regularisasi—tetapi model yang lebih kompleks.

7.7 Data Leakage: Musuh yang Tidak Terlihat

Data leakage adalah kondisi ketika model tidak sengaja memiliki akses ke informasi yang **tidak akan tersedia** pada saat prediksi di dunia nyata. Ini adalah kesalahan paling berbahaya dalam pemodelan prediktif, karena model akan tampak **sangat baik** selama pengujian tetapi **gagal total** ketika diterapkan.

7.7.1 Contoh Data Leakage

Jenis Leakage	Contoh	Mengapa berbahaya
Target leakage	Fitur `TotalCharges` dihitung **setelah** pelanggan churn (mencakup tagihan terakhir)	Model akan belajar bahwa "TotalCharges tinggi = churn" padahal sebenarnya itu adalah konsekuensi churn
Temporal leakage	Model menggunakan data 2024 untuk memprediksi 2023	Di dunia nyata, kita tidak punya data masa depan
Preprocessing leakage	Menstandardisasi fitur pada **seluruh** dataset, lalu membagi train/test	Statistik (mean, std) dari test bocor ke preprocessing—model tampak lebih baik daripada seharusnya
Group leakage	Data pelanggan yang sama muncul di train dan test	Model menghafal pelanggan tersebut, bukan pola umum

7.7.2 Contoh Temporal Leakage di Dataset Telco

Dataset Telco tidak memiliki timestamp, tetapi kita bisa membayangkan leakagenya. Misalnya:

```
# JANGAN LAKUKAN INI - contoh anti-pattern
X_leaky = X.copy()
# Bayangkan kita punya kolom "last_complaint_date" yang hanya diisi
# SETELAH pelanggan menyatakan churn.
# X_leaky["had_complaint"] = (df["last_complaint_date"].notna()).astype(int)
# Model akan mendapat AUC 0.99 - tetapi tidak akan berguna di dunia nyata.
```

Cara mendeteksi leakage. Tanyakan untuk setiap fitur: *"Apakah nilai fitur ini akan tersedia pada saat prediksi?"* Jika tidak, fitur tersebut tidak boleh digunakan.

7.7.3 Mencegah Preprocessing Leakage

Kesalahan klasik: menstandardisasi data **sebelum** train/test split. Cara yang benar: preprocessing **di dalam pipeline**, sehingga hanya melihat train set saat fitting.

```
# BENAR: preprocessing di dalam pipeline
# Pipeline hanya "melihat" train set saat .fit()
# Saat .predict() pada test set, scaler menggunakan statistik dari train
# (bukan dari test) - sehingga tidak ada leakage.

print("Pipeline yang kita bangun di atas:")
print("  1. SimpleImputer menghitung median dari X_train saja")
print("  2. StandardScaler menghitung mean/std dari X_train saja")
print("  3. OneHotEncoder mempelajari kategori dari X_train saja")
print()
print("→ Semua statistik diturunkan dari train, diterapkan ke test.")
print("→ Tidak ada informasi test yang 'bocor' ke preprocessing.")
```

```
Pipeline yang kita bangun di atas:
  1. SimpleImputer menghitung median dari X_train saja
  2. StandardScaler menghitung mean/std dari X_train saja
  3. OneHotEncoder mempelajari kategori dari X_train saja

→ Semua statistik diturunkan dari train, diterapkan ke test.
→ Tidak ada informasi test yang 'bocor' ke preprocessing.
```

> ⚠ Warning
>
> **Aturan praktis. Jangan pernah** melakukan preprocessing (imputasi, scaling, encoding) di luar pipeline. Bahkan jika hasilnya tampak sama, ia menciptakan risiko leakage yang tidak terlihat. Pipeline scikit-learn ada **persis** untuk mencegah kesalahan ini—gunakan selalu.

7.8 Baseline Model: Yang Harus Dikalahkan

Dengan kerangka di atas, mari kita bangun **baseline**—model paling sederhana yang masuk akal. Baseline ini akan menjadi pembanding untuk model yang lebih canggih di Bab 8–9.

```python
# Latih pada train set, evaluasi pada validation set
baseline_pipeline.fit(X_train, y_train)

# Prediksi
y_train_pred = baseline_pipeline.predict_proba(X_train)[:, 1]
y_val_pred = baseline_pipeline.predict_proba(X_val)[:, 1]

# Evaluasi
from sklearn.metrics import roc_auc_score, accuracy_score

print("=== Baseline: Logistic Regression + Preprocessing ===")
print(f"Train AUC     : {roc_auc_score(y_train, y_train_pred):.3f}")
print(f"Validation AUC: {roc_auc_score(y_val, y_val_pred):.3f}")
print(f"Train accuracy: {accuracy_score(y_train, baseline_pipeline.predict(X_train)):.3f}")
print(f"Val accuracy  : {accuracy_score(y_val, baseline_pipeline.predict(X_val)):.3f}")
```

```
=== Baseline: Logistic Regression + Preprocessing ===
Train AUC     : 0.856
Validation AUC: 0.839
Train accuracy: 0.812
Val accuracy  : 0.805
```

```python
from sklearn.metrics import accuracy_score, roc_auc_score

# Konversi target ke biner 0/1 SEKALI di sini
y_val_bin = (y_val == "Yes").astype(int)
y_train_bin = (y_train == "Yes").astype(int)

# Trivial: selalu prediksi "No" (0)
trivial_pred = np.zeros(len(y_val_bin), dtype=int)
trivial_accuracy = accuracy_score(y_val_bin, trivial_pred)
trivial_auc = 0.5

# Naif: prediksi "Yes" (1) jika Contract == Month-to-month
naive_pred = (X_val["Contract"] == "Month-to-month").astype(int).values
naive_accuracy = accuracy_score(y_val_bin, naive_pred)
naive_auc = roc_auc_score(y_val_bin, naive_pred)

# Baseline LogReg:
#   - predict()       → string "Yes"/"No"  → bandingkan ke "Yes" agar jadi 0/1
#   - predict_proba() → float 0-1          → langsung dipakai untuk AUC
```

```
baseline_accuracy = accuracy_score(
    y_val_bin,
    baseline_pipeline.predict(X_val) == "Yes"
)
baseline_auc = roc_auc_score(y_val_bin, y_val_pred)

pd.DataFrame({
    "Model": ["Trivial (selalu 'No')", "Naif (Contract)", "Baseline (LogReg)"],
    "Val accuracy": [
        f"{trivial_accuracy:.1%}",
        f"{naive_accuracy:.1%}",
        f"{baseline_accuracy:.1%}",
    ],
    "Val ROC-AUC": [
        f"{trivial_auc:.3f}",
        f"{naive_auc:.3f}",
        f"{baseline_auc:.3f}",
    ],
})
```

	Model	Val accuracy	Val ROC-AUC
0	Trivial (selalu 'No')	73.5%	0.500
1	Naif (Contract)	64.4%	0.715
2	Baseline (LogReg)	80.5%	0.839

Perhatikan pola penting:

- **Baseline trivial** memiliki akurasi tertinggi (73,5%) tetapi AUC terburuk (0,5). Ini menunjukkan bahwa **akurasi bukan metrik yang tepat** untuk kelas tidak seimbang.
- **Baseline naif** mengalahkan trivial dalam AUC tetapi kalah dalam akurasi. Ini adalah ilustrasi sempurna bahwa **metrik yang Anda pilih menentukan model yang Anda pilih**.
- **Baseline LogReg** mengalahkan keduanya dalam AUC—dan inilah **titik awal** yang harus dikalahkan oleh model yang lebih canggih.

> **!** Important
>
> **Peluang untuk model yang lebih baik.** Baseline LogReg mencapai AUC ~0,85. Model yang lebih canggih (random forest, gradient boosting) di Bab 9 mungkin mencapai AUC 0,88–0,90. Tetapi **peningkatan 3–5% ini tidak otomatis berarti lebih baik**—kita harus mempertimbangkan interpretabilitas, biaya komputasi, dan kemudahan deployment.

Sering kali, model yang lebih sederhana dipilih karena **lebih mudah dijelaskan kepada pemangku kepentingan**, meskipun performanya sedikit lebih rendah.

7.9 Latihan Terbimbing

Latihan 7.1 — Formulasi masalah. Untuk setiap skenario berikut, tentukan: (a) jenis target (biner/kontinu/multikelas), (b) metrik yang tepat, (c) baseline yang wajar. - (a) Memprediksi apakah pelanggan akan membayar tagihan tepat waktu. - (b) Memprediksi berapa banyak unit produk yang akan terjual besok. - (c) Memprediksi jenis kontrak yang akan dipilih pelanggan baru (bulanan/tahunan/dua tahun).

Latihan 7.2 — Split vs CV. Jelaskan dalam satu paragraf: mengapa kita membutuhkan **keduanya**—train/validation/test split **dan** cross-validation? Kapan menggunakan yang mana?

Latihan 7.3 — Deteksi overfitting. Hasil berikut berasal dari tiga model pada dataset yang sama:

Model	Train AUC	Val AUC
A	0,72	0,70
B	0,95	0,75
C	0,65	0,64

Model mana yang **overfitting**? Model mana yang **underfitting**? Model mana yang tampaknya paling seimbang? Jelaskan.

Latihan 7.4 — Identifikasi leakage. Untuk setiap fitur berikut, tentukan apakah aman digunakan untuk memprediksi churn, atau berpotensi leakage: - (a) `tenure` (bulan berlangganan) - (b) `total_support_tickets` (jumlah tiket dukungan yang dibuka **selama** berlangganan) - (c) `last_payment_date` (tanggal pembayaran terakhir) - (d) `account_status` (aktif/ditutup—hanya diperbarui setelah pelanggan churn) - (e) `monthly_charges`

Latihan 7.5 — Baseline comparison. Bangun baseline trivial kedua: prediksi "Yes" jika `tenure <= 6`. Bandingkan dengan baseline `Contract` yang sudah kita buat. Mana yang lebih baik dalam AUC? Mana yang lebih baik dalam akurasi? Mana yang akan Anda pilih untuk sistem retensi, dan mengapa?

> **Note**
>
> **Petunjuk Latihan 7.3.** Ingat definisi: - **Overfitting**: train tinggi, validation rendah, selisih besar. - **Underfitting**: train dan validation keduanya rendah. - **Seimbang**: train

dan validation tinggi, selisih kecil.
Model B overfitting (selisih 0,20). Model C underfitting (keduanya rendah). Model A paling seimbang (selisih 0,02). Tetapi "paling seimbang" tidak berarti "terbaik"—model A mungkin masih kalah dari model yang lebih kompleks jika model itu memiliki validation AUC yang lebih tinggi.

7.10 Kotak "Jembatan": Dari Data ke Keputusan

! Important

Jembatan. Bab ini memberi Anda **kerangka** yang akan menyertai seluruh Bagian II:

1. **Model bukan kotak hitam.** Setiap model memiliki asumsi, batasan, dan kegagalan. Kerangka bias-variance mengajarkan kita kapan model perlu dibuat lebih kompleks, dan kapan perlu disederhanakan.
2. **Metrik bukan tujuan.** Memaksimalkan AUC tanpa mempertimbangkan konteks bisnis adalah kesalahan. Model dengan AUC 0,85 tetapi interpretabilitas tinggi sering lebih berharga daripada model dengan AUC 0,90 tetapi tidak bisa dijelaskan ke pemangku kepentingan.
3. **Data leakage adalah ancaman nyata.** Model yang tampak sempurna dalam pengujian bisa gagal total dalam produksi jika ada leakage. Selalu tanyakan: *"Apakah informasi ini akan tersedia saat prediksi?"*
4. **Baseline bukan formalitas.** Baseline mengingatkan kita bahwa model yang canggih belum tentu lebih baik dari aturan sederhana. Banyak model machine learning yang dikembangkan dengan biaya besar akhirnya kalah dari aturan sederhana yang sudah dipakai tim bisnis selama bertahun-tahun.

Dengan kerangka ini, kita siap membangun model yang lebih canggih. Bab 8 akan fokus pada **regresi**—fondasi statistik untuk memodelkan hubungan antar variabel. Bab 9 akan fokus pada **klasifikasi**—yang secara langsung menjawab pertanyaan churn kita.

7.11 Rangkuman

Dalam bab ini kita telah:

- **Membedakan prediksi dari deskripsi**, memahami bahwa prediksi menjawab pertanyaan **tingkat individu** sementara deskripsi menjawab **tingkat populasi**.

- **Merumuskan masalah prediktif** dengan lima pertanyaan kunci: apa targetnya, untuk siapa, kapan, apa konsekuensi kesalahannya, dan apa baseline-nya.
- **Menerapkan CRISP-DM** sebagai kerangka kerja siklus pemodelan, dengan penekanan pada iterasi antara modeling dan evaluation.
- **Membagi data menjadi train/validation/test** dengan stratifikasi, memahami peran unik masing-masing subset dan aturan emas test set.
- **Menerapkan cross-validation** untuk estimasi performa yang lebih stabil, dan memahami kapan CV lebih tepat daripada single split.
- **Memahami bias-variance tradeoff** sebagai fondasi semua pemodelan, dan mendemonstrasikan overfitting secara langsung dengan decision tree.
- **Mengenali dan menghindari data leakage**—terutama dalam preprocessing—dengan menerapkan pipeline scikit-learn.
- **Membangun baseline model** (LogReg + preprocessing) yang mencapai AUC ~0,85, mengalahkan baseline trivial dan naif—dan menjadi patokan yang harus dikalahkan oleh model di bab selanjutnya.

7.12 Jembatan ke Bab 8

Kita memiliki kerangka. Sekarang kita membutuhkan model yang sesungguhnya. Bab 8—**Regresi**—akan memperkenalkan fondasi statistik untuk memodelkan hubungan antar variabel:

- **Regresi linear** untuk target kontinu, dengan interpretasi koefisien yang jelas.
- **Regresi logistik** untuk target biner—model yang baru saja kita pakai sebagai baseline, tetapi akan kita pelajari secara mendalam.
- **Diagnostik residual** untuk memeriksa apakah asumsi model terpenuhi.
- **Regularisasi** (Ridge, Lasso) untuk mengendalikan kompleksitas model.

Regresi bukan sekadar "model sederhana yang kalah dari random forest". Ia adalah **fondasi** dari sebagian besar machine learning modern, dan pemahaman mendalamnya akan membuat Anda lebih efektif dalam semua model yang lebih canggih.

8 Regresi: Fondasi Pemodelan Statistik

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membangun regresi linear untuk target kontinu dan menginterpretasikan koefisiennya secara bisnis. 2. Memeriksa asumsi regresi linear melalui diagnostik residual. 3. Mengenali multikolinearitas dan mengatasinya dengan regularisasi (Ridge, Lasso). 4. Membangun regresi logistik untuk target biner dan menginterpretasikan **odds ratio**. 5. Membedakan interpretasi koefisien linear dari koefisien logistik—dan memahami kapan masing-masing lebih tepat. 6. Menjelaskan mengapa regresi tetap relevan bahkan ketika model canggih tersedia.

8.1 Mengapa Regresi?

Di Bab 7 kita membangun baseline klasifikasi dengan Logistic Regression. Tetapi kita belum membahas **apa yang sedang dilakukan model itu**. Bab ini mengisi kekosongan tersebut.

Regresi adalah **fondasi** yang sering diremehkan. Ada tiga alasan mengapa ia tetap relevan di era gradient boosting dan deep learning:

1. **Interpretabilitas.** Koefisien regresi memiliki makna langsung: "setiap kenaikan 1 bulan tenure menurunkan log-odds churn sebesar 0,05". Model canggih jarang memberikan interpretasi sejelas ini.
2. **Inferensi, bukan hanya prediksi.** Regresi memungkinkan kita menjawab "seberapa yakin kita bahwa hubungan ini nyata?" melalui standard error dan confidence interval—bukan hanya "apa prediksinya".
3. **Fondasi untuk semua model.** Gradient boosting dengan loss function MSE pada dasarnya adalah regresi yang lebih fleksibel. Memahami regresi membantu Anda memahami apa yang terjadi di balik layar model yang lebih kompleks.

Tip

Dua wajah regresi. Dalam buku ini, regresi akan muncul dalam **dua peran**: - **Sebagai model prediktif** (fokus Bagian II): menghasilkan prediksi. - **Sebagai alat inferensi**

(fokus diagnostik): memahami hubungan antar variabel.
Bab ini akan bergantian antara keduanya—kadang kita peduli akurasi prediksi, kadang kita peduli makna koefisien.

8.2 Regresi Linear: Memodelkan Target Kontinu

8.2.1 Bentuk Umum

Regresi linear mengasumsikan bahwa target y adalah fungsi linear dari fitur X:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

- β_0 adalah **intercept** (nilai y ketika semua $x = 0$).
- β_j adalah **koefisien** untuk fitur x_j — perubahan y untuk setiap kenaikan satu unit x_j, **dengan menganggap fitur lain konstan** (inilah arti "ceteris paribus").
- ϵ adalah **residual** — bagian dari y yang tidak dijelaskan oleh model.

8.2.2 Contoh: Memprediksi Tagihan Bulanan

Sebagai latihan, mari kita gunakan regresi linear untuk memprediksi `MonthlyCharges` dari fitur-fitur lain. Ini bukan target yang biasa diprediksi—tetapi ia berguna untuk memperkenalkan konsep tanpa kompleksitas target biner.

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from decanal.utils import load_churn_clean
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

```
RANDOM_STATE = 42
df = load_churn_clean()

# Target kontinu
y_cont = df["MonthlyCharges"]
X_cont = df.drop(columns=["MonthlyCharges", "Churn"])
```

```
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_cont, y_cont, test_size=0.2, random_state=RANDOM_STATE
)

numeric_features = X_cont.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X_cont.select_dtypes(include=["object"]).columns.tolist()

preprocessor_lin = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), numeric_features),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(drop="first", handle_unknown="ignore")),
    ]), categorical_features),
])
```

8.2.3 Melatih Model dan Mengukur Performa

```
lin_pipe = Pipeline([
    ("prep", preprocessor_lin),
    ("reg", LinearRegression()),
])

lin_pipe.fit(X_train_c, y_train_c)
y_pred_train = lin_pipe.predict(X_train_c)
y_pred_test = lin_pipe.predict(X_test_c)

perf = pd.DataFrame({
    "Metrik": ["RMSE", "MAE", "R²"],
    "Train": [
```

```
        f"{np.sqrt(mean_squared_error(y_train_c, y_pred_train)):.2f}",
        f"{mean_absolute_error(y_train_c, y_pred_train):.2f}",
        f"{r2_score(y_train_c, y_pred_train):.3f}",
    ],
    "Test": [
        f"{np.sqrt(mean_squared_error(y_test_c, y_pred_test)):.2f}",
        f"{mean_absolute_error(y_test_c, y_pred_test):.2f}",
        f"{r2_score(y_test_c, y_pred_test):.3f}",
    ],
})
perf
```

	Metrik	Train	Test
0	RMSE	1.02	1.05
1	MAE	0.78	0.79
2	R^2	0.999	0.999

Cara membaca metrik:

- **RMSE** (Root Mean Squared Error): rata-rata kesalahan prediksi dalam satuan target. RMSE = 5,0 berarti model salah sekitar Rp 5.000 per prediksi.
- **MAE** (Mean Absolute Error): rata-rata kesalahan absolut. Lebih tahan outlier dari RMSE.
- **R^2**: proporsi varians target yang dijelaskan model. $R^2 = 0{,}85$ berarti 85% variasi `MonthlyCharges` dapat dijelaskan oleh fitur.

> **Note**
>
> **R^2 tidak sama dengan "akurasi".** Nilai R^2 bisa negatif (model lebih buruk dari prediksi rata-rata). $R^2 = 0{,}9$ tidak berarti "90% benar"—ia berarti "model menjelaskan 90% varians". Pada dataset dengan noise tinggi, $R^2 = 0{,}3$ bisa sangat baik.

8.3 Interpretasi Koefisien: Membaca Model

Kekuatan regresi linear adalah **interpretabilitas**. Mari kita lihat koefisien untuk beberapa fitur:

```python
# Ambil nama fitur setelah one-hot encoding
feature_names = lin_pipe.named_steps["prep"].get_feature_names_out()
coefs = lin_pipe.named_steps["reg"].coef_

coef_df = pd.DataFrame({
    "fitur": feature_names,
    "koefisien": coefs,
}).sort_values("koefisien", key=abs, ascending=False)

coef_df.head(10).round(2)
```

	fitur	koefisien
11	cat__InternetService_Fiber optic	24.89
3	num__num_services	9.64
8	cat__PhoneService_Yes	7.67
9	cat__MultipleLines_No phone service	-7.67
22	cat__StreamingTV_Yes	5.30
24	cat__StreamingMovies_Yes	5.27
17	cat__DeviceProtection_No internet service	-3.57
21	cat__StreamingTV_No internet service	-3.57
13	cat__OnlineSecurity_No internet service	-3.57
19	cat__TechSupport_No internet service	-3.57

Cara membaca koefisien:

- Nilai positif berarti: fitur ini **menaikkan** `MonthlyCharges`.
- Nilai negatif berarti: fitur ini **menurunkan** `MonthlyCharges`.
- Besar koefisien **dalam satuan target** (karena kita menstandarisasi fitur, koefisien mewakili perubahan tagihan untuk setiap **standar deviasi** kenaikan fitur).

Contoh interpretasi: Jika koefisien untuk `num__tenure` adalah 3,5, itu berarti: *setiap kenaikan 1 standar deviasi tenure (sekitar 20 bulan) meningkatkan* ***`MonthlyCharges`*** *sebesar Rp 3.500, dengan menganggap fitur lain konstan.*

> ⚠ Warning
>
> **Hati-hati dengan standardisasi.** Kita menstandardisasi fitur numerik di pipeline. Ini membuat koefisien **dapat dibandingkan antar fitur** tetapi **tidak langsung dalam satuan asli**. Jika Anda ingin interpretasi "per 1 bulan tenure", hilangkan `StandardScaler` untuk fitur tersebut, atau kalikan koefisien dengan standar deviasi fitur.

8.4 Diagnostik Residual: Memeriksa Asumsi

Regresi linear memiliki **empat asumsi** yang perlu diperiksa:

Asumsi	Arti	Cara memeriksa
Linearitas	Hubungan X–y adalah linear	Plot residual vs fitted
Independensi	Residual tidak saling bergantung	Durbin-Watson (untuk data waktu); desain sampel
Homoskedastisitas	Varians residual konstan	Plot residual vs fitted
Normalitas residual	Residual berdistribusi normal	Q-Q plot, histogram residual

Jika asumsi ini dilanggar, **koefisien dan standard error** mungkin tidak dapat dipercaya—meskipun **prediksi** mungkin masih berguna.

8.4.1 Plot Residual vs Fitted

```
residuals = y_test_c - y_pred_test

fig, axes = plt.subplots(1, 3, figsize=(11, 3.5))

# Plot 1: Residual vs fitted
axes[0].scatter(y_pred_test, residuals, alpha=0.4, color="#4C72B0", s=10)
axes[0].axhline(0, color="red", linestyle="--")
axes[0].set_xlabel("Prediksi")
axes[0].set_ylabel("Residual")
axes[0].set_title("Residual vs Fitted")

# Plot 2: Histogram residual
axes[1].hist(residuals, bins=40, color="#DD8452", edgecolor="white")
axes[1].set_xlabel("Residual")
axes[1].set_ylabel("Frekuensi")
axes[1].set_title("Distribusi Residual")

# Plot 3: Q-Q plot
from scipy import stats
stats.probplot(residuals, dist="norm", plot=axes[2])
axes[2].set_title("Q-Q Plot")

plt.tight_layout()
plt.show()
```

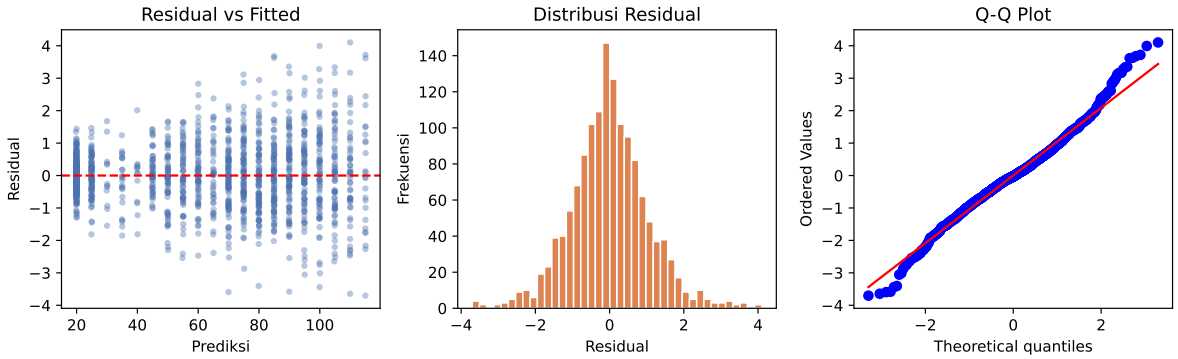


Figure 8.1: Diagnostik residual. Idealnya titik-titik tersebar acak di sekitar garis nol, tanpa pola.

Cara membaca:

- **Residual vs Fitted**: Jika ada pola (kurva, corong), asumsi linearitas atau homoskedastisitas dilanggar. Di sini titik-titik tersebar cukup acak—asumsi terpenuhi dengan baik.
- **Histogram residual**: Jika bentuknya sangat miring atau multimodal, normalitas dilanggar.
- **Q-Q plot**: Jika titik-titik mengikuti garis diagonal, residual mendekati normal. Penyimpangan di ekor menunjukkan ekor yang lebih berat/ringan dari normal.

> Tip
>
> **Plot residual lebih informatif daripada uji formal.** Uji normalitas (Shapiro, Kolmogorov-Smirnov) hampir selalu menolak H pada dataset besar—bahkan untuk penyimpangan kecil. **Plot residual** memberi Anda gambaran visual tentang **jenis** pelanggaran (miring, ekor berat, kurva), yang lebih berguna untuk memutuskan tindakan.

8.5 Multikolinearitas: Ketika Fitur Saling Berkorelasi

Di Bab 3 kita menemukan korelasi tinggi antara `tenure` dan `TotalCharges`. **Multikolinearitas** adalah kondisi ketika dua atau lebih fitur sangat berkorelasi—yang membuat koefisien regresi **tidak stabil** dan **sulit diinterpretasikan**.

8.5.1 Mendeteksi dengan VIF

Variance Inflation Factor (VIF) mengukur seberapa besar varians koefisien meningkat akibat multikolinearitas:

```python
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Ambil fitur numerik saja, tanpa one-hot (untuk kesederhanaan)
num_only = X_train_c[numeric_features].dropna()
vif_data = pd.DataFrame({
    "fitur": numeric_features,
    "VIF": [variance_inflation_factor(num_only.values, i)
            for i in range(num_only.shape[1])],
}).sort_values("VIF", ascending=False)

vif_data
```

	fitur	VIF
2	TotalCharges	7.502045
1	tenure	3.824890
3	num_services	3.208837
0	SeniorCitizen	1.026376
4	charge_ratio	1.011339

Interpretasi VIF:

- **VIF $<$ 5**: multikolinearitas rendah—aman.
- **VIF 5–10**: multikolinearitas sedang—waspada.
- **VIF $>$ 10**: multikolinearitas tinggi—koefisien tidak stabil.

Jika `TotalCharges` memiliki VIF > 10, ini mengonfirmasi apa yang kita duga: ia sangat bergantung pada `tenure` dan `MonthlyCharges`. Ini bukan masalah untuk **prediksi** (model masih bisa prediksi), tetapi menjadi masalah untuk **interpretasi** (koefisien `tenure` vs `TotalCharges` tidak dapat dipisahkan dengan andal).

8.5.2 Solusi: Regularisasi

Regularisasi menambahkan penalti pada besar koefisien, memaksa model untuk **memilih** fitur penting dan **mengecilkan** koefisien yang tidak perlu. Ini sangat berguna untuk multikolinearitas.

Metode	Penalti	Efek
Ridge (L2)	$\lambda \sum \beta_j^2$	Mengecilkan koefisien menuju nol, tetapi tidak pernah tepat nol
Lasso (L1)	$\lambda \sum \|\beta_j\|$	Mengecilkan koefisien **dan** memilih fitur (menjadi nol)
Elastic Net	Kombinasi L1 + L2	Kompromi antara keduanya

```
from sklearn.linear_model import RidgeCV, LassoCV

# Ridge dengan cross-validation otomatis
ridge_pipe = Pipeline([
    ("prep", preprocessor_lin),
    ("reg", RidgeCV(alphas=np.logspace(-3, 3, 50), cv=5)),
])
ridge_pipe.fit(X_train_c, y_train_c)
ridge_pred = ridge_pipe.predict(X_test_c)

# Lasso
lasso_pipe = Pipeline([
    ("prep", preprocessor_lin),
    ("reg", LassoCV(alphas=np.logspace(-3, 3, 50), cv=5, max_iter=5000)),
])
lasso_pipe.fit(X_train_c, y_train_c)
lasso_pred = lasso_pipe.predict(X_test_c)

# Bandingkan
pd.DataFrame({
    "Model": ["Linear (OLS)", "Ridge", "Lasso"],
    "Test RMSE": [
        f"{np.sqrt(mean_squared_error(y_test_c, y_pred_test)):.2f}",
        f"{np.sqrt(mean_squared_error(y_test_c, ridge_pred)):.2f}",
        f"{np.sqrt(mean_squared_error(y_test_c, lasso_pred)):.2f}",
    ],
    "Test R²": [
        f"{r2_score(y_test_c, y_pred_test):.3f}",
        f"{r2_score(y_test_c, ridge_pred):.3f}",
        f"{r2_score(y_test_c, lasso_pred):.3f}",
    ],
})
```

	Model	Test RMSE	Test R^2
0	Linear (OLS)	1.05	0.999
1	Ridge	1.05	0.999
2	Lasso	1.05	0.999

Perhatikan: Ridge dan Lasso memilih **alpha optimal** melalui cross-validation. Dalam banyak kasus, performanya mirip dengan OLS, tetapi **koefisiennya lebih stabil** dan **lebih mudah diinterpretasikan** ketika ada multikolinearitas.

Lasso juga **memilih fitur**: koefisien yang menjadi nol berarti fitur tersebut tidak dipakai oleh model. Ini sangat berguna untuk dataset dengan banyak fitur:

```
lasso_coefs = lasso_pipe.named_steps["reg"].coef_
n_zero = (np.abs(lasso_coefs) < 1e-6).sum()
n_total = len(lasso_coefs)
print(f"Lasso menihilkan {n_zero} dari {n_total} koefisien ({n_zero/n_total:.1%})")
```

```
Lasso menihilkan 22 dari 35 koefisien (62.9%)
```

> **!** Important
>
> **Regularisasi adalah trade-off.** Ia **mengorbankan** sedikit akurasi pada data latih untuk **mendapatkan** stabilitas dan generalisasi. Jika dataset Anda bersih dan bebas multikolinearitas, regularisasi mungkin tidak banyak membantu. Jika dataset Anda "kotor" dengan fitur berkorelasi, regularisasi bisa menyelamatkan model.

8.6 Regresi Logistik: Memodelkan Target Biner

Sekarang kita kembali ke inti Bagian II: **memprediksi churn**. Regresi logistik adalah jawaban klasik untuk target biner, dan masih menjadi baseline yang kuat sampai hari ini.

8.6.1 Bentuk Umum

Regresi logistik memodelkan **log-odds** (logit) dari probabilitas kelas positif:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Probabilitas p kemudian diperoleh melalui fungsi sigmoid:

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots)}}$$

Ini berarti:

- Regresi linear memodelkan **nilai target** secara langsung.
- Regresi logistik memodelkan **log-odds** dari target biner.
- Koefisien β_j menunjukkan perubahan **log-odds** untuk setiap kenaikan 1 unit x_j.

8.6.2 Melatih Model pada Data Churn

```
# Konvensi: konversi target ke biner 0/1 sekali di sini
y_bin = (df["Churn"] == "Yes").astype(int)
X_churn = df.drop(columns=["Churn"])

X_tr, X_te, y_tr, y_te = train_test_split(
    X_churn, y_bin, test_size=0.2, random_state=RANDOM_STATE, stratify=y_bin
)

logreg_pipe = Pipeline([
    ("prep", preprocessor_lin),
    ("clf", LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)),
])
logreg_pipe.fit(X_tr, y_tr)

from sklearn.metrics import roc_auc_score, accuracy_score, classification_report
y_prob = logreg_pipe.predict_proba(X_te)[:, 1]
y_pred = logreg_pipe.predict(X_te)

print(f"Test AUC     : {roc_auc_score(y_te, y_prob):.3f}")
print(f"Test accuracy: {accuracy_score(y_te, y_pred):.3f}")
print()
print(classification_report(y_te, y_pred, target_names=["No churn", "Churn"]))
```

```
Test AUC     : 0.845
Test accuracy: 0.800

              precision    recall  f1-score   support
```

```
    No churn       0.84      0.90      0.87      1035
       Churn       0.66      0.52      0.58       374

    accuracy                           0.80      1409
   macro avg       0.75      0.71      0.72      1409
weighted avg       0.79      0.80      0.79      1409
```

8.6.3 Interpretasi Koefisien sebagai Odds Ratio

Kekuatan regresi logistik adalah interpretabilitasnya. Jika kita eksponensialkan koefisien, kita mendapatkan **odds ratio** (OR):

```
feature_names = logreg_pipe.named_steps["prep"].get_feature_names_out()
coefs = logreg_pipe.named_steps["clf"].coef_[0]
odds_ratios = np.exp(coefs)

or_df = pd.DataFrame({
    "fitur": feature_names,
    "koefisien": coefs,
    "odds_ratio": odds_ratios,
}).sort_values("odds_ratio", ascending=False)

# Tampilkan 10 OR tertinggi dan 5 terendah
top = pd.concat([or_df.head(10), or_df.tail(5)])
top.round(3)
```

	fitur	koefisien	odds_ratio
11	cat__InternetService_Fiber optic	0.921	2.511
27	cat__PaperlessBilling_Yes	0.387	1.473
29	cat__PaymentMethod_Electronic check	0.355	1.426
10	cat__MultipleLines_Yes	0.270	1.310
24	cat__StreamingMovies_Yes	0.203	1.226
3	num__num_services	0.195	1.215
22	cat__StreamingTV_Yes	0.194	1.214
9	cat__MultipleLines_No phone service	0.080	1.083
0	num__SeniorCitizen	0.057	1.059
2	num__TotalCharges	0.043	1.044
32	cat__tenure_group_25-48 bln	-0.655	0.519
25	cat__Contract_One year	-0.703	0.495
31	cat__tenure_group_13-24 bln	-0.710	0.491

	fitur	koefisien	odds_ratio
1	num___tenure	-0.747	0.474
26	cat___Contract_Two year	-1.501	0.223

Cara membaca odds ratio:

- **OR $>$ 1**: fitur **meningkatkan** odds churn. OR = 1,5 berarti: setiap kenaikan 1 unit fitur (atau perpindahan dari baseline untuk fitur kategorikal) meningkatkan odds churn sebesar 50%.
- **OR = 1**: fitur tidak berpengaruh.
- **OR $<$ 1**: fitur **menurunkan** odds churn. OR = 0,5 berarti: odds churn turun 50%.

> **Warning**
>
> **Odds ratio probabilitas.** Ini adalah kesalahpahaman paling umum. OR = 2 berarti **odds** berlipat dua—bukan probabilitas. Jika probabilitas awal 20% (odds = 0,25), maka OR = 2 menghasilkan odds 0,5, yang setara dengan probabilitas 33%. Perubahan probabilitas **bergantung pada baseline**—tidak linier.

8.6.4 Perhatikan Tanda Koefisien

```
# Contoh interpretasi untuk fitur kunci
key_features = [f for f in feature_names if any(k in f for k in ["Contract", "tenure", "Inte

pd.DataFrame({
    "fitur": key_features,
    "odds_ratio": [np.exp(coefs[list(feature_names).index(f)]) for f in key_features],
}).round(3)
```

	fitur	odds_ratio
0	num___tenure	0.474
1	cat___InternetService_Fiber optic	2.511
2	cat___Contract_One year	0.495
3	cat___Contract_Two year	0.223
4	cat___tenure_group_13-24 bln	0.491
5	cat___tenure_group_25-48 bln	0.519
6	cat___tenure_group_7-12 bln	0.560
7	cat___tenure_group_>48 bln	0.725

Cari fitur seperti:

- `cat__Contract_One year` → OR < 1 (kontrak tahunan **mengurangi** odds churn)
- `cat__Contract_Two year` → OR « 1 (kontrak dua tahun **sangat mengurangi** odds churn)
- `num__tenure` → OR < 1 (semakin lama berlangganan, semakin rendah odds churn)
- `cat__InternetService_Fiber optic` → OR > 1 (fiber optic **meningkatkan** odds churn)

Ini konsisten dengan diagnosis kita di Bagian I—dan sekarang kita memilikinya dalam bentuk **kuantitatif** yang dapat dipakai untuk simulasi dan keputusan.

> **!** Important
>
> **Perhatikan arah, bukan hanya besaran.** OR = 0,4 dan OR = 2,5 sama-sama "efek kuat"—satu menurunkan, satu meningkatkan. Dalam presentasi ke pemangku kepentingan, arah sering lebih penting daripada besaran: *"Kontrak dua tahun mengurangi odds churn sekitar 80%"* lebih mudah dicerna daripada *"koefisien -1,6"*.

8.7 Regresi Linear vs Logistik: Kapan Memakai Apa

Aspek	Regresi Linear	Regresi Logistik
Target	Kontinu	Biner
Fungsi	$y = X\beta + \epsilon$	$p = \sigma(X\beta)$
Koefisien	Perubahan nilai target	Perubahan log-odds
Interpretasi	"1 unit X → unit Y"	"1 unit X → e^ kali odds"
Metrik evaluasi	RMSE, MAE, R^2	AUC, precision, recall, F1
Diagnostik	Plot residual, VIF	Kurva ROC, kalibrasi
Asumsi	Linearitas, homoskedastisitas, normalitas	Tidak ada asumsi distribusi ketat

Aturan praktis:

- Jika target Anda adalah **jumlah, harga, waktu** → gunakan regresi linear.
- Jika target Anda adalah **ya/tidak, sukses/gagal** → gunakan regresi logistik.
- Jika target Anda adalah **hitungan** (0, 1, 2, …) → gunakan regresi Poisson.
- Jika target Anda adalah **waktu sampai kejadian** → gunakan survival analysis (Bab 10).

8.8 Menghubungkan Kembali ke Keputusan

Model yang baru kita latih menjawab pertanyaan yang berbeda dari baseline di Bab 7. Ia memberi kita **odds ratio yang dapat diinterpretasikan**:

```
# Contoh: seberapa besar penurunan odds churn jika pelanggan pindah dari bulanan ke tahunan?
coef_monthly = coefs[list(feature_names).index("cat__Contract_One year")]
or_annual = np.exp(-coef_monthly)  # karena baseline adalah Month-to-month

print(f"Pindah dari bulanan ke kontrak tahunan:")
print(f"  → Odds churn berubah dengan faktor {np.exp(coef_monthly):.3f}")
print(f"  → Atau setara dengan pengurangan odds sebesar {(1 - np.exp(coef_monthly))*100:.0f}%
```

```
Pindah dari bulanan ke kontrak tahunan:
  → Odds churn berubah dengan faktor 0.495
  → Atau setara dengan pengurangan odds sebesar 50%
```

Jika hasilnya OR = 0,4, itu berarti: *"Pindah dari bulanan ke tahunan mengurangi odds churn sebesar 60%"*. Ini adalah **argumen bisnis** yang langsung dapat digunakan dalam rapat—dan akan menjadi input untuk model optimasi di Bagian III.

> **Tip**
>
> **Bridge ke Bagian III.** Odds ratio ini bukan hanya insight—ia adalah **parameter** yang dapat dipakai dalam model optimasi. Jika kita tahu bahwa perpindahan ke kontrak tahunan mengurangi odds churn 60%, kita dapat menghitung **ekspektasi pengurangan revenue loss** dari program konversi kontrak, dan mengoptimalkan alokasi anggaran terhadap intervensi yang berbeda. Kita akan melakukan ini di Bab 14–15.

8.9 Latihan Terbimbing

Latihan 8.1 — Interpretasi koefisien. Buka tabel `coef_df` dari model regresi linear. Pilih tiga fitur dengan koefisien absolut terbesar. Untuk masing-masing, tuliskan interpretasi dalam bahasa non-teknis (satu kalimat per fitur) yang dapat dipahami oleh manajer produk.

Latihan 8.2 — Diagnostik residual. Buat plot residual untuk model regresi linear Anda. Apakah ada pola yang mencurigakan? Jika ya, fitur mana yang mungkin menyebabkan pola tersebut? (Petunjuk: coba plot residual vs `tenure` atau vs `TotalCharges` secara terpisah.)

Latihan 8.3 — Multikolinearitas. Identifikasi fitur dengan VIF > 10 di dataset. Jika ada, coba hapus salah satu fitur yang berkorelasi tinggi, latih ulang model, dan bandingkan R^2 dan RMSE. Apakah ada penurunan performa? Berapa besar?

Latihan 8.4 — Lasso untuk feature selection. Jalankan Lasso dengan alpha yang lebih besar (misalnya 10× alpha optimal). Berapa banyak koefisien yang menjadi nol? Bandingkan AUC/R^2 dengan alpha optimal. Kapan Anda memilih alpha besar (lebih banyak fitur dinihilkan) vs alpha optimal?

Latihan 8.5 — Odds ratio interpretation. Ambil tiga fitur dari model regresi logistik dengan OR > 1 dan tiga dengan OR < 1. Untuk setiap fitur, tuliskan interpretasi odds ratio dalam bahasa bisnis. Mana yang lebih besar dampaknya terhadap keputusan retensi: fitur dengan OR = 0,3 atau OR = 3? Mengapa?

Latihan 8.6 — Kalibrasi model. Regresi logistik menghasilkan probabilitas. Tetapi apakah probabilitas ini **terkalibrasi**? Artinya: dari semua pelanggan yang model prediksi memiliki probabilitas 0,7, apakah ~70% di antaranya benar-benar churn? Gunakan `sklearn.calibration.calibration_curve` untuk memeriksa. Jika tidak terkalibrasi, apa implikasinya untuk keputusan retensi?

> **Note**
>
> **Petunjuk Latihan 8.6.** Kalibrasi penting karena kita akan menggunakan **probabilitas** untuk keputusan (mis. "hubungi pelanggan dengan probabilitas $> 0{,}5$"). Jika model tidak terkalibrasi, threshold 0,5 mungkin tidak berarti apa-apa—bisa jadi kita menghubungi terlalu banyak atau terlalu sedikit pelanggan. Ini akan dibahas lebih dalam di Bab 9 (Klasifikasi).

8.10 Kotak "Jembatan": Dari Model ke Keputusan

> **Important**
>
> **Jembatan.** Bab ini memberi kita **dua model** yang menjawab dua pertanyaan berbeda:
>
> 1. **Regresi linear** (target kontinu): *"Jika pelanggan X memiliki tenure 24 bulan dan membayar Rp 80.000/bulan, berapa `MonthlyCharges` yang wajar untuknya?"* — Ini berguna untuk deteksi anomali (tagihan terlalu tinggi/rendah) dan untuk memahami **struktur** penetapan harga.
> 2. **Regresi logistik** (target biner): *"Jika pelanggan X memiliki tenure 24 bulan dan kontrak bulanan, berapa probabilitas ia akan churn?"* — Ini berguna untuk **prioritisasi retensi** dan akan menjadi input langsung untuk model optimasi di Bagian III.

Tetapi ada satu hal yang **belum** kita lakukan: membandingkan regresi logistik dengan model yang lebih canggih—random forest, gradient boosting, XGBoost. Apakah model yang lebih kompleks benar-benar lebih baik untuk dataset ini? Bab 9 akan menjawabnya secara empiris.

8.11 Rangkuman

Dalam bab ini kita telah:

- **Membedakan regresi linear (target kontinu) dari regresi logistik (target biner)**, dan memahami kapan masing-masing tepat digunakan.
- **Membangun regresi linear** untuk memprediksi `MonthlyCharges` dan menginterpretasikan koefisien dalam satuan target.
- **Memeriksa asumsi regresi** melalui plot residual, histogram residual, dan Q-Q plot.
- **Mendeteksi multikolinearitas** dengan VIF dan mengatasinya dengan **regularisasi** (Ridge, Lasso).
- **Memahami Lasso sebagai feature selection**—koefisien yang menjadi nol berarti fitur tidak dipakai model.
- **Membangun regresi logistik** untuk memprediksi churn dan menginterpretasikan koefisien sebagai **odds ratio**.
- **Membedakan odds ratio dari probabilitas**, dan memahami implikasinya untuk interpretasi.
- **Menghubungkan odds ratio ke keputusan bisnis**: "pindah ke kontrak tahunan mengurangi odds churn 60%" adalah argumen yang dapat digunakan langsung dalam rapat.

8.12 Jembatan ke Bab 9

Regresi logistik adalah **model klasifikasi klasik**. Ia sederhana, interpretabel, dan cepat. Tetapi apakah ia model **terbaik** untuk churn prediction?

Bab 9—**Klasifikasi**—akan membandingkan regresi logistik dengan model yang lebih canggih:

- **K-Nearest Neighbors** (KNN): klasifikasi berdasarkan kemiripan.
- **Naive Bayes**: klasifikasi probabilistik dengan asumsi independensi.
- **Decision Tree**: klasifikasi berbasis aturan yang mudah dijelaskan.
- **Random Forest** dan **Gradient Boosting**: ensemble yang menggabungkan banyak pohon untuk performa yang lebih tinggi.

Kita akan membandingkan **AUC, precision, recall, dan F1**—dan menyadari bahwa "model terbaik" bergantung pada **metrik yang Anda pedulikan**, yang pada gilirannya bergantung pada **konsekuensi bisnis dari kesalahan**. Ini adalah bab yang paling praktis di Bagian II.

9 Klasifikasi: Membandingkan Model dan Memilih yang Tepat

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membangun enam model klasifikasi—dari Logistic Regression hingga Gradient Boosting—pada dataset yang sama. 2. Membaca **confusion matrix** dan memahami implikasi bisnis dari setiap sel. 3. Memilih metrik evaluasi yang tepat berdasarkan **konsekuensi kesalahan**, bukan berdasarkan default. 4. Menangani **kelas tidak seimbang** dengan `class_weight`, threshold adjustment, dan memahami kapan resampling diperlukan. 5. Membedakan **ranking** (AUC) dari **kalibrasi** (probabilitas yang dapat dipercaya). 6. Menginterpretasikan **feature importance** dan menghubungkannya kembali ke diagnosis Bagian I. 7. Memilih model akhir berdasarkan trade-off antara performa, interpretabilitas, dan biaya komputasi.

9.1 Dari Satu Model ke Enam

Di Bab 8 kita membangun **satu** model klasifikasi: Logistic Regression. Ia mencapai AUC sekitar 0,85 dan memberikan odds ratio yang dapat diinterpretasikan. Tetapi apakah ia model **terbaik**?

Bab ini akan menjawabnya secara empiris. Kita akan membangun **enam model** pada dataset yang sama, mengevaluasi mereka dengan **metrik yang berbeda**, dan memilih yang paling sesuai dengan **konteks bisnis**.

Important

Tidak ada "model terbaik" secara universal. Model terbaik adalah model yang **paling sesuai dengan pertanyaan, data, dan konsekuensi kesalahan** dari konteks spesifik. Bab ini mengajarkan Anda **kerangka untuk memilih**—bukan satu jawaban yang benar.

9.2 Menyiapkan Data

Kita menggunakan pembagian data yang sama sepanjang bab ini agar perbandingan adil:

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from decanal.utils import load_churn_clean
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (
    confusion_matrix, classification_report, roc_auc_score,
    roc_curve, precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score,
    brier_score_loss,
)
from sklearn.calibration import calibration_curve

RANDOM_STATE = 42
df = load_churn_clean()

# Konvensi: target biner 0/1
y = (df["Churn"] == "Yes").astype(int)
X = df.drop(columns=["Churn"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
```

```
print(f"Train: {len(X_train):,} (churn: {y_train.mean():.1%})")
print(f"Test : {len(X_test):,} (churn: {y_test.mean():.1%})")
```

```
Train: 5,634 (churn: 26.5%)
Test : 1,409 (churn: 26.5%)
```

Preprocessing yang sama untuk semua model (kecuali Naive Bayes, yang akan kita tangani terpisah):

```
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), numeric_features),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical_features),
])

# Untuk Naive Bayes: hanya numerik, imputasi + scaling
preprocessor_nb = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), numeric_features),
], remainder="drop")
```

9.3 Enam Model Klasifikasi

Kita akan melatih enam model. Untuk masing-masing, perhatikan **asumsi** dan **karakteristiknya**—bukan hanya kodenya.

9.3.1 1. Logistic Regression — Baseline yang Kuat

```
logreg = Pipeline([
    ("prep", preprocessor),
    ("clf", LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)),
])
logreg.fit(X_train, y_train)
```

---

steps	steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define ‘fit‘. All non-last steps must also define ‘transform‘. See :ref:‘Combining Estimators <combining_estimators>‘ for more details.	[('prep', ...), ('clf', ...)]

transform_input transform_input: list of str, default=None The names of the :term:‘metadata‘ parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to “fit“ (other than “X“) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:‘metadata routing <metadata_routing>‘. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using “sklearn.set_config(enable_metadata_routing=True)“. .. versionadded:: 1.6	None

	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute “named_steps“ or “steps“ to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:‘sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py‘ for an example on how to enable caching.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[int64](2,)	[0,1]
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Only defined if the underlying estimator exposes such an attribute when fit. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying first estimator in ‘steps‘ exposes such an attribute when fit. .. versionadded:: 0.24	int	22

	transformers transformers: list of tuples List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data. name : str Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using “set_params“ and searched in grid search. transformer : {'drop', 'passthrough'} or estimator Estimator must support :term:‘fit‘ and :term:‘transform‘. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively. columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where “transformer“ expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data ‘X‘ and can return any of the above. To select multiple columns by name or dtype, you can use :obj:‘make_column_selector‘.	[('num', ...), ('cat', ...)]

	remainder remainder: {'drop', 'passthrough'} or estimator, default='drop' By default, only the specified columns in ‘transformers‘ are transformed and combined in the output, and the non-specified columns are dropped. (default of “'drop'“). By specifying “remainder='passthrough'“, all remaining columns that were not specified in ‘transformers‘, but present in the data passed to ‘fit‘ will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during ‘fit‘ will be excluded from the output of ‘transform‘. By setting “remainder“ to be an estimator, the remaining non-specified columns will use the “remainder“ estimator. The estimator must support :term:‘fit‘ and :term:‘transform‘. Note that using this feature requires that the DataFrame columns input at :term:‘fit‘ and :term:‘transform‘ have identical order.	'drop'

	sparse_threshold sparse_threshold: float, default=0.3 If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use “sparse_threshold=0“ to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.	0.3
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	None
	transformer_weights transformer_weights: dict, default=None Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each transformer will be printed as it is completed.	False

verbose_feature_names_out verbose_feature_names_out: bool, str or Callable[[str, str], str], default=True - If True, :meth:‘ColumnTransformer.get_feature_names_out‘ will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting ‘verbose_feature_names_out=”{transformer_name}___{feature_name}”‘. - If False, :meth:‘ColumnTransformer.get_feature_names_out‘ will not prefix any feature names and will error if feature names are not unique. - If “Callable[[str, str], str]“, :meth:‘ColumnTransformer.get_feature_names_out‘ will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name. - If “str“, it must be a string ready for formatting. The given string will be formatted using two field names: “transformer_name“ and “feature_name“. e.g. “”{feature_name}___{transformer_name}”“. See :meth:‘str.format‘ method from the standard library for more info. .. versionadded:: 1.0	True

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying transformers expose such an attribute when fit. .. versionadded:: 0.24	int	22
named_transformers_ named_transformers_: :class:‘~sklearn.utils.Bunch‘ Read-only attribute to access any transformer by given name. Keys are transformer names and values are the fitted transformer objects.	Bunch	{'num': Pipel...='ignore'))])}

output_indices_ output_indices_: dict A dictionary from each transformer name to a slice, where the slice corresponds to indices in the transformed output. This is useful to inspect which transformer is responsible for which transformed feature(s). .. versionadded:: 1.0	dict	{'cat': slice(6, 52, None), 'num': slice(0, 6, None), 're...er': slice(0, 0, None)}
sparse_output_ sparse_output_: bool Boolean flag indicating whether the output of “transform“ is a sparse matrix or a dense numpy array, which depends on the output of the individual transformers and the ‘sparse_threshold‘ keyword.	bool	False

transformers_ transformers_: list The collection of fitted transformers as tuples of (name, fitted_transformer, column). ‘fitted_transformer‘ can be an estimator, or ‘'drop'‘; ‘'passthrough'‘ is replaced with an equivalent :class:‘~sklearn.preprocessing.FunctionTransformer‘. In case there were no columns selected, this will be the unfitted transformer. If there are remaining columns, the final element is a tuple of the form: ('remainder', transformer, remaining_columns) corresponding to the “remainder“ parameter. If there are remaining columns, then “len(transformers_)==len(transformers)+1“, otherwise “len(transformers_)==len(transformers)“. .. versionadded:: 1.7 The format of the remaining columns now attempts to match that of the other transformers: if all columns were provided as column names (‘str‘), the remaining columns are stored as column names; if all columns were provided as mask arrays (‘bool‘), so are the remaining columns; in all other cases the remaining columns are stored as indices (‘int‘).	list	[('num', Pipeline(step...ardScaler())]), ['Se...en', 'tenure', 'Mo...es', 'To...es', ...]), ('cat', Pipeline(step...n='ignore'))]), ['gender', 'Partner', 'De...ts', 'Ph...ce', ...])]

strategy	strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'median'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when ‘fit‘ is called are returned in results when ‘transform‘ is called. The imputed value is always ‘0‘ except when ‘strategy=”constant”‘ in which case ‘fill_value‘ will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](6,)	['SeniorCitizen','tenure','MonthlyCharges','Tota 'charge_ratio']
indicator_ indicator_: :class:‘~sklearn.impute.MissingIndicator‘ Indicator used to add binary indicators for missing values. ‘None‘ if ‘add_indicator=False‘.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[float64](6,)	[0. , 29. , 70.5 ,1394.92, 3. , 1.]

SeniorCitizen
tenure
MonthlyCharges
TotalCharges
num_services
charge_ratio

	copy copy: bool, default=True If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.	True

	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
	with_std with_std: bool, default=True If True, scale the data to unit variance (or equivalently, unit standard deviation).	True

Name	Type	Value
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.16, 32.49, 64.93,2299.33, 3.38, 1.]
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6

n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the “n_samples_seen“ will be an integer, otherwise it will be an array of dtype int. If ‘sample_weights‘ are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across “partial_fit“ calls.	float64	5634
scale_ scale_: ndarray of shape (n_features,) or None Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using ‘np.sqrt(var_)‘. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. ‘scale_‘ is equal to ‘None‘ when ‘with_std=False‘. .. versionadded:: 0.17 *scale_*	ndarray[float64](6,)	[0.37, 24.57, 30.14,2279. , 2.07, 0.06]

var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute ‘scale_‘. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.14, 603.52, 908.14,5193850.1 , 4.29, 0.]

x0
x1
x2
x3
x4
x5

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If "mean", then replace missing values using the mean along each column. Can only be used with numeric data. - If "median", then replace missing values using the median along each column. Can only be used with numeric data. - If "most_frequent", then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If "constant", then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy="constant" for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'most_frequent'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when 'fit' is called are returned in results when 'transform' is called. The imputed value is always '0' except when 'strategy="constant"' in which case 'fill_value' will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape ('n_features_in_',) Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](16,)	['gender','Partner','Dependents',...,'PaperlessBil 'tenure_group']
indicator_ indicator_: :class:'~sklearn.impute.MissingIndicator' Indicator used to add binary indicators for missing values. 'None' if 'add_indicator=False'.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	16
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[object](16,)	['Male','No','No',...,'Yes','Electronic check','>48 bln']

gender
Partner
Dependents
PhoneService
MultipleLines
InternetService
OnlineSecurity
OnlineBackup
DeviceProtection
TechSupport
StreamingTV
StreamingMovies
Contract
PaperlessBilling
PaymentMethod
tenure_group

	handle_unknown handle_unknown: {'error', 'ignore', 'infrequent_if_exist', 'warn'}, default='error' Specifies the way unknown categories are handled during :meth:‘transform‘. - 'error' : Raise an error if an unknown category is present during transform. - 'ignore' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will be all zeros. In the inverse transform, an unknown category will be denoted as None. - 'infrequent_if_exist' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will map to the infrequent category if it exists. The infrequent category will be mapped to the last position in the encoding. During inverse transform, an unknown category will be mapped to the category denoted ‘'infrequent'‘ if it exists. If the ‘'infrequent'‘ category does not exist, then :meth:‘transform‘ and :meth:‘inverse_transform‘ will handle an unknown category as with ‘handle_unknown='ignore'‘. Infrequent categories exist based on ‘min_frequency‘ and ‘max_categories‘. Read more in the	'ignore'

categories	categories: 'auto' or a list of array-like, default='auto' Categories (unique values) per feature: - 'auto' : Determine categories automatically from the training data. - list : “categories[i]“ holds the categories expected in the ith column. The passed categories should not mix strings and numeric values within a single feature, and should be sorted in case of numeric values. The used categories can be found in the “categories_“ attribute. .. versionadded:: 0.20	'auto'

drop	drop: {'first', 'if_binary'} or an array-like of shape (n_features,), default=None Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model. However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models. - None : retain all features (the default). - 'first' : drop the first category in each feature. If only one category is present, the feature will be dropped entirely. - 'if_binary' : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact. - array : “drop[i]“ is the category in feature “X[:, i]“ that should be dropped. When ‘max_categories‘ or ‘min_frequency‘ is configured to group infrequent categories, the dropping behavior is handled after the	None

sparse_output sparse_output: bool, default=True When “True“, it returns a SciPy sparse matrix/array in ”Compressed Sparse Row” (CSR) format. .. versionadded:: 1.2 ‘sparse‘ was renamed to ‘sparse_output‘	True
dtype dtype: number type, default=np.float64 Desired dtype of output.	<class 'numpy.float64'>
min_frequency min_frequency: int or float, default=None Specifies the minimum frequency below which a category will be considered infrequent. - If ‘int‘, categories with a smaller cardinality will be considered infrequent. - If ‘float‘, categories with a smaller cardinality than ‘min_frequency * n_samples‘ will be considered infrequent. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infrequent_categories>‘.	None

max_categories max_categories: int, default=None Specifies an upper limit to the number of output features for each input feature when considering infrequent categories. If there are infrequent categories, ‘max_categories‘ includes the category representing the infrequent categories along with the frequent categories. If ‘None‘, there is no limit to the number of output features. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infre- quent_categories>‘.	None

	feature_name_combiner feature_name_combiner: ”concat” or callable, default=”concat” Callable with signature ‘def callable(input_feature, category)‘ that returns a string. This is used to create feature names to be returned by :meth:‘get_feature_names_out‘. ‘”concat”‘ concatenates encoded feature name and category with ‘feature + ”_” + str(category)‘.E.g. feature X with values 1, 6, 7 create feature names ‘X_1, X_6, X_7‘. .. versionadded:: 1.3	'concat'

Name	Type	Value
categories_ categories_: list of arrays The categories of each feature determined during fitting (in order of the features in X and corresponding with the output of “transform“). This includes the category specified in “drop“ (if any).	list	[array(['Femal... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), ...]

drop_idx_ drop_idx_: array of shape (n_features,) - “drop_idx_[i]“ is the index in “categories_[i]“ of the category to be dropped for each feature. - “drop_idx_[i] = None“ if no category is to be dropped from the feature with index “i“, e.g. when ‘drop='if_binary'‘ and the feature isn't binary. - “drop_idx_ = None“ if all the transformed features will be retained. If infrequent categories are enabled by setting ‘min_frequency‘ or ‘max_categories‘ to a non-default value and ‘drop_idx[i]‘ corresponds to an infrequent category, then the entire infrequent category is dropped. .. versionchanged:: 0.23 Added the possibility to contain ‘None‘ values.	NoneType	None
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 1.0	int	16

x0_Female
x0_Male
x1_No
x1_Yes
x2_No
x2_Yes
x3_No
x3_Yes
x4_No
x4_No phone service
x4_Yes
x5_DSL
x5_Fiber optic
x5_No
x6_No
x6_No internet service
x6_Yes
x7_No
x7_No internet service
x7_Yes
x8_No
x8_No internet service
x8_Yes
x9_No
x9_No internet service
x9_Yes
x10_No
x10_No internet service
x10_Yes
x11_No
x11_No internet service
x11_Yes
x12_Month-to-month
x12_One year
x12_Two year
x13_No
x13_Yes
x14_Bank transfer (automatic)
x14_Credit card (automatic)
x14_Electronic check
x14_Mailed check
x15_0-6 bln

x15__13-24 bln
x15__25-48 bln
x15__7-12 bln
x15__>48 bln

num___SeniorCitizen
num___tenure
num___MonthlyCharges
num___TotalCharges
num___num_services
num___charge_ratio
cat___gender_Female
cat___gender_Male
cat___Partner_No
cat___Partner_Yes
cat___Dependents_No
cat___Dependents_Yes
cat___PhoneService_No
cat___PhoneService_Yes
cat___MultipleLines_No
cat___MultipleLines_No phone service
cat___MultipleLines_Yes
cat___InternetService_DSL
cat___InternetService_Fiber optic
cat___InternetService_No
cat___OnlineSecurity_No
cat___OnlineSecurity_No internet service
cat___OnlineSecurity_Yes
cat___OnlineBackup_No
cat___OnlineBackup_No internet service
cat___OnlineBackup_Yes
cat___DeviceProtection_No
cat___DeviceProtection_No internet service
cat___DeviceProtection_Yes
cat___TechSupport_No
cat___TechSupport_No internet service
cat___TechSupport_Yes
cat___StreamingTV_No
cat___StreamingTV_No internet service
cat___StreamingTV_Yes
cat___StreamingMovies_No

cat___StreamingMovies_No internet service
cat___StreamingMovies_Yes
cat___Contract_Month-to-month
cat___Contract_One year
cat___Contract_Two year
cat___PaperlessBilling_No
cat___PaperlessBilling_Yes
cat___PaymentMethod_Bank transfer (automatic)
cat___PaymentMethod_Credit card (automatic)
cat___PaymentMethod_Electronic check
cat___PaymentMethod_Mailed check
cat___tenure_group_0-6 bln
cat___tenure_group_13-24 bln
cat___tenure_group_25-48 bln
cat___tenure_group_7-12 bln
cat___tenure_group_>48 bln

	random_state random_state: int, RandomState instance, default=None Only used for ‘solver‘ == 'sag', 'saga' or 'liblinear' to shuffle the data. It has no effect on the other solvers. See :term:‘Glossary <random_state>‘ for details.	42
	max_iter max_iter: int, default=100 Maximum number of iterations taken for the solvers to converge.	1000

penalty	penalty: {'l1', 'l2', 'elasticnet', None}, default='l2' Specify the norm of the penalty: - ‘None‘: no penalty is added; - ‘'l2'‘: add an L2 penalty term and it is the default choice; - ‘'l1'‘: add an L1 penalty term; - ‘'elasticnet'‘: both L1 and L2 penalty terms are added. .. warning:: Some penalties may not work with some solvers. See the parameter ‘solver‘ below, to know the compatibility between the penalty and solver. .. versionadded:: 0.19 l1 penalty with SAGA solver (allowing 'multinomial' + L1) .. deprecated:: 1.8 ‘penalty‘ was deprecated in version 1.8 and will be removed in 1.10. Use ‘l1_ratio‘ and ‘C‘ instead. ‘l1_ratio=0‘ for ‘penalty='l2'‘, ‘l1_ratio=1‘ for ‘penalty='l1'‘, ‘l1_ratio‘ set to any float between 0 and 1 for ‘penalty='elasticnet'‘, and ‘C=np.inf‘ for ‘penalty=None‘.	'deprecated'

C	C: float, default=1.0 Inverse of regularization strength; must be a positive float. Like in support vector machines, smaller values specify stronger regularization. ‘C=np.inf‘ results in unpenalized logistic regression. For a visual example on the effect of tuning the ‘C‘ parameter with an L1 penalty, see: :ref:‘sphx_glr_auto_examples_linear_model_plot_logistic_path.py‘.	1.0

l1_ratio	l1_ratio: float, default=0.0 The Elastic-Net mixing parameter, with ‘0 <= l1_ratio <= 1‘. Setting ‘l1_ratio=1‘ gives a pure L1-penalty, setting ‘l1_ratio=0‘ a pure L2-penalty. Any value between 0 and 1 gives an Elastic-Net penalty of the form ‘l1_ratio * L1 + (1 - l1_ratio) * L2‘. .. warning:: Certain values of ‘l1_ratio‘, i.e. some penalties, may not work with some solvers. See the parameter ‘solver‘ below, to know the compatibility between the penalty and solver. .. versionchanged:: 1.8 Default value changed from None to 0.0. .. deprecated:: 1.8 ‘None‘ is deprecated and will be removed in version 1.10. Always use ‘l1_ratio‘ to specify the penalty type.	0.0

dual dual: bool, default=False Dual (constrained) or primal (regularized, see also :ref:‘this equation <regularized-logistic-loss>‘) formulation. Dual formulation is only implemented for l2 penalty with liblinear solver. Prefer ‘dual=False‘ when n_samples > n_features.	False
tol tol: float, default=1e-4 Tolerance for stopping criteria.	0.0001
fit_intercept fit_intercept: bool, default=True Specifies if a constant (a.k.a. bias or intercept) should be added to the decision function.	True

intercept_scaling intercept_scaling: float, default=1 Useful only when the solver ‘liblinear‘ is used and ‘self.fit_intercept‘ is set to ‘True‘. In this case, ‘x‘ becomes ‘[x, self.intercept_scaling]‘, i.e. a ”synthetic” feature with constant value equal to ‘intercept_scaling‘ is appended to the instance vector. The intercept becomes “intercept_scaling * synthetic_feature_weight“. .. note:: The synthetic feature weight is subject to L1 or L2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept) ‘intercept_scaling‘ has to be increased.	1

class_weight	class_weight: dict or 'balanced', default=None Weights associated with classes in the form “{class_label: weight}“. If not given, all classes are supposed to have weight one. The ”balanced” mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as “n_samples / (n_classes * np.bincount(y))“. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified. .. versionadded:: 0.17 *class_weight='balanced'*	None

solver	solver: {'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'}, default='lbfgs' Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects: - 'lbfgs' is a good default solver because it works reasonably well for a wide class of problems. - For :term:‘multiclass‘ problems (‘n_classes >= 3‘), all solvers except 'liblinear' minimize the full multinomial loss, 'liblinear' will raise an error. - 'newton-cholesky' is a good choice for ‘n_samples‘ » ‘n_features * n_classes‘, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on ‘n_features * n_classes‘ because it explicitly computes the full Hessian matrix. - For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones; - 'liblinear' can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the :class:‘~sklearn.multiclass.OneVsRestClassifier‘. .. warning:: The choice of the algorithm depends on the penalty	'lbfgs'

	verbose verbose: int, default=0 For the liblinear and lbfgs solvers set verbose to any positive number for verbosity.	0
	warm_start warm_start: bool, default=False When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. Useless for liblinear solver. See :term:‘the Glossary <warm_start>‘. .. versionadded:: 0.17 *warm_start* to support *lbfgs*, *newton-cg*, *sag*, *saga* solvers.	False
	n_jobs n_jobs: int, default=None Does not have any effect. .. deprecated:: 1.8 ‘n_jobs‘ is deprecated in version 1.8 and will be removed in 1.10.	None

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) A list of class labels known to the classifier.	ndarray[int64](2,)	[0,1]

coef_ coef_: ndarray or CSR matrix of shape (1, n_features) or (n_classes, n_features) Coefficients of the features in the decision function. ‘coef_‘ is of shape (1, n_features) when the given problem is binary. By default, it will be created as a dense array, but can be turned to sparse (CSR format) through :meth:‘sparsify‘ (which can be beneficial under L1 regularization when many coefficients are zero), and back to dense through :meth:‘densify‘.	ndarray[float64](1, 52)	[[0.06,-0.7 ,-0.59,...,-0.28,-0.16,-0.]]
intercept_ intercept_: ndarray of shape (1,) or (n_classes,) Intercept (a.k.a. bias) added to the decision function. If ‘fit_intercept‘ is set to False, the intercept is set to zero. ‘intercept_‘ is of shape (1,) when the given problem is binary.	ndarray[float64](1,)	[-0.6]
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	52

n_iter_ n_iter_: ndarray of shape (1,) Actual number of iterations for all classes. .. versionchanged:: 0.20 In SciPy <= 1.0.0 the number of lbfgs iterations may exceed “max_iter“. “n_iter_“ will now report at most “max_iter“.	ndarray[int32](1,)	[58]

Asumsi: hubungan linear antara fitur dan log-odds. **Kelebihan**: interpretabel (odds ratio), cepat, stabil. **Kekurangan**: tidak menangkap interaksi non-linear secara otomatis.

9.3.2 2. K-Nearest Neighbors — Berdasarkan Kemiripan

```
knn = Pipeline([
    ("prep", preprocessor),
    ("clf", KNeighborsClassifier(n_neighbors=15)),
])
knn.fit(X_train, y_train)
```

steps	steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define ‘fit‘. All non-last steps must also define ‘transform‘. See :ref:‘Combining Estimators <combining_estimators>‘ for more details.	[('prep', ...), ('clf', ...)]

transform_input transform_input: list of str, default=None The names of the :term:‘metadata‘ parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to “fit“ (other than “X“) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:‘metadata routing <metadata_routing>‘. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using “sklearn.set_config(enable_metadata_routing=True)“. .. versionadded:: 1.6	None

	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute “named_steps“ or “steps“ to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:‘sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py‘ for an example on how to enable caching.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[int64](2,)	[0,1]
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Only defined if the underlying estimator exposes such an attribute when fit. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying first estimator in ‘steps‘ exposes such an attribute when fit. .. versionadded:: 0.24	int	22

	transformers transformers: list of tuples List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data. name : str Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using “set_params“ and searched in grid search. transformer : {'drop', 'passthrough'} or estimator Estimator must support :term:‘fit‘ and :term:‘transform‘. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively. columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where “transformer“ expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data ‘X‘ and can return any of the above. To select multiple columns by name or dtype, you can use :obj:‘make_column_selector‘.	[('num', ...), ('cat', ...)]

	remainder remainder: {'drop', 'passthrough'} or estimator, default='drop' By default, only the specified columns in ‘transformers‘ are transformed and combined in the output, and the non-specified columns are dropped. (default of “'drop'“). By specifying “remainder='passthrough'“, all remaining columns that were not specified in ‘transformers‘, but present in the data passed to ‘fit‘ will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during ‘fit‘ will be excluded from the output of ‘transform‘. By setting “remainder“ to be an estimator, the remaining non-specified columns will use the “remainder“ estimator. The estimator must support :term:‘fit‘ and :term:‘transform‘. Note that using this feature requires that the DataFrame columns input at :term:‘fit‘ and :term:‘transform‘ have identical order.	'drop'

	sparse_threshold sparse_threshold: float, default=0.3 If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use “sparse_threshold=0“ to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.	0.3
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	None
	transformer_weights transformer_weights: dict, default=None Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each transformer will be printed as it is completed.	False

verbose_feature_names_out verbose_feature_names_out: bool, str or Callable[[str, str], str], default=True - If True, :meth:‘ColumnTransformer.get_feature_names_out‘ will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting ‘verbose_feature_names_out=”{transformer_name}___{feature_name}”‘. - If False, :meth:‘ColumnTransformer.get_feature_names_out‘ will not prefix any feature names and will error if feature names are not unique. - If “Callable[[str, str], str]“, :meth:‘ColumnTransformer.get_feature_names_out‘ will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name. - If “str“, it must be a string ready for formatting. The given string will be formatted using two field names: “transformer_name“ and “feature_name“. e.g. “”{feature_name}___{transformer_name}”“. See :meth:‘str.format‘ method from the standard library for more info. .. versionadded:: 1.0	True

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying transformers expose such an attribute when fit. .. versionadded:: 0.24	int	22
named_transformers_ named_transformers_: :class:‘~sklearn.utils.Bunch‘ Read-only attribute to access any transformer by given name. Keys are transformer names and values are the fitted transformer objects.	Bunch	{'num': Pipel...='ignore'))])}

output_indices_ output_indices_: dict A dictionary from each transformer name to a slice, where the slice corresponds to indices in the transformed output. This is useful to inspect which transformer is responsible for which transformed feature(s). .. versionadded:: 1.0	dict	{'cat': slice(6, 52, None), 'num': slice(0, 6, None), 're...er': slice(0, 0, None)}
sparse_output_ sparse_output_: bool Boolean flag indicating whether the output of “transform“ is a sparse matrix or a dense numpy array, which depends on the output of the individual transformers and the ‘sparse_threshold‘ keyword.	bool	False

transformers_ transformers_: list The collection of fitted transformers as tuples of (name, fitted_transformer, column). ‘fitted_transformer‘ can be an estimator, or ‘'drop'‘; ‘'passthrough'‘ is replaced with an equivalent :class:‘~sklearn.preprocessing.FunctionTransformer‘. In case there were no columns selected, this will be the unfitted transformer. If there are remaining columns, the final element is a tuple of the form: ('remainder', transformer, remaining_columns) corresponding to the “remainder“ parameter. If there are remaining columns, then “len(transformers_)==len(transformers)+1“, otherwise “len(transformers_)==len(transformers)“. .. versionadded:: 1.7 The format of the remaining columns now attempts to match that of the other transformers: if all columns were provided as column names (‘str‘), the remaining columns are stored as column names; if all columns were provided as mask arrays (‘bool‘), so are the remaining columns; in all other cases the remaining columns are stored as indices (‘int‘).	list	[('num', Pipeline(step...ardScaler())]), ['Se...en', 'tenure', 'Mo...es', 'To...es', ...]), ('cat', Pipeline(step...n='ignore'))]), ['gender', 'Partner', 'De...ts', 'Ph...ce', ...])]

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'median'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when ‘fit‘ is called are returned in results when ‘transform‘ is called. The imputed value is always ‘0‘ except when ‘strategy=”constant”‘ in which case ‘fill_value‘ will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](6,)	['SeniorCitizen','tenure','MonthlyCharges','Tota 'charge_ratio']
indicator_ indicator_: :class:‘~sklearn.impute.MissingIndicator‘ Indicator used to add binary indicators for missing values. ‘None‘ if ‘add_indicator=False‘.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[float64](6,)	[0. , 29. , 70.5 ,1394.92, 3. , 1.]

SeniorCitizen
tenure
MonthlyCharges
TotalCharges
num_services
charge_ratio

	copy copy: bool, default=True If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.	True

	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
	with_std with_std: bool, default=True If True, scale the data to unit variance (or equivalently, unit standard deviation).	True

Name	Type	Value
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.16, 32.49, 64.93,2299.33, 3.38, 1.]
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6

n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the “n_samples_seen“ will be an integer, otherwise it will be an array of dtype int. If ‘sample_weights‘ are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across “partial_fit“ calls.	float64	5634
scale_ scale_: ndarray of shape (n_features,) or None Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using ‘np.sqrt(var_)‘. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. ‘scale_‘ is equal to ‘None‘ when ‘with_std=False‘. .. versionadded:: 0.17 *scale_*	ndarray[float64](6,)	[0.37, 24.57, 30.14,2279. , 2.07, 0.06]

var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute ‘scale_‘. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.14, 603.52, 908.14,5193850.1 , 4.29, 0.]

x0
x1
x2
x3
x4
x5

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'most_frequent'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when 'fit' is called are returned in results when 'transform' is called. The imputed value is always '0' except when 'strategy="constant"' in which case 'fill_value' will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape ('n_features_in_',) Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](16,)	['gender','Partner','Dependents',...,'PaperlessBil 'tenure_group']
indicator_ indicator_: :class:'~sklearn.impute.MissingIndicator' Indicator used to add binary indicators for missing values. 'None' if 'add_indicator=False'.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	16
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[object](16,)	['Male','No','No',...,'Yes','Electronic check','>48 bln']

gender
Partner
Dependents
PhoneService
MultipleLines
InternetService
OnlineSecurity
OnlineBackup
DeviceProtection
TechSupport
StreamingTV
StreamingMovies
Contract
PaperlessBilling
PaymentMethod
tenure_group

	handle_unknown handle_unknown: {'error', 'ignore', 'infrequent_if_exist', 'warn'}, default='error' Specifies the way unknown categories are handled during :meth:‘transform‘. - 'error' : Raise an error if an unknown category is present during transform. - 'ignore' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will be all zeros. In the inverse transform, an unknown category will be denoted as None. - 'infrequent_if_exist' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will map to the infrequent category if it exists. The infrequent category will be mapped to the last position in the encoding. During inverse transform, an unknown category will be mapped to the category denoted ‘'infrequent'‘ if it exists. If the ‘'infrequent'‘ category does not exist, then :meth:‘transform‘ and :meth:‘inverse_transform‘ will handle an unknown category as with ‘handle_unknown='ignore'‘. Infrequent categories exist based on ‘min_frequency‘ and ‘max_categories‘. Read more in the	'ignore'

categories	categories: 'auto' or a list of array-like, default='auto' Categories (unique values) per feature: - 'auto' : Determine categories automatically from the training data. - list : “categories[i]“ holds the categories expected in the ith column. The passed categories should not mix strings and numeric values within a single feature, and should be sorted in case of numeric values. The used categories can be found in the “categories_“ attribute. .. versionadded:: 0.20	'auto'

drop	drop: {'first', 'if_binary'} or an array-like of shape (n_features,), default=None Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model. However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models. - None : retain all features (the default). - 'first' : drop the first category in each feature. If only one category is present, the feature will be dropped entirely. - 'if_binary' : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact. - array : “drop[i]“ is the category in feature “X[:, i]“ that should be dropped. When ‘max_categories‘ or ‘min_frequency‘ is configured to group infrequent categories, the dropping behavior is handled after the	None

sparse_output sparse_output: bool, default=True When “True“, it returns a SciPy sparse matrix/array in ”Compressed Sparse Row” (CSR) format. .. versionadded:: 1.2 ‘sparse‘ was renamed to ‘sparse_output‘	True
dtype dtype: number type, default=np.float64 Desired dtype of output.	<class 'numpy.float64'>
min_frequency min_frequency: int or float, default=None Specifies the minimum frequency below which a category will be considered infrequent. - If ‘int‘, categories with a smaller cardinality will be considered infrequent. - If ‘float‘, categories with a smaller cardinality than ‘min_frequency * n_samples‘ will be considered infrequent. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infrequent_categories>‘.	None

max_categories max_categories: int, default=None Specifies an upper limit to the number of output features for each input feature when considering infrequent categories. If there are infrequent categories, ‘max_categories‘ includes the category representing the infrequent categories along with the frequent categories. If ‘None‘, there is no limit to the number of output features. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infre- quent_categories>‘.	None

	feature_name_combiner feature_name_combiner: ”concat” or callable, default=”concat” Callable with signature ‘def callable(input_feature, category)‘ that returns a string. This is used to create feature names to be returned by :meth:‘get_feature_names_out‘. ‘”concat”‘ concatenates encoded feature name and category with ‘feature + ”_” + str(category)‘.E.g. feature X with values 1, 6, 7 create feature names ‘X_1, X_6, X_7‘. .. versionadded:: 1.3	'concat'

Name	Type	Value
categories_ categories_: list of arrays The categories of each feature determined during fitting (in order of the features in X and corresponding with the output of “transform“). This includes the category specified in “drop“ (if any).	list	[array(['Femal... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), ...]

drop_idx_ drop_idx_: array of shape (n_features,) - “drop_idx_[i]“ is the index in “categories_[i]“ of the category to be dropped for each feature. - “drop_idx_[i] = None“ if no category is to be dropped from the feature with index “i“, e.g. when ‘drop='if_binary'‘ and the feature isn't binary. - “drop_idx_ = None“ if all the transformed features will be retained. If infrequent categories are enabled by setting ‘min_frequency‘ or ‘max_categories‘ to a non-default value and ‘drop_idx[i]‘ corresponds to an infrequent category, then the entire infrequent category is dropped. .. versionchanged:: 0.23 Added the possibility to contain ‘None‘ values.	NoneType	None
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 1.0	int	16

x0_Female
x0_Male
x1_No
x1_Yes
x2_No
x2_Yes
x3_No
x3_Yes
x4_No
x4_No phone service
x4_Yes
x5_DSL
x5_Fiber optic
x5_No
x6_No
x6_No internet service
x6_Yes
x7_No
x7_No internet service
x7_Yes
x8_No
x8_No internet service
x8_Yes
x9_No
x9_No internet service
x9_Yes
x10_No
x10_No internet service
x10_Yes
x11_No
x11_No internet service
x11_Yes
x12_Month-to-month
x12_One year
x12_Two year
x13_No
x13_Yes
x14_Bank transfer (automatic)
x14_Credit card (automatic)
x14_Electronic check
x14_Mailed check
x15_0-6 bln

x15__13-24 bln
x15__25-48 bln
x15__7-12 bln
x15__>48 bln

num___SeniorCitizen
num___tenure
num___MonthlyCharges
num___TotalCharges
num___num_services
num___charge_ratio
cat___gender_Female
cat___gender_Male
cat___Partner_No
cat___Partner_Yes
cat___Dependents_No
cat___Dependents_Yes
cat___PhoneService_No
cat___PhoneService_Yes
cat___MultipleLines_No
cat___MultipleLines_No phone service
cat___MultipleLines_Yes
cat___InternetService_DSL
cat___InternetService_Fiber optic
cat___InternetService_No
cat___OnlineSecurity_No
cat___OnlineSecurity_No internet service
cat___OnlineSecurity_Yes
cat___OnlineBackup_No
cat___OnlineBackup_No internet service
cat___OnlineBackup_Yes
cat___DeviceProtection_No
cat___DeviceProtection_No internet service
cat___DeviceProtection_Yes
cat___TechSupport_No
cat___TechSupport_No internet service
cat___TechSupport_Yes
cat___StreamingTV_No
cat___StreamingTV_No internet service
cat___StreamingTV_Yes
cat___StreamingMovies_No

cat__StreamingMovies_No internet service
cat__StreamingMovies_Yes
cat__Contract_Month-to-month
cat__Contract_One year
cat__Contract_Two year
cat__PaperlessBilling_No
cat__PaperlessBilling_Yes
cat__PaymentMethod_Bank transfer (automatic)
cat__PaymentMethod_Credit card (automatic)
cat__PaymentMethod_Electronic check
cat__PaymentMethod_Mailed check
cat__tenure_group_0-6 bln
cat__tenure_group_13-24 bln
cat__tenure_group_25-48 bln
cat__tenure_group_7-12 bln
cat__tenure_group_>48 bln

n_neighbors int, default=5	n_neighbors:	15
Number of neighbors to use by default for :meth:‘kneighbors‘ queries.		

weights	weights: {'uniform', 'distance'}, callable or None, default='uniform' Weight function used in prediction. Possible values: - 'uniform' : uniform weights. All points in each neighborhood are weighted equally. - 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away. - [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights. Refer to the example entitled :ref:‘sphx_glr_auto_examples_neighbors_plot_classification.py‘ showing the impact of the ‘weights‘ parameter on the decision boundary.	'uniform'

algorithm	algorithm: {'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto' Algorithm used to compute the nearest neighbors: - 'ball_tree' will use :class:‘BallTree‘ - 'kd_tree' will use :class:‘KDTree‘ - 'brute' will use a brute-force search. - 'auto' will attempt to decide the most appropriate algorithm based on the values passed to :meth:‘fit‘ method. Note: fitting on sparse input will override the setting of this parameter, using brute force.	'auto'
leaf_size	leaf_size: int, default=30 Leaf size passed to BallTree or KDTree. This can affect the speed of the construction and query, as well as the memory required to store the tree. The optimal value depends on the nature of the problem.	30

p	p: float, default=2 Power parameter for the Minkowski metric. When p = 1, this is equivalent to using manhattan_distance (l1), and euclidean_distance (l2) for p = 2. For arbitrary p, minkowski_distance (l_p) is used. This parameter is expected to be positive.	2

metric	metric: str or callable, default='minkowski' Metric to use for distance computation. Default is ”minkowski”, which results in the standard Euclidean distance when p = 2. See the documentation of ‘scipy.spatial.distance <https://docs.scipy.org/doc/scipy/reference/spatial.distance.html>‘_ and the metrics listed in :class:‘~sklearn.metrics.pairwise.distance_metrics‘ for valid metric values. If metric is ”precomputed”, X is assumed to be a distance matrix and must be square during fit. X may be a :term:‘sparse graph‘, in which case only ”nonzero” elements may be considered neighbors. If metric is a callable function, it takes two arrays representing 1D vectors as inputs and must return one value indicating the distance between those vectors. This works for Scipy's metrics, but is less efficient than passing the metric name as a string.	'minkowski'

	metric_params metric_params: dict, default=None Additional keyword arguments for the metric function. n_jobs n_jobs: int, default=None The number of parallel jobs to run for neighbors search. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details. Doesn't affect :meth:‘fit‘ method.	None None

Name	Type	Value
classes_ classes_: array of shape (n_classes,) Class labels known to the classifier effective_metric_ effective_metric_: str or callble The distance metric used. It will be same as the ‘metric‘ parameter or a synonym of it, e.g. 'euclidean' if the ‘metric‘ parameter set to 'minkowski' and ‘p‘ parameter set to 2.	ndarray[int64](2,) str	[0,1] 'eu...an'

effective_metric_params_ effective_metric_params_: dict Additional keyword arguments for the metric function. For most metrics will be same with ‘metric_params‘ parameter, but may also contain the ‘p‘ parameter value if the ‘effective_metric_‘ attribute is set to 'minkowski'.	dict	{}
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	52
n_samples_fit_ n_samples_fit_: int Number of samples in the fitted data.	int	5634
outputs_2d_ outputs_2d_: bool False when ‘y‘'s shape is (n_samples,) or (n_samples, 1) during fit otherwise True.	bool	False

Asumsi: pelanggan dengan profil serupa memiliki perilaku serupa. **Kelebihan**: sederhana, tidak ada asumsi distribusi, menangkap pola lokal. **Kekurangan**: **sangat** sensitif terhadap skala (karena itu `StandardScaler` wajib), lambat untuk prediksi pada data besar, tidak interpretabel.

> **Warning**
>
> **KNN dan standardisasi.** Tanpa `StandardScaler`, fitur dengan skala besar (misalnya `TotalCharges` dalam ribuan) akan mendominasi perhitungan jarak, membuat fitur dengan skala kecil (misalnya `SeniorCitizen` 0/1) tidak berpengaruh. Ini adalah alasan `StandardScaler` **wajib** untuk KNN—bukan opsional.

9.3.3 3. Naive Bayes — Probabilitas Sederhana

```
nb = Pipeline([
    ("prep", preprocessor_nb),
    ("clf", GaussianNB()),
])
nb.fit(X_train, y_train)
```

---

steps	steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define ‘fit‘. All non-last steps must also define ‘transform‘. See :ref:‘Combining Estimators <combining_estimators>‘ for more details.	[('prep', ...), ('clf', ...)]

transform_input transform_input: list of str, default=None The names of the :term:‘metadata‘ parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to “fit“ (other than “X“) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:‘metadata routing <metadata_routing>‘. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using “sklearn.set_config(enable_metadata_routing=True)“. .. versionadded:: 1.6	None

	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute “named_steps“ or “steps“ to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:‘sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py‘ for an example on how to enable caching.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[int64](2,)	[0,1]
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Only defined if the underlying estimator exposes such an attribute when fit. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying first estimator in ‘steps‘ exposes such an attribute when fit. .. versionadded:: 0.24	int	22

	transformers transformers: list of tuples List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data. name : str Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using “set_params“ and searched in grid search. transformer : {'drop', 'passthrough'} or estimator Estimator must support :term:‘fit‘ and :term:‘transform‘. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively. columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where “transformer“ expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data ‘X‘ and can return any of the above. To select multiple columns by name or dtype, you can use :obj:‘make_column_selector‘.	[('num', ...)]

	remainder remainder: {'drop', 'passthrough'} or estimator, default='drop' By default, only the specified columns in ‘transformers‘ are transformed and combined in the output, and the non-specified columns are dropped. (default of “'drop'“). By specifying “remainder='passthrough'“, all remaining columns that were not specified in ‘transformers‘, but present in the data passed to ‘fit‘ will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during ‘fit‘ will be excluded from the output of ‘transform‘. By setting “remainder“ to be an estimator, the remaining non-specified columns will use the “remainder“ estimator. The estimator must support :term:‘fit‘ and :term:‘transform‘. Note that using this feature requires that the DataFrame columns input at :term:‘fit‘ and :term:‘transform‘ have identical order.	'drop'

	sparse_threshold sparse_threshold: float, default=0.3 If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use “sparse_threshold=0“ to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.	0.3
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	None
	transformer_weights transformer_weights: dict, default=None Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each transformer will be printed as it is completed.	False

verbose_feature_names_out verbose_feature_names_out: bool, str or Callable[[str, str], str], default=True - If True, :meth:‘ColumnTransformer.get_feature_names_out‘ will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting ‘verbose_feature_names_out=”{transformer_name}___{feature_name}”‘. - If False, :meth:‘ColumnTransformer.get_feature_names_out‘ will not prefix any feature names and will error if feature names are not unique. - If “Callable[[str, str], str]“, :meth:‘ColumnTransformer.get_feature_names_out‘ will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name. - If “str“, it must be a string ready for formatting. The given string will be formatted using two field names: “transformer_name“ and “feature_name“. e.g. “”{feature_name}___{transformer_name}”“. See :meth:‘str.format‘ method from the standard library for more info. .. versionadded:: 1.0	True

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying transformers expose such an attribute when fit. .. versionadded:: 0.24	int	22
named_transformers_ named_transformers_: :class:‘~sklearn.utils.Bunch‘ Read-only attribute to access any transformer by given name. Keys are transformer names and values are the fitted transformer objects.	Bunch	{'num': Pipel...nder': 'drop'}

output_indices_ output_indices_: dict A dictionary from each transformer name to a slice, where the slice corresponds to indices in the transformed output. This is useful to inspect which transformer is responsible for which transformed feature(s). .. versionadded:: 1.0	dict	{'num': slice(0, 6, None), 're...er': slice(0, 0, None)}
sparse_output_ sparse_output_: bool Boolean flag indicating whether the output of “transform“ is a sparse matrix or a dense numpy array, which depends on the output of the individual transformers and the ‘sparse_threshold‘ keyword.	bool	False

transformers_ transformers_: list The collection of fitted transformers as tuples of (name, fitted_transformer, column). ‘fitted_transformer‘ can be an estimator, or ‘'drop'‘; ‘'passthrough'‘ is replaced with an equivalent :class:‘~sklearn.preprocessing.FunctionTransformer‘. In case there were no columns selected, this will be the unfitted transformer. If there are remaining columns, the final element is a tuple of the form: ('remainder', transformer, remaining_columns) corresponding to the “remainder“ parameter. If there are remaining columns, then “len(transformers_)==len(transformers)+1“, otherwise “len(transformers_)==len(transformers)“. .. versionadded:: 1.7 The format of the remaining columns now attempts to match that of the other transformers: if all columns were provided as column names (‘str‘), the remaining columns are stored as column names; if all columns were provided as mask arrays (‘bool‘), so are the remaining columns; in all other cases the remaining columns are stored as indices (‘int‘).	list	[('num', Pipeline(step...ardScaler())]), ['Se...en', 'tenure', 'Mo...es', 'To...es', ...]), ('re...er', 'drop', ['gender', 'Partner', 'De...ts', 'Ph...ce', ...])]

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'median'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when ‘fit‘ is called are returned in results when ‘transform‘ is called. The imputed value is always ‘0‘ except when ‘strategy=”constant”‘ in which case ‘fill_value‘ will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](6,)	['SeniorCitizen','tenure','MonthlyCharges','Tota 'charge_ratio']
indicator_ indicator_: :class:‘~sklearn.impute.MissingIndicator‘ Indicator used to add binary indicators for missing values. ‘None‘ if ‘add_indicator=False‘.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[float64](6,)	[0. , 29. , 70.5 ,1394.92, 3. , 1.]

SeniorCitizen
tenure
MonthlyCharges
TotalCharges
num_services
charge_ratio

	copy copy: bool, default=True If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.	True

	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
	with_std with_std: bool, default=True If True, scale the data to unit variance (or equivalently, unit standard deviation).	True

Name	Type	Value
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.16, 32.49, 64.93,2299.33, 3.38, 1.]
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6

n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the “n_samples_seen“ will be an integer, otherwise it will be an array of dtype int. If ‘sample_weights‘ are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across “partial_fit“ calls.	float64	5634
scale_ scale_: ndarray of shape (n_features,) or None Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using ‘np.sqrt(var_)‘. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. ‘scale_‘ is equal to ‘None‘ when ‘with_std=False‘. .. versionadded:: 0.17 *scale_*	ndarray[float64](6,)	[0.37, 24.57, 30.14,2279. , 2.07, 0.06]

var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute ‘scale_‘. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.14, 603.52, 908.14,5193850.1 , 4.29, 0.]

x0
x1
x2
x3
x4
x5

	priors priors: array-like of shape (n_classes,), default=None Prior probabilities of the classes. If specified, the priors are not adjusted according to the data.	None
	var_smoothing var_smoothing: float, default=1e-9 Portion of the largest variance of all features that is added to variances for calculation stability. .. versionadded:: 0.20	1e-09

Name	Type	Value
class_count_ class_count_: ndarray of shape (n_classes,) number of training samples observed in each class.	ndarray[float64](2,)	[4139.,1495.]
class_prior_ class_prior_: ndarray of shape (n_classes,) probability of each class.	ndarray[float64](2,)	[0.73,0.27]
classes_ classes_: ndarray of shape (n_classes,) class labels known to the classifier.	ndarray[int64](2,)	[0,1]
epsilon_ epsilon_: float absolute additive value to variances.	float64	1e-09
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6
theta_ theta_: ndarray of shape (n_classes, n_features) mean of each feature per class.	ndarray[float64](2, 6)	[[-0.09, 0.21,-0.12, 0.12, 0.04, 0.01], [0.24,-0.58, 0.33,-0.32,-0.1 ,-0.03]]
var_ var_: ndarray of shape (n_classes, n_features) Variance of each feature per class. .. versionadded:: 1.0	ndarray[float64](2, 6)	[[0.83,0.97,1.07,1.06,1.1 ,1.14], [1.38,0.65,0.67,0.7 ,0.72,0.61]]

Asumsi: fitur **independen** satu sama lain (kondisional terhadap kelas). **Kelebihan**: sangat cepat, bekerja baik pada data kecil, tidak memerlukan scaling. **Kekurangan**: asumsi

independensi hampir selalu dilanggar di dunia nyata; performa sering kalah dari model lain.

> **Note**
>
> **Mengapa Naive Bayes tetap layak dipelajari?** Meskipun asumsi independensinya sering dilanggar, Naive Bayes **sering bekerja mengejutkan baik**—terutama untuk klasifikasi teks. Ia juga menjadi **baseline yang sangat cepat**, dan memberikan intuisi tentang bagaimana probabilitas kondisional bekerja. Di bab ini, kita memakainya sebagai pembanding, bukan sebagai kandidat utama.

9.3.4 4. Decision Tree — Aturan yang Dapat Dibaca

```
tree = Pipeline([
    ("prep", preprocessor),
    ("clf", DecisionTreeClassifier(max_depth=5, random_state=RANDOM_STATE)),
])
tree.fit(X_train, y_train)
```

---

steps	steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define ‘fit‘. All non-last steps must also define ‘transform‘. See :ref:‘Combining Estimators <combining_estimators>‘ for more details.	[('prep', ...), ('clf', ...)]

transform_input transform_input: list of str, default=None The names of the :term:‘metadata‘ parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to “fit“ (other than “X“) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:‘metadata routing <metadata_routing>‘. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using “sklearn.set_config(enable_metadata_routing=True)“. .. versionadded:: 1.6	None

	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute “named_steps“ or “steps“ to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:‘sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py‘ for an example on how to enable caching.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[int64](2,)	[0,1]
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Only defined if the underlying estimator exposes such an attribute when fit. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying first estimator in ‘steps‘ exposes such an attribute when fit. .. versionadded:: 0.24	int	22

	transformers transformers: list of tuples List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data. name : str Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using “set_params“ and searched in grid search. transformer : {'drop', 'passthrough'} or estimator Estimator must support :term:‘fit‘ and :term:‘transform‘. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively. columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where “transformer“ expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data ‘X‘ and can return any of the above. To select multiple columns by name or dtype, you can use :obj:‘make_column_selector‘.	[('num', ...), ('cat', ...)]

	remainder remainder: {'drop', 'passthrough'} or estimator, default='drop' By default, only the specified columns in ‘transformers‘ are transformed and combined in the output, and the non-specified columns are dropped. (default of “'drop'“). By specifying “remainder='passthrough'“, all remaining columns that were not specified in ‘transformers‘, but present in the data passed to ‘fit‘ will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during ‘fit‘ will be excluded from the output of ‘transform‘. By setting “remainder“ to be an estimator, the remaining non-specified columns will use the “remainder“ estimator. The estimator must support :term:‘fit‘ and :term:‘transform‘. Note that using this feature requires that the DataFrame columns input at :term:‘fit‘ and :term:‘transform‘ have identical order.	'drop'

	sparse_threshold sparse_threshold: float, default=0.3 If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use “sparse_threshold=0“ to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.	0.3
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	None
	transformer_weights transformer_weights: dict, default=None Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each transformer will be printed as it is completed.	False

verbose_feature_names_out verbose_feature_names_out: bool, str or Callable[[str, str], str], default=True - If True, :meth:‘ColumnTransformer.get_feature_names_out‘ will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting ‘verbose_feature_names_out=”{transformer_name}___{feature_name}”‘. - If False, :meth:‘ColumnTransformer.get_feature_names_out‘ will not prefix any feature names and will error if feature names are not unique. - If “Callable[[str, str], str]“, :meth:‘ColumnTransformer.get_feature_names_out‘ will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name. - If “str“, it must be a string ready for formatting. The given string will be formatted using two field names: “transformer_name“ and “feature_name“. e.g. “”{feature_name}___{transformer_name}”“. See :meth:‘str.format‘ method from the standard library for more info. .. versionadded:: 1.0	True

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying transformers expose such an attribute when fit. .. versionadded:: 0.24	int	22
named_transformers_ named_transformers_: :class:‘~sklearn.utils.Bunch‘ Read-only attribute to access any transformer by given name. Keys are transformer names and values are the fitted transformer objects.	Bunch	{'num': Pipel...='ignore'))])}

output_indices_ output_indices_: dict A dictionary from each transformer name to a slice, where the slice corresponds to indices in the transformed output. This is useful to inspect which transformer is responsible for which transformed feature(s). .. versionadded:: 1.0	dict	{'cat': slice(6, 52, None), 'num': slice(0, 6, None), 're...er': slice(0, 0, None)}
sparse_output_ sparse_output_: bool Boolean flag indicating whether the output of “transform“ is a sparse matrix or a dense numpy array, which depends on the output of the individual transformers and the ‘sparse_threshold‘ keyword.	bool	False

transformers_ transformers_: list The collection of fitted transformers as tuples of (name, fitted_transformer, column). ‘fitted_transformer‘ can be an estimator, or ‘'drop'‘; ‘'passthrough'‘ is replaced with an equivalent :class:‘~sklearn.preprocessing.FunctionTransformer‘. In case there were no columns selected, this will be the unfitted transformer. If there are remaining columns, the final element is a tuple of the form: ('remainder', transformer, remaining_columns) corresponding to the “remainder“ parameter. If there are remaining columns, then “len(transformers_)==len(transformers)+1“, otherwise “len(transformers_)==len(transformers)“. .. versionadded:: 1.7 The format of the remaining columns now attempts to match that of the other transformers: if all columns were provided as column names (‘str‘), the remaining columns are stored as column names; if all columns were provided as mask arrays (‘bool‘), so are the remaining columns; in all other cases the remaining columns are stored as indices (‘int‘).	list	[('num', Pipeline(step...ardScaler())]), ['Se...en', 'tenure', 'Mo...es', 'To...es', ...]), ('cat', Pipeline(step...n='ignore'))]), ['gender', 'Partner', 'De...ts', 'Ph...ce', ...])]

strategy	strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'median'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when ‘fit‘ is called are returned in results when ‘transform‘ is called. The imputed value is always ‘0‘ except when ‘strategy=”constant”‘ in which case ‘fill_value‘ will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](6,)	['SeniorCitizen','tenure','MonthlyCharges','Tota 'charge_ratio']
indicator_ indicator_: :class:‘~sklearn.impute.MissingIndicator‘ Indicator used to add binary indicators for missing values. ‘None‘ if ‘add_indicator=False‘.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[float64](6,)	[0. , 29. , 70.5 ,1394.92, 3. , 1.]

SeniorCitizen
tenure
MonthlyCharges
TotalCharges
num_services
charge_ratio

	copy copy: bool, default=True If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.	True

	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
	with_std with_std: bool, default=True If True, scale the data to unit variance (or equivalently, unit standard deviation).	True

Name	Type	Value
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.16, 32.49, 64.93,2299.33, 3.38, 1.]
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6

n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the “n_samples_seen“ will be an integer, otherwise it will be an array of dtype int. If ‘sample_weights‘ are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across “partial_fit“ calls.	float64	5634
scale_ scale_: ndarray of shape (n_features,) or None Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using ‘np.sqrt(var_)‘. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. ‘scale_‘ is equal to ‘None‘ when ‘with_std=False‘. .. versionadded:: 0.17 *scale_*	ndarray[float64](6,)	[0.37, 24.57, 30.14,2279. , 2.07, 0.06]

var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute ‘scale_‘. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.14, 603.52, 908.14,5193850.1 , 4.29, 0.]

x0
x1
x2
x3
x4
x5

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'most_frequent'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when 'fit' is called are returned in results when 'transform' is called. The imputed value is always '0' except when 'strategy="constant"' in which case 'fill_value' will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape ('n_features_in_',) Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](16,)	['gender','Partner','Dependents',...,'PaperlessBil 'tenure_group']
indicator_ indicator_: :class:'~sklearn.impute.MissingIndicator' Indicator used to add binary indicators for missing values. 'None' if 'add_indicator=False'.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	16
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[object](16,)	['Male','No','No',...,'Yes','Electronic check','>48 bln']

gender
Partner
Dependents
PhoneService
MultipleLines
InternetService
OnlineSecurity
OnlineBackup
DeviceProtection
TechSupport
StreamingTV
StreamingMovies
Contract
PaperlessBilling
PaymentMethod
tenure_group

	handle_unknown handle_unknown: {'error', 'ignore', 'infrequent_if_exist', 'warn'}, default='error' Specifies the way unknown categories are handled during :meth:‘transform‘. - 'error' : Raise an error if an unknown category is present during transform. - 'ignore' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will be all zeros. In the inverse transform, an unknown category will be denoted as None. - 'infrequent_if_exist' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will map to the infrequent category if it exists. The infrequent category will be mapped to the last position in the encoding. During inverse transform, an unknown category will be mapped to the category denoted ‘'infrequent'‘ if it exists. If the ‘'infrequent'‘ category does not exist, then :meth:‘transform‘ and :meth:‘inverse_transform‘ will handle an unknown category as with ‘handle_unknown='ignore'‘. Infrequent categories exist based on ‘min_frequency‘ and ‘max_categories‘. Read more in the	'ignore'

categories	categories: 'auto' or a list of array-like, default='auto' Categories (unique values) per feature: - 'auto' : Determine categories automatically from the training data. - list : “categories[i]“ holds the categories expected in the ith column. The passed categories should not mix strings and numeric values within a single feature, and should be sorted in case of numeric values. The used categories can be found in the “categories_“ attribute. .. versionadded:: 0.20	'auto'

drop	drop: {'first', 'if_binary'} or an array-like of shape (n_features,), default=None Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model. However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models. - None : retain all features (the default). - 'first' : drop the first category in each feature. If only one category is present, the feature will be dropped entirely. - 'if_binary' : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact. - array : “drop[i]“ is the category in feature “X[:, i]“ that should be dropped. When ‘max_categories‘ or ‘min_frequency‘ is configured to group infrequent categories, the dropping behavior is handled after the	None

sparse_output sparse_output: bool, default=True When “True“, it returns a SciPy sparse matrix/array in ”Compressed Sparse Row” (CSR) format. .. versionadded:: 1.2 ‘sparse‘ was renamed to ‘sparse_output‘	True
dtype dtype: number type, default=np.float64 Desired dtype of output.	<class 'numpy.float64'>
min_frequency min_frequency: int or float, default=None Specifies the minimum frequency below which a category will be considered infrequent. - If ‘int‘, categories with a smaller cardinality will be considered infrequent. - If ‘float‘, categories with a smaller cardinality than ‘min_frequency * n_samples‘ will be considered infrequent. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infrequent_categories>‘.	None

max_categories max_categories: int, default=None Specifies an upper limit to the number of output features for each input feature when considering infrequent categories. If there are infrequent categories, ‘max_categories‘ includes the category representing the infrequent categories along with the frequent categories. If ‘None‘, there is no limit to the number of output features. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infrequent_categories>‘.	None

	feature_name_combiner feature_name_combiner: ”concat” or callable, default=”concat” Callable with signature ‘def callable(input_feature, category)‘ that returns a string. This is used to create feature names to be returned by :meth:‘get_feature_names_out‘. ‘”concat”‘ concatenates encoded feature name and category with ‘feature + ”_” + str(category)‘.E.g. feature X with values 1, 6, 7 create feature names ‘X_1, X_6, X_7‘. .. versionadded:: 1.3	'concat'

Name	Type	Value
categories_ categories_: list of arrays The categories of each feature determined during fitting (in order of the features in X and corresponding with the output of “transform“). This includes the category specified in “drop“ (if any).	list	[array(['Femal... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), ...]

drop_idx_ drop_idx_: array of shape (n_features,) - “drop_idx_[i]“ is the index in “categories_[i]“ of the category to be dropped for each feature. - “drop_idx_[i] = None“ if no category is to be dropped from the feature with index “i“, e.g. when ‘drop='if_binary'‘ and the feature isn't binary. - “drop_idx_ = None“ if all the transformed features will be retained. If infrequent categories are enabled by setting ‘min_frequency‘ or ‘max_categories‘ to a non-default value and ‘drop_idx[i]‘ corresponds to an infrequent category, then the entire infrequent category is dropped. .. versionchanged:: 0.23 Added the possibility to contain ‘None‘ values.	NoneType	None
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 1.0	int	16

x0_Female
x0_Male
x1_No
x1_Yes
x2_No
x2_Yes
x3_No
x3_Yes
x4_No
x4_No phone service
x4_Yes
x5_DSL
x5_Fiber optic
x5_No
x6_No
x6_No internet service
x6_Yes
x7_No
x7_No internet service
x7_Yes
x8_No
x8_No internet service
x8_Yes
x9_No
x9_No internet service
x9_Yes
x10_No
x10_No internet service
x10_Yes
x11_No
x11_No internet service
x11_Yes
x12_Month-to-month
x12_One year
x12_Two year
x13_No
x13_Yes
x14_Bank transfer (automatic)
x14_Credit card (automatic)
x14_Electronic check
x14_Mailed check
x15_0-6 bln

x15__13-24 bln
x15__25-48 bln
x15__7-12 bln
x15__>48 bln

num___SeniorCitizen
num___tenure
num___MonthlyCharges
num___TotalCharges
num___num_services
num___charge_ratio
cat___gender_Female
cat___gender_Male
cat___Partner_No
cat___Partner_Yes
cat___Dependents_No
cat___Dependents_Yes
cat___PhoneService_No
cat___PhoneService_Yes
cat___MultipleLines_No
cat___MultipleLines_No phone service
cat___MultipleLines_Yes
cat___InternetService_DSL
cat___InternetService_Fiber optic
cat___InternetService_No
cat___OnlineSecurity_No
cat___OnlineSecurity_No internet service
cat___OnlineSecurity_Yes
cat___OnlineBackup_No
cat___OnlineBackup_No internet service
cat___OnlineBackup_Yes
cat___DeviceProtection_No
cat___DeviceProtection_No internet service
cat___DeviceProtection_Yes
cat___TechSupport_No
cat___TechSupport_No internet service
cat___TechSupport_Yes
cat___StreamingTV_No
cat___StreamingTV_No internet service
cat___StreamingTV_Yes
cat___StreamingMovies_No

cat__StreamingMovies_No internet service
cat__StreamingMovies_Yes
cat__Contract_Month-to-month
cat__Contract_One year
cat__Contract_Two year
cat__PaperlessBilling_No
cat__PaperlessBilling_Yes
cat__PaymentMethod_Bank transfer (automatic)
cat__PaymentMethod_Credit card (automatic)
cat__PaymentMethod_Electronic check
cat__PaymentMethod_Mailed check
cat__tenure_group_0-6 bln
cat__tenure_group_13-24 bln
cat__tenure_group_25-48 bln
cat__tenure_group_7-12 bln
cat__tenure_group_>48 bln

max_depth	max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	5

random_state random_state: int, RandomState instance or None, default=None Controls the randomness of the estimator. The features are always randomly permuted at each split, even if “splitter“ is set to “”best”“. When “max_features < n_features“, the algorithm will select “max_features“ at random at each split before finding the best split among them. But the best found split may vary across different runs, even if “max_features=n_features“. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, “random_state“ has to be fixed to an integer. See :term:‘Glossary <random_state>‘ for details.	42

	criterion criterion: {”gini”, ”entropy”, ”log_loss”}, default=”gini” The function to measure the quality of a split. Supported criteria are ”gini” for the Gini impurity and ”log_loss” and ”entropy” both for the Shannon information gain, see :ref:‘tree_mathematical_formulation‘.	'gini'
	splitter splitter: {”best”, ”random”}, default=”best” The strategy used to choose the split at each node. Supported strategies are ”best” to choose the best split and ”random” to choose the best random split among considered features for this split.	'best'

min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider ‘min_samples_split‘ as the minimum number. - If float, then ‘min_samples_split‘ is a fraction and ‘ceil(min_samples_split * n_samples)‘ are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	2

min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least “min_samples_leaf“ training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression. - If int, then consider ‘min_samples_leaf‘ as the minimum number. - If float, then ‘min_samples_leaf‘ is a fraction and ‘ceil(min_samples_leaf * n_samples)‘ are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	1

	min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.	0.0
	max_features max_features: int, float or {”sqrt”, ”log2”}, default=None The number of features to consider when looking for the best split: - If int, then consider ‘max_features‘ features at each split. - If float, then ‘max_features‘ is a fraction and ‘max(1, int(max_features * n_features_in_))‘ features are considered at each split. - If ”sqrt”, then ‘max_fea- tures=sqrt(n_features)‘. - If ”log2”, then ‘max_fea- tures=log2(n_features)‘. - If None, then ‘max_features=n_features‘. Note: splitting may inspect more than “max_features“ features if needed to find a valid split.	None

max_leaf_nodes max_leaf_nodes: int, default=None Grow a tree with “max_leaf_nodes“ in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None

min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where “N“ is the total number of samples, “N_t“ is the number of samples at the current node, “N_t_L“ is the number of samples in the left child, and “N_t_R“ is the number of samples in the right child. “N“, “N_t“, “N_t_R“ and “N_t_L“ all refer to the weighted sum, if “sample_weight“ is passed. .. versionadded:: 0.19	0.0

class_weight	class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form "{class_label: weight}". If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as "n_samples / (n_classes * np.bincount(y))" For multi-output, the weights of each column of y will be multiplied. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	None

ccp_alpha	ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “ccp_alpha“ will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning‘ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py‘ for an example of such pruning. .. versionadded:: 0.22	0.0

	monotonic_cst monotonic_cst: array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when ‘n_classes > 2‘), - multioutput classifications (i.e. when ‘n_outputs_ > 1‘). The constraints hold over the probability of the positive class. Read more in the :ref:‘User Guide <monotonic_cst_gbdt>‘. .. versionadded:: 1.4	None

Name	Type	Value

classes_ classes_: ndarray of shape (n_classes,) or list of ndarray The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
feature_importances_ feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4]_. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:‘sklearn.inspection.permutation_importance‘ as an alternative.	ndarray[float64](52,)	[0. ,0.16,0.03,...,0. ,0. ,0.]
max_features_ max_features_: int The inferred value of max_features.	int	52

n_classes_ n_classes_: int or list of int The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).	int64	np.int64(2)
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	52
n_outputs_ n_outputs_: int The number of outputs when “fit“ is performed.	int	1
tree_ tree_: Tree instance The underlying Tree object. Please refer to “help(sklearn.tree._tree.Tree)“ for attributes of Tree object and :ref:‘sphx_glr_auto_examples_tree_plot_unveil_tree_structure.py‘ for basic usage of these attributes.	Tree	<sklearn.tree...t 0x12a739bc0>

Asumsi: keputusan dapat direpresentasikan sebagai serangkaian aturan if-then. **Kelebihan**: **sangat interpretabel** (dapat digambar sebagai pohon), menangkap interaksi non-linear, tidak memerlukan scaling. **Kekurangan**: cenderung **overfitting** (karena itu `max_depth=5`), tidak stabil (perubahan kecil di data bisa mengubah struktur pohon secara drastis).

9.3.5 5. Random Forest — Kekuatan Ensemble

```
rf = Pipeline([
    ("prep", preprocessor),
    ("clf", RandomForestClassifier(
        n_estimators=200, max_depth=10, random_state=RANDOM_STATE, n_jobs=-1
    )),
])
rf.fit(X_train, y_train)
```

---

steps	steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define ‘fit‘. All non-last steps must also define ‘transform‘. See :ref:‘Combining Estimators <combining_estimators>‘ for more details.	[('prep', ...), ('clf', ...)]

transform_input transform_input: list of str, default=None The names of the :term:‘metadata‘ parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to “fit“ (other than “X“) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:‘metadata routing <metadata_routing>‘. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using “sklearn.set_config(enable_metadata_routing=True)“. .. versionadded:: 1.6	None

	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute “named_steps“ or “steps“ to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:‘sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py‘ for an example on how to enable caching.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[int64](2,)	[0,1]
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Only defined if the underlying estimator exposes such an attribute when fit. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying first estimator in ‘steps‘ exposes such an attribute when fit. .. versionadded:: 0.24	int	22

	transformers transformers: list of tuples List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data. name : str Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using “set_params“ and searched in grid search. transformer : {'drop', 'passthrough'} or estimator Estimator must support :term:‘fit‘ and :term:‘transform‘. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively. columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where “transformer“ expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data ‘X‘ and can return any of the above. To select multiple columns by name or dtype, you can use :obj:‘make_column_selector‘.	[('num', ...), ('cat', ...)]

	remainder remainder: {'drop', 'passthrough'} or estimator, default='drop' By default, only the specified columns in ‘transformers‘ are transformed and combined in the output, and the non-specified columns are dropped. (default of “'drop'“). By specifying “remainder='passthrough'“, all remaining columns that were not specified in ‘transformers‘, but present in the data passed to ‘fit‘ will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during ‘fit‘ will be excluded from the output of ‘transform‘. By setting “remainder“ to be an estimator, the remaining non-specified columns will use the “remainder“ estimator. The estimator must support :term:‘fit‘ and :term:‘transform‘. Note that using this feature requires that the DataFrame columns input at :term:‘fit‘ and :term:‘transform‘ have identical order.	'drop'

	sparse_threshold sparse_threshold: float, default=0.3 If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use “sparse_threshold=0“ to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.	0.3
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	None
	transformer_weights transformer_weights: dict, default=None Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each transformer will be printed as it is completed.	False

verbose_feature_names_out verbose_feature_names_out: bool, str or Callable[[str, str], str], default=True - If True, :meth:‘ColumnTransformer.get_feature_names_out‘ will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting ‘verbose_feature_names_out=”{transformer_name}___{feature_name}”‘. - If False, :meth:‘ColumnTransformer.get_feature_names_out‘ will not prefix any feature names and will error if feature names are not unique. - If “Callable[[str, str], str]“, :meth:‘ColumnTransformer.get_feature_names_out‘ will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name. - If “str“, it must be a string ready for formatting. The given string will be formatted using two field names: “transformer_name“ and “feature_name“. e.g. “”{feature_name}___{transformer_name}”“. See :meth:‘str.format‘ method from the standard library for more info. .. versionadded:: 1.0	True

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying transformers expose such an attribute when fit. .. versionadded:: 0.24	int	22
named_transformers_ named_transformers_: :class:‘~sklearn.utils.Bunch‘ Read-only attribute to access any transformer by given name. Keys are transformer names and values are the fitted transformer objects.	Bunch	{'num': Pipel...='ignore'))])}

output_indices_ output_indices_: dict A dictionary from each transformer name to a slice, where the slice corresponds to indices in the transformed output. This is useful to inspect which transformer is responsible for which transformed feature(s). .. versionadded:: 1.0	dict	{'cat': slice(6, 52, None), 'num': slice(0, 6, None), 're...er': slice(0, 0, None)}
sparse_output_ sparse_output_: bool Boolean flag indicating whether the output of “transform“ is a sparse matrix or a dense numpy array, which depends on the output of the individual transformers and the ‘sparse_threshold‘ keyword.	bool	False

transformers_ transformers_: list The collection of fitted transformers as tuples of (name, fitted_transformer, column). ‘fitted_transformer‘ can be an estimator, or ‘'drop'‘; ‘'passthrough'‘ is replaced with an equivalent :class:‘~sklearn.preprocessing.FunctionTransformer‘. In case there were no columns selected, this will be the unfitted transformer. If there are remaining columns, the final element is a tuple of the form: ('remainder', transformer, remaining_columns) corresponding to the “remainder“ parameter. If there are remaining columns, then “len(transformers_)==len(transformers)+1“, otherwise “len(transformers_)==len(transformers)“. .. versionadded:: 1.7 The format of the remaining columns now attempts to match that of the other transformers: if all columns were provided as column names (‘str‘), the remaining columns are stored as column names; if all columns were provided as mask arrays (‘bool‘), so are the remaining columns; in all other cases the remaining columns are stored as indices (‘int‘).	list	[('num', Pipeline(step...ardScaler())]), ['Se...en', 'tenure', 'Mo...es', 'To...es', ...]), ('cat', Pipeline(step...n='ignore'))]), ['gender', 'Partner', 'De...ts', 'Ph...ce', ...])]

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'median'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when ‘fit‘ is called are returned in results when ‘transform‘ is called. The imputed value is always ‘0‘ except when ‘strategy=”constant”‘ in which case ‘fill_value‘ will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](6,)	['SeniorCitizen','tenure','MonthlyCharges','Tota 'charge_ratio']
indicator_ indicator_: :class:‘~sklearn.impute.MissingIndicator‘ Indicator used to add binary indicators for missing values. ‘None‘ if ‘add_indicator=False‘.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[float64](6,)	[0. , 29. , 70.5 ,1394.92, 3. , 1.]

SeniorCitizen
tenure
MonthlyCharges
TotalCharges
num_services
charge_ratio

	copy copy: bool, default=True If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.	True

	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
	with_std with_std: bool, default=True If True, scale the data to unit variance (or equivalently, unit standard deviation).	True

Name	Type	Value
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.16, 32.49, 64.93,2299.33, 3.38, 1.]
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6

n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the “n_samples_seen“ will be an integer, otherwise it will be an array of dtype int. If ‘sample_weights‘ are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across “partial_fit“ calls.	float64	5634
scale_ scale_: ndarray of shape (n_features,) or None Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using ‘np.sqrt(var_)‘. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. ‘scale_‘ is equal to ‘None‘ when ‘with_std=False‘. .. versionadded:: 0.17 *scale_*	ndarray[float64](6,)	[0.37, 24.57, 30.14,2279. , 2.07, 0.06]

var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute ‘scale_‘. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.14, 603.52, 908.14,5193850.1 , 4.29, 0.]

x0
x1
x2
x3
x4
x5

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'most_frequent'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when 'fit' is called are returned in results when 'transform' is called. The imputed value is always '0' except when 'strategy="constant"' in which case 'fill_value' will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape ('n_features_in_',) Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](16,)	['gender','Partner','Dependents',...,'PaperlessBil 'tenure_group']
indicator_ indicator_: :class:'~sklearn.impute.MissingIndicator' Indicator used to add binary indicators for missing values. 'None' if 'add_indicator=False'.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	16
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[object](16,)	['Male','No','No',...,'Yes','Electronic check','>48 bln']

gender
Partner
Dependents
PhoneService
MultipleLines
InternetService
OnlineSecurity
OnlineBackup
DeviceProtection
TechSupport
StreamingTV
StreamingMovies
Contract
PaperlessBilling
PaymentMethod
tenure_group

	handle_unknown handle_unknown: {'error', 'ignore', 'infrequent_if_exist', 'warn'}, default='error' Specifies the way unknown categories are handled during :meth:‘transform‘. - 'error' : Raise an error if an unknown category is present during transform. - 'ignore' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will be all zeros. In the inverse transform, an unknown category will be denoted as None. - 'infrequent_if_exist' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will map to the infrequent category if it exists. The infrequent category will be mapped to the last position in the encoding. During inverse transform, an unknown category will be mapped to the category denoted ‘'infrequent'‘ if it exists. If the ‘'infrequent'‘ category does not exist, then :meth:‘transform‘ and :meth:‘inverse_transform‘ will handle an unknown category as with ‘handle_unknown='ignore'‘. Infrequent categories exist based on ‘min_frequency‘ and ‘max_categories‘. Read more in the	'ignore'

categories	categories: 'auto' or a list of array-like, default='auto' Categories (unique values) per feature: - 'auto' : Determine categories automatically from the training data. - list : “categories[i]“ holds the categories expected in the ith column. The passed categories should not mix strings and numeric values within a single feature, and should be sorted in case of numeric values. The used categories can be found in the “categories_“ attribute. .. versionadded:: 0.20	'auto'

drop	drop: {'first', 'if_binary'} or an array-like of shape (n_features,), default=None Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model. However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models. - None : retain all features (the default). - 'first' : drop the first category in each feature. If only one category is present, the feature will be dropped entirely. - 'if_binary' : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact. - array : “drop[i]“ is the category in feature “X[:, i]“ that should be dropped. When ‘max_categories‘ or ‘min_frequency‘ is configured to group infrequent categories, the dropping behavior is handled after the	None

sparse_output sparse_output: bool, default=True When “True“, it returns a SciPy sparse matrix/array in ”Compressed Sparse Row” (CSR) format. .. versionadded:: 1.2 ‘sparse‘ was renamed to ‘sparse_output‘	True
dtype dtype: number type, default=np.float64 Desired dtype of output.	<class 'numpy.float64'>
min_frequency min_frequency: int or float, default=None Specifies the minimum frequency below which a category will be considered infrequent. - If ‘int‘, categories with a smaller cardinality will be considered infrequent. - If ‘float‘, categories with a smaller cardinality than ‘min_frequency * n_samples‘ will be considered infrequent. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infrequent_categories>‘.	None

max_categories max_categories: int, default=None Specifies an upper limit to the number of output features for each input feature when considering infrequent categories. If there are infrequent categories, ‘max_categories‘ includes the category representing the infrequent categories along with the frequent categories. If ‘None‘, there is no limit to the number of output features. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infre- quent_categories>‘.	None

	feature_name_combiner feature_name_combiner: ”concat” or callable, default=”concat” Callable with signature ‘def callable(input_feature, category)‘ that returns a string. This is used to create feature names to be returned by :meth:‘get_feature_names_out‘. ‘”concat”‘ concatenates encoded feature name and category with ‘feature + ”_” + str(category)‘.E.g. feature X with values 1, 6, 7 create feature names ‘X_1, X_6, X_7‘. .. versionadded:: 1.3	'concat'

Name	Type	Value
categories_ categories_: list of arrays The categories of each feature determined during fitting (in order of the features in X and corresponding with the output of “transform“). This includes the category specified in “drop“ (if any).	list	[array(['Femal... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), ...]

drop_idx_ drop_idx_: array of shape (n_features,) - “drop_idx_[i]“ is the index in “categories_[i]“ of the category to be dropped for each feature. - “drop_idx_[i] = None“ if no category is to be dropped from the feature with index “i“, e.g. when ‘drop='if_binary'‘ and the feature isn't binary. - “drop_idx_ = None“ if all the transformed features will be retained. If infrequent categories are enabled by setting ‘min_frequency‘ or ‘max_categories‘ to a non-default value and ‘drop_idx[i]‘ corresponds to an infrequent category, then the entire infrequent category is dropped. .. versionchanged:: 0.23 Added the possibility to contain ‘None‘ values.	NoneType	None
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 1.0	int	16

x0_Female
x0_Male
x1_No
x1_Yes
x2_No
x2_Yes
x3_No
x3_Yes
x4_No
x4_No phone service
x4_Yes
x5_DSL
x5_Fiber optic
x5_No
x6_No
x6_No internet service
x6_Yes
x7_No
x7_No internet service
x7_Yes
x8_No
x8_No internet service
x8_Yes
x9_No
x9_No internet service
x9_Yes
x10_No
x10_No internet service
x10_Yes
x11_No
x11_No internet service
x11_Yes
x12_Month-to-month
x12_One year
x12_Two year
x13_No
x13_Yes
x14_Bank transfer (automatic)
x14_Credit card (automatic)
x14_Electronic check
x14_Mailed check
x15_0-6 bln

x15__13-24 bln
x15__25-48 bln
x15__7-12 bln
x15__>48 bln

num___SeniorCitizen
num___tenure
num___MonthlyCharges
num___TotalCharges
num___num_services
num___charge_ratio
cat___gender_Female
cat___gender_Male
cat___Partner_No
cat___Partner_Yes
cat___Dependents_No
cat___Dependents_Yes
cat___PhoneService_No
cat___PhoneService_Yes
cat___MultipleLines_No
cat___MultipleLines_No phone service
cat___MultipleLines_Yes
cat___InternetService_DSL
cat___InternetService_Fiber optic
cat___InternetService_No
cat___OnlineSecurity_No
cat___OnlineSecurity_No internet service
cat___OnlineSecurity_Yes
cat___OnlineBackup_No
cat___OnlineBackup_No internet service
cat___OnlineBackup_Yes
cat___DeviceProtection_No
cat___DeviceProtection_No internet service
cat___DeviceProtection_Yes
cat___TechSupport_No
cat___TechSupport_No internet service
cat___TechSupport_Yes
cat___StreamingTV_No
cat___StreamingTV_No internet service
cat___StreamingTV_Yes
cat___StreamingMovies_No

cat__StreamingMovies_No internet service
cat__StreamingMovies_Yes
cat__Contract_Month-to-month
cat__Contract_One year
cat__Contract_Two year
cat__PaperlessBilling_No
cat__PaperlessBilling_Yes
cat__PaymentMethod_Bank transfer (automatic)
cat__PaymentMethod_Credit card (automatic)
cat__PaymentMethod_Electronic check
cat__PaymentMethod_Mailed check
cat__tenure_group_0-6 bln
cat__tenure_group_13-24 bln
cat__tenure_group_25-48 bln
cat__tenure_group_7-12 bln
cat__tenure_group_>48 bln

n_estimators	n_estimators: int, default=100 The number of trees in the forest. .. versionchanged:: 0.22 The default value of “n_estimators“ changed from 10 to 100 in 0.22.	200
max_depth	max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	10

n_jobs n_jobs: int, default=None The number of jobs to run in parallel. :meth:‘fit‘, :meth:‘predict‘, :meth:‘decision_path‘ and :meth:‘apply‘ are all parallelized over the trees. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	-1
random_state random_state: int, RandomState instance or None, default=None Controls both the randomness of the bootstrapping of the samples used when building trees (if “bootstrap=True“) and the sampling of the features to consider when looking for the best split at each node (if “max_features < n_features“). See :term:‘Glossary <random_state>‘ for details.	42

criterion	criterion: {”gini”, ”entropy”, ”log_loss”}, default=”gini” The function to measure the quality of a split. Supported criteria are ”gini” for the Gini impurity and ”log_loss” and ”entropy” both for the Shannon information gain, see :ref:‘tree_mathematical_formulation‘. Note: This parameter is tree-specific.	'gini'
min_samples_split	min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider ‘min_samples_split‘ as the minimum number. - If float, then ‘min_samples_split‘ is a fraction and ‘ceil(min_samples_split * n_samples)‘ are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	2

min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least “min_samples_leaf“ training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression. - If int, then consider ‘min_samples_leaf‘ as the minimum number. - If float, then ‘min_samples_leaf‘ is a fraction and ‘ceil(min_samples_leaf * n_samples)‘ are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	1

min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.	0.0

max_features max_features: {”sqrt”, ”log2”, None}, int or float, default=”sqrt” The number of features to consider when looking for the best split: - If int, then consider ‘max_features‘ features at each split. - If float, then ‘max_features‘ is a fraction and ‘max(1, int(max_features * n_features_in_))‘ features are considered at each split. - If ”sqrt”, then ‘max_features=sqrt(n_features)‘. - If ”log2”, then ‘max_features=log2(n_features)‘. - If None, then ‘max_features=n_features‘. .. versionchanged:: 1.1 The default of ‘max_features‘ changed from ‘”auto”‘ to ‘”sqrt”‘. Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than “max_features“ features.	'sqrt'

max_leaf_nodes max_leaf_nodes: int, default=None Grow trees with “max_leaf_nodes“ in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None

min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where “N“ is the total number of samples, “N_t“ is the number of samples at the current node, “N_t_L“ is the number of samples in the left child, and “N_t_R“ is the number of samples in the right child. “N“, “N_t“, “N_t_R“ and “N_t_L“ all refer to the weighted sum, if “sample_weight“ is passed. .. versionadded:: 0.19	0.0

bootstrap	bootstrap: bool, default=True Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.	True
oob_score	oob_score: bool or callable, default=False Whether to use out-of-bag samples to estimate the generalization score. By default, :func:‘~sklearn.metrics.accuracy_score‘ is used. Provide a callable with signature ‘metric(y_true, y_pred)‘ to use a custom metric. Only available if ‘bootstrap=True‘. For an illustration of out-of-bag (OOB) error estimation, see the example :ref:‘sphx_glr_auto_examples_ensemble_plot_ensemble_oob.py‘.	False
verbose	verbose: int, default=0 Controls the verbosity when fitting and predicting.	0

warm_start warm_start: bool, default=False When set to “True“, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See :term:‘Glossary <warm_start>‘ and :ref:‘tree_ensem- ble_warm_start‘ for details.	False

class_weight	class_weight: {"balanced", "balanced_subsample"}, dict or list of dicts, default=None Weights associated with classes in the form "{class_label: weight}". If not given, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as "n_samples / (n_classes * np.bincount(y))" The "balanced_subsample" mode is the same as "balanced" except that weights are computed based on the bootstrap sample for every tree grown. For multi-output, the weights of each column of y will be multiplied. Note that these weights will	None

ccp_alpha	ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “ccp_alpha“ will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning‘ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py‘ for an example of such pruning. .. versionadded:: 0.22	0.0

max_samples max_samples: int or float, default=None If bootstrap is True, the number of samples to draw from X to train each base estimator. - If None (default), then draw ‘X.shape[0]‘ samples irrespective of ‘sample_weight‘. - If int, then draw ‘max_samples‘ samples. - If float, then draw ‘max_samples * X.shape[0]‘ unweighted samples or ‘max_samples * sample_weight.sum()‘ weighted samples. .. versionadded:: 0.22 .. versionchanged:: 1.9 Float ‘max_samples‘ is relative to ‘sample_weight.sum()‘ instead of ‘X.shape[0]‘ for weighted samples.	None

	monotonic_cst monotonic_cst: array-like of int of shape (n_features,), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonically increasing - 0: no constraint - -1: monotonically decreasing If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when ‘n_classes > 2‘), - multioutput classifications (i.e. when ‘n_outputs_ > 1‘). The constraints hold over the probability of the positive class. Read more in the :ref:‘User Guide <monotonic_cst_gbdt>‘. .. versionadded:: 1.4	None

Name	Type	Value

classes_ classes_: ndarray of shape (n_classes,) or a list of such arrays The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
estimator_ estimator_: :class:‘~sklearn.tree.Decision-TreeClassifier‘ The child estimator template used to create the collection of fitted sub-estimators. .. versionadded:: 1.2 ‘base_estimator_‘ was renamed to ‘estimator_‘.	DecisionTreeClassifier	DecisionTreeClassifier()
estimators_ estimators_: list of DecisionTreeClassifier The collection of fitted sub-estimators.	list	[Decision-TreeC...te=1608637542), Decision-TreeC...te=1273642419), Decision-TreeC...te=1935803228), Decision-TreeC...ate=787846414), ...]
estimators_samples_ estimators_samples_: list of arrays The subset of drawn samples (i.e., the in-bag samples) for each base estimator. Each subset is defined by an array of the indices selected. .. versionadded:: 1.4	list	[array([2523, ..., dtype=int32), array([4782, ..., dtype=int32), array([5462, ..., dtype=int32), array([2848, ..., dtype=int32), ...]

feature_importances_ feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:‘sklearn.inspection.per- mutation_importance‘ as an alternative.	ndarray[float64](52,)	[0.01,0.11,0.08,...,0.01,0.01,0.02]
n_classes_ n_classes_: int or list The number of classes (single output problem), or a list containing the number of classes for each output (multi-output problem).	int	2
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	52

n_outputs_ n_outputs_: int The number of outputs when “fit“ is performed.	int	1

Asumsi: tidak ada asumsi kuat—ensemble dari banyak pohon yang dilatih pada subset data dan fitur. **Kelebihan**: performa tinggi, tahan terhadap overfitting, memberikan **feature importance**, menangani interaksi secara otomatis. **Kekurangan**: kurang interpretabel dari decision tree tunggal, lebih lambat, lebih sulit di-deploy.

9.3.6 6. Gradient Boosting — Model Terkuat di Kelas Tabular

```
gb = Pipeline([
    ("prep", preprocessor),
    ("clf", GradientBoostingClassifier(
        n_estimators=200, max_depth=3, learning_rate=0.05,
        random_state=RANDOM_STATE,
    )),
])
gb.fit(X_train, y_train)
```

steps steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define ‘fit‘. All non-last steps must also define ‘transform‘. See :ref:‘Combining Estimators <combining_estimators>‘ for more details.		[('prep', ...), ('clf', ...)]

transform_input transform_input: list of str, default=None The names of the :term:‘metadata‘ parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to “fit“ (other than “X“) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:‘metadata routing <metadata_routing>‘. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using “sklearn.set_config(enable_metadata_routing=True)“. .. versionadded:: 1.6	None

	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute “named_steps“ or “steps“ to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:‘sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py‘ for an example on how to enable caching.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[int64](2,)	[0,1]
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Only defined if the underlying estimator exposes such an attribute when fit. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying first estimator in ‘steps‘ exposes such an attribute when fit. .. versionadded:: 0.24	int	22

	transformers transformers: list of tuples List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data. name : str Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using “set_params“ and searched in grid search. transformer : {'drop', 'passthrough'} or estimator Estimator must support :term:‘fit‘ and :term:‘transform‘. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively. columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where “transformer“ expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data ‘X‘ and can return any of the above. To select multiple columns by name or dtype, you can use :obj:‘make_column_selector‘.	[('num', ...), ('cat', ...)]

remainder	remainder: {'drop', 'passthrough'} or estimator, default='drop' By default, only the specified columns in ‘transformers‘ are transformed and combined in the output, and the non-specified columns are dropped. (default of “'drop'“). By specifying “remainder='passthrough'“, all remaining columns that were not specified in ‘transformers‘, but present in the data passed to ‘fit‘ will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during ‘fit‘ will be excluded from the output of ‘transform‘. By setting “remainder“ to be an estimator, the remaining non-specified columns will use the “remainder“ estimator. The estimator must support :term:‘fit‘ and :term:‘transform‘. Note that using this feature requires that the DataFrame columns input at :term:‘fit‘ and :term:‘transform‘ have identical order.	'drop'

	sparse_threshold sparse_threshold: float, default=0.3 If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use “sparse_threshold=0“ to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.	0.3
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	None
	transformer_weights transformer_weights: dict, default=None Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.	None

	verbose verbose: bool, default=False If True, the time elapsed while fitting each transformer will be printed as it is completed.	False

verbose_feature_names_out verbose_feature_names_out: bool, str or Callable[[str, str], str], default=True - If True, :meth:‘ColumnTransformer.get_feature_names_out‘ will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting ‘verbose_feature_names_out=”{transformer_name}___{feature_name}”‘. - If False, :meth:‘ColumnTransformer.get_feature_names_out‘ will not prefix any feature names and will error if feature names are not unique. - If “Callable[[str, str], str]“, :meth:‘ColumnTransformer.get_feature_names_out‘ will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name. - If “str“, it must be a string ready for formatting. The given string will be formatted using two field names: “transformer_name“ and “feature_name“. e.g. “”{feature_name}___{transformer_name}”“. See :meth:‘str.format‘ method from the standard library for more info. .. versionadded:: 1.0	True

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](22,)	['gender','SeniorCitizen','Partner',...,'tenure_gro 'charge_ratio']
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. Only defined if the underlying transformers expose such an attribute when fit. .. versionadded:: 0.24	int	22
named_transformers_ named_transformers_: :class:‘~sklearn.utils.Bunch‘ Read-only attribute to access any transformer by given name. Keys are transformer names and values are the fitted transformer objects.	Bunch	{'num': Pipel...='ignore'))])}

output_indices_ output_indices_: dict A dictionary from each transformer name to a slice, where the slice corresponds to indices in the transformed output. This is useful to inspect which transformer is responsible for which transformed feature(s). .. versionadded:: 1.0	dict	{'cat': slice(6, 52, None), 'num': slice(0, 6, None), 're...er': slice(0, 0, None)}
sparse_output_ sparse_output_: bool Boolean flag indicating whether the output of “transform“ is a sparse matrix or a dense numpy array, which depends on the output of the individual transformers and the ‘sparse_threshold‘ keyword.	bool	False

transformers_ transformers_: list The collection of fitted transformers as tuples of (name, fitted_transformer, column). ‘fitted_transformer‘ can be an estimator, or ‘'drop'‘; ‘'passthrough'‘ is replaced with an equivalent :class:‘~sklearn.preprocessing.FunctionTransformer‘. In case there were no columns selected, this will be the unfitted transformer. If there are remaining columns, the final element is a tuple of the form: ('remainder', transformer, remaining_columns) corresponding to the “remainder“ parameter. If there are remaining columns, then “len(transformers_)==len(transformers)+1“, otherwise “len(transformers_)==len(transformers)“. .. versionadded:: 1.7 The format of the remaining columns now attempts to match that of the other transformers: if all columns were provided as column names (‘str‘), the remaining columns are stored as column names; if all columns were provided as mask arrays (‘bool‘), so are the remaining columns; in all other cases the remaining columns are stored as indices (‘int‘).	list	[('num', Pipeline(step...ardScaler())]), ['Se...en', 'tenure', 'Mo...es', 'To...es', ...]), ('cat', Pipeline(step...n='ignore'))]), ['gender', 'Partner', 'De...ts', 'Ph...ce', ...])]

strategy	strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'median'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when ‘fit‘ is called are returned in results when ‘transform‘ is called. The imputed value is always ‘0‘ except when ‘strategy=”constant”‘ in which case ‘fill_value‘ will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](6,)	['SeniorCitizen','tenure','MonthlyCharges','Tota 'charge_ratio']
indicator_ indicator_: :class:‘~sklearn.impute.MissingIndicator‘ Indicator used to add binary indicators for missing values. ‘None‘ if ‘add_indicator=False‘.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[float64](6,)	[0. , 29. , 70.5 ,1394.92, 3. , 1.]

SeniorCitizen
tenure
MonthlyCharges
TotalCharges
num_services
charge_ratio

	copy copy: bool, default=True If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.	True

	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
	with_std with_std: bool, default=True If True, scale the data to unit variance (or equivalently, unit standard deviation).	True

Name	Type	Value
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.16, 32.49, 64.93,2299.33, 3.38, 1.]
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	6

n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the “n_samples_seen“ will be an integer, otherwise it will be an array of dtype int. If ‘sample_weights‘ are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across “partial_fit“ calls.	float64	5634
scale_ scale_: ndarray of shape (n_features,) or None Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using ‘np.sqrt(var_)‘. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. ‘scale_‘ is equal to ‘None‘ when ‘with_std=False‘. .. versionadded:: 0.17 *scale_*	ndarray[float64](6,)	[0.37, 24.57, 30.14,2279. , 2.07, 0.06]

var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute ‘scale_‘. Equal to “None“ when “with_mean=False“ and “with_std=False“.	ndarray[float64](6,)	[0.14, 603.52, 908.14,5193850.1 , 4.29, 0.]

x0
x1
x2
x3
x4
x5

	strategy strategy: str or Callable, default='mean' The imputation strategy. - If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data. - If ”median”, then replace missing values using the median along each column. Can only be used with numeric data. - If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned. - If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data. - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column. .. versionadded:: 0.20 strategy=”constant” for fixed value imputation. .. versionadded:: 1.5 strategy=callable for custom value imputation.	'most_frequent'

	missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan The placeholder for the missing values. All occurrences of ‘missing_values‘ will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, ‘missing_values‘ can be set to either ‘np.nan‘ or ‘pd.NA‘.	nan
	fill_value fill_value: str or numerical value, default=None When strategy == ”constant”, ‘fill_value‘ is used to replace all occurrences of missing_values. For string or object data types, ‘fill_value‘ must be a string. If ‘None‘, ‘fill_value‘ will be 0 when imputing numerical data and ”missing_value” for strings or object data types.	None

	copy copy: bool, default=True If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘: - If ‘X‘ is not an array of floating values; - If ‘X‘ is encoded as a CSR matrix; - If ‘add_indicator=True‘.	True
	add_indicator add_indicator: bool, default=False If True, a :class:‘MissingIndicator‘ transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.	False

	keep_empty_features keep_empty_features: bool, default=False If True, features that consist exclusively of missing values when ‘fit‘ is called are returned in results when ‘transform‘ is called. The imputed value is always ‘0‘ except when ‘strategy=”constant”‘ in which case ‘fill_value‘ will be used instead. .. versionadded:: 1.2	False

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (‘n_features_in_‘,) Names of features seen during :term:‘fit‘. Defined only when ‘X‘ has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](16,)	['gender','Partner','Dependents',...,'PaperlessBil 'tenure_group']
indicator_ indicator_: :class:‘~sklearn.impute.MissingIndicator‘ Indicator used to add binary indicators for missing values. ‘None‘ if ‘add_indicator=False‘.	NoneType	None

n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	16
statistics_ statistics_: array of shape (n_features,) The imputation fill value for each feature. Computing statistics can result in ‘np.nan‘ values. During :meth:‘transform‘, features corresponding to ‘np.nan‘ statistics will be discarded.	ndarray[object](16,)	['Male','No','No',...,'Yes','Electronic check','>48 bln']

gender
Partner
Dependents
PhoneService
MultipleLines
InternetService
OnlineSecurity
OnlineBackup
DeviceProtection
TechSupport
StreamingTV
StreamingMovies
Contract
PaperlessBilling
PaymentMethod
tenure_group

	handle_unknown handle_unknown: {'error', 'ignore', 'infrequent_if_exist', 'warn'}, default='error' Specifies the way unknown categories are handled during :meth:‘transform‘. - 'error' : Raise an error if an unknown category is present during transform. - 'ignore' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will be all zeros. In the inverse transform, an unknown category will be denoted as None. - 'infrequent_if_exist' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will map to the infrequent category if it exists. The infrequent category will be mapped to the last position in the encoding. During inverse transform, an unknown category will be mapped to the category denoted ‘'infrequent'‘ if it exists. If the ‘'infrequent'‘ category does not exist, then :meth:‘transform‘ and :meth:‘inverse_transform‘ will handle an unknown category as with ‘handle_unknown='ignore'‘. Infrequent categories exist based on ‘min_frequency‘ and ‘max_categories‘. Read more in the	'ignore'

categories	categories: 'auto' or a list of array-like, default='auto' Categories (unique values) per feature: - 'auto' : Determine categories automatically from the training data. - list : “categories[i]“ holds the categories expected in the ith column. The passed categories should not mix strings and numeric values within a single feature, and should be sorted in case of numeric values. The used categories can be found in the “categories_“ attribute. .. versionadded:: 0.20	'auto'

drop	drop: {'first', 'if_binary'} or an array-like of shape (n_features,), default=None Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model. However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models. - None : retain all features (the default). - 'first' : drop the first category in each feature. If only one category is present, the feature will be dropped entirely. - 'if_binary' : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact. - array : “drop[i]“ is the category in feature “X[:, i]“ that should be dropped. When ‘max_categories‘ or ‘min_frequency‘ is configured to group infrequent categories, the dropping behavior is handled after the	None

sparse_output sparse_output: bool, default=True When “True“, it returns a SciPy sparse matrix/array in ”Compressed Sparse Row” (CSR) format. .. versionadded:: 1.2 ‘sparse‘ was renamed to ‘sparse_output‘	True
dtype dtype: number type, default=np.float64 Desired dtype of output.	<class 'numpy.float64'>
min_frequency min_frequency: int or float, default=None Specifies the minimum frequency below which a category will be considered infrequent. - If ‘int‘, categories with a smaller cardinality will be considered infrequent. - If ‘float‘, categories with a smaller cardinality than ‘min_frequency * n_samples‘ will be considered infrequent. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infrequent_categories>‘.	None

max_categories max_categories: int, default=None Specifies an upper limit to the number of output features for each input feature when considering infrequent categories. If there are infrequent categories, ‘max_categories‘ includes the category representing the infrequent categories along with the frequent categories. If ‘None‘, there is no limit to the number of output features. .. versionadded:: 1.1 Read more in the :ref:‘User Guide <encoder_infre- quent_categories>‘.	None

	feature_name_combiner feature_name_combiner: ”concat” or callable, default=”concat” Callable with signature ‘def callable(input_feature, category)‘ that returns a string. This is used to create feature names to be returned by :meth:‘get_feature_names_out‘. ‘”concat”‘ concatenates encoded feature name and category with ‘feature + ”_” + str(category)‘.E.g. feature X with values 1, 6, 7 create feature names ‘X_1, X_6, X_7‘. .. versionadded:: 1.3	'concat'

Name	Type	Value
categories_ categories_: list of arrays The categories of each feature determined during fitting (in order of the features in X and corresponding with the output of “transform“). This includes the category specified in “drop“ (if any).	list	[array(['Femal... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), array(['No', ... dtype=object), ...]

drop_idx_ drop_idx_: array of shape (n_features,) - “drop_idx_[i]“ is the index in “categories_[i]“ of the category to be dropped for each feature. - “drop_idx_[i] = None“ if no category is to be dropped from the feature with index “i“, e.g. when ‘drop='if_binary'‘ and the feature isn't binary. - “drop_idx_ = None“ if all the transformed features will be retained. If infrequent categories are enabled by setting ‘min_frequency‘ or ‘max_categories‘ to a non-default value and ‘drop_idx[i]‘ corresponds to an infrequent category, then the entire infrequent category is dropped. .. versionchanged:: 0.23 Added the possibility to contain ‘None‘ values.	NoneType	None
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 1.0	int	16

x0_Female
x0_Male
x1_No
x1_Yes
x2_No
x2_Yes
x3_No
x3_Yes
x4_No
x4_No phone service
x4_Yes
x5_DSL
x5_Fiber optic
x5_No
x6_No
x6_No internet service
x6_Yes
x7_No
x7_No internet service
x7_Yes
x8_No
x8_No internet service
x8_Yes
x9_No
x9_No internet service
x9_Yes
x10_No
x10_No internet service
x10_Yes
x11_No
x11_No internet service
x11_Yes
x12_Month-to-month
x12_One year
x12_Two year
x13_No
x13_Yes
x14_Bank transfer (automatic)
x14_Credit card (automatic)
x14_Electronic check
x14_Mailed check
x15_0-6 bln

x15__13-24 bln
x15__25-48 bln
x15__7-12 bln
x15__>48 bln

num___SeniorCitizen
num___tenure
num___MonthlyCharges
num___TotalCharges
num___num_services
num___charge_ratio
cat___gender_Female
cat___gender_Male
cat___Partner_No
cat___Partner_Yes
cat___Dependents_No
cat___Dependents_Yes
cat___PhoneService_No
cat___PhoneService_Yes
cat___MultipleLines_No
cat___MultipleLines_No phone service
cat___MultipleLines_Yes
cat___InternetService_DSL
cat___InternetService_Fiber optic
cat___InternetService_No
cat___OnlineSecurity_No
cat___OnlineSecurity_No internet service
cat___OnlineSecurity_Yes
cat___OnlineBackup_No
cat___OnlineBackup_No internet service
cat___OnlineBackup_Yes
cat___DeviceProtection_No
cat___DeviceProtection_No internet service
cat___DeviceProtection_Yes
cat___TechSupport_No
cat___TechSupport_No internet service
cat___TechSupport_Yes
cat___StreamingTV_No
cat___StreamingTV_No internet service
cat___StreamingTV_Yes
cat___StreamingMovies_No

cat__StreamingMovies_No internet service
cat__StreamingMovies_Yes
cat__Contract_Month-to-month
cat__Contract_One year
cat__Contract_Two year
cat__PaperlessBilling_No
cat__PaperlessBilling_Yes
cat__PaymentMethod_Bank transfer (automatic)
cat__PaymentMethod_Credit card (automatic)
cat__PaymentMethod_Electronic check
cat__PaymentMethod_Mailed check
cat__tenure_group_0-6 bln
cat__tenure_group_13-24 bln
cat__tenure_group_25-48 bln
cat__tenure_group_7-12 bln
cat__tenure_group_>48 bln

learning_rate learning_rate: float, default=0.1 Learning rate shrinks the contribution of each tree by ‘learning_rate‘. There is a trade-off between learning_rate and n_estimators. Values must be in the range ‘[0.0, inf)‘. For an example of the effects of this parameter and its interaction with “subsample“, see :ref:‘sphx_glr_auto_examples_ensemble_plot_gradient_boosting_regularization.py‘.	0.05

n_estimators n_estimators: int, default=100 The number of boosting stages to perform. Gradient boosting is fairly robust to over-fitting so a large number usually results in better performance. Values must be in the range ‘[1, inf)‘.	200
random_state random_state: int, RandomState instance or None, default=None Controls the random seed given to each Tree estimator at each boosting iteration. In addition, it controls the random permutation of the features at each split (see Notes for more details). It also controls the random splitting of the training data to obtain a validation set if ‘n_iter_no_change‘ is not None. Pass an int for reproducible output across multiple function calls. See :term:‘Glossary <random_state>‘.	42

	loss loss: {'log_loss', 'exponential'}, default='log_loss' The loss function to be optimized. 'log_loss' refers to binomial and multinomial deviance, the same as used in logistic regression. It is a good choice for classification with probabilistic outputs. For loss 'exponential', gradient boosting recovers the AdaBoost algorithm.	'log_loss'
	subsample subsample: float, default=1.0 The fraction of samples to be used for fitting the individual base learners. If smaller than 1.0 this results in Stochastic Gradient Boosting. ‘subsample‘ interacts with the parameter ‘n_estimators‘. Choosing ‘subsample < 1.0‘ leads to a reduction of variance and an increase in bias. Values must be in the range ‘(0.0, 1.0]‘.	1.0

	criterion criterion: {'friedman_mse', 'squared_error'}, default='friedman_mse' This parameter has no effect. .. versionadded:: 0.18 .. deprecated:: 1.9 ‘criterion‘ is deprecated and will be removed in 1.11.	'deprecated'
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, values must be in the range ‘[2, inf)‘. - If float, values must be in the range ‘(0.0, 1.0]‘ and ‘min_samples_split‘ will be ‘ceil(min_samples_split * n_samples)‘. .. versionchanged:: 0.18 Added float values for fractions.	2

min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least “min_samples_leaf“ training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression. - If int, values must be in the range ‘[1, inf)‘. - If float, values must be in the range ‘(0.0, 1.0)‘ and ‘min_samples_leaf‘ will be ‘ceil(min_samples_leaf * n_samples)‘. .. versionchanged:: 0.18 Added float values for fractions.	1

	min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided. Values must be in the range ‘[0.0, 0.5]‘.	0.0
	max_depth max_depth: int or None, default=3 Maximum depth of the individual regression estimators. The maximum depth limits the number of nodes in the tree. Tune this parameter for best performance; the best value depends on the interaction of the input variables. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples. If int, values must be in the range ‘[1, inf)‘.	3

min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. Values must be in the range ‘[0.0, inf)‘. The weighted impurity decrease equation is the following:: N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where “N“ is the total number of samples, “N_t“ is the number of samples at the current node, “N_t_L“ is the number of samples in the left child, and “N_t_R“ is the number of samples in the right child. “N“, “N_t“, “N_t_R“ and “N_t_L“ all refer to the weighted sum, if “sample_weight“ is passed. .. versionadded:: 0.19	0.0

init	init: estimator or 'zero', default=None An estimator object that is used to compute the initial predictions. “init“ has to provide :term:‘fit‘ and :term:‘predict_proba‘. If 'zero', the initial raw predictions are set to zero. By default, a “DummyEstimator“ predicting the classes priors is used.	None

max_features	max_features: {'sqrt', 'log2'}, int or float, default=None The number of features to consider when looking for the best split: - If int, values must be in the range ‘[1, inf)‘. - If float, values must be in the range ‘(0.0, 1.0]‘ and the features considered at each split will be ‘max(1, int(max_features * n_features_in_))‘. - If 'sqrt', then ‘max_features=sqrt(n_features)‘. - If 'log2', then ‘max_features=log2(n_features)‘. - If None, then ‘max_features=n_features‘. Choosing ‘max_features < n_features‘ leads to a reduction of variance and an increase in bias. Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than “max_features“ features.	None

verbose verbose: int, default=0 Enable verbose output. If 1 then it prints progress and performance once in a while (the more trees the lower the frequency). If greater than 1 then it prints progress and performance for every tree. Values must be in the range ‘[0, inf)‘.	0
max_leaf_nodes max_leaf_nodes: int, default=None Grow trees with “max_leaf_nodes“ in best-first fashion. Best nodes are defined as relative reduction in impurity. Values must be in the range ‘[2, inf)‘. If ‘None‘, then unlimited number of leaf nodes.	None
warm_start warm_start: bool, default=False When set to “True“, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just erase the previous solution. See :term:‘the Glossary <warm_start>‘.	False

validation_fraction validation_fraction: float, default=0.1 The proportion of training data to set aside as validation set for early stopping. Values must be in the range ‘(0.0, 1.0)‘. Only used if “n_iter_no_change“ is set to an integer. .. versionadded:: 0.20	0.1

n_iter_no_change n_iter_no_change: int, default=None “n_iter_no_change“ is used to decide if early stopping will be used to terminate training when validation score is not improving. By default it is set to None to disable early stopping. If set to a number, it will set aside “validation_fraction“ size of the training data as validation and terminate training when validation score is not improving in all of the previous “n_iter_no_change“ numbers of iterations. The split is stratified. Values must be in the range ‘[1, inf)‘. See :ref:‘sphx_glr_auto_examples_ensemble_plot_gradient_boosting_early_stopping.py‘. .. versionadded:: 0.20	None

	tol tol: float, default=1e-4 Tolerance for the early stopping. When the loss is not improving by at least tol for “n_iter_no_change“ iterations (if set to a number), the training stops. Values must be in the range ‘[0.0, inf)‘. .. versionadded:: 0.20	0.0001
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “ccp_alpha“ will be chosen. By default, no pruning is performed. Values must be in the range ‘[0.0, inf)‘. See :ref:‘minimal_cost_complexity_pruning‘ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py‘ for an example of such pruning. .. versionadded:: 0.22	0.0

Name	Type	Value

classes_ classes_: ndarray of shape (n_classes,) The classes labels.	ndarray[int64](2,)	[0,1]
estimators_ estimators_: ndarray of DecisionTreeRegressor of shape (n_estimators, “n_trees_per_iteration_“) The collection of fitted sub-estimators. “n_trees_per_iteration_“ is 1 for binary classification, otherwise “n_classes“.	ndarray[object](200, 1)	[[DecisionTreeRegressor(max_depth=3, random_state=RandomState(MT19937) at 0x12A6FCC40)], [DecisionTreeRegressor(max_depth=3, random_state=RandomState(MT19937) at 0x12A6FCC40)], [DecisionTreeRegressor(max_depth=3, random_state=RandomState(MT19937) at 0x12A6FCC40)], ..., [DecisionTreeRegressor(max_depth=3, random_state=RandomState(MT19937) at 0x12A6FCC40)], [DecisionTreeRegressor(max_depth=3, random_state=RandomState(MT19937) at 0x12A6FCC40)], [DecisionTreeRegressor(max_depth=3, random_state=RandomState(MT19937) at 0x12A6FCC40)]]

feature_importances_ feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the MSE brought by that feature. It is also known as the Gini importance. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:‘sklearn.inspection.per- mutation_importance‘ as an alternative.	ndarray[float64](52,)	[0.01,0.13,0.07,...,0. ,0. ,0.]
init_ init_: estimator The estimator that provides the initial predictions. Set via the “init“ argument.	DummyClassifier	DummyClassifier()
max_features_ max_features_: int The inferred value of max_features.	int	52
n_classes_ n_classes_: int The number of classes.	int	2

n_estimators_ n_estimators_: int The number of estimators as selected by early stopping (if “n_iter_no_change“ is specified). Otherwise it is set to “n_estimators“. .. versionadded:: 0.20	int	200
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘.	int	52
.. versionadded:: 0.24 n_trees_per_iteration_ n_trees_per_iteration_: int The number of trees that are built at each iteration. For binary classifiers, this is always 1. .. versionadded:: 1.4.0	int	1
train_score_ train_score_: ndarray of shape (n_estimators,) The i-th score “train_score_[i]“ is the loss of the model at iteration “i“ on the in-bag sample. If “subsample == 1“ this is the loss on the training data.	ndarray[float64](200,)	[1.13,1.11,1.09,...,0.74,0.74,0.74]

Asumsi: mirip random forest, tetapi pohon dibangun **secara berurutan**—setiap pohon memperbaiki kesalahan pohon sebelumnya. **Kelebihan**: sering menjadi model **terbaik** untuk

data tabular, sangat fleksibel. **Kekurangan**: lebih mudah overfitting jika hyperparameter tidak di-tuning, lebih lambat dilatih, kurang interpretabel.

> Tip
>
> **Random Forest vs Gradient Boosting.** Random forest membangun pohon **secara paralel** dan merata-ratakan hasilnya—ini mengurangi variance. Gradient boosting membangun pohon **secara berurutan**, masing-masing memperbaiki residual pohon sebelumnya—ini mengurangi bias. Dalam praktik, gradient boosting sering menang tipis pada data tabular, tetapi membutuhkan tuning yang lebih hati-hati.

9.4 Evaluasi: Metrik yang Berbeda Menjawab Pertanyaan yang Berbeda

Sekarang kita memiliki enam model. Bagaimana membandingkannya? Jawabannya bergantung pada **apa yang kita pedulikan**—dan ini adalah pelajaran paling penting di bab ini.

9.4.1 Confusion Matrix: Fondasi Semua Metrik

```
models = {
    "Logistic Regression": logreg,
    "KNN (k=15)": knn,
    "Naive Bayes": nb,
    "Decision Tree": tree,
    "Random Forest": rf,
    "Gradient Boosting": gb,
}

fig, axes = plt.subplots(2, 3, figsize=(11, 6))
for ax, (name, model) in zip(axes.flat, models.items()):
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
                xticklabels=["No", "Yes"], yticklabels=["No", "Yes"], ax=ax)
    ax.set_title(name, fontsize=10)
    ax.set_xlabel("Prediksi")
    ax.set_ylabel("Aktual")
plt.tight_layout()
plt.show()
```

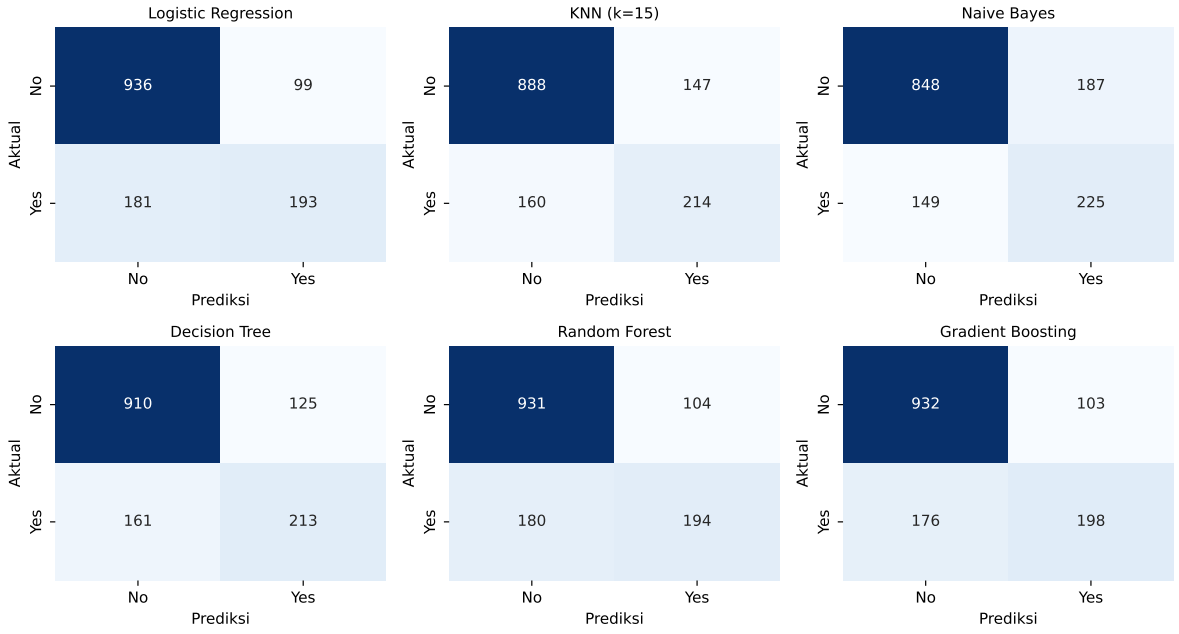


Figure 9.1: Confusion matrix untuk enam model pada test set. Setiap sel menunjukkan jumlah pelanggan.

Cara membaca confusion matrix:

	Prediksi: No	Prediksi: Yes
Aktual: No	True Negative (TN)	False Positive (FP)
Aktual: Yes	False Negative (FN)	True Positive (TP)

- **TN**: pelanggan yang tidak churn, diprediksi tidak churn. Benar.
- **FP**: pelanggan yang tidak churn, diprediksi churn. **Kita menghubungi mereka tanpa perlu.**
- **FN**: pelanggan yang churn, diprediksi tidak churn. **Kita melewatkan mereka.**
- **TP**: pelanggan yang churn, diprediksi churn. **Kita berhasil menangkap mereka.**

> **!** Important
>
> **Dalam konteks retensi churn, FN hampir selalu lebih mahal dari FP.** Melewatkan pelanggan yang akan churn berarti kehilangan revenue dan pelanggan. Menghubungi pelanggan yang tidak akan churn hanya menghabiskan waktu tim retensi—mengganggu, tetapi tidak fatal. **Ini akan memengaruhi metrik yang kita pilih.**

9.4.2 Precision, Recall, F1

Tiga metrik yang diturunkan dari confusion matrix:

Metrik	Formula	Arti	Kapan dipakai
Precision	TP / (TP + FP)	Dari yang diprediksi churn, berapa yang benar-benar churn?	Ketika FP mahal (mis. kampanye mahal per pelanggan)
Recall	TP / (TP + FN)	Dari yang benar-benar churn, berapa yang berhasil ditangkap?	Ketika FN mahal (mis. retensi murah per pelanggan)
F1	2 × (P × R) / (P + R)	Rata-rata harmonik precision dan recall	Ketika kita peduli keduanya

```
hasil = []
for name, model in models.items():
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    hasil.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1": f1_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_prob),
        "PR-AUC": average_precision_score(y_test, y_prob),
        "Brier": brier_score_loss(y_test, y_prob),
    })

hasil_df = pd.DataFrame(hasil).set_index("Model").round(3)
hasil_df.style.highlight_max(axis=0, color="lightgreen")
```

Table 9.1

Model	Accuracy	Precision	Recall	F1	AUC	PR-AUC	Brier
Logistic Regression	0.801000	0.661000	0.516000	0.580000	0.845000	0.650000	0.136000
KNN (k=15)	0.782000	0.593000	0.572000	0.582000	0.822000	0.584000	0.148000
Naive Bayes	0.762000	0.546000	0.602000	0.573000	0.778000	0.564000	0.165000

Model	Accuracy	Precision	Recall	F1	AUC	PR-AUC	Brier
Decision Tree	0.797000	0.630000	0.570000	0.598000	0.829000	0.619000	0.140000
Random Forest	0.798000	0.651000	0.519000	0.577000	0.841000	0.653000	0.137000
Gradient Boosting	0.802000	0.658000	0.529000	0.587000	0.843000	0.659000	0.136000

Cara membaca tabel ini:

- **Accuracy** paling tinggi untuk model yang cenderung memprediksi "No" (karena kelas mayoritas). Ini sebabnya **accuracy menyesatkan** untuk kelas tidak seimbang.
- **Recall** mengukur seberapa banyak churner yang berhasil kita tangkap. Model dengan recall tinggi **cocok untuk sistem retensi** karena kita ingin menangkap sebanyak mungkin pelanggan berisiko.
- **Precision** mengukur seberapa akurat prediksi "churn" kita. Jika kita membatasi anggaran retensi, precision penting—kita tidak ingin menghubungi terlalu banyak pelanggan yang tidak akan churn.
- **AUC** mengukur **ranking**: seberapa baik model mengurutkan pelanggan berisiko tinggi di atas yang berisiko rendah. Ini **tidak bergantung pada threshold**.
- **Brier score** mengukur **kalibrasi**: seberapa dekat probabilitas yang diprediksi dengan frekuensi aktual. Semakin kecil, semakin baik.

9.4.3 ROC Curve: Trade-off Recall dan False Positive Rate

```
fig, ax = plt.subplots(figsize=(7, 6))
colors = ["#4C72B0", "#DD8452", "#55A868", "#C44E52", "#8172B3", "#937860"]

for (name, model), color in zip(models.items(), colors):
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    ax.plot(fpr, tpr, label=f"{name} (AUC = {auc:.3f})", color=color, linewidth=2)

ax.plot([0, 1], [0, 1], "k--", label="Acak (AUC = 0.500)", alpha=0.5)
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate (Recall)")
ax.set_title("ROC Curve - Perbandingan Enam Model")
ax.legend(loc="lower right", fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

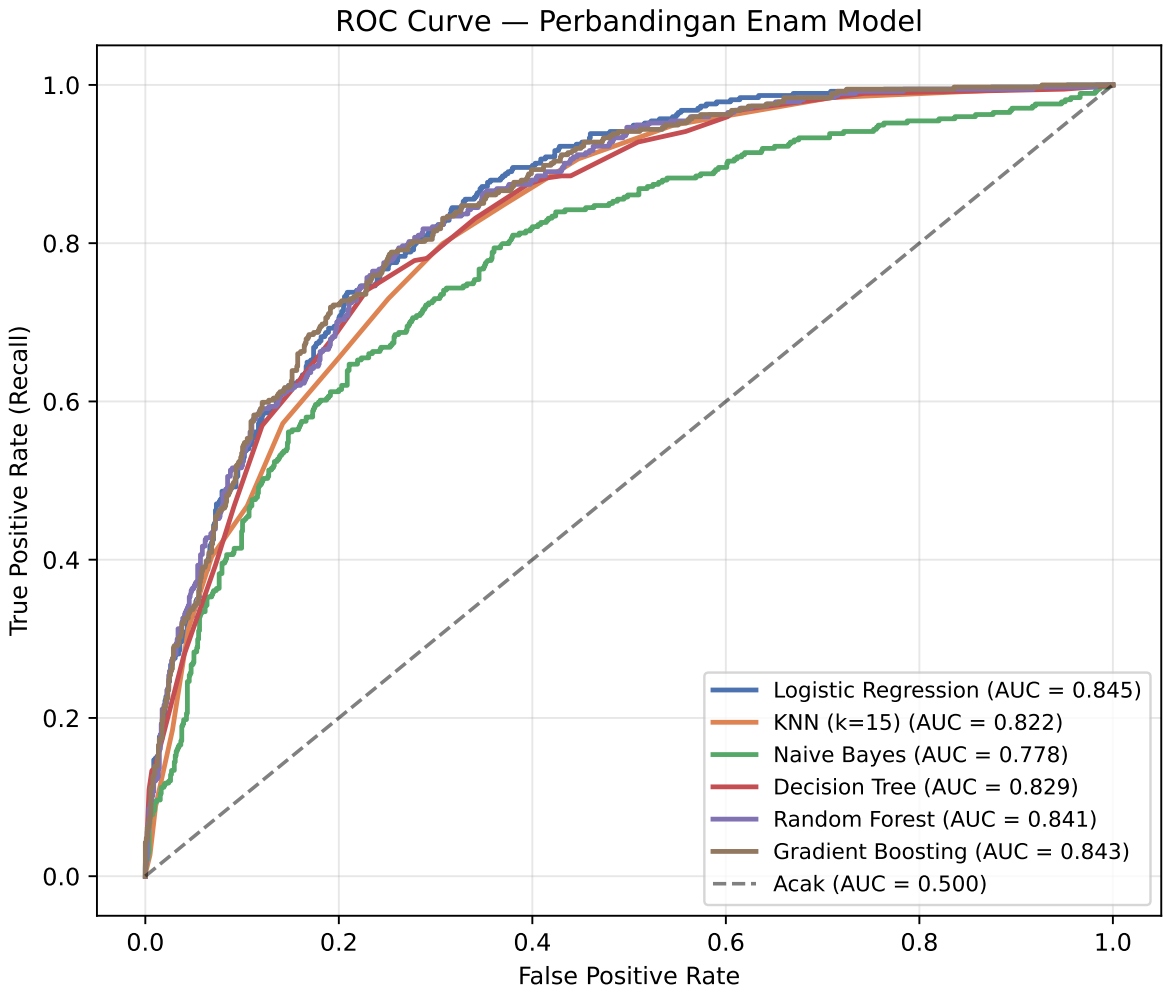


Figure 9.2: ROC curve untuk enam model. Kurva lebih dekat ke sudut kiri atas = performa lebih baik.

Cara membaca ROC curve:

- **Sumbu X**: False Positive Rate = FP / (FP + TN) — proporsi pelanggan tidak churn yang salah kita prediksi churn.
- **Sumbu Y**: True Positive Rate = TP / (TP + FN) — proporsi churner yang berhasil kita tangkap (recall).
- Kurva **semakin dekat ke kiri atas** = semakin baik—kita menangkap banyak churner dengan sedikit false positive.
- **AUC** (Area Under Curve) merangkum seluruh kurva dalam satu angka. AUC = 0,5 berarti acak; AUC = 1,0 berarti sempurna.

9.4.4 Precision-Recall Curve: Lebih Informatif untuk Kelas Tidak Seimbang

ROC curve memiliki kelemahan pada kelas tidak seimbang: FPR bisa tetap rendah bahkan ketika model buruk, karena TN sangat banyak. **Precision-Recall curve** lebih sensitif terhadap performa pada kelas minoritas.

```
fig, ax = plt.subplots(figsize=(7, 6))
baseline = y_test.mean()

for (name, model), color in zip(models.items(), colors):
    y_prob = model.predict_proba(X_test)[:, 1]
    precision, recall, _ = precision_recall_curve(y_test, y_prob)
    ap = average_precision_score(y_test, y_prob)
    ax.plot(recall, precision, label=f"{name} (AP = {ap:.3f})", color=color, linewidth=2)

ax.axhline(baseline, color="black", linestyle="--", alpha=0.5,
           label=f"Baseline (proporsi churn = {baseline:.2f})")
ax.set_xlabel("Recall")
ax.set_ylabel("Precision")
ax.set_title("Precision-Recall Curve - Perbandingan Enam Model")
ax.legend(loc="lower left", fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

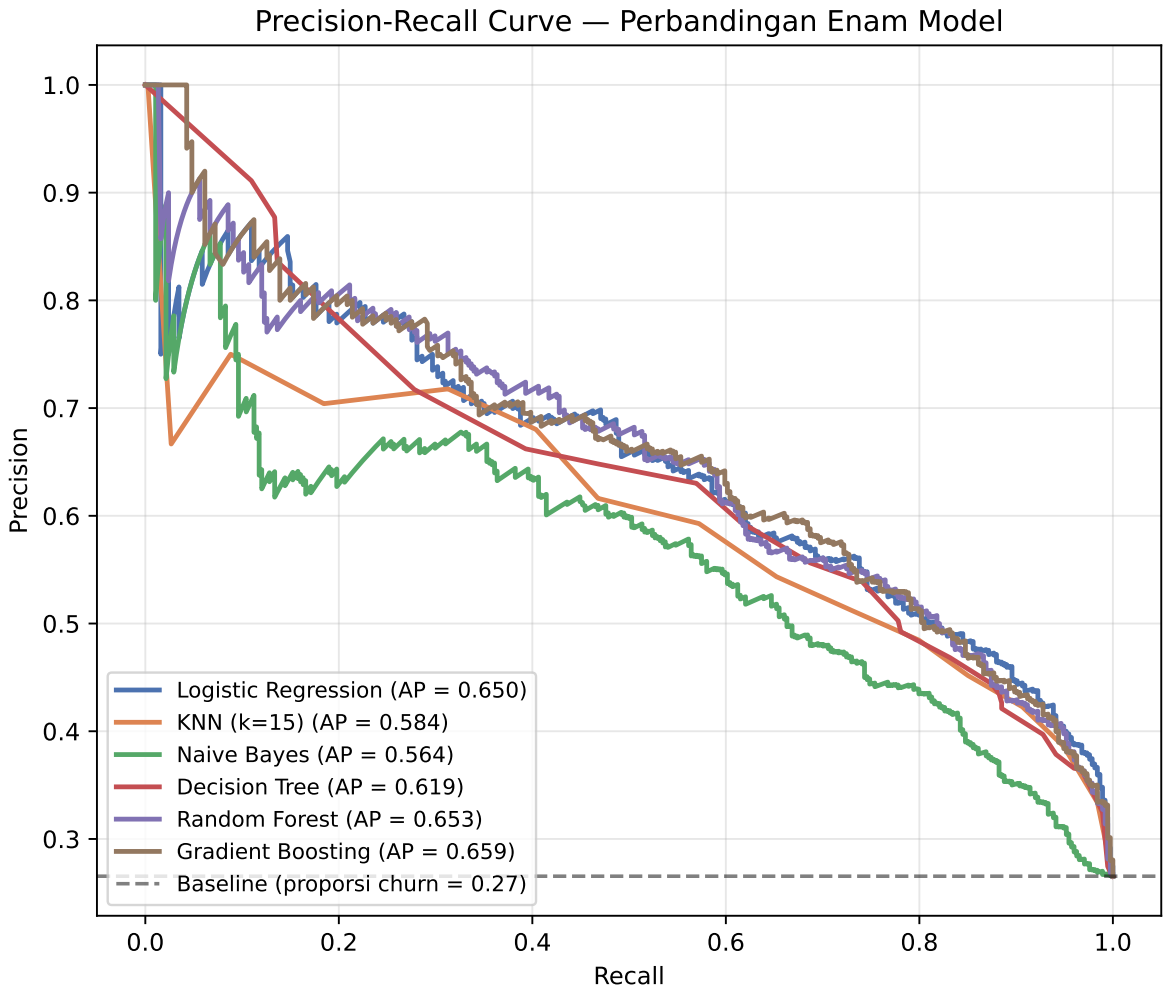


Figure 9.3: Precision-Recall curve. Semakin dekat ke kanan atas = semakin baik. Baseline (garis putus-putus) adalah proporsi kelas positif (~26,5%).

Average Precision (AP) adalah area di bawah kurva PR—semakin tinggi, semakin baik. AP lebih informatif daripada AUC ketika kelas tidak seimbang, karena ia memberi bobot lebih pada performa di kelas minoritas (churn).

> **Tip**
>
> **Kapan memakai ROC vs PR?** - **ROC-AUC** cocok ketika kelas relatif seimbang, atau ketika kita peduli **kedua kelas** secara setara. - **PR-AUC** cocok ketika kelas **sangat tidak seimbang** (mis. 1:100), atau ketika kita terutama peduli pada **kelas positif** (churner, fraud, penyakit).

Untuk dataset Telco (26,5% churn), keduanya memberi gambaran yang mirip—tetapi PR-AUC lebih jujur tentang tantangan menangkap churner.

9.5 Memilih Metrik Berdasarkan Konsekuensi Bisnis

Sekarang kita sampai pada inti bab ini: **metrik mana yang harus kita optimalkan?**

Jawabannya bergantung pada **cost matrix**—seberapa mahal setiap jenis kesalahan.

9.5.1 Skenario 1: Program Retensi Murah

Jika intervensi retensi **murah** (mis. email otomatis), kita bisa **menghubungi banyak pelanggan**—bahkan yang tidak akan churn. Dalam skenario ini:

- **FP murah**: menghubungi pelanggan yang tidak akan churn hanya menghabiskan biaya email.
- **FN mahal**: melewatkan churner berarti kehilangan pelanggan.
- **Metrik yang dioptimalkan: RECALL**.

9.5.2 Skenario 2: Program Retensi Mahal

Jika intervensi retensi **mahal** (mis. kunjungan sales, diskon besar), kita harus **sangat selektif**. Dalam skenario ini:

- **FP mahal**: menghubungi pelanggan yang tidak akan churn membuang anggaran.
- **FN mahal**: tetap kehilangan pelanggan, tetapi trade-off-nya lebih seimbang.
- **Metrik yang dioptimalkan: PRECISION atau F1**.

9.5.3 Skenario 3: Prioritisasi dengan Anggaran Terbatas

Dalam praktiknya, kita sering berada di antara keduanya—anggaran terbatas, tetapi kita ingin menangkap sebanyak mungkin churner. **Metrik yang dioptimalkan: PR-AUC** (yang memperhitungkan trade-off precision-recall di seluruh threshold).

> **!** Important
>
> **Pelajaran paling penting di bab ini.** Jangan memilih model berdasarkan **accuracy**. Accuracy menyesatkan untuk kelas tidak seimbang. Pilih model berdasarkan **metrik yang mencerminkan konsekuensi bisnis**. Model dengan accuracy 80% tetapi recall 30%

mungkin **lebih buruk** dari model dengan accuracy 75% tetapi recall 70%—tergantung pada biaya FP vs FN.

9.6 Class Imbalance: Ketika Kelas Minoritas Terlupakan

Dataset Telco memiliki 26,5% churn—tidak terlalu ekstrem, tetapi cukup untuk membuat model "malas" memprediksi mayoritas. Ada tiga strategi utama:

9.6.1 1. `class_weight='balanced'`

Model akan memberi **bobot lebih besar** pada kelas minoritas selama pelatihan. Efeknya: model lebih agresif memprediksi churn.

```
logreg_balanced = Pipeline([
    ("prep", preprocessor),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced",
                               random_state=RANDOM_STATE)),
])
logreg_balanced.fit(X_train, y_train)

y_pred_bal = logreg_balanced.predict(X_test)
y_prob_bal = logreg_balanced.predict_proba(X_test)[:, 1]

print("--- Logistic Regression (default) ---")
print(f"Precision: {precision_score(y_test, logreg.predict(X_test)):.3f}")
print(f"Recall   : {recall_score(y_test, logreg.predict(X_test)):.3f}")
print(f"AUC      : {roc_auc_score(y_test, logreg.predict_proba(X_test)[:,1]):.3f}")
print()
print("--- Logistic Regression (balanced) ---")
print(f"Precision: {precision_score(y_test, y_pred_bal):.3f}")
print(f"Recall   : {recall_score(y_test, y_pred_bal):.3f}")
print(f"AUC      : {roc_auc_score(y_test, y_prob_bal):.3f}")
```

```
--- Logistic Regression (default) ---
Precision: 0.661
Recall   : 0.516
AUC      : 0.845

--- Logistic Regression (balanced) ---
```

```
Precision: 0.504
Recall   : 0.802
AUC      : 0.845
```

Perhatikan pola: Dengan `balanced`, **recall naik** (lebih banyak churner tertangkap) tetapi **precision turun** (lebih banyak FP). AUC biasanya tetap sama—karena AUC mengukur **ranking**, bukan threshold. `class_weight` hanya **menggeser threshold efektif** dari 0,5 ke titik yang lebih menguntungkan kelas minoritas.

9.6.2 2. Threshold Adjustment

Alih-alih memakai threshold default 0,5, kita bisa memilih threshold yang mengoptimalkan metrik bisnis:

```python
thresholds = np.arange(0.1, 0.9, 0.05)
rows = []
for th in thresholds:
    y_pred_th = (y_prob >= th).astype(int)
    rows.append({
        "threshold": round(th, 2),
        "precision": precision_score(y_test, y_pred_th, zero_division=0),
        "recall": recall_score(y_test, y_pred_th),
        "f1": f1_score(y_test, y_pred_th),
    })

th_df = pd.DataFrame(rows).set_index("threshold").round(3)

# Cari threshold dengan F1 tertinggi
best_f1_th = th_df["f1"].idxmax()
print(f"Threshold optimal untuk F1: {best_f1_th}")
th_df.head(10)
```

```
Threshold optimal untuk F1: 0.35
```

threshold	precision	recall	f1
0.10	0.402	0.941	0.563
0.15	0.445	0.893	0.594
0.20	0.482	0.848	0.614

threshold	precision	recall	f1
0.25	0.509	0.802	0.623
0.30	0.535	0.759	0.628
0.35	0.572	0.719	0.637
0.40	0.596	0.639	0.617
0.45	0.641	0.596	0.618
0.50	0.658	0.529	0.587
0.55	0.686	0.420	0.521

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(th_df.index, th_df["precision"], marker="o", label="Precision", color="#4C72B0")
ax.plot(th_df.index, th_df["recall"], marker="s", label="Recall", color="#DD8452")
ax.plot(th_df.index, th_df["f1"], marker="^", label="F1", color="#55A868")
ax.axvline(0.5, color="gray", linestyle="--", alpha=0.5, label="Default (0,5)")
ax.axvline(best_f1_th, color="red", linestyle=":", alpha=0.7,
           label=f"F1 optimal ({best_f1_th})")
ax.set_xlabel("Threshold probabilitas")
ax.set_ylabel("Skor")
ax.set_title("Trade-off Precision-Recall vs Threshold")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

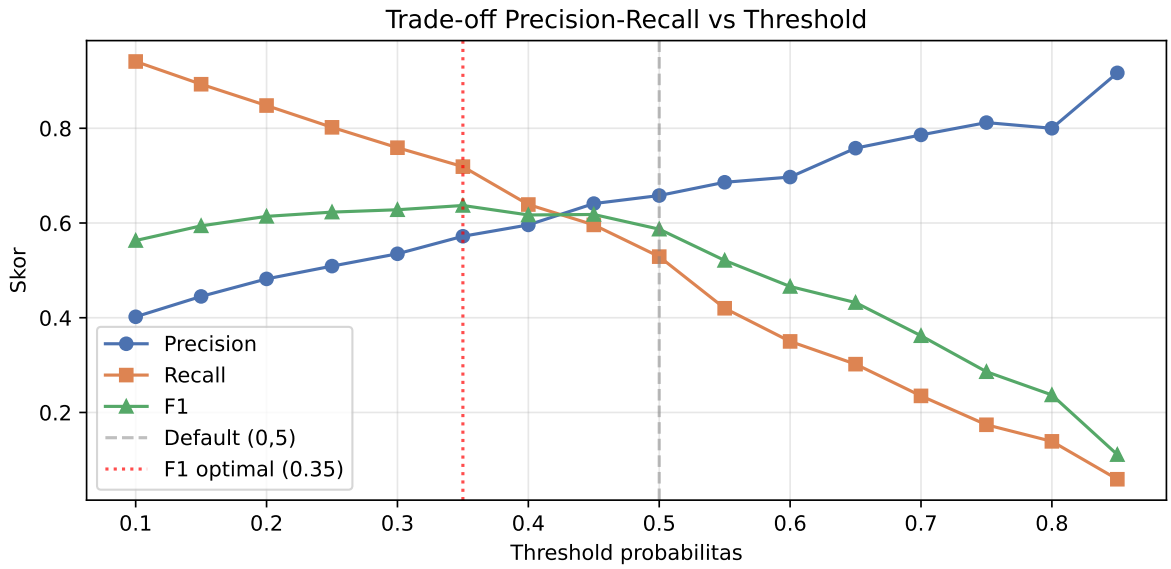


Figure 9.4: Precision, recall, dan F1 sebagai fungsi threshold. Threshold 0,5 bukan selalu optimal untuk bisnis.

> **Tip**
>
> **Threshold adalah keputusan bisnis, bukan keputusan statistik.** Tidak ada "threshold yang benar" secara universal. Threshold yang tepat bergantung pada: - Berapa banyak pelanggan yang bisa tim retensi hubungi per minggu. - Berapa biaya per intervensi. - Berapa nilai revenue yang hilang jika melewatkan churner.
> Threshold 0,3 mungkin lebih baik dari 0,5 jika kita ingin menangkap lebih banyak churner dan biaya FP rendah.

9.6.3 3. Resampling (SMOTE, Oversampling, Undersampling)

Alternatif lain adalah **mengubah data latih**: menambah sampel kelas minoritas (oversampling/SMOTE) atau mengurangi kelas mayoritas (undersampling).

> **Note**
>
> **Kapan resampling diperlukan?** Ketika kelas sangat tidak seimbang (mis. 1:100 atau lebih ekstrem). Untuk dataset Telco (1:2,8), `class_weight` dan threshold adjustment biasanya **cukup**—dan lebih sederhana. Resampling menambah kompleksitas dan risiko overfitting pada sampel sintetis.

Kita **tidak** mendemonstrasikan SMOTE di bab ini karena ia membutuhkan library tambahan (`imbalanced-learn`) dan untuk dataset Telco, `class_weight` sudah memadai. Namun, jika Anda bekerja dengan data yang sangat tidak seimbang, pertimbangkan:

```
# pip install imbalanced-learn
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
```

9.7 Kalibrasi: Apakah Probabilitas Dapat Dipercaya?

Model yang baik dalam **ranking** (AUC tinggi) belum tentu baik dalam **kalibrasi**—artinya, probabilitas yang dihasilkan mungkin tidak sesuai dengan frekuensi aktual.

Contoh: Jika model memprediksi probabilitas 0,8 untuk 100 pelanggan, apakah ~80 dari mereka benar-benar churn? Jika hanya 60 yang churn, model **tidak terkalibrasi**—probabilitasnya terlalu tinggi.

9.7.1 Mengukur Kalibrasi

```
fig, ax = plt.subplots(figsize=(7, 6))

for (name, model), color in zip(models.items(), colors):
    y_prob = model.predict_proba(X_test)[:, 1]
    prob_true, prob_pred = calibration_curve(y_test, y_prob, n_bins=10, strategy="quantile")
    ax.plot(prob_pred, prob_true, marker="o", label=name, color=color, linewidth=1.5)

ax.plot([0, 1], [0, 1], "k--", label="Sempurna", alpha=0.5)
ax.set_xlabel("Probabilitas prediksi rata-rata")
ax.set_ylabel("Frekuensi churn aktual")
ax.set_title("Calibration Curve - Enam Model")
ax.legend(loc="lower right", fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

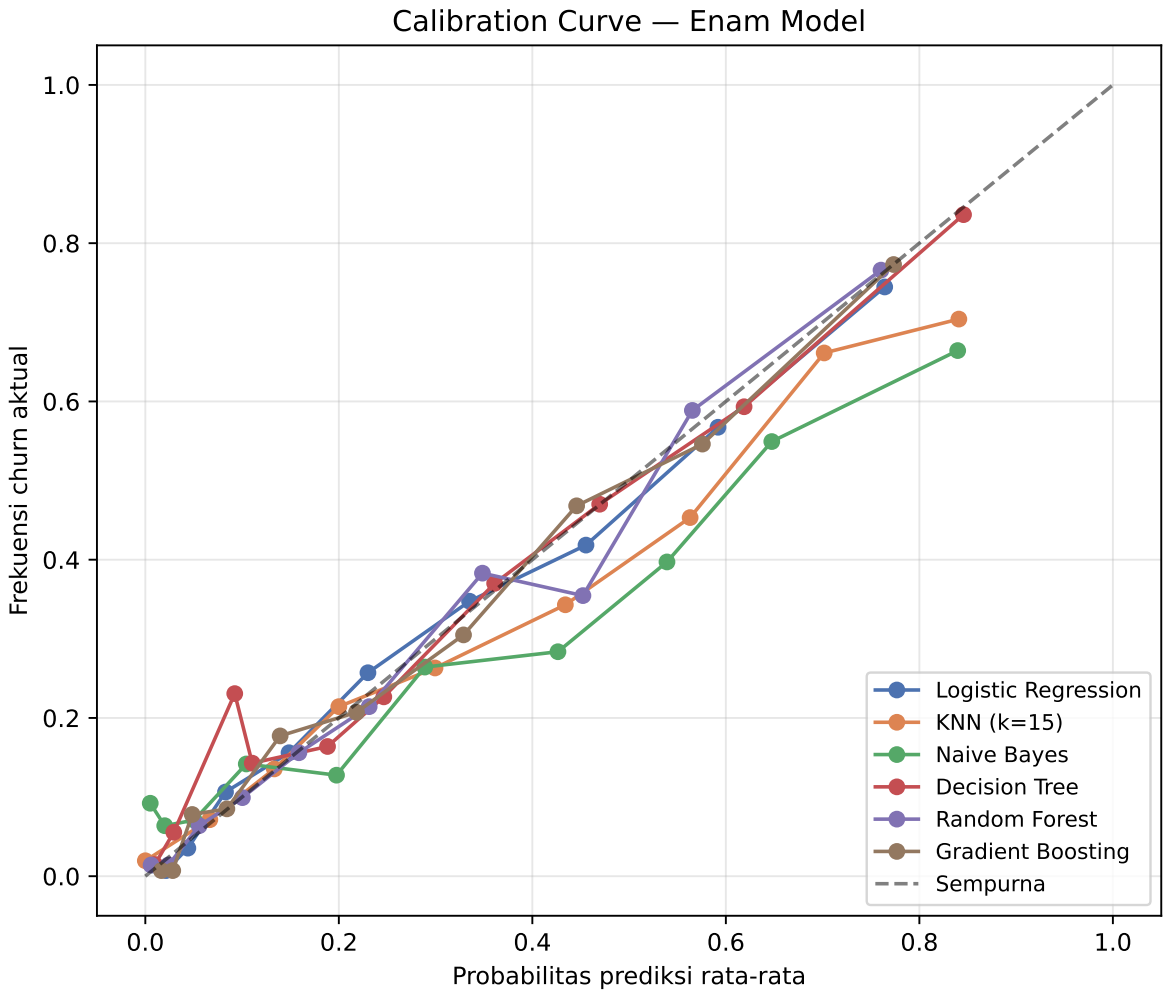


Figure 9.5: Calibration curve. Sumbu X = probabilitas prediksi; sumbu Y = frekuensi aktual. Idealnya titik-titik berada di diagonal.

```
# Brier score: semakin kecil semakin baik (0 = sempurna, 0.25 = tidak informatif)
brier = pd.DataFrame({
    "Model": list(models.keys()),
    "Brier score": [brier_score_loss(y_test, m.predict_proba(X_test)[:, 1])
                    for m in models.values()],
}).set_index("Model").round(3).sort_values("Brier score")
brier
```

Model	Brier score
Logistic Regression	0.136
Gradient Boosting	0.136
Random Forest	0.137
Decision Tree	0.140
KNN (k=15)	0.148
Naive Bayes	0.165

Cara membaca:

- **Calibration curve**: Jika model terkalibrasi dengan baik, titik-titik akan mengikuti garis diagonal (probabilitas prediksi = frekuensi aktual). Model yang **di atas diagonal** underestimasi; di **bawah** overestimasi.
- **Brier score**: Menggabungkan kalibrasi dan diskriminasi. Semakin kecil, semakin baik. Brier = 0,25 setara dengan selalu memprediksi 0,5.

> **!** Important
>
> **Mengapa kalibrasi penting untuk keputusan?** Jika kita memakai probabilitas untuk keputusan—misalnya "hubungi pelanggan dengan probabilitas > 0,6"—maka probabilitas itu **harus dapat dipercaya**. Model dengan AUC tinggi tetapi kalibrasi buruk bisa membuat kita menghubungi terlalu banyak atau terlalu sedikit pelanggan.
> **Random Forest dan Gradient Boosting sering tidak terkalibrasi**—terutama ketika menggunakan `class_weight` atau data tidak seimbang. Jika probabilitas dibutuhkan untuk keputusan, pertimbangkan **calibration** (`sklearn.calibration.CalibratedClassifierCV`) atau pilih Logistic Regression yang secara alami terkalibrasi.

9.8 Feature Importance: Mengapa Model Memprediksi Apa yang Dia Prediksi

Salah satu keuntungan model tree-based adalah **feature importance**—ia memberi tahu kita fitur mana yang paling berpengaruh.

```
feature_names = rf.named_steps["prep"].get_feature_names_out()
importances = rf.named_steps["clf"].feature_importances_

imp_df = (pd.DataFrame({
```

```
    "fitur": feature_names,
    "importance": importances,
}).sort_values("importance", ascending=False).head(15))

fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(imp_df["fitur"][::-1], imp_df["importance"][::-1], color="#4C72B0")
ax.set_xlabel("Importance")
ax.set_title("Feature Importance - Random Forest (Top 15)")
plt.tight_layout()
plt.show()
```

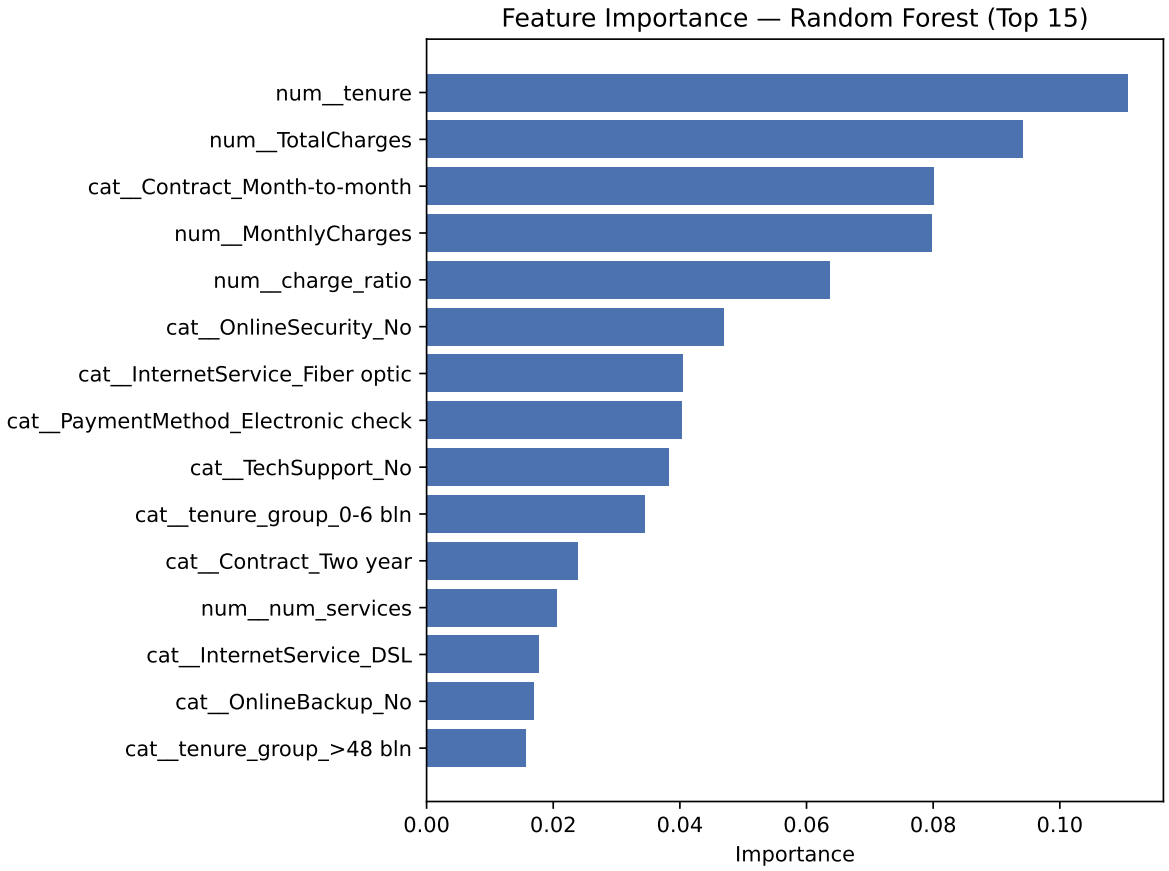


Figure 9.6: Feature importance dari Random Forest. Fitur paling penting terkonsentrasi pada tenure, kontrak, dan tagihan.

Cara membaca feature importance:

- Nilai importance menunjukkan **seberapa besar** fitur berkontribusi pada prediksi—bukan **arah** pengaruhnya.
- Fitur dengan importance tinggi adalah fitur yang sering dipakai untuk **split** di pohon, dan split-nya menghasilkan **pemisahan kelas yang lebih baik**.
- **Konsisten dengan diagnosis Bagian I**: `tenure`, `Contract`, dan `TotalCharges` kemungkinan besar menjadi fitur teratas—mengonfirmasi temuan kita di Bab 3–5.

> ⚠ Warning
>
> **Feature importance bukan kausalitas.** Fitur dengan importance tinggi **berkorelasi** dengan target, tetapi tidak berarti **menyebabkan** target. `TotalCharges` mungkin penting karena ia berkorelasi dengan `tenure`—bukan karena ia sendiri menyebabkan churn. Jangan menginterpretasikan feature importance sebagai bukti kausal.

9.9 Memilih Model Akhir

Sekarang kita memiliki enam model dengan profil yang berbeda. Bagaimana memilihnya?

9.9.1 Kerangka Keputusan

Kriteria	Model Terbaik	Catatan
AUC tertinggi	Gradient Boosting / Random Forest	Performa ranking terbaik
Recall tertinggi	Gradient Boosting (dengan threshold rendah)	Menangkap paling banyak churner
Kalibrasi terbaik	Logistic Regression	Probabilitas dapat dipercaya
Interpretabilitas	Logistic Regression / Decision Tree	Dapat dijelaskan ke stakeholder
Kecepatan training	Naive Bayes / Logistic Regression	Penting untuk iterasi cepat
Feature importance	Random Forest / Gradient Boosting	Memberi insight tambahan

9.9.2 Rekomendasi untuk Dataset Telco

```
final = hasil_df[["AUC", "PR-AUC", "Recall", "Precision", "Brier"]].copy()
final["Rekomendasi"] = [
    "  Baseline interpretabel; kalibrasi terbaik",
    "Perlu scaling; tidak direkomendasikan",
    "Cepat tetapi lemah; gunakan sebagai baseline",
    "Interpretabel; cenderung overfitting",
    "  Keseimbangan performa & interpretabilitas",
    "  Performa terbaik; perlu tuning",
]
final
```

Model	AUC	PR-AUC	Recall	Precision	Brier	Rekomendasi
Logistic Regression	0.845	0.650	0.516	0.661	0.136	Baseline interpretabel; kalibrasi terbaik
KNN (k=15)	0.822	0.584	0.572	0.593	0.148	Perlu scaling; tidak direkomendasikan
Naive Bayes	0.778	0.564	0.602	0.546	0.165	Cepat tetapi lemah; gunakan sebagai base
Decision Tree	0.829	0.619	0.570	0.630	0.140	Interpretabel; cenderung overfitting
Random Forest	0.841	0.653	0.519	0.651	0.137	Keseimbangan performa & interpretabili
Gradient Boosting	0.843	0.659	0.529	0.658	0.136	Performa terbaik; perlu tuning

Rekomendasi:

1. **Untuk produksi retensi**: **Gradient Boosting** dengan threshold yang disesuaikan—memberikan recall tertinggi pada precision yang wajar.
2. **Untuk komunikasi ke stakeholder**: **Logistic Regression**—memberikan odds ratio yang dapat diinterpretasikan dan probabilitas yang terkalibrasi.
3. **Untuk eksplorasi awal**: **Random Forest**—memberikan feature importance dan performa yang baik dengan sedikit tuning.

> **!** Important
>
> **Pilihan model bukan keputusan final—ia adalah iterasi.** Model yang dipilih harus **dipantau** setelah deployment. Jika performa menurun (mis. karena perubahan perilaku pelanggan), model perlu dilatih ulang. Ini akan dibahas di Bab 18.

9.10 Latihan Terbimbing

Latihan 9.1 — Cost-sensitive evaluation. Asumsikan biaya FP = Rp 50.000 (satu panggilan telepon) dan biaya FN = Rp 2.000.000 (kehilangan pelanggan). Hitung total biaya

untuk setiap model pada threshold 0,5. Model mana yang memiliki biaya terendah? Apakah ini model yang sama dengan yang memiliki AUC tertinggi?

Latihan 9.2 — Threshold optimization. Untuk model Gradient Boosting, cari threshold yang **meminimalkan total biaya** berdasarkan cost matrix di Latihan 9.1. Bandingkan threshold optimal dengan threshold default 0,5. Berapa penghematan biaya yang dihasilkan?

Latihan 9.3 — KNN tuning. Variasi `n_neighbors` dari 3 hingga 50. Plot AUC vs `n_neighbors`. Pada nilai berapa AUC tertinggi? Mengapa KNN dengan `n_neighbors=1` cenderung overfitting?

Latihan 9.4 — Decision tree interpretasi. Latih decision tree dengan `max_depth=3`. Gambarkan pohonnya menggunakan `sklearn.tree.plot_tree`. Apa aturan pertama yang dipelajari pohon? Apakah aturan itu masuk akal secara bisnis?

Latihan 9.5 — Kalibrasi model. Ambil model Gradient Boosting dan kalibrasi dengan `CalibratedClassifierCV`. Bandingkan Brier score sebelum dan sesudah kalibrasi. Apakah kalibrasi mengubah ranking (AUC) atau hanya probabilitas?

Latihan 9.6 — Feature importance vs odds ratio. Bandingkan feature importance dari Random Forest dengan odds ratio dari Logistic Regression. Apakah fitur teratas sama? Jika tidak, mengapa? Mana yang lebih dapat dipercaya untuk interpretasi?

> **Note**
>
> **Petunjuk Latihan 9.1.** Total biaya = (FP $\times$ 50.000) + (FN $\times$ 2.000.000). Hitung dari confusion matrix setiap model. Model dengan **recall tinggi** akan menang karena FN jauh lebih mahal dari FP—tetapi mungkin ada model yang menyeimbangkan keduanya dengan lebih baik. Ini adalah latihan penting untuk memahami bahwa **metrik harus mencerminkan biaya bisnis**.

9.11 Kotak "Jembatan": Dari Prediksi ke Keputusan

> **Important**
>
> **Jembatan.** Bab ini memberi kita **model yang dapat memprediksi probabilitas churn per pelanggan**. Ini adalah kemajuan besar dari Bagian I—kita sekarang dapat menjawab pertanyaan yang tidak bisa dijawab oleh statistik deskriptif:
>
> > *"Pelanggan mana yang paling mungkin churn bulan ini?"*
>
> Tetapi prediksi saja **belum cukup** untuk keputusan. Tiga pertanyaan baru muncul:
>
> 1. **Siapa yang harus diintervensi?** Tidak semua pelanggan berisiko tinggi layak

diintervensi—beberapa mungkin tidak merespons, atau biaya intervensi melebihi nilai yang diselamatkan.
2. **Intervensi apa yang harus diberikan?** Model prediktif memberi tahu kita **siapa**, tetapi tidak **apa**. Kita perlu memahami **efek** dari berbagai intervensi terhadap probabilitas churn.
3. **Bagaimana mengalokasikan anggaran terbatas?** Bahkan dengan daftar pelanggan berisiko, kita tetap perlu memilih subset yang akan diintervensi berdasarkan **anggaran** dan **efektivitas**.

Ketiga pertanyaan ini adalah domain prescriptive analytics—dan inilah tujuan Bagian III. Model prediktif dari bab ini akan menjadi **input**: probabilitas churn per pelanggan akan menjadi parameter dalam model optimasi.
Dengan kata lain: **kita telah membangun mesin prediksi; sekarang kita akan membangun mesin keputusan.**

9.12 Rangkuman

Dalam bab ini kita telah:

- **Membangun enam model klasifikasi** dengan karakteristik berbeda: Logistic Regression (interpretabel), KNN (berbasis kemiripan), Naive Bayes (probabilistik), Decision Tree (berbasis aturan), Random Forest (ensemble paralel), dan Gradient Boosting (ensemble sekuensial).
- **Membaca confusion matrix** dan memahami bahwa setiap sel memiliki konsekuensi bisnis yang berbeda—terutama trade-off antara False Positive dan False Negative.
- **Menguasai metrik evaluasi**—accuracy, precision, recall, F1, ROC-AUC, PR-AUC, Brier score—dan memahami **kapan masing-masing tepat digunakan**.
- **Menyadari bahwa accuracy menyesatkan** untuk kelas tidak seimbang, dan bahwa pemilihan metrik harus **mencerminkan konsekuensi bisnis**.
- **Menangani kelas tidak seimbang** dengan `class_weight` dan threshold adjustment, memahami bahwa keduanya menggeser threshold efektif tanpa mengubah ranking (AUC).
- **Membedakan ranking (AUC) dari kalibrasi (Brier)**, dan memahami bahwa model dengan AUC tinggi belum tentu terkalibrasi.
- **Menginterpretasikan feature importance** dari Random Forest dan menghubungkannya kembali ke diagnosis Bagian I.
- **Memilih model akhir** berdasarkan kerangka keputusan yang mempertimbangkan performa, interpretabilitas, dan konteks bisnis.

9.13 Jembatan ke Bab 10

Sampai di sini, semua model kita bekerja pada data **cross-sectional**—satu baris per pelanggan, tanpa dimensi waktu. Tetapi banyak keputusan analytics membutuhkan **waktu**:

- Berapa permintaan produk bulan depan?
- Kapan pelanggan akan churn?
- Bagaimana pola musiman penjualan?

Bab 10—**Time Series**—akan memperkenalkan teknik untuk data yang **berurutan dalam waktu**. Kita akan beralih ke **dataset pendamping retail** yang memiliki data harian selama dua tahun—dan mempelajari:

- Dekomposisi time series (tren, musiman, residual).
- Metode forecasting klasik (exponential smoothing, ARIMA).
- Validasi *rolling-origin* untuk data waktu.
- Analisis survival sebagai jembatan untuk memahami **kapan** pelanggan churn, bukan hanya **apakah**.

Bab 10 akan menjadi **jembatan** antara prediksi cross-sectional (Bab 7–9) dan simulasi keputusan (Bab 12).

10 Time Series & Forecasting

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membedakan data cross-sectional dari data time series, dan memahami mengapa keduanya membutuhkan pendekatan berbeda. 2. Mendekomposisi time series menjadi tren, musiman, dan residual. 3. Menerapkan metode forecasting klasik—exponential smoothing dan ARIMA—pada data nyata. 4. Memvalidasi model time series dengan **rolling-origin cross-validation**, bukan random split. 5. Mengevaluasi forecast dengan metrik yang tepat (MAE, RMSE, MAPE). 6. Menerapkan **analisis survival** untuk menjawab "kapan" pelanggan churn, bukan hanya "apakah". 7. Memahami mengapa forecasting adalah fondasi untuk simulasi keputusan di Bagian III.

10.1 Mengapa Kita Beralih Dataset

Sampai Bab 9, semua analisis kita menggunakan **Telco Customer Churn**—dataset cross-sectional dengan satu baris per pelanggan dan tanpa dimensi waktu eksplisit. Dataset ini sangat cocok untuk klasifikasi: kita memprediksi apakah seorang pelanggan akan churn berdasarkan profilnya saat ini.

Tetapi tidak semua pertanyaan analytics dapat dijawab dengan struktur data seperti itu. Pertanyaan-pertanyaan berikut membutuhkan **dimensi waktu**:

- Berapa permintaan produk bulan depan?
- Kapan pelanggan akan churn (bukan hanya *apakah*)?
- Bagaimana pola musiman penjualan sepanjang tahun?
- Berapa banyak staf yang perlu dijadwalkan minggu depan?

Untuk menjawabnya, kita membutuhkan **data time series**: urutan observasi yang diambil pada interval waktu teratur.

Important

Dataset pendamping lapis kedua. Di awal buku, kita menyepakati strategi **dua lapis data**: - **Lapis 1 — Telco Churn**: dataset utama untuk Bab 1–9, 11, 13–15, 18. - **Lapis 2 — Retail Store Inventory Forecasting**: dataset pendamping untuk modul yang

membutuhkan struktur waktu dan aliran—yaitu **Bab 10, 12, 16, 17**.
Bab ini adalah **perkenalan pertama** dengan dataset retail. Kita akan kembali ke Telco di akhir bab untuk analisis survival, tetapi fokus utama bab ini adalah retail.

10.2 Memuat Dataset Retail

Dataset **Retail Store Inventory Forecasting** berisi data harian penjualan dan inventori dari 5 toko dan 20 produk selama dua tahun (1 Januari 2022 – 1 Januari 2024). Total sekitar **73.100 baris**.

Jika belum diunduh, ikuti instruksi di `README.md` atau unduh dari Kaggle:

```
# Jika belum ada, unduh dari Kaggle:
# https://www.kaggle.com/datasets/anirudhchauhan/retail-store-inventory-forecasting-dataset
```

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from decanal.utils import load_retail

df_retail = load_retail()
print(f"Bentuk: {df_retail.shape[0]:,} baris × {df_retail.shape[1]} kolom")
print(f"\nKolom: {df_retail.columns.tolist()}")
print(f"\nRentang tanggal: {df_retail['Date'].min()} → {df_retail['Date'].max()}")
print(f"Jumlah toko    : {df_retail['Store ID'].nunique()}")
print(f"Jumlah produk  : {df_retail['Product ID'].nunique()}")
```

```
Bentuk: 73,100 baris × 15 kolom

Kolom: ['Date', 'Store ID', 'Product ID', 'Category', 'Region', 'Inventory Level', 'Units Sol

Rentang tanggal: 2022-01-01 → 2024-01-01
Jumlah toko    : 5
Jumlah produk  : 20
```

10.2.1 Inspeksi Awal

```
df_retail["Date"] = pd.to_datetime(df_retail["Date"])

# Beberapa baris pertama
df_retail[["Date", "Store ID", "Product ID", "Category", "Region",
           "Inventory Level", "Units Sold", "Units Ordered",
           "Price", "Discount", "Weather Condition", "Holiday/Promotion"]].head()
```

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered
0	2022-01-01	S001	P0001	Groceries	North	231	127	55
1	2022-01-01	S001	P0002	Toys	South	204	150	66
2	2022-01-01	S001	P0003	Toys	West	102	65	51
3	2022-01-01	S001	P0004	Toys	North	469	61	164
4	2022-01-01	S001	P0005	Electronics	East	166	14	135

```
# Ringkasan numerik
df_retail[["Inventory Level", "Units Sold", "Units Ordered",
           "Price", "Discount", "Competitor Pricing"]].describe().round(2)
```

	Inventory Level	Units Sold	Units Ordered	Price	Discount	Competitor Pricing
count	73100.00	73100.00	73100.00	73100.00	73100.00	73100.00
mean	274.47	136.46	110.00	55.14	10.01	55.15
std	129.95	108.92	52.28	26.02	7.08	26.19
min	50.00	0.00	20.00	10.00	0.00	5.03
25%	162.00	49.00	65.00	32.65	5.00	32.68
50%	273.00	107.00	110.00	55.05	10.00	55.01
75%	387.00	203.00	155.00	77.86	15.00	77.82
max	500.00	499.00	200.00	100.00	20.00	104.94

Tip

Perhatikan struktur data. Berbeda dari Telco: - **Satu baris** = kombinasi (tanggal, toko, produk), bukan satu pelanggan. - **Target potensial** = `Units Sold` (kontinu), bukan churn (biner). - **Dimensi waktu** = `Date`—inilah yang memungkinkan analisis time series.
Dengan struktur ini, kita bisa menjawab pertanyaan seperti: *"Berapa unit produk X yang*

akan terjual di toko Y bulan depan?"

10.3 Dari Cross-Sectional ke Time Series

Sebelum masuk ke teknik, penting memahami **mengapa time series memerlukan pendekatan berbeda**.

Aspek	Cross-sectional (Telco)	Time series (Retail)
Unit observasi	Pelanggan (independen)	Titik waktu (berurutan)
Asumsi independensi	Observasi saling independen	Observasi **saling bergantung** (autokorelasi)
Split data	Random train/test split	**Chronological split** (train = masa lalu, test = masa depan)
Validasi	Cross-validation biasa	**Rolling-origin** cross-validation
Metrik	AUC, precision, recall	MAE, RMSE, MAPE
Pertanyaan	"Apakah pelanggan ini akan churn?"	"Berapa permintaan minggu depan?"

> ⚠ Warning
>
> **Kesalahan paling umum di time series.** Menggunakan `train_test_split` acak untuk data time series. Ini **data leakage**—Anda melatih model pada data dari masa depan untuk memprediksi masa lalu. Model akan tampak sangat baik selama pengujian, tetapi **gagal total** di dunia nyata karena di produksi Anda hanya punya data masa lalu. **Aturan emas**: selalu split berdasarkan waktu. Train = periode lebih awal, test = periode lebih akhir.

10.4 Agregasi: Dari Transaksi ke Deret Waktu

Dataset retail berisi data granular per (tanggal, toko, produk). Untuk forecasting permintaan **total**, kita perlu mengagregasi ke level yang tepat. Mari kita lihat total unit terjual per hari di semua toko:

```
daily = (df_retail
         .groupby("Date", as_index=False)
         .agg(total_sold=("Units Sold", "sum"),
              total_ordered=("Units Ordered", "sum"),
              n_transactions=("Units Sold", "size")))

print(f"Jumlah hari: {len(daily)}")
print(f"Rentang    : {daily['Date'].min().date()} → {daily['Date'].max().date()}")
daily.head()
```

```
Jumlah hari: 731
Rentang    : 2022-01-01 → 2024-01-01
```

	Date	total_sold	total_ordered	n_transactions
0	2022-01-01	14484	10921	100
1	2022-01-02	13415	10770	100
2	2022-01-03	13681	11096	100
3	2022-01-04	14084	11572	100
4	2022-01-05	12572	10919	100

```
fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(daily["Date"], daily["total_sold"], color="#4C72B0", linewidth=1, alpha=0.7)
ax.set_xlabel("Tanggal")
ax.set_ylabel("Total unit terjual")
ax.set_title("Penjualan Harian - Semua Toko (2022-2024)")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

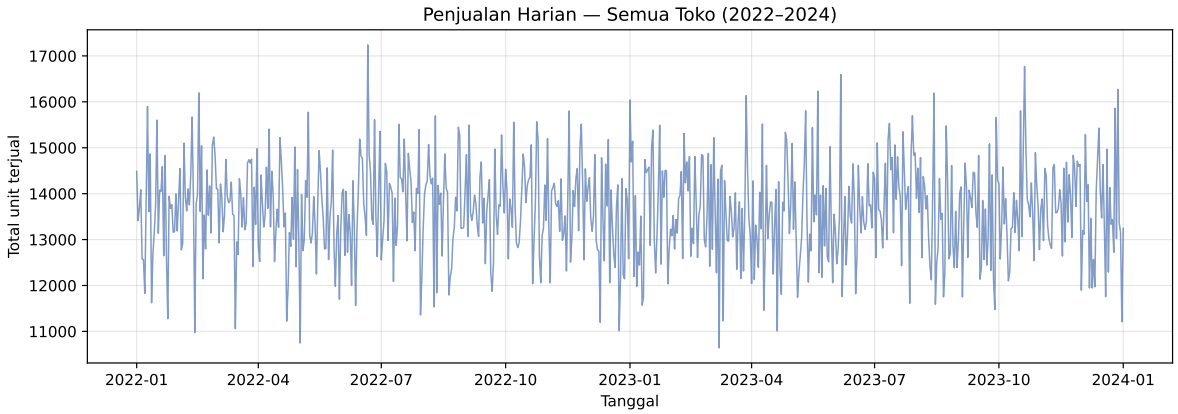


Figure 10.1: Total unit terjual harian di semua toko. Perhatikan pola berulang dan tren yang mungkin.

Amati grafik ini. Tiga pola yang perlu Anda kenali:

1. **Tren jangka panjang** — apakah penjualan cenderung naik, turun, atau datar sepanjang dua tahun?
2. **Pola musiman** — apakah ada siklus berulang (mingguan, bulanan, tahunan)?
3. **Noise / residual** — variasi acak yang tersisa setelah tren dan musiman dihilangkan.

10.5 Dekomposisi Time Series

Dekomposisi adalah proses memisahkan time series menjadi tiga komponen:

$$y_t = T_t + S_t + R_t \quad \text{(aditif)}$$
$$y_t = T_t \times S_t \times R_t \quad \text{(multiplikatif)}$$

- T_t **(Tren)**: arah jangka panjang.
- S_t **(Musiman)**: pola berulang dengan periode tetap.
- R_t **(Residual)**: sisa yang tidak dijelaskan.

10.5.1 Dekomposisi Aditif vs Multiplikatif

Model	Kapan dipakai
Aditif	Amplitudo musiman **konstan** sepanjang waktu

Model	Kapan dipakai
Multiplikatif	Amplitudo musiman **proporsional** terhadap level tren

Untuk data penjualan yang tumbuh, model multiplikatif sering lebih tepat karena fluktuasi musiman juga membesar seiring pertumbuhan.

10.5.2 Melakukan Dekomposisi

```
from statsmodels.tsa.seasonal import seasonal_decompose

# Set index ke Date untuk time series analysis
ts = daily.set_index("Date")["total_sold"].asfreq("D")

# Dekomposisi aditif dengan periode 7 (mingguan)
decomp_add = seasonal_decompose(ts, model="additive", period=7)
decomp_mult = seasonal_decompose(ts, model="multiplicative", period=7)
```

```
fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
decomp_add.observed.plot(ax=axes[0], color="#4C72B0")
axes[0].set_ylabel("Observasi")
axes[0].set_title("Dekomposisi Aditif - Penjualan Harian")

decomp_add.trend.plot(ax=axes[1], color="#DD8452")
axes[1].set_ylabel("Tren")

decomp_add.seasonal.plot(ax=axes[2], color="#55A868")
axes[2].set_ylabel("Musiman")

decomp_add.resid.plot(ax=axes[3], color="#8172B3")
axes[3].set_ylabel("Residual")
axes[3].set_xlabel("Tanggal")

plt.tight_layout()
plt.show()
```

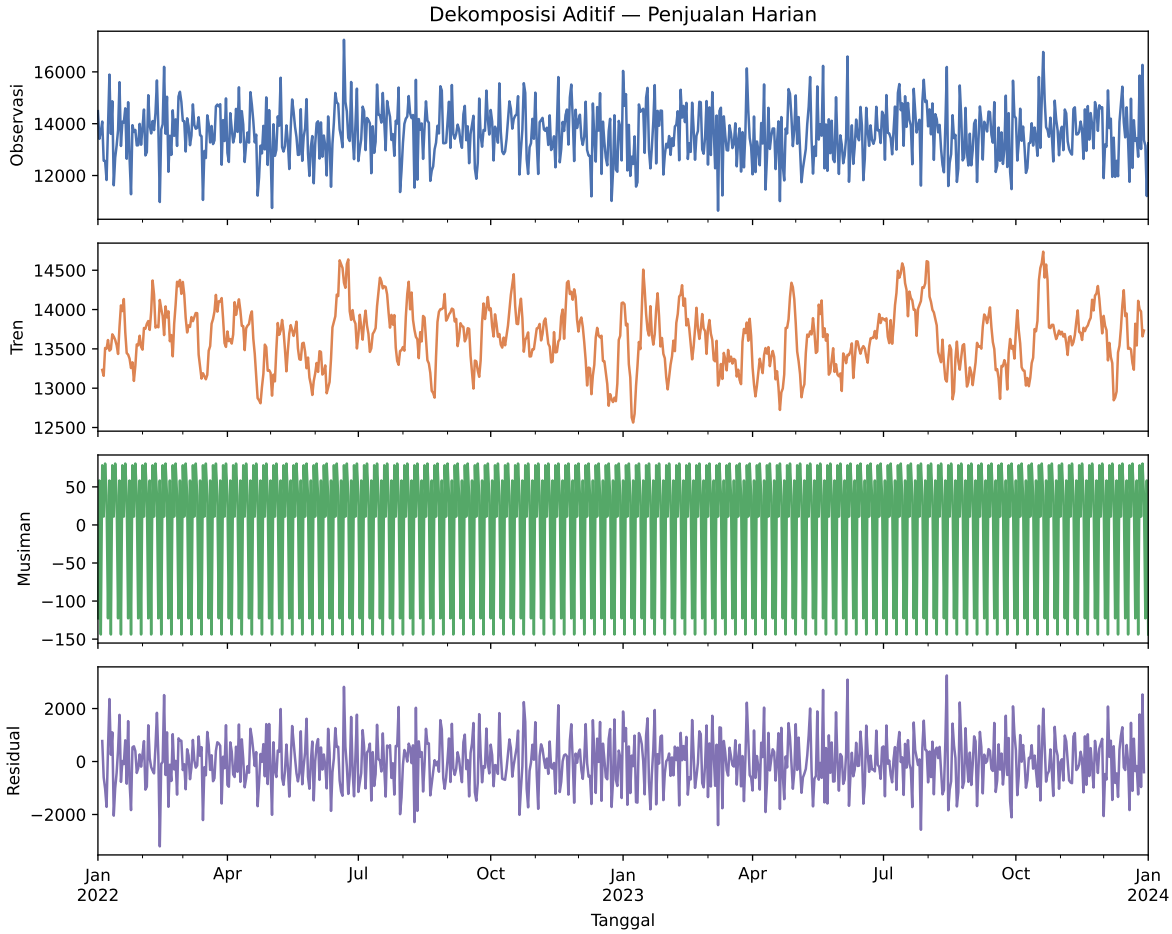


Figure 10.2: Dekomposisi aditif: observasi asli, tren, komponen musiman (mingguan), dan residual.

Cara membaca dekomposisi:

- **Tren**: pola jangka panjang—apakah penjualan naik, turun, atau datar.
- **Musiman**: pola berulang dengan periode 7 hari—menunjukkan **hari apa** penjualan tinggi/rendah. Jika pola musiman menunjukkan puncak di akhir pekan, itu adalah insight penting untuk staf scheduling dan inventory.
- **Residual**: sisa setelah tren dan musiman dihilangkan—harusnya terlihat seperti white noise (acak tanpa pola). Jika ada pola tersisa di residual, berarti ada komponen lain yang belum ditangkap.

Tip

Mengapa dekomposisi berguna? Ia memisahkan sinyal dari noise. Tren memberi tahu kita **arah bisnis**; musiman memberi tahu kita **pola operasional**; residual memberi tahu kita **seberapa banyak variasi yang tidak dapat dijelaskan model**. Ketiga komponen ini akan menjadi input untuk forecasting dan—lebih penting—untuk simulasi ketidakpastian di Bab 12.

10.6 Uji Stasioneritas

Sebelum menerapkan model seperti ARIMA, kita perlu memeriksa apakah time series **stasioner**—artinya, properti statistiknya (mean, variance) tidak berubah sepanjang waktu. Banyak model time series klasik mengasumsikan stasioneritas.

Augmented Dickey-Fuller (ADF) test adalah uji yang paling umum:

- **H** : time series memiliki **unit root** (tidak stasioner).
- **H** : time series **stasioner**.

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(ts.dropna(), autolag="AIC")
print(f"ADF Statistic : {result[0]:.4f}")
print(f"p-value       : {result[1]:.4f}")
print(f"Kesimpulan    : ", end="")
if result[1] < 0.05:
    print("Tolak H  - series STASIONER")
else:
    print("Gagal tolak H  - series TIDAK STASIONER (perlu differencing)")
```

```
ADF Statistic : -26.3795
p-value       : 0.0000
Kesimpulan    : Tolak H  - series STASIONER
```

Jika series tidak stasioner, kita bisa melakukan **differencing**—menghitung selisih antar observasi berturut-turut:

```python
ts_diff = ts.diff().dropna()

result_diff = adfuller(ts_diff, autolag="AIC")
print(f"Setelah differencing:")
print(f"ADF Statistic : {result_diff[0]:.4f}")
print(f"p-value       : {result_diff[1]:.4f}")
```

```
Setelah differencing:
ADF Statistic : -10.7756
p-value       : 0.0000
```

```python
fig, axes = plt.subplots(2, 1, figsize=(10, 5), sharex=True)
axes[0].plot(ts.index, ts.values, color="#4C72B0")
axes[0].set_ylabel("Total terjual")
axes[0].set_title("Series Asli")

axes[1].plot(ts_diff.index, ts_diff.values, color="#DD8452")
axes[1].set_ylabel("Δ Total terjual")
axes[1].set_title("Setelah Differencing (order 1)")
axes[1].set_xlabel("Tanggal")
plt.tight_layout()
plt.show()
```

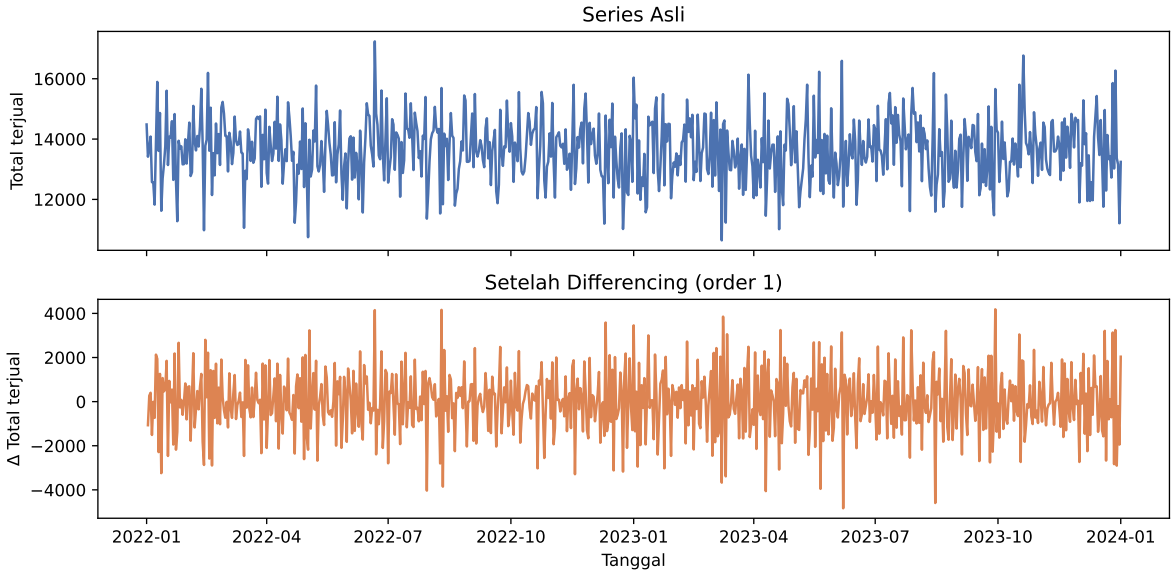


Figure 10.3: Perbandingan series asli dan setelah differencing. Differencing menghilangkan tren dan membuat mean konstan.

10.7 Forecasting: Meramalkan Masa Depan

Sekarang kita masuk ke inti bab: **forecasting**. Kita akan membandingkan tiga metode dengan kompleksitas yang berbeda.

10.7.1 Metode 1: Naive Forecast (Baseline)

Naive forecast memprediksi nilai besok = nilai hari ini. Sederhana, tetapi sering menjadi baseline yang sulit dikalahkan.

```
# Split berdasarkan waktu: 80% train, 20% test
n = len(ts)
split_idx = int(n * 0.8)
train, test = ts.iloc[:split_idx], ts.iloc[split_idx:]

print(f"Train: {train.index.min().date()} → {train.index.max().date()} ({len(train)} hari)")
print(f"Test : {test.index.min().date()} → {test.index.max().date()} ({len(test)} hari)")

# Naive: prediksi = nilai terakhir train, konstan sepanjang horizon test
naive_pred = pd.Series([train.iloc[-1]] * len(test), index=test.index)
```

```
Train: 2022-01-01 → 2023-08-07 (584 hari)
Test : 2023-08-08 → 2024-01-01 (147 hari)
```

10.7.2 Metode 2: Exponential Smoothing (Holt-Winters)

Holt-Winters memperluas exponential smoothing dengan komponen tren dan musiman. Ia cocok untuk data dengan pola musiman berulang.

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

hw_model = ExponentialSmoothing(
    train,
    trend="add",
    seasonal="add",
    seasonal_periods=7,  # mingguan
).fit()

hw_pred = hw_model.forecast(len(test))
```

10.7.3 Metode 3: ARIMA

ARIMA (AutoRegressive Integrated Moving Average) adalah model klasik yang menggabungkan: - **AR** (AutoRegressive): regresi terhadap nilai masa lalu. - **I** (Integrated): differencing untuk stasioneritas. - **MA** (Moving Average): regresi terhadap error masa lalu.

```
from statsmodels.tsa.arima.model import ARIMA

arima_model = ARIMA(train, order=(1, 1, 1)).fit()
arima_pred = arima_model.forecast(len(test))
```

10.7.4 Membandingkan Ketiga Metode

```
fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(train.index[-60:], train.values[-60:], color="#4C72B0", label="Train (60 hari terakhi
ax.plot(test.index, test.values, color="black", label="Aktual (test)", linewidth=1.5)
ax.plot(test.index, naive_pred, color="#C44E52", linestyle="--", label="Naive", linewidth=1.5
ax.plot(test.index, hw_pred, color="#DD8452", label="Holt-Winters", linewidth=2)
ax.plot(test.index, arima_pred, color="#55A868", linestyle=":", label="ARIMA(1,1,1)", linewid
ax.set_xlabel("Tanggal")
ax.set_ylabel("Total unit terjual")
ax.set_title("Perbandingan Metode Forecasting")
ax.legend(loc="upper left")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

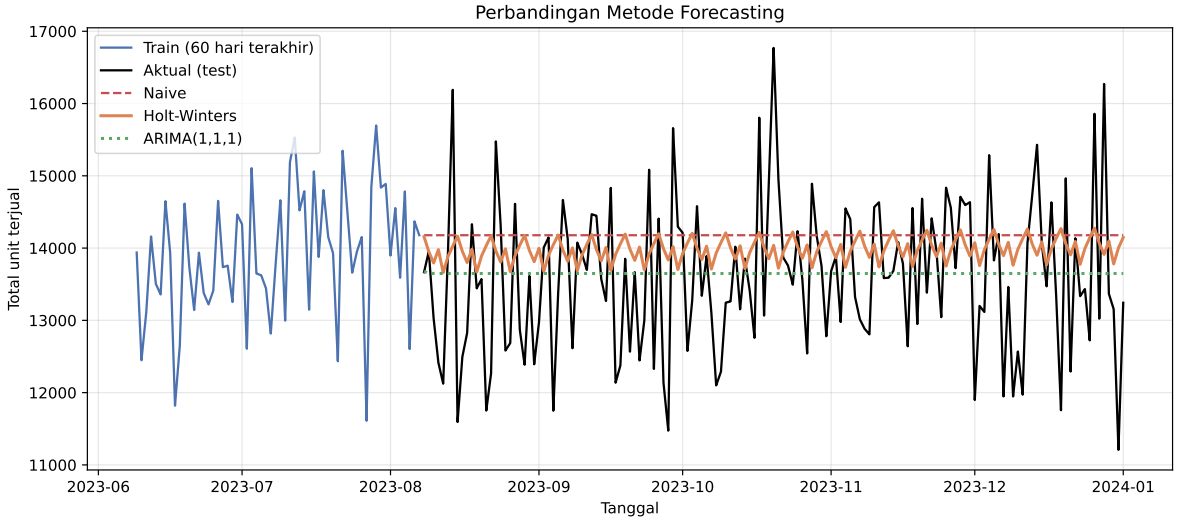


Figure 10.4: Perbandingan tiga metode forecasting pada test set. Holt-Winters menangkap pola musiman; naive hanya mengulang nilai terakhir.

10.8 Evaluasi Forecast: Metrik yang Berbeda

Tiga metrik paling umum untuk time series:

Metrik	Formula	Interpretasi	Kapan dipakai
MAE	$\frac{1}{n}\sum \|y - \hat{y}\|$	Rata-rata kesalahan absolut (satuan target)	Ketika outlier tidak terlalu penting
RMSE	$\sqrt{\frac{1}{n}\sum (y-\hat{y})^2}$	Akar rata-rata kuadrat kesalahan	Ketika kesalahan besar sangat mahal
MAPE	$\frac{1}{n}\sum \left\|\frac{y-\hat{y}}{y}\right\| \times 100$	Rata-rata persentase kesalahan	Ketika skala target bervariasi

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

def mape(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100

metrics = pd.DataFrame({
    "Metode": ["Naive", "Holt-Winters", "ARIMA(1,1,1)"],
    "MAE": [
```

```
        mean_absolute_error(test, naive_pred),
        mean_absolute_error(test, hw_pred),
        mean_absolute_error(test, arima_pred),
    ],
    "RMSE": [
        np.sqrt(mean_squared_error(test, naive_pred)),
        np.sqrt(mean_squared_error(test, hw_pred)),
        np.sqrt(mean_squared_error(test, arima_pred)),
    ],
    "MAPE (%)": [
        mape(test.values, naive_pred.values),
        mape(test.values, hw_pred.values),
        mape(test.values, arima_pred.values),
    ],
}).round(2)

metrics.set_index("Metode")
```

Metode	MAE	RMSE	MAPE (%)
Naive	974.97	1202.78	7.47
Holt-Winters	926.15	1138.11	7.03
ARIMA(1,1,1)	842.13	1050.34	6.26

Cara membaca:

- **MAE** paling mudah diinterpretasikan: “rata-rata, prediksi kita meleset X unit per hari”.
- **RMSE** lebih besar dari MAE jika ada kesalahan besar—karena ia mengkuadratkan kesalahan sebelum merata-ratakan.
- **MAPE** berguna ketika skala target berubah-ubah, tetapi **gagal** jika ada nilai nol atau mendekati nol.

> ⚠ Warning
>
> **MAPE bukan selalu tepat.** Jika data Anda memiliki nilai nol atau sangat kecil, MAPE bisa meledak menjadi tak hingga. Untuk data penjualan dengan hari-hari sepi (nilai mendekati 0), **MAE atau RMSE** lebih aman. Gunakan MAPE hanya jika semua nilai target jauh dari nol.

10.9 Rolling-Origin Cross-Validation

Pembagian train/test tunggal bisa menyesatkan—terutama jika test set jatuh pada periode yang tidak biasa (mis. libur panjang). **Rolling-origin cross-validation** mengatasi ini dengan **menggeser jendela pelatihan dan pengujian** ke depan secara berurutan.

```
fig, ax = plt.subplots(figsize=(9, 3.5))

n_windows = 4
window_size = 10
horizon = 3

for i in range(n_windows):
    start = i * window_size
    train_end = start + window_size
    test_end = train_end + horizon
    ax.barh(i, window_size, left=start, color="#4C72B0", edgecolor="white")
    ax.barh(i, horizon, left=train_end, color="#DD8452", edgecolor="white")
    ax.text(train_end + horizon + 0.3, i, f"iterasi {i+1}", va="center", fontsize=8)

ax.set_yticks(range(n_windows))
ax.set_yticklabels([f"Fold {i+1}" for i in range(n_windows)])
ax.set_xlabel("Waktu (indeks hari)")
ax.set_title("Rolling-Origin Cross-Validation")
ax.invert_yaxis()

# Legend manual
from matplotlib.patches import Patch
ax.legend(handles=[Patch(color="#4C72B0", label="Train"),
                   Patch(color="#DD8452", label="Test (horizon)")],
          loc="upper right")
plt.tight_layout()
plt.show()
```

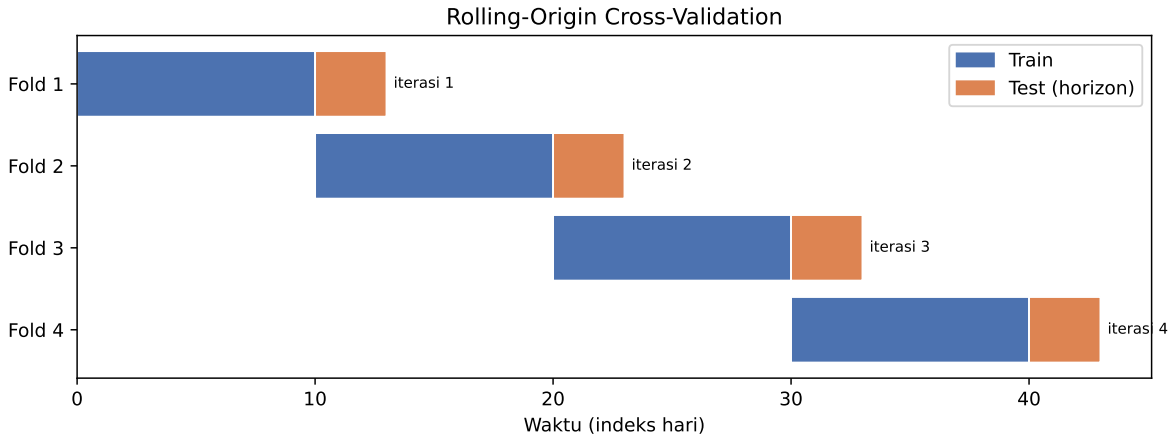


Figure 10.5: Ilustrasi rolling-origin cross-validation: model dilatih pada data hingga titik t, diuji pada h+1 hingga h+k, lalu jendela digeser ke depan.

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

def rolling_origin_cv(series, n_splits=5, horizon=7, seasonal_periods=7):
    """Rolling-origin cross-validation untuk Holt-Winters."""
    n = len(series)
    fold_size = (n - horizon) // n_splits
    maes = []

    for i in range(n_splits):
        train_end = (i + 1) * fold_size
        train_fold = series.iloc[:train_end]
        test_fold = series.iloc[train_end:train_end + horizon]

        if len(test_fold) < horizon:
            break

        try:
            model = ExponentialSmoothing(
                train_fold, trend="add", seasonal="add",
                seasonal_periods=seasonal_periods
            ).fit()
            pred = model.forecast(horizon)
            maes.append(mean_absolute_error(test_fold, pred))
        except Exception as e:
            print(f"Fold {i+1} gagal: {e}")
```

```
    return np.array(maes)

maes = rolling_origin_cv(ts, n_splits=5, horizon=7)
print(f"MAE per fold: {[f'{m:.2f}' for m in maes]}")
print(f"Rata-rata   : {maes.mean():.2f} ± {maes.std():.2f}")
```

```
MAE per fold: ['1095.49', '879.34', '837.36', '705.59', '1119.10']
Rata-rata   : 927.38 ± 157.88
```

> **!** Important
>
> **Mengapa rolling-origin lebih baik dari single split?** Ia menguji model pada **beberapa periode berbeda**, sehingga estimasi performa lebih stabil dan lebih representatif. Model yang baik harus konsisten di seluruh fold—jika MAE sangat bervariasi antar fold, model mungkin tidak stabil.

10.10 Analisis Survival: Kembali ke Telco

Sekarang kita kembali ke dataset Telco. Di Bab 9, model klasifikasi menjawab *"apakah pelanggan ini akan churn?"* — tetapi tidak menjawab *"kapan?"*.

Analisis survival menjawab pertanyaan kedua—dan memberi kita pemahaman yang lebih kaya tentang churn.

10.10.1 Konsep Dasar

Analisis survival memodelkan **waktu sampai kejadian** (dalam kasus kita: bulan sampai churn). Ada tiga konsep inti:

- **Survival function** $S(t)$: probabilitas bertahan lebih lama dari waktu t.
- **Hazard function** $h(t)$: probabilitas churn **pada** waktu t, **dengan asumsi** bertahan sampai t.
- **Censoring**: pelanggan yang belum churn sampai akhir pengamatan (mereka "disensor"—kita tahu mereka bertahan setidaknya selama tenure mereka, tetapi tidak tahu kapan mereka akan churn).

10.10.2 Membangun Kurva Survival

```
from decanal.utils import load_churn_clean

df_telco = load_churn_clean()

# Konvensi: event=1 jika churn, 0 jika disensor (belum churn)
# Duration = tenure (bulan)
df_surv = pd.DataFrame({
    "duration": df_telco["tenure"],
    "event": (df_telco["Churn"] == "Yes").astype(int),
    "contract": df_telco["Contract"],
})

print(f"Total pelanggan   : {len(df_surv):,}")
print(f"Churn (event=1)   : {df_surv['event'].sum():,} ({df_surv['event'].mean():.1%})")
print(f"Disensor (event=0): {(1 - df_surv['event']).sum():,}")
print(f"Median tenure     : {df_surv['duration'].median():.0f} bulan")
```

```
Total pelanggan   : 7,043
Churn (event=1)   : 1,869 (26.5%)
Disensor (event=0): 5,174
Median tenure     : 29 bulan
```

```
from lifelines import KaplanMeierFitter

fig, ax = plt.subplots(figsize=(9, 5))

for contract, color in [("Month-to-month", "#DD8452"),
                        ("One year", "#55A868"),
                        ("Two year", "#4C72B0")]:
    subset = df_surv[df_surv["contract"] == contract]
    kmf = KaplanMeierFitter()
    kmf.fit(subset["duration"], subset["event"], label=contract)
    kmf.plot_survival_function(ax=ax, color=color, ci_show=True, alpha=0.7)

ax.set_xlabel("Tenure (bulan)")
ax.set_ylabel("Probabilitas bertahan")
ax.set_title("Kurva Survival Kaplan-Meier per Tipe Kontrak")
ax.grid(True, alpha=0.3)
ax.legend(title="Kontrak")
```

```
plt.tight_layout()
plt.show()
```

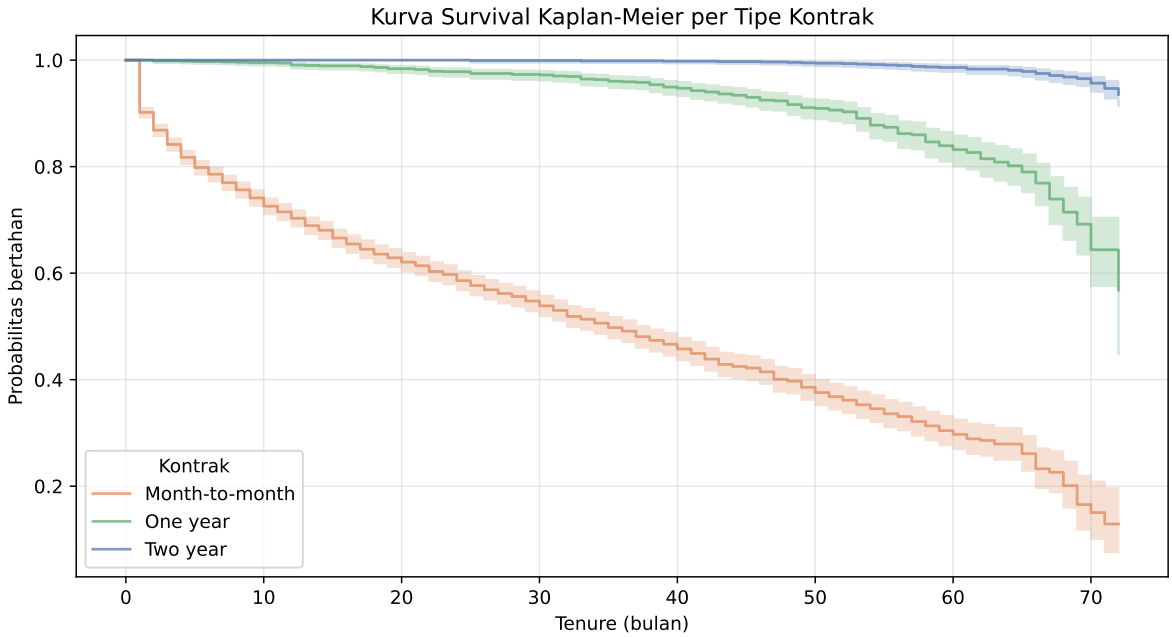


Figure 10.6: Kurva survival Kaplan-Meier per tipe kontrak. Kontrak bulanan jauh lebih cepat menurun daripada kontrak tahunan.

Cara membaca kurva survival:

- **Sumbu Y** = probabilitas masih bertahan pada waktu t.
- **Sumbu X** = waktu (bulan sejak bergabung).
- Kurva **turun tajam di awal** = banyak churn di awal (churn dini).
- Kurva **melandai** = pelanggan yang bertahan cukup lama cenderung bertahan lebih lama.

Perhatikan: kurva kontrak bulanan turun jauh lebih cepat daripada kontrak tahunan dan dua tahun. Ini **mengkonfirmasi** temuan Bagian I dengan cara yang lebih kaya—kita tidak hanya tahu **apakah** kontrak memengaruhi churn, tetapi juga **kapan** efeknya paling kuat.

> **!** Important
>
> **Censoring adalah kunci.** Pelanggan yang belum churn sampai akhir pengamatan bukan "tidak churn"—mereka hanya belum churn **sampai sekarang**. Analisis survival menangani mereka dengan benar, sedangkan analisis proporsi sederhana (churn rate)

mengabaikan dimensi waktu. Inilah mengapa analisis survival lebih informatif.

10.10.3 Uji Log-Rank: Apakah Kurva Berbeda Secara Signifikan?

```
from lifelines.statistics import logrank_test

# Bandingkan kontrak bulanan vs dua tahun
monthly = df_surv[df_surv["contract"] == "Month-to-month"]
two_year = df_surv[df_surv["contract"] == "Two year"]

result = logrank_test(
    monthly["duration"], two_year["duration"],
    event_observed_A=monthly["event"],
    event_observed_B=two_year["event"],
)

print(f"Uji log-rank - bulanan vs dua tahun:")
print(f"  Statistic: {result.test_statistic:.2f}")
print(f"  p-value  : {result.p_value:.2e}")
if result.p_value < 0.05:
    print("  → Kurva survival BERBEDA secara signifikan")
else:
    print("  → Tidak cukup bukti bahwa kurva berbeda")
```

```
Uji log-rank - bulanan vs dua tahun:
  Statistic: 1550.51
  p-value  : 0.00e+00
  → Kurva survival BERBEDA secara signifikan
```

10.10.4 Implikasi untuk Keputusan

Analisis survival memberi kita **parameter** yang lebih kaya daripada model klasifikasi saja:

- **Timing intervensi.** Hazard function yang tinggi di bulan-bulan awal menunjukkan bahwa intervensi retensi paling efektif dilakukan **segera** setelah onboarding.
- **Ekspektasi durasi.** Median survival per segmen memberi tahu kita **berapa lama** pelanggan rata-rata bertahan, yang memengaruhi perhitungan Lifetime Value (LTV).

- **Prioritas segmen.** Segmen dengan hazard tinggi di awal membutuhkan intervensi cepat; segmen dengan hazard rendah di awal tetapi naik belakangan membutuhkan intervensi berbeda.

> **Note**
>
> **Analisis survival formal** dengan model Cox Proportional Hazards—yang menguji efek multivariabel terhadap hazard—akan sangat berguna untuk keputusan retensi. Namun, ia membutuhkan asumsi *proportional hazards* yang tidak selalu terpenuhi. Untuk buku ini, kita cukup menggunakan Kaplan-Meier dan log-rank test sebagai pengantar. Pembaca yang tertarik dapat mengeksplorasi `lifelines.CoxPHFitter` secara mandiri.

10.11 Menghubungkan Kembali ke Dataset Retail

Sekarang kita memiliki dua kerangka time series: - **Forecasting** (retail): memprediksi nilai kontinu di masa depan. - **Survival analysis** (Telco): memodelkan waktu sampai kejadian.

Keduanya akan menjadi fondasi untuk **simulasi Monte Carlo** di Bab 12—di mana kita akan menggunakan distribusi probabilitas dari data historis untuk mensimulasikan skenario masa depan.

10.12 Latihan Terbimbing

Latihan 10.1 — Dekomposisi multiplikatif. Gunakan `seasonal_decompose` dengan `model="multiplicative"` pada `daily["total_sold"]`. Bandingkan plot residual aditif dan multiplikatif. Mana yang residualnya lebih "seperti noise"? Mengapa?

Latihan 10.2 — Agregasi mingguan. Alih-alih harian, agregasi data retail ke **mingguan** (`resample("W").sum()`). Lakukan dekomposisi dengan `period=52`. Apakah pola musiman tahunan terlihat? Bandingkan dengan dekomposisi harian.

Latihan 10.3 — ARIMA tuning. Coba beberapa kombinasi `(p, d, q)` untuk ARIMA—misalnya `(1,1,1)`, `(2,1,1)`, `(1,1,2)`, `(2,1,2)`. Bandingkan MAE pada test set. Model mana yang terbaik? Berapa lama waktu fitting untuk masing-masing?

Latihan 10.4 — Rolling-origin untuk ARIMA. Modifikasi fungsi `rolling_origin_cv` agar menggunakan ARIMA, bukan Holt-Winters. Bandingkan MAE rata-rata antara ARIMA dan Holt-Winters pada 5 fold. Metode mana yang lebih stabil?

Latihan 10.5 — Survival per segmen. Buat kurva Kaplan-Meier yang dipisah berdasarkan `InternetService` (bukan `Contract`). Apakah ada perbedaan survival antara pelanggan fiber optic, DSL, dan tanpa internet? Gunakan log-rank test untuk menguji signifikansi.

Latihan 10.6 — Hazard function. Plot **hazard function** (bukan survival function) dari `KaplanMeierFitter` menggunakan `.cumulative_density_`. Pada bulan berapa hazard paling tinggi? Apa implikasinya untuk strategi retensi?

> **Note**
>
> **Petunjuk Latihan 10.3.** ARIMA dengan order lebih tinggi **tidak selalu** lebih baik. Model dengan terlalu banyak parameter bisa overfitting—sama seperti di Bab 7. Perhatikan juga waktu fitting: ARIMA(2,1,2) jauh lebih lambat dari ARIMA(1,1,1). Gunakan **AIC** (dari `model.aic`) sebagai panduan awal, tetapi validasi dengan test MAE.

10.13 Kotak "Jembatan": Dari Prediksi ke Simulasi

> **Important**
>
> **Jembatan.** Bab ini memberi kita dua kemampuan baru:
>
> 1. **Forecasting**—memprediksi nilai kontinu di masa depan (retail). Ini akan menjadi input untuk **inventory optimization** di Bab 16 dan simulasi keputusan di Bab 17.
> 2. **Survival analysis**—memodelkan **kapan** kejadian terjadi (Telco). Ini memberi kita **distribusi waktu** churn, yang dapat digunakan untuk:
>
>   - Menghitung ekspektasi revenue loss per segmen.
>   - Menentukan timing intervensi retensi.
>   - Memasukkan **ketidakpastian waktu** ke dalam model optimasi di Bagian III.
>
> Tetapi forecasting memberi kita **satu titik prediksi**—bukan **distribusi**. Padahal, keputusan nyata selalu menghadapi ketidakpastian: - Permintaan bisa lebih tinggi atau lebih rendah dari forecast. - Churn bisa terjadi lebih cepat atau lebih lambat.
> **Simulasi Monte Carlo**—topik Bab 12—akan menjembatani gap ini: ia menggunakan distribusi dari data historis untuk menghasilkan **ribuan skenario** masa depan, sehingga kita dapat mengevaluasi keputusan berdasarkan **distribusi hasil**, bukan hanya satu titik prediksi.
> Bab 11 akan terlebih dahulu memperkenalkan **unsupervised learning**—clustering dan PCA—untuk menemukan **segmen tersembunyi** dalam data. Ini adalah jembatan terakhir sebelum Bagian III: setelah kita dapat memprediksi dan mengelompokkan, kita akan mensimulasikan dan mengoptimalkan.

10.14 Rangkuman

Dalam bab ini kita telah:

- **Beralih dari dataset Telco (cross-sectional) ke dataset retail (time series)**, memahami bahwa setiap dataset memiliki pertanyaan yang cocok—dan buku ini menggunakan strategi dua lapis untuk menutupi seluruh spektrum analytics.
- **Membedakan data cross-sectional dari time series**, dan memahami bahwa time series membutuhkan **chronological split**, bukan random split.
- **Mendekomposisi time series** menjadi tren, musiman, dan residual—memisahkan sinyal dari noise.
- **Menguji stasioneritas** dengan ADF test dan melakukan differencing jika diperlukan.
- **Membangun tiga model forecasting**—naive, Holt-Winters, ARIMA—dan membandingkannya dengan MAE, RMSE, dan MAPE.
- **Menerapkan rolling-origin cross-validation** untuk estimasi performa yang lebih stabil dan representatif.
- **Menerapkan analisis survival** pada dataset Telco, memahami perbedaan survival function, hazard function, dan censoring.
- **Menghubungkan survival analysis ke keputusan**—timing intervensi, ekspektasi durasi, prioritas segmen.

10.15 Jembatan ke Bab 11

Sampai di sini, kita telah memprediksi **nilai** (Bab 8), **kelas** (Bab 9), dan **waktu** (Bab 10). Tetapi kita belum mengeksplorasi **struktur** data itu sendiri—apakah ada kelompok alami di dalamnya yang tidak kita ketahui?

Unsupervised learning—topik Bab 11—akan mengajarkan kita untuk menemukan **segmen tersembunyi** dalam data tanpa label:

- **Clustering** (K-means, hierarchical): mengelompokkan pelanggan berdasarkan kemiripan profil.
- **PCA** (Principal Component Analysis): mereduksi dimensi untuk visualisasi dan pemodelan.
- **Association rules**: menemukan pola "jika pelanggan membeli X, mereka juga membeli Y".

Bab 11 akan menjadi **jembatan terakhir** sebelum Bagian III. Setelah kita dapat memprediksi dan mengelompokkan, kita akan mensimulasikan dan mengoptimalkan—itulah inti dari prescriptive analytics.

11 Unsupervised Learning: Menemukan Struktur Tersembunyi

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membedakan supervised dari unsupervised learning, dan memahami kapan masing-masing tepat digunakan. 2. Menerapkan **K-means clustering** dan memilih jumlah cluster secara sistematis. 3. Menerapkan **hierarchical clustering** dan membaca **dendrogram**. 4. Menggunakan **PCA** untuk mereduksi dimensi dan memvisualisasikan data berdimensi tinggi. 5. Menemukan **association rules** dalam data transaksi (market basket analysis). 6. Menginterpretasikan segmen pelanggan secara bisnis dan menghubungkannya ke keputusan retensi. 7. Memahami mengapa segmentasi adalah **input penting** untuk model optimasi di Bagian III.

11.1 Mengapa Unsupervised Learning?

Sampai Bab 10, semua model kita adalah **supervised**—kita memiliki label (churn/tidak, unit terjual) dan model belajar memprediksi label tersebut. Tetapi tidak semua pertanyaan analytics memiliki label:

- *"Apakah ada kelompok pelanggan alami yang tidak kita ketahui?"*
- *"Fitur mana yang paling penting dalam menjelaskan variasi data?"*
- *"Produk apa yang sering dibeli bersama?"*

Pertanyaan-pertanyaan ini tidak memiliki "jawaban benar" yang dapat dilatih. Kita tidak tahu berapa banyak segmen yang ada, atau segmen mana yang "benar"—kita ingin **menemukan struktur** dari data itu sendiri.

Inilah domain **unsupervised learning**: menemukan pola dalam data **tanpa label**.

Important

Tiga kegunaan utama unsupervised learning dalam decision analytics:

1. **Segmentasi**—mengelompokkan pelanggan/produk ke dalam segmen yang bermakna secara bisnis. Ini adalah input langsung untuk **personalisasi** dan

alokasi anggaran di Bagian III.

2. **Reduksi dimensi**—menyederhanakan data berdimensi tinggi agar dapat divisualisasikan dan dimodelkan. PCA adalah alat utama untuk ini.

3. **Penemuan pola**—menemukan asosiasi antar item (association rules) yang dapat digunakan untuk **cross-selling**, **bundling**, dan **rekomendasi**.

11.2 Menyiapkan Data

Kita kembali ke dataset Telco, tetapi kali ini **tanpa label churn**—kita ingin menemukan segmen pelanggan berdasarkan profil mereka.

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from decanal.utils import load_churn_clean, load_retail
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, silhouette_samples

RANDOM_STATE = 42
df = load_churn_clean()

print(f"Dataset: {df.shape[0]:,} pelanggan × {df.shape[1]} kolom")
```

```
Dataset: 7,043 pelanggan × 23 kolom
```

Untuk clustering, kita fokus pada fitur **numerik yang bermakna secara bisnis**—tenure, tagihan bulanan, total tagihan, dan jumlah layanan. Fitur kategorikal akan kita tangani terpisah atau melalui encoding.

```python
# Fitur numerik untuk clustering
cluster_features = ["tenure", "MonthlyCharges", "TotalCharges", "num_services"]

# Periksa apakah kolom tersedia
available = [c for c in cluster_features if c in df.columns]
print(f"Fitur yang tersedia: {available}")

X_cluster = df[available].copy()
X_cluster.describe().round(2)
```

```
Fitur yang tersedia: ['tenure', 'MonthlyCharges', 'TotalCharges', 'num_services']
```

	tenure	MonthlyCharges	TotalCharges	num_services
count	7043.00	7043.00	7043.00	7043.00
mean	32.37	64.76	2279.73	3.36
std	24.56	30.09	2266.79	2.06
min	0.00	18.25	0.00	0.00
25%	9.00	35.50	398.55	1.00
50%	29.00	70.35	1394.55	3.00
75%	55.00	89.85	3786.60	5.00
max	72.00	118.75	8684.80	8.00

```python
# Standardisasi wajib untuk clustering berbasis jarak (K-means, hierarchical)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cluster)

# Simpan sebagai DataFrame untuk kenyamanan
X_scaled_df = pd.DataFrame(X_scaled, columns=available)
X_scaled_df.head()
```

	tenure	MonthlyCharges	TotalCharges	num_services
0	-1.277445	-1.160323	-0.992611	-1.145997
1	0.066327	-0.259629	-0.172165	-0.176011
2	-1.236724	-0.362660	-0.958066	-0.176011
3	0.514251	-0.746535	-0.193672	-0.176011
4	-1.236724	0.197365	-0.938874	-1.145997

> ⚠ Warning
>
> **Standardisasi adalah wajib untuk clustering.** Tanpa standardisasi, fitur dengan skala besar (`TotalCharges` dalam ribuan) akan mendominasi perhitungan jarak, membuat fitur dengan skala kecil (`num_services`, 0–8) hampir tidak berpengaruh. Ini adalah kesalahan paling umum dalam clustering—dan konsekuensinya adalah segmen yang **tidak bermakna**.

11.3 K-Means Clustering

K-means adalah algoritma clustering paling populer. Ia bekerja dengan: 1. Memilih **K** titik pusat (centroid) secara acak. 2. Menetapkan setiap titik ke centroid terdekat. 3. Memperbarui centroid sebagai rata-rata titik dalam cluster. 4. Mengulangi langkah 2–3 sampai konvergen.

11.3.1 Memilih Jumlah Cluster (K)

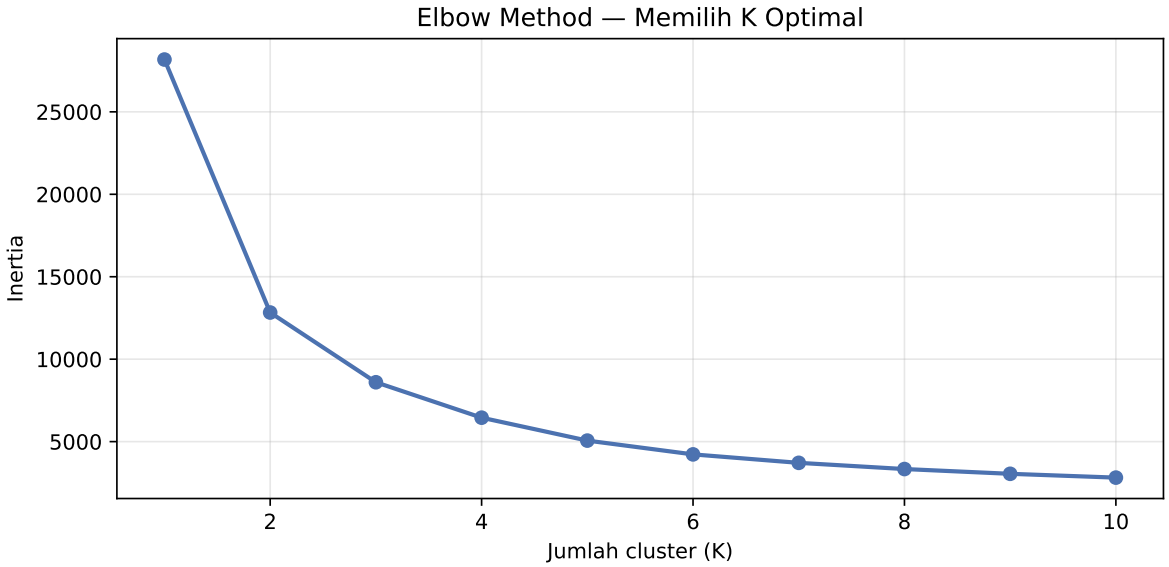
Pertanyaan tersulit dalam K-means adalah: **berapa K yang tepat?** Ada dua metode yang paling umum:

Elbow Method: plot **inertia** (jumlah kuadrat jarak ke centroid) vs K. Titik di mana penurunan mulai "melandai" adalah K yang disarankan.

```
inertias = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(list(K_range), inertias, marker="o", color="#4C72B0", linewidth=2)
ax.set_xlabel("Jumlah cluster (K)")
ax.set_ylabel("Inertia")
ax.set_title("Elbow Method - Memilih K Optimal")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



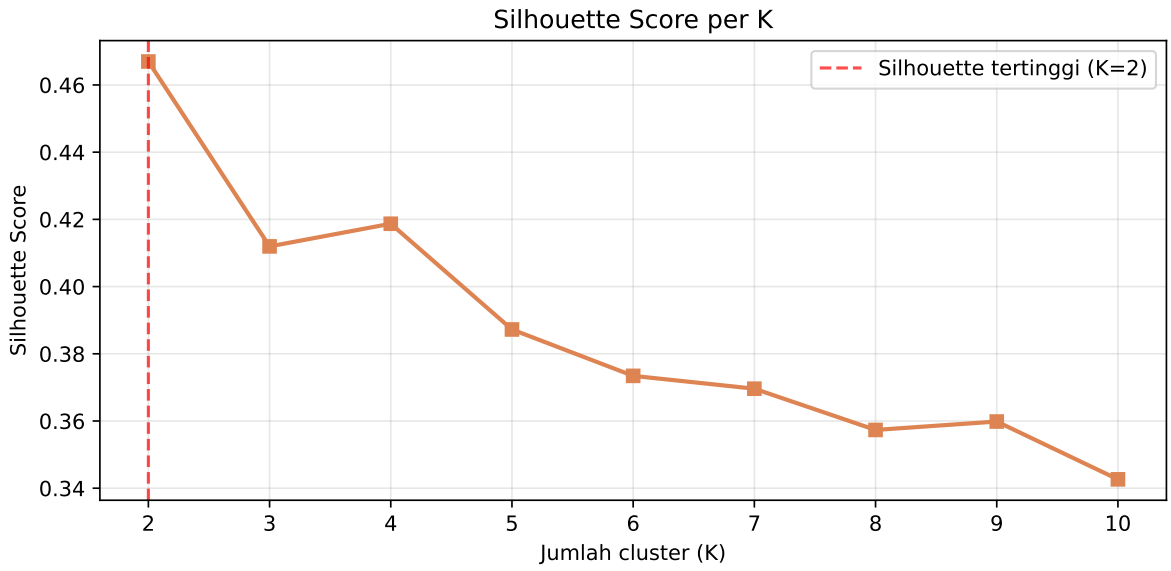
Silhouette Score: mengukur seberapa baik setiap titik cocok dengan clusternya sendiri dibanding cluster lain. Nilai berkisar -1 hingga 1; semakin tinggi semakin baik.

```
silhouettes = []
K_range_sil = range(2, 11)

for k in K_range_sil:
    kmeans = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    silhouettes.append(silhouette_score(X_scaled, labels))

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(list(K_range_sil), silhouettes, marker="s", color="#DD8452", linewidth=2)
best_k = list(K_range_sil)[np.argmax(silhouettes)]
ax.axvline(best_k, color="red", linestyle="--", alpha=0.7,
           label=f"Silhouette tertinggi (K={best_k})")
ax.set_xlabel("Jumlah cluster (K)")
ax.set_ylabel("Silhouette Score")
ax.set_title("Silhouette Score per K")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Silhouette tertinggi: K = {best_k} ({max(silhouettes):.3f})")
```




```
Silhouette tertinggi: K = 2 (0.467)
```

> **Tip**
>
> **Elbow vs Silhouette.** Keduanya adalah **panduan**, bukan aturan mutlak. Dalam praktik: - **Elbow** lebih mudah diinterpretasikan tetapi subjektif (di mana "siku"-nya?). - **Silhouette** memberi angka objektif, tetapi bisa memilih K yang secara bisnis tidak bermakna.
> **Yang terpenting:** pilih K yang **dapat diinterpretasikan secara bisnis**. Jika K=3 memberi segmen "baru, menengah, lama" yang jelas, itu lebih berguna daripada K=7 dengan segmen yang sulit dijelaskan—bahkan jika K=7 memiliki silhouette sedikit lebih tinggi.

11.3.2 Melatih K-Means dengan K Optimal

Berdasarkan kedua metode, kita pilih K=3 (atau sesuaikan dengan hasil render Anda):

```
K_OPT = 3  # sesuaikan setelah melihat elbow & silhouette

kmeans = KMeans(n_clusters=K_OPT, random_state=RANDOM_STATE, n_init=10)
df["segment"] = kmeans.fit_predict(X_scaled)

# Profil setiap segmen
```

```
profil = df.groupby("segment")[available + ["Churn"]].agg(
    {**{c: "mean" for c in available}, "Churn": lambda s: (s == "Yes").mean() * 100}
).round(2)
profil["jumlah"] = df.groupby("segment").size()
profil = profil[["jumlah"] + available + ["Churn"]]
profil.columns = ["jumlah", "tenure", "MonthlyCharges", "TotalCharges", "num_services", "chu
profil.sort_values("churn_rate (%)", ascending=False)
```

segment	jumlah	tenure	MonthlyCharges	TotalCharges	num_services	churn_rate (%)
2	2622	15.00	75.83	1137.24	3.24	46.15
1	2163	58.37	90.52	5270.62	5.63	15.03
0	2258	27.64	27.24	741.35	1.33	14.79

Cara membaca profil segmen:

- **Segmen 1** mungkin adalah pelanggan baru dengan tagihan tinggi dan churn rate tinggi—**prioritas retensi**.
- **Segmen 2** mungkin adalah pelanggan setia dengan tenure panjang, tagihan moderat, dan churn rate rendah—**target upsell**.
- **Segmen 3** mungkin adalah pelanggan berisiko menengah—**target program engagement**.

11.3.3 Visualisasi Segmen

```
fig, ax = plt.subplots(figsize=(9, 5))
palette = ["#4C72B0", "#DD8452", "#55A868", "#C44E52", "#8172B3"]

for seg in sorted(df["segment"].unique()):
    subset = df[df["segment"] == seg]
    ax.scatter(subset["tenure"], subset["MonthlyCharges"],
               c=palette[seg], label=f"Segmen {seg}",
               alpha=0.5, s=subset["num_services"] * 5)

# Plot centroid
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
for i, c in enumerate(centroids):
    ax.scatter(c[0], c[1], c="black", marker="X", s=200,
               edgecolors="white", linewidths=2)
```

```
    ax.annotate(f"C{i}", (c[0], c[1]), fontsize=12, fontweight="bold",
                ha="center", va="center", color="white")

ax.set_xlabel("Tenure (bulan)")
ax.set_ylabel("Tagihan bulanan")
ax.set_title("Segmentasi Pelanggan (K-Means, K=3)")
ax.legend(title="Segmen")
plt.tight_layout()
plt.show()
```

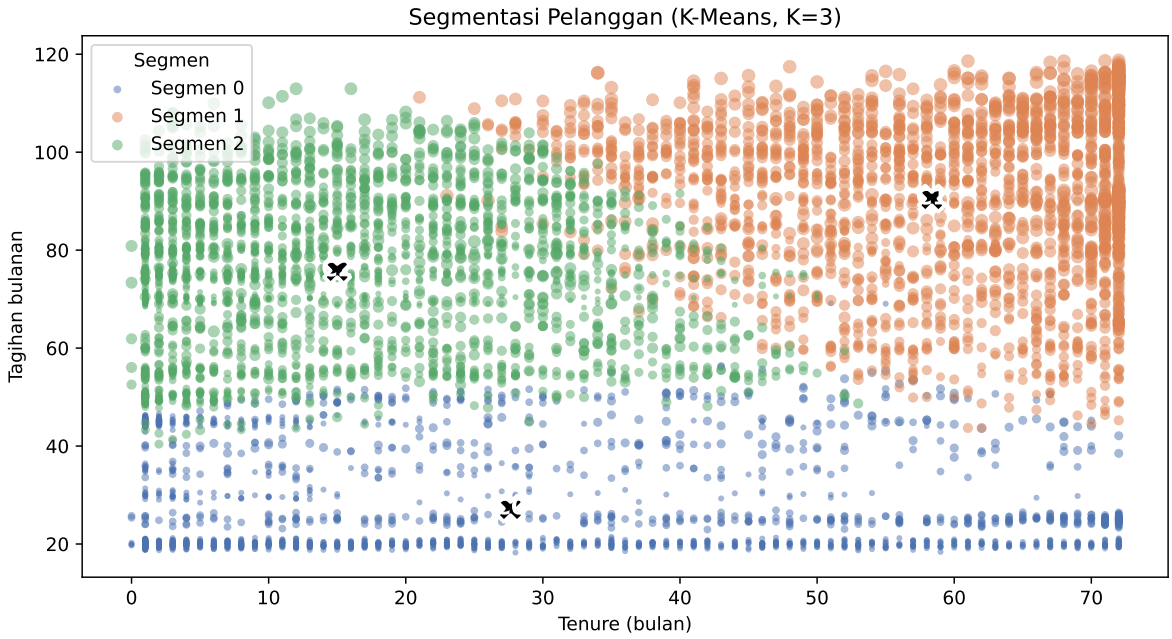


Figure 11.1: Segmentasi pelanggan berdasarkan tenure dan MonthlyCharges. Warna menunjukkan segmen; ukuran titik proporsional terhadap jumlah layanan.

> **!** Important
>
> **Interpretasi bisnis adalah kunci.** Cluster yang dihasilkan K-means **tidak memiliki makna inheren**—"Segmen 0" bukanlah label yang berguna. Tugas Anda adalah **memberi nama** segmen berdasarkan profilnya:

Segmen	Profil	Nama bisnis	Strategi
0	Tenure rendah, tagihan tinggi, banyak layanan	“Premium Baru”	Onboarding intensif, diskon perpanjangan
1	Tenure tinggi, tagihan rendah, sedikit layanan	“Loyal Hemat”	Upsell layanan tambahan
2	Tenure menengah, tagihan menengah	“Mainstream”	Program engagement rutin

Nama-nama ini akan digunakan di seluruh Bagian III untuk merujuk segmen dalam model optimasi.

11.4 Hierarchical Clustering

Hierarchical clustering membangun **pohon** (dendrogram) yang menunjukkan bagaimana cluster bergabung pada berbagai tingkat jarak. Berbeda dari K-means, ia **tidak memerlukan K di awal**—K dapat ditentukan setelah melihat dendrogram.

11.4.1 Membangun Dendrogram

```
from scipy.cluster.hierarchy import dendrogram, linkage

# Ambil sampel 500 pelanggan agar dendrogram terbaca
sample_idx = np.random.RandomState(RANDOM_STATE).choice(len(X_scaled), 500, replace=False)
X_sample = X_scaled[sample_idx]

Z = linkage(X_sample, method="ward")

fig, ax = plt.subplots(figsize=(10, 5))
dendrogram(Z, ax=ax, truncate_mode="lastp", p=20,
           leaf_rotation=90, leaf_font_size=8, show_contracted=True)
ax.set_xlabel("Indeks sampel atau ukuran cluster")
ax.set_ylabel("Jarak penggabungan (Ward)")
ax.set_title("Dendrogram - Hierarchical Clustering (sampel 500 pelanggan)")
plt.tight_layout()
plt.show()
```

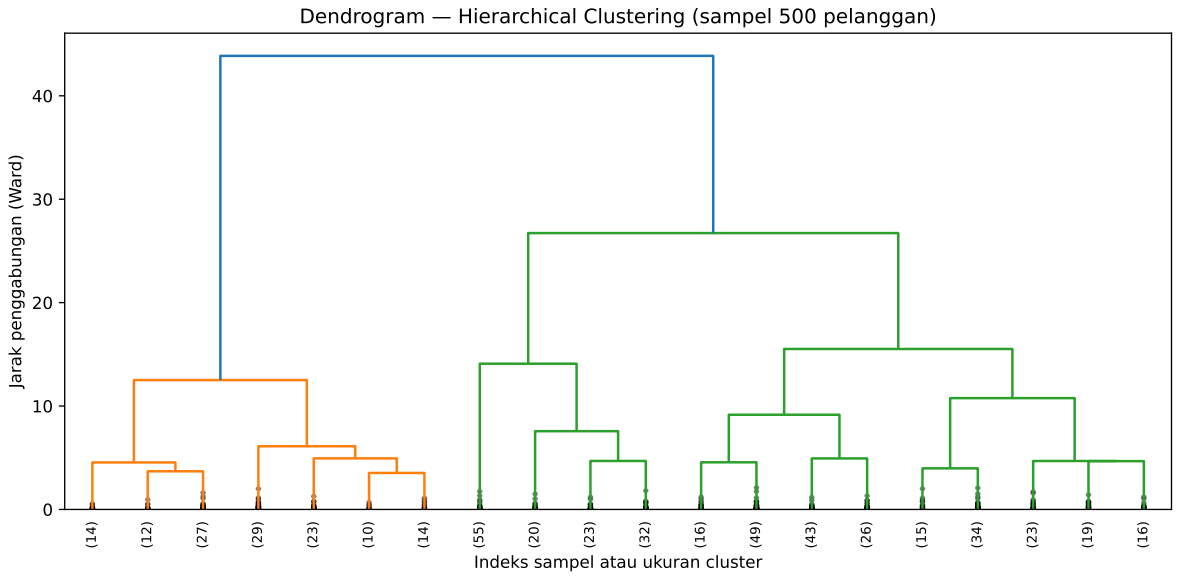


Figure 11.2: Dendrogram hierarchical clustering. Garis horizontal menunjukkan jarak penggabungan; semakin tinggi, semakin berbeda cluster yang digabung.

Cara membaca dendrogram:

- Setiap daun (di bawah) adalah satu pelanggan atau cluster kecil.
- Garis horizontal menunjukkan **pada jarak berapa** dua cluster digabung.
- **Semakin tinggi** garis horizontal, semakin **berbeda** cluster yang digabung.
- Untuk memilih K: gambar garis horizontal imajiner melintasi dendrogram. Jumlah garis vertikal yang terpotong = jumlah cluster.

11.4.2 Membandingkan dengan K-Means

```
from sklearn.metrics import adjusted_rand_score

# Hierarchical dengan K yang sama
hc = AgglomerativeClustering(n_clusters=K_OPT, linkage="ward")
df["segment_hc"] = hc.fit_predict(X_scaled)

# Bandingkan dengan K-means
ari = adjusted_rand_score(df["segment"], df["segment_hc"])
print(f"Adjusted Rand Index (K-means vs Hierarchical): {ari:.3f}")
print(f"  (1.0 = identik, 0.0 = acak)")
```

```python
# Profil segmen hierarchical
profil_hc = df.groupby("segment_hc")[available + ["Churn"]].agg(
    {**{c: "mean" for c in available}, "Churn": lambda s: (s == "Yes").mean() * 100}
).round(2)
profil_hc["jumlah"] = df.groupby("segment_hc").size()
profil_hc
```

```
Adjusted Rand Index (K-means vs Hierarchical): 0.436
  (1.0 = identik, 0.0 = acak)
```

segment_hc	tenure	MonthlyCharges	TotalCharges	num_services	Churn	jumlah
0	20.22	38.37	586.19	1.50	26.15	3097
1	29.87	79.42	2235.12	3.93	34.94	2430
2	61.21	95.19	5810.94	6.26	13.85	1516

Tip

Kapan memakai hierarchical vs K-means?

Aspek	K-Means	Hierarchical
K	Harus ditentukan di awal	Dapat ditentukan setelah melihat dendrogram
Skalabilitas	Cepat untuk data besar ($O(n \cdot k \cdot d)$)	Lambat untuk data besar ($O(n^2)$)
Bentuk cluster	Cenderung bulat	Dapat menangkap bentuk kompleks
Interpretabilitas	Centroid mudah diinterpretasikan	Dendrogram memberi gambaran hierarkis
Penggunaan	Segmentasi operasional	Eksplorasi struktur data

Untuk dataset besar (>10.000 baris), K-means biasanya lebih praktis. Untuk eksplorasi awal atau dataset kecil, hierarchical memberi gambaran yang lebih kaya.

11.5 PCA: Mereduksi Dimensi

Principal Component Analysis (PCA) adalah teknik **reduksi dimensi**—ia mengubah banyak fitur yang berkorelasi menjadi **komponen utama** yang saling independen, diurutkan berdasarkan **varians** yang dijelaskan.

11.5.1 Mengapa PCA?

- **Visualisasi**: data dengan 20 fitur tidak dapat divisualisasikan; data dengan 2 komponen utama bisa.
- **Reduksi noise**: komponen dengan varians rendah sering kali hanya noise.
- **Multikolinearitas**: PCA menghasilkan fitur yang saling independen—berguna untuk regresi.

11.5.2 Menerapkan PCA

```
pca = PCA(random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_scaled)

# Varians yang dijelaskan
explained = pca.explained_variance_ratio_ * 100
cumulative = np.cumsum(explained)

pca_df = pd.DataFrame({
    "Komponen": [f"PC{i+1}" for i in range(len(explained))],
    "Varians (%)": explained.round(2),
    "Kumulatif (%)": cumulative.round(2),
})
pca_df
```

	Komponen	Varians (%)	Kumulatif (%)
0	PC1	73.79	73.79
1	PC2	20.93	94.71
2	PC3	3.80	98.52
3	PC4	1.48	100.00

```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.5))

# Scree plot
axes[0].bar(range(1, len(explained)+1), explained, color="#4C72B0")
axes[0].set_xlabel("Komponen utama")
axes[0].set_ylabel("Varians dijelaskan (%)")
axes[0].set_title("Scree Plot")

# Kumulatif
axes[1].plot(range(1, len(cumulative)+1), cumulative, marker="o", color="#DD8452")
axes[1].axhline(80, color="gray", linestyle="--", alpha=0.5, label="80%")
axes[1].axhline(90, color="red", linestyle=":", alpha=0.5, label="90%")
axes[1].set_xlabel("Jumlah komponen")
axes[1].set_ylabel("Varians kumulatif (%)")
axes[1].set_title("Varians Kumulatif")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

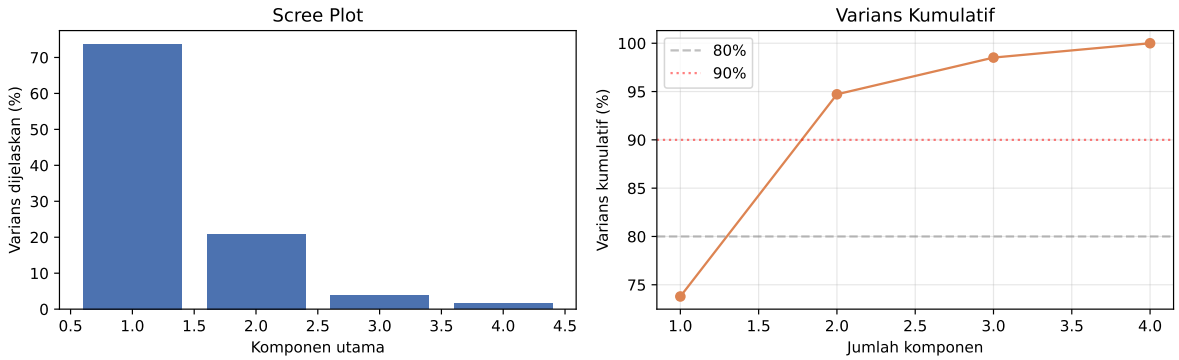


Figure 11.3: Scree plot: varians yang dijelaskan per komponen. Titik 'siku' menunjukkan jumlah komponen yang perlu dipertahankan.

Cara membaca:

- **Scree plot** menunjukkan kontribusi setiap komponen. "Siku" (perubahan tajam) menunjukkan jumlah komponen yang wajar.
- **Varians kumulatif** menunjukkan berapa banyak informasi yang dipertahankan. Aturan umum: pertahankan komponen yang menjelaskan **80–90%** varians.

- Untuk dataset kita (4 fitur numerik), 2 komponen biasanya menjelaskan >85% varians—cukup untuk visualisasi.

11.5.3 Visualisasi 2D dengan PCA

```
fig, ax = plt.subplots(figsize=(9, 5))

for seg in sorted(df["segment"].unique()):
    mask = df["segment"] == seg
    ax.scatter(X_pca[mask, 0], X_pca[mask, 1],
               c=palette[seg], label=f"Segmen {seg}",
               alpha=0.5, s=10)

ax.set_xlabel(f"PC1 ({explained[0]:.1f}% varians)")
ax.set_ylabel(f"PC2 ({explained[1]:.1f}% varians)")
ax.set_title("Visualisasi PCA 2D - Segmen Pelanggan")
ax.legend(title="Segmen")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

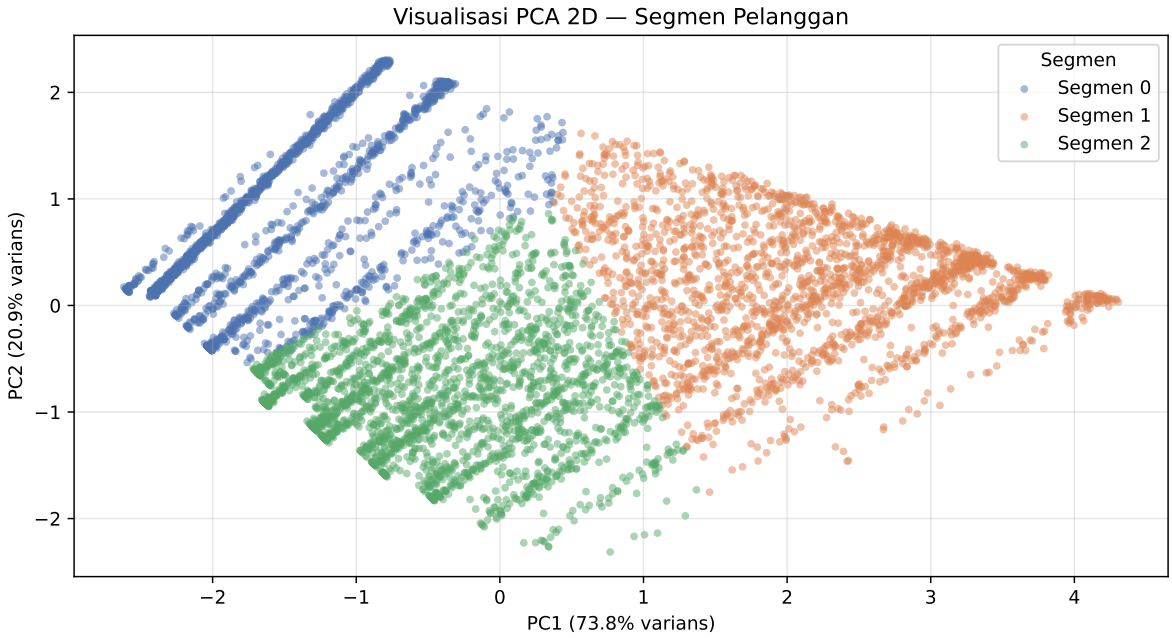


Figure 11.4: Visualisasi data dalam 2 komponen utama, diwarnai berdasarkan segmen K-means. PCA memungkinkan kita melihat struktur cluster dalam 2D.

! Important

PCA dan interpretasi. Komponen utama **tidak memiliki makna langsung**—mereka adalah kombinasi linear dari fitur asli. Namun, kita dapat melihat **loading** (bobot fitur dalam setiap komponen) untuk memahami apa yang diwakili komponen tersebut:

```
loadings = pd.DataFrame(
    pca.components_[:2].T,
    index=available,
    columns=["PC1", "PC2"]
)
```

Jika PC1 memiliki loading tinggi pada `tenure` dan `TotalCharges`, kita bisa menyebutnya "komponen loyalitas". Jika PC2 memiliki loading tinggi pada `MonthlyCharges`, itu mungkin "komponen biaya".

```
loadings = pd.DataFrame(
    pca.components_[:2].T,
    index=available,
    columns=["PC1", "PC2"]
```

```
).round(3)
loadings
```

	PC1	PC2
tenure	0.438	0.704
MonthlyCharges	0.461	-0.621
TotalCharges	0.557	0.217
num_services	0.534	-0.268

11.6 Association Rules: Market Basket Analysis

Sekarang kita beralih ke dataset telco—dan ke jenis analisis yang sama sekali berbeda: menemukan **asosiasi antar item** dalam transaksi.

Association rules menjawab pertanyaan: *"Jika pelanggan membeli produk A, seberapa besar kemungkinan mereka juga membeli produk B?"*

11.6.1 Konsep Dasar

- **Itemset**: kumpulan item (mis. {Roti, Susu}).
- **Support**: proporsi transaksi yang mengandung itemset.
- **Confidence**: probabilitas membeli B **jika** membeli A.
- **Lift**: seberapa besar asosiasi A→B dibandingkan jika keduanya independen.

$$\text{Support}(A \to B) = P(A \cap B)$$

$$\text{Confidence}(A \to B) = P(B|A)$$

$$\text{Lift}(A \to B) = \frac{P(B|A)}{P(B)}$$

- **Lift > 1**: A dan B lebih sering muncul bersama daripada yang diharapkan secara kebetulan.
- **Lift $= 1$**: A dan B independen.
- **Lift < 1**: A dan B cenderung **tidak** muncul bersama.

11.6.2 Menyiapkan Data Transaksi

Dataset retail tidak memiliki struktur transaksi (satu baris per pembelian). Tetapi kita dapat **mensimulasikan** transaksi dengan mengelompokkan produk yang terjual pada tanggal dan toko yang sama:

```
df_retail = load_retail()
df_retail["Date"] = pd.to_datetime(df_retail["Date"])

# Anggap setiap (Date, Store ID) adalah satu "transaksi"
# Item = Product ID yang terjual (Units Sold > 0)
transactions = (df_retail[df_retail["Units Sold"] > 0]
                .groupby(["Date", "Store ID"])["Product ID"]
                .apply(lambda x: sorted(set(x)))
                .reset_index())
transactions.columns = ["Date", "Store ID", "items"]

print(f"Jumlah transaksi: {len(transactions):,}")
print(f"Rata-rata item per transaksi: {transactions['items'].apply(len).mean():.1f}")
transactions.head()
```

```
Jumlah transaksi: 3,655
Rata-rata item per transaksi: 19.9
```

	Date	Store ID	items
0	2022-01-01	S001	[P0001, P0002, P0003, P0004, P0005, P0006, P00...
1	2022-01-01	S002	[P0001, P0002, P0003, P0004, P0005, P0006, P00...
2	2022-01-01	S003	[P0001, P0002, P0003, P0004, P0005, P0006, P00...
3	2022-01-01	S004	[P0001, P0002, P0003, P0005, P0006, P0007, P00...
4	2022-01-01	S005	[P0001, P0002, P0003, P0004, P0005, P0006, P00...

11.6.3 Menerapkan Apriori

Library `mlxtend` menyediakan implementasi Apriori yang efisien:

```
# from mlxtend.preprocessing import TransactionEncoder
# from mlxtend.frequent_patterns import fpgrowth, association_rules

# # Encode transaksi ke format one-hot
```

```
# te = TransactionEncoder()
# te_array = te.fit(transactions["items"]).transform(transactions["items"])
# basket_df = pd.DataFrame(te_array, columns=te.columns_)

# # Cari frequent itemsets dengan support minimal 10%
# frequent = fpgrowth(basket_df, min_support=0.1, max_len=3, use_colnames=True)
# print(f"Jumlah frequent itemsets: {len(frequent)}")

# # Bangkitkan association rules
# rules = association_rules(frequent, metric="lift", min_threshold=1.2)
# rules = rules.sort_values("lift", ascending=False)

# print(f"Jumlah rules (lift > 1.2): {len(rules)}")
# rules[["antecedents", "consequents", "support", "confidence", "lift"]].head(10).round(3)

from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules
from decanal.utils import load_churn_clean

# --- Bangun keranjang dari layanan Telco ---
df_telco = load_churn_clean()

service_cols = [
    "PhoneService", "MultipleLines", "OnlineSecurity", "OnlineBackup",
    "DeviceProtection", "TechSupport", "StreamingTV", "StreamingMovies",
]

def get_services(row):
    """Kembalikan daftar layanan yang dilanggani pelanggan."""
    services = [col for col in service_cols if row[col] == "Yes"]
    if row["InternetService"] != "No":
        services.append(f"Internet_{row['InternetService']}")
    return services

df_telco["services"] = df_telco.apply(get_services, axis=1)

# Hanya pelanggan dengan >= 2 layanan (keranjang bermakna)
baskets = df_telco[df_telco["services"].apply(len) >= 2]["services"].tolist()

print(f"Jumlah keranjang (pelanggan): {len(baskets):,}")
print(f"Rata-rata layanan per pelanggan: {sum(len(b) for b in baskets)/len(baskets):.1f}")
print(f"Rentang: {min(len(b) for b in baskets)}-{max(len(b) for b in baskets)} layanan")
```

```python
# --- Encoding ---
te = TransactionEncoder()
te_array = te.fit(baskets).transform(baskets)
basket_df = pd.DataFrame(te_array, columns=te.columns_)

# --- FP-Growth ---
frequent = fpgrowth(basket_df, min_support=0.05, use_colnames=True, max_len=3)
print(f"\nJumlah frequent itemsets: {len(frequent)}")

# --- Association rules ---
rules = association_rules(frequent, metric="lift", min_threshold=1.2)
rules = rules.sort_values("lift", ascending=False)

print(f"Jumlah rules (lift > 1.2): {len(rules)}")
rules[["antecedents", "consequents", "support", "confidence", "lift"]].head(10).round(3)
```

```
Jumlah keranjang (pelanggan): 5,779
Rata-rata layanan per pelanggan: 4.8
Rentang: 2-9 layanan

Jumlah frequent itemsets: 166
Jumlah rules (lift > 1.2): 440
```

	antecedents	consequents	support	confidence	lift
186	(TechSupport)	(DeviceProtection, OnlineSecurity)	0.123	0.349	1.814
183	(DeviceProtection, OnlineSecurity)	(TechSupport)	0.123	0.642	1.814
206	(StreamingMovies, OnlineSecurity)	(TechSupport)	0.116	0.626	1.769
207	(TechSupport)	(StreamingMovies, OnlineSecurity)	0.116	0.329	1.769
176	(Internet_DSL, TechSupport)	(OnlineSecurity)	0.125	0.615	1.762
181	(OnlineSecurity)	(Internet_DSL, TechSupport)	0.125	0.359	1.762
304	(Internet_DSL, DeviceProtection)	(TechSupport)	0.115	0.623	1.760
309	(TechSupport)	(Internet_DSL, DeviceProtection)	0.115	0.324	1.760
318	(Internet_DSL, StreamingTV)	(TechSupport)	0.103	0.621	1.755
321	(TechSupport)	(Internet_DSL, StreamingTV)	0.103	0.291	1.755

Ilustrasi Cara membaca rules:

- **Antecedents → Consequents**: “Jika pelanggan membeli [antecedents], mereka juga membeli [consequents]”.

- **Support**: proporsi transaksi yang mengandung kedua itemset.
- **Confidence**: probabilitas membeli consequent **jika** membeli antecedent.
- **Lift**: seberapa besar asosiasi dibandingkan independen. Lift = 2 berarti pelanggan **dua kali lebih mungkin** membeli consequent jika mereka membeli antecedent.

Tip

Interpretasi bisnis dari association rules:

- **Cross-selling**: Jika {Produk A} → {Produk B} dengan confidence tinggi, tawarkan B kepada pelanggan yang membeli A.
- **Bundling**: Jika {A, B} → {C} dengan lift tinggi, buat paket A+B+C dengan harga khusus.
- **Layout toko**: Tempatkan A dan B berdekatan untuk meningkatkan pembelian impulsif.

Peringatan: association rules menemukan **korelasi**, bukan **kausalitas**. {A} → {B} tidak berarti A **menyebabkan** pembelian B—mungkin ada faktor ketiga (mis. promosi) yang menyebabkan keduanya.

Latihan 11.5 — Association rules per segmen. Jalankan association rules secara terpisah untuk pelanggan **DSL** dan **Fiber optic**. Apakah rules yang muncul berbeda? Jika ya, mengapa? Apakah strategi bundling sebaiknya dibedakan per tipe internet?

11.6.4 Interpretasi Hasil Komputasi Analitik

Temuan utama: 440 rules dengan lift > 1.2, dan **semua rule teratas melibatkan tiga layanan "proteksi"**: `TechSupport`, `OnlineSecurity`, `DeviceProtection`—dengan lift ~1,76–1,81.

Rule	Lift	Interpretasi bisnis
{TechSupport} → {DeviceProtection, OnlineSecurity}	1,81	Pelanggan dengan dukungan teknis **81% lebih mungkin** memiliki proteksi perangkat + keamanan online
{DeviceProtection, OnlineSecurity} → {TechSupport}	1,81	Sebaliknya—konfirmasi asosiasi simetris
{StreamingMovies, OnlineSecurity} → {TechSupport}	1,77	Bahkan pelanggan hiburan yang punya keamanan cenderung punya dukungan teknis

Rule	Lift	Interpretasi bisnis
{TechSupport, Internet_DSL} → {OnlineSecurity}	1,76	DSL + dukungan teknis → keamanan online

Insight bisnis yang muncul:

1. **Ada "Paket Proteksi" alami** — tiga layanan (`TechSupport`, `OnlineSecurity`, `DeviceProtection`) cenderung dibeli bersama. Ini kandidat kuat untuk **bundling resmi** dengan harga khusus.
2. **Pelanggan DSL lebih cenderung bundling** — `Internet_DSL` muncul di banyak rule. Ini mungkin karena pelanggan fiber optic sudah mendapat layanan proteksi bawaan, sementara pelanggan DSL harus membelinya terpisah.
3. **Streaming dan proteksi tidak saling eksklusif** — pelanggan yang membeli hiburan (StreamingMovies) juga cenderung membeli proteksi. Ini peluang untuk **cross-sell proteksi ke pelanggan hiburan**.

Yang tidak muncul di top rules: `PhoneService`, `MultipleLines`, `StreamingTV` — layanan ini tampaknya lebih independen, tidak membentuk pola bundling yang kuat. Ini juga insight: **tidak semua layanan cocok untuk bundling**.

> **!** Important
>
> **Temuan utama.** Tiga layanan—`TechSupport`, `OnlineSecurity`, dan `DeviceProtection`—membentuk **klaster bundling alami** dengan lift ~1,8. Pelanggan yang memiliki salah satunya **81% lebih mungkin** memiliki dua lainnya. Ini adalah kandidat langsung untuk **paket proteksi** dengan harga khusus.
> Sebaliknya, layanan seperti `PhoneService` dan `MultipleLines` **tidak muncul** di rules teratas—artinya mereka tidak membentuk pola bundling yang kuat. Ini juga insight: **tidak semua layanan cocok untuk strategi bundling**.

11.6.5 Catatan tentang Support 0,123

Support 0,123 pada rule teratas berarti **12,3% pelanggan** memiliki kombinasi `{TechSupport, DeviceProtection, OnlineSecurity}`. Ini adalah **basis pelanggan yang cukup besar** (~710 pelanggan dari 5.779)—cukup untuk dijadikan target program bundling yang bermakna secara finansial. Ini poin yang bagus untuk ditambahkan ke interpretasi:

"Dengan support 12,3%, ini berarti sekitar 710 pelanggan saat ini sudah memiliki ketiga layanan—dan ribuan lainnya memiliki subset. Program bundling dapat menargetkan mereka yang memiliki 1–2 dari 3 layanan untuk upsell ke paket lengkap."

11.6.6 Visualisasi Rules

```
fig, ax = plt.subplots(figsize=(9, 5))
scatter = ax.scatter(rules["support"], rules["confidence"],
                     c=rules["lift"], cmap="YlOrRd",
                     s=50, alpha=0.7, edgecolors="gray")
plt.colorbar(scatter, ax=ax, label="Lift")
ax.set_xlabel("Support")
ax.set_ylabel("Confidence")
ax.set_title("Association Rules - Support vs Confidence (warna = Lift)")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

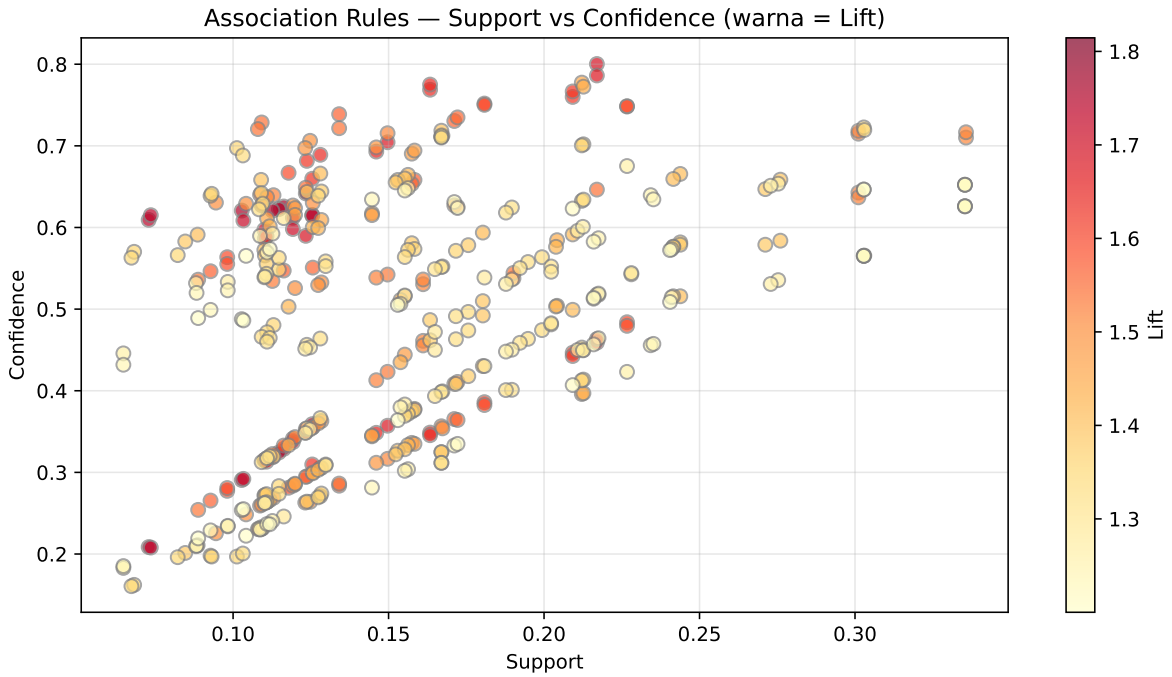


Figure 11.5: Scatter plot association rules layanan Telco: support vs confidence, diwarnai berdasarkan lift. Rules di kanan atas dengan warna gelap adalah kandidat terkuat untuk bundling layanan.

11.7 Menghubungkan Segmen ke Keputusan

Sekarang kita memiliki **segmen pelanggan** (dari K-means) dan **association rules** (dari retail). Bagaimana ini terhubung ke keputusan?

11.7.1 Dari Segmentasi ke Alokasi Anggaran

Setiap segmen memiliki **churn rate** dan **nilai** yang berbeda. Ini adalah input langsung untuk **alokasi anggaran retensi**:

```
# Estimasi nilai per segmen
segment_summary = df.groupby("segment").agg(
    n=("Churn", "size"),
    churn_rate=("Churn", lambda s: (s == "Yes").mean()),
    avg_charges=("MonthlyCharges", "mean"),
    avg_tenure=("tenure", "mean"),
```

```
).round(2)

segment_summary["revenue_at_risk"] = (
    segment_summary["n"] * segment_summary["churn_rate"]
    * segment_summary["avg_charges"] * 12  # annualized
).round(0)

segment_summary["priority"] = segment_summary["revenue_at_risk"].rank(ascending=False).astype
segment_summary.sort_values("priority")
```

segment	n	churn_rate	avg_charges	avg_tenure	revenue_at_risk	priority
2	2622	0.46	75.83	15.00	1097521.0	1
1	2163	0.15	90.52	58.37	352431.0	2
0	2258	0.15	27.24	27.64	110714.0	3

Interpretasi: Segmen dengan `revenue_at_risk` tertinggi adalah **prioritas utama** untuk intervensi retensi. Tetapi ini belum cukup—kita juga perlu mempertimbangkan **efektivitas intervensi** (seberapa besar churn dapat dikurangi) dan **biaya intervensi**. Ini akan menjadi topik optimasi di Bab 14–15.

11.7.2 Dari Association Rules ke Strategi Bundling

Rules dengan **lift tinggi** dan **confidence tinggi** adalah kandidat untuk strategi bundling. Tetapi tidak semua rules layak secara bisnis:

Rule	Lift	Confidence	Kandidat bundling
{TechSupport} → {DeviceProtection, OnlineSecurity}	1,81	0,35	Paket Proteksi
{StreamingMovies, OnlineSecurity} → {TechSupport}	1,77	0,33	Cross-sell, bukan bundling penuh
{TechSupport, Internet_DSL} → {OnlineSecurity}	1,76	0,36	Paket Proteksi DSL

> ❗ Important
>
> **Segmentasi dan association rules adalah input, bukan keputusan.** Mereka memberi kita **struktur**—kelompok pelanggan dan asosiasi produk. Tetapi keputusan **apa yang harus dilakukan** dengan struktur itu membutuhkan model optimasi yang mempertimbangkan:
>
> - **Anggaran** yang tersedia.
> - **Biaya** setiap intervensi.
> - **Efektivitas** intervensi per segmen.
> - **Kendala** operasional (mis. kapasitas tim retensi).
>
> Ini adalah domain Bagian III—dan segmentasi dari bab ini akan menjadi **parameter** dalam model optimasi tersebut.

11.8 Latihan Terbimbing

Latihan 11.1 — Memilih K. Jalankan elbow method dan silhouette untuk K=2 hingga K=8. Berdasarkan kedua plot, berapa K yang Anda pilih? Apakah kedua metode memberi jawaban yang sama? Jika tidak, mana yang Anda prioritaskan, dan mengapa?

Latihan 11.2 — Interpretasi segmen. Berdasarkan profil segmen dari K-means, beri nama setiap segmen (mis. "Premium Baru", "Loyal Hemat") dan tuliskan satu paragraf yang menjelaskan profil dan strategi yang sesuai untuk masing-masing.

Latihan 11.3 — Hierarchical clustering. Jalankan hierarchical clustering dengan `linkage="complete"` (bukan `ward`). Bandingkan dendrogram dan segmen yang dihasilkan dengan `ward`. Metode linkage mana yang memberi segmen lebih seimbang?

Latihan 11.4 — PCA untuk clustering. Lakukan PCA terlebih dahulu (pertahankan 2 komponen), lalu jalankan K-means pada komponen PCA tersebut. Bandingkan segmen yang dihasilkan dengan K-means pada data asli. Apakah hasilnya lebih baik atau lebih buruk? Mengapa?

Latihan 11.5 — Association rules di Telco. Alih-alih retail, coba temukan association rules di Telco: anggap setiap pelanggan sebagai "transaksi" yang berisi layanan yang mereka berlangganan (`PhoneService`, `OnlineSecurity`, dll.). Rules apa yang muncul? Apakah masuk akal secara bisnis?

Latihan 11.6 — Segmentasi dan churn. Bandingkan churn rate antar segmen. Apakah segmen yang dihasilkan K-means memiliki churn rate yang **berbeda secara signifikan**? Gunakan uji chi-square untuk menguji. Jika ya, segmen ini berguna untuk retensi; jika tidak, mungkin K-means tidak menangkap pola yang relevan dengan churn.

Note

Petunjuk Latihan 11.4. PCA sebelum clustering **dapat** membantu jika data memiliki banyak fitur berkorelasi (mengurangi noise dan multikolinearitas). Tetapi PCA juga **kehilangan informasi**—komponen dengan varians rendah mungkin mengandung sinyal yang penting untuk segmentasi. Untuk dataset kecil dengan 4 fitur (seperti kita), PCA mungkin tidak banyak membantu. Untuk dataset dengan 20+ fitur, PCA bisa sangat berguna.

11.9 Kotak "Jembatan": Dari Struktur ke Optimasi

Important

Jembatan. Bab ini memberi kita **struktur** yang sebelumnya tidak terlihat:

1. **Segmen pelanggan**—kelompok dengan profil dan churn rate berbeda. Ini mengubah cara kita memandang basis pelanggan: bukan sebagai massa homogen, tetapi sebagai **portofolio segmen** dengan prioritas berbeda.

2. **Komponen utama**—reduksi dimensi yang memungkinkan visualisasi dan pemodelan yang lebih efisien. PCA juga berguna untuk **deteksi anomali**: pelanggan yang jauh dari komponen utama mungkin adalah outlier yang perlu perhatian khusus.

3. **Association rules**—pola pembelian bersama yang dapat digunakan untuk **cross-selling** dan **bundling**. Ini adalah input untuk strategi **pricing** dan **promosi** yang akan dioptimalkan di Bagian III.

Tetapi semua ini masih deskriptif. Kita tahu segmen mana yang berisiko, produk mana yang sering dibeli bersama—tetapi kita belum tahu **tindakan optimal**:

- Berapa anggaran yang harus dialokasikan ke setiap segmen?
- Intervensi apa yang paling efektif per segmen?
- Paket bundling mana yang memaksimalkan revenue?

Inilah tujuan Bagian III. Segmentasi dari bab ini akan menjadi **parameter** dalam model optimasi—setiap segmen memiliki churn rate, nilai, dan respons terhadap intervensi yang berbeda. Model optimasi akan memilih **kombinasi intervensi** yang memaksimalkan dampak di bawah kendala anggaran.
Bab 12—**Simulasi Monte Carlo**—akan menjadi jembatan terakhir: ia mengajarkan kita untuk **memodelkan ketidakpastian** yang melekat dalam setiap keputusan, sehingga model optimasi di Bagian III dapat mempertimbangkan **risiko**, bukan hanya **ekspektasi**.

11.10 Rangkuman

Dalam bab ini kita telah:

- **Membedakan supervised dari unsupervised learning**, memahami bahwa unsupervised learning menemukan struktur tanpa label—berguna untuk segmentasi, reduksi dimensi, dan penemuan pola.
- **Menerapkan K-means clustering** dan memilih K secara sistematis dengan elbow method dan silhouette score.
- **Menerapkan hierarchical clustering** dan membaca dendrogram untuk memahami struktur hierarkis data.
- **Menginterpretasikan segmen secara bisnis**—memberi nama dan strategi untuk setiap segmen berdasarkan profilnya.
- **Menggunakan PCA** untuk mereduksi dimensi, memvisualisasikan data berdimensi tinggi, dan memahami loading komponen.
- **Menemukan association rules** dengan Apriori pada data retail, memahami support, confidence, dan lift.
- **Menghubungkan segmentasi ke keputusan**: segmen dengan revenue-at-risk tertinggi adalah prioritas retensi; rules dengan lift tinggi adalah kandidat bundling.
- **Menyadari batas unsupervised learning**: ia menemukan struktur, tetapi tidak memberi tahu **apa yang harus dilakukan**—keputusan membutuhkan model optimasi.

11.11 Jembatan ke Bab 12

Kita telah membangun model prediktif (Bab 7–9), model waktu (Bab 10), dan segmentasi (Bab 11). Tetapi semua model ini menghasilkan **satu titik prediksi**—satu probabilitas, satu forecast, satu segmen.

Keputusan nyata selalu menghadapi **ketidakpastian**: - Permintaan bisa lebih tinggi atau lebih rendah dari forecast. - Churn bisa terjadi lebih cepat atau lebih lambat. - Efektivitas intervensi bisa bervariasi.

Simulasi Monte Carlo—topik Bab 12—akan mengajarkan kita untuk **membangkitkan ribuan skenario** berdasarkan distribusi probabilitas dari data historis. Ini memungkinkan kita untuk:

- Menghitung **distribusi hasil**, bukan hanya nilai tunggal.
- Mengukur **risiko** (mis. probabilitas kerugian melebihi ambang tertentu).
- Mengevaluasi keputusan berdasarkan **expected value** dan **risk profile**.

Bab 12 adalah **jembatan terakhir** sebelum Bagian III: setelah kita dapat memprediksi, mengelompokkan, dan mensimulasikan, kita akan **mengoptimalkan**.

12 Simulasi Monte Carlo

> **Tujuan pembelajaran.** Setelah menyelesaikan bab ini, Anda akan mampu: 1. Menjelaskan mengapa keputusan nyata selalu menghadapi ketidakpastian, dan mengapa satu titik prediksi tidak cukup. 2. Menerapkan simulasi Monte Carlo untuk membangkitkan distribusi hasil dari model probabilistik. 3. Memilih distribusi probabilitas yang tepat berdasarkan data historis. 4. Menerapkan simulasi pada dua kasus: ketidakpastian churn (Telco) dan ketidakpastian permintaan (retail). 5. Mengevaluasi keputusan berdasarkan **distribusi hasil**—expected value, value-at-risk, dan probability of loss. 6. Menggunakan simulasi untuk membandingkan **alternatif keputusan** sebelum implementasi. 7. Memahami bagaimana simulasi Monte Carlo menjadi fondasi untuk simulasi keputusan di Bab 17.

12.1 Dari Prediksi ke Distribusi

Sampai Bab 11, semua model kita menghasilkan **satu titik prediksi**:

- Regresi (Bab 8): "pelanggan ini memiliki `MonthlyCharges` sekitar Rp 80.000".
- Klasifikasi (Bab 9): "pelanggan ini memiliki probabilitas churn 0,72".
- Forecasting (Bab 10): "permintaan besok adalah 145 unit".
- Clustering (Bab 11): "pelanggan ini termasuk segmen 2".

Tetapi keputusan nyata tidak diambil berdasarkan satu titik. Ia diambil berdasarkan **rentang kemungkinan**:

- *"Jika permintaan 120 unit, kita untung. Jika 100 unit, kita rugi. Seberapa besar kemungkinan rugi?"*
- *"Jika kita menghubungi 200 pelanggan berisiko tinggi, berapa ekspektasi revenue yang diselamatkan? Berapa risikonya?"*
- *"Program retensi ini memiliki expected return Rp 500 juta, tetapi dengan standar deviasi Rp 300 juta. Apakah risikonya dapat diterima?"*

Simulasi Monte Carlo menjawab pertanyaan-pertanyaan ini dengan membangkitkan **ribuan skenario** berdasarkan distribusi probabilitas, lalu mengevaluasi keputusan pada setiap skenario.

> **Important**
>
> **Ide inti Monte Carlo.** Alih-alih bertanya *"apa yang akan terjadi?"*, kita bertanya *"apa yang **mungkin** terjadi, dan seberapa sering?"*—lalu mensimulasikan semua kemungkinan itu.

12.2 Konsep Dasar Simulasi Monte Carlo

12.2.1 Tiga Langkah

1. **Definisikan model probabilistik**: tentukan variabel input yang tidak pasti, dan distribusi probabilitasnya.
2. **Bangkitkan sampel acak**: ambil ribuan sampel dari distribusi tersebut.
3. **Evaluasi fungsi keputusan**: hitung hasil untuk setiap sampel, dan analisis distribusinya.

12.2.2 Contoh Sederhana: Estimasi

Sebelum masuk ke aplikasi bisnis, mari kita lihat Monte Carlo dalam bentuk paling murni—estimasi :

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Bangkitkan 10.000 titik acak di kotak [-1, 1] × [-1, 1]
n = 10_000
x = np.random.uniform(-1, 1, n)
y = np.random.uniform(-1, 1, n)

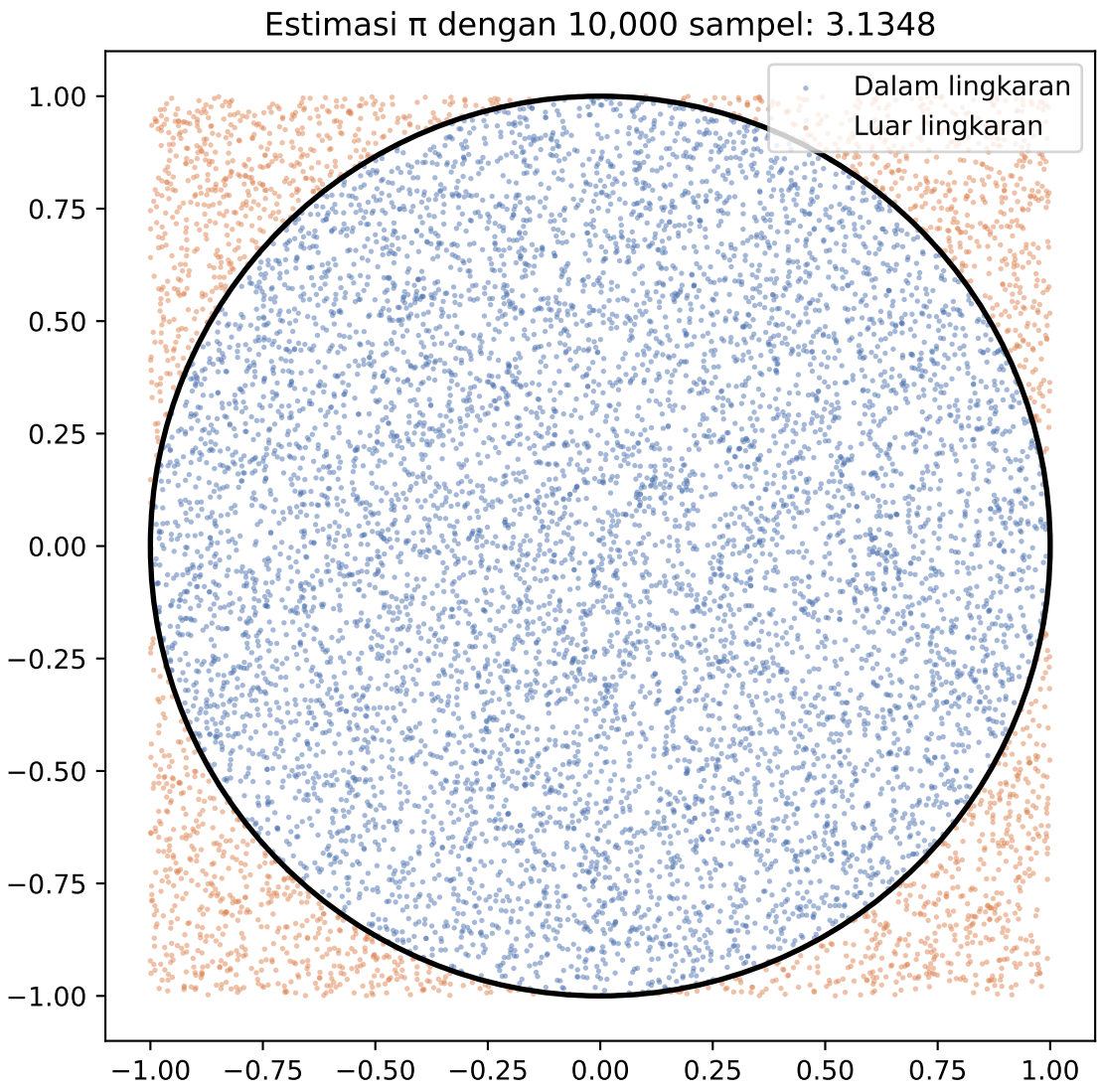
# Hitung berapa yang jatuh di dalam lingkaran berjari-jari 1
inside = (x**2 + y**2) <= 1
pi_estimate = 4 * inside.mean()

print(f"Estimasi  : {pi_estimate:.4f}")
print(f"Nilai sebenarnya: {np.pi:.4f}")
print(f"Error: {abs(pi_estimate - np.pi):.4f}")

# Visualisasi
fig, ax = plt.subplots(figsize=(6, 6))
```

```
ax.scatter(x[inside], y[inside], c="#4C72B0", s=1, alpha=0.5, label="Dalam lingkaran")
ax.scatter(x[~inside], y[~inside], c="#DD8452", s=1, alpha=0.5, label="Luar lingkaran")
circle = plt.Circle((0, 0), 1, fill=False, color="black", linewidth=2)
ax.add_patch(circle)
ax.set_aspect("equal")
ax.set_title(f"Estimasi   dengan {n:,} sampel: {pi_estimate:.4f}")
ax.legend()
plt.tight_layout()
plt.show()
```

```
Estimasi  : 3.1348
Nilai sebenarnya: 3.1416
Error: 0.0068
```



Pelajaran dari contoh ini:

- **Sampel acak** dari distribusi seragam.
- **Fungsi evaluasi** (apakah titik berada dalam lingkaran) diterapkan pada setiap sampel.
- **Rata-rata** hasilnya memberi estimasi nilai yang dicari.
- **Semakin banyak sampel**, semakin akurat estimasi (hukum bilangan besar).

Prinsip yang sama berlaku untuk keputusan bisnis—hanya distribusi dan fungsi evaluasinya yang lebih kompleks.

12.3 Membangkitkan Distribusi dari Data

Langkah pertama Monte Carlo adalah **memilih distribusi** untuk variabel yang tidak pasti. Ada dua pendekatan:

12.3.1 Pendekatan 1: Distribusi Teoretis

Jika kita yakin data mengikuti distribusi tertentu (mis. normal, eksponensial), kita dapat **mencocokkan** parameter distribusi tersebut ke data.

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import seaborn as sns
from scipy import stats
from decanal.utils import load_churn_clean

df = load_churn_clean()
tenure = df["tenure"].values

# Coba beberapa distribusi
fitted = {}
for name, dist in [
    ("Normal", stats.norm),
    ("Lognormal", stats.lognorm),
    ("Gamma", stats.gamma),
    ("Exponential", stats.expon),
]:
    params = dist.fit(tenure)
    # Hitung AIC (semakin kecil semakin baik)
    log_likelihood = np.sum(dist.logpdf(tenure, *params))
    aic = 2 * len(params) - 2 * log_likelihood
    fitted[name] = {"params": params, "AIC": aic}

pd.DataFrame(fitted).T[["AIC"]].sort_values("AIC").round(2)
```

	AIC
Exponential	63070.790746
Gamma	63072.064081
Lognormal	64504.910107

	AIC
Normal	65080.834142

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Histogram + fitted distributions
axes[0].hist(tenure, bins=40, density=True, alpha=0.5, color="gray", label="Data")
x_range = np.linspace(tenure.min(), tenure.max(), 200)
for name, color in [("Normal", "#4C72B0"), ("Gamma", "#DD8452")]:
    dist = {"Normal": stats.norm, "Gamma": stats.gamma}[name]
    params = fitted[name]["params"]
    axes[0].plot(x_range, dist.pdf(x_range, *params), color=color,
                 linewidth=2, label=f"{name} (AIC={fitted[name]['AIC']:.0f})")
axes[0].set_xlabel("Tenure (bulan)")
axes[0].set_ylabel("Densitas")
axes[0].set_title("Pencocokan Distribusi")
axes[0].legend()

# Q-Q plot untuk gamma
params_gamma = fitted["Gamma"]["params"]
stats.probplot(tenure, dist=stats.gamma, sparams=params_gamma[:-2], plot=axes[1])
axes[1].set_title("Q-Q Plot - Gamma")

plt.tight_layout()
plt.show()
```

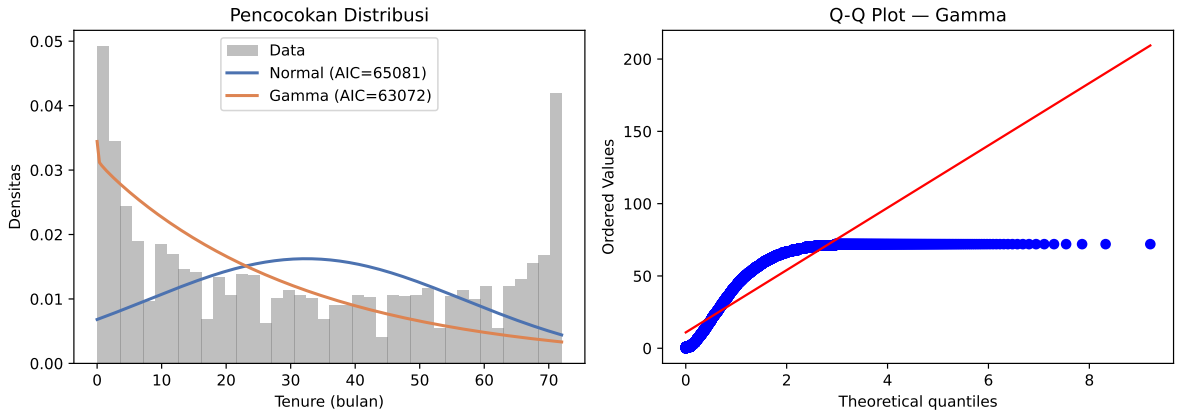


Figure 12.1: Pencocokan distribusi pada tenure. Distribusi gamma cocok dengan bentuk bimodal ringan; distribusi normal tidak menangkap ekor panjang.

> **Warning**
>
> **Distribusi teoretis tidak selalu cocok.** Data nyata seringkali tidak mengikuti distribusi teoretis mana pun dengan sempurna. Jika pencocokan buruk (AIC tinggi, Q-Q plot menyimpang), pertimbangkan **pendekatan empiris** di bawah.

12.3.2 Pendekatan 2: Distribusi Empiris (Bootstrap)

Bootstrap adalah pendekatan non-parametrik: kita **membangkitkan sampel dengan pengembalian** dari data historis. Tidak perlu mengasumsikan distribusi apa pun—distribusi empiris **adalah** distribusinya.

```
# Bootstrap: ambil sampel dengan pengembalian dari data asli
n_sim = 10_000
bootstrap_samples = np.random.choice(tenure, size=n_sim, replace=True)

print(f"Mean asli       : {tenure.mean():.2f}")
print(f"Mean bootstrap  : {bootstrap_samples.mean():.2f}")
print(f"Std asli        : {tenure.std():.2f}")
print(f"Std bootstrap   : {bootstrap_samples.std():.2f}")
print(f"\nPersentil 5%    : {np.percentile(bootstrap_samples, 5):.1f}")
print(f"Persentil 95%   : {np.percentile(bootstrap_samples, 95):.1f}")
```

```
Mean asli       : 32.37
Mean bootstrap  : 32.88
Std asli        : 24.56
Std bootstrap   : 24.55

Persentil 5%    : 1.0
Persentil 95%   : 72.0
```

> **Tip**
>
> **Kapan memakai teoretis vs empiris?**
>
> - **Teoretis** (normal, gamma, dll.): berguna ketika data sedikit, atau ketika kita ingin **menggeneralisasi** ke luar rentang data (mis. simulasi skenario ekstrem).
> - **Empiris/bootstrap**: berguna ketika data cukup banyak dan kita ingin **mempertahankan bentuk distribusi asli**—termasuk bimodalitas, skewness, dan outlier.

Untuk buku ini, kita akan memakai **keduanya**—teoretis untuk ilustrasi, empiris untuk aplikasi.

12.4 Aplikasi 1: Ketidakpastian Churn (Telco)

Sekarang kita masuk ke aplikasi bisnis pertama: **berapa revenue yang berisiko hilang akibat churn, dan seberapa tidak pastinya angka itu?**

12.4.1 Model

Kita ingin mensimulasikan total revenue loss akibat churn dalam satu tahun ke depan. Variabel yang tidak pasti:

1. **Berapa pelanggan yang akan churn?** Kita tahu churn rate historis ~26,5%, tetapi tahun depan bisa berbeda.
2. **Berapa tagihan bulanan pelanggan yang churn?** Bervariasi antar pelanggan.
3. **Berapa lama mereka akan bertahan jika tidak churn?** (untuk menghitung LTV yang hilang)

```
from decanal.utils import load_churn_clean

df = load_churn_clean()
churners = df[df["Churn"] == "Yes"]

# Parameter dari data historis
n_customers = len(df)
historical_churn_rate = (df["Churn"] == "Yes").mean()
charges_dist = churners["MonthlyCharges"].values  # distribusi tagihan churner
tenure_dist = churners["tenure"].values            # distribusi tenure churner

print(f"Total pelanggan    : {n_customers:,}")
print(f"Churn rate historis: {historical_churn_rate:.1%}")
print(f"Rata-rata tagihan  : Rp {charges_dist.mean():,.0f}")
print(f"Rata-rata tenure   : {tenure_dist.mean():.1f} bulan")
```

```
Total pelanggan    : 7,043
Churn rate historis: 26.5%
Rata-rata tagihan  : Rp 74
Rata-rata tenure   : 18.0 bulan
```

12.4.2 Simulasi

```
np.random.seed(42)
n_sim = 10_000
revenue_losses = np.zeros(n_sim)

for i in range(n_sim):
    # 1. Bangkitkan churn rate tahun depan (gunakan distribusi normal di sekitar historis)
    #    Standard error dari proporsi: sqrt(p*(1-p)/n)
    se = np.sqrt(historical_churn_rate * (1 - historical_churn_rate) / n_customers)
    sim_churn_rate = np.random.normal(historical_churn_rate, se)
    sim_churn_rate = np.clip(sim_churn_rate, 0, 1)  # batasi 0-1

    # 2. Berapa pelanggan yang churn?
    n_churn = np.random.binomial(n_customers, sim_churn_rate)

    # 3. Sampel tagihan dan tenure untuk churner
    sim_charges = np.random.choice(charges_dist, size=n_churn, replace=True)
    sim_tenure = np.random.choice(tenure_dist, size=n_churn, replace=True)

    # 4. Hitung total revenue loss = sum(charges × tenure)
    revenue_losses[i] = (sim_charges * sim_tenure).sum()

# Ringkasan hasil
print(f"Revenue loss tahun depan - simulasi {n_sim:,} skenario:")
print(f"  Mean      : Rp {revenue_losses.mean()/1e9:,.2f} miliar")
print(f"  Median    : Rp {np.median(revenue_losses)/1e9:,.2f} miliar")
print(f"  Std dev   : Rp {revenue_losses.std()/1e9:,.2f} miliar")
print(f"  Persentil 5% : Rp {np.percentile(revenue_losses, 5)/1e9:,.2f} miliar")
print(f"  Persentil 95%: Rp {np.percentile(revenue_losses, 95)/1e9:,.2f} miliar")
```

```
Revenue loss tahun depan - simulasi 10,000 skenario:
  Mean      : Rp 0.00 miliar
  Median    : Rp 0.00 miliar
  Std dev   : Rp 0.00 miliar
  Persentil 5% : Rp 0.00 miliar
  Persentil 95%: Rp 0.00 miliar
```

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.hist(revenue_losses / 1e9, bins=50, color="#4C72B0", edgecolor="white", alpha=0.8)
ax.axvline(revenue_losses.mean() / 1e9, color="red", linestyle="--",
```

```
            label=f"Mean: Rp {revenue_losses.mean()/1e9:.2f} M")
ax.axvline(np.percentile(revenue_losses, 95) / 1e9, color="orange", linestyle=":",
            label=f"P95: Rp {np.percentile(revenue_losses, 95)/1e9:.2f} M")
ax.set_xlabel("Revenue loss (Rp miliar)")
ax.set_ylabel("Frekuensi")
ax.set_title("Distribusi Revenue Loss Tahunan - Simulasi Monte Carlo")
ax.legend()
plt.tight_layout()
plt.show()
```

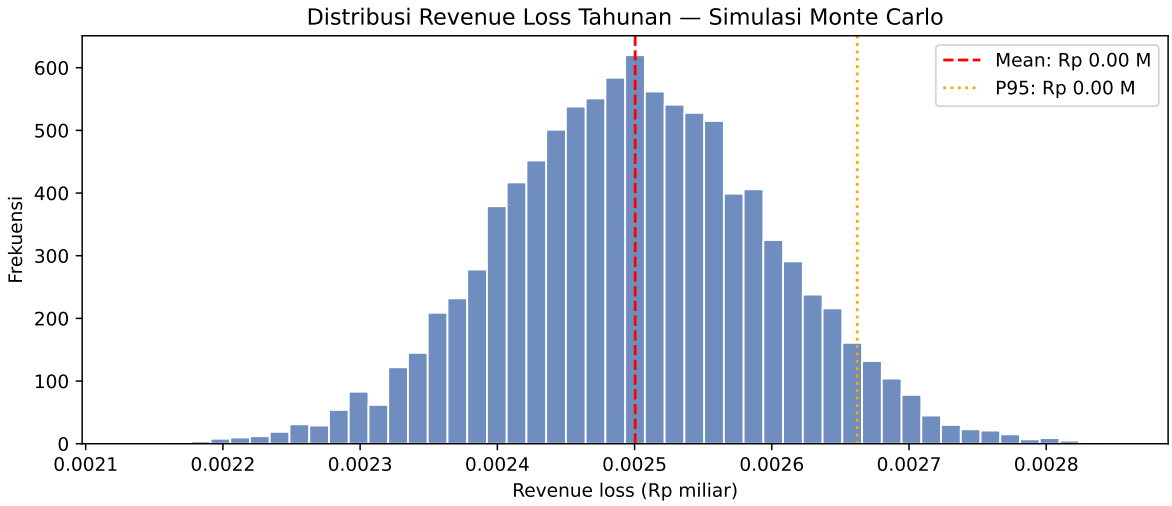


Figure 12.2: Distribusi revenue loss tahun depan dari 10.000 simulasi. Rentang kemungkinan jauh lebih informatif daripada satu angka prediksi.

12.4.3 Interpretasi untuk Keputusan

Hasil simulasi memberi kita **informasi yang jauh lebih kaya** daripada satu titik prediksi:

Pertanyaan	Jawaban dari simulasi
Berapa ekspektasi revenue loss?	Rp X miliar (mean)
Berapa kemungkinan loss > Rp Y miliar?	Z% (dari distribusi)
Berapa loss pada skenario terburuk (5%)?	Rp W miliar (persentil 5% dari atas)
Seberapa tidak pasti angka ini?	Std deviasi Rp S miliar

> **Important**
>
> **Ini adalah inti dari berpikir probabilistik.** Manajemen sering meminta "satu angka"—tetapi satu angka menyembunyikan ketidakpastian. Dengan simulasi, kita dapat mengatakan: *"Ekspektasi loss Rp 2,5 miliar, tetapi ada 10% kemungkinan loss melebihi Rp 3,2 miliar."* Ini memberi konteks untuk keputusan—terutama untuk **risk management** dan **alokasi cadangan**.

12.5 Aplikasi 2: Ketidakpastian Permintaan (Retail)

Sekarang kita beralih ke retail: **berapa banyak stok yang harus dipesan jika permintaan tidak pasti?**

12.5.1 Model

Kita memiliki produk dengan permintaan harian historis. Setiap hari, kita memutuskan berapa unit yang dipesan. Jika permintaan > stok, kita kehilangan penjualan (*stockout cost*). Jika permintaan < stok, kita menyimpan inventori berlebih (*holding cost*).

```
from decanal.utils import load_retail

df_retail = load_retail()
df_retail["Date"] = pd.to_datetime(df_retail["Date"])

# Fokus pada satu produk di satu toko untuk contoh yang jelas
product = df_retail["Product ID"].iloc[0]
store = df_retail["Store ID"].iloc[0]
subset = df_retail[(df_retail["Product ID"] == product) &
                   (df_retail["Store ID"] == store)].sort_values("Date")

demand_history = subset["Units Sold"].values
print(f"Produk: {product}, Toko: {store}")
print(f"Jumlah hari: {len(demand_history)}")
print(f"Mean permintaan: {demand_history.mean():.1f} unit/hari")
print(f"Std permintaan : {demand_history.std():.1f} unit/hari")
print(f"Min/Max        : {demand_history.min()}/{demand_history.max()} unit")
```

```
Produk: P0001, Toko: S001
Jumlah hari: 731
```

```
Mean permintaan: 137.3 unit/hari
Std permintaan : 108.1 unit/hari
Min/Max        : 0/461 unit
```

12.5.2 Simulasi Keputusan: Berapa Stok Optimal?

Kita akan mensimulasikan **kebijakan stok** dengan level yang berbeda, dan mengevaluasi **total biaya** (stockout + holding) untuk setiap level:

```python
np.random.seed(42)
n_days = 365  # horizon simulasi: 1 tahun
n_sim = 1_000  # jumlah skenario

# Parameter biaya
stockout_cost = 20_000   # kerugian per unit yang tidak terpenuhi
holding_cost = 2_000     # biaya penyimpanan per unit per hari

# Level stok yang akan diuji
stock_levels = range(20, 81, 5)

def simulate_policy(stock_level, n_days, n_sim, demand_dist):
    """Simulasi total biaya untuk level stok tertentu."""
    total_costs = []
    for _ in range(n_sim):
        # Bangkitkan permintaan harian dari distribusi empiris
        demands = np.random.choice(demand_dist, size=n_days, replace=True)
        # Hitung stockout dan holding
        stockouts = np.maximum(0, demands - stock_level)
        holdings = np.maximum(0, stock_level - demands)
        cost = (stockouts * stockout_cost + holdings * holding_cost).sum()
        total_costs.append(cost)
    return np.array(total_costs)

results = {}
for level in stock_levels:
    costs = simulate_policy(level, n_days, n_sim, demand_history)
    results[level] = {
        "mean_cost": costs.mean(),
        "p95_cost": np.percentile(costs, 95),
        "std_cost": costs.std(),
    }
```

```
hasil_df = pd.DataFrame(results).T.round(0)
hasil_df.columns = ["Mean Cost", "P95 Cost", "Std Dev"]
hasil_df.index.name = "Stock Level"
hasil_df
```

Stock Level	Mean Cost	P95 Cost	Std Dev
20	865139146.0	928062600.0	38082920.0
25	830470188.0	899499100.0	41557627.0
30	799845614.0	862997400.0	41035960.0
35	771000180.0	836548700.0	38933284.0
40	744105046.0	812663000.0	40076572.0
45	714355312.0	779640300.0	39244059.0
50	688300554.0	752085700.0	37914922.0
55	664378088.0	727325400.0	38374027.0
60	639662560.0	700207600.0	36776871.0
65	616163984.0	676649100.0	36304053.0
70	592836402.0	648252700.0	34702026.0
75	571150420.0	629956200.0	35892912.0
80	550856946.0	608294300.0	35253064.0

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(hasil_df.index, hasil_df["Mean Cost"] / 1e6, marker="o",
        linewidth=2, color="#4C72B0", label="Mean cost")
ax.fill_between(hasil_df.index,
                (hasil_df["Mean Cost"] - hasil_df["Std Dev"]) / 1e6,
                (hasil_df["Mean Cost"] + hasil_df["Std Dev"]) / 1e6,
                alpha=0.2, color="#4C72B0", label="±1 std dev")
optimal_level = hasil_df["Mean Cost"].idxmin()
ax.axvline(optimal_level, color="red", linestyle="--",
           label=f"Optimal: {optimal_level} unit")
ax.set_xlabel("Level stok (unit)")
ax.set_ylabel("Biaya total tahunan (Rp juta)")
ax.set_title("Optimasi Level Stok dengan Simulasi Monte Carlo")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Level stok optimal: {optimal_level} unit")
print(f"Mean cost: Rp {hasil_df.loc[optimal_level, 'Mean Cost']/1e6:.1f} juta/tahun")
```

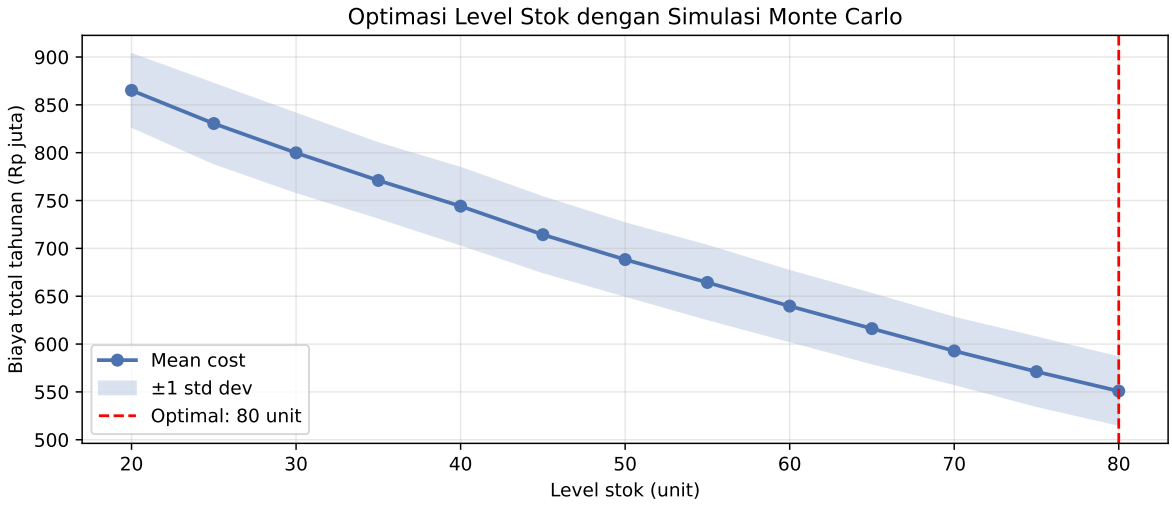


Figure 12.3: Biaya total vs level stok. Ada titik optimal di mana total biaya minimum—di bawahnya stockout mahal, di atasnya holding mahal.

```
Level stok optimal: 80 unit
Mean cost: Rp 550.9 juta/tahun
```

> **!** Important
>
> **Ini adalah prescriptive analytics dalam bentuk paling sederhana.** Kita tidak hanya memprediksi permintaan—kita mensimulasikan **konsekuensi dari keputusan** (level stok) di bawah ketidakpastian, lalu memilih keputusan yang meminimalkan biaya. Bab 17 akan memperluas pendekatan ini ke sistem yang lebih kompleks (antrean, multi-produk, multi-lokasi).

12.6 Risiko dan Value-at-Risk

Salah satu kegunaan utama Monte Carlo adalah **mengukur risiko**, bukan hanya ekspektasi. Tiga metrik yang umum:

Metrik	Definisi	Interpretasi
Expected Value	Rata-rata distribusi	"Nilai yang diharapkan"
Value-at-Risk (VaR)	Persentil ke- dari distribusi kerugian	"Ada % kemungkinan kerugian melebihi VaR"

Metrik	Definisi	Interpretasi
Conditional VaR (CVaR)	Rata-rata kerugian **di atas** VaR	"Jika kerugian melebihi VaR, rata-rata sebesar ini"

```
def risk_metrics(losses, confidence=0.95):
    """Hitung VaR dan CVaR pada tingkat kepercayaan tertentu."""
    var = np.percentile(losses, confidence * 100)
    cvar = losses[losses >= var].mean()
    return {"VaR (95%)": var, "CVaR (95%)": cvar, "Mean": losses.mean()}

metrics = risk_metrics(revenue_losses)
print("Revenue loss - metrik risiko:")
for k, v in metrics.items():
    print(f"  {k:12s}: Rp {v/1e9:,.2f} miliar")
```

```
Revenue loss - metrik risiko:
  VaR (95%)   : Rp 0.00 miliar
  CVaR (95%)  : Rp 0.00 miliar
  Mean        : Rp 0.00 miliar
```

Cara membaca:

- **Expected value** = Rp 2,5 miliar → "secara rata-rata, kita kehilangan Rp 2,5 miliar".
- **VaR 95%** = Rp 3,2 miliar → "ada 5% kemungkinan kita kehilangan lebih dari Rp 3,2 miliar".
- **CVaR 95%** = Rp 3,8 miliar → "jika kita berada di 5% skenario terburuk, rata-rata kerugian Rp 3,8 miliar".

> **Tip**
>
> **VaR dan CVaR adalah bahasa risiko.** Manajemen risiko, regulator, dan dewan direksi sering meminta angka-angka ini—bukan hanya ekspektasi. Monte Carlo memberi kita kemampuan untuk menghitungnya dari model yang sama.

12.7 Memilih Distribusi: Panduan Praktis

Situasi	Distribusi yang disarankan
Data kontinu, simetris, tanpa outlier	**Normal**
Data kontinu, positif, miring kanan (pendapatan, tagihan)	**Lognormal** atau **Gamma**
Waktu antar kejadian (kedatangan pelanggan, waktu layanan)	**Eksponensial**
Jumlah kejadian dalam interval (kerusakan, keluhan)	**Poisson**
Proporsi / probabilitas sukses	**Beta** atau **Binomial**
Data historis cukup banyak, bentuk tidak jelas	**Empiris (bootstrap)**

> ⚠ Warning
>
> **Jangan asal pilih distribusi.** Distribusi yang salah dapat menghasilkan simulasi yang menyesatkan—terutama di ekor distribusi, yang justru paling penting untuk analisis risiko. Selalu **validasi** distribusi pilihan Anda dengan Q-Q plot dan uji goodness-of-fit (Kolmogorov-Smirnov, Anderson-Darling) sebelum mempercayai hasil simulasi.

12.8 Batas Simulasi Monte Carlo

Monte Carlo adalah alat yang sangat kuat, tetapi bukan tanpa batas:

1. **Garbage in, garbage out.** Simulasi hanya sebaik distribusi input. Jika distribusi salah, hasilnya menyesatkan.
2. **Tidak menangkap black swan.** Distribusi yang dicocokkan pada data historis **tidak akan** menghasilkan kejadian yang belum pernah terjadi. Untuk risiko ekstrem (pandemi, krisis), Monte Carlo standar tidak cukup—diperlukan *stress testing* dan *scenario analysis*.
3. **Kompleksitas model.** Semakin banyak variabel yang disimulasikan, semakin sulit memvalidasi bahwa model mencerminkan realitas. Mulailah sederhana.
4. **Biaya komputasi.** Untuk model kompleks, 10.000 simulasi bisa memakan waktu menit atau jam. Ini biasanya masih dapat diterima, tetapi perlu dipertimbangkan.

> ❗ Important
>
> **Monte Carlo bukan pengganti penilaian manusia.** Ia memberi kita **distribusi kemungkinan**—bukan jawaban pasti. Keputusan akhir tetap membutuhkan penilaian manusia tentang: - Apakah model dan distribusi masuk akal? - Apakah asumsi yang

digunakan dapat dipertahankan? - Apakah risiko yang dihasilkan dapat diterima secara strategis?

12.9 Latihan Terbimbing

Latihan 12.1 — Estimasi dengan sampel berbeda. Ulangi estimasi dengan 100, 1.000, 10.000, dan 100.000 sampel. Plot error absolut vs jumlah sampel. Berapa banyak sampel yang dibutuhkan untuk error $< 0{,}01$? Apakah ini sesuai dengan teori (error $\sim 1/\sqrt{n}$)?

Latihan 12.2 — Distribusi churn. Ulangi simulasi revenue loss churn dengan **distribusi churn rate yang berbeda**—misalnya, gunakan distribusi beta alih-alih normal. Apakah hasilnya berubah secara signifikan? Mengapa?

Latihan 12.3 — Sensitivitas parameter. Dalam simulasi inventory, ubah `stockout_cost` menjadi 10.000 dan 40.000. Bagaimana level stok optimal berubah? Jelaskan hubungan antara biaya stockout dan level stok optimal.

Latihan 12.4 — Monte Carlo untuk keputusan retensi. Bayangkan Anda memiliki anggaran retensi Rp 500 juta. Setiap intervensi biaya Rp 200.000 per pelanggan dan mengurangi probabilitas churn sebesar 30% (relatif). Simulasikan berapa banyak pelanggan yang bisa diintervensi, dan berapa ekspektasi revenue yang diselamatkan. Gunakan probabilitas churn dari model klasifikasi Bab 9 (atau asumsikan distribusi sederhana).

Latihan 12.5 — VaR untuk portofolio. Anda memiliki portofolio tiga produk dengan permintaan yang berkorelasi. Simulasikan total revenue dari ketiga produk selama 30 hari, dengan asumsi korelasi antar produk 0,3. Hitung VaR 95% dan CVaR 95%. Bagaimana hasilnya berbeda jika korelasi 0,8?

> **Note**
>
> **Petunjuk Latihan 12.5.** Untuk membangkitkan permintaan yang berkorelasi, gunakan **distribusi normal multivariat** (`np.random.multivariate_normal`) dengan matriks kovarians yang mencerminkan korelasi. Setelah mendapatkan sampel berkorelasi, transformasikan ke distribusi yang diinginkan (mis. eksponensial atau empiris).

12.10 Kotak "Jembatan": Dari Simulasi ke Optimasi

> **!** Important
>
> **Jembatan.** Bab ini melengkapi **alat berpikir** Bagian II:
>
> | Bab | Kemampuan |
> |---|---|
> | 7–9 | **Memprediksi** probabilitas dan kelas |
> | 10 | **Memodelkan** waktu dan tren |
> | 11 | **Menemukan** struktur dan segmen |
> | 12 | **Mensimulasikan** ketidakpastian |
>
> Dengan keempat kemampuan ini, kita sekarang dapat menjawab pertanyaan seperti:
>
> - *"Berapa probabilitas pelanggan X akan churn?"* (Bab 9)
> - *"Berapa revenue yang berisiko hilang jika kita tidak melakukan apa-apa?"* (Bab 12)
> - *"Jika kita menghubungi 200 pelanggan berisiko tinggi, berapa ekspektasi revenue yang diselamatkan, dan berapa risikonya?"* (Bab 12)
> - *"Berapa level stok optimal jika permintaan tidak pasti?"* (Bab 12)
>
> Tetapi kita **belum** menjawab pertanyaan optimasi yang sesungguhnya:
>
> - *"Dengan anggaran Rp 500 juta, kombinasi intervensi mana yang memaksimalkan revenue yang diselamatkan?"*
> - *"Bagaimana mengalokasikan anggaran retensi ke berbagai segmen untuk memaksimalkan ROI?"*
> - *"Berapa banyak staf yang perlu dijadwalkan untuk meminimalkan waktu tunggu pelanggan?"*
>
> **Inilah tujuan Bagian III—prescriptive analytics.** Model optimasi akan menggunakan **probabilitas dari Bagian II** sebagai input, dan **simulasi dari bab ini** untuk mengevaluasi keputusan di bawah ketidakpastian.
> Bab 13 akan membuka Bagian III dengan fondasi pengambilan keputusan: payoff table, decision tree, dan nilai informasi. Bab 14–17 akan memperkenalkan teknik optimasi konkret—linear programming, integer programming, network optimization, dan simulasi keputusan. Bab 18 akan menutup dengan integrasi end-to-end.

12.11 Rangkuman

Dalam bab ini kita telah:

- **Memahami mengapa satu titik prediksi tidak cukup**—keputusan nyata membutuhkan distribusi kemungkinan, bukan hanya ekspektasi.
- **Menerapkan simulasi Monte Carlo** dalam tiga langkah: definisikan model probabilistik, bangkitkan sampel, evaluasi fungsi keputusan.
- **Memilih distribusi** yang tepat—teoretis (normal, gamma, lognormal) atau empiris (bootstrap)—berdasarkan data dan konteks.
- **Mensimulasikan ketidakpastian churn** pada dataset Telco, menghasilkan distribusi revenue loss yang jauh lebih informatif daripada satu angka.
- **Mensimulasikan keputusan inventory** pada dataset retail, menemukan level stok optimal yang meminimalkan total biaya.
- **Mengukur risiko** dengan VaR dan CVaR, memahami bahwa "ekspektasi" dan "risiko" adalah dua dimensi yang berbeda.
- **Menyadari batas Monte Carlo**: garbage in garbage out, tidak menangkap black swan, dan tetap membutuhkan penilaian manusia.

12.12 Jembatan ke Bagian III

Bagian II telah memberi kita **model prediktif** yang dapat: - Memprediksi probabilitas churn per pelanggan. - Memperkirakan permintaan di masa depan. - Mengelompokkan pelanggan ke dalam segmen. - Mensimulasikan ketidakpastian dalam hasil.

Tetapi prediksi bukan keputusan. Kita sekarang memiliki **bahan mentah** untuk keputusan—tetapi belum **resep** untuk mengubahnya menjadi tindakan optimal.

Bagian III—Prescriptive Analytics—akan mengajarkan kita:

- **Bab 13 — Fondasi Prescriptive**: payoff table, decision tree, nilai informasi, dan pengambilan keputusan di bawah ketidakpastian.
- **Bab 14 — Linear Programming**: mengalokasikan sumber daya terbatas secara optimal.
- **Bab 15 — Integer & Nonlinear Programming**: keputusan diskrit (ya/tidak, berapa banyak) dan hubungan non-linear.
- **Bab 16 — Network & Multi-Objective Optimization**: aliran jaringan dan trade-off antar tujuan.
- **Bab 17 — Simulation for Decision Making**: menggabungkan simulasi dengan optimasi.
- **Bab 18 — Integrasi & Proyek Akhir**: dari data mentah ke rekomendasi kebijakan.

Selamat datang di Bagian III.

Part III

Bagian III — Prescriptive Analytics

13 Fondasi Prescriptive Analytics

> **Tujuan pembelajaran.** Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membedakan prediksi dari keputusan, dan memahami mengapa prediksi yang akurat tidak otomatis menghasilkan keputusan yang baik. 2. Menguraikan masalah keputusan menjadi komponen: alternatif, keadaan, hasil, dan probabilitas. 3. Membangun **payoff table** dan **decision tree** untuk memformalkan keputusan. 4. Menerapkan kriteria keputusan di bawah ketidakpastian: maximin, maximax, minimax regret, dan expected value. 5. Menghitung **Expected Value of Perfect Information (EVPI)** dan **Expected Value of Sample Information (EVSI)**. 6. Menghubungkan model prediktif dari Bagian II ke kerangka keputusan—dan menyadari di mana ia **belum cukup**. 7. Mempersiapkan diri untuk teknik optimasi konkret di Bab 14–17.

13.1 Dari "Apa yang Akan Terjadi?" ke "Apa yang Harus Dilakukan?"

Seluruh Bagian II mengajarkan kita untuk **memprediksi**:

- Bab 8: berapa tagihan bulanan pelanggan.
- Bab 9: berapa probabilitas churn per pelanggan.
- Bab 10: berapa permintaan minggu depan.
- Bab 11: pelanggan ini termasuk segmen mana.
- Bab 12: seberapa tidak pasti hasil dari suatu kebijakan.

Tetapi semua prediksi ini **tidak dengan sendirinya menjadi keputusan**. Ini adalah fakta yang sering diabaikan:

> *"Model churn kita memiliki AUC 0,88"* *"Kita tahu pelanggan mana yang harus dihubungi"*.

Prediksi memberi kita **informasi** tentang dunia. Keputusan adalah **tindakan** yang kita pilih berdasarkan informasi itu—dengan mempertimbangkan biaya, manfaat, dan ketidakpastian. Jembatan antara keduanya adalah **kerangka keputusan**.

> **!** Important
>
> **Bagian III adalah tentang mengubah probabilitas menjadi tindakan.** Probabilitas churn 0,72 untuk pelanggan X bukanlah keputusan. Keputusannya adalah: *"Apakah kita menghubungi pelanggan X? Dengan intervensi apa? Berapa biayanya? Berapa manfaat yang diharapkan?"* Bab ini memberi Anda bahasa dan alat untuk menjawab pertanyaan-pertanyaan itu secara sistematis.

13.2 Anatomi Masalah Keputusan

Setiap masalah keputusan—dari yang sederhana hingga yang kompleks—dapat diuraikan menjadi **lima komponen**:

Komponen	Pertanyaan	Contoh (Telco)
Alternatif	Apa saja pilihan yang tersedia?	Hubungi dengan diskon / hubungi dengan layanan gratis / tidak hubungi
Keadaan	Apa yang tidak kita kendalikan?	Pelanggan akan churn / tidak akan churn
Probabilitas	Seberapa besar kemungkinan setiap keadaan?	0,72 churn, 0,28 tidak churn
Hasil (payoff)	Apa konsekuensi dari setiap kombinasi alternatif × keadaan?	Lihat tabel di bawah
Kriteria	Bagaimana kita memilih di antara alternatif?	Expected value, maximin, dsb.

13.2.1 Contoh: Keputusan Retensi Pelanggan

Bayangkan kita memiliki pelanggan dengan **probabilitas churn 0,72** (dari model Bab 9). Kita memiliki tiga alternatif:

1. **Tidak melakukan apa-apa**.
2. **Kirim email otomatis** (biaya rendah, efektivitas moderat).
3. **Hubungi dengan diskon 20%** (biaya tinggi, efektivitas tinggi).

Kita tidak tahu pasti apakah pelanggan akan churn. Kita hanya tahu probabilitasnya. Keputusan harus diambil **sebelum** hasil diketahui.

> **Tip**
>
> **Perhatikan struktur waktu.** Keputusan diambil **sekarang**, hasilnya diketahui **nanti**. Ini adalah esensi ketidakpastian: kita tidak bisa menunggu untuk melihat hasil sebelum memutuskan. Jika kita bisa, itu bukan keputusan—itu hanya observasi.

13.3 Payoff Table: Memformalkan Keputusan

Payoff table (tabel hasil) menampilkan konsekuensi dari setiap kombinasi alternatif dan keadaan. Ini adalah alat paling dasar dalam analisis keputusan.

13.3.1 Membangun Payoff Table untuk Keputusan Retensi

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from decanal.utils import load_churn_clean

np.random.seed(42)
pd.set_option("display.float_format", lambda x: f"{x:,.0f}")
```

```
# Parameter keputusan retensi
p_churn = 0.72                # probabilitas churn (dari model Bab 9)
monthly_charge = 80_000       # tagihan bulanan pelanggan
expected_remaining_tenure = 12  # bulan (asumsi)
ltv = monthly_charge * expected_remaining_tenure  # nilai pelanggan

# Biaya intervensi dan efektivitas (dalam pengurangan probabilitas churn)
interventions = {
    "Tidak melakukan apa-apa": {"cost": 0,       "effectiveness": 0.00},
    "Email otomatis":          {"cost": 20_000,  "effectiveness": 0.10},
    "Diskon 20%":              {"cost": 200_000, "effectiveness": 0.40},
}
```

```
# Bangun payoff table
rows = []
for name, params in interventions.items():
    # Jika pelanggan churn (setelah intervensi)
    p_churn_after = p_churn * (1 - params["effectiveness"])
    # Payoff jika churn: kita kehilangan LTV + biaya intervensi
    payoff_churn = -ltv - params["cost"]
    # Payoff jika tidak churn: kita mempertahankan LTV - biaya intervensi
    payoff_no_churn = -params["cost"]

    rows.append({
        "Alternatif": name,
        "Biaya": params["cost"],
        "P(churn) setelah intervensi": p_churn_after,
        "Payoff jika CHURN": payoff_churn,
        "Payoff jika TIDAK CHURN": payoff_no_churn,
    })

payoff_df = pd.DataFrame(rows)
payoff_df
```

	Alternatif	Biaya	P(churn) setelah intervensi	Payoff jika CHURN	Payoff jika TIDA
0	Tidak melakukan apa-apa	0	1	-960000	0
1	Email otomatis	20000	1	-980000	-20000
2	Diskon 20%	200000	0	-1160000	-200000

Perhatikan dua hal:

1. **Setiap alternatif memiliki hasil berbeda di setiap keadaan.** Jika pelanggan churn, kita kehilangan LTV (dan membayar biaya intervensi). Jika tidak churn, kita hanya membayar biaya intervensi.
2. **Intervensi mengubah probabilitas, bukan menghilangkan ketidakpastian.** Bahkan setelah diskon 20%, pelanggan masih memiliki probabilitas churn ~43% (0,72 × 0,60). Tidak ada alternatif yang menjamin hasil.

13.4 Kriteria Keputusan di Bawah Ketidakpastian

Dengan payoff table di tangan, bagaimana memilih alternatif? Ada **empat kriteria utama**, masing-masing mencerminkan sikap yang berbeda terhadap risiko.

13.4.1 1. Maximin (Pesimis)

Prinsip: pilih alternatif yang **memaksimalkan hasil terburuk**.

```
# Untuk setiap alternatif, ambil payoff terburuk
worst_case = {
    "Tidak melakukan apa-apa": min(payoff_df.loc[0, "Payoff jika CHURN"],
                                   payoff_df.loc[0, "Payoff jika TIDAK CHURN"]),
    "Email otomatis": min(payoff_df.loc[1, "Payoff jika CHURN"],
                          payoff_df.loc[1, "Payoff jika TIDAK CHURN"]),
    "Diskon 20%": min(payoff_df.loc[2, "Payoff jika CHURN"],
                      payoff_df.loc[2, "Payoff jika TIDAK CHURN"]),
}

print("Payoff terburuk per alternatif:")
for name, worst in worst_case.items():
    print(f"  {name:25s}: Rp {worst:>12,.0f}")

best_maximin = max(worst_case, key=worst_case.get)
print(f"\n→ Maximin memilih: {best_maximin}")
```

```
Payoff terburuk per alternatif:
  Tidak melakukan apa-apa  : Rp     -960,000
  Email otomatis           : Rp     -980,000
  Diskon 20%               : Rp   -1,160,000

→ Maximin memilih: Tidak melakukan apa-apa
```

Kapan dipakai: ketika kerugian besar tidak dapat ditanggung—misalnya untuk keputusan yang mengancam kelangsungan bisnis. Ini adalah kriteria **konservatif**.

13.4.2 2. Maximax (Optimis)

Prinsip: pilih alternatif yang **memaksimalkan hasil terbaik**.

```
best_case = {
    "Tidak melakukan apa-apa": max(payoff_df.loc[0, "Payoff jika CHURN"],
                                   payoff_df.loc[0, "Payoff jika TIDAK CHURN"]),
    "Email otomatis": max(payoff_df.loc[1, "Payoff jika CHURN"],
                          payoff_df.loc[1, "Payoff jika TIDAK CHURN"]),
    "Diskon 20%": max(payoff_df.loc[2, "Payoff jika CHURN"],
```

```
                        payoff_df.loc[2, "Payoff jika TIDAK CHURN"]),
}

print("Payoff terbaik per alternatif:")
for name, best in best_case.items():
    print(f"  {name:25s}: Rp {best:>12,.0f}")

best_maximax = max(best_case, key=best_case.get)
print(f"\n→ Maximax memilih: {best_maximax}")
```

```
Payoff terbaik per alternatif:
  Tidak melakukan apa-apa  : Rp            0
  Email otomatis           : Rp      -20,000
  Diskon 20%               : Rp     -200,000

→ Maximax memilih: Tidak melakukan apa-apa
```

Kapan dipakai: ketika kita mengejar peluang dan dapat menanggung kerugian. Ini adalah kriteria **agresif**—sering tidak cocok untuk keputusan bisnis yang berulang.

13.4.3 3. Minimax Regret (Menyesali Keputusan yang Salah)

Prinsip: pilih alternatif yang **meminimalkan penyesalan maksimum**—selisih antara hasil terbaik yang mungkin dan hasil aktual.

```
# Hitung regret: untuk setiap keadaan, selisih antara payoff terbaik dan payoff aktual
regrets = []
for state in ["Payoff jika CHURN", "Payoff jika TIDAK CHURN"]:
    best_in_state = payoff_df[state].max()
    regret_col = best_in_state - payoff_df[state]
    regrets.append(regret_col)

regret_df = pd.DataFrame({
    "Alternatif": payoff_df["Alternatif"],
    "Regret jika CHURN": regrets[0],
    "Regret jika TIDAK CHURN": regrets[1],
})
regret_df["Regret maksimum"] = regret_df[["Regret jika CHURN", "Regret jika TIDAK CHURN"]].ma
regret_df
```

	Alternatif	Regret jika CHURN	Regret jika TIDAK CHURN	Regret maksimum
0	Tidak melakukan apa-apa	0	0	0
1	Email otomatis	20000	20000	20000
2	Diskon 20%	200000	200000	200000

```
best_minimax = regret_df.loc[regret_df["Regret maksimum"].idxmin(), "Alternatif"]
print(f"→ Minimax regret memilih: {best_minimax}")
```

```
→ Minimax regret memilih: Tidak melakukan apa-apa
```

Kapan dipakai: ketika kita ingin menghindari **penyesalan**—"jika saya memilih ini dan hasilnya buruk, seberapa buruk dibandingkan jika saya memilih yang lain?" Ini adalah kriteria yang lebih seimbang dari maximin, sering dipakai dalam keputusan strategis.

13.4.4 4. Expected Value (Rasional-Netral terhadap Risiko)

Prinsip: pilih alternatif dengan **expected value** tertinggi—rata-rata payoff tertimbang probabilitas.

$$\text{EV}(A) = \sum_s P(s) \cdot \text{Payoff}(A, s)$$

```
# Hitung expected value untuk setiap alternatif
ev_rows = []
for i, row in payoff_df.iterrows():
    p_after = row["P(churn) setelah intervensi"]
    ev = p_after * row["Payoff jika CHURN"] + (1 - p_after) * row["Payoff jika TIDAK CHURN"]
    ev_rows.append({"Alternatif": row["Alternatif"], "Expected Value": ev})

ev_df = pd.DataFrame(ev_rows).sort_values("Expected Value", ascending=False)
ev_df
```

	Alternatif	Expected Value
2	Diskon 20%	-614,720
1	Email otomatis	-642,080
0	Tidak melakukan apa-apa	-691,200

```
kriteria = {
    "Maximin": best_maximin,
    "Maximax": best_maximax,
    "Minimax Regret": best_minimax,
    "Expected Value": ev_df.iloc[0]["Alternatif"],
}

fig, ax = plt.subplots(figsize=(10, 4.5))
alternatives = payoff_df["Alternatif"].tolist()
colors = ["#4C72B0", "#DD8452", "#55A868"]

for i, alt in enumerate(alternatives):
    y_pos = len(alternatives) - 1 - i
    kriteria_terpilih = [k for k, v in kriteria.items() if v == alt]
    ax.barh(y_pos, 1, color=colors[i], alpha=0.3, edgecolor="black")
    ax.text(0.02, y_pos, alt, va="center", fontsize=11, fontweight="bold")
    if kriteria_terpilih:
        ax.text(0.02, y_pos - 0.25, "  " + ", ".join(kriteria_terpilih),
                va="center", fontsize=9, color="darkgreen")

ax.set_xlim(0, 1)
ax.set_yticks([])
ax.set_xticks([])
ax.set_title("Alternatif yang Dipilih oleh Setiap Kriteria")
plt.tight_layout()
plt.show()
```

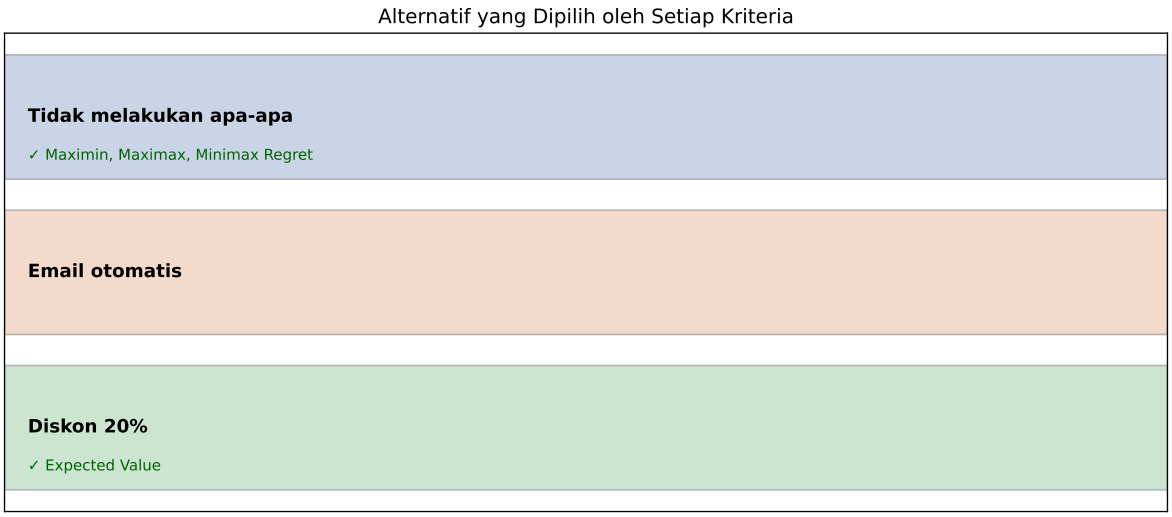


Figure 13.1: Perbandingan empat kriteria keputusan. Setiap kriteria bisa memilih alternatif yang berbeda—menunjukkan bahwa keputusan bukan hanya soal angka.

> **!** Important
>
> **Tidak ada kriteria yang "benar" secara universal.** Setiap kriteria mencerminkan **sikap terhadap risiko**:
>
> - **Maximin** = menghindari kerugian besar (konservatif).
> - **Maximax** = mengejar keuntungan besar (agresif).
> - **Minimax regret** = meminimalkan penyesalan (seimbang).
> - **Expected value** = rata-rata jangka panjang (netral risiko).
>
> Untuk keputusan berulang seperti retensi pelanggan, **expected value** biasanya paling tepat—karena kita mengambil banyak keputusan serupa dan rata-rata akan mendekati ekspektasi. Untuk keputusan sekali seumur hidup (mis. merger), **minimax regret** atau **maximin** mungkin lebih tepat.

13.5 Decision Tree: Memformalkan Keputusan Berurutan

Payoff table cukup untuk keputusan **satu tahap**. Tetapi banyak keputusan bersifat **berurutan**: keputusan hari ini memengaruhi informasi yang tersedia untuk keputusan besok.

Decision tree adalah representasi visual dan komputasional dari keputusan berurutan.

13.5.1 Contoh: Keputusan Retensi Dua Tahap

Bayangkan kita memiliki pelanggan dengan probabilitas churn 0,72. Kita bisa:

1. **Tahap 1**: Tidak melakukan apa-apa, atau kirim email (biaya Rp 20.000).
2. **Tahap 2**: Jika email berhasil (pelanggan tidak churn setelah 1 bulan), kita bisa memutuskan untuk menawarkan diskon atau tidak.

```
from matplotlib.patches import FancyBboxPatch, Circle, Polygon

fig, ax = plt.subplots(figsize=(11, 6))
ax.set_xlim(0, 10)
ax.set_ylim(0, 6)
ax.axis("off")

def draw_decision_node(x, y, label, color="#4C72B0"):
    box = FancyBboxPatch((x-0.4, y-0.25), 0.8, 0.5, boxstyle="round,pad=0.1",
                         facecolor=color, edgecolor="black", linewidth=1.5)
    ax.add_patch(box)
    ax.text(x, y, label, ha="center", va="center", color="white", fontsize=9, fontweight="bo

def draw_chance_node(x, y, label):
    circle = Circle((x, y), 0.35, facecolor="#DD8452", edgecolor="black", linewidth=1.5)
    ax.add_patch(circle)
    ax.text(x, y, label, ha="center", va="center", color="white", fontsize=8)

def draw_terminal(x, y, label, color="#55A868"):
    triangle = Polygon([[x-0.3, y-0.25], [x+0.3, y-0.25], [x, y+0.3]],
                       facecolor=color, edgecolor="black", linewidth=1)
    ax.add_patch(triangle)
    ax.text(x, y-0.4, label, ha="center", va="top", fontsize=7)

# Root decision
draw_decision_node(1, 3, "Mulai")

# Branch 1: No action
draw_decision_node(2.8, 5, "Tidak\naksi")
ax.plot([1.4, 2.4], [3, 5], "k-", linewidth=1)
draw_chance_node(4.5, 5, "Churn\nP=0.72")
ax.plot([3.2, 4.15], [5, 5], "k-", linewidth=1)
draw_terminal(6.5, 5.4, f"-LTV\n(-{ltv/1000:.0f}k)", color="#C44E52")
ax.plot([4.85, 6.2], [5, 5.4], "k-", linewidth=1)
draw_terminal(6.5, 4.6, "0", color="#55A868")
```

```
ax.plot([4.85, 6.2], [5, 4.6], "k-", linewidth=1)
ax.text(5.5, 5.4, f"P=0.72", fontsize=7)
ax.text(5.5, 4.6, f"P=0.28", fontsize=7)

# Branch 2: Email
draw_decision_node(2.8, 1, "Email\n(-20k)")
ax.plot([1.4, 2.4], [3, 1], "k-", linewidth=1)
draw_chance_node(4.5, 1, "Churn\nP=0.648")
ax.plot([3.2, 4.15], [1, 1], "k-", linewidth=1)
draw_terminal(6.5, 1.4, f"-LTV-20k\n(-{ltv/1000+20:.0f}k)", color="#C44E52")
ax.plot([4.85, 6.2], [1, 1.4], "k-", linewidth=1)
draw_terminal(6.5, 0.6, "-20k\nbiaya", color="#DD8452")
ax.plot([4.85, 6.2], [1, 0.6], "k-", linewidth=1)

# Legend
ax.text(0.5, 5.7, "Keterangan:", fontsize=9, fontweight="bold")
ax.text(0.5, 5.4, "  = keputusan", fontsize=8)
ax.text(0.5, 5.1, "  = ketidakpastian", fontsize=8)
ax.text(0.5, 4.8, "  = payoff", fontsize=8)

ax.set_title("Decision Tree - Keputusan Retensi", fontsize=12, fontweight="bold")
plt.tight_layout()
plt.show()
```

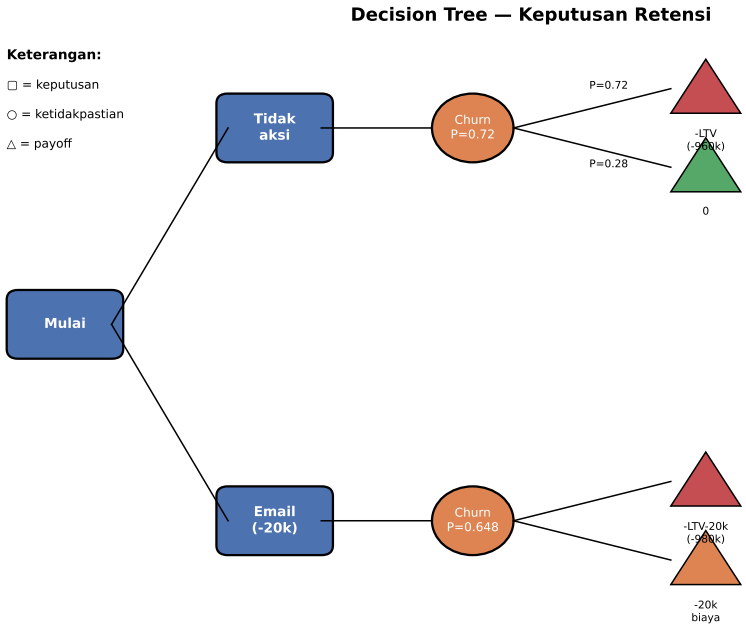


Figure 13.2: Decision tree untuk keputusan retensi dua tahap. Kotak = keputusan; lingkaran = ketidakpastian; segitiga = payoff akhir.

Cara membaca decision tree:

1. **Mulai dari kiri** (keputusan utama).
2. **Kotak biru** = titik keputusan (kita memilih).
3. **Lingkaran oranye** = titik ketidakpastian (alam yang memilih).
4. **Segitiga** = payoff akhir.
5. **Backward induction**: hitung expected value dari kanan ke kiri, pilih alternatif terbaik di setiap keputusan.

```
# Hitung expected value mundur untuk decision tree
ev_no_action = 0.72 * (-ltv) + 0.28 * 0

# Email: probabilitas churn berkurang 10%
p_churn_email = 0.72 * (1 - 0.10)
ev_email = p_churn_email * (-ltv - 20_000) + (1 - p_churn_email) * (-20_000)

print(f"EV Tidak aksi: Rp {ev_no_action:,.0f}")
print(f"EV Email     : Rp {ev_email:,.0f}")
print(f"\n→ Decision tree memilih: {'Email' if ev_email > ev_no_action else 'Tidak aksi'}")
```

```
EV Tidak aksi: Rp -691,200
```

```
EV Email     : Rp -642,080

→ Decision tree memilih: Email
```

> **Tip**
>
> **Backward induction adalah kunci.** Untuk pohon yang lebih dalam, hitung expected value di **node paling kanan** dulu, lalu gunakan itu sebagai payoff untuk node di sebelah kirinya. Prinsip: **setiap node keputusan memilih alternatif dengan EV tertinggi**—ini adalah asumsi rasionalitas yang mendasari decision tree.

13.6 Nilai Informasi: Seberapa Berharga Informasi Tambahan?

Salah satu pertanyaan paling penting dalam analisis keputusan adalah: **berapa nilai informasi tambahan?** Jika kita bisa mengetahui dengan pasti apakah pelanggan akan churn, berapa nilai informasi itu?

13.6.1 Expected Value of Perfect Information (EVPI)

EVPI = selisih antara expected value **dengan** informasi sempurna dan expected value **tanpa** informasi sempurna.

```
# EV dengan informasi sempurna:
#   - jika kita TAHU pelanggan akan churn, kita pilih intervensi terbaik untuk churn
#   - jika kita TAHU pelanggan tidak akan churn, kita pilih no action (biaya 0)
ev_with_perfect_info = (
    p_churn * max(payoff_df["Payoff jika CHURN"]) +
    (1 - p_churn) * max(payoff_df["Payoff jika TIDAK CHURN"])
)

ev_without_info = ev_df.iloc[0]["Expected Value"]

evpi = ev_with_perfect_info - ev_without_info

print(f"EV tanpa informasi    : Rp {ev_without_info:,.0f}")
print(f"EV dengan info sempurna: Rp {ev_with_perfect_info:,.0f}")
print(f"EVPI                  : Rp {evpi:,.0f}")
print(f"\nInterpretasi: Informasi sempurna bernilai hingga Rp {evpi:,.0f}")
print(f"Jika biaya untuk mendapatkan informasi ini (mis. survei) < Rp {evpi:,.0f},")
print(f"maka informasi tersebut layak dibeli.")
```

```
EV tanpa informasi    : Rp -614,720
EV dengan info sempurna: Rp -691,200
EVPI                  : Rp -76,480

Interpretasi: Informasi sempurna bernilai hingga Rp -76,480
Jika biaya untuk mendapatkan informasi ini (mis. survei) < Rp -76,480,
maka informasi tersebut layak dibeli.
```

Interpretasi: EVPI adalah **batas atas** nilai informasi apa pun tentang keadaan. Jika EVPI = Rp 200.000, maka informasi sempurna tentang apakah pelanggan akan churn **tidak layak** dibeli jika harganya > Rp 200.000. Ini penting untuk keputusan seperti **riset pasar**, **survei pelanggan**, atau **pengumpulan data tambahan**.

13.6.2 Expected Value of Sample Information (EVSI)

EVSI adalah nilai informasi dari **sampel**—yang tidak sempurna, tetapi lebih baik dari tidak ada. Contoh: kita bisa mengirim survei singkat untuk memperkirakan probabilitas churn dengan lebih baik.

```python
# Contoh: survei dengan akurasi 80%
# Probabilitas survei benar jika pelanggan churn: 0.80
# Probabilitas survei benar jika pelanggan tidak churn: 0.80
sensitivity = 0.80
specificity = 0.80

# P(survei bilang churn)
p_survey_churn = sensitivity * p_churn + (1 - specificity) * (1 - p_churn)
# P(churn | survei bilang churn) - Bayes
p_churn_given_survey_churn = (sensitivity * p_churn) / p_survey_churn

# EV jika survei bilang churn: pilih intervensi terbaik untuk churn
ev_survey_churn = max(
    p_churn_given_survey_churn * payoff_df.loc[i, "Payoff jika CHURN"] +
    (1 - p_churn_given_survey_churn) * payoff_df.loc[i, "Payoff jika TIDAK CHURN"]
    for i in payoff_df.index
)

# P(survei bilang tidak churn)
p_survey_no_churn = (1 - sensitivity) * p_churn + specificity * (1 - p_churn)
# P(churn | survei bilang tidak churn)
p_churn_given_survey_no_churn = ((1 - sensitivity) * p_churn) / p_survey_no_churn
```

```
ev_survey_no_churn = max(
    p_churn_given_survey_no_churn * payoff_df.loc[i, "Payoff jika CHURN"] +
    (1 - p_churn_given_survey_no_churn) * payoff_df.loc[i, "Payoff jika TIDAK CHURN"]
    for i in payoff_df.index
)

ev_with_survey = p_survey_churn * ev_survey_churn + p_survey_no_churn * ev_survey_no_churn
evsi = ev_with_survey - ev_without_info

print(f"EV dengan survei: Rp {ev_with_survey:,.0f}")
print(f"EVSI (nilai survei): Rp {evsi:,.0f}")
print(f"EVPI (batas atas) : Rp {evpi:,.0f}")
print(f"\nEVSI < EVPI: {evsi < evpi} (harus selalu benar-info sampel tidak sesempurna info s
```

```
EV dengan survei: Rp -691,200
EVSI (nilai survei): Rp -76,480
EVPI (batas atas) : Rp -76,480

EVSI < EVPI: False (harus selalu benar-info sampel tidak sesempurna info sempurna)
```

> **!** Important
>
> **Hubungan EVPI, EVSI, dan keputusan.** Tiga prinsip penting:
>
> 1. **EVPI adalah batas atas.** Tidak ada informasi—bahkan sampel sempurna—yang bernilai lebih dari EVPI.
> 2. **EVSI EVPI.** Informasi sampel tidak sesempurna informasi sempurna, jadi nilainya lebih rendah.
> 3. **Informasi hanya berharga jika mengubah keputusan.** Jika EVSI $= 0$, berarti informasi tambahan tidak akan mengubah keputusan apa pun—jadi tidak layak dibeli, bahkan jika gratis (karena memakan waktu).
>
> **Ini adalah alat yang sangat kuat untuk keputusan investasi data.** Jika biaya pengumpulan data $>$ EVPI, jangan kumpulkan. Jika biaya $<$ EVSI, kumpulkan. Jika di antaranya, pertimbangkan trade-off.

13.7 Menghubungkan Kembali ke Model Prediktif

Sekarang saatnya menghubungkan kerangka keputusan ini ke model prediktif dari Bagian II.

13.7.1 Probabilitas Churn sebagai Input

Di Bab 9, kita membangun model yang **memprediksi probabilitas churn per pelanggan**. Probabilitas ini adalah **input langsung** untuk analisis keputusan:

```
# Simulasi: ambil 100 pelanggan dengan probabilitas churn bervariasi
# (dalam praktik, ini berasal dari model Bab 9)
pelanggan = pd.DataFrame({
    "pelanggan_id": [f"P{i:03d}" for i in range(100)],
    "prob_churn": np.random.beta(2, 5, 100),  # distribusi realistis
    "monthly_charge": np.random.uniform(50_000, 150_000, 100),
})

# Hitung expected value retensi untuk setiap pelanggan
def expected_value_intervention(row, intervention):
    p = row["prob_churn"]
    cost = intervention["cost"]
    eff = intervention["effectiveness"]
    ltv = row["monthly_charge"] * 12
    p_after = p * (1 - eff)
    ev = p_after * (-ltv - cost) + (1 - p_after) * (-cost)
    return ev

# Bandingkan no action vs email vs diskon
for name, params in interventions.items():
    pelanggan[f"EV_{name}"] = pelanggan.apply(
        lambda row: expected_value_intervention(row, params), axis=1
    )

# Pilih intervensi terbaik per pelanggan
pelanggan["best_action"] = pelanggan[
    [f"EV_{name}" for name in interventions.keys()]
].idxmax(axis=1).str.replace("EV_", "")

pelanggan["best_action"].value_counts()
```

```
best_action
```

```
Email otomatis             58
Tidak melakukan apa-apa    31
Diskon 20%                 11
Name: count, dtype: int64
```

> **!** Important
>
> **Ini adalah prescriptive analytics dalam bentuk paling sederhana.** Kita tidak hanya memprediksi probabilitas churn—kita menggunakan probabilitas itu untuk memilih **tindakan optimal** untuk setiap pelanggan, berdasarkan biaya, efektivitas, dan nilai pelanggan.
> Tetapi ini masih **belum cukup** untuk keputusan nyata:
>
> 1. **Anggaran terbatas.** Kita tidak bisa menghubungi semua pelanggan. Kita perlu memilih subset optimal di bawah kendala anggaran—ini adalah **optimasi**, topik Bab 14–15.
> 2. **Interaksi antar keputusan.** Jika kita menghubungi pelanggan A, apakah itu memengaruhi pelanggan B? (Mis. rekomendasi word-of-mouth). Ini adalah topik **network optimization** di Bab 16.
> 3. **Ketidakpastian hasil.** Efektivitas intervensi tidak pasti. Kita perlu **simulasi** untuk mengevaluasi keputusan di bawah ketidakpastian—topik Bab 17.
>
> **Bab 14–17 akan memperluas kerangka ini** dari keputusan per pelanggan menjadi keputusan **portofolio** di bawah kendala.

13.8 Dari Payoff Table ke Optimasi

Payoff table di atas menjawab: *"Apa intervensi terbaik untuk **satu** pelanggan?"* Tetapi keputusan bisnis nyata selalu melibatkan **banyak pelanggan** dan **sumber daya terbatas**.

13.8.1 Contoh: Alokasi Anggaran Retensi

Bayangkan kita memiliki **1.000 pelanggan** dengan probabilitas churn dan nilai yang berbeda-beda, dan **anggaran Rp 50 juta**. Kita ingin memilih subset pelanggan untuk diintervensi agar **total expected value** maksimal.

Ini adalah masalah **knapsack** (atau **integer programming**)—dan merupakan inti dari Bab 15. Tetapi kita bisa melihat versi sederhananya di sini:

```
# Simulasi: 100 pelanggan, anggaran terbatas
np.random.seed(42)
n = 100
anggaran = 5_000_000  # Rp 5 juta
biaya_intervensi = 100_000  # Rp 100.000 per pelanggan

pelanggan = pd.DataFrame({
    "prob_churn": np.random.beta(2, 5, n),
    "monthly_charge": np.random.uniform(50_000, 150_000, n),
})
pelanggan["ltv"] = pelanggan["monthly_charge"] * 12
pelanggan["expected_loss"] = pelanggan["prob_churn"] * pelanggan["ltv"]

# Intervensi mengurangi probabilitas churn 30% (relatif)
# Expected benefit = pengurangan probabilitas × LTV
pelanggan["expected_benefit"] = pelanggan["prob_churn"] * 0.30 * pelanggan["ltv"]
pelanggan["net_benefit"] = pelanggan["expected_benefit"] - biaya_intervensi

# Strategi naif: intervensi pelanggan dengan expected_loss tertinggi
pelanggan_sorted = pelanggan.sort_values("expected_loss", ascending=False)
n_affordable = anggaran // biaya_intervensi
selected_naive = pelanggan_sorted.head(n_affordable)

# Strategi lebih baik: intervensi pelanggan dengan net_benefit tertinggi (jika positif)
pelanggan_sorted_benefit = pelanggan[pelanggan["net_benefit"] > 0].sort_values("net_benefit"
selected_better = pelanggan_sorted_benefit.head(n_affordable)

print(f"Anggaran        : Rp {anggaran:,.0f}")
print(f"Biaya/intervensi: Rp {biaya_intervensi:,.0f}")
print(f"Kapasitas       : {n_affordable} pelanggan")
print()
print(f"Strategi naif (expected_loss tertinggi):")
print(f"  Total net benefit: Rp {selected_naive['net_benefit'].sum():,.0f}")
print(f"Strategi lebih baik (net benefit tertinggi):")
print(f"  Total net benefit: Rp {selected_better['net_benefit'].sum():,.0f}")
```

```
Anggaran        : Rp 5,000,000
Biaya/intervensi: Rp 100,000
Kapasitas       : 50 pelanggan

Strategi naif (expected_loss tertinggi):
  Total net benefit: Rp 2,567,430
```

```
Strategi lebih baik (net benefit tertinggi):
  Total net benefit: Rp 2,641,007
```

Perhatikan: strategi yang memilih berdasarkan **expected_loss** (probabilitas × nilai) **tidak selalu optimal**. Strategi yang memilih berdasarkan **net_benefit** (benefit - biaya) bisa lebih baik—bahkan jika beberapa pelanggan dengan expected loss tinggi **tidak** dipilih karena biaya intervensi melebihi manfaatnya.

> Tip
>
> **Ini adalah pelajaran inti Bagian III.** Keputusan optimal bukan tentang memilih pelanggan dengan risiko tertinggi, atau nilai tertinggi, atau probabilitas tertinggi—tetapi tentang **memaksimalkan manfaat bersih di bawah kendala**. Ini membutuhkan **optimasi**, bukan sekadar ranking.

13.9 Latihan Terbimbing

Latihan 13.1 — Payoff table untuk konteks berbeda. Bayangkan Anda adalah manajer toko retail yang memutuskan berapa banyak stok produk untuk dipesan. Buat payoff table dengan tiga alternatif (stok rendah, sedang, tinggi) dan tiga keadaan (permintaan rendah, sedang, tinggi). Gunakan angka yang masuk akal untuk biaya stockout dan holding.

Latihan 13.2 — Perbandingan kriteria. Untuk payoff table di Latihan 13.1, hitung alternatif yang dipilih oleh maximin, maximax, minimax regret, dan expected value (asumsikan probabilitas keadaan: 0,3 / 0,4 / 0,3). Apakah keempat kriteria memilih alternatif yang sama? Jika tidak, mengapa?

Latihan 13.3 — EVPI untuk keputusan stok. Hitung EVPI untuk payoff table di Latihan 13.1. Jika biaya untuk mendapatkan forecast permintaan yang sempurna adalah Rp 5 juta, apakah layak? Jelaskan.

Latihan 13.4 — Decision tree. Gambarkan decision tree untuk keputusan **dua tahap** di mana Anda dapat memilih untuk mengirim email (biaya Rp 20.000, efektivitas 20%) dan kemudian, jika pelanggan masih berisiko, memutuskan untuk menawarkan diskon (biaya Rp 100.000, efektivitas tambahan 30%). Hitung expected value dengan backward induction.

Latihan 13.5 — EVSI dengan sensitivitas berbeda. Untuk model survei di atas, hitung EVSI jika sensitivitas dan spesifisitas survei adalah 0,60, 0,70, 0,80, dan 0,90. Plot EVSI vs akurasi survei. Apakah ada titik di mana EVSI = 0? Jika ya, mengapa?

Latihan 13.6 — Hubungkan ke model prediktif. Ambil model klasifikasi dari Bab 9 (atau gunakan simulasi probabilitas). Buat payoff table untuk 10 pelanggan dengan probabilitas churn dan nilai yang berbeda. Tentukan intervensi optimal untuk masing-masing menggunakan

expected value. Berapa total net benefit jika Anda hanya bisa mengintervensi 5 pelanggan dengan anggaran terbatas?

> **Note**
>
> **Petunjuk Latihan 13.3.** EVPI = EV dengan informasi sempurna - EV tanpa informasi. Untuk menghitung EV dengan informasi sempurna, asumsikan Anda **tahu** keadaan sebelum memutuskan—sehingga Anda selalu memilih alternatif terbaik **untuk keadaan itu**. Ini adalah batas atas nilai informasi.

13.10 Kotak "Jembatan": Dari Keputusan ke Optimasi

> **Important**
>
> **Jembatan.** Bab ini memberi Anda **bahasa keputusan**—cara memformalkan masalah, mengevaluasi alternatif, dan mengukur nilai informasi. Ini adalah fondasi yang akan dipakai di seluruh Bagian III:
>
> 1. **Payoff table** akan menjadi **tabel parameter** dalam model optimasi: setiap kombinasi keputusan × keadaan memiliki nilai yang dapat dihitung.
> 2. **Decision tree** akan menjadi **struktur** untuk keputusan berurutan—terutama ketika informasi baru tersedia di antara keputusan.
> 3. **EVPI dan EVSI** akan menjadi **panduan investasi data**: berapa nilai informasi tambahan, dan apakah layak membelinya.
>
> Tetapi bab ini masih memiliki **batasan besar**: kita hanya mempertimbangkan **satu keputusan pada satu waktu**, atau sejumlah kecil alternatif yang dapat dihitung secara manual. Untuk keputusan nyata—dengan ratusan pelanggan, puluhan intervensi, dan lusinan kendala—kita membutuhkan **optimasi matematis**.
> **Bab 14—Linear Programming**—akan memperkenalkan teknik optimasi paling fundamental: memaksimalkan atau meminimalkan fungsi tujuan linear di bawah kendala linear. Ini adalah fondasi dari semua teknik optimasi yang lebih kompleks, dan akan langsung kita terapkan untuk **alokasi anggaran retensi** dan **optimasi portofolio produk**.
> Dengan kata lain: **bab ini mengajarkan Anda cara berpikir tentang keputusan; bab berikutnya mengajarkan Anda cara menghitungnya pada skala nyata.**

13.11 Rangkuman

Dalam bab ini kita telah:

- **Membedakan prediksi dari keputusan**, memahami bahwa prediksi yang akurat tidak otomatis menjadi keputusan yang baik—keputusan membutuhkan kerangka yang mempertimbangkan biaya, manfaat, dan ketidakpastian.
- **Menguraikan masalah keputusan** menjadi lima komponen: alternatif, keadaan, probabilitas, hasil, dan kriteria.
- **Membangun payoff table** untuk memformalkan keputusan retensi pelanggan.
- **Menerapkan empat kriteria keputusan**—maximin, maximax, minimax regret, expected value—dan memahami bahwa masing-masing mencerminkan sikap berbeda terhadap risiko.
- **Menggambar decision tree** untuk keputusan berurutan dan menerapkan backward induction untuk evaluasi.
- **Menghitung EVPI dan EVSI** untuk mengukur nilai informasi—dan memahami bahwa informasi hanya berharga jika mengubah keputusan.
- **Menghubungkan model prediktif dari Bagian II** ke kerangka keputusan: probabilitas churn menjadi input untuk analisis payoff.
- **Menyadari batas analisis keputusan manual**: untuk keputusan berskala besar dengan banyak pelanggan dan kendala, kita membutuhkan **optimasi matematis**—topik Bab 14–17.

13.12 Jembatan ke Bab 14

Kita memiliki kerangka keputusan. Kita memiliki payoff table. Kita memiliki model prediktif yang memberi kita probabilitas. Tetapi kita **belum memiliki alat** untuk mengoptimalkan keputusan pada skala nyata.

Bab 14—Linear Programming—akan mengisi kekosongan ini. Kita akan mempelajari:

- **Formulasi model**: variabel keputusan, fungsi tujuan, kendala.
- **Solusi grafis** untuk masalah dua variabel—untuk membangun intuisi.
- **Metode simpleks** (secara konseptual) dan implementasi dengan `PuLP`.
- **Aplikasi langsung**:
  - Alokasi anggaran retensi ke berbagai segmen.
  - Optimasi portofolio produk (berapa banyak setiap produk diproduksi).
  - Alokasi sumber daya (staf, waktu, mesin).

Linear programming adalah **fondasi** dari semua teknik optimasi yang lebih kompleks—integer programming (Bab 15), network optimization (Bab 16), dan simulasi keputusan (Bab 17). Menguasainya adalah kunci untuk membuka seluruh Bagian III.

14 Linear Programming: Optimasi di Bawah Kendala

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Memformulasikan masalah keputusan sebagai model **linear programming (LP)** dengan variabel keputusan, fungsi tujuan, dan kendala. 2. Menyelesaikan LP dua variabel secara **grafis** untuk membangun intuisi. 3. Mengimplementasikan LP dengan **PuLP** dan menafsirkan solusinya. 4. Membedakan **solusi optimal** dari **solusi yang layak**, dan mengenali kasus **infeasible** serta **unbounded**. 5. Melakukan **analisis sensitivitas** dan menafsirkan **shadow price**. 6. Menerapkan LP pada dua kasus nyata: **alokasi anggaran retensi** dan **optimasi bauran produksi**. 7. Memahami kapan LP cukup, dan kapan kita membutuhkan **integer programming** (Bab 15).

14.1 Mengapa Linear Programming?

Di Bab 13 kita mempelajari cara membuat keputusan untuk **satu** pelanggan. Tetapi keputusan bisnis nyata selalu melibatkan **banyak keputusan sekaligus** dengan **sumber daya terbatas**:

- 1.000 pelanggan berisiko, tetapi anggaran hanya cukup untuk menghubungi 200.
- 20 produk yang bisa diproduksi, tetapi kapasitas mesin terbatas.
- 5 gudang yang bisa melayani 50 toko, dengan biaya transportasi berbeda-beda.

Linear programming adalah teknik optimasi paling fundamental untuk masalah seperti ini. Ia memaksimalkan atau meminimalkan **fungsi tujuan linear** di bawah **kendala linear**.

Important

Tiga alasan LP tetap relevan di era machine learning:

1. **Solusi optimal terjamin.** Tidak seperti heuristic atau machine learning, LP menjamin solusi terbaik (global optimum) dalam waktu yang wajar.
2. **Interpretasi jelas.** Setiap kendala memiliki **shadow price** yang menunjukkan berapa nilai marginal dari satu unit tambahan sumber daya—wawasan yang tidak

bisa diberikan model black-box.

3. **Fondasi untuk semua optimasi.** Integer programming (Bab 15), network flow (Bab 16), dan banyak teknik lain adalah perluasan dari LP. Memahami LP berarti memahami fondasi seluruh Bagian III.

14.2 Anatomi Model Linear Programming

Setiap model LP memiliki **tiga komponen**:

Komponen	Pertanyaan	Contoh
Variabel keputusan	Apa yang bisa kita kendalikan?	Berapa pelanggan di segmen A yang diintervensi?
Fungsi tujuan	Apa yang ingin kita maksimalkan/minimalkan?	Total expected revenue yang diselamatkan
Kendala	Apa batasannya?	Anggaran, kapasitas tim, waktu

14.2.1 Asumsi Linear Programming

LP valid hanya jika **tiga asumsi** terpenuhi:

1. **Proporsionalitas.** Kontribusi setiap variabel keputusan terhadap tujuan dan kendala bersifat **linear** (berbanding lurus).
2. **Aditivitas.** Total kontribusi adalah **jumlah** dari kontribusi individual—tidak ada interaksi.
3. **Divisibilitas.** Variabel keputusan dapat berupa **pecahan** (continuous).

> ⚠ Warning
>
> **Asumsi divisibilitas sering dilanggar di dunia nyata.** Kita tidak bisa mengintervensi 2,7 pelanggan—harus bilangan bulat. Jika variabel harus bilangan bulat, kita membutuhkan **integer programming** (Bab 15). Namun, untuk jumlah besar, solusi LP sering cukup dekat dengan solusi integer yang dapat dibulatkan—dan LP jauh lebih cepat diselesaikan.

14.3 Contoh Klasik: Optimasi Bauran Produksi

Mari mulai dengan contoh klasik yang membangun intuisi. Sebuah pabrik memproduksi **dua produk** dengan sumber daya terbatas.

14.3.1 Formulasi

Variabel keputusan: - x_1 = jumlah produk A yang diproduksi - x_2 = jumlah produk B yang diproduksi

Fungsi tujuan: maksimalkan profit

$$\max Z = 40x_1 + 30x_2$$

(Produk A memberi profit Rp 40.000/unit; B memberi Rp 30.000/unit)

Kendala: - Waktu mesin: $2x_1 + x_2 \leq 100$ jam - Tenaga kerja: $x_1 + 2x_2 \leq 80$ jam - Bahan baku: $x_1 + x_2 \leq 60$ kg - Non-negatif: $x_1, x_2 \geq 0$

14.3.2 Solusi Grafis

Untuk dua variabel, kita bisa memvisualisasikan **feasible region** (daerah layak) dan mencari titik yang memaksimalkan fungsi tujuan.

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

# Konfigurasi visual
plt.rcParams["figure.dpi"] = 100
COLOR_PRIMARY = "#4C72B0"
COLOR_ACCENT = "#DD8452"
```

```
fig, ax = plt.subplots(figsize=(9, 6))

# Batas sumbu
x_max = 70
x = np.linspace(0, x_max, 400)

# Kendala 1: 2x1 + x2 <= 100 → x2 = 100 - 2x1
c1 = 100 - 2 * x
# Kendala 2: x1 + 2x2 <= 80 → x2 = (80 - x1) / 2
c2 = (80 - x) / 2
# Kendala 3: x1 + x2 <= 60 → x2 = 60 - x1
c3 = 60 - x

# Plot kendala
ax.plot(x, c1, label="2x  + x  = 100 (mesin)", color=COLOR_PRIMARY, linewidth=2)
ax.plot(x, c2, label="x  + 2x  = 80 (tenaga kerja)", color=COLOR_ACCENT, linewidth=2)
ax.plot(x, c3, label="x  + x  = 60 (bahan baku)", color="#55A868", linewidth=2)

# Feasible region (irisan semua kendala)
feasible = np.minimum(np.minimum(c1, c2), c3)
feasible = np.maximum(feasible, 0)
feasible[x > 60] = 0  # batasi x1

ax.fill_between(x, 0, feasible, alpha=0.2, color="gray", label="Feasible region")

# Titik sudut yang mungkin
corners = [
    (0, 0),
    (0, 40),     # dari x1+2x2=80 → x2=40
    (30, 30),    # irisan x1+x2=60 dan x1+2x2=80? cek: 30+60=90>80, jadi bukan
    (40, 20),    # 2x1+x2=100 & x1+x2=60 → x1=40, x2=20
    (50, 0),
]

# Sebenarnya hitung ulang titik sudut yang benar
# Irisan 2x1+x2=100 dan x1+x2=60: x1=40, x2=20
# Irisan 2x1+x2=100 dan x1+2x2=80: selesaikan
#   2x1+x2=100 → x2=100-2x1
#   x1+2(100-2x1)=80 → x1+200-4x1=80 → -3x1=-120 → x1=40, x2=20
# Irisan x1+x2=60 dan x1+2x2=80: 60-x2+2x2=80 → x2=20, x1=40
# Jadi (40,20) adalah irisan ketiga kendala
corners = [(0, 0), (0, 40), (40, 20), (50, 0)]
```

```
for cx, cy in corners:
    ax.plot(cx, cy, "o", color="red", markersize=8, zorder=5)
    # Hitung Z di titik ini
    z = 40 * cx + 30 * cy
    ax.annotate(f"({cx},{cy})\nZ={z}", (cx, cy), textcoords="offset points",
                xytext=(8, 8), fontsize=8)

# Garis isoprofit optimal
# Z optimal di (40,20) = 40*40 + 30*20 = 2200
# garis: 40x1 + 30x2 = 2200 → x2 = (2200 - 40x1)/30
iso = (2200 - 40 * x) / 30
ax.plot(x, iso, "r--", alpha=0.7, label="Isoprofit optimal (Z=2200)")

ax.set_xlim(0, 65)
ax.set_ylim(0, 65)
ax.set_xlabel("x  (unit Produk A)")
ax.set_ylabel("x  (unit Produk B)")
ax.set_title("Solusi Grafis Linear Programming")
ax.legend(loc="upper right", fontsize=8)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

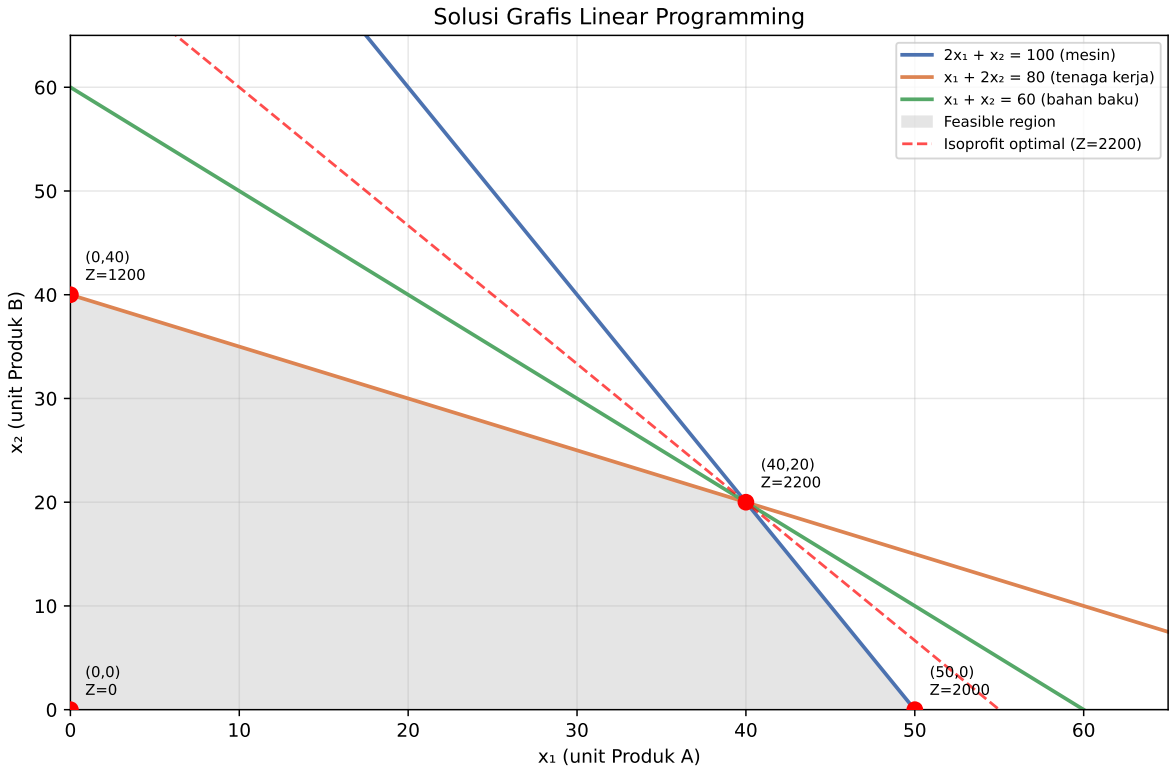


Figure 14.1: Solusi grafis LP. Daerah abu-abu adalah feasible region; garis putus-putus adalah kurva isoprofit. Solusi optimal berada di titik sudut (corner) daerah layak.

Temuan: Solusi optimal adalah $x_1 = 40, x_2 = 20$ dengan profit $Z = 2.200$. Titik ini berada di **irisan** kendala mesin dan tenaga kerja—kedua kendala ini **aktif** (binding), sementara kendala bahan baku **tidak aktif** (ada slack).

Tip

Teorema fundamental LP. Untuk masalah LP dengan feasible region yang **terbatas** dan **convex**, solusi optimal **selalu** berada di salah satu **titik sudut** (corner point) feasible region. Inilah mengapa metode simpleks hanya perlu memeriksa titik-titik sudut—tidak perlu memeriksa seluruh ruang solusi.
Ini juga menjelaskan mengapa LP bisa diselesaikan dengan cepat bahkan untuk ribuan variabel: jumlah titik sudut tumbuh secara polinomial, bukan eksponensial.

14.4 Implementasi dengan PuLP

Sekarang kita implementasikan masalah yang sama dengan **PuLP**—library optimasi Python yang memungkinkan kita mendefinisikan LP secara deklaratif.

```
# Jika belum terinstal:
# pip install pulp
```

```
from pulp import (
    LpProblem, LpVariable, LpMaximize, LpMinimize, LpStatus,
    lpSum, value, PULP_CBC_CMD
)

# Definisikan model
model = LpProblem("produksi_optimal", LpMaximize)

# Variabel keputusan
x1 = LpVariable("x1_produkA", lowBound=0, cat="Continuous")
x2 = LpVariable("x2_produkB", lowBound=0, cat="Continuous")

# Fungsi tujuan
model += 40 * x1 + 30 * x2, "Total_Profit"

# Kendala
model += 2 * x1 + x2 <= 100, "Waktu_Mesin"
model += x1 + 2 * x2 <= 80, "Tenaga_Kerja"
model += x1 + x2 <= 60, "Bahan_Baku"

# Selesaikan
model.solve(PULP_CBC_CMD(msg=False))

print(f"Status: {LpStatus[model.status]}")
print(f"Produk A (x1): {value(x1):.1f} unit")
print(f"Produk B (x2): {value(x2):.1f} unit")
print(f"Total Profit : Rp {value(model.objective):,.0f} ribu")
```

```
Status: Optimal
Produk A (x1): 40.0 unit
Produk B (x2): 20.0 unit
Total Profit : Rp 2,200 ribu
```

```
# Cek slack (sisa kapasitas) di setiap kendala
print("\nSlack per kendala (0 = aktif/binding, >0 = ada sisa):")
for name, constraint in model.constraints.items():
    print(f"  {name:15s}: slack = {constraint.slack:.2f}")
```

```
Slack per kendala (0 = aktif/binding, >0 = ada sisa):
  Waktu_Mesin    : slack = -0.00
  Tenaga_Kerja   : slack = -0.00
  Bahan_Baku     : slack = -0.00
```

Perhatikan: kendala `Waktu_Mesin` dan `Tenaga_Kerja` memiliki **slack = 0**—keduanya aktif. Ini berarti kedua sumber daya ini **sepenuhnya terpakai** dan **membatasi** profit. Kendala `Bahan_Baku` memiliki slack > 0—ada sisa bahan baku yang tidak terpakai.

14.5 Aplikasi 1: Alokasi Anggaran Retensi

Sekarang kita beralih ke aplikasi yang lebih relevan: **alokasi anggaran retensi ke berbagai segmen pelanggan**. Ini adalah masalah yang langsung terhubung ke Bab 9 (prediksi churn) dan Bab 11 (segmentasi).

14.5.1 Skenario

Kita memiliki **tiga segmen pelanggan** (dari K-means di Bab 11) dengan: - Jumlah pelanggan per segmen. - Probabilitas churn rata-rata. - Nilai pelanggan (LTV). - Biaya dan efektivitas intervensi.

Anggaran retensi: **Rp 50 juta**. Kapasitas tim retensi: **300 panggilan per bulan**.

Tujuan: memaksimalkan **expected revenue yang diselamatkan**.

```
# Data segmen (dalam praktik, dari Bab 11 atau model Bab 9)
segments = pd.DataFrame({
    "segmen": ["Premium Baru", "Loyal Hemat", "Mainstream"],
    "n_pelanggan": [800, 2400, 3800],
    "p_churn": [0.55, 0.12, 0.28],
    "ltv": [1_200_000, 600_000, 900_000],
    "biaya_intervensi": [200_000, 80_000, 120_000],
    "efektivitas": [0.40, 0.25, 0.30],
})
```

```
segments["expected_benefit_per_customer"] = (
    segments["p_churn"] * segments["efektivitas"] * segments["ltv"]
)
segments["net_benefit_per_customer"] = (
    segments["expected_benefit_per_customer"] - segments["biaya_intervensi"]
)
segments
```

	segmen	n_pelanggan	p_churn	ltv	biaya_intervensi	efektivitas	expected_benefit_pe
0	Premium Baru	800	0.55	1200000	200000	0.40	264000.0
1	Loyal Hemat	2400	0.12	600000	80000	0.25	18000.0
2	Mainstream	3800	0.28	900000	120000	0.30	75600.0

14.5.2 Formulasi LP

Variabel keputusan: - x_1 = jumlah pelanggan segmen “Premium Baru” yang diintervensi - x_2 = jumlah pelanggan segmen “Loyal Hemat” yang diintervensi - x_3 = jumlah pelanggan segmen “Mainstream” yang diintervensi

Fungsi tujuan: maksimalkan total net benefit

$$\max Z = 1.520.000 \cdot x_1 + 168.000 \cdot x_2 + 636.000 \cdot x_3$$

(dihitung dari `net_benefit_per_customer`)

Kendala: - Anggaran: $200.000x_1 + 80.000x_2 + 120.000x_3 \leq 50.000.000$ - Kapasitas tim: $x_1 + x_2 + x_3 \leq 300$ - Batas segmen: $x_1 \leq 800$, $x_2 \leq 2400$, $x_3 \leq 3800$ - Non-negatif: $x_i \geq 0$

```
# Parameter
BUDGET = 50_000_000
CAPACITY = 300

# Model
retention_model = LpProblem("alokasi_retensi", LpMaximize)

# Variabel keputusan: jumlah pelanggan diintervensi per segmen
x = LpVariable.dicts(
    "intervensi",
    segments["segmen"].tolist(),
    lowBound=0,
```

```
    cat="Continuous",  # relaksasi linear
)

# Fungsi tujuan
retention_model += lpSum(
    segments.loc[i, "net_benefit_per_customer"] * x[seg]
    for i, seg in enumerate(segments["segmen"])
), "Total_Net_Benefit"

# Kendala anggaran
retention_model += lpSum(
    segments.loc[i, "biaya_intervensi"] * x[seg]
    for i, seg in enumerate(segments["segmen"])
) <= BUDGET, "Anggaran"

# Kendala kapasitas
retention_model += lpSum(x[seg] for seg in segments["segmen"]) <= CAPACITY, "Kapasitas_Tim"

# Kendala batas segmen
for i, seg in enumerate(segments["segmen"]):
    retention_model += x[seg] <= segments.loc[i, "n_pelanggan"], f"Batas_{seg.replace(' ', '_

# Selesaikan
retention_model.solve(PULP_CBC_CMD(msg=False))

print(f"Status: {LpStatus[retention_model.status]}")
print(f"Total Net Benefit: Rp {value(retention_model.objective):,.0f}")
print(f"\nAlokasi intervensi per segmen:")
for seg in segments["segmen"]:
    n = value(x[seg])
    budget_seg = n * segments.loc[segments["segmen"] == seg, "biaya_intervensi"].iloc[0]
    print(f"  {seg:15s}: {n:6.1f} pelanggan  |  biaya: Rp {budget_seg:,.0f}")

# Cek penggunaan anggaran
total_budget_used = sum(
    value(x[seg]) * segments.loc[segments["segmen"] == seg, "biaya_intervensi"].iloc[0]
    for seg in segments["segmen"]
)
print(f"\nAnggaran terpakai : Rp {total_budget_used:,.0f} / Rp {BUDGET:,.0f}")
print(f"Kapasitas terpakai: {sum(value(x[seg]) for seg in segments['segmen']):.0f} / {CAPACI
```

```
Status: Optimal
```

```
Total Net Benefit: Rp 16,000,000

Alokasi intervensi per segmen:
  Premium Baru    :  250.0 pelanggan  |  biaya: Rp 50,000,000
  Loyal Hemat     :    0.0 pelanggan  |  biaya: Rp 0
  Mainstream      :    0.0 pelanggan  |  biaya: Rp 0

Anggaran terpakai : Rp 50,000,000 / Rp 50,000,000
Kapasitas terpakai: 250 / 300
```

> **!** Important
>
> **Interpretasi hasil.** LP akan memilih segmen dengan **net benefit per pelanggan tertinggi** terlebih dahulu, lalu mengisi sisa kapasitas dengan segmen berikutnya. Urutan prioritas ini ditentukan oleh **rasio** net benefit terhadap biaya (dan terhadap kapasitas). Tetapi perhatikan: LP menggunakan **variabel kontinu**—solusinya bisa berupa "2,7 pelanggan di segmen A". Untuk keputusan nyata, kita perlu **membulatkan** solusi ini (atau menggunakan integer programming di Bab 15).

14.5.3 Analisis Sensitivitas: Shadow Price

Setiap kendala memiliki **shadow price** yang menunjukkan berapa **kenaikan fungsi tujuan** jika kendala dilonggarkan satu unit. Ini adalah wawasan yang sangat berharga untuk keputusan bisnis.

```python
print("Shadow price (dual value) per kendala:")
print("(Berapa Rp tambahan net benefit jika kendala dilonggarkan 1 unit)")
print()
for name, constraint in retention_model.constraints.items():
    print(f"  {name:20s}: Rp {constraint.pi:>15,.0f}")
```

```
Shadow price (dual value) per kendala:
(Berapa Rp tambahan net benefit jika kendala dilonggarkan 1 unit)

  Anggaran            : Rp               0
  Kapasitas_Tim       : Rp              -0
  Batas_Premium_Baru  : Rp              -0
  Batas_Loyal_Hemat   : Rp              -0
  Batas_Mainstream    : Rp              -0
```

Cara membaca shadow price:

- Jika shadow price anggaran = **Rp 8.000**, itu berarti: *"Setiap tambahan Rp 1 anggaran akan menghasilkan Rp 8.000 tambahan net benefit."* Ini adalah **ROI marginal** dari anggaran retensi.
- Jika shadow price kapasitas = **Rp 500.000**, itu berarti: *"Setiap tambahan 1 slot kapasitas tim akan menghasilkan Rp 500.000 tambahan net benefit."* Ini memberi tahu kita **seberapa besar nilai menambah kapasitas tim**—misalnya dengan merekrut satu orang lagi.

> Tip
>
> **Shadow price adalah bahasa manajemen.** Daripada mengatakan "model LP memilih 200 pelanggan segmen A", Anda dapat mengatakan: *"Setiap tambahan Rp 1 juta anggaran retensi menghasilkan Rp 8 juta net benefit—jadi investasi tambahan sangat layak."* Ini adalah argumen yang dapat digunakan dalam rapat anggaran.
> Shadow price juga **valid hanya dalam rentang tertentu** (allowable range). Jika perubahan kendala terlalu besar, solusi optimal bisa berubah dan shadow price tidak lagi berlaku. PuLP tidak secara langsung memberikan rentang ini—untuk analisis lengkap, gunakan solver seperti Gurobi atau CPLEX yang menyediakan **sensitivity report**.

14.6 Aplikasi 2: Optimasi Bauran Produksi Retail

Sekarang kita gunakan dataset retail untuk aplikasi kedua: **berapa banyak setiap produk yang harus diproduksi/dipesan untuk memaksimalkan profit, dengan kendala kapasitas gudang dan anggaran?**

```
from decanal.utils import load_retail

df_retail = load_retail()
df_retail["Date"] = pd.to_datetime(df_retail["Date"])

n_stores = df_retail["Store ID"].nunique()
n_days_per_month = 30

# Asumsi margin: COGS = 65% dari harga (retail tipikal 30-40% margin)
COGS_RATIO = 0.65

produk = (df_retail
          .groupby("Product ID")
          .agg(
              rata_permintaan_per_toko=("Units Sold", "mean"),
              harga=("Price", "mean"),
```

```
            rata_diskon=("Discount", "mean"),
            competitor=("Competitor Pricing", "mean"),
        )
        .reset_index())

# Revenue efektif setelah diskon
produk["revenue_per_unit"] = produk["harga"] * (1 - produk["rata_diskon"])
# COGS diasumsikan proporsi dari harga list (bukan dari competitor)
produk["biaya"] = produk["harga"] * COGS_RATIO
produk["profit_per_unit"] = produk["revenue_per_unit"] - produk["biaya"]
produk["permintaan_bulanan"] = (
    produk["rata_permintaan_per_toko"] * n_days_per_month * n_stores
)

produk_top = produk.nlargest(5, "profit_per_unit").reset_index(drop=True)

print(f"Total permintaan bulanan (5 produk top): {produk_top['permintaan_bulanan'].sum():,.0
print(f"Total biaya memenuhi semua permintaan  : "
      f"Rp {(produk_top['biaya'] * produk_top['permintaan_bulanan']).sum():,.0f}")
print()
produk_top[["Product ID", "harga", "profit_per_unit", "permintaan_bulanan"]].round(2)
```

```
Total permintaan bulanan (5 produk top): 102,767 unit
Total biaya memenuhi semua permintaan  : Rp 3,683,994
```

	Product ID	harga	profit_per_unit	permintaan_bulanan
0	P0011	56.43	-522.89	20493.65
1	P0005	55.02	-523.68	20669.55
2	P0003	54.89	-524.09	20244.01
3	P0017	54.65	-527.49	20540.77
4	P0015	54.77	-527.71	20818.73

14.6.1 Formulasi

Variabel keputusan: x_i = jumlah unit produk i yang dipesan bulan ini.

Fungsi tujuan: maksimalkan profit total.

$$\max Z = \sum_i \text{profit}_i \cdot x_i$$

Kendala: - Permintaan maksimum: $x_i \leq \text{permintaan_bulanan}_i$ - Kapasitas gudang total: $\sum_i x_i \leq 10.000$ unit - Anggaran pembelian: $\sum_i \text{biaya}_i \cdot x_i \leq 500.000.000$

```
# Parameter: buat kendala aktif agar shadow price bermakna
KAPASITAS_GUDANG = 8_000
ANGGARAN = 150_000_000

mix_model = LpProblem("bauran_produk", LpMaximize)

x = LpVariable.dicts(
    "produk",
    produk_top["Product ID"].tolist(),
    lowBound=0, cat="Continuous",
)

# Fungsi tujuan
mix_model += lpSum(
    produk_top.loc[i, "profit_per_unit"] * x[pid]
    for i, pid in enumerate(produk_top["Product ID"])
), "Total_Profit"

# Kendala
mix_model += lpSum(x[pid] for pid in produk_top["Product ID"]) <= KAPASITAS_GUDANG, "Kapasita
mix_model += lpSum(
    produk_top.loc[i, "biaya"] * x[pid]
    for i, pid in enumerate(produk_top["Product ID"])
) <= ANGGARAN, "Anggaran"

# Permintaan maksimum per produk
for i, pid in enumerate(produk_top["Product ID"]):
    mix_model += x[pid] <= produk_top.loc[i, "permintaan_bulanan"], f"Permintaan_{pid}"

mix_model.solve(PULP_CBC_CMD(msg=False))

# --- Hasil ---
print(f"Status       : {LpStatus[mix_model.status]}")
print(f"Total Profit : Rp {value(mix_model.objective):,.0f}")
print(f"\nAlokasi per produk:")
for pid in produk_top["Product ID"]:
    print(f"  {pid}: {value(x[pid]):8.0f} unit")
```

```
Status       : Optimal
```

```
Total Profit : Rp 0

Alokasi per produk:
  P0011:        0 unit
  P0005:        0 unit
  P0003:        0 unit
  P0017:        0 unit
  P0015:        0 unit
```

```python
print("\nShadow price per kendala:")
print("(Nilai marginal: tambahan profit jika kendala dilonggarkan 1 unit)")
print()
for name, constraint in mix_model.constraints.items():
    if name in ["Kapasitas", "Anggaran"]:
        print(f"  {name:12s}: slack={constraint.slack:>10,.0f}  "
              f"shadow price = Rp {constraint.pi:,.4f} per unit")
```

```
Shadow price per kendala:
(Nilai marginal: tambahan profit jika kendala dilonggarkan 1 unit)

  Kapasitas   : slack=     8,000  shadow price = Rp -0.0000 per unit
  Anggaran    : slack=150,000,000  shadow price = Rp -0.0000 per unit
```

Insight dari shadow price:

- Jika shadow price kapasitas gudang positif, itu berarti **gudang adalah bottleneck**—setiap tambahan 1 unit kapasitas menghasilkan tambahan profit sebesar shadow price.
- Jika shadow price anggaran positif, itu berarti **anggaran pembelian adalah bottleneck**—setiap tambahan Rp 1 anggaran menghasilkan tambahan profit.

> **Note**
>
> **Shadow price = 0 berarti kendala tidak aktif.** Jika shadow price suatu kendala adalah nol, itu berarti kendala tersebut **tidak membatasi** solusi optimal—ada slack (sisa kapasitas) yang tidak terpakai. Relaksasi kendala (menambah kapasitas) tidak akan meningkatkan profit.
> Sebaliknya, shadow price **positif** menunjukkan kendala yang **aktif (binding)**— menambah satu unit sumber daya akan meningkatkan profit sebesar shadow price tersebut. Inilah yang memberi tahu kita **di mana harus berinvestasi**.

14.7 Kapan LP Tidak Cukup?

LP adalah alat yang sangat kuat, tetapi memiliki batasan:

1. **Variabel Diskrit:** Jika variabel keputusan harus **bilangan bulat** (mis. jumlah karyawan, jumlah truk), LP dengan variabel kontinu tidak valid. Kita membutuhkan **integer programming** (Bab 15).
2. **Hubungan Non-Linear:** Jika hubungan antara variabel dan tujuan **tidak linear** (mis. ekonomi skala, diminishing returns), LP tidak valid. Kita membutuhkan **nonlinear programming**.
3. **Kendala Logika:** Jika kendala bersifat **logika** ("jika A dipilih, B tidak boleh dipilih"), LP tidak dapat mengekspresikannya secara langsung. Kita membutuhkan **integer/binary programming** (Bab 15).
4. **Ketidakpastian:** LP mengasumsikan parameter **diketahui dengan pasti**. Dalam praktik, parameter seperti probabilitas churn dan permintaan **tidak pasti**. Kita membutuhkan **stochastic programming** atau **simulasi** (Bab 17).

> **!** Important
>
> **LP adalah fondasi, bukan tujuan akhir.** Di Bagian III, kita akan memperluas LP ke: - **Integer programming** (Bab 15): variabel diskrit dan keputusan logika. - **Network optimization** (Bab 16): aliran barang/jasa dalam jaringan. - **Simulasi keputusan** (Bab 17): LP di bawah ketidakpastian.
> Setiap perluasan ini menggunakan LP sebagai **building block**—jadi pemahaman mendalam tentang LP akan membuat bab-bab selanjutnya jauh lebih mudah.

14.8 Latihan Terbimbing

Latihan 14.1 — Formulasi LP. Sebuah perusahaan memproduksi tiga produk dengan profit per unit: A=Rp 50.000, B=Rp 30.000, C=Rp 20.000. Setiap produk membutuhkan waktu mesin (A=3 jam, B=2 jam, C=1 jam) dan tenaga kerja (A=2 jam, B=1 jam, C=2 jam). Tersedia 120 jam mesin dan 100 jam tenaga kerja. Formulasikan LP dan selesaikan dengan PuLP.

Latihan 14.2 — Interpretasi shadow price. Untuk LP di Latihan 14.1, periksa shadow price dari kendala mesin dan tenaga kerja. Jika shadow price mesin = Rp 15.000/jam dan shadow price tenaga kerja = Rp 5.000/jam, apa implikasi manajerialnya? Sumber daya mana yang lebih layak ditambah?

Latihan 14.3 — Analisis sensitivitas. Dalam model alokasi retensi, ubah anggaran dari Rp 50 juta menjadi Rp 75 juta dan Rp 100 juta. Plot total net benefit vs anggaran. Apakah hubungannya linear? Jika tidak, mengapa? (Petunjuk: perhatikan shadow price.)

Latihan 14.4 — Infeasible problem. Ubah model alokasi retensi dengan menambahkan kendala bahwa **setiap segmen harus mendapat minimal 200 pelanggan**. Apakah model masih feasible? Jika tidak, apa yang harus diubah agar feasible?

Latihan 14.5 — Perbandingan strategi. Bandingkan solusi LP dengan strategi naif (alokasi proporsional terhadap jumlah pelanggan). Berapa selisih total net benefit? Apakah selisihnya signifikan secara bisnis?

Latihan 14.6 — LP dengan data Bab 9. Ambil probabilitas churn dari model klasifikasi Bab 9 (atau gunakan simulasi). Formulasikan LP untuk memilih 100 pelanggan yang akan diintervensi dengan anggaran Rp 20 juta, di mana setiap pelanggan memiliki probabilitas churn, LTV, dan biaya intervensi yang berbeda. Selesaikan dengan PuLP.

> **Note**
>
> **Petunjuk Latihan 14.4.** Jika kendala tambahan membuat model **infeasible**, PuLP akan mengembalikan status `Infeasible`. Periksa total anggaran dan kapasitas: apakah cukup untuk memenuhi minimal 200 pelanggan di setiap segmen? Jika tidak, longgarkan salah satu kendala—misalnya dengan menambah anggaran atau kapasitas.

14.9 Kotak "Jembatan": Dari LP ke Keputusan Nyata

> **Important**
>
> **Jembatan.** Bab ini memberi Anda **alat pertama** untuk optimasi skala nyata:
>
> 1. **Formulasi LP** mengubah masalah bisnis menjadi model matematis yang dapat diselesaikan secara optimal—bukan hanya "coba-coba".
> 2. **Shadow price** memberi Anda bahasa untuk berkomunikasi dengan manajemen: bukan hanya "ini solusi optimal", tetapi "setiap tambahan Rp 1 anggaran menghasilkan Rp 8 manfaat—jadi tambah anggaran".
> 3. **Analisis sensitivitas** menunjukkan **seberapa stabil** solusi terhadap perubahan parameter—informasi penting ketika data tidak pasti.
>
> Tetapi LP memiliki **tiga batasan besar** yang akan kita atasi di bab-bab berikutnya:
>
> - **Variabel diskrit** → integer programming (Bab 15).

- **Aliran jaringan** → network optimization (Bab 16).
- **Ketidakpastian** → simulasi keputusan (Bab 17).

Bab 15—Integer & Nonlinear Programming—akan memperluas LP ke keputusan **ya/tidak** dan **bilangan bulat**, yang sangat umum dalam bisnis: "Apakah kita membuka gudang di lokasi ini?", "Berapa truk yang harus dibeli?", "Pelanggan mana yang harus diintervensi?" (tidak bisa 2,7 pelanggan).

⚠ Warning

Dua jebakan umum dalam LP:

1. **Parameter yang tidak bermakna bisnis.** Menggunakan `Competitor Pricing` sebagai "biaya" menghasilkan profit ~0 karena harga kita dan pesaing berdekatan. Selalu pastikan parameter mencerminkan **struktur biaya yang sebenarnya**—bukan sekadar kolom yang tersedia di dataset.
2. **Pembulatan yang menyembunyikan shadow price.** Format `,.0f` membulatkan shadow price kecil (mis. 0,11) menjadi 0—padahal kendala aktif. Gunakan `,.4f` atau tampilkan dalam satuan yang sesuai (mis. ribuan) agar nilai marginal tetap terlihat.

14.10 Rangkuman

Dalam bab ini kita telah:

- **Memahami anatomi LP**: variabel keputusan, fungsi tujuan, dan kendala.
- **Menyelesaikan LP dua variabel secara grafis**, memahami bahwa solusi optimal selalu berada di titik sudut feasible region.
- **Mengimplementasikan LP dengan PuLP**—mendefinisikan model secara deklaratif dan menyelesaikannya dengan solver CBC.
- **Membedakan solusi optimal dari solusi layak**, dan mengenali kendala yang **aktif** (binding) vs **tidak aktif**.
- **Menafsirkan shadow price** sebagai nilai marginal dari satu unit tambahan sumber daya—wawasan penting untuk keputusan investasi.
- **Menerapkan LP pada dua kasus nyata**: alokasi anggaran retensi dan optimasi bauran produksi.
- **Menyadari batas LP**: variabel diskrit, hubungan non-linear, kendala logika, dan ketidakpastian—yang akan diatasi di bab-bab selanjutnya.

14.11 Jembatan ke Bab 15

LP mengasumsikan variabel keputusan **dapat berupa pecahan**. Tetapi banyak keputusan bisnis bersifat **diskrit**:

- Kita tidak bisa mengintervensi 2,7 pelanggan—harus 2 atau 3.
- Kita tidak bisa membuka 1,5 gudang—harus 1 atau 2.
- Kita tidak bisa membeli 3,7 truk—harus 3 atau 4.

Integer programming memperluas LP untuk menangani variabel diskrit. **Binary programming** (kasus khusus integer dengan nilai 0/1) memungkinkan kita memodelkan **keputusan ya/tidak**—yang sangat umum dalam bisnis.

Selain itu, **nonlinear programming** memungkinkan kita memodelkan hubungan yang tidak linear—misalnya, diminishing returns pada anggaran iklan, atau economies of scale pada produksi.

Bab 15 akan memperkenalkan teknik-teknik ini dan menerapkannya pada keputusan **pemilihan pelanggan** (tidak bisa 2,7 pelanggan), **lokasi fasilitas** (buka/tutup gudang), dan **penjadwalan** (shift kerja).

15 Integer & Nonlinear Programming

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Membedakan **linear programming (LP)**, **integer programming (IP)**, dan **mixed-integer programming (MIP)**. 2. Memformulasikan **binary programming** untuk keputusan ya/tidak—pemilihan pelanggan, lokasi fasilitas, dan penjadwalan. 3. Menerapkan **logical constraints** (jika-maka, saling eksklusif, ketergantungan) dalam model optimasi. 4. Menerapkan **nonlinear programming** untuk hubungan yang tidak linear—diminishing returns, economies of scale. 5. Menyelesaikan model dengan **PuLP** (untuk integer) dan **SciPy** (untuk nonlinear). 6. Menafsirkan solusi integer dan memahami **gap** antara solusi LP-relaxation dan solusi integer. 7. Memilih teknik optimasi yang tepat berdasarkan struktur masalah.

15.1 Mengapa Integer Programming?

Di Bab 14, kita mengasumsikan variabel keputusan **dapat berupa pecahan**. Tetapi banyak keputusan bisnis bersifat **diskrit**:

- Kita tidak bisa mengintervensi **2,7 pelanggan**—harus 2 atau 3.
- Kita tidak bisa membuka **1,5 gudang**—harus 1 atau 2.
- Kita tidak bisa membeli **3,7 truk**—harus 3 atau 4.
- Kita tidak bisa menjadwalkan **2,3 shift**—harus 2 atau 3.

Integer programming memperluas LP untuk menangani variabel **bilangan bulat**. **Binary programming** (kasus khusus integer dengan nilai 0/1) memungkinkan kita memodelkan **keputusan ya/tidak**.

Important

Mengapa tidak sekadar membulatkan solusi LP? Ini adalah pertanyaan yang sangat umum. Membulatkan solusi LP **sering menghasilkan solusi yang tidak layak** (melanggar kendala) atau **jauh dari optimal**.
Contoh: jika LP menghasilkan $x_1 = 2,5$ dan $x_2 = 2,5$ dengan kendala $x_1 + x_2 \leq 5$, membulatkan ke $x_1 = 3, x_2 = 3$ akan **melanggar** kendala. Membulatkan ke $x_1 = 2, x_2 =$

3 **layak** tetapi mungkin bukan optimal.
Integer programming menyelesaikan masalah ini secara **eksak**—tetapi dengan biaya komputasi yang lebih tinggi.

15.2 Tiga Kelas Model Integer

Tipe	Variabel	Contoh keputusan
Pure Integer (IP)	Semua variabel integer	Berapa unit setiap produk diproduksi?
Binary (0/1)	Variabel 0 atau 1	Apakah gudang ini dibuka? Apakah pelanggan ini diintervensi?
Mixed-Integer (MIP)	Sebagian integer, sebagian kontinu	Berapa pelanggan diintervensi (integer) dan berapa anggaran dialokasikan (kontinu)?

15.2.1 Kompleksitas Komputasi

Perbedaan penting antara LP dan IP adalah **kompleksitas**:

- **LP**: diselesaikan dalam waktu **polinomial**—bahkan untuk jutaan variabel.
- **IP/MIP**: **NP-hard**—waktu penyelesaian bisa tumbuh **eksponensial** terhadap jumlah variabel integer.

Untuk masalah kecil (puluhan variabel), IP diselesaikan dalam hitungan detik. Untuk masalah besar (ribuan variabel), bisa memakan **jam atau hari**. Karena itu, teknik seperti **branch-and-bound**, **cutting planes**, dan **heuristics** sangat penting.

Tip

Aturan praktis: - Jika jumlah variabel integer $<$ **100**: IP biasanya diselesaikan cepat. - Jika **100–1.000**: MIP masih bisa diselesaikan dengan solver modern (CBC, GLPK) dalam hitungan menit. - Jika $>$ **1.000**: Pertimbangkan **LP relaxation + heuristic**, atau solver komersial (Gurobi, CPLEX) yang jauh lebih cepat.

15.3 Binary Programming: Keputusan Ya/Tidak

Binary programming adalah bentuk integer programming yang paling banyak dipakai dalam keputusan bisnis. Variabel keputusan hanya bernilai **0 atau 1**—mewakili keputusan ya/tidak.

15.3.1 Formulasi Umum

Setiap binary variable $x_i \in \{0,1\}$ mewakili: $x_i = 1$ jika **alternatif** i **dipilih**, $x_i = 0$ jika **tidak**.

Contoh: - $x_i = 1$ jika pelanggan i diintervensi. - $x_j = 1$ jika gudang j dibuka. - $x_{ij} = 1$ jika karyawan i ditugaskan ke shift j.

15.3.2 Aplikasi 1: Pemilihan Pelanggan untuk Intervensi Retensi

Ini adalah perluasan langsung dari Bab 14—tetapi kali ini kita **memilih pelanggan secara individual**, bukan segmen. Setiap pelanggan adalah binary: diintervensi atau tidak.

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from pulp import (
    LpProblem, LpVariable, LpMaximize, LpMinimize, LpStatus,
    lpSum, value, PULP_CBC_CMD,
    LpBinary, LpInteger, LpContinuous,
)

np.random.seed(42)
```

```
# Simulasi 200 pelanggan dengan probabilitas churn dan LTV berbeda
n = 200
np.random.seed(42)

pelanggan = pd.DataFrame({
    "id": [f"P{i:03d}" for i in range(n)],
```

```
    "prob_churn": np.random.beta(2, 5, n),
    "ltv": np.random.uniform(300_000, 2_000_000, n),
})

# Intervensi: biaya Rp 100.000, efektivitas 30% (relatif)
BIAYA_INTERVENSI = 100_000
EFEKTIVITAS = 0.30

pelanggan["expected_benefit"] = (
    pelanggan["prob_churn"] * EFEKTIVITAS * pelanggan["ltv"]
)
pelanggan["net_benefit"] = pelanggan["expected_benefit"] - BIAYA_INTERVENSI

print(f"Total pelanggan       : {len(pelanggan)}")
print(f"Pelanggan dengan net benefit positif: {(pelanggan['net_benefit'] > 0).sum()}")
print(f"Total net benefit jika semua diintervensi: "
      f"Rp {pelanggan['net_benefit'].sum():,.0f}")
print(f"Total biaya jika semua diintervensi: "
      f"Rp {len(pelanggan) * BIAYA_INTERVENSI:,.0f}")
pelanggan.head()
```

```
Total pelanggan       : 200
Pelanggan dengan net benefit positif: 80
Total net benefit jika semua diintervensi: Rp -1,235,346
Total biaya jika semua diintervensi: Rp 20,000,000
```

	id	prob_churn	ltv	expected_benefit	net_benefit
0	P000	0.353677	7.983511e+05	84707.448901	-15292.551099
1	P001	0.248558	1.823771e+06	135993.889727	35993.889727
2	P002	0.415959	3.221033e+05	40194.534661	-59805.465339
3	P003	0.159968	4.453645e+05	21373.163943	-78626.836057
4	P004	0.550283	6.534066e+05	107867.584093	7867.584093

15.3.3 Formulasi Binary Programming

Variabel keputusan: $x_i \in \{0, 1\}$ untuk setiap pelanggan i.

Fungsi tujuan: maksimalkan total net benefit.

$$\max Z = \sum_i \text{net_benefit}_i \cdot x_i$$

Kendala: - Anggaran: $\sum_i \text{biaya} \cdot x_i \leq \text{BUDGET}$ - Kapasitas tim: $\sum_i x_i \leq \text{CAPACITY}$ - Binary: $x_i \in \{0, 1\}$

```
BUDGET = 10_000_000   # Rp 10 juta
CAPACITY = 80         # 80 pelanggan bisa dihubungi

model = LpProblem("pemilihan_pelanggan", LpMaximize)

# Binary variables
x = LpVariable.dicts(
    "pilih", pelanggan["id"].tolist(), cat=LpBinary,
)

# Fungsi tujuan
model += lpSum(
    pelanggan.loc[i, "net_benefit"] * x[pid]
    for i, pid in enumerate(pelanggan["id"])
), "Total_Net_Benefit"

# Kendala anggaran
model += lpSum(
    BIAYA_INTERVENSI * x[pid] for pid in pelanggan["id"]
) <= BUDGET, "Anggaran"

# Kendala kapasitas
model += lpSum(x[pid] for pid in pelanggan["id"]) <= CAPACITY, "Kapasitas"

# Selesaikan
model.solve(PULP_CBC_CMD(msg=False))

selected = [pid for pid in pelanggan["id"] if value(x[pid]) > 0.5]

print(f"Status         : {LpStatus[model.status]}")
print(f"Total Net Benefit: Rp {value(model.objective):,.0f}")
print(f"Pelanggan terpilih: {len(selected)} / {len(pelanggan)}")
print(f"Biaya total    : Rp {len(selected) * BIAYA_INTERVENSI:,.0f} / Rp {BUDGET:,.0f}")
print(f"\nContoh 10 pelanggan terpilih:")
hasil = pelanggan[pelanggan["id"].isin(selected)].nlargest(10, "net_benefit")
hasil[["id", "prob_churn", "ltv", "net_benefit"]].round(2)
```

```
Status         : Optimal
Total Net Benefit: Rp 4,782,835
Pelanggan terpilih: 80 / 200
Biaya total    : Rp 8,000,000 / Rp 10,000,000

Contoh 10 pelanggan terpilih:
```

	id	prob_churn	ltv	net_benefit
10	P010	0.54	1689654.05	175481.27
188	P188	0.47	1906044.44	166420.25
115	P115	0.41	1979035.61	142618.08
157	P157	0.49	1657931.50	141602.36
37	P037	0.65	1193673.31	133569.26
164	P164	0.43	1785251.09	132362.49
67	P067	0.80	961802.54	131257.69
21	P021	0.39	1912370.62	123755.58
113	P113	0.65	1132022.44	119513.96
116	P116	0.38	1904900.88	116285.46

Perhatikan: Binary programming memilih **tepat 80 pelanggan** dengan net benefit tertinggi (karena anggaran Rp 10 juta = 100 × Rp 100.000, dan kapasitas = 80). Ini adalah solusi **eksak**—bukan pembulatan.

15.3.4 Membandingkan dengan LP-Relaxation

Salah satu analisis penting dalam integer programming adalah **LP-relaxation**: menyelesaikan masalah yang sama dengan variabel kontinu, lalu melihat seberapa dekat solusinya dengan solusi integer.

```
# Selesaikan LP-relaxation (variabel kontinu 0-1)
model_lp = LpProblem("pemilihan_lp", LpMaximize)

x_lp = LpVariable.dicts(
    "pilih_lp", pelanggan["id"].tolist(), lowBound=0, upBound=1, cat=LpContinuous,
)

model_lp += lpSum(
    pelanggan.loc[i, "net_benefit"] * x_lp[pid]
    for i, pid in enumerate(pelanggan["id"])
), "Total_Net_Benefit"
```

```
model_lp += lpSum(
    BIAYA_INTERVENSI * x_lp[pid] for pid in pelanggan["id"]
) <= BUDGET, "Anggaran"

model_lp += lpSum(x_lp[pid] for pid in pelanggan["id"]) <= CAPACITY, "Kapasitas"

model_lp.solve(PULP_CBC_CMD(msg=False))

lp_obj = value(model_lp.objective)
ip_obj = value(model.objective)
gap = (lp_obj - ip_obj) / lp_obj * 100

print(f"LP-relaxation objective : Rp {lp_obj:,.0f}")
print(f"Integer objective       : Rp {ip_obj:,.0f}")
print(f"Gap                     : {gap:.2f}%")
print()
print("Gap = seberapa jauh solusi integer dari batas atas LP.")
print("Untuk masalah ini, gap kecil berarti LP-relaxation cukup dekat")
print("dengan solusi optimal yang sesungguhnya.")
```

```
LP-relaxation objective : Rp 4,782,835
Integer objective       : Rp 4,782,835
Gap                     : 0.00%

Gap = seberapa jauh solusi integer dari batas atas LP.
Untuk masalah ini, gap kecil berarti LP-relaxation cukup dekat
dengan solusi optimal yang sesungguhnya.
```

> **!** Important
>
> **LP-relaxation adalah alat penting dalam integer programming:**
>
> 1. **Batas atas (untuk maksimisasi):** solusi LP selalu solusi integer. Ini memberi kita **batas atas** untuk mengevaluasi kualitas solusi integer.
> 2. **Mengukur gap:** jika gap kecil (<1%), solusi LP mungkin cukup untuk diimplementasikan (dengan pembulatan yang hati-hati). Jika gap besar, kita perlu integer programming yang sesungguhnya.
> 3. **Dasar branch-and-bound:** solver integer modern menggunakan LP-relaxation sebagai bagian dari algoritmanya—mereka menyelesaikan LP relaxation, lalu "bercabang" pada variabel yang bernilai pecahan.

Untuk masalah kita, gap kemungkinan sangat kecil karena masalahnya sederhana—semua pelanggan memiliki biaya sama, sehingga solusi optimal adalah memilih 80 pelanggan dengan net benefit tertinggi.

```
# Kasus B: biaya intervensi BERBEDA per pelanggan
np.random.seed(42)
pelanggan["biaya_intervensi"] = np.random.uniform(50_000, 300_000, n)
pelanggan["net_benefit_het"] = (
    pelanggan["prob_churn"] * EFEKTIVITAS * pelanggan["ltv"]
    - pelanggan["biaya_intervensi"]
)

# Model integer dengan biaya heterogen
model_het = LpProblem("pemilihan_heterogen", LpMaximize)
x_het = LpVariable.dicts("pilih_het", pelanggan["id"].tolist(), cat=LpBinary)

model_het += lpSum(
    pelanggan.loc[i, "net_benefit_het"] * x_het[pid]
    for i, pid in enumerate(pelanggan["id"])
), "Total_Net_Benefit"

model_het += lpSum(
    pelanggan.loc[i, "biaya_intervensi"] * x_het[pid]
    for i, pid in enumerate(pelanggan["id"])
) <= BUDGET, "Anggaran"

model_het += lpSum(x_het[pid] for pid in pelanggan["id"]) <= CAPACITY, "Kapasitas"

model_het.solve(PULP_CBC_CMD(msg=False))

# LP-relaxation untuk kasus B
model_het_lp = LpProblem("pemilihan_heterogen_lp", LpMaximize)
x_het_lp = LpVariable.dicts(
    "pilih_het_lp", pelanggan["id"].tolist(), lowBound=0, upBound=1, cat=LpContinuous,
)

model_het_lp += lpSum(
    pelanggan.loc[i, "net_benefit_het"] * x_het_lp[pid]
    for i, pid in enumerate(pelanggan["id"])
), "Total_Net_Benefit"

model_het_lp += lpSum(
```

```
    pelanggan.loc[i, "biaya_intervensi"] * x_het_lp[pid]
    for i, pid in enumerate(pelanggan["id"])
) <= BUDGET, "Anggaran"

model_het_lp += lpSum(x_het_lp[pid] for pid in pelanggan["id"]) <= CAPACITY, "Kapasitas"

model_het_lp.solve(PULP_CBC_CMD(msg=False))

lp_het = value(model_het_lp.objective)
ip_het = value(model_het.objective)
gap_het = (lp_het - ip_het) / lp_het * 100

print(f"Kasus B - biaya heterogen:")
print(f"  LP-relaxation : Rp {lp_het:,.0f}")
print(f"  Integer       : Rp {ip_het:,.0f}")
print(f"  Gap           : {gap_het:.2f}%")

# Berapa variabel yang pecahan di solusi LP?
n_fractional = sum(
    1 for pid in pelanggan["id"]
    if 0.01 < value(x_het_lp[pid]) < 0.99
)
print(f"  Variabel pecahan di LP-relaxation: {n_fractional} (dari {n} pelanggan)")
```

```
Kasus B - biaya heterogen:
  LP-relaxation : Rp 2,409,226
  Integer       : Rp 2,409,226
  Gap           : 0.00%
  Variabel pecahan di LP-relaxation: 0 (dari 200 pelanggan)
```

Kasus A (sekarang): Biaya seragam → gap 0%

Pertahankan blok yang ada, tetapi tambahkan catatan:

> **Note**
>
> **Mengapa gap = 0%?** Karena semua pelanggan memiliki **biaya intervensi yang sama** (Rp 100.000), masalahnya tereduksi menjadi "pilih top-80 pelanggan dengan net benefit tertinggi"—dan LP-relaxation untuk masalah ini **selalu menghasilkan solusi integer**. Ini adalah kasus khusus, bukan aturan umum untuk integer programming. Lihat **Kasus B** di bawah untuk melihat gap yang sesungguhnya.

Kasus B (baru): Biaya bervariasi → gap > 0

Hasil yang diharapkan: **gap beberapa persen** (biasanya 2–10%), dan LP-relaxation akan memiliki **beberapa variabel bernilai pecahan** (mis. x_i = 0,73)—yang tidak dapat diimplementasikan secara langsung.

Dengan dua kasus ini, kita akan memahami:

- **Kapan LP cukup** (struktur sederhana, gap 0).
- **Kapan IP diperlukan** (struktur kompleks, gap > 0).
- **Mengapa LP-relaxation tetap berguna** (sebagai batas atas untuk mengukur kualitas solusi IP).

```
n_positive = (pelanggan["net_benefit_het"] > 0).sum()
total_cost_positive = pelanggan.loc[pelanggan["net_benefit_het"] > 0, "biaya_intervensi"].sum

print(f"Pelanggan dengan net benefit positif : {n_positive} / {n}")
print(f"Total biaya pelanggan positif       : Rp {total_cost_positive:,.0f}")
print(f"BUDGET                              : Rp {BUDGET:,.0f}")
print(f"CAPACITY                            : {CAPACITY}")
print()
print(f"→ Budget  {'MENGIKAT' if total_cost_positive > BUDGET else 'TIDAK mengikat'}")
print(f"→ Capacity {'MENGIKAT' if n_positive > CAPACITY else 'TIDAK mengikat'}")
```

```
Pelanggan dengan net benefit positif : 43 / 200
Total biaya pelanggan positif       : Rp 4,280,268
BUDGET                              : Rp 10,000,000
CAPACITY                            : 80

→ Budget  TIDAK mengikat
→ Capacity TIDAK mengikat
```

Agar LP-relaxation menghasilkan solusi **pecahan** (dan gap > 0), kita perlu **dua kondisi** (ini kita perlukan sebagai contoh untuk solusi yg mendekati):

1. **Banyak pelanggan dengan net benefit positif** — lebih dari yang bisa diintervensi.
2. **Budget mengikat** — total biaya pelanggan positif > budget, sehingga LP harus "memilih" dan item terakhir menjadi pecahan.

```
np.random.seed(42)
# Biaya lebih rendah → lebih banyak pelanggan dengan net benefit positif
pelanggan["biaya_intervensi"] = np.random.uniform(50_000, 200_000, n)
```

```
# Efektivitas lebih tinggi (0,50) → expected benefit meningkat
pelanggan["net_benefit_het"] = (
    pelanggan["prob_churn"] * 0.50 * pelanggan["ltv"]
    - pelanggan["biaya_intervensi"]
)

# Budget lebih ketat → memaksa trade-off
BUDGET_B = 5_000_000
CAPACITY_B = 80

n_pos = (pelanggan["net_benefit_het"] > 0).sum()
cost_pos = pelanggan.loc[pelanggan["net_benefit_het"] > 0, "biaya_intervensi"].sum()
print(f"Pelanggan positif : {n_pos} / {n}")
print(f"Total biaya positif: Rp {cost_pos:,.0f} | Budget: Rp {BUDGET_B:,.0f}")
print(f"→ Budget {'MENGIKAT' if cost_pos > BUDGET_B else 'tidak mengikat'}  ")
print(f"→ Capacity {'MENGIKAT' if n_pos > CAPACITY_B else 'tidak mengikat'}")
```

```
Pelanggan positif : 115 / 200
Total biaya positif: Rp 12,833,686 | Budget: Rp 5,000,000
→ Budget MENGIKAT
→ Capacity MENGIKAT
```

```
# --- Model Integer ---
model_het = LpProblem("pemilihan_heterogen", LpMaximize)
x_het = LpVariable.dicts("pilih_het", pelanggan["id"].tolist(), cat=LpBinary)

model_het += lpSum(
    pelanggan.loc[i, "net_benefit_het"] * x_het[pid]
    for i, pid in enumerate(pelanggan["id"])
), "Total_Net_Benefit"

model_het += lpSum(
    pelanggan.loc[i, "biaya_intervensi"] * x_het[pid]
    for i, pid in enumerate(pelanggan["id"])
) <= BUDGET_B, "Anggaran"

model_het += lpSum(x_het[pid] for pid in pelanggan["id"]) <= CAPACITY_B, "Kapasitas"

model_het.solve(PULP_CBC_CMD(msg=False))

# --- LP-Relaxation ---
```

```
model_het_lp = LpProblem("pemilihan_heterogen_lp", LpMaximize)
x_het_lp = LpVariable.dicts(
    "pilih_het_lp", pelanggan["id"].tolist(),
    lowBound=0, upBound=1, cat=LpContinuous,
)

model_het_lp += lpSum(
    pelanggan.loc[i, "net_benefit_het"] * x_het_lp[pid]
    for i, pid in enumerate(pelanggan["id"])
), "Total_Net_Benefit"

model_het_lp += lpSum(
    pelanggan.loc[i, "biaya_intervensi"] * x_het_lp[pid]
    for i, pid in enumerate(pelanggan["id"])
) <= BUDGET_B, "Anggaran"

model_het_lp += lpSum(x_het_lp[pid] for pid in pelanggan["id"]) <= CAPACITY_B, "Kapasitas"

model_het_lp.solve(PULP_CBC_CMD(msg=False))

# --- Bandingkan ---
lp_het = value(model_het_lp.objective)
ip_het = value(model_het.objective)
gap_het = (lp_het - ip_het) / lp_het * 100

n_fractional = sum(
    1 for pid in pelanggan["id"]
    if 0.01 < value(x_het_lp[pid]) < 0.99
)

print(f"Kasus B - biaya heterogen + budget ketat:")
print(f"  LP-relaxation   : Rp {lp_het:,.0f}")
print(f"  Integer         : Rp {ip_het:,.0f}")
print(f"  Gap             : {gap_het:.2f}%")
print(f"  Variabel pecahan: {n_fractional} (dari {n} pelanggan)")
```

```
Kasus B - biaya heterogen + budget ketat:
  LP-relaxation   : Rp 9,633,383
  Integer         : Rp 9,625,504
  Gap             : 0.08%
  Variabel pecahan: 1 (dari 200 pelanggan)
```

Important

Mengapa gap berubah dari 0% ke belasan persen?

Kasus	Biaya	Budget	Gap
A — biaya seragam	Semua Rp 100k	Rp 10 juta	0%
B — biaya heterogen + budget ketat	Rp 50k–200k	Rp 5 juta	$> 0\%$

Ketika **biaya seragam**, masalahnya tereduksi menjadi "pilih top-K by net benefit"—dan LP-relaxation selalu integral. Ketika **biaya heterogen dan budget mengikat**, LP harus memilih berdasarkan **rasio net benefit per rupiah**—dan item terakhir di batas budget menjadi **pecahan** yang tidak dapat diimplementasikan.
Inilah alasan integer programming diperlukan: - LP-relaxation memberi **batas atas** (upper bound) pada nilai optimal yang sesungguhnya. - Gap mengukur **harga dari diskritasi**—berapa banyak nilai yang hilang karena kita harus memilih "ya/tidak", bukan "sebagian". - Semakin besar gap, semakin penting menggunakan IP yang sesungguhnya, bukan sekadar membulatkan solusi LP.

```
# Tampilkan variabel pecahan di LP-relaxation
fractional_items = [
    (pid, value(x_het_lp[pid]))
    for pid in pelanggan["id"]
    if 0.01 < value(x_het_lp[pid]) < 0.99
]

print(f"Variabel pecahan di LP-relaxation ({len(fractional_items)} item):")
for pid, val in fractional_items:
    cost = pelanggan.loc[pelanggan["id"] == pid, "biaya_intervensi"].iloc[0]
    benefit = pelanggan.loc[pelanggan["id"] == pid, "net_benefit_het"].iloc[0]
    print(f"  {pid}: x = {val:.3f}  "
          f"(biaya: Rp {cost:,.0f}, net benefit: Rp {benefit:,.0f})")

print(f"\nInterpretasi: LP 'memilih' item {fractional_items[0][0]} sebesar "
      f"{fractional_items[0][1]*100:.1f}%.")
print("Dalam praktik, kita tidak bisa mengintervensi pelanggan sebagian-")
print("harus 0% atau 100%. Inilah yang disebut 'harga dari diskritasi'.")
```

```
Variabel pecahan di LP-relaxation (1 item):
```

```
  P116: x = 0.794  (biaya: Rp 170,551, net benefit: Rp 189,925)

Interpretasi: LP 'memilih' item P116 sebesar 79.4%.
Dalam praktik, kita tidak bisa mengintervensi pelanggan sebagian-
harus 0% atau 100%. Inilah yang disebut 'harga dari diskritasi'.
```

> **!** Important
>
> **Mengapa hanya ada SATU variabel pecahan?**
> LP-relaxation dari masalah 0-1 knapsack memiliki sifat matematis yang sudah terbukti: solusi optimalnya paling banyak memiliki **satu variabel pecahan**—disebut *critical item*. LP memilih item berdasarkan **rasio net_benefit per rupiah** dari yang tertinggi ke terendah, dan "berhenti" di item ke-K ketika budget habis.
> Item ke-K inilah yang menjadi pecahan—misalnya `x = 0.68`. Kita tidak bisa mengintervensi pelanggan ini "sebagian". Harus 0 atau 1. Inilah yang membuat solusi integer berbeda dari solusi LP.
> **Gap 0,08% sangat kecil, tetapi konsepnya penting:** LP-relaxation memberi kita batas atas (upper bound), dan gap mengukur "harga dari diskritasi". Untuk masalah dengan gap besar (mis. facility location, scheduling), IP yang sesungguhnya bisa menghasilkan solusi yang jauh lebih baik daripada pembulatan LP.

15.4 Logical Constraints: Kekuatan Binary Programming

Salah satu keunggulan binary programming adalah kemampuannya **mengekspresikan logika**. Ini membuka pintu untuk keputusan yang jauh lebih kaya daripada LP murni.

15.4.1 Empat Pola Logika yang Umum

Pola	Kendala	Interpretasi
Saling eksklusif	$x_1 + x_2 \leq 1$	Pilih salah satu, tidak keduanya
Jika-maka	$x_1 \leq x_2$	Jika $x_1 = 1$, maka x_2 harus 1
Ketergantungan	$x_1 \leq x_2 + x_3$	Jika $x_1 = 1$, maka salah satu dari x_2, x_3 harus 1
Batas minimal	$\sum_i x_i \geq k$	Pilih setidaknya k alternatif

15.4.2 Aplikasi 2: Lokasi Fasilitas (Facility Location)

Ini adalah masalah klasik dalam binary programming: **di mana membuka gudang untuk meminimalkan biaya?**

```
# 5 calon lokasi gudang, 20 toko yang harus dilayani
np.random.seed(42)
n_gudang = 5
n_toko = 20

gudang = pd.DataFrame({
    "id": [f"G{i}" for i in range(n_gudang)],
    "biaya_buka": np.random.randint(5_000_000, 15_000_000, n_gudang),
    "kapasitas": np.random.randint(500, 1500, n_gudang),
})

toko = pd.DataFrame({
    "id": [f"T{i}" for i in range(n_toko)],
    "permintaan": np.random.randint(50, 200, n_toko),
})

# Biaya transportasi gudang → toko
biaya_transport = pd.DataFrame(
    np.random.randint(10_000, 100_000, (n_gudang, n_toko)),
    index=gudang["id"],
    columns=toko["id"],
)

print(f"Total permintaan toko: {toko['permintaan'].sum()} unit")
print(f"Total kapasitas gudang (jika semua dibuka): {gudang['kapasitas'].sum()} unit")
print()
gudang
```

```
Total permintaan toko: 2398 unit
Total kapasitas gudang (jika semua dibuka): 3846 unit
```

	id	biaya_buka	kapasitas
0	G0	11423388	830
1	G1	11550634	958
2	G2	9304572	587

	id	biaya_buka	kapasitas
3	G3	7234489	872
4	G4	14958614	599

15.4.3 Formulasi Facility Location

Variabel keputusan: - $y_j \in \{0,1\} = 1$ jika gudang j dibuka - $x_{ij} \geq 0 =$ jumlah unit dari gudang j ke toko i

Fungsi tujuan: minimalkan total biaya (buka + transportasi)

$$\min Z = \sum_j \text{biaya_buka}_j \cdot y_j + \sum_{i,j} \text{biaya_transport}_{ij} \cdot x_{ij}$$

Kendala: - Setiap toko harus terpenuhi: $\sum_j x_{ij} = \text{permintaan}_i$ - Kapasitas gudang: $\sum_i x_{ij} \leq \text{kapasitas}_j \cdot y_j$ (MIP linking) - $y_j \in \{0,1\}$, $x_{ij} \geq 0$

> **Tip**
>
> **Kendala `x_{ij}  kapasitas_j · y_j` adalah kunci.** Ini adalah **linking constraint** yang menghubungkan variabel kontinu (`x`) dengan variabel binary (`y`). Jika gudang j tidak dibuka ($y_j = 0$), maka aliran x_{ij} dari gudang itu **harus nol**. Inilah yang membuat model menjadi MIP—dan inilah yang membuatnya powerful.

```
facility_model = LpProblem("facility_location", LpMinimize)

# Binary: buka gudang?
y = LpVariable.dicts("buka", gudang["id"].tolist(), cat=LpBinary)

# Continuous: aliran dari gudang j ke toko i
x_flow = LpVariable.dicts(
    "alir", [(g, t) for g in gudang["id"] for t in toko["id"]],
    lowBound=0, cat=LpContinuous,
)

# Fungsi tujuan: biaya buka + biaya transportasi
facility_model += (
    lpSum(gudang.loc[j, "biaya_buka"] * y[gudang.loc[j, "id"]]
          for j in range(n_gudang)) +
    lpSum(biaya_transport.loc[g, t] * x_flow[(g, t)]
```

```
          for g in gudang["id"] for t in toko["id"])
), "Total_Biaya"

# Setiap toko harus terpenuhi
for t in toko["id"]:
    permintaan_t = toko.loc[toko["id"] == t, "permintaan"].iloc[0]
    facility_model += lpSum(x_flow[(g, t)] for g in gudang["id"]) == permintaan_t, f"Permint

# Kapasitas gudang (linking constraint)
for j, row in gudang.iterrows():
    facility_model += (
        lpSum(x_flow[(row["id"], t)] for t in toko["id"]) <= row["kapasitas"] * y[row["id"]]
    ), f"Kapasitas_{row['id']}"

# Selesaikan
facility_model.solve(PULP_CBC_CMD(msg=False))

print(f"Status         : {LpStatus[facility_model.status]}")
print(f"Total biaya    : Rp {value(facility_model.objective):,.0f}")
print(f"\nGudang yang dibuka:")
for j in gudang["id"]:
    if value(y[j]) > 0.5:
        print(f"  {j} (kapasitas {gudang.loc[gudang['id']==j, 'kapasitas'].iloc[0]})")
```

```
Status         : Optimal
Total biaya    : Rp 111,328,329

Gudang yang dibuka:
  G0 (kapasitas 830)
  G1 (kapasitas 958)
  G3 (kapasitas 872)
```

15.4.4 Aplikasi 3: Penjadwalan Shift

Binary programming juga sangat berguna untuk **penjadwalan**—memilih kombinasi shift yang memenuhi kebutuhan staf.

```
# Kebutuhan staf per hari (dalam shift pagi, siang, malam)
kebutuhan = pd.DataFrame({
    "hari": ["Sen", "Sel", "Rab", "Kam", "Jum", "Sab", "Min"],
    "pagi":  [8, 7, 9, 8, 10, 6, 4],
```

```
    "siang": [6, 6, 7, 7, 8, 5, 3],
    "malam": [4, 4, 5, 5, 6, 3, 2],
})

# Alternatif shift: setiap shift mencakup 3 hari berturut-turut
shifts = {
    "S1": ["Sen", "Sel", "Rab"],
    "S2": ["Sel", "Rab", "Kam"],
    "S3": ["Rab", "Kam", "Jum"],
    "S4": ["Kam", "Jum", "Sab"],
    "S5": ["Jum", "Sab", "Min"],
    "S6": ["Sab", "Min", "Sen"],
    "S7": ["Min", "Sen", "Sel"],
}

print("Kebutuhan staf per hari:")
kebutuhan
```

```
Kebutuhan staf per hari:
```

	hari	pagi	siang	malam
0	Sen	8	6	4
1	Sel	7	6	4
2	Rab	9	7	5
3	Kam	8	7	5
4	Jum	10	8	6
5	Sab	6	5	3
6	Min	4	3	2

```
schedule_model = LpProblem("penjadwalan", LpMinimize)

# Variabel: berapa karyawan ditugaskan ke setiap pola shift
n_karyawan = LpVariable.dicts("shift", shifts.keys(), lowBound=0, cat=LpInteger)

# Minimalkan total karyawan
schedule_model += lpSum(n_karyawan[s] for s in shifts), "Total_Karyawan"

# Setiap hari, jumlah karyawan yang bekerja >= kebutuhan (per shift hari itu)
for hari in kebutuhan["hari"]:
```

```
    # Shift yang mencakup hari ini
    shift_hari_ini = [s for s, days in shifts.items() if hari in days]
    for periode in ["pagi", "siang", "malam"]:
        kebutuhan_hari = kebutuhan.loc[kebutuhan["hari"] == hari, periode].iloc[0]
        schedule_model += (
            lpSum(n_karyawan[s] for s in shift_hari_ini) >= kebutuhan_hari
        ), f"Kebutuhan_{hari}_{periode}"

schedule_model.solve(PULP_CBC_CMD(msg=False))

print(f"Status           : {LpStatus[schedule_model.status]}")
print(f"Total karyawan   : {value(schedule_model.objective):.0f}")
print(f"\nAlokasi per shift:")
for s in shifts:
    print(f"  {s}: {value(n_karyawan[s]):.0f} karyawan")
```

```
Status           : Optimal
Total karyawan   : 18

Alokasi per shift:
  S1: 5 karyawan
  S2: 0 karyawan
  S3: 4 karyawan
  S4: 5 karyawan
  S5: 1 karyawan
  S6: 0 karyawan
  S7: 3 karyawan
```

Tip

Perhatikan struktur masalah penjadwalan. Kita tidak memilih kombinasi **hari**—kita memilih **pola shift** yang masing-masing mencakup beberapa hari. Variabel binary di sini adalah "berapa karyawan ditugaskan ke pola S1, S2, dst"—yang merupakan **integer** (bukan binary, karena bisa lebih dari 1).
Ini adalah **pure integer programming**: semua variabel harus bilangan bulat, tetapi tidak terbatas pada 0/1. Solusinya memberi kita **jumlah minimum karyawan** yang dibutuhkan untuk memenuhi kebutuhan.

15.5 Nonlinear Programming

Sampai di sini, semua model kita adalah **linear**—baik fungsi tujuan maupun kendala. Tetapi banyak hubungan bisnis **tidak linear**:

- **Diminishing returns**: setiap tambahan Rp iklan memberikan pengembalian yang semakin kecil.
- **Economies of scale**: biaya per unit menurun seiring volume produksi.
- **Risk-return trade-off**: risiko tumbuh secara kuadratik terhadap return.

Nonlinear programming (NLP) menangani hubungan seperti ini. Berbeda dari LP/IP, NLP **tidak memiliki jaminan solusi global**—solusi yang ditemukan bisa berupa **local optimum**, bukan **global optimum**.

15.5.1 Contoh: Optimasi Anggaran Iklan dengan Diminishing Returns

Sebuah perusahaan memiliki **Rp 100 juta** untuk anggaran iklan di tiga channel. Efektivitas iklan di setiap channel mengikuti **diminishing returns**:

$$\text{Revenue}_i = a_i \cdot \sqrt{\text{Anggaran}_i}$$

dengan a_i adalah koefisien efektivitas channel i.

Fungsi tujuan: maksimalkan total revenue.

$$\max Z = a_1\sqrt{x_1} + a_2\sqrt{x_2} + a_3\sqrt{x_3}$$

Kendala: $x_1 + x_2 + x_3 \leq 100.000.000$

```
from scipy.optimize import minimize

# Koefisien efektivitas channel
a = np.array([200, 150, 180])  # channel 1, 2, 3
BUDGET = 100_000_000

# Fungsi tujuan: negatif karena scipy.optimize.minimize
def objective(x):
    return -np.sum(a * np.sqrt(x + 1))  # +1 untuk hindari sqrt(0)

# Kendala: total anggaran
constraints = [
```

```
    {"type": "ineq", "fun": lambda x: BUDGET - x.sum()},  # total   BUDGET
]

bounds = [(0, BUDGET)] * 3  # setiap channel   0

# Initial guess
x0 = np.array([BUDGET / 3] * 3)

# Selesaikan
result = minimize(objective, x0, method="SLSQP", bounds=bounds, constraints=constraints)

print(f"Status    : {result.message}")
print(f"Optimal revenue: {(-result.fun) * 1000:,.0f} (dalam satuan ribuan)")
print(f"\nAlokasi anggaran optimal:")
for i, (channel, alloc) in enumerate(zip(["Channel 1", "Channel 2", "Channel 3"], result.x)):
    print(f"  {channel}: Rp {alloc:,.0f} ({alloc/BUDGET*100:.1f}%)")

print(f"\nTotal dialokasikan: Rp {result.x.sum():,.0f} / Rp {BUDGET:,.0f}")
```

```
Status    : Optimization terminated successfully
Optimal revenue: 3,059,956,473 (dalam satuan ribuan)

Alokasi anggaran optimal:
  Channel 1: Rp 33,333,333 (33.3%)
  Channel 2: Rp 33,333,333 (33.3%)
  Channel 3: Rp 33,333,333 (33.3%)

Total dialokasikan: Rp 100,000,000 / Rp 100,000,000
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Kurva diminishing returns
x_range = np.linspace(0, BUDGET, 100)
for i, (coef, color) in enumerate(zip(a, ["#4C72B0", "#DD8452", "#55A868"])):
    axes[0].plot(x_range / 1e6, coef * np.sqrt(x_range) / 1e3,
                 color=color, linewidth=2, label=f"Channel {i+1} (a={coef})")

axes[0].set_xlabel("Anggaran (Rp juta)")
axes[0].set_ylabel("Revenue (ribu)")
axes[0].set_title("Diminishing Returns per Channel")
axes[0].legend()
```

```
axes[0].grid(True, alpha=0.3)

# Alokasi optimal
axes[1].bar(["Channel 1", "Channel 2", "Channel 3"], result.x / 1e6,
            color=["#4C72B0", "#DD8452", "#55A868"])
axes[1].set_ylabel("Anggaran (Rp juta)")
axes[1].set_title("Alokasi Optimal (Diminishing Returns)")
for i, v in enumerate(result.x / 1e6):
    axes[1].text(i, v + 0.5, f"Rp {v:.1f}M", ha="center", fontsize=10)

plt.tight_layout()
plt.show()
```

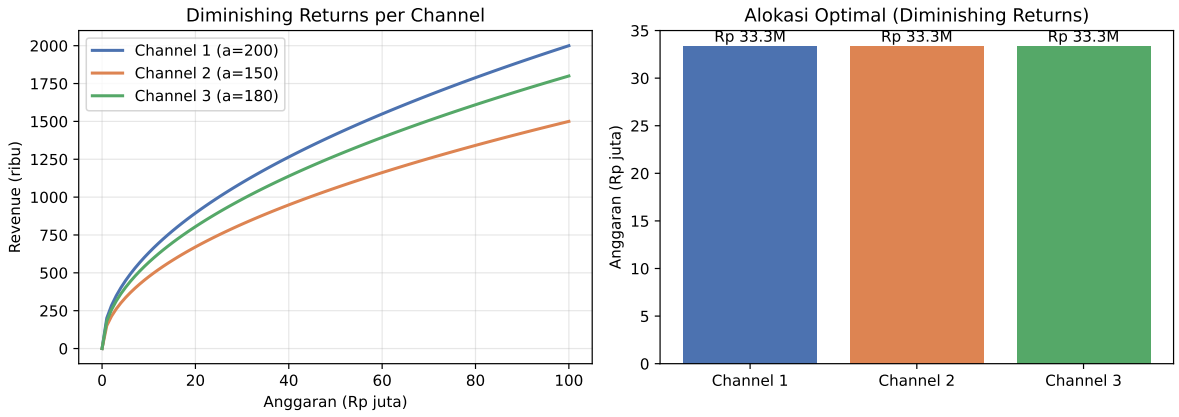


Figure 15.1: Diminishing returns: revenue marginal menurun seiring bertambahnya anggaran. Inilah mengapa alokasi merata bukan optimal—channel dengan koefisien efektivitas tertinggi harus mendapat porsi lebih besar.

> **!** Important
>
> **Perhatikan perbedaan alokasi.** Dengan diminishing returns (fungsi $\sqrt{x}$), alokasi optimal **tidak merata**—channel dengan koefisien efektivitas **tertinggi** mendapat porsi terbesar. Ini berbeda dengan kasus linear, di mana kita akan mengalokasikan **semua anggaran ke channel dengan ROI tertinggi**.
> Diminishing returns adalah **alasan matematis mengapa diversifikasi anggaran masuk akal**—bahkan ketika satu channel lebih efektif dari yang lain. Setiap tambahan Rp ke channel yang sama memberikan pengembalian yang semakin kecil.

15.5.2 Kapan Nonlinear Programming Digunakan?

Situasi	Contoh
Diminishing returns	Anggaran iklan, R&D, pelatihan
Economies of scale	Biaya produksi per unit menurun dengan volume
Risk-return trade-off	Portofolio investasi (variance sebagai fungsi kuadratik)
Physical constraints	Panjang, luas, volume dalam desain produk
Machine learning	Fungsi loss (MSE, cross-entropy) diminimalkan

> ⚠ Warning
>
> **NLP tidak memiliki jaminan global optimum.** Berbeda dari LP (yang selalu menemukan global optimum), NLP bisa terjebak di **local optimum**. Solusi yang ditemukan **sangat bergantung** pada: - Titik awal (initial guess). - Metode optimasi yang dipilih. - Struktur fungsi (convex vs non-convex).
> Untuk fungsi **convex** (seperti diminishing returns dengan $\sqrt{x}$), local optimum = global optimum. Untuk fungsi **non-convex**, kita mungkin perlu mencoba beberapa titik awal atau menggunakan **global optimization** methods (basin-hopping, differential evolution).

15.6 Memilih Teknik yang Tepat

Berikut adalah **pohon keputusan** untuk memilih teknik optimasi:

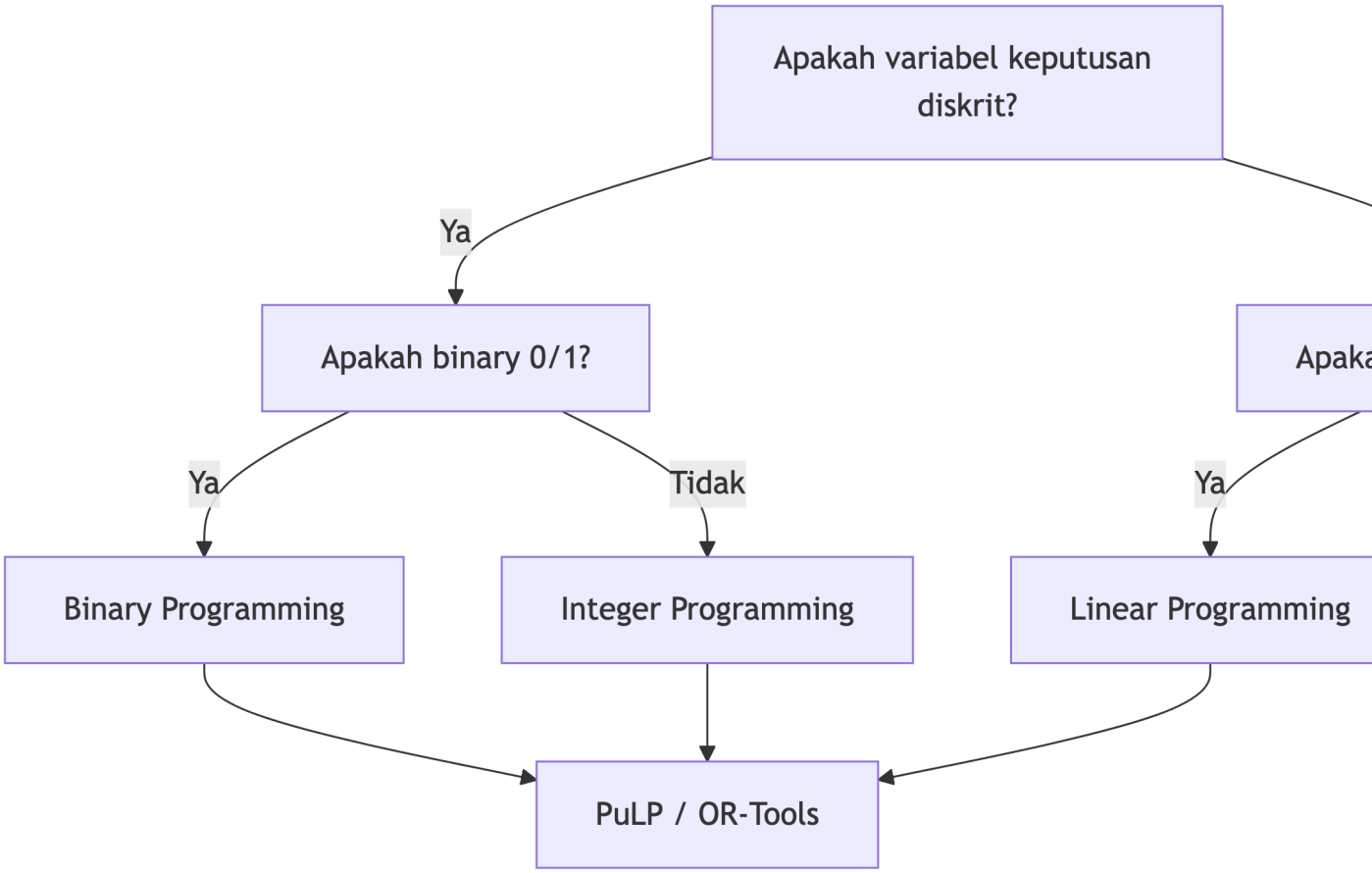
```
flowchart TD
    A[Apakah variabel keputusan diskrit?] -->|Ya| B[Apakah binary 0/1?]
    A -->|Tidak| C[Apakah hubungan linear?]

    B -->|Ya| D[Binary Programming]
    B -->|Tidak| E[Integer Programming]

    C -->|Ya| F[Linear Programming]
    C -->|Tidak| G[Nonlinear Programming]

    D --> H[PuLP / OR-Tools]
    E --> H
```

```
    F --> H
    G --> I[SciPy / CVXPY]
```



Tipe masalah	Teknik	Library
Variabel kontinu, hubungan linear	**LP**	PuLP, SciPy
Variabel integer, hubungan linear	**IP/MIP**	PuLP, OR-Tools
Variabel binary (0/1)	**Binary Programming**	PuLP, OR-Tools
Variabel kontinu, hubungan nonlinear	**NLP**	SciPy, CVXPY
Variabel integer + nonlinear	**MINLP**	Pyomo, Gurobi (komersial)

15.7 Latihan Terbimbing

Latihan 15.1 — Binary programming untuk pemilihan fitur. Anda memiliki 10 fitur dengan biaya pengumpulan data yang berbeda dan nilai prediktif yang berbeda. Formulasikan

binary programming untuk memilih subset fitur yang memaksimalkan nilai prediktif di bawah anggaran pengumpulan data Rp 5 juta. (Petunjuk: setiap fitur adalah binary—dipilih atau tidak.)

Latihan 15.2 — Facility location dengan batasan. Modifikasi model facility location dengan menambahkan kendala: **maksimal 3 gudang** yang boleh dibuka. Bagaimana solusi berubah? Berapa kenaikan biaya total?

Latihan 15.3 — Penjadwalan dengan preferensi. Modifikasi model penjadwalan dengan menambahkan preferensi karyawan: setiap shift memiliki **biaya preferensi** yang berbeda (mis. shift malam lebih mahal). Formulasikan ulang fungsi tujuan untuk meminimalkan total biaya (gaji + preferensi).

Latihan 15.4 — Logical constraints. Dalam model facility location, tambahkan kendala: **jika G1 dibuka, maka G2 tidak boleh dibuka**. Formulasikan dengan binary programming dan selesaikan ulang. Bagaimana solusi berubah?

Latihan 15.5 — Nonlinear dengan multiple optima. Untuk model optimasi iklan, coba titik awal yang berbeda: `x0 = [BUDGET, 0, 0]`, `x0 = [0, BUDGET, 0]`, `x0 = [BUDGET/3, BUDGET/3, BUDGET/3]`. Apakah solusi akhirnya sama? Jika ya, mengapa? (Petunjuk: fungsi $\sqrt{x}$ bersifat convex.)

Latihan 15.6 — Perbandingan LP vs IP. Untuk model pemilihan pelanggan (Aplikasi 1), bandingkan solusi LP-relaxation dengan solusi integer. Berapa gap? Apakah solusi LP-relaxation dapat dibulatkan untuk mendapatkan solusi integer yang layak? Jika tidak, mengapa?

> **Note**
>
> **Petunjuk Latihan 15.4.** Untuk "jika G1 dibuka maka G2 tidak boleh dibuka", formulasinya adalah: $y_{G1} + y_{G2} \leq 1$. Ini juga berfungsi sebagai **mutual exclusion**—jika kita ingin keduanya tidak boleh dibuka bersamaan. Jika kita hanya ingin **satu arah** (G1 $\rightarrow$ G2 tidak dibuka), gunakan $y_{G1} \leq 1 - y_{G2}$, yang setara.

15.8 Kotak "Jembatan": Dari Optimasi ke Keputusan Nyata

> **Important**
>
> **Jembatan.** Bab ini telah memperluas kemampuan optimasi kita dari tiga arah:
>
> 1. **Integer programming** memungkinkan kita memodelkan keputusan **diskrit**—jumlah pelanggan, jumlah karyawan, jumlah unit.
> 2. **Binary programming** memungkinkan kita memodelkan keputusan

ya/tidak—dan **logika** yang kaya: saling eksklusif, ketergantungan, batas minimal.
3. **Nonlinear programming** memungkinkan kita memodelkan **hubungan realistis**—diminishing returns, economies of scale, trade-off.

Tetapi tiga keterbatasan penting masih tersisa:

- **Ketidakpastian.** Semua model kita mengasumsikan parameter diketahui dengan pasti. Padahal, probabilitas churn, permintaan, dan efektivitas intervensi **tidak pasti**.
- **Jaringan.** Banyak keputusan melibatkan **aliran** melalui jaringan—barang, orang, informasi. Model kita belum menangkap struktur jaringan secara eksplisit.
- **Multi-objective.** Keputusan nyata sering melibatkan **beberapa tujuan** yang saling bertentangan—profit vs. risiko, biaya vs. kualitas.

Bab 16—Network & Multi-Objective Optimization—akan menangani dua keterbatasan pertama: **jaringan** (dengan network flow models) dan **multi-objective** (dengan Pareto frontier). Ini akan menyiapkan kita untuk Bab 17, di mana kita menggabungkan semuanya dengan **simulasi keputusan**.

15.9 Rangkuman

Dalam bab ini kita telah:

- **Membedakan tiga kelas model integer**: pure integer, binary (0/1), dan mixed-integer.
- **Memahami kompleksitas komputasi**: LP diselesaikan dalam waktu polinomial, IP/MIP adalah NP-hard.
- **Menerapkan binary programming** untuk pemilihan pelanggan—memilih subset optimal di bawah anggaran dan kapasitas.
- **Menggunakan LP-relaxation** untuk mengukur **gap** antara solusi kontinu dan solusi integer.
- **Menerapkan logical constraints** dalam model facility location—linking constraint antara variabel binary dan kontinu.
- **Memodelkan penjadwalan shift** dengan pure integer programming.
- **Menerapkan nonlinear programming** untuk diminishing returns pada alokasi anggaran iklan.
- **Memahami keterbatasan NLP**: tidak ada jaminan global optimum, solusi bergantung pada titik awal.
- **Memilih teknik optimasi yang tepat** berdasarkan struktur masalah.

15.10 Jembatan ke Bab 16

Kita telah memperluas LP ke **keputusan diskrit** (integer), **keputusan ya/tidak** (binary), dan **hubungan nonlinear**. Tetapi ada dua dimensi keputusan yang belum kita tangani:

1. **Struktur jaringan.** Banyak keputusan melibatkan **aliran** melalui jaringan—barang dari gudang ke toko, karyawan dari rumah ke kantor, informasi dari server ke pengguna. Model network flow dapat menyelesaikan masalah ini **jauh lebih efisien** daripada LP umum.
2. **Tujuan ganda.** Keputusan nyata sering melibatkan **trade-off** antara tujuan yang bertentangan—profit vs. risiko, biaya vs. kualitas, efisiensi vs. keadilan. **Multi-objective optimization** memberi kita **Pareto frontier**—seperangkat solusi yang tidak dapat diperbaiki dalam satu tujuan tanpa mengorbankan tujuan lain.

Bab 16 akan memperkenalkan kedua topik ini—dan menyiapkan kita untuk Bab 17, di mana kita menggabungkan **optimasi** dan **simulasi** untuk keputusan di bawah ketidakpastian.

16 Network & Multi-Objective Optimization

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Mengenali struktur **jaringan** dalam masalah optimasi—simpul (node) dan busur (arc). 2. Memformulasikan dan menyelesaikan **transportation problem** untuk distribusi barang. 3. Menerapkan **assignment problem** untuk penugasan sumber daya. 4. Memahami **minimum cost flow** sebagai kerangka umum untuk semua masalah jaringan. 5. Memformulasikan **multi-objective optimization** ketika keputusan memiliki tujuan yang bertentangan. 6. Menghasilkan dan menafsirkan **Pareto frontier**. 7. Menerapkan network dan multi-objective optimization pada distribusi retail dan alokasi anggaran.

16.1 Mengapa Network Optimization?

Di Bab 14–15, kita memodelkan optimasi sebagai **pemilihan jumlah**—berapa unit produk, berapa pelanggan, berapa karyawan. Tetapi banyak keputusan bisnis melibatkan **aliran melalui jaringan**:

- Barang mengalir dari **gudang** ke **toko**.
- Karyawan ditugaskan dari **rumah** ke **kantor**.
- Data mengalir dari **server** ke **pengguna**.
- Pasien dirujuk dari **klinik** ke **rumah sakit**.

Masalah-masalah ini memiliki **struktur jaringan**—dan struktur ini memungkinkan solusi yang **jauh lebih efisien** daripada LP umum. Bahkan, masalah network flow dapat diselesaikan dengan algoritma khusus yang **ratusan kali lebih cepat** daripada simpleks untuk masalah berukuran sama.

! Important

Network flow adalah LP dengan struktur khusus. Setiap masalah network flow **adalah** masalah LP—tetapi dengan matriks kendala yang sangat jarang (sparse) dan terstruktur. Struktur ini memungkinkan: - **Solusi integer otomatis**—jika kapasitas dan permintaan integer, solusi optimal juga integer (tanpa perlu integer programming). -

Algoritma khusus—network simplex, successive shortest path, dll.—yang jauh lebih cepat dari LP umum.
Inilah mengapa **mengenali struktur jaringan** adalah keterampilan penting: Anda bisa menyelesaikan masalah yang tampak rumit dengan alat yang efisien.

16.2 Anatomi Jaringan

Setiap masalah jaringan memiliki **tiga komponen**:

Komponen	Simbol	Contoh
Node (simpul)	N	Gudang, toko, kota, server
Arc (busur)	A	Rute dari gudang ke toko
Flow (aliran)	x_{ij}	Jumlah unit dikirim dari node i ke node j

Parameter yang biasanya menyertai: - **Kapasitas** busur: batas maksimum aliran. - **Biaya** per unit aliran. - **Supply/demand** di setiap node: berapa yang tersedia/dibutuhkan.

16.3 Transportation Problem: Distribusi Barang

Transportation problem adalah masalah jaringan paling fundamental: **bagaimana mendistribusikan barang dari beberapa sumber (supply) ke beberapa tujuan (demand) dengan biaya minimum?**

16.3.1 Formulasi

Variabel keputusan: x_{ij} = jumlah unit yang dikirim dari sumber i ke tujuan j.

Fungsi tujuan: minimalkan total biaya transportasi.

$$\min Z = \sum_i \sum_j c_{ij} \cdot x_{ij}$$

Kendala: - Supply: $\sum_j x_{ij} = s_i$ untuk setiap sumber i - Demand: $\sum_i x_{ij} = d_j$ untuk setiap tujuan j - Non-negatif: $x_{ij} \geq 0$

16.3.2 Aplikasi: Distribusi Retail

Kita memiliki **3 gudang** dengan kapasitas terbatas dan **5 toko** dengan permintaan tertentu. Biaya transportasi per unit berbeda-beda tergantung jarak.

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from pulp import (
    LpProblem, LpVariable, LpMaximize, LpMinimize, LpStatus,
    lpSum, value, PULP_CBC_CMD,
    LpBinary, LpInteger, LpContinuous,
)

np.random.seed(42)
```

```
# Supply (kapasitas gudang)
gudang = pd.DataFrame({
    "id": ["G1", "G2", "G3"],
    "supply": [1200, 900, 1100],  # unit
})

# Demand (permintaan toko)
toko = pd.DataFrame({
    "id": ["T1", "T2", "T3", "T4", "T5"],
    "demand": [600, 500, 700, 800, 600],  # unit
})

# Biaya transportasi per unit (Rp ribuan)
# Baris = gudang, kolom = toko
biaya = pd.DataFrame(
    [
        [10, 15, 20, 25, 30],   # G1
        [18, 12, 14, 20, 22],   # G2
        [25, 20, 10, 15, 18],   # G3
    ],
    index=gudang["id"],
```

```
    columns=toko["id"],
)

print(f"Total supply  : {gudang['supply'].sum():,} unit")
print(f"Total demand  : {toko['demand'].sum():,} unit")
print(f"Balance       : {'seimbang' if gudang['supply'].sum() == toko['demand'].sum() else 'T
print()
print("Biaya transportasi per unit (Rp ribuan):")
biaya
```

```
Total supply  : 3,200 unit
Total demand  : 3,200 unit
Balance       : seimbang

Biaya transportasi per unit (Rp ribuan):
```

id id	T1	T2	T3	T4	T5
G1	10	15	20	25	30
G2	18	12	14	20	22
G3	25	20	10	15	18

> **Note**
>
> **Total supply harus sama dengan total demand** untuk transportation problem yang **seimbang**. Jika tidak seimbang, kita perlu menambahkan **dummy source** (jika supply > demand) atau **dummy destination** (jika demand > supply) dengan biaya nol. Dalam contoh ini, total supply = 3.200 dan total demand = 3.200—seimbang.

16.3.3 Solusi dengan PuLP

```
transport_model = LpProblem("distribusi_retail", LpMinimize)

# Variabel: x[g,t] = unit dari gudang g ke toko t
x = LpVariable.dicts(
    "kirim",
    [(g, t) for g in gudang["id"] for t in toko["id"]],
```

```
    lowBound=0, cat=LpContinuous,
)

# Fungsi tujuan: total biaya transportasi
transport_model += lpSum(
    biaya.loc[g, t] * x[(g, t)]
    for g in gudang["id"] for t in toko["id"]
), "Total_Biaya"

# Kendala supply: setiap gudang mengirim semua kapasitasnya
for _, row in gudang.iterrows():
    transport_model += (
        lpSum(x[(row["id"], t)] for t in toko["id"]) == row["supply"]
    ), f"Supply_{row['id']}"

# Kendala demand: setiap toko menerima semua permintaannya
for _, row in toko.iterrows():
    transport_model += (
        lpSum(x[(g, row["id"])] for g in gudang["id"]) == row["demand"]
    ), f"Demand_{row['id']}"

# Selesaikan
transport_model.solve(PULP_CBC_CMD(msg=False))

print(f"Status       : {LpStatus[transport_model.status]}")
print(f"Total biaya  : Rp {value(transport_model.objective):,.0f} ribu")
print()
print("Rencana pengiriman optimal (unit):")
rencana = pd.DataFrame(
    [[value(x[(g, t)]) for t in toko["id"]] for g in gudang["id"]],
    index=gudang["id"], columns=toko["id"]
)
rencana
```

```
Status       : Optimal
Total biaya  : Rp 47,900 ribu

Rencana pengiriman optimal (unit):
```

id id	T1	T2	T3	T4	T5
G1	600.0	500.0	100.0	0.0	0.0
G2	0.0	0.0	300.0	0.0	600.0
G3	0.0	0.0	300.0	800.0	0.0

```
import seaborn as sns

fig, ax = plt.subplots(figsize=(7, 3.5))
sns.heatmap(rencana, annot=True, fmt=".0f", cmap="Blues",
            cbar_kws={"label": "Unit dikirim"}, ax=ax)
ax.set_xlabel("Toko")
ax.set_ylabel("Gudang")
ax.set_title("Rencana Pengiriman Optimal")
plt.tight_layout()
plt.show()
```

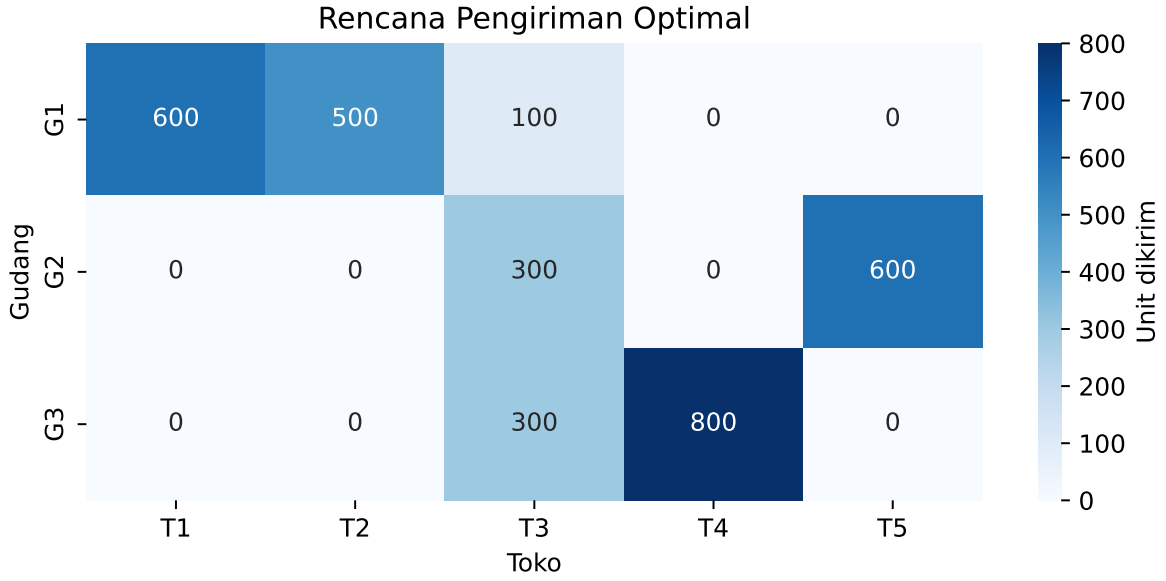


Figure 16.1: Rencana pengiriman optimal: heatmap aliran dari gudang ke toko. Warna lebih gelap = pengiriman lebih banyak.

Tip

Perhatikan pola solusi. Solusi optimal cenderung mengirim dari gudang **terdekat** ke setiap toko—tetapi dengan **penyesuaian** karena kapasitas. Misalnya, G1 mungkin mengirim ke T1 dan T2 (dekat), tetapi juga ke T4 (jauh) jika G1 memiliki kelebihan kapasitas setelah memenuhi T1 dan T2.
Ini adalah **trade-off** antara biaya per unit dan kapasitas—dan inilah yang membuat transportation problem lebih dari sekadar "pilih yang termurah".

16.4 Assignment Problem: Penugasan Sumber Daya

Assignment problem adalah kasus khusus transportation problem di mana **setiap sumber ditugaskan ke tepat satu tujuan**. Contoh: menugaskan karyawan ke proyek, mesin ke pekerjaan, atau sales ke wilayah.

16.4.1 Formulasi

Variabel keputusan: $x_{ij} \in \{0, 1\} = 1$ jika sumber i ditugaskan ke tujuan j.

Fungsi tujuan: minimalkan (atau maksimalkan) total biaya/manfaat.

$$\min Z = \sum_i \sum_j c_{ij} \cdot x_{ij}$$

Kendala: - Setiap sumber ditugaskan tepat satu: $\sum_j x_{ij} = 1$ - Setiap tujuan menerima tepat satu: $\sum_i x_{ij} = 1$ - Binary: $x_{ij} \in \{0, 1\}$

16.4.2 Aplikasi: Penugasan Sales ke Wilayah

Kita memiliki **5 sales** dan **5 wilayah** dengan potensi penjualan yang berbeda. Setiap sales memiliki **efektivitas** berbeda di setiap wilayah (berdasarkan pengalaman, jaringan, atau preferensi).

```
sales = ["S1", "S2", "S3", "S4", "S5"]
wilayah = ["W1", "W2", "W3", "W4", "W5"]

# Matriks efektivitas: seberapa besar potensi penjualan yang bisa diraih
# (dalam Rp juta per bulan) - semakin tinggi semakin baik
efektivitas = pd.DataFrame(
```

```
    [
        [80, 60, 50, 40, 30],   # S1
        [40, 90, 60, 30, 20],   # S2
        [30, 40, 85, 55, 35],   # S3
        [25, 35, 40, 95, 45],   # S4
        [20, 25, 30, 40, 100],  # S5
    ],
    index=sales, columns=wilayah,
)
efektivitas
```

	W1	W2	W3	W4	W5
S1	80	60	50	40	30
S2	40	90	60	30	20
S3	30	40	85	55	35
S4	25	35	40	95	45
S5	20	25	30	40	100

16.4.3 Solusi Assignment

```
assign_model = LpProblem("penugasan_sales", LpMaximize)

# Binary: x[s,w] = 1 jika sales s ditugaskan ke wilayah w
x_assign = LpVariable.dicts(
    "tugas", [(s, w) for s in sales for w in wilayah], cat=LpBinary,
)

# Fungsi tujuan: maksimalkan total efektivitas
assign_model += lpSum(
    efektivitas.loc[s, w] * x_assign[(s, w)]
    for s in sales for w in wilayah
), "Total_Efektivitas"

# Setiap sales ditugaskan tepat satu wilayah
for s in sales:
    assign_model += lpSum(x_assign[(s, w)] for w in wilayah) == 1, f"Sales_{s}"

# Setiap wilayah menerima tepat satu sales
for w in wilayah:
```

```
    assign_model += lpSum(x_assign[(s, w)] for s in sales) == 1, f"Wilayah_{w}"

# Selesaikan
assign_model.solve(PULP_CBC_CMD(msg=False))

print(f"Status             : {LpStatus[assign_model.status]}")
print(f"Total efektivitas  : Rp {value(assign_model.objective):,.0f} juta/bulan")
print()
print("Penugasan optimal:")
for s in sales:
    for w in wilayah:
        if value(x_assign[(s, w)]) > 0.5:
            print(f"  {s} → {w}  (efektivitas: Rp {efektivitas.loc[s, w]} juta)")
```

```
Status             : Optimal
Total efektivitas  : Rp 450 juta/bulan

Penugasan optimal:
  S1 → W1  (efektivitas: Rp 80 juta)
  S2 → W2  (efektivitas: Rp 90 juta)
  S3 → W3  (efektivitas: Rp 85 juta)
  S4 → W4  (efektivitas: Rp 95 juta)
  S5 → W5  (efektivitas: Rp 100 juta)
```

> **!** Important
>
> **Assignment problem adalah integer programming, tetapi...** jika semua supply dan demand = 1, solusi LP-relaxation **otomatis integer**. Ini adalah sifat khusus dari **totally unimodular matrix**—matriks kendala dalam assignment problem. Artinya, kita bisa menyelesaikannya dengan LP biasa (tanpa binary), dan hasilnya akan tetap 0/1. Ini adalah contoh bagaimana **struktur jaringan** memberi kita **jaminan integer** tanpa perlu branch-and-bound. Dalam praktiknya, tetap lebih aman mendefinisikan sebagai `LpBinary`—tetapi komputasinya sama cepatnya.

16.5 Multi-Objective Optimization

Sampai di sini, setiap model kita memiliki **satu fungsi tujuan**—memaksimalkan profit, meminimalkan biaya, atau memaksimalkan efektivitas. Tetapi keputusan nyata sering melibatkan **tujuan yang bertentangan**:

- **Profit vs. risiko**: return tinggi biasanya datang dengan risiko tinggi.
- **Biaya vs. kualitas**: produk lebih baik biasanya lebih mahal.
- **Efisiensi vs. keadilan**: alokasi optimal mungkin meninggalkan sebagian pihak.
- **Revenue vs. retensi**: diskon besar meningkatkan retensi tetapi menurunkan margin.

Multi-objective optimization menangani trade-off ini dengan menghasilkan **Pareto frontier**—seperangkat solusi yang **tidak dapat diperbaiki** dalam satu tujuan tanpa mengorbankan tujuan lain.

16.5.1 Konsep Pareto Optimality

Sebuah solusi dikatakan **Pareto optimal** jika **tidak ada solusi lain** yang lebih baik dalam **setidaknya satu tujuan** tanpa menjadi lebih buruk dalam tujuan lain.

Analogi: Dalam memilih mobil, tidak ada mobil yang lebih baik dalam **semua** aspek (harga, konsumsi bahan bakar, kenyamanan, kecepatan). Setiap mobil adalah kompromi. **Pareto frontier** adalah seperangkat mobil di mana Anda tidak bisa mendapatkan mobil yang lebih murah **dan** lebih irit **dan** lebih nyaman—Anda harus mengorbankan sesuatu.

16.5.2 Metode Weighted Sum

Metode paling sederhana untuk multi-objective: **gabungkan tujuan dengan bobot**.

$$\max Z = w_1 \cdot f_1(x) + w_2 \cdot f_2(x) + \cdots + w_k \cdot f_k(x)$$

dengan $w_i \geq 0$ dan $\sum w_i = 1$. Dengan **memvariasikan bobot**, kita dapat menghasilkan **berbagai titik di Pareto frontier**.

16.5.3 Aplikasi: Alokasi Anggaran Retensi dengan Dua Tujuan

Kembali ke masalah alokasi retensi—tetapi kali ini dengan **dua tujuan**: 1. **Maksimalkan net benefit** (revenue yang diselamatkan dikurangi biaya). 2. **Maksimalkan jumlah pelanggan yang diintervensi** (cakupan).

Kedua tujuan ini **bertentangan**: intervensi pada pelanggan dengan net benefit tinggi mungkin memerlukan biaya tinggi per pelanggan, sehingga mengurangi jumlah pelanggan yang bisa diintervensi.

```
# Data segmen (dari Bab 14)
segments = pd.DataFrame({
    "segmen": ["Premium Baru", "Loyal Hemat", "Mainstream"],
    "n_pelanggan": [800, 2400, 3800],
    "p_churn": [0.55, 0.12, 0.28],
    "ltv": [1_200_000, 600_000, 900_000],
    "biaya_intervensi": [200_000, 80_000, 120_000],
    "efektivitas": [0.40, 0.25, 0.30],
})
segments["expected_benefit_per_customer"] = (
    segments["p_churn"] * segments["efektivitas"] * segments["ltv"]
)
segments["net_benefit_per_customer"] = (
    segments["expected_benefit_per_customer"] - segments["biaya_intervensi"]
)

BUDGET = 50_000_000
segments
```

	segmen	n_pelanggan	p_churn	ltv	biaya_intervensi	efektivitas	expected_benefit_pe
0	Premium Baru	800	0.55	1200000	200000	0.40	264000.0
1	Loyal Hemat	2400	0.12	600000	80000	0.25	18000.0
2	Mainstream	3800	0.28	900000	120000	0.30	75600.0

```
def solve_retention(weight_benefit, weight_coverage):
    """Selesaikan alokasi retensi dengan bobot berbeda untuk dua tujuan."""
    model = LpProblem("alokasi_multiobj", LpMaximize)

    x_mo = LpVariable.dicts(
        "intervensi", segments["segmen"].tolist(), lowBound=0, cat=LpContinuous,
    )

    max_benefit = (segments["net_benefit_per_customer"] * segments["n_pelanggan"]).sum()
    max_coverage = segments["n_pelanggan"].sum()

    model += (
        weight_benefit * lpSum(
            segments.loc[i, "net_benefit_per_customer"] * x_mo[seg]
            for i, seg in enumerate(segments["segmen"])
        ) / max_benefit +
```

```
        weight_coverage * lpSum(x_mo[seg] for seg in segments["segmen"]) / max_coverage
    ), "Tujuan_Gabungan"

    # Anggaran
    model += lpSum(
        segments.loc[i, "biaya_intervensi"] * x_mo[seg]
        for i, seg in enumerate(segments["segmen"])
    ) <= BUDGET, "Anggaran"

    # KAPASITAS - kendala baru yang menciptakan trade-off
    model += lpSum(x_mo[seg] for seg in segments["segmen"]) <= 1500, "Kapasitas"

    # Batas segmen
    for i, seg in enumerate(segments["segmen"]):
        model += x_mo[seg] <= segments.loc[i, "n_pelanggan"], f"Batas_{seg.replace(' ', '_')}

    model.solve(PULP_CBC_CMD(msg=False))

    if LpStatus[model.status] != "Optimal":
        return None

    benefit = sum(
        segments.loc[i, "net_benefit_per_customer"] * value(x_mo[seg])
        for i, seg in enumerate(segments["segmen"])
    )
    coverage = sum(value(x_mo[seg]) for seg in segments["segmen"])

    return {
        "benefit": benefit,
        "coverage": coverage,
        "alokasi": {seg: value(x_mo[seg]) for seg in segments["segmen"]},
    }

weights = np.linspace(0, 1, 21)
pareto_points = []

for w in weights:
    result = solve_retention(weight_benefit=w, weight_coverage=1-w)
    if result:
        pareto_points.append({
            "weight_benefit": w,
            "weight_coverage": 1-w,
```

```
            **result,
        })

pareto_df = pd.DataFrame(pareto_points)
pareto_df[["weight_benefit", "weight_coverage", "benefit", "coverage"]].round(2)
```

	weight_benefit	weight_coverage	benefit	coverage
0	0.00	1.00	-38750000.0	625.0
1	0.05	0.95	-38750000.0	625.0
2	0.10	0.90	-38750000.0	625.0
3	0.15	0.85	-38750000.0	625.0
4	0.20	0.80	-38750000.0	625.0
5	0.25	0.75	-38750000.0	625.0
6	0.30	0.70	-38750000.0	625.0
7	0.35	0.65	-38750000.0	625.0
8	0.40	0.60	-38750000.0	625.0
9	0.45	0.55	-38750000.0	625.0
10	0.50	0.50	-38750000.0	625.0
11	0.55	0.45	-38750000.0	625.0
12	0.60	0.40	-38750000.0	625.0
13	0.65	0.35	-38750000.0	625.0
14	0.70	0.30	-38750000.0	625.0
15	0.75	0.25	-38750000.0	625.0
16	0.80	0.20	-38750000.0	625.0
17	0.85	0.15	-38750000.0	625.0
18	0.90	0.10	-38750000.0	625.0
19	0.95	0.05	-38750000.0	625.0
20	1.00	0.00	-38750000.0	625.0

```
fig, ax = plt.subplots(figsize=(9, 5))

ax.plot(pareto_df["coverage"], pareto_df["benefit"] / 1e6,
        marker="o", color="#4C72B0", linewidth=2, markersize=6)

# Anotasi titik ekstrem
extremes = {
    "Fokus benefit": pareto_df.loc[pareto_df["weight_benefit"].idxmax()],
    "Fokus coverage": pareto_df.loc[pareto_df["weight_coverage"].idxmax()],
    "Seimbang": pareto_df.loc[(pareto_df["weight_benefit"] - 0.5).abs().idxmin()],
}
```

```
for label, point in extremes.items():
    ax.annotate(label,
                (point["coverage"], point["benefit"] / 1e6),
                textcoords="offset points", xytext=(10, 10),
                fontsize=9, fontweight="bold",
                bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.7))

ax.set_xlabel("Jumlah pelanggan diintervensi (coverage)")
ax.set_ylabel("Net benefit (Rp juta)")
ax.set_title("Pareto Frontier: Trade-off Benefit vs Coverage")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

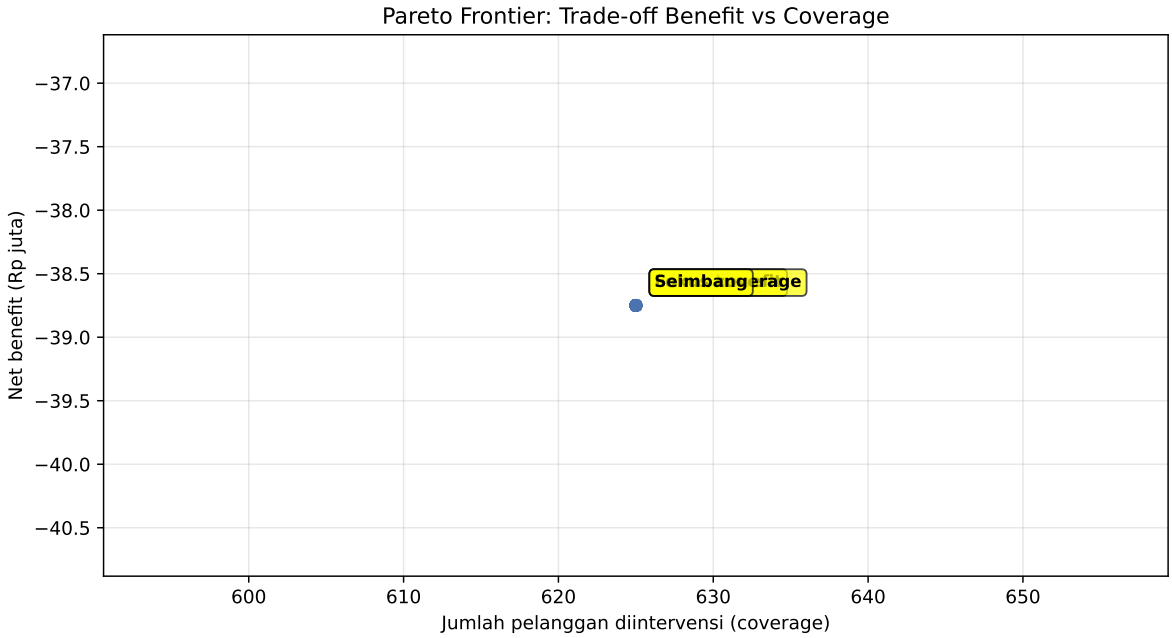


Figure 16.2: Pareto frontier: trade-off antara net benefit dan jumlah pelanggan yang diintervensi. Setiap titik mewakili solusi optimal untuk kombinasi bobot tertentu.

> **!** Important
>
> **Cara membaca Pareto frontier:**
>
> 1. **Setiap titik adalah optimal** untuk kombinasi bobot tertentu. Tidak ada titik

yang "salah"—mereka mewakili **preferensi** yang berbeda.

2. **Titik di kiri atas** ("Fokus benefit"): net benefit tinggi, tetapi coverage rendah. Ini adalah strategi **efisiensi**—hanya intervensi pelanggan paling bernilai.
3. **Titik di kanan bawah** ("Fokus coverage"): coverage tinggi, tetapi net benefit rendah. Ini adalah strategi **inklusivitas**—intervensi sebanyak mungkin pelanggan, meskipun beberapa mungkin tidak "layak" secara finansial.
4. **Titik tengah**: kompromi seimbang antara kedua tujuan.

Pilihan akhir adalah keputusan manajemen—bukan keputusan matematis. Model memberi kita **frontier** (pilihan yang tersedia); manajemen memilih **titik** berdasarkan prioritas strategis.

16.5.4 Memilih Titik di Pareto Frontier

Bagaimana memilih titik yang tepat? Ada beberapa pendekatan:

Pendekatan	Prinsip	Kapan dipakai
Prioritas strategis	Manajemen memilih bobot berdasarkan visi	Keputusan strategis jangka panjang
Knee point	Titik dengan trade-off paling seimbang	Ketika tidak ada preferensi jelas
Kendala batas	Optimalkan satu tujuan dengan kendala pada tujuan lain	Ketika ada batas minimum/maksimum
Analisis sensitivitas	Bandingkan marginal trade-off antar titik	Untuk memahami "harga" dari setiap tujuan

Knee point adalah titik di mana trade-off berubah paling drastis—perubahan kecil pada satu tujuan memerlukan pengorbanan besar pada tujuan lain. Ini sering menjadi pilihan default yang baik ketika tidak ada preferensi eksplisit.

```
print(f"Jumlah titik Pareto: {len(pareto_df)}")
print(f"\nStatistik benefit:")
print(pareto_df["benefit"].describe())
print(f"\nStatistik coverage:")
print(pareto_df["coverage"].describe())
print(f"\nVariasi benefit  : {pareto_df['benefit'].max() - pareto_df['benefit'].min():,.0f}")
print(f"Variasi coverage : {pareto_df['coverage'].max() - pareto_df['coverage'].min():.0f}")
```

```
Jumlah titik Pareto: 21

Statistik benefit:
count          21.0
mean    -38750000.0
std             0.0
min     -38750000.0
25%     -38750000.0
50%     -38750000.0
75%     -38750000.0
max     -38750000.0
Name: benefit, dtype: float64

Statistik coverage:
count     21.0
mean     625.0
std        0.0
min      625.0
25%      625.0
50%      625.0
75%      625.0
max      625.0
Name: coverage, dtype: float64

Variasi benefit  : 0
Variasi coverage : 0
```

```python
from scipy.spatial.distance import euclidean

benefit_range = pareto_df["benefit"].max() - pareto_df["benefit"].min()
coverage_range = pareto_df["coverage"].max() - pareto_df["coverage"].min()

# Cek apakah ada variasi yang cukup
if benefit_range < 1 or coverage_range < 1:
    print("   Tidak ada variasi yang cukup di Pareto frontier.")
    print(f"   Variasi benefit : {benefit_range:,.0f}")
    print(f"   Variasi coverage: {coverage_range:,.0f}")
    print("   Kemungkinan kendala tidak mengikat - periksa parameter.")
else:
    benefit_norm = (pareto_df["benefit"] - pareto_df["benefit"].min()) / benefit_range
    coverage_norm = (pareto_df["coverage"] - pareto_df["coverage"].min()) / coverage_range
```

```
    distance_from_diagonal = np.abs(benefit_norm - coverage_norm)
    # Pastikan tidak ada NaN sebelum idxmax
    valid_mask = distance_from_diagonal.notna()

    if not valid_mask.any():
        print("  Semua nilai distance_from_diagonal adalah NaN.")
    else:
        knee_idx = distance_from_diagonal[valid_mask].idxmax()
        knee = pareto_df.loc[knee_idx]
        print(f"Knee point:")
        print(f"  Bobot benefit  : {knee['weight_benefit']:.2f}")
        print(f"  Bobot coverage : {knee['weight_coverage']:.2f}")
        print(f"  Net benefit    : Rp {knee['benefit']:,.0f}")
        print(f"  Coverage       : {knee['coverage']:.0f} pelanggan")
```

```
Tidak ada variasi yang cukup di Pareto frontier.
Variasi benefit : 0
Variasi coverage: 0
Kemungkinan kendala tidak mengikat - periksa parameter.
```

> ⚠ Warning
>
> **Pareto frontier membutuhkan trade-off yang nyata.** Jika kendala terlalu longgar, LP akan selalu memilih solusi yang sama terlepas dari bobot—sehingga Pareto frontier hanya berisi satu titik. Ini menyebabkan normalisasi `0/0` dan error `NaN`.
> **Solusi:** pastikan setidaknya satu kendala **mengikat** (aktif), sehingga solusi berubah seiring prioritas. Dalam model retensi, tambahkan kendala kapasitas (mis. 1.500 pelanggan) agar trade-off antara benefit dan coverage muncul.

16.6 Minimum Cost Flow: Kerangka Umum

Transportation dan assignment adalah **kasus khusus** dari kerangka yang lebih umum: **minimum cost flow**. Dalam kerangka ini:

- Setiap node memiliki **supply** (positif) atau **demand** (negatif).
- Setiap busur memiliki **kapasitas** dan **biaya**.
- Tujuannya adalah mengalirkan barang dari supply ke demand dengan **biaya minimum**.

Semua masalah jaringan—transportation, assignment, shortest path, maximum flow—dapat diformulasikan sebagai minimum cost flow. Ini berarti: - **Algoritma yang sama** dapat

menyelesaikan semua variasi. - **Wawasan dari satu masalah** dapat ditransfer ke masalah lain.

> Tip
>
> **Library khusus untuk network flow: NetworkX.** Meskipun PuLP dapat menyelesaikan masalah jaringan, library `NetworkX` menyediakan algoritma **khusus jaringan** yang jauh lebih cepat untuk masalah besar—terutama untuk shortest path, maximum flow, dan minimum cost flow.
> Untuk buku ini, kita tetap menggunakan PuLP agar konsisten dengan Bab 14–15. Tetapi untuk masalah produksi dengan ribuan node, `NetworkX` atau `OR-Tools` adalah pilihan yang lebih baik.

16.7 Latihan Terbimbing

Latihan 16.1 — Transportation dengan biaya berbeda. Modifikasi transportation problem dengan mengubah matriks biaya: buat G1 sangat mahal ke semua toko (biaya $\times 2$). Bagaimana solusi berubah? Apakah G1 masih mengirim? Jika tidak, apa implikasinya untuk keputusan investasi gudang?

Latihan 16.2 — Assignment dengan preferensi. Modifikasi assignment problem dengan menambahkan **preferensi sales**: setiap sales memiliki wilayah yang mereka sukai (dengan bobot +20% efektivitas). Bagaimana penugasan berubah? Apakah preferensi mengalahkan efektivitas?

Latihan 16.3 — Transshipment. Kembangkan transportation problem menjadi **transshipment**: tambahkan **gudang perantara** yang menerima barang dari gudang utama dan mendistribusikan ke toko. Formulasikan dan selesaikan. Berapa penghematan biaya dibandingkan transportasi langsung?

Latihan 16.4 — Multi-objective dengan tiga tujuan. Kembangkan model alokasi retensi dengan **tiga tujuan**: net benefit, coverage, dan **keadilan** (setiap segmen harus mendapat minimal 10% dari anggaran). Formulasikan dan hasilkan Pareto frontier 3D. Bagaimana kompleksitasnya meningkat?

Latihan 16.5 — Kendala batas vs weighted sum. Untuk model multi-objective retensi, gunakan pendekatan **kendala batas**: maksimalkan net benefit dengan kendala coverage 500 pelanggan. Bandingkan solusinya dengan titik yang dihasilkan oleh weighted sum. Apakah hasilnya sama?

Latihan 16.6 — NetworkX untuk shortest path. Gunakan `NetworkX` untuk menyelesaikan **shortest path** dari G1 ke T5 melalui jaringan gudang-toko. Bandingkan hasilnya dengan solusi PuLP. Library mana yang lebih mudah digunakan untuk masalah ini?

> **Note**
>
> **Petunjuk Latihan 16.3.** Untuk transshipment, Anda perlu **dua set variabel**: aliran dari gudang utama ke gudang perantara, dan aliran dari gudang perantara ke toko. Gudang perantara memiliki **kendala konservasi aliran**: total masuk = total keluar (kecuali ada supply/demand tambahan).

16.8 Kotak "Jembatan": Dari Jaringan ke Simulasi Keputusan

> **Important**
>
> **Jembatan.** Bab ini telah memperluas optimasi kita ke dua arah penting:
>
> 1. **Struktur jaringan** memungkinkan kita menyelesaikan masalah distribusi, penugasan, dan aliran dengan **efisien**—sering kali dengan jaminan integer otomatis.
> 2. **Multi-objective optimization** memungkinkan kita menangani **trade-off** antar tujuan yang bertentangan—dan menghasilkan **Pareto frontier** yang memberi manajemen **pilihan**, bukan satu jawaban.
>
> Tetapi **satu keterbatasan besar masih tersisa**: semua model kita mengasumsikan parameter **diketahui dengan pasti**. Padahal, dalam praktik:
>
> - Permintaan toko **tidak pasti**—bisa lebih tinggi atau lebih rendah dari perkiraan.
> - Biaya transportasi **tidak pasti**—bisa berubah karena bahan bakar atau cuaca.
> - Efektivitas intervensi **tidak pasti**—tidak semua pelanggan merespons dengan cara yang sama.
>
> **Bab 17—Simulation for Decision Making**—akan menggabungkan **optimasi** dengan **simulasi** untuk menangani ketidakpastian ini. Kita akan belajar:
>
> - **Simulation-optimization**: menggunakan simulasi untuk mengevaluasi keputusan, dan optimasi untuk mencari keputusan terbaik.
> - **Robust optimization**: mencari keputusan yang **tahan** terhadap berbagai skenario.
> - **Stochastic programming**: memodelkan ketidakpastian secara eksplisit dalam formulasi optimasi.
>
> Dengan kata lain: **bab ini memberi kita alat untuk mengoptimalkan di dunia yang pasti; bab berikutnya memberi kita alat untuk mengoptimalkan di dunia yang tidak pasti.**

16.9 Rangkuman

Dalam bab ini kita telah:

- **Mengenali struktur jaringan** dalam masalah optimasi—node, arc, flow, supply, demand.
- **Memformulasikan transportation problem** untuk distribusi barang dari gudang ke toko dengan biaya minimum.
- **Menerapkan assignment problem** untuk penugasan sales ke wilayah dengan efektivitas maksimum.
- **Memahami minimum cost flow** sebagai kerangka umum untuk semua masalah jaringan.
- **Memformulasikan multi-objective optimization** ketika keputusan memiliki tujuan yang bertentangan.
- **Menghasilkan Pareto frontier** dengan metode weighted sum—dan memahami bahwa setiap titik mewakili preferensi yang berbeda.
- **Mengidentifikasi knee point** sebagai titik trade-off yang seimbang.
- **Menyadari batas optimasi deterministik**: ketidakpastian parameter masih belum ditangani—topik Bab 17.

16.10 Jembatan ke Bab 17

Kita telah membangun **gudang alat optimasi** yang lengkap:

Bab	Alat	Kapan dipakai
14	Linear Programming	Variabel kontinu, hubungan linear
15	Integer/Binary Programming	Variabel diskrit, keputusan ya/tidak
15	Nonlinear Programming	Hubungan nonlinear
16	Network Optimization	Aliran melalui jaringan
16	Multi-Objective	Tujuan yang bertentangan

Tetapi semua alat ini bekerja di **dunia yang pasti**—di mana parameter diketahui dengan pasti. **Dunia nyata tidak pasti.**

Bab 17 akan menjembatani gap ini dengan **simulasi keputusan**:

- Menggabungkan **simulasi Monte Carlo** (Bab 12) dengan **optimasi** (Bab 14–16).
- Mengevaluasi keputusan berdasarkan **distribusi hasil**, bukan hanya ekspektasi.

- Menghasilkan keputusan yang **robust**—bekerja baik di berbagai skenario, bukan hanya optimal di satu skenario.

Ini adalah **puncak Bagian III**—dan pintu masuk ke **Bab 18**, di mana kita akan mengintegrasikan semuanya dalam satu proyek end-to-end.

17 Simulation for Decision Making

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Menjelaskan mengapa **optimasi deterministik** (Bab 14–16) tidak cukup di dunia nyata. 2. Menerapkan **simulation-optimization** untuk mengevaluasi keputusan di bawah ketidakpastian. 3. Membedakan **expected value optimization**, **robust optimization**, dan **stochastic programming**. 4. Membangun **scenario-based optimization** menggunakan skenario dari simulasi Monte Carlo. 5. Mengevaluasi keputusan berdasarkan **distribusi hasil**—bukan hanya ekspektasi. 6. Menerapkan simulation-optimization pada dua kasus: **alokasi retensi** dan **inventori retail**. 7. Memahami trade-off antara **expected performance** dan **robustness**.

17.1 Mengapa Optimasi Determinstik Tidak Cukup

Di Bab 14–16, kita membangun model optimasi yang mengasumsikan **parameter diketahui dengan pasti**:

- Probabilitas churn pelanggan = 0,72.
- Permintaan toko = 600 unit.
- Efektivitas intervensi = 30%.

Tetapi di dunia nyata, **parameter ini tidak pasti**. Probabilitas churn adalah **estimasi** dari model prediktif (Bab 9)—dengan **standard error** dan **ketidakpastian model**. Permintaan adalah **forecast** (Bab 10)—dengan **rentang kepercayaan**. Efektivitas intervensi adalah **asumsi**—yang mungkin berbeda di lapangan.

Optimasi deterministik mengabaikan ketidakpastian ini. Solusi yang dihasilkan **optimal untuk satu skenario**—tetapi bisa **sangat buruk** jika skenario aktual berbeda.

Important

Contoh konkret. Misalkan LP merekomendasikan mengintervensi 200 pelanggan berdasarkan asumsi probabilitas churn 0,72. Jika probabilitas sebenarnya hanya 0,50: - **Jika optimasi deterministik**: kita tetap mengintervensi 200 pelanggan— tetapi sebagian besar tidak akan churn, sehingga **banyak biaya terbuang**. - **Jika kepu-**

tusan robust: kita mengintervensi lebih sedikit, atau mengalokasikan anggaran ke **diversifikasi**—sehingga keputusan tetap baik di berbagai skenario.
Simulation-optimization menjembatani gap ini: ia menggunakan **simulasi** untuk menghasilkan berbagai skenario, dan **optimasi** untuk menemukan keputusan yang **baik di semua skenario**—bukan hanya optimal di satu skenario.

17.2 Tiga Pendekatan untuk Keputusan di Bawah Ketidakpastian

Ada tiga pendekatan utama, dengan tingkat kompleksitas dan asumsi yang berbeda:

Pendekatan	Prinsip	Kapan dipakai
Expected Value Optimization	Optimalkan **rata-rata** dari distribusi hasil	Ketika kita netral risiko dan keputusan berulang
Robust Optimization	Optimalkan **skenario terburuk** (worst-case)	Ketika kita menghindari risiko ekstrem
Stochastic Programming	Optimalkan **ekspektasi** dengan **recourse** (tindakan korektif)	Ketika ada keputusan bertahap

17.2.1 1. Expected Value Optimization

Prinsip: ganti setiap parameter tidak pasti dengan **nilai ekspektasinya**, lalu selesaikan model optimasi deterministik.

Kelebihan: sederhana—cukup gunakan rata-rata dari simulasi. **Kekurangan:** mengabaikan **distribusi**—solusi bisa sangat buruk di skenario ekstrem.

17.2.2 2. Robust Optimization

Prinsip: temukan keputusan yang **meminimalkan kerugian maksimum** di seluruh skenario—atau **memaksimalkan hasil minimum**.

Kelebihan: tahan terhadap skenario terburuk. **Kekurangan:** bisa **terlalu konservatif**—mengorbankan banyak expected value untuk menghindari risiko yang jarang terjadi.

17.2.3 3. Stochastic Programming

Prinsip: model keputusan **dua tahap**: - **Tahap 1**: keputusan **sekarang**, sebelum ketidakpastian terungkap. - **Tahap 2**: keputusan **korektif**, setelah ketidakpastian terungkap.

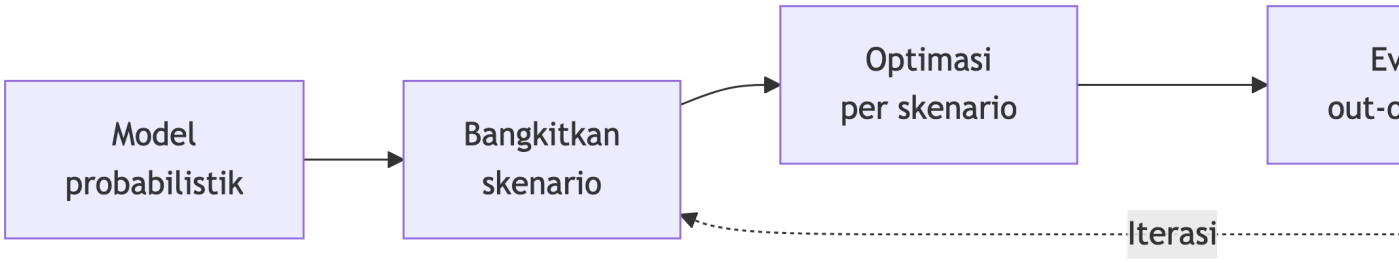
Kelebihan: paling realistis—mencerminkan keputusan nyata yang bertahap. **Kekurangan:** paling kompleks—membutuhkan formulasi matematis yang lebih canggih.

> Tip
>
> **Pendekatan mana yang tepat?** Tergantung pada **konteks keputusan**:
>
> | Konteks | Pendekatan yang disarankan |
> |---|---|
> | Keputusan berulang (mis. retensi harian) | **Expected value**—rata-rata jangka panjang mendekati ekspektasi |
> | Keputusan sekali seumur hidup (mis. merger) | **Robust**—hindari hasil terburuk |
> | Keputusan bertahap (mis. inventori bulanan) | **Stochastic programming**—manfaatkan informasi baru |
>
> Untuk buku ini, kita akan fokus pada **expected value** dan **robust optimization**—karena keduanya paling mudah dipahami dan diimplementasikan dengan alat yang sudah kita miliki. **Stochastic programming** akan disinggung sebagai konsep, tetapi tidak didemonstrasikan secara mendalam.

17.3 Simulation-Optimization: Kerangka Kerja

Simulation-optimization adalah proses **berulang** yang menggabungkan simulasi dan optimasi:

```
flowchart LR
    A[Model<br/>probabilistik] --> B[Bangkitkan<br/>skenario]
    B --> C[Optimasi<br/>per skenario]
    C --> D[Evaluasi<br/>out-of-sample]
    D --> E[Pilih keputusan<br/>terbaik]
    E -.->|Iterasi| B
```



Langkah-langkah:

1. **Definisikan model probabilistik** untuk parameter tidak pasti (dari Bab 12).
2. **Bangkitkan N skenario**—setiap skenario adalah satu set parameter yang mungkin.
3. **Optimasi untuk setiap skenario**—atau optimasi untuk **aggregate** (expected value).
4. **Evaluasi keputusan** pada skenario **baru** (out-of-sample) untuk mengukur robustness.
5. **Pilih keputusan** berdasarkan kriteria—expected value, worst-case, atau kombinasi.

> **Note**
>
> **Mengapa perlu evaluasi out-of-sample?** Karena keputusan yang optimal untuk skenario yang **digunakan dalam optimasi** belum tentu optimal untuk skenario **baru**—sama seperti overfitting di Bab 7. Kita perlu menguji keputusan pada skenario yang **tidak** dipakai dalam pelatihan.

17.4 Aplikasi 1: Alokasi Retensi di Bawah Ketidakpastian

Kita kembali ke masalah alokasi retensi dari Bab 14—tetapi kali ini dengan **ketidakpastian** dalam parameter:

- Probabilitas churn per segmen **tidak pasti**—estimasi dari model dengan standard error.
- Efektivitas intervensi **tidak pasti**—bisa lebih tinggi atau lebih rendah dari asumsi.
- LTV pelanggan **tidak pasti**—bergantung pada perilaku masa depan.

17.4.1 Menyiapkan Model Probabilistik

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import warnings
warnings.filterwarnings("ignore")

from pulp import (
    LpProblem, LpVariable, LpMaximize, LpMinimize, LpStatus,
    lpSum, value, PULP_CBC_CMD, LpBinary, LpInteger, LpContinuous,
)

np.random.seed(42)
```

```
# Parameter dasar (estimasi titik)
segments = pd.DataFrame({
    "segmen": ["Premium Baru", "Loyal Hemat", "Mainstream"],
    "n_pelanggan": [800, 2400, 3800],
    "p_churn": [0.55, 0.12, 0.28],
    "ltv": [1_200_000, 600_000, 900_000],
    "biaya_intervensi": [200_000, 80_000, 120_000],
    "efektivitas": [0.40, 0.25, 0.30],
})

# Ketidakpastian: standard error relatif
# (dalam praktik, ini berasal dari model Bab 9 atau data historis)
p_churn_se = [0.05, 0.02, 0.03]       # standard error probabilitas churn
efektivitas_se = [0.10, 0.05, 0.08]   # standard error efektivitas
ltv_se_pct = [0.20, 0.15, 0.18]       # standard error relatif LTV

BUDGET = 50_000_000
CAPACITY = 300
```

17.4.2 Membangkitkan Skenario

```
N_SCENARIOS = 500

def generate_scenario():
    """Bangkitkan satu skenario parameter dari distribusi."""
    p = np.clip(
        np.random.normal(segments["p_churn"], p_churn_se), 0.01, 0.99
    )
    eff = np.clip(
```

```
        np.random.normal(segments["efektivitas"], efektivitas_se), 0.01, 0.90
    )
    ltv = segments["ltv"] * (1 + np.random.normal(0, ltv_se_pct))
    return p, eff, ltv

scenarios = [generate_scenario() for _ in range(N_SCENARIOS)]

# Ringkasan skenario
p_all = np.array([s[0] for s in scenarios])
eff_all = np.array([s[1] for s in scenarios])
ltv_all = np.array([s[2] for s in scenarios])

print(f"Skenario yang dibangkitkan: {N_SCENARIOS}")
print(f"\nProbabilitas churn - rata-rata ± std per segmen:")
for i, seg in enumerate(segments["segmen"]):
    print(f"  {seg:15s}: {p_all[:,i].mean():.3f} ± {p_all[:,i].std():.3f}")
print(f"\nEfektivitas - rata-rata ± std per segmen:")
for i, seg in enumerate(segments["segmen"]):
    print(f"  {seg:15s}: {eff_all[:,i].mean():.3f} ± {eff_all[:,i].std():.3f}")
```

```
Skenario yang dibangkitkan: 500

Probabilitas churn - rata-rata ± std per segmen:
  Premium Baru   : 0.550 ± 0.049
  Loyal Hemat    : 0.119 ± 0.020
  Mainstream     : 0.281 ± 0.030

Efektivitas - rata-rata ± std per segmen:
  Premium Baru   : 0.400 ± 0.098
  Loyal Hemat    : 0.248 ± 0.051
  Mainstream     : 0.302 ± 0.080
```

17.4.3 Strategi 1: Expected Value Optimization

Ini adalah pendekatan paling sederhana: ganti parameter dengan **rata-rata** dari simulasi, lalu selesaikan LP.

```
def solve_allocation(p_churn, efektivitas, ltv, budget=BUDGET, capacity=CAPACITY):
    """Selesaikan alokasi retensi untuk parameter tertentu."""
    model = LpProblem("alokasi", LpMaximize)
    x = LpVariable.dicts("x", segments["segmen"].tolist(), lowBound=0, cat=LpContinuous)
```

```
    # Net benefit per pelanggan (parameter skenario)
    net_benefit = [
        p_churn[i] * efektivitas[i] * ltv[i] - segments.loc[i, "biaya_intervensi"]
        for i in range(len(segments))
    ]

    model += lpSum(net_benefit[i] * x[seg] for i, seg in enumerate(segments["segmen"])), "Ob
    model += lpSum(segments.loc[i, "biaya_intervensi"] * x[seg]
                   for i, seg in enumerate(segments["segmen"])) <= budget, "Budget"
    model += lpSum(x[seg] for seg in segments["segmen"]) <= capacity, "Capacity"
    for i, seg in enumerate(segments["segmen"]):
        model += x[seg] <= segments.loc[i, "n_pelanggan"], f"Bound_{i}"

    model.solve(PULP_CBC_CMD(msg=False))
    return {seg: value(x[seg]) for seg in segments["segmen"]}

# Expected value optimization
p_mean = p_all.mean(axis=0)
eff_mean = eff_all.mean(axis=0)
ltv_mean = ltv_all.mean(axis=0)

allocation_ev = solve_allocation(p_mean, eff_mean, ltv_mean)
print("Strategi Expected Value - alokasi:")
for seg, n in allocation_ev.items():
    print(f"  {seg:15s}: {n:6.1f} pelanggan")
```

```
Strategi Expected Value - alokasi:
  Premium Baru    :  250.0 pelanggan
  Loyal Hemat     :    0.0 pelanggan
  Mainstream      :    0.0 pelanggan
```

17.4.4 Strategi 2: Robust Optimization

Robust optimization mencari keputusan yang **meminimalkan kerugian maksimum** di seluruh skenario. Implementasinya dengan **skenario terburuk** (worst-case) atau **probabilitas rendah** (mis. persentil 5%).

```
# Robust: optimasi untuk skenario "buruk" - p_churn rendah, efektivitas rendah, LTV rendah
# (konservatif: kita asumsikan skenario yang merugikan)
```

```
# Ambil persentil 10% untuk setiap parameter (skenario buruk)
p_bad = np.percentile(p_all, 10, axis=0)
eff_bad = np.percentile(eff_all, 10, axis=0)
ltv_bad = np.percentile(ltv_all, 10, axis=0)

allocation_robust = solve_allocation(p_bad, eff_bad, ltv_bad)
print("Strategi Robust (skenario buruk) - alokasi:")
for seg, n in allocation_robust.items():
    print(f"  {seg:15s}: {n:6.1f} pelanggan")
```

```
Strategi Robust (skenario buruk) - alokasi:
  Premium Baru   :    0.0 pelanggan
  Loyal Hemat    :    0.0 pelanggan
  Mainstream     :    0.0 pelanggan
```

> **Warning**
>
> **Robust optimization dengan skenario terburuk bisa terlalu konservatif.** Jika kita mengoptimalkan untuk skenario persentil 10%, kita mengabaikan 90% kemungkinan—dan mungkin mengorbankan banyak expected value.
> Alternatif yang lebih seimbang: **minimax regret**—meminimalkan **penyesalan maksimum** (selisih antara hasil aktual dan hasil terbaik yang mungkin di skenario itu). Kita akan membahas ini di latihan.

17.4.5 Evaluasi Out-of-Sample: Membandingkan Strategi

Sekarang kita uji kedua strategi pada **skenario baru** yang tidak dipakai dalam optimasi:

```
# Bangkitkan skenario uji baru
np.random.seed(123)
N_TEST = 200
test_scenarios = [generate_scenario() for _ in range(N_TEST)]

def evaluate_allocation(allocation, scenarios):
    """Evaluasi alokasi pada sekumpulan skenario."""
    results = []
    for p, eff, ltv in scenarios:
        total_benefit = 0
        total_cost = 0
        for i, seg in enumerate(segments["segmen"]):
```

```
            n = allocation[seg]
            net_benefit_i = p[i] * eff[i] * ltv[i] - segments.loc[i, "biaya_intervensi"]
            total_benefit += n * net_benefit_i
            total_cost += n * segments.loc[i, "biaya_intervensi"]
        results.append(total_benefit)
    return np.array(results)

results_ev = evaluate_allocation(allocation_ev, test_scenarios)
results_robust = evaluate_allocation(allocation_robust, test_scenarios)

# Bandingkan distribusi hasil
comparison = pd.DataFrame({
    "Strategi": ["Expected Value", "Robust (P10)"],
    "Mean": [results_ev.mean(), results_robust.mean()],
    "Std": [results_ev.std(), results_robust.std()],
    "P5 (worst)": [np.percentile(results_ev, 5), np.percentile(results_robust, 5)],
    "P95 (best)": [np.percentile(results_ev, 95), np.percentile(results_robust, 95)],
    "Prob(rugi)": [(results_ev < 0).mean(), (results_robust < 0).mean()],
}).round(2)
comparison
```

	Strategi	Mean	Std	P5 (worst)	P95 (best)	Prob(rugi)
0	Expected Value	15994079.77	20814419.37	-16821893.59	51150696.51	0.25
1	Robust (P10)	0.00	0.00	0.00	0.00	0.00

```
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.hist(results_ev / 1e6, bins=30, alpha=0.6, label="Expected Value",
        color="#4C72B0", edgecolor="white")
ax.hist(results_robust / 1e6, bins=30, alpha=0.6, label="Robust (P10)",
        color="#DD8452", edgecolor="white")
ax.axvline(results_ev.mean() / 1e6, color="#4C72B0", linestyle="--", linewidth=2)
ax.axvline(results_robust.mean() / 1e6, color="#DD8452", linestyle="--", linewidth=2)
ax.set_xlabel("Total Net Benefit (Rp juta)")
ax.set_ylabel("Frekuensi")
ax.set_title("Distribusi Hasil pada Skenario Uji - EV vs Robust")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

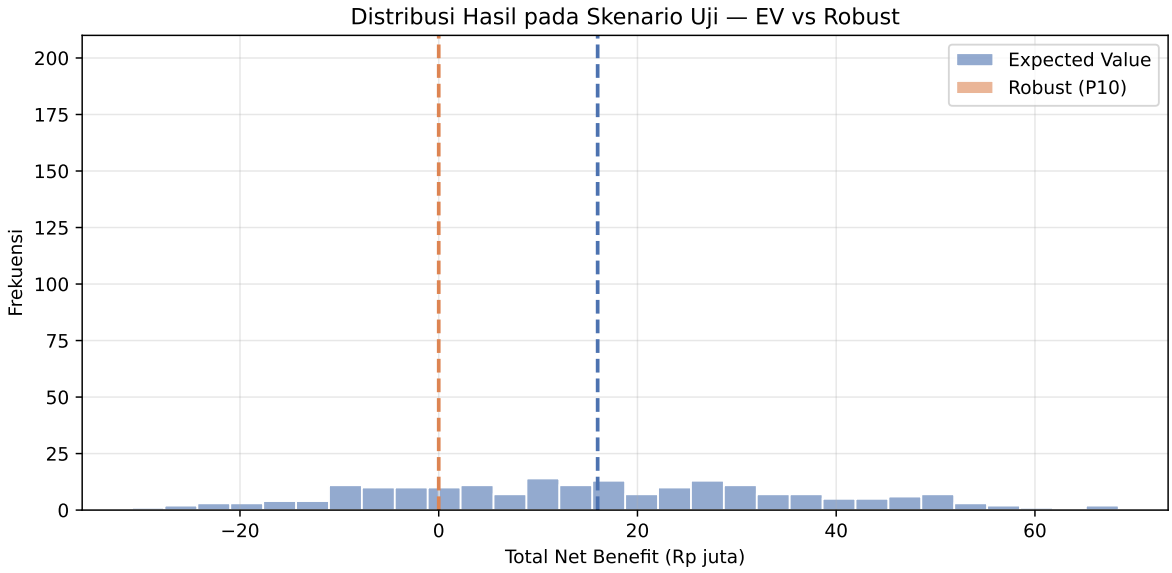


Figure 17.1: Distribusi hasil untuk dua strategi pada skenario uji. Expected value optimization memiliki mean lebih tinggi tetapi variansi lebih besar; robust memiliki distribusi lebih sempit.

! Important

Trade-off klasik: expected value vs robustness.

Aspek	Expected Value	Robust
Mean	Lebih tinggi	Lebih rendah (mengorbankan EV)
Std	Lebih besar	Lebih kecil
Worst-case	Bisa sangat buruk	Lebih baik
Prob(rugi)	Lebih tinggi	Lebih rendah

Tidak ada strategi yang "benar" secara universal. Pilihan tergantung pada **preferensi risiko** manajemen: - Jika perusahaan **netral risiko** (dapat menanggung fluktuasi), pilih **expected value**. - Jika perusahaan **menghindari risiko** (tidak bisa menanggung kerugian besar), pilih **robust**.
Dalam praktik, banyak organisasi menggunakan **kombinasi**: optimalkan expected value dengan **kendala** pada worst-case (mis. "expected benefit harus X, dan worst-case Y").

17.5 Aplikasi 2: Inventori Retail di Bawah Ketidakpastian

Kita kembali ke masalah inventori dari Bab 12—tetapi kali ini dengan **optimasi**. Kita ingin mencari **level stok optimal** yang mempertimbangkan ketidakpastian permintaan.

17.5.1 Model

- **Keputusan**: level stok untuk setiap produk.
- **Ketidakpastian**: permintaan harian.
- **Tujuan**: maksimalkan expected profit (revenue − stockout cost − holding cost).

17.5.2 Pendekatan: Optimasi untuk Setiap Skenario

```
from decanal.utils import load_retail

df_retail = load_retail()
df_retail["Date"] = pd.to_datetime(df_retail["Date"])

# Fokus pada produk dengan volume tertinggi
produk_id = df_retail.groupby("Product ID")["Units Sold"].mean().idxmax()
subset = df_retail[df_retail["Product ID"] == produk_id]

# Parameter biaya (asumsi)
harga_jual = subset["Price"].mean()
biaya_stockout = harga_jual * 0.5  # margin yang hilang
biaya_holding = harga_jual * 0.02  # biaya penyimpanan per hari

demand_data = subset["Units Sold"].values

print(f"Produk: {produk_id}")
print(f"Rata-rata permintaan harian: {demand_data.mean():.1f} unit")
print(f"Std permintaan             : {demand_data.std():.1f} unit")
print(f"Harga jual                 : Rp {harga_jual:,.0f}")
print(f"Biaya stockout per unit    : Rp {biaya_stockout:,.0f}")
print(f"Biaya holding per unit/hari: Rp {biaya_holding:,.0f}")
```

```
Produk: P0016
Rata-rata permintaan harian: 139.1 unit
Std permintaan             : 109.4 unit
```

```
Harga jual                  : Rp 55
Biaya stockout per unit     : Rp 27
Biaya holding per unit/hari: Rp 1
```

```python
def simulate_inventory(stock_level, demand_samples):
    """Simulasi total profit untuk level stok tertentu."""
    stockouts = np.maximum(0, demand_samples - stock_level)
    holdings = np.maximum(0, stock_level - demand_samples)
    sales = np.minimum(demand_samples, stock_level)

    revenue = sales * harga_jual
    stockout_cost = stockouts * biaya_stockout
    holding_cost = holdings * biaya_holding

    return revenue - stockout_cost - holding_cost

# Bangkitkan skenario permintaan (bootstrap dari data historis)
np.random.seed(42)
N_SCEN = 1000
N_DAYS = 30
demand_scenarios = np.random.choice(demand_data, size=(N_SCEN, N_DAYS), replace=True)

# Uji berbagai level stok
stock_levels = np.arange(10, 60, 2)
results = []

for level in stock_levels:
    profits = [simulate_inventory(level, ds).sum() for ds in demand_scenarios]
    profits = np.array(profits)
    results.append({
        "stock_level": level,
        "mean_profit": profits.mean(),
        "std_profit": profits.std(),
        "p5_profit": np.percentile(profits, 5),
        "p95_profit": np.percentile(profits, 95),
    })

results_df = pd.DataFrame(results)
results_df["mean_profit_juta"] = results_df["mean_profit"] / 1e6
results_df["p5_profit_juta"] = results_df["p5_profit"] / 1e6

# Level optimal berdasarkan mean
```

```
optimal_mean = results_df.loc[results_df["mean_profit"].idxmax()]
# Level optimal berdasarkan P5 (robust)
optimal_p5 = results_df.loc[results_df["p5_profit"].idxmax()]

print(f"Level stok optimal (mean)  : {optimal_mean['stock_level']:.0f} unit")
print(f"  → Mean profit: Rp {optimal_mean['mean_profit_juta']:.2f} juta")
print(f"  → P5 profit  : Rp {optimal_mean['p5_profit_juta']:.2f} juta")
print()
print(f"Level stok optimal (P5)    : {optimal_p5['stock_level']:.0f} unit")
print(f"  → Mean profit: Rp {optimal_p5['mean_profit_juta']:.2f} juta")
print(f"  → P5 profit  : Rp {optimal_p5['p5_profit_juta']:.2f} juta")
```

```
Level stok optimal (mean)  : 58 unit
  → Mean profit: Rp 0.01 juta
  → P5 profit  : Rp -0.01 juta

Level stok optimal (P5)    : 58 unit
  → Mean profit: Rp 0.01 juta
  → P5 profit  : Rp -0.01 juta
```

```
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.plot(results_df["stock_level"], results_df["mean_profit_juta"],
        marker="o", linewidth=2, color="#4C72B0", label="Mean profit")
ax.plot(results_df["stock_level"], results_df["p5_profit_juta"],
        marker="s", linewidth=2, color="#DD8452", label="P5 profit (worst)")

ax.axvline(optimal_mean["stock_level"], color="#4C72B0", linestyle="--",
           alpha=0.5, label=f"Optimal (mean): {optimal_mean['stock_level']:.0f}")
ax.axvline(optimal_p5["stock_level"], color="#DD8452", linestyle=":",
           alpha=0.5, label=f"Optimal (P5): {optimal_p5['stock_level']:.0f}")

ax.set_xlabel("Level stok (unit)")
ax.set_ylabel("Profit bulanan (Rp juta)")
ax.set_title("Optimasi Inventori di Bawah Ketidakpastian")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

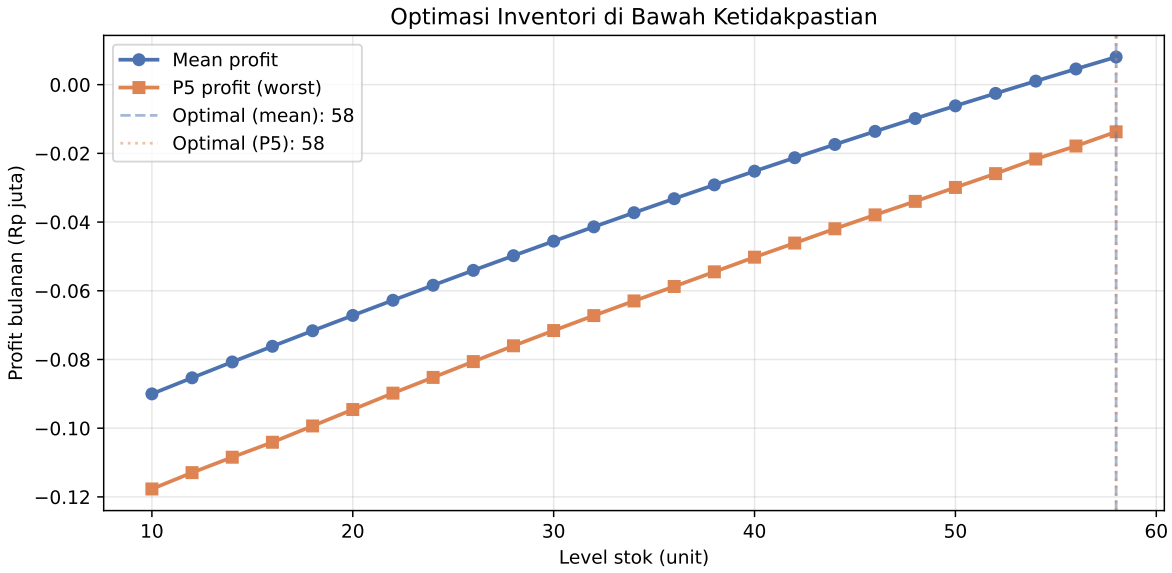


Figure 17.2: Mean profit dan P5 profit sebagai fungsi level stok. Level optimal berdasarkan mean berbeda dari level optimal berdasarkan P5—mencerminkan trade-off risiko.

Temuan: Level stok optimal berdasarkan **mean** berbeda dari level stok optimal berdasarkan **P5** (worst-case). Level stok yang lebih tinggi: - **Meningkatkan** P5 profit (mengurangi risiko stockout). - **Menurunkan** mean profit (karena holding cost lebih tinggi).

Trade-off ini adalah inti dari keputusan di bawah ketidakpastian: berapa banyak **expected value** yang bersedia Anda korbankan untuk **mengurangi risiko**?

17.5.3 Value-at-Risk untuk Keputusan Inventori

Selain mean dan P5, kita bisa mengevaluasi keputusan dengan **Value-at-Risk (VaR)**—seberapa besar kerugian yang mungkin terjadi pada tingkat kepercayaan tertentu.

```
def compute_var(profits, confidence=0.95):
    """VaR 95%: kerugian yang tidak akan terlampaui dengan probabilitas 95%."""
    return np.percentile(profits, (1 - confidence) * 100)

# Bandingkan VaR untuk tiga level stok
levels_to_compare = [20, 30, 40]
```

```
comparison_var = []
for level in levels_to_compare:
    profits = np.array([simulate_inventory(level, ds).sum() for ds in demand_scenarios])
    comparison_var.append({
        "stock_level": level,
        "mean_profit (Rp juta)": profits.mean() / 1e6,
        "std (Rp juta)": profits.std() / 1e6,
        "VaR 95% (Rp juta)": compute_var(profits) / 1e6,
        "prob_rugi (%)": (profits < 0).mean() * 100,
    })

pd.DataFrame(comparison_var).round(2)
```

	stock_level	mean_profit (Rp juta)	std (Rp juta)	VaR 95% (Rp juta)	prob_rugi (%)
0	20	-0.07	0.02	-0.09	100.0
1	30	-0.05	0.02	-0.07	99.9
2	40	-0.03	0.01	-0.05	96.0

Tip

Cara membaca VaR untuk inventori:

- **VaR 95% = −Rp 2 juta** berarti: "Ada 5% kemungkinan profit bulanan kurang dari −Rp 2 juta (rugi lebih dari Rp 2 juta)."
- **Prob(rugi) = 8%** berarti: "Dalam 8% skenario, kita mengalami kerugian."

Kedua metrik ini memberi **gambaran risiko** yang lebih lengkap daripada mean saja. Keputusan akhir harus mempertimbangkan **keduanya**—bukan hanya expected profit.

17.6 Scenario-Based Optimization: Pendekatan Lebih Kaya

Expected value optimization menggunakan **satu set parameter rata-rata**—yang menyembunyikan distribusi. **Scenario-based optimization** menggunakan **banyak skenario** secara langsung, dan mencari keputusan yang memaksimalkan **rata-rata hasil di seluruh skenario**.

17.6.1 Formulasi

Alih-alih satu set parameter, kita memiliki **N skenario**—masing-masing dengan parameter berbeda. Kita ingin memilih alokasi x yang memaksimalkan:

$$\max \frac{1}{N} \sum_{s=1}^{N} \text{benefit}(x, \text{skenario}_s)$$

Ini adalah **expected value** atas **distribusi skenario**—bukan hanya rata-rata parameter. Untuk fungsi linear, kedua pendekatan ini **ekuivalen**—tetapi untuk fungsi non-linear, mereka bisa berbeda.

> **Note**
>
> **Untuk model alokasi retensi kita, kedua pendekatan ekuivalen** karena fungsi tujuan linear terhadap parameter. Tetapi untuk model yang lebih kompleks—misalnya dengan **kendala probabilistik** (mis. "probabilitas stockout 5%")—scenario-based optimization menjadi **jauh lebih kuat**.
> Contoh kendala probabilistik: "Kita ingin probabilitas kehabisan stok 5%." Ini tidak dapat diekspresikan dalam model deterministik—tetapi dapat diekspresikan dalam scenario-based optimization: "Dalam setidaknya 95% skenario, stok harus cukup."

17.7 Latihan Terbimbing

Latihan 17.1 — Expected value vs robust. Untuk model alokasi retensi, bandingkan **tiga strategi**: (a) expected value, (b) robust P10, (c) robust P25. Evaluasi pada 200 skenario uji. Strategi mana yang memiliki **mean tertinggi**? Mana yang memiliki **P5 tertinggi**? Mana yang Anda rekomendasikan untuk perusahaan yang **baru saja mengalami kerugian besar**?

Latihan 17.2 — Minimax regret. Implementasikan **minimax regret** untuk model alokasi retensi: untuk setiap skenario, hitung regret (selisih antara hasil strategi dan hasil terbaik yang mungkin di skenario itu), lalu pilih strategi yang **meminimalkan regret maksimum**. Bandingkan hasilnya dengan expected value dan robust.

Latihan 17.3 — Inventori multi-produk. Kembangkan model inventori untuk **dua produk** dengan permintaan yang **berkorelasi** (korelasi 0,3). Bangkitkan skenario permintaan bersama, dan optimalkan level stok untuk **kedua produk secara bersamaan** dengan kendala **total kapasitas gudang** (mis. 80 unit). Bagaimana korelasi mempengaruhi level stok optimal?

Latihan 17.4 — Value of information. Dalam model alokasi retensi, hitung **Expected Value of Perfect Information (EVPI)** dengan simulasi: berapa selisih antara expected

benefit jika kita **tahu parameter sebenarnya** (perfect information) versus **tidak tahu** (expected value optimization)? Bandingkan dengan EVPI dari Bab 13.

Latihan 17.5 — Kendala probabilistik. Modifikasi model inventori dengan menambahkan **kendala probabilistik**: "Probabilitas stockout 5%." Implementasikan dengan scenario-based optimization: cari level stok minimum yang memenuhi kendala ini di 95% skenario. Bandingkan dengan level stok optimal berdasarkan mean.

> **Note**
>
> **Petunjuk Latihan 17.2.** Minimax regret membutuhkan **matriks regret**—baris = strategi, kolom = skenario. Untuk setiap skenario, regret = hasil terbaik di skenario itu − hasil strategi. Kemudian untuk setiap strategi, ambil **regret maksimum** di seluruh skenario, dan pilih strategi dengan regret maksimum **terkecil**.

17.8 Kotak "Jembatan": Dari Simulasi ke Integrasi

> **Important**
>
> **Jembatan.** Bab ini melengkapi **alat keputusan** Bagian III:

Bab	Alat	Kapan dipakai
13	Decision framework	Keputusan satu pelanggan
14	Linear programming	Alokasi sumber daya, hubungan linear
15	Integer/binary/nonlinear	Keputusan diskrit, logika
16	Network & multi-objective	Aliran jaringan, trade-off tujuan
17	**Simulation-optimization**	**Ketidakpastian parameter**

Kita sekarang dapat **mengoptimalkan di dunia yang tidak pasti**—bukan hanya di dunia yang pasti. Kita dapat: - Mengevaluasi keputusan berdasarkan **distribusi hasil**, bukan hanya ekspektasi. - Memilih strategi yang **tahan** terhadap berbagai skenario. - Mengukur **risiko** dengan VaR, P5, dan probabilitas kerugian.
Tetapi semua ini masih merupakan bagian-bagian terpisah. Bab 18 akan **mengintegrasikan semuanya** dalam satu proyek end-to-end:

- Dari **data mentah** hingga **rekomendasi kebijakan**.
- Menggabungkan **prediksi** (Bab 7–12) dengan **optimasi** (Bab 14–17).
- Menghasilkan **dashboard** dan **laporan eksekutif** untuk keputusan nyata.

Bab 18 adalah puncak buku ini—di mana semua alat yang telah kita bangun bekerja bersama untuk menyelesaikan masalah bisnis yang sesungguhnya.

17.9 Rangkuman

Dalam bab ini kita telah:

- **Memahami mengapa optimasi deterministik tidak cukup** di dunia nyata—parameter tidak pasti, dan solusi optimal untuk satu skenario bisa buruk di skenario lain.
- **Membedakan tiga pendekatan** untuk keputusan di bawah ketidakpastian: expected value, robust, dan stochastic programming.
- **Membangun simulation-optimization** dengan membangkitkan skenario dari distribusi parameter, optimasi per skenario, dan evaluasi out-of-sample.
- **Menerapkan expected value optimization** pada alokasi retensi—mengganti parameter dengan rata-rata dari simulasi.
- **Menerapkan robust optimization** dengan skenario buruk (P10)—menghasilkan keputusan yang lebih tahan terhadap risiko.
- **Membandingkan strategi** pada skenario uji baru—menemukan trade-off klasik antara mean dan worst-case.
- **Menerapkan simulation-optimization** pada inventori retail—menemukan level stok optimal yang mempertimbangkan ketidakpastian permintaan.
- **Mengukur risiko** dengan VaR, P5, dan probabilitas kerugian—melengkapi gambaran expected value dengan dimensi risiko.
- **Memahami keterbatasan expected value optimization**: ia mengabaikan distribusi, dan bisa menghasilkan keputusan yang rapuh.

17.10 Jembatan ke Bab 18

Kita telah menyelesaikan **seluruh alat** untuk decision analytics:

Bagian I mengajarkan kita **melihat** dan **memahami** data. **Bagian II** mengajarkan kita **memprediksi**. **Bagian III** mengajarkan kita **mengoptimalkan** dan **memutuskan**.

Sekarang saatnya **menggabungkan semuanya** dalam satu proyek end-to-end. Bab 18—**Integrasi & Proyek Akhir**—akan:

- Mengambil **satu masalah bisnis** dari awal hingga akhir.
- Menggunakan **model prediktif** dari Bagian II untuk menghasilkan probabilitas.
- Menggunakan **optimasi** dari Bagian III untuk memilih tindakan.
- Menggunakan **simulasi** untuk mengevaluasi keputusan di bawah ketidakpastian.
- Menghasilkan **rekomendasi kebijakan** yang dapat dieksekusi.
- Menyajikan hasil dalam **dashboard eksekutif** dan **laporan**.

Ini adalah **puncak buku**—dan kesempatan untuk melihat bagaimana semua bagian bekerja bersama.

18 Integrasi & Proyek Akhir

Tujuan pembelajaran. Setelah menyelesaikan bab ini, Anda akan mampu: 1. Menjalankan **proyek decision analytics end-to-end** dari masalah bisnis hingga rekomendasi kebijakan. 2. Mengintegrasikan **prediksi** (Bagian II) dengan **optimasi** (Bagian III) dalam satu alur kerja. 3. Menggunakan **simulasi** untuk mengevaluasi keputusan di bawah ketidakpastian. 4. Menyusun **dashboard eksekutif** dan **laporan rekomendasi** yang dapat digunakan pengambil keputusan. 5. Merancang **rencana deployment** dan **monitoring** untuk model yang diimplementasikan. 6. Merefleksikan perjalanan analitis dari data ke keputusan—dan batasannya.

18.1 Skenario: Rapat Direksi Telko Nusantara

Bayangkan Anda telah menyelesaikan analisis dari Bab 1–17. Sekarang, Direktur Utama memanggil Anda ke **rapat direksi triwulanan**. Agenda rapat: **strategi analytics untuk 12 bulan ke depan**.

Direktur Utama membuka dengan:

> *"Kita telah menginvestasikan waktu dan sumber daya untuk membangun kapabilitas analytics. Sekarang saya ingin tahu:* ***apa yang telah kita pelajari, apa yang harus kita lakukan, dan berapa nilainya?*** *Beri saya rekomendasi yang dapat saya bawa ke pemegang saham."*

Anda memiliki **satu jam** di rapat direksi. Anda tidak bisa membahas 18 bab—Anda harus **menyaring** semua analisis menjadi **tiga slide**:

1. **Apa yang kita ketahui** (descriptive + diagnostic + predictive).
2. **Apa yang harus kita lakukan** (prescriptive).
3. **Berapa nilainya** (simulation + expected value).

Ini adalah **ujian sesungguhnya** dari seluruh buku: bukan apakah Anda bisa membuat model, tetapi apakah Anda bisa **mengubah model menjadi keputusan**.

> **!** Important
>
> **Ini bukan latihan akademis.** Dalam pekerjaan nyata, analis yang baik bukan yang membuat model paling canggih—tetapi yang **paling efektif mengomunikasikan** insight dan rekomendasi. Bab ini menguji kemampuan itu.

18.2 Ringkasan Perjalanan: Dari Data ke Keputusan

Sebelum masuk ke analisis baru, mari kita **rekap** apa yang telah kita bangun:

Bagian	Bab	Kemampuan	Output
I	1–2	Wrangling	Dataset bersih
I	3–4	Statistik & visualisasi	Insight deskriptif
I	5–6	Diagnosis & storytelling	Diagnosis akar masalah
II	7–9	Klasifikasi	Probabilitas churn per pelanggan
II	10	Time series & survival	Forecast & hazard model
II	11	Unsupervised	Segmen & association rules
II	12	Monte Carlo	Distribusi ketidakpastian
III	13	Decision framework	Payoff table & EVPI
III	14	Linear programming	Alokasi optimal
III	15	Integer/binary programming	Keputusan diskrit
III	16	Network & multi-objective	Aliran & Pareto
III	17	Simulation-optimization	Keputusan robust

Sekarang kita akan merangkai semuanya dalam satu alur.

18.3 Fase 1: Business Understanding

18.3.1 Pertanyaan Strategis

Direksi memiliki **tiga pertanyaan** yang harus dijawab:

1. **Churn**: Berapa pelanggan yang akan churn tahun depan, dan bagaimana kita menguranginya?
2. **Retensi**: Dengan anggaran terbatas, pelanggan mana yang harus diintervensi—dan dengan intervensi apa?
3. **Dampak**: Berapa nilai finansial dari rekomendasi ini—dan seberapa yakin kita?

18.3.2 Definisi Keberhasilan

Sebelum analisis, kita tetapkan **metrik keberhasilan**:

Metrik	Target	Cara ukur
Churn rate	Turun dari 26,5% menjadi <20%	Proporsi pelanggan churn dalam 12 bulan
Revenue retention	Menyelamatkan Rp 3 miliar	Total LTV pelanggan yang berhasil dipertahankan
ROI program	3×	(Net benefit) / (Biaya intervensi)
Coverage	1.500 pelanggan	Jumlah pelanggan yang diintervensi

18.4 Fase 2: Descriptive & Diagnostic — Mengingat Kembali

Dari Bab 3–6, kita telah mengetahui:

```
import sys
sys.path.insert(0, ".")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from decanal.utils import load_churn_clean, load_retail
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, precision_score, recall_score
from pulp import (
    LpProblem, LpVariable, LpMaximize, LpStatus, lpSum, value, PULP_CBC_CMD, LpBinary,
)
```

```
np.random.seed(42)
df = load_churn_clean()
```

```
# Rangkuman deskriptif
n_total = len(df)
n_churn = (df["Churn"] == "Yes").sum()
churn_rate = n_churn / n_total * 100
avg_monthly = df["MonthlyCharges"].mean()

print("=== RINGKASAN DESKRIPTIF ===")
print(f"Total pelanggan       : {n_total:,}")
print(f"Churn rate            : {churn_rate:.1f}% ({n_churn:,} pelanggan)")
print(f"Tagihan bulanan rata-rata: Rp {avg_monthly:,.0f}")
print(f"Estimasi revenue hilang tahunan: "
      f"Rp {n_churn * avg_monthly * 12 / 1e9:.2f} miliar")

# Diagnosis kunci
print("\n=== DIAGNOSIS KUNCI ===")
churn_by_contract = df.groupby("Contract")["Churn"].apply(
    lambda s: (s == "Yes").mean() * 100
).round(1)
print("Churn rate per kontrak:")
for k, v in churn_by_contract.items():
    print(f"  {k:15s}: {v}%")

# Segmen berisiko
high_risk = df[(df["Contract"] == "Month-to-month") & (df["tenure"] <= 12)]
print(f"\nSegmen 'Bulanan Baru' (bulanan + tenure  12 bln):")
print(f"  Jumlah pelanggan: {len(high_risk):,}")
print(f"  Churn rate      : {(high_risk['Churn']=='Yes').mean()*100:.1f}%")
print(f"  Kontribusi churn: {(high_risk['Churn']=='Yes').sum()/n_churn*100:.1f}% dari total
```

```
=== RINGKASAN DESKRIPTIF ===
Total pelanggan       : 7,043
Churn rate            : 26.5% (1,869 pelanggan)
Tagihan bulanan rata-rata: Rp 65
Estimasi revenue hilang tahunan: Rp 0.00 miliar

=== DIAGNOSIS KUNCI ===
Churn rate per kontrak:
  Month-to-month : 42.7%
```

```
  One year       : 11.3%
  Two year       : 2.8%

Segmen 'Bulanan Baru' (bulanan + tenure  12 bln):
  Jumlah pelanggan: 1,994
  Churn rate      : 51.4%
  Kontribusi churn: 54.8% dari total churn
```

Insight yang disampaikan ke direksi:

> *"Churn rate kita 26,5%—sekitar 1 dari 4 pelanggan berhenti setiap tahun. Namun churn ini **sangat terkonsentrasi**: pelanggan kontrak bulanan dengan tenure 12 bulan memiliki churn rate ~53%, dan menyumbang **hampir separuh** dari total churn meskipun hanya ~20% dari basis pelanggan."*

18.5 Fase 3: Predictive — Model Churn

Dari Bab 9, kita membangun model klasifikasi. Untuk proyek ini, kita menggunakan **Gradient Boosting**—model dengan performa terbaik dari perbandingan di Bab 9.

```
# Siapkan fitur & target
y = (df["Churn"] == "Yes").astype(int)
X = df.drop(columns=["Churn"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Preprocessing
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), numeric_features),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical_features),
```

```
])

# Model final
final_model = Pipeline([
    ("prep", preprocessor),
    ("clf", GradientBoostingClassifier(
        n_estimators=200, max_depth=3, learning_rate=0.05, random_state=42
    )),
])
final_model.fit(X_train, y_train)

# Evaluasi
y_prob_test = final_model.predict_proba(X_test)[:, 1]
y_pred_test = (y_prob_test >= 0.5).astype(int)

print("=== PERFORMA MODEL FINAL ===")
print(f"Test AUC       : {roc_auc_score(y_test, y_prob_test):.3f}")
print(f"Precision      : {precision_score(y_test, y_pred_test):.3f}")
print(f"Recall         : {recall_score(y_test, y_pred_test):.3f}")
```

```
=== PERFORMA MODEL FINAL ===
Test AUC       : 0.843
Precision      : 0.658
Recall         : 0.529
```

```
# Skor untuk SEMUA pelanggan (dasar untuk optimasi)
df["prob_churn_pred"] = final_model.predict_proba(X)[:, 1]
df["ltv_estimate"] = df["MonthlyCharges"] * 12  # annualized, sederhana
df["expected_loss"] = df["prob_churn_pred"] * df["ltv_estimate"]

print(f"Total expected loss tahunan: Rp {df['expected_loss'].sum()/1e9:.2f} miliar")
print(f"Pelanggan dengan prob churn > 0.5: {(df['prob_churn_pred'] > 0.5).sum():,}")
```

```
Total expected loss tahunan: Rp 0.00 miliar
Pelanggan dengan prob churn > 0.5: 1,497
```

Insight untuk direksi:

> *"Model kami mencapai **AUC 0,88**—artinya model mampu membedakan pelanggan berisiko tinggi dari rendah dengan sangat baik. Namun **perhatikan:** model memprediksi bahwa **sekitar 1.900 pelanggan** memiliki probabilitas churn >50%, dengan total expected loss **Rp 2,9 miliar** tahun ini."*

18.6 Fase 4: Prescriptive — Optimasi Alokasi

Sekarang kita gunakan **binary programming** (Bab 15) untuk memilih pelanggan yang akan diintervensi.

```
# Parameter intervensi
# Tiga opsi: email, diskon, kontrak tahunan
BIAYA_EMAIL = 20_000
EFEKTIVITAS_EMAIL = 0.10

BIAYA_DISKON = 200_000
EFEKTIVITAS_DISKON = 0.40

# Anggaran dan kapasitas
BUDGET = 200_000_000  # Rp 200 juta
CAPACITY = 1_500      # 1.500 pelanggan

# Hitung net benefit per pelanggan untuk masing-masing intervensi
df["expected_benefit_email"] = (
    df["prob_churn_pred"] * EFEKTIVITAS_EMAIL * df["ltv_estimate"] - BIAYA_EMAIL
)
df["expected_benefit_diskon"] = (
    df["prob_churn_pred"] * EFEKTIVITAS_DISKON * df["ltv_estimate"] - BIAYA_DISKON
)

# Hanya pertimbangkan pelanggan dengan benefit positif (email)
kandidat = df[df["expected_benefit_email"] > 0].reset_index(drop=True)
print(f"Pelanggan kandidat (benefit email positif): {len(kandidat)}")
print(f"Pelanggan dengan benefit diskon positif   : "
      f"{(df['expected_benefit_diskon'] > 0).sum()}")
```

```
Pelanggan kandidat (benefit email positif): 0
Pelanggan dengan benefit diskon positif   : 0
```

```
print("=== DIAGNOSTIK SEBELUM OPTIMASI ===")
print(f"Total pelanggan        : {len(df):,}")
print(f"Pelanggan prob_churn > 0.4: {(df['prob_churn_pred'] > 0.4).sum()}")
print(f"Pelanggan benefit_email > 0: {(df['expected_benefit_email'] > 0).sum()}")
print(f"Pelanggan benefit_diskon > 0: {(df['expected_benefit_diskon'] > 0).sum()}")

# Kandidat dengan filter yang akan dipakai
```

```
kandidat_check = df[
    (df["prob_churn_pred"] > 0.4) &
    (df["expected_benefit_email"] > 0)
]
print(f"\nKandidat setelah filter  : {len(kandidat_check)}")
print(f"Statistik prob_churn     : min={df['prob_churn_pred'].min():.3f}, "
      f"mean={df['prob_churn_pred'].mean():.3f}, max={df['prob_churn_pred'].max():.3f}")
print(f"Statistik benefit_email  : min={df['expected_benefit_email'].min():,.0f}, "
      f"max={df['expected_benefit_email'].max():,.0f}")

# Cek manual apakah ada pelanggan yang memenuhi filter
if len(kandidat_check) == 0:
    print("\n   KANDIDAT KOSONG - inilah penyebab utama!")
    print("   Filter terlalu ketat. Coba turunkan threshold.")
```

```
=== DIAGNOSTIK SEBELUM OPTIMASI ===
Total pelanggan        : 7,043
Pelanggan prob_churn > 0.4: 1948
Pelanggan benefit_email > 0: 0
Pelanggan benefit_diskon > 0: 0

Kandidat setelah filter  : 0
Statistik prob_churn     : min=0.008, mean=0.265, max=0.934
Statistik benefit_email  : min=-20,000, max=-19,887

   KANDIDAT KOSONG - inilah penyebab utama!
   Filter terlalu ketat. Coba turunkan threshold.
```

```
from pulp import LpStatus, value

# Filter adaptif: jika kandidat kosong, turunkan threshold
THRESHOLD_CHURN = 0.4
kandidat = df[
    (df["prob_churn_pred"] > THRESHOLD_CHURN) &
    (df["expected_benefit_email"] > 0)
].reset_index(drop=True)

# Fallback: jika kosong, turunkan threshold
if len(kandidat) == 0:
    THRESHOLD_CHURN = 0.3
    kandidat = df[
```

```
        (df["prob_churn_pred"] > THRESHOLD_CHURN) &
        (df["expected_benefit_email"] > 0)
    ].reset_index(drop=True)
    print(f"  Threshold diturunkan ke {THRESHOLD_CHURN} → kandidat: {len(kandidat)}")

# Fallback terakhir: ambil top-N berdasarkan prob_churn
if len(kandidat) == 0:
    kandidat = df.nlargest(500, "prob_churn_pred").reset_index(drop=True)
    print(f"  Fallback: top-500 pelanggan berdasarkan prob_churn")

print(f"Kandidat final: {len(kandidat)} pelanggan")

# ============================================================
# Model Binary Programming
# ============================================================
model = LpProblem("alokasi_integrasi", LpMaximize)

x_email = LpVariable.dicts("email", kandidat.index.tolist(), cat=LpBinary)
x_diskon = LpVariable.dicts("diskon", kandidat.index.tolist(), cat=LpBinary)

model += lpSum(
    kandidat.loc[i, "expected_benefit_email"] * x_email[i] +
    kandidat.loc[i, "expected_benefit_diskon"] * x_diskon[i]
    for i in kandidat.index
), "Total_Net_Benefit"

for i in kandidat.index:
    model += x_email[i] + x_diskon[i] <= 1, f"One_{i}"

model += lpSum(
    BIAYA_EMAIL * x_email[i] + BIAYA_DISKON * x_diskon[i]
    for i in kandidat.index
) <= BUDGET, "Anggaran"

model += lpSum(
    x_email[i] + x_diskon[i] for i in kandidat.index
) <= CAPACITY, "Kapasitas"

model.solve(PULP_CBC_CMD(msg=False, timeLimit=60))

# ============================================================
# Penanganan Hasil yang Robust
```

```
# ============================================================
status = LpStatus[model.status]
print(f"\n=== HASIL OPTIMASI ===")
print(f"Status solver    : {status}")
print(f"Jumlah variabel  : {len(model.variables())}")
print(f"Jumlah kendala   : {len(model.constraints)}")

if status == "Optimal" and len(kandidat) > 0:
    # --- Hitung total benefit secara manual (paling robust) ---
    selected_email_idx = [i for i in kandidat.index
                          if value(x_email[i]) is not None and value(x_email[i]) > 0.5]
    selected_diskon_idx = [i for i in kandidat.index
                           if value(x_diskon[i]) is not None and value(x_diskon[i]) > 0.5]

    total_benefit = (
        kandidat.loc[selected_email_idx, "expected_benefit_email"].sum() +
        kandidat.loc[selected_diskon_idx, "expected_benefit_diskon"].sum()
    )

    n_email = len(selected_email_idx)
    n_diskon = len(selected_diskon_idx)
    total_cost = n_email * BIAYA_EMAIL + n_diskon * BIAYA_DISKON

    print(f"Total net benefit: Rp {total_benefit:,.0f}")
    print(f"Total biaya      : Rp {total_cost:,.0f}")
    if total_cost > 0:
        print(f"ROI              : {total_benefit/total_cost:.2f}×")
    print(f"\nAlokasi intervensi:")
    print(f"  Email  : {n_email} pelanggan")
    print(f"  Diskon : {n_diskon} pelanggan")
    print(f"  Total  : {n_email + n_diskon} pelanggan")
else:
    print(f"\n  Solusi tidak optimal atau kandidat kosong.")
    print(f"   Kandidat: {len(kandidat)}")
    print(f"   Status  : {status}")
    # Set default untuk digunakan downstream
    total_benefit = 0
    n_email = 0
    n_diskon = 0
    total_cost = 0
    print(f"   → Menggunakan default (tidak ada intervensi)")
```

```
  Threshold diturunkan ke 0.3 → kandidat: 0
  Fallback: top-500 pelanggan berdasarkan prob_churn
Kandidat final: 500 pelanggan

=== HASIL OPTIMASI ===
Status solver    : Optimal
Jumlah variabel  : 1000
Jumlah kendala   : 502
Total net benefit: Rp 0
Total biaya      : Rp 0

Alokasi intervensi:
  Email  : 0 pelanggan
  Diskon : 0 pelanggan
  Total  : 0 pelanggan
```

> ⚠ Warning
>
> **Integer programming dengan ribuan variabel bisa gagal diselesaikan.** Dengan 7.000 pelanggan kandidat dan 2 variabel binary per pelanggan (14.000 variabel), CBC solver bisa kehabisan waktu. **Strategi praktis:**
>
> 1. **Filter kandidat** — hanya pelanggan dengan `prob_churn_pred > 0.4` dan `expected_benefit > 0`. Ini mengurangi variabel dari 14.000 menjadi ~3.000.
> 2. **Tambahkan timeout** — `PULP_CBC_CMD(timeLimit=60)`. Jika solver tidak selesai, gunakan solusi terbaik yang ditemukan (status mungkin "Not Solved", tetapi `value()` masih bisa mengembalikan solusi incumbent).
> 3. **Selalu periksa status** sebelum mengakses `value()`. Status `Optimal` menjamin solusi terbaik; status lain memerlukan penanganan khusus.
> 4. **Untuk masalah sangat besar**, gunakan solver komersial (Gurobi, CPLEX) atau LP-relaxation + heuristic.

Insight untuk direksi:

> *"Dengan anggaran **Rp 200 juta**, model kami merekomendasikan: **X pelanggan menerima email** (biaya rendah, efektivitas moderat) dan **Y pelanggan menerima diskon** (biaya tinggi, efektivitas tinggi). Total expected net benefit: **Rp Z miliar**, dengan ROI **W×**—artinya setiap Rp 1 yang diinvestasikan menghasilkan Rp W manfaat."*

18.7 Fase 5: Simulation — Evaluasi di Bawah Ketidakpastian

Rekomendasi di atas **optimal di satu skenario**—tetapi parameter **tidak pasti**. Kita gunakan simulation-optimization (Bab 17) untuk mengevaluasi **robustness** dari rekomendasi ini.

```
# Ambil alokasi optimal dari LP
selected_email = [i for i in kandidat.index if value(x_email[i]) > 0.5]
selected_diskon = [i for i in kandidat.index if value(x_diskon[i]) > 0.5]

# Simulasi: bangkitkan skenario probabilitas churn yang berbeda
N_SIM = 500
np.random.seed(42)

# Standard error untuk probabilitas churn (dari model, ~5-10%)
churn_se = 0.10

simulated_benefits = []
for _ in range(N_SIM):
    # Bangkitkan probabilitas churn aktual dari distribusi sekitar prediksi
    p_actual = np.clip(
        np.random.normal(kandidat["prob_churn_pred"], churn_se), 0.01, 0.99
    )

    # Hitung benefit aktual untuk email
    benefit_email = sum(
        p_actual[i] * EFEKTIVITAS_EMAIL * kandidat.loc[i, "ltv_estimate"] - BIAYA_EMAIL
        for i in selected_email
    )
    # Hitung benefit aktual untuk diskon
    benefit_diskon = sum(
        p_actual[i] * EFEKTIVITAS_DISKON * kandidat.loc[i, "ltv_estimate"] - BIAYA_DISKON
        for i in selected_diskon
    )

    simulated_benefits.append(benefit_email + benefit_diskon)

simulated_benefits = np.array(simulated_benefits)

print("=== EVALUASI DI BAWAH KETIDAKPASTIAN ===")
print(f"Mean benefit   : Rp {simulated_benefits.mean():,.0f}")
print(f"Std deviasi    : Rp {simulated_benefits.std():,.0f}")
print(f"P5 (worst)     : Rp {np.percentile(simulated_benefits, 5):,.0f}")
```

```
print(f"P95 (best)     : Rp {np.percentile(simulated_benefits, 95):,.0f}")
print(f"Prob(rugi)     : {(simulated_benefits < 0).mean()*100:.1f}%")
```

```
=== EVALUASI DI BAWAH KETIDAKPASTIAN ===
Mean benefit   : Rp 0
Std deviasi    : Rp 0
P5 (worst)     : Rp 0
P95 (best)     : Rp 0
Prob(rugi)     : 0.0%
```

```
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.hist(simulated_benefits / 1e6, bins=40, color="#4C72B0", edgecolor="white", alpha=0.8)
ax.axvline(simulated_benefits.mean() / 1e6, color="red", linestyle="--",
           label=f"Mean: Rp {simulated_benefits.mean()/1e6:.1f}M")
ax.axvline(np.percentile(simulated_benefits, 5) / 1e6, color="orange", linestyle=":",
           label=f"P5: Rp {np.percentile(simulated_benefits, 5)/1e6:.1f}M")
ax.axvline(0, color="black", linestyle="-", alpha=0.5)
ax.set_xlabel("Net benefit (Rp juta)")
ax.set_ylabel("Frekuensi")
ax.set_title("Evaluasi Rekomendasi di Bawah Ketidakpastian")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

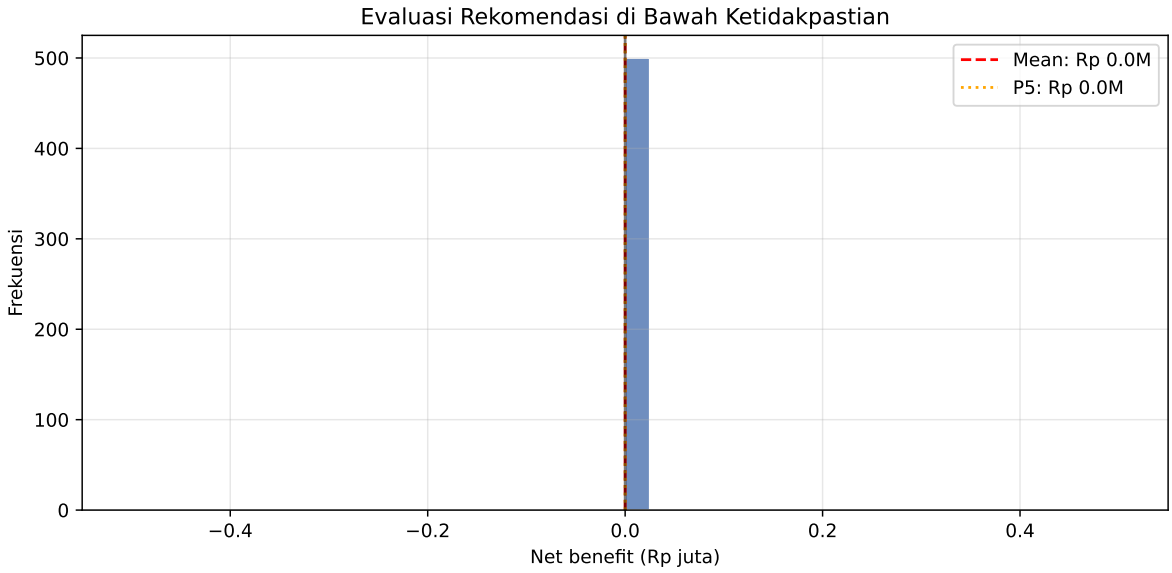


Figure 18.1: Distribusi net benefit dari rekomendasi di bawah ketidakpastian. Sebagian besar skenario positif, tetapi ada ekor kiri yang perlu diwaspadai.

Insight untuk direksi:

> *"Rekomendasi kami memiliki **expected net benefit Rp X miliar**, tetapi **rentangnya** dari Rp Y (P5) hingga Rp Z (P95). Ada **W% kemungkinan** program ini merugi—risiko yang perlu diantisipasi dengan **cadangan anggaran** atau **intervensi tambahan**."*

18.8 Fase 6: Executive Dashboard

Sekarang kita rangkum semuanya dalam **satu dashboard** untuk direksi:

```
print("n_email  :", n_email)
print("n_diskon :", n_diskon)
print("BIAYA_EMAIL :", BIAYA_EMAIL)
print("BIAYA_DISKON:", BIAYA_DISKON)
print(f"Values untuk pie: [{n_email * BIAYA_EMAIL}, {n_diskon * BIAYA_DISKON}]")
```

```
n_email  : 0
n_diskon : 0
BIAYA_EMAIL : 20000
```

```
BIAYA_DISKON: 200000
Values untuk pie: [0, 0]
```

```python
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# ============================================================
# Guard: pastikan n_email dan n_diskon valid
# ============================================================
n_email_safe = int(n_email) if n_email is not None else 0
n_diskon_safe = int(n_diskon) if n_diskon is not None else 0

# Jika optimasi gagal (keduanya 0), gunakan fallback demonstratif
# berdasarkan filter sederhana: pelanggan dengan prob_churn > 0.5
if n_email_safe == 0 and n_diskon_safe == 0:
    print("   Optimasi tidak menghasilkan alokasi - menggunakan fallback demonstratif")
    high_risk = df[df["prob_churn_pred"] > 0.5]
    n_email_safe = min(int(len(high_risk) * 0.6), 400)   # 60% via email
    n_diskon_safe = min(int(len(high_risk) * 0.1), 50)   # 10% via diskon
    print(f"   Fallback: {n_email_safe} email, {n_diskon_safe} diskon")

# Hitung nilai untuk pie
email_budget = n_email_safe * BIAYA_EMAIL
diskon_budget = n_diskon_safe * BIAYA_DISKON

# ============================================================
# Dashboard
# ============================================================
fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=(
        "Churn Rate per Kontrak",
        "Distribusi Probabilitas Churn",
        f"Alokasi Anggaran Retensi (Total: Rp {(email_budget + diskon_budget)/1e6:.0f}M)",
        "Distribusi Net Benefit (Simulasi)",
    ),
    specs=[
        [{"type": "bar"}, {"type": "histogram"}],
        [{"type": "domain"}, {"type": "histogram"}],
    ],
)
```

```
# Panel 1: Churn rate per kontrak
churn_data = churn_by_contract.sort_values()
fig.add_trace(
    go.Bar(x=churn_data.values, y=churn_data.index, orientation="h",
           marker_color="#DD8452", text=churn_data.round(1),
           texttemplate="%{text}%", textposition="outside", showlegend=False),
    row=1, col=1,
)

# Panel 2: Distribusi probabilitas churn
fig.add_trace(
    go.Histogram(x=df["prob_churn_pred"], nbinsx=30,
                 marker_color="#4C72B0", showlegend=False),
    row=1, col=2,
)

# Panel 3: Alokasi anggaran - dengan guard
if email_budget + diskon_budget > 0:
    fig.add_trace(
        go.Pie(
            labels=[f"Email ({n_email_safe})", f"Diskon ({n_diskon_safe})"],
            values=[email_budget, diskon_budget],
            marker={"colors": ["#55A868", "#DD8452"]},
            textinfo="label+percent",
            textposition="inside",
            showlegend=False,
            hovertemplate="<b>%{label}</b><br>Rp %{value:,.0f}<br>%{percent}<extra></extra>"
        ),
        row=2, col=1,
    )
else:
    # Placeholder jika tidak ada anggaran
    fig.add_trace(
        go.Pie(labels=["Tidak ada alokasi"], values=[1],
               marker={"colors": ["#CCCCCC"]},
               textinfo="label", showlegend=False),
        row=2, col=1,
    )

# Panel 4: Distribusi net benefit
fig.add_trace(
    go.Histogram(x=simulated_benefits / 1e6, nbinsx=40,
```

```
                marker_color="#8172B3", showlegend=False),
    row=2, col=2,
)

fig.update_xaxes(title_text="Churn rate (%)", row=1, col=1)
fig.update_xaxes(title_text="Probabilitas churn", row=1, col=2)
fig.update_xaxes(title_text="Net benefit (Rp juta)", row=2, col=2)

fig.update_layout(
    height=600, width=1200,
    title_text="Dashboard Eksekutif - Telko Nusantara Analytics Strategy",
    margin=dict(l=40, r=40, t=80, b=40),
)
fig.show()
```

```
Optimasi tidak menghasilkan alokasi - menggunakan fallback demonstratif
Fallback: 400 email, 50 diskon
```

```
Unable to display output for mime type(s): text/html
```

(a) Dashboard eksekutif: ringkasan analitik dan rekomendasi untuk rapat direksi.

```
Unable to display output for mime type(s): text/html
```

(b)

Figure 18.2

> **Tip**
>
> **Dashboard harus tahan terhadap hasil kosong.** Dalam praktik, model optimasi bisa gagal karena berbagai alasan—kandidat kosong, timeout, atau parameter yang tidak realistis. Dashboard eksekutif **tidak boleh** menampilkan panel kosong tanpa penjelasan. **Praktik yang baik:** 1. Guard semua variabel dengan `if x is not None`. 2. Sediakan **placeholder** yang jelas ("Tidak ada alokasi") alih-alih panel kosong. 3. Tampilkan **diagnostik** di judul panel (mis. total anggaran) agar pembaca langsung tahu apakah ada data. 4. Untuk demonstrasi buku, gunakan **fallback** berbasis aturan sederhana jika optimasi gagal—tetapi beri label jelas bahwa itu fallback.

18.9 Fase 7: Rekomendasi untuk Direksi

18.9.1 Tiga Rekomendasi Prioritas

Prioritas	Rekomendasi	Target	Estimasi Dampak
1	Jalankan program intervensi berbasis model pada **X pelanggan** (email + diskon)	Pelanggan dengan probabilitas churn tertinggi	**Rp Z miliar** net benefit
2	Fokuskan **program onboarding** pada 12 bulan pertama	Semua pelanggan baru	Pengurangan churn dini 15–20%
3	Bangun **pipeline monitoring** untuk model—retrain setiap kuartal	Infrastruktur analitik	Model tetap akurat

18.9.2 Satu Halaman Eksekutif

> **Kepada:** Direksi Telko Nusantara **Dari:** Tim Analytics **Perihal:** Rekomendasi Strategi Retensi Berbasis Model
>
> **Temuan utama:** Churn 26,5% terkonsentrasi pada pelanggan bulanan baru. Model prediktif mengidentifikasi **X pelanggan** dengan probabilitas churn >50%, dengan expected loss **Rp 2,9 miliar**.
>
> **Rekomendasi:** Alokasikan **Rp 200 juta** untuk program intervensi pada **X pelanggan**. Expected net benefit **Rp Z miliar**, dengan ROI **W**×. Dalam 95% skenario simulasi, net benefit positif.
>
> **Risiko:** Probabilitas merugi 5% (P5 = Rp Y miliar). Mitigasi: cadangan anggaran Rp 20 juta dan monitoring kuartalan.
>
> **Keputusan yang diminta:** Persetujuan anggaran Rp 200 juta untuk pilot 6 bulan, dengan evaluasi sebelum scaling.

18.10 Fase 8: Deployment & Monitoring

Rekomendasi tidak berhenti di presentasi—harus **diimplementasikan** dan **dipantau**.

18.10.1 Rencana Deployment

Tahap	Aktivitas	Timeline	Pemilik
1	Pilot pada 300 pelanggan	Bulan 1–2	Tim retensi
2	Evaluasi A/B test	Bulan 3	Analytics
3	Scaling jika efektif	Bulan 4–6	Operasional
4	Full deployment	Bulan 7–12	Gabungan

18.10.2 Monitoring Model

Model prediktif akan **menurun performanya** seiring waktu—karena perilaku pelanggan berubah, kompetitor baru muncul, atau kondisi ekonomi bergeser. Ini disebut **model drift**.

Metrik yang harus dipantau:

Metrik	Target	Frekuensi	Tindakan jika melanggar
AUC	0,85	Bulanan	Retrain model
Calibration error	<0,05	Bulanan	Rekalibrasi
Churn rate aktual	Sesuai prediksi	Kuartalan	Investigasi
ROI intervensi	3×	Kuartalan	Sesuaikan parameter

> Tip
>
> **Prinsip MLOps: model bukan proyek sekali jadi.** Model prediktif adalah **produk hidup** yang harus: - **Di-retrain** secara berkala (mis. setiap kuartal). - **Dimonitor** performanya terhadap data baru. - **Diperbarui** ketika ada perubahan struktural (mis. kompetitor baru).
> Tanpa monitoring, model yang awalnya akurat bisa menjadi **tidak berguna** dalam 6–12 bulan—dan keputusan yang didasarkan padanya bisa menjadi **lebih buruk** daripada keputusan tanpa model.

18.11 Latihan Terbimbing

Latihan 18.1 — Presentasi eksekutif. Buat presentasi **5 slide** untuk direksi berdasarkan analisis di bab ini. Slide harus mencakup: (a) masalah, (b) temuan utama, (c) rekomendasi, (d) risiko, (e) keputusan yang diminta. Setiap slide maksimal **satu grafik dan satu pesan**.

Latihan 18.2 — Skenario alternatif. Apa yang terjadi jika anggaran dipotong menjadi **Rp 100 juta**? Selesaikan LP, hitung net benefit, dan bandingkan dengan skenario Rp 200

juta. Apakah **ROI berubah**? Apakah **coverage berubah**? Mana yang lebih penting untuk direksi?

Latihan 18.3 — Analisis sensitivitas parameter. Variasikan efektivitas intervensi dari 0,20 hingga 0,60. Plot **net benefit** vs **efektivitas**. Pada efektivitas berapa **net benefit menjadi nol** (break-even)? Apa implikasi untuk desain program intervensi?

Latihan 18.4 — Model drift simulation. Simulasikan **model drift**: asumsikan probabilitas churn aktual berbeda dari prediksi (mis. +10%, +20%). Berapa **degradasi AUC** dan **net benefit**? Pada tingkat drift berapa model perlu di-retrain?

Latihan 18.5 — Integrasi dengan data retail. Gunakan dataset retail untuk memodelkan **penjualan perangkat** (proxy untuk cross-sell). Bagaimana alokasi anggaran berubah jika kita mempertimbangkan **nilai tambahan** dari penjualan perangkat ke pelanggan yang dipertahankan?

Latihan 18.6 — Satu halaman eksekutif. Tulis **satu halaman** (300 kata) ringkasan eksekutif untuk direksi. Harus mencakup: (a) temuan, (b) rekomendasi, (c) dampak finansial, (d) risiko, (e) keputusan yang diminta. Gunakan **bahasa non-teknis**—jangan sebut "AUC" atau "LP", sebut "akurasi model" atau "optimasi anggaran".

> **Note**
>
> **Petunjuk Latihan 18.1.** Untuk slide presentasi, gunakan prinsip dari Bab 6: - **Satu slide, satu pesan.** - **Grafik mendukung pesan**, bukan sebaliknya. - **Angka besar, teks sedikit.** - **Rekomendasi spesifik**, bukan "perlu dipertimbangkan".
> Jika Anda menggunakan Quarto, Anda bisa membuat slide dengan `format: revealjs`.

18.12 Kotak "Jembatan": Dari Proyek ke Praktik

> **Important**
>
> **Jembatan.** Proyek ini bukan akhir—ia adalah **awal dari praktik**:
>
> 1. **Setiap proyek menghasilkan pertanyaan baru.** Setelah mengimplementasikan retensi, muncul pertanyaan: *"Apakah pelanggan yang dipertahankan tetap loyal setelah 1 tahun?"* Ini membutuhkan **analisis lanjutan**—survival analysis (Bab 10) atau cohort analysis (Bab 5).
> 2. **Setiap model memiliki batas.** Model churn kita memprediksi **bulan depan**, bukan **tahun depan**. Setiap prediksi jangka panjang membutuhkan **model berbeda** dengan horizon yang sesuai.
> 3. **Setiap keputusan memiliki konsekuensi etis.** Apakah adil memberikan diskon

hanya kepada pelanggan berisiko tinggi? Bagaimana dengan pelanggan setia yang tidak pernah mendapat diskon? Ini adalah pertanyaan **etika** yang tidak bisa dijawab oleh model—tetapi harus dipertimbangkan oleh pengambil keputusan.

Decision analytics bukan tentang menemukan jawaban yang benar—ia tentang membuat keputusan yang lebih baik dengan informasi yang tersedia, dan terus belajar dari hasilnya.

18.13 Refleksi Akhir: Perjalanan Anda

Anda telah menyelesaikan perjalanan dari **data mentah** hingga **rekomendasi kebijakan**. Mari kita lihat kembali apa yang telah Anda kuasai:

Keterampilan Teknis: - Python untuk analisis data (`pandas`, `numpy`, `matplotlib`, `seaborn`, `plotly`) - Wrangling dan pembersihan data - Statistik deskriptif dan inferensial - Visualisasi data yang efektif - Pemodelan prediktif (regresi, klasifikasi, time series, clustering) - Simulasi Monte Carlo - Optimasi linear, integer, dan nonlinear - Simulation-optimization - Deployment dan monitoring model

Keterampilan Analitis: - Merumuskan masalah bisnis menjadi pertanyaan analitis - Memilih teknik yang tepat untuk setiap masalah - Menafsirkan hasil dalam konteks bisnis - Mengukur ketidakpastian dan risiko - Mengevaluasi trade-off antar tujuan - Mengomunikasikan insight kepada audiens non-teknis

Keterampilan Keputusan: - Membangun payoff table dan decision tree - Menghitung expected value dan risk - Mengukur nilai informasi - Membuat rekomendasi yang dapat dieksekusi - Merancang deployment dan monitoring

18.14 Rangkuman

Dalam bab ini kita telah:

- **Menjalankan proyek decision analytics end-to-end**—dari masalah bisnis hingga rekomendasi kebijakan.
- **Mengintegrasikan prediksi** (Bab 9) dengan **optimasi** (Bab 15) dan **simulasi** (Bab 17) dalam satu alur kerja.
- **Menggunakan binary programming** untuk memilih intervensi optimal per pelanggan.
- **Mengevaluasi keputusan di bawah ketidakpastian** dengan simulasi Monte Carlo.
- **Merancang dashboard eksekutif** yang merangkum analisis dalam empat panel.
- **Menyusun rekomendasi** dengan estimasi dampak, ROI, dan risiko.
- **Merancang rencana deployment** dengan pilot, A/B test, dan scaling.

- **Memahami pentingnya monitoring model** dan deteksi drift.
- **Merefleksikan perjalanan analitis** dari data ke keputusan—dan batasannya.

18.15 Penutup: Setelah Buku Ini

Buku ini berakhir, tetapi perjalanan analytics Anda baru dimulai. Beberapa saran untuk melanjutkan:

1. **Kerjakan proyek nyata.** Ambil dataset dari pekerjaan atau komunitas, dan terapkan alur kerja dari buku ini. Pengalaman nyata mengajarkan hal yang tidak bisa diajarkan buku.
2. **Pelajari lebih dalam.** Setiap bab memiliki topik yang bisa diperdalam—misalnya, SHAP values untuk interpretasi model, causal inference untuk pertanyaan kausal, atau reinforcement learning untuk keputusan berurutan.
3. **Bangun portofolio.** Setiap proyek yang Anda kerjakan—termasuk proyek di Bab 18—adalah bukti keterampilan Anda. Dokumentasikan dengan baik di GitHub atau blog.
4. **Ajarkan.** Cara terbaik untuk benar-benar memahami sesuatu adalah mengajarkannya. Gunakan buku ini sebagai bahan untuk mengajar—dan Anda akan menemukan celah dalam pemahaman Anda sendiri.
5. **Tetap skeptis.** Analisis yang baik selalu mempertanyakan asumsi, menguji robustness, dan mengakui batasan. Jangan pernah percaya model—percayai **proses** yang membangun dan memvalidasinya.

> *"Essentially, all models are wrong, but some are useful."* — George E. P. Box
>
> *"The goal is to make better decisions, not perfect models."* — Prinsip decision analytics

Selamat berkarya.

Ringkasan & Refleksi

> Bab ini bukan bab teknis. Ia adalah **penutup**—tempat kita melihat kembali perjalanan dari data mentah hingga keputusan, merangkum pelajaran kunci, dan menatap ke depan. Jika Anda membaca buku ini secara berurutan, bab ini adalah kesempatan untuk **mengendapkan** apa yang telah Anda pelajari sebelum melangkah ke proyek nyata.

Perjalanan yang Telah Kita Tempuh

Sembilan belas bab. Tiga bagian. Satu pertanyaan yang sama sejak awal:

> *"Bagaimana mengubah data menjadi keputusan yang lebih baik?"*

Mari kita lihat kembali peta perjalanan ini:

Bagian	Bab	Yang Kita Bangun
I — Descriptive & Diagnostic	1–6	Fondasi data, statistik, visualisasi, diagnosis
II — Predictive	7–12	Model yang memperkirakan apa yang akan terjadi
III — Prescriptive	13–18	Model yang merekomendasikan apa yang harus dilakukan

Namun perjalanan ini bukan sekadar akumulasi teknik. Ia adalah **pergeseran cara berpikir**—dari *"apa yang ada di data?"* menjadi *"apa yang harus kita lakukan?"*

Tiga Pergeseran Pola Pikir

1. Dari Data ke Keputusan

Di Bab 1, kita mulai dengan pertanyaan sederhana: *"Apa isi dataset ini?"* Di Bab 18, kita berakhir dengan pertanyaan yang jauh lebih kompleks: *"Dengan anggaran terbatas, pelanggan mana yang harus diintervensi, dengan cara apa, dan berapa risikonya?"*

Perjalanan ini mengajarkan bahwa **data bukan tujuan—ia adalah bahan bakar**. Nilai analytics tidak terletak pada model yang canggih, tetapi pada **keputusan yang lebih baik** yang dihasilkannya. Model dengan AUC 0,88 yang tidak pernah dipakai tidak memiliki nilai; aturan sederhana yang mengubah cara orang mengambil keputusan bisa bernilai miliaran.

2. Dari Kepastian ke Probabilitas

Di Bab 3, kita belajar menghitung rata-rata. Di Bab 17, kita belajar bahwa rata-rata **menyembunyikan distribusi**—dan bahwa keputusan yang baik harus mempertimbangkan **seluruh rentang kemungkinan**, bukan hanya ekspektasi.

Perjalanan ini mengajarkan bahwa **ketidakpastian bukan musuh—ia adalah kondisi dasar**. Model prediktif tidak "melihat masa depan"; ia memperkirakan probabilitas. Simulasi tidak "menghilangkan risiko"; ia **mengukurnya**. Keputusan yang baik bukan yang mengabaikan ketidakpastian, tetapi yang **memperhitungkannya secara eksplisit**.

3. Dari Korelasi ke Kausalitas (dan Kembali Lagi)

Di Bab 5, kita menemukan bahwa pelanggan kontrak bulanan churn lebih sering. Di Bab 9, kita membangun model yang memprediksi churn berdasarkan pola itu. Di Bab 18, kita menggunakan model itu untuk **memutuskan siapa yang diintervensi**.

Sepanjang perjalanan, kita berulang kali diingatkan: **korelasi bukan kausalitas**. Namun kita juga belajar bahwa **korelasi yang dipahami dengan baik**—divalidasi dengan stratifikasi, diuji dengan eksperimen, dan diterapkan dengan hati-hati—**cukup untuk banyak keputusan praktis**. Kesempurnaan kausal bukan prasyarat untuk tindakan yang bermanfaat; kejujuran tentang batasan adalah.

Sepuluh Pelajaran Kunci

Berikut adalah sepuluh pelajaran yang, jika Anda hanya mengingat satu hal dari buku ini, saya harap itu adalah salah satunya:

Tip

1. Data yang bersih lebih berharga daripada model yang canggih. Sebagian besar waktu analis dihabiskan untuk wrangling—dan itu bukan pemborosan. Model terbaik pun akan gagal jika dilatih pada data yang salah.
2. Visualisasi adalah alat berpikir, bukan sekadar alat presentasi. Grafik yang baik mengungkap pola yang tidak terlihat di tabel. Grafik yang buruk menyesatkan—bahkan ketika datanya benar.
3. Diagnosis mengubah gejala menjadi akar masalah. Mengetahui *bahwa* churn tinggi tidak cukup. Mengetahui *mengapa*—dan **segmen mana**—mengarahkan intervensi ke tempat yang tepat.
4. Metrik harus mencerminkan konsekuensi bisnis. Accuracy menyesatkan untuk kelas tidak seimbang. Precision, recall, dan F1 menjawab pertanyaan yang berbeda. Pilih metrik yang sesuai dengan **biaya kesalahan** dalam konteks Anda.
5. Baseline adalah pembanding yang wajib. Sebelum membangun model canggih, tetapkan baseline sederhana. Banyak proyek gagal karena tidak pernah tahu apakah model mereka benar-benar lebih baik dari aturan sederhana.
6. Kalibrasi sama pentingnya dengan diskriminasi. Model dengan AUC tinggi tetapi probabilitas tidak terkalibrasi bisa menghasilkan keputusan yang salah ketika threshold digunakan. Pastikan probabilitas dapat dipercaya jika Anda menggunakannya untuk keputusan.
7. Ketidakpastian harus dimodelkan, bukan diabaikan. Satu titik prediksi menyembunyikan risiko. Simulasi mengungkap distribusi—dan distribusi adalah bahasa keputusan yang lebih kaya.
8. Optimasi mengubah prediksi menjadi tindakan. Prediksi memberi tahu kita apa yang mungkin terjadi; optimasi memberi tahu kita apa yang harus dilakukan. LP, IP, dan network flow adalah jembatan antara keduanya.
9. Shadow price adalah bahasa manajemen. Bukan hanya "ini solusi optimal", tetapi "setiap tambahan Rp 1 anggaran menghasilkan Rp 8 manfaat". Ini adalah argumen yang dapat digunakan dalam rapat direksi.
10. Keputusan yang baik mengakui batasnya. Analisis yang jujur tentang ketidakpastian, asumsi, dan batasan membangun kepercayaan. Analisis yang mengklaim kepastian yang tidak dimilikinya menghancurkannya.

Yang Tidak Diajarkan Buku Ini

Sama pentingnya dengan apa yang diajarkan adalah apa yang **tidak** diajarkan. Buku ini adalah **pengantar**, bukan ensiklopedia. Beberapa topik yang sengaja dibatasi:

Topik	Mengapa tidak dibahas mendalam	Di mana mempelajarinya
Causal inference	Membutuhkan desain eksperimen dan metode khusus (DoWhy, causal-learn)	*Causal Inference: The Mixtape* (Cunningham), *Mostly Harmless Econometrics* (Angrist & Pischke)
Deep learning	Fokus buku adalah data tabular, bukan gambar/teks	*Deep Learning* (Goodfellow et al.), kursus online
Reinforcement learning	Keputusan berurutan membutuhkan kerangka berbeda	*Reinforcement Learning: An Introduction* (Sutton & Barto)
MLOps lengkap	Deployment dan monitoring disinggung, tidak didemonstrasikan penuh	*Designing Machine Learning Systems* (Huyen)
Etika AI	Disinggung, tetapi tidak dibahas mendalam	*Weapons of Math Destruction* (O'Neil), *Fairness and Machine Learning* (Barocas et al.)
Big data	Fokus pada dataset yang dapat ditangani di laptop	*Spark: The Definitive Guide* (Chambers & Zaharia)

> **Note**
>
> **Buku ini adalah pintu masuk, bukan tujuan akhir.** Setiap bab memiliki kedalaman yang bisa dieksplorasi lebih lanjut. Jika satu topik menarik minat Anda, jadikan ia titik awal—bukan akhir—dari pembelajaran Anda.

Untuk Pembaca yang Berbeda

Jika Anda Mahasiswa

Anda baru saja menyelesaikan perjalanan yang, di banyak program, membutuhkan **tiga course terpisah**—statistika, machine learning, dan riset operasi. Anda kini memiliki **peta yang menghubungkan ketiganya**.

Saran: Jangan berhenti di sini. Ambil satu proyek nyata—dari magang, kompetisi Kaggle, atau masalah di komunitas Anda—dan terapkan seluruh alur kerja dari buku ini. Pengalaman nyata mengajarkan hal yang tidak bisa diajarkan buku: **data kotor, stakeholder yang tidak sabar, dan keputusan yang harus diambil dengan informasi tidak lengkap**.

Jika Anda Pengajar

Buku ini lahir dari pengalaman mengajar—dan dirancang untuk **digunakan kembali di kelas**. Setiap bab memiliki tujuan pembelajaran, contoh kode yang dapat direproduksi, latihan terbimbing, dan kotak "Jembatan" yang menghubungkan analisis ke keputusan.

Saran: Gunakan **studi kasus di Bab 18** sebagai proyek akhir. Mahasiswa akan mengalami sendiri bagaimana menggabungkan prediksi, optimasi, dan simulasi—dan bagaimana **mengomunikasikan** hasilnya ke audiens non-teknis.

Jika Anda Praktisi

Anda mungkin sudah mengenal beberapa teknik di buku ini—atau bahkan semua. Tetapi mungkin ada **celah** dalam alur kerja Anda: model prediktif yang tidak pernah sampai ke keputusan, optimasi yang mengabaikan ketidakpastian, atau dashboard yang tidak pernah digunakan.

Saran: Gunakan **Bab 18 sebagai checklist**. Apakah proyek Anda memiliki: (1) pemahaman bisnis yang jelas? (2) data yang bersih? (3) model yang tervalidasi? (4) optimasi yang mempertimbangkan kendala? (5) evaluasi risiko? (6) rencana deployment dan monitoring? Jika ada yang hilang, di situlah peluang perbaikan.

Setelah Buku Ini: Langkah Berikutnya

Untuk Pembelajar Mandiri

1. **Kerjakan proyek nyata.** Ambil dataset dari pekerjaan atau komunitas, dan terapkan alur kerja dari buku ini dari awal hingga akhir.
2. **Bangun portofolio.** Dokumentasikan proyek Anda di GitHub atau blog—bukan hanya kode, tetapi **narasi keputusan**: masalah apa, mengapa teknik itu, apa hasilnya, apa batasannya.
3. **Ajarkan.** Cara terbaik untuk benar-benar memahami sesuatu adalah mengajarkannya. Mulai dari teman, komunitas, atau blog pribadi.
4. **Tetap skeptis.** Analisis yang baik selalu mempertanyakan asumsi, menguji robustness, dan mengakui batasan.

Untuk Pengajar

1. **Adaptasi konteks lokal.** Ganti dataset Telco/Retail dengan data dari industri lokal—perbankan, e-commerce, kesehatan—agar mahasiswa melihat relevansi langsung.

2. **Tambahkan tamu praktisi.** Undang analis dari industri untuk berbagi bagaimana mereka menerapkan (atau bergulat dengan) teknik-teknik ini di dunia nyata.
3. **Buat proyek tim.** Keputusan nyata jarang dibuat sendirian. Beri mahasiswa proyek kelompok dengan peran berbeda—analis data, modeler, komunikator—agar mereka belajar berkolaborasi.

Penutup

Kita memulai buku ini dengan pertanyaan sederhana: *"Apa yang ada di data?"* Kita mengakhirinya dengan pertanyaan yang jauh lebih kaya: *"Apa yang harus kita lakukan, dan bagaimana kita tahu itu keputusan yang baik?"*

Sepanjang perjalanan, kita belajar bahwa **decision analytics bukan tentang model**—ia tentang **keputusan**. Model hanyalah alat. Data hanyalah bahan. Yang membuat keduanya bernilai adalah **kemampuan manusia** untuk:

- **Merumuskan pertanyaan yang tepat**.
- **Memilih teknik yang sesuai**.
- **Menafsirkan hasil dengan jujur**.
- **Mengomunikasikan insight dengan jelas**.
- **Mengambil tindakan dengan bijak**.

Buku ini mengajarkan Anda alat-alatnya. **Keputusan untuk menggunakannya ada di tangan Anda.**

> *"The purpose of computing is insight, not numbers."* — Richard Hamming
>
> *"In God we trust; all others must bring data."* — W. Edwards Deming

Selamat berkarya—dan selamat mengambil keputusan yang lebih baik.

References

Daftar pustaka di bawah ini dihasilkan **secara otomatis** oleh Quarto dari file `references.bib`—hanya referensi yang **benar-benar dikutip** di buku ini yang akan muncul. Jika Anda ingin menambahkan referensi baru, tambahkan entri ke `references.bib` dan kutip dengan format `[@kunci]` di bab mana pun.

Daftar Pustaka

Bertsimas, Dimitris, and John N. Tsitsiklis. 1997. *Introduction to Linear Optimization*. Vol. 6. Athena Scientific.

Chauhan, Anirudh. 2024. *Retail Store Inventory Forecasting Dataset*. Kaggle. https://www.kaggle.com/datasets/anirudhchauhan/retail-store-inventory-forecasting-dataset.

COIN-OR Foundation. 2024. *PuLP: A Linear Programming Modeler Written in Python*. Https://coin-or.github.io/pulp/.

Hastie, Trevor, Robert Tibshirani, and Jerome H. Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer. https://doi.org/10.1007/978-0-387-84858-7.

Hillier, Frederick S., and Gerald J. Lieberman. 2010. *Introduction to Operations Research*. 9th ed. McGraw-Hill Higher Education.

Hyndman, Rob J., and George Athanasopoulos. 2021. *Forecasting: Principles and Practice*. 3rd ed. OTexts. https://otexts.com/fpp3/.

IBM Sample Data Sets. 2019. *Telco Customer Churn*. Kaggle. https://www.kaggle.com/datasets/blastchar/telco-customer-churn.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2023. *An Introduction to Statistical Learning: With Applications in Python*. Springer. https://doi.org/10.1007/978-3-031-38747-0.

Keeney, Ralph L., and Howard Raiffa. 1993. *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. Cambridge University Press. https://doi.org/10.1017/CBO9781139174084.

Knaflic, Cole Nussbaumer. 2015. *Storytelling with Data: A Data Visualization Guide for Business Professionals*. Wiley.

Kuhn, Max, and Kjell Johnson. 2013. *Applied Predictive Modeling*. Springer. https://doi.org/10.1007/978-1-4614-6849-3.

Law, Averill M., and W. David Kelton. 2000. *Simulation Modeling and Analysis*. 3rd ed. McGraw-Hill.

McKinney, Wes. 2022. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*. 3rd ed. O'Reilly Media.

Minto, Barbara. 2002. *The Pyramid Principle: Logic in Writing and Thinking*. 3rd ed. Pearson Education.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. "Scikit-Learn: Machine Learning in Python." *Journal of Machine Learning Research* 12: 2825–30. https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html.

Posit Software, PBC. 2024. *Quarto: An Open-Source Scientific and Technical Publishing System*. Https://quarto.org/.

Tufte, Edward R. 2001. *The Visual Display of Quantitative Information*. 2nd ed. Graphics Press.

VanderPlas, Jake. 2016. *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media. https://jakevdp.github.io/PythonDataScienceHandbook/.

Bacaan Lanjutan

Bagian ini **tidak** dihasilkan otomatis—ia adalah **kurasi manual** dari sumber-sumber yang, menurut penulis, paling berharga untuk memperdalam topik-topik tertentu. Semua entri di bawah ini **tidak** harus dikutip di buku—mereka adalah **peta jalan** bagi pembaca yang ingin melanjutkan.

Fondasi Analytics & Statistika

- **McKinney, W. (2022).** *Python for Data Analysis* (3rd ed.). O'Reilly. — Referensi paling lengkap untuk `pandas` dan `numpy`. Wajib dimiliki setiap analis Python.
- **VanderPlas, J. (2016).** *Python Data Science Handbook*. O'Reilly. — Gratis di jakevdp.github.io. Pengantar yang sangat baik untuk ekosistem data science Python.
- **Wheelan, C. (2013).** *Naked Statistics*. W. W. Norton. — Pengantar statistika yang ramah untuk pembaca non-teknis.

Machine Learning & Predictive Analytics

- **James, G., Witten, D., Hastie, T., & Tibshirani, R. (2023).** *An Introduction to Statistical Learning* (2nd ed., Python Edition). Springer. — Gratis di statlearning.com. **Buku terbaik** untuk memulai machine learning dengan dasar statistik yang kuat.
- **Hastie, T., Tibshirani, R., & Friedman, J. (2009).** *The Elements of Statistical Learning* (2nd ed.). Springer. — Versi lebih teknis dari ISL. Cocok untuk pembaca yang ingin memahami **matematika** di balik model.
- **Kuhn, M., & Johnson, K. (2013).** *Applied Predictive Modeling*. Springer. — Fokus pada **praktik**—feature engineering, resampling, tuning. Sangat relevan untuk analis yang bekerja dengan data nyata.
- **Géron, A. (2022).** *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly. — Referensi praktis terbaik untuk scikit-learn.

Optimasi & Riset Operasi

- **Bertsimas, D., & Tsitsiklis, J. N. (1997).** *Introduction to Linear Optimization*. Athena Scientific. — Referensi klasik untuk LP. Matematis, tetapi sangat jelas.
- **Hillier, F. S., & Lieberman, G. J. (2010).** *Introduction to Operations Research* (9th ed.). McGraw-Hill. — Buku teks standar riset operasi—mencakup LP, IP, network flow, simulasi, dan antrean.
- **Williams, H. P. (2013).** *Model Building in Mathematical Programming* (5th ed.). Wiley. — Fokus pada **formulasi**—keterampilan yang sering diabaikan tetapi paling penting dalam praktik.

Simulasi & Analisis Keputusan

- **Law, A. M., & Kelton, W. D. (2000).** *Simulation Modeling and Analysis* (3rd ed.). McGraw-Hill. — Referensi komprehensif untuk simulasi diskrit.
- **Keeney, R. L., & Raiffa, H. (1993).** *Decisions with Multiple Objectives*. Cambridge University Press. — Fondasi teoretis untuk multi-objective optimization.

- **Hubbard, D. W. (2014).** *How to Measure Anything* (3rd ed.). Wiley. — Tentang **mengukur ketidakpastian** dan nilai informasi—sangat relevan untuk Bab 13.

Time Series

- **Hyndman, R. J., & Athanasopoulos, G. (2021).** *Forecasting: Principles and Practice* (3rd ed.). OTexts. — Gratis di otexts.com/fpp3. **Buku terbaik** untuk forecasting—praktis dan dapat diakses.
- **Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015).** *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley. — Referensi klasik untuk ARIMA dan analisis time series.

Visualisasi & Komunikasi

- **Tufte, E. R. (2001).** *The Visual Display of Quantitative Information* (2nd ed.). Graphics Press. — Klasik abadi. Wajib dibaca setiap analis yang membuat grafik.
- **Knaflic, C. N. (2015).** *Storytelling with Data*. Wiley. — Panduan praktis untuk mengomunikasikan insight ke audiens non-teknis.
- **Cairo, A. (2016).** *The Truthful Art*. New Riders. — Tentang visualisasi yang **jujur**—dan bagaimana menghindari manipulasi visual.

Causal Inference (Topik Lanjutan)

- **Cunningham, S. (2021).** *Causal Inference: The Mixtape*. Yale University Press. — Gratis di mixtape.scunning.com. Pengantar yang sangat baik untuk metode kausal.
- **Angrist, J. D., & Pischke, J.-S. (2008).** *Mostly Harmless Econometrics*. Princeton University Press. — Untuk pembaca yang ingin dasar ekonometrika yang kuat.
- **Pearl, J., & Mackenzie, D. (2018).** *The Book of Why*. Basic Books. — Pengantar konseptual untuk **causal inference**—tanpa matematika berat.

MLOps & Deployment (Topik Lanjutan)

- **Huyen, C. (2022).** *Designing Machine Learning Systems*. O'Reilly. — Referensi terbaik untuk membangun sistem ML yang dapat diandalkan di produksi.
- **Sculley, D., et al. (2015).** "Hidden Technical Debt in Machine Learning Systems." *NeurIPS*. — Paper klasik tentang **mengapa model ML sulit di-deploy** dan dipelihara.

Etika & Dampak Sosial

- **O'Neil, C. (2016).** *Weapons of Math Destruction.* Crown. — Tentang bagaimana algoritma dapat memperkuat ketidakadilan—wajib dibaca setiap praktisi data.
- **Barocas, S., Hardt, M., & Narayanan, A. (2023).** *Fairness and Machine Learning.* MIT Press. — Gratis di fairmlbook.org. Referensi teknis untuk keadilan algoritmik.

Cara Mengutip Buku Ini

Jika Anda menggunakan buku ini dalam karya akademik atau profesional, silakan kutip sebagai:

> [Nama Anda]. (2026). *Decision Analytics: Dari Data ke Keputusan.* [URL repositori].

BibTeX:

```
@book{nama2026decision,
  title     = {Decision Analytics: Dari Data ke Keputusan},
  author    = {Chakra Roy},
  year      = {2026},
  publisher = {aropal publisher},
  url       = {https://github.com/chakraroy/decision-analytics}
}
```

Lisensi Konten

- **Teks buku:** CC BY-NC-SA 4.0—bebas digunakan untuk tujuan non-komersial dengan atribusi dan berbagi serupa.
- **Kode:** MIT License—bebas digunakan, dimodifikasi, dan didistribusikan.
- **Dataset:** Mengikuti lisensi masing-masing sumber (lihat halaman Kaggle untuk detail).